ABG-VBG Analysis

Brian Locke, Anila Mehta

Table of contents

1 Data Pre-Processing and Unweighted Analyses **3**
1.0.1 Package Set Up . . . 22
1.1 Helper functions for model diagnostics . . . 38
1.1.1 Configuration for the IPW models . . . 103
1.1.2 Outcome Variable Creation . . . 119
1.2 Baseline tables . . . 121
1.3 Three-level PCO2 categories (unweighted) . . . 143
1.4 Observed outcome rates by CO2 bin (unweighted) . . . 150
1.5 Restricted cubic spline regressions (unweighted) . . . 151
1.5.1 Unweighted, Restricted Cubic Spline Regression - ABG by PaCO2 . . . 151
1.5.2 Unweighted, Restricted Cubic Spline - VBG . . . 153

2 Inverse Propensity Weighting (GBM) **159**
2.0.1 ABG IPW weighting and diagnostics . . . 159
2.0.2 ABG IPW spline models . . . 161
2.0.3 ABG IPW spline models (2–98th percentile) . . . 163
2.0.4 VBG IPW weighting and spline models . . . 167
2.0.5 Three-level PCO2 categories (weighted; ABG, VBG) . . . 176
2.0.6 Observed outcome rates by CO2 bin (GBM IPSW, non-MI) . . . 180
2.1 Propensity score diagnostics . . . 181

3 Multiple Imputation (and logistic IPSW) **189**
3.0.1 Missingness structure and drivers . . . 195

3.0.2 Monte Carlo error check after MI . . . 200
3.1 Pre-imputation data prep (consistent types & predictors) . . . 200
3.2 Imputation model specification (MICE) . . . 201
3.2.1 Predictor matrix & methods. Run MICE (moderate settings for scale) . . . 201
3.3 Refit propensity models within each imputation . . . 261
3.3.1 FAIL-FAST CHECKS . . . 261
3.3.2 ABG propensity (has_abg) . . . 323
3.3.3 Balance diagnostics across imputations . . . 324
3.3.4 VBG propensity (has_vbg) . . . 325
3.3.5 VBG balance . . . 326
3.4 Weighted outcome models within each imputation + pooling . . . 328
3.4.1 Helper: fit + extract log-OR and SE from svyglm . . . 328
3.4.2 VBG: MI pooled spline models (treated cohort only) . . . 328
3.5 Explainability . . . 328
3.6 MI three-level PCO2 helpers and checks . . . 338
3.7 MI + IPW three-level PCO2 (ABG & VBG) . . . 339
3.7.1 ABG: MI + IPW, three-level PCO2 outcomes . . . 339
3.7.2 VBG: MI + IPW, three-level PCO2 outcomes . . . 339
3.7.3 MI-pooled IPW associations (3-level CO2) . . . 340
3.7.4 Summary: adjusted CO2-category associations across analysis tracks . . . 341
3.8 Canonical Categorical Key-Results Figures Across Analysis Tracks . . . 358
3.9 Observed outcome rates by CO2 bin (MI-logistic IPSW) . . . 361
3.10 Manuscript outputs summary . . . 363
3.10.1 Visualization: pooled three-level ORs . . . 367
3.10.2 Visualization: pooled three-level predicted probabilities . . . 368
3.10.3 Visualization . . . 374
3.11 Canonical Key-Results Figures Across Analysis Tracks . . . 376

4 Diagnostics and Exports **420**
4.0.1 MI convergence and mixing . . . 422
4.0.2 MI stability across m . . . 425
4.0.3 MI maxit sensitivity (sampled) . . . 430
4.0.4 Balance diagnostics . . . 435
4.0.5 Outcome diagnostics . . . 439
4.0.6 Diagnostics summary and audit . . . 440
4.0.7 Performance / runtime log . . . 446
4.0.8 Performance / runtime log . . . 450

5 Analysis of the discordance between predicted probabilities and OR for NIV and IMV **452**
5.1 Pilot validation and scope 453
5.2 Save, export, and session info 523

```
# TODO (future work):
# P1: ABG residual imbalance >0.10 - consider redefining target population / stratified standardization / overlap
 ↪  restriction.
# P3: Separation/nonconvergence in adjusted outcome models - consider penalized logistic / level collapsing, and
 ↪  define reporting policy.
# TODO consider adding discontinuity (at 45 mmHg CO2 on ABG and 50 mmHg CO2 on VBG; start with raw binning and LOESS
 ↪  to evaluate if there's a jump, then consider regression discontinuity or similar if so).
```

1 Data Pre-Processing and Unweighted Analyses

This code pulls in the master database (a STATA file) and does some initial cleaning - this will only need to be run once, and then the data can be accessed in the usual way.

```
# setup-packages chunk is the authoritative loader.
stopifnot(
  requireNamespace("kableExtra", quietly = TRUE),
  requireNamespace("gtsummary", quietly = TRUE),
  requireNamespace("purrr", quietly = TRUE)
)

# globally tighten gtsummary/gt tables (smaller font + tighter padding)
gtsummary::theme_gtsummary_compact()

project_root <- here::here()
knitr::opts_knit$set(root.dir = project_root)

# keep figures anchored in PDF to reduce heading-only/blank pages
if (knitr::is_latex_output()) {
  knitr::opts_chunk$set(fig.pos = "H")
}

# helper: scale generic data.frames for PDF output
```

```r
kable_pdf <- function(df, caption = NULL, font_size = 7) {
  kbl <- knitr::kable(df, format = "latex", booktabs = TRUE, caption = caption)
  kbl <- kableExtra::kable_styling(
    kbl,
    latex_options = c("hold_position", "scale_down"),
    font_size     = font_size,
    full_width    = TRUE,
    position      = "center"
  )
  mark_float_emitted()
  kbl
}

# helper: resolve a notebook display path relative to the repo root so the
# canonical render path never depends on the notebook subdirectory.
display_path <- function(path, must_exist = TRUE) {
  vapply(path, function(one_path) {
    path_abs <- tryCatch(
      fs::path_abs(one_path, start = project_root),
      error = function(e) NA_character_
    )
    if (is.na(path_abs)) {
      stop("Could not resolve display path: ", one_path)
    }
    if (must_exist && !file.exists(path_abs)) {
      stop("Missing display artifact: ", path_abs)
    }
    if (!(fs::path_has_parent(path_abs, project_root) || identical(path_abs, project_root))) {
      stop("Display path escapes project root: ", path_abs)
    }
    fs::path_rel(path_abs, start = project_root)
  }, character(1), USE.NAMES = FALSE)
}

# helper: pretty, relative paths for PDF output
pretty_path <- function(p) {
```

```r
  vapply(p, function(one_path) {
    out <- tryCatch(display_path(one_path, must_exist = FALSE)[[1]], error = function(e) NA_character_)
    if (is.na(out) || out == "." || grepl("^\\.\\.", out)) {
      return(basename(one_path))
    }
    out
  }, character(1), USE.NAMES = FALSE)
}

# helper: emit inline notebook graphics through one canonical path. For PDF
# renders we cap the height explicitly so headings plus image can fit on the
# page without triggering TeX pagination loops.
render_inline_graphic <- function(file,
                                  width = "0.94\\linewidth",
                                  height = "0.72\\textheight") {
  inline_path <- display_path(file)
  if (knitr::is_latex_output()) {
    out <- c(
      "\\begin{center}",
      sprintf(
        "\\includegraphics[width=%s,height=%s,keepaspectratio]{%s}",
        width, height, inline_path
      ),
      "\\end{center}"
    )
    return(knitr::asis_output(paste(out, collapse = "\n")))
  }
  knitr::include_graphics(inline_path)
}

# Canonical PDF path: export all figure/table artifacts, keep executed source
# visible, and reserve rendered output for manuscript-facing displays plus
# compact provenance/status summaries. QA/debug figures remain sidecar artifacts.
VALIDATION_INLINE_ANALYSIS_PREVIEWS <- FALSE

# helper: normalize manuscript-facing text to a plain-ASCII representation so
```

```r
# PDF labels/captions do not depend on platform-specific glyph handling.
safe_manuscript_text <- function(x) {
  if (is.null(x) || !length(x)) return(x)
  x <- as.character(x)
  replacements <- c(
    "\u00a0" = " ",
    "\u2010" = "-",
    "\u2011" = "-",
    "\u2012" = "-",
    "\u2013" = "-",
    "\u2014" = "-",
    "\u2212" = "-",
    "\u2026" = "...",
    "\u03bc" = "u",
    "\u00b5" = "u",
    "\u2264" = "<=",
    "\u2265" = ">=",
    "\u0394" = "Delta",
    "\u03b4" = "delta",
    "\u03b2" = "beta",
    "\u2082" = "2",
    "\u00b2" = "^2",
    "\u00b3" = "^3",
    "\u2080" = "0",
    "\u2081" = "1",
    "\u2083" = "3",
    "\u2084" = "4",
    "\u2085" = "5",
    "\u2086" = "6",
    "\u2087" = "7",
    "\u2088" = "8",
    "\u2089" = "9"
  )
  for (i in seq_along(replacements)) {
    x <- gsub(names(replacements)[i], replacements[[i]], x, fixed = TRUE)
  }
```

```r
  x <- iconv(x, from = "", to = "ASCII//TRANSLIT", sub = "?")
  x <- gsub("\\s+", " ", x)
  trimws(x)
}

# helper: flag non-ASCII manuscript labels early so validation renders fail
# before the final PDF silently substitutes or drops glyphs.
is_ascii_text <- function(x) {
  if (is.null(x) || !length(x)) return(logical(0))
  !grepl("[^ -~]", x)
}

# helper: derive one stable manuscript caption from the centralized registry.
manuscript_caption <- function(label, title) {
  label <- safe_manuscript_text(label)
  title <- safe_manuscript_text(title)
  stopifnot(length(label) == length(title))
  ifelse(!nzchar(title), label, paste0(label, ". ", title))
}

# helper: collapse component-level validation states into one build status.
collapse_validation_status <- function(status_values) {
  status_values <- unique(status_values[!is.na(status_values) & nzchar(status_values)])
  if ("FAIL" %in% status_values) return("FAIL")
  if ("PASS_WITH_WARNINGS" %in% status_values) return("PASS_WITH_WARNINGS")
  "PASS"
}

# helper: strip manual "Table X." prefixes in PDF to avoid double numbering
strip_manual_table_number <- function(caption) {
  if (is.null(caption)) return(caption)
  if (!knitr::is_latex_output()) return(caption)
  cap <- gsub("^\\s*(\\*\\*)?Table\\s+[0-9A-Za-z]+\\.?\\s*:?\\s*", "", caption)
  cap
}
```

```r
# helper: escape LaTeX special chars in plain captions
escape_latex_text <- function(x) {
  if (is.null(x)) return(x)
  if (!knitr::is_latex_output()) return(x)
  gsub("([\\\\%$&#{}_\\^~])", "\\\\\\1", x, perl = TRUE)
}

# helper: strip markdown/LaTeX numbering and formatting so captions can be
# safely reused in both inline Quarto output and standalone PDF exports.
plain_caption_text <- function(caption) {
  if (is.null(caption) || !nzchar(caption)) return("")
  cap <- strip_manual_table_number(caption)
  cap <- gsub("\\*\\*", "", cap)
  cap <- gsub("\\s+", " ", cap)
  trimws(cap)
}

# helper: replace CO2 labels with LaTeX-safe subscripts in table text so PDF
# tables render predictably even when the visible code uses plain ASCII labels.
latex_safe_table_text <- function(x) {
  if (!knitr::is_latex_output() || is.null(x) || !length(x)) return(x)
  out <- x
  out <- gsub("PaCO ", "PaCO\\\\textsubscript{2}", out, fixed = TRUE)
  out <- gsub("PvCO ", "PvCO\\\\textsubscript{2}", out, fixed = TRUE)
  out <- gsub("PaCO2", "PaCO\\\\textsubscript{2}", out, fixed = TRUE)
  out <- gsub("PvCO2", "PvCO\\\\textsubscript{2}", out, fixed = TRUE)
  out
}

# helper: build one plain-text heading string for manually managed table
# displays, where the notebook wants explicit manuscript labels.
table_heading_block <- function(title = NULL, display_label = NULL) {
  title <- plain_caption_text(title)
  label <- if (is.null(display_label) || !nzchar(display_label)) "" else trimws(as.character(display_label))
  heading <- trimws(paste(if (nzchar(label)) paste0(label, ".") else "", title))
  if (!nzchar(heading)) return("")
```

```r
  if (!knitr::is_latex_output()) return(paste0("**", heading, "**\n\n"))
  heading <- latex_safe_table_text(escape_latex_text(heading))
  paste0("\\begin{center}\\textbf{", heading, "}\\end{center}")
}

# helper: emit a table heading plus one or more LaTeX/Markdown table blocks,
# keeping page-break and landscape behavior explicit in one place.
render_table_blocks <- function(table_blocks, title = NULL, display_label = NULL,
                                landscape = FALSE, page_break_before = FALSE) {
  heading <- table_heading_block(title = title, display_label = display_label)
  out <- c(if (nzchar(heading)) heading else character(0), table_blocks)
  if (knitr::is_latex_output() && isTRUE(landscape)) {
    out <- c("\\begin{landscape}", out, "\\end{landscape}")
  }
  if (knitr::is_latex_output() && isTRUE(page_break_before)) {
    out <- c("\\clearpage", out)
  }
  mark_float_emitted()
  knitr::asis_output(paste(out, collapse = "\n\n"))
}

# helper: mark points where a visible table/figure has been emitted; this is a
# semantic hook more than a data object, but it keeps display code readable.
mark_float_emitted <- function() {
  invisible(TRUE)
}
# helper: flush accumulated LaTeX floats at important manuscript boundaries so
# tables and figures stay near the code that generated them.
float_barrier <- function(mode = c("soft", "section", "hard", "flush")) {
  if (!knitr::is_latex_output()) return(invisible(NULL))
  mode <- match.arg(mode)
  cmd <- switch(
    mode,
    soft = "\\FloatBarrier\\nopagebreak",
    section = "\\FloatBarrier\\nopagebreak\\par\\noindent",
    hard = "\\FloatBarrier\\clearpage",
```

```r
    flush = "\\FloatBarrier\\pagebreak"
  )
  knitr::asis_output(cmd)
}

# helper: render full tables for PDF (no previews)
render_table_pdf <- function(df, caption, file_stub,
                             wide = FALSE,
                             digits = 2,
                             max_cols = 6,
                             landscape = NULL,
                             display_label = NULL,
                             caption_mode = c("plain", "numbered", "none"),
                             page_break_before = FALSE) {
  stopifnot(is.data.frame(df))
  caption_mode <- match.arg(caption_mode)
  out_csv <- results_path(paste0(file_stub, ".csv"))
  write_csv_safely(df, out_csv, row_names = FALSE)

  if (is.null(landscape)) landscape <- (wide || ncol(df) > max_cols)
  use_longtable <- nrow(df) > 40
  n_cols <- ncol(df)
  parts <- if (n_cols <= max_cols) list(df) else {
    idx <- split(seq_len(n_cols), ceiling(seq_len(n_cols) / max_cols))
    lapply(idx, function(ii) df[, ii, drop = FALSE])
  }

  build_kbl <- function(df_part, cap) {
    cap <- plain_caption_text(cap)
    cap_numbered <- if (identical(caption_mode, "numbered")) {
      escape_latex_text(latex_safe_table_text(cap))
    } else {
      NULL
    }
    kbl <- knitr::kable(
      df_part,
```

```r
      format    = "latex",
      booktabs  = TRUE,
      longtable = use_longtable,
      caption   = cap_numbered,
      digits    = digits
    )
    kbl <- kableExtra::kable_styling(
      kbl,
      latex_options = if (use_longtable) c("repeat_header") else c("hold_position", "scale_down"),
      font_size     = if (ncol(df_part) > max_cols || wide) 6 else 7,
      full_width    = FALSE,
      position      = "center"
    )
    latex_safe_table_text(as.character(kbl))
  }

  letters_part <- LETTERS[seq_along(parts)]
  out <- unlist(lapply(seq_along(parts), function(i) {
    cap_i <- if (length(parts) == 1L) caption else paste0(caption, " (Part ", letters_part[i], ")")
    build_kbl(parts[[i]], cap_i)
  }))
  if (identical(caption_mode, "numbered")) {
    mark_float_emitted()
    return(knitr::asis_output(paste(out, collapse = "\n\n")))
  }
  title_i <- if (identical(caption_mode, "none")) NULL else caption
  render_table_blocks(
    out,
    title = title_i,
    display_label = display_label,
    landscape = landscape,
    page_break_before = page_break_before
  )
}

# helper: conditionally show a rendered table in the notebook while still
```

```r
# always exporting the CSV/PDF artifact needed downstream.
render_table_pdf_maybe <- function(df, caption, file_stub,
                                   wide = FALSE,
                                   digits = 2,
                                   max_cols = 6,
                                   landscape = NULL,
                                   show = TRUE,
                                   display_label = NULL,
                                   caption_mode = c("plain", "numbered", "none")) {
  caption_mode <- match.arg(caption_mode)
  if (!isTRUE(show)) {
    write_csv_safely(df, results_path(paste0(file_stub, ".csv")), row_names = FALSE)
    return(invisible(NULL))
  }
  tbl <- render_table_pdf(
    df,
    caption = caption,
    file_stub = file_stub,
    wide = wide,
    digits = digits,
    max_cols = max_cols,
    landscape = landscape,
    display_label = display_label,
    caption_mode = caption_mode
  )
  if (isTRUE(show)) {
    return(tbl)
  }
  invisible(NULL)
}

# helper: render one numbered manuscript table as multiple explicit panels.
# This avoids automatic wide-table splitting, which can detach the final column
# from the table body in LaTeX output.
render_panel_table_pdf <- function(panels, caption, file_stub,
                                   digits = 2,
```

```r
                                  display_label = NULL,
                                  landscape = TRUE,
                                  page_break_before = TRUE,
                                  csv_df = NULL) {
stopifnot(is.list(panels), length(panels) > 0L)
panel_names <- names(panels)
if (is.null(panel_names) || any(!nzchar(panel_names))) {
  panel_names <- paste("Panel", LETTERS[seq_along(panels)])
}
stopifnot(all(vapply(panels, is.data.frame, logical(1L))))

if (is.null(csv_df)) {
  csv_df <- dplyr::bind_rows(
    lapply(seq_along(panels), function(idx) {
      dplyr::mutate(panels[[idx]], panel = panel_names[[idx]], .before = 1L)
    })
  )
}
write_csv_safely(csv_df, results_path(paste0(file_stub, ".csv")), row_names = FALSE)

build_kbl <- function(df_part) {
  kbl <- knitr::kable(
    df_part,
    format = "latex",
    booktabs = TRUE,
    longtable = FALSE,
    digits = digits
  )
  kbl <- kableExtra::kable_styling(
    kbl,
    latex_options = c("hold_position", "scale_down"),
    font_size = 7,
    full_width = FALSE,
    position = "center"
  )
  latex_safe_table_text(as.character(kbl))
```

```r
  }

  panel_blocks <- unlist(lapply(seq_along(panels), function(idx) {
    panel_title <- paste0("Panel ", LETTERS[[idx]], ". ", panel_names[[idx]])
    if (knitr::is_latex_output()) {
      c(
        paste0("\\medskip\\noindent\\textbf{", escape_latex_text(latex_safe_table_text(panel_title)), "}\\par"),
        build_kbl(panels[[idx]])
      )
    } else {
      c(paste0("**", panel_title, "**"), as.character(knitr::kable(panels[[idx]], digits = digits)))
    }
  }))

  render_table_blocks(
    panel_blocks,
    title = caption,
    display_label = display_label,
    landscape = landscape,
    page_break_before = page_break_before
  )
}

# helper: render a standalone table PDF that can be cited outside the notebook.
# This path is used for manuscript/supplement table artifacts with stable names.
resolve_latex_command <- function(cmd = "pdflatex") {
  cmd_path <- Sys.which(cmd)
  if (nzchar(cmd_path)) return(cmd_path)

  fallback <- fs::path(path.expand("~"), "Library", "TinyTeX", "bin", "universal-darwin", cmd)
  if (file.exists(fallback)) {
    tex_dir <- fs::path_dir(fallback)
    current_path <- Sys.getenv("PATH", unset = "")
    path_parts <- strsplit(current_path, .Platform$path.sep, fixed = TRUE)[[1]]
    if (!tex_dir %in% path_parts) {
      Sys.setenv(PATH = paste(c(tex_dir, current_path), collapse = .Platform$path.sep))
```

```r
    }
    return(fallback)
  }

  stop(
    "Required LaTeX command not found on PATH: ", cmd,
    ". Install TinyTeX or expose the TeX bin directory before rendering."
  )
}

write_standalone_table_pdf <- function(df, caption, file_stub,
                                       wide = FALSE,
                                       digits = 2,
                                       max_cols = 6,
                                       landscape = NULL,
                                       display_label = NULL) {
  stopifnot(is.data.frame(df))
  out_csv <- results_path(paste0(file_stub, ".csv"))
  write_csv_safely(df, out_csv, row_names = FALSE)

  if (is.null(landscape)) landscape <- (wide || ncol(df) > max_cols)
  use_longtable <- nrow(df) > 40
  n_cols <- ncol(df)
  parts <- if (n_cols <= max_cols) list(df) else {
    idx <- split(seq_len(n_cols), ceiling(seq_len(n_cols) / max_cols))
    lapply(idx, function(ii) df[, ii, drop = FALSE])
  }

  build_kbl <- function(df_part) {
    kbl <- knitr::kable(
      df_part,
      format = "latex",
      booktabs = TRUE,
      longtable = use_longtable,
      caption = NULL,
      digits = digits
```

```r
  )
  kbl <- kableExtra::kable_styling(
    kbl,
    latex_options = if (use_longtable) c("repeat_header") else c("hold_position", "scale_down"),
    font_size = if (ncol(df_part) > max_cols || wide) 6 else 7,
    full_width = FALSE,
    position = "center"
  )
  latex_safe_table_text(as.character(kbl))
}

letters_part <- LETTERS[seq_along(parts)]
table_blocks <- unlist(lapply(seq_along(parts), function(i) {
  cap_i <- if (length(parts) == 1L) caption else paste0(caption, " (Part ", letters_part[i], ")")
  c(
    table_heading_block(cap_i, display_label = display_label),
    build_kbl(parts[[i]])
  )
}))
if (isTRUE(landscape)) {
  table_blocks <- c("\\begin{landscape}", table_blocks, "\\end{landscape}")
}

tex_doc <- c(
  "\\documentclass{article}",
  "\\usepackage[margin=0.75in]{geometry}",
  "\\usepackage{booktabs}",
  "\\usepackage{longtable}",
  "\\usepackage{pdflscape}",
  "\\usepackage{float}",
  "\\usepackage{graphicx}",
  "\\begin{document}",
  table_blocks,
  "\\end{document}"
)
```

```r
  build_dir <- fs::path(tempdir(), paste0("table_pdf_", file_stub))
  fs::dir_create(build_dir, recurse = TRUE)
  tex_file <- fs::path(build_dir, paste0(file_stub, ".tex"))
  writeLines(tex_doc, tex_file, useBytes = TRUE)

  latex_cmd <- resolve_latex_command("pdflatex")
  latex_log <- tryCatch(
    system2(
      latex_cmd,
      args = c(
        "-interaction=nonstopmode",
        "-halt-on-error",
        "-output-directory", build_dir,
        tex_file
      ),
      stdout = TRUE,
      stderr = TRUE
    ),
    error = function(e) stop("Standalone table PDF render failed for ", file_stub, ": ", conditionMessage(e))
  )
  latex_status <- attr(latex_log, "status")
  if (!is.null(latex_status) && latex_status != 0) {
    stop(
      "Standalone table PDF render failed for ", file_stub, " via pdflatex. Last log lines: ",
      paste(utils::tail(latex_log, 20), collapse = " | ")
    )
  }
  pdf_file <- sub("\\.tex$", ".pdf", tex_file)
  if (!file.exists(pdf_file)) {
    stop("Standalone table PDF render failed for ", file_stub, ": expected PDF not found at ", pdf_file)
  }
  out_pdf <- results_path(paste0(file_stub, ".pdf"))
  fs::file_copy(pdf_file, out_pdf, overwrite = TRUE)
  invisible(out_pdf)
}
```

```r
# helper: write a standalone sidecar PDF for a manuscript table that is rendered
# as explicit panels in the notebook PDF.
write_standalone_panel_table_pdf <- function(panels, caption, file_stub,
                                             digits = 2,
                                             display_label = NULL,
                                             landscape = TRUE,
                                             csv_df = NULL) {
  stopifnot(is.list(panels), length(panels) > 0L)
  panel_names <- names(panels)
  if (is.null(panel_names) || any(!nzchar(panel_names))) {
    panel_names <- paste("Panel", LETTERS[seq_along(panels)])
  }
  stopifnot(all(vapply(panels, is.data.frame, logical(1L))))

  if (is.null(csv_df)) {
    csv_df <- dplyr::bind_rows(
      lapply(seq_along(panels), function(idx) {
        dplyr::mutate(panels[[idx]], panel = panel_names[[idx]], .before = 1L)
      })
    )
  }
  write_csv_safely(csv_df, results_path(paste0(file_stub, ".csv")), row_names = FALSE)

  build_kbl <- function(df_part) {
    kbl <- knitr::kable(
      df_part,
      format = "latex",
      booktabs = TRUE,
      longtable = FALSE,
      digits = digits
    )
    kbl <- kableExtra::kable_styling(
      kbl,
      latex_options = c("hold_position", "scale_down"),
      font_size = 7,
      full_width = FALSE,
```

```
    position = "center"
  )
  latex_safe_table_text(as.character(kbl))
}

table_blocks <- unlist(lapply(seq_along(panels), function(idx) {
  panel_title <- paste0("Panel ", LETTERS[[idx]], ". ", panel_names[[idx]])
  c(
    paste0("\\medskip\\noindent\\textbf{", escape_latex_text(latex_safe_table_text(panel_title)), "}\\par"),
    build_kbl(panels[[idx]])
  )
}))
if (isTRUE(landscape)) {
  table_blocks <- c("\\begin{landscape}", table_blocks, "\\end{landscape}")
}

tex_doc <- c(
  "\\documentclass{article}",
  "\\usepackage[margin=0.75in]{geometry}",
  "\\usepackage{booktabs}",
  "\\usepackage{pdflscape}",
  "\\usepackage{float}",
  "\\usepackage{graphicx}",
  "\\begin{document}",
  table_heading_block(caption, display_label = display_label),
  table_blocks,
  "\\end{document}"
)

build_dir <- fs::path(tempdir(), paste0("table_pdf_", file_stub))
fs::dir_create(build_dir, recurse = TRUE)
tex_file <- fs::path(build_dir, paste0(file_stub, ".tex"))
writeLines(tex_doc, tex_file, useBytes = TRUE)

latex_cmd <- resolve_latex_command("pdflatex")
latex_log <- tryCatch(
```

```r
    system2(
      latex_cmd,
      args = c(
        "-interaction=nonstopmode",
        "-halt-on-error",
        "-output-directory", build_dir,
        tex_file
      ),
      stdout = TRUE,
      stderr = TRUE
    ),
    error = function(e) stop("Standalone panel table PDF render failed for ", file_stub, ": ", conditionMessage(e))
  )
  latex_status <- attr(latex_log, "status")
  if (!is.null(latex_status) && latex_status != 0) {
    stop(
      "Standalone panel table PDF render failed for ", file_stub, " via pdflatex. Last log lines: ",
      paste(utils::tail(latex_log, 20), collapse = " | ")
    )
  }
  pdf_file <- sub("\\.tex$", ".pdf", tex_file)
  if (!file.exists(pdf_file)) {
    stop("Standalone panel table PDF render failed for ", file_stub, ": expected PDF not found at ", pdf_file)
  }
  out_pdf <- results_path(paste0(file_stub, ".pdf"))
  fs::file_copy(pdf_file, out_pdf, overwrite = TRUE)
  invisible(out_pdf)
}

# helper: turn any gtsummary table into a PDF-safe LaTeX table while keeping
# numbering under explicit notebook control.
to_pdf_table <- function(tbl, font_size = 8, landscape = FALSE,
                         label_col_width = NULL, longtable = FALSE,
                         caption_mode = c("plain", "numbered", "none"),
                         display_label = NULL,
                         page_break_before = FALSE) {
```

```r
caption_mode <- match.arg(caption_mode)
tbl_ncol <- tryCatch(ncol(gtsummary::as_tibble(tbl)), error = function(e) NA_integer_)
if (is.finite(tbl_ncol) && tbl_ncol > 6 && font_size > 6) font_size <- 6

caption_text <- tryCatch(tbl$table_styling$caption, error = function(e) NULL)
tbl_local <- tbl
if (!identical(caption_mode, "numbered")) {
  tbl_local$table_styling$caption <- NULL
}

kbl <- gtsummary::as_kable(
  tbl_local,
  format    = "latex",
  booktabs  = TRUE,
  longtable = longtable
)

if (!is.null(label_col_width)) {
  kbl <- kableExtra::column_spec(kbl, 1, width = label_col_width)
}

latex_opts <- if (longtable) {
  c("repeat_header")
} else {
  c("hold_position", "scale_down")
}
kbl <- kableExtra::kable_styling(
  kbl,
  latex_options = latex_opts,
  font_size     = font_size,
  full_width    = !longtable,
  position      = "center"
)

kbl_txt <- latex_safe_table_text(as.character(kbl))
if (identical(caption_mode, "numbered")) {
```

```
    mark_float_emitted()
    return(knitr::asis_output(kbl_txt))
  }
  title_i <- if (identical(caption_mode, "none")) NULL else caption_text
  render_table_blocks(
    kbl_txt,
    title = title_i,
    display_label = display_label,
    landscape = landscape,
    page_break_before = page_break_before
  )
}
```

Package dependencies are managed via `renv` for reproducibility. Use `renv::restore()` before rendering.

1.0.1 Package Set Up

```
# Consolidated package management ------------------------------------------------
seed_pkgs <- c(
  "Matrix", "WeightIt", "broom", "cobalt", "codebookr", "dagitty", "digest",
  "consort", "dplyr", "fastshap", "flextable", "fs", "future", "future.apply", "gbm",
  "ggdag", "ggplot2", "gt", "gtsummary", "haven", "here", "kableExtra",
  "knitr", "labelled", "lubridate", "mice", "miceadds", "mitools", "naniar",
  "officer", "parallel", "patchwork", "pdftools", "progressr", "purrr",
  "ragg", "renv", "rlang", "rms", "scales", "skimr", "survey",
  "tibble", "tidyr", "visdat"
)

# helper: scan the notebook source for package usage so the render fails fast
# if a collaborator has not restored the required `renv` environment.
extract_notebook_pkgs <- function(path) {
  if (!file.exists(path)) return(character())
  lines <- readLines(path, warn = FALSE)
  # Local extractor used for both `library(pkg)` and `pkg::fn()` patterns.
  capture_group <- function(pattern, x) {
    hits <- regexec(pattern, x, perl = TRUE)
```

```r
    matches <- regmatches(x, hits)
    vals <- vapply(matches, function(m) if (length(m) >= 2) m[2] else NA_character_, character(1))
    vals[is.finite(nchar(vals))]
  }
  lib_pkgs <- capture_group("library\\s*\\(\\s*['\"]?([A-Za-z][A-Za-z0-9.]*)['\"]?", lines)
  ns_pkgs <- capture_group("([A-Za-z][A-Za-z0-9.]*)::[A-Za-z][A-Za-z0-9._]*", lines)
  unique(c(lib_pkgs, ns_pkgs))
}

qmd_source <- here::here("Code Drafts", "ABG-VBG analysis 2026-4-21.qmd")
notebook_pkgs <- extract_notebook_pkgs(qmd_source)

renv_dep_pkgs <- character()
if ("renv" %in% rownames(installed.packages())) {
  renv_dep_pkgs <- tryCatch(
    {
      dep_tbl <- renv::dependencies(qmd_source, progress = FALSE)
      if (!is.null(dep_tbl$Package)) unique(dep_tbl$Package) else character()
    },
    error = function(e) character()
  )
}

base_drop <- c("base", "datasets", "graphics", "grDevices", "methods", "stats", "tools", "utils")
ignore_pkgs <- c("pkg")
required_pkgs <- sort(unique(setdiff(c(seed_pkgs, notebook_pkgs), c(base_drop, ignore_pkgs))))
if (length(renv_dep_pkgs)) {
  renv_only <- setdiff(renv_dep_pkgs, c(required_pkgs, base_drop, ignore_pkgs))
  if (length(renv_only)) {
    message("Dependency audit note (renv::dependencies only): ", paste(sort(renv_only), collapse = ", "))
  }
}

# Fail fast if packages are missing (use renv to install)
missing_pkgs <- setdiff(required_pkgs, rownames(installed.packages()))
if (length(missing_pkgs)) {
```

```r
  stop(
    "Missing packages: ", paste(missing_pkgs, collapse = ", "),
    ". Install with renv::install(c(...)); if renv is absent, install 'renv' first; then run renv::snapshot()."
  )
}

# Attach packages from one authoritative location.
ensure_packages_loaded <- function(pkgs) {
  invisible(lapply(pkgs, function(pkg) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
      stop("Required package is not available: ", pkg)
    }
    if (!paste0("package:", pkg) %in% search()) {
      suppressPackageStartupMessages(library(pkg, character.only = TRUE))
    }
  }))
}
ensure_packages_loaded(required_pkgs)

# Centralize outputs under Results/, with paths relative to project root
data_dir_param <- Sys.getenv("ABGVBG_DATA_DIR", unset = params$data_dir)
results_dir_param <- Sys.getenv("ABGVBG_RESULTS_DIR", unset = params$results_dir)
data_dir_name <- fs::path_abs(data_dir_param, start = project_root)
results_dir <- fs::path_abs(results_dir_param, start = project_root)
if (!dir.exists(data_dir_name)) {
  stop("Data directory does not exist: ", data_dir_name)
}
fs::dir_create(results_dir, recurse = TRUE)
fig_dir      <- fs::path(results_dir, "figs")
fig_path     <- paste0(display_path(fig_dir, must_exist = FALSE), "/")
# helper: build paths under `Results/` from one canonical root so later code
# never has to guess where artifacts should be written.
results_path <- function(...) fs::path(results_dir, ...)
fs::dir_create(fig_dir, recurse = TRUE)

knitr::opts_chunk$set(
```

```r
  fig.path   = fig_path,
  dev        = "ragg_png",
  dpi = 300
)
# on macOS and some setups this prevents device headaches
options(bitmapType = "cairo")
```

```r
# Purpose: cohort flow helpers.
format_count <- function(x) {
  format(as.integer(x), big.mark = ",", trim = TRUE, scientific = FALSE)
}

cohort_flow_input_path <- fs::path(data_dir_name, "full_db.dta")
cohort_flow_cols <- c("encounter_type", "has_abg", "has_vbg", "paco2", "vbg_co2")

# helper: read only the columns needed for Figure 1 so the cohort-flow section
# stays fast and does not depend on unrelated variables.
read_cohort_flow_input <- function(path, cols = cohort_flow_cols) {
  stopifnot(file.exists(path))
  haven::read_dta(path, col_select = dplyr::all_of(cols))
}

# helper: compute the count objects that feed the cohort-flow diagram. Keeping
# this logic in one function makes later figure edits less error-prone.
compute_cohort_flow_counts <- function(df) {
  encounter_type_num <- suppressWarnings(as.integer(df$encounter_type))
  if (anyNA(encounter_type_num)) {
    stop(
      "Cohort flow count input has missing encounter_type values (n = ",
      sum(is.na(encounter_type_num)), ")."
    )
  }

  has_abg_num <- suppressWarnings(as.integer(df$has_abg))
  has_vbg_num <- suppressWarnings(as.integer(df$has_vbg))
  has_abg_num[is.na(has_abg_num)] <- 0L
  has_vbg_num[is.na(has_vbg_num)] <- 0L
```

```r
  analytic_flag <- encounter_type_num != 1L
  has_abg_flag <- analytic_flag & has_abg_num == 1L
  has_vbg_flag <- analytic_flag & has_vbg_num == 1L
  both_flag <- has_abg_flag & has_vbg_flag
  abg_only_flag <- has_abg_flag & !has_vbg_flag
  vbg_only_flag <- has_vbg_flag & !has_abg_flag
  neither_flag <- analytic_flag & !has_abg_flag & !has_vbg_flag
  abg_paco2_flag <- has_abg_flag & !is.na(df$paco2)
  vbg_co2_flag <- has_vbg_flag & !is.na(df$vbg_co2)

  analytic_n <- sum(analytic_flag, na.rm = TRUE)
  stopifnot(sum(abg_only_flag, na.rm = TRUE) + sum(vbg_only_flag, na.rm = TRUE) +
              sum(both_flag, na.rm = TRUE) + sum(neither_flag, na.rm = TRUE) == analytic_n)
  stopifnot(sum(abg_only_flag, na.rm = TRUE) + sum(both_flag, na.rm = TRUE) ==
              sum(has_abg_flag, na.rm = TRUE))
  stopifnot(sum(vbg_only_flag, na.rm = TRUE) + sum(both_flag, na.rm = TRUE) ==
              sum(has_vbg_flag, na.rm = TRUE))

  list(
    total_extracted = nrow(df),
    excluded_ambulatory = sum(encounter_type_num == 1L, na.rm = TRUE),
    analytic_ed_inpatient = analytic_n,
    abg_tested = sum(has_abg_flag, na.rm = TRUE),
    vbg_tested = sum(has_vbg_flag, na.rm = TRUE),
    both_abg_vbg = sum(both_flag, na.rm = TRUE),
    abg_only = sum(abg_only_flag, na.rm = TRUE),
    vbg_only = sum(vbg_only_flag, na.rm = TRUE),
    neither_abg_nor_vbg = sum(neither_flag, na.rm = TRUE),
    abg_with_paco2 = sum(abg_paco2_flag, na.rm = TRUE),
    vbg_with_vbg_co2 = sum(vbg_co2_flag, na.rm = TRUE)
  )
}

# helper: add the shared comparability annotation beneath the two weighted
# endpoints without changing the underlying consort graph geometry.
```

```r
decorate_weighted_comparability <- function(built_graph, weighted_nodes,
                                            label_text,
                                            text_gp = grid::gpar(col = "grey25", cex = 0.78),
                                            label_x = grid::unit(0.5, "npc"),
                                            label_y = grid::unit(0.035, "npc")) {
  stopifnot(inherits(built_graph, "gTree"))
  stopifnot(length(weighted_nodes) == 2L)

  band_label <- grid::textGrob(
    label = label_text,
    x = label_x,
    y = label_y,
    just = "center",
    gp = text_gp,
    name = "weighted_comparability_label"
  )

  list(
    built_graph = built_graph,
    pre_draw = NULL,
    post_draw = function() {
      grid::grid.draw(band_label)
    }
  )
}

# helper: export the cohort-flow graph to manuscript artifacts plus an optional
# low-DPI inline preview while drawing the same built grob on each device to
# avoid device-specific layout drift.
save_cohort_flow_outputs <- function(graph, png_file, pdf_file,
                                     width_mm = 180, height_mm = 150,
                                     dpi = 600, bg = "white",
                                     decorate_fn = NULL,
                                     preview_file = NULL,
                                     preview_dpi = 96) {
  width_in <- width_mm / 25.4
```

```r
height_in <- height_mm / 25.4
draw_graph <- function(new_page = TRUE) {
  built_graph <- consort::build_grid(graph)
  decoration <- NULL
  if (!is.null(decorate_fn)) {
    decoration <- decorate_fn(built_graph)
    if (inherits(decoration, "gTree")) {
      built_graph <- decoration
      decoration <- NULL
    } else if (is.list(decoration) && inherits(decoration$built_graph, "gTree")) {
      built_graph <- decoration$built_graph
    }
  }
  # Raster devices and inline replay paths often need an explicit new page,
  # but a freshly opened PDF device already has page 1 active. Calling
  # grid.newpage() there creates a blank first page and pushes the diagram to
  # page 2, which LaTeX then includes incorrectly.
  if (isTRUE(new_page)) {
    grid::grid.newpage()
  }
  if (is.list(decoration) && is.function(decoration$pre_draw)) {
    decoration$pre_draw()
  }
  grid::grid.draw(built_graph)
  if (is.list(decoration) && is.function(decoration$post_draw)) {
    decoration$post_draw()
  }
}

fs::dir_create(fs::path_dir(png_file), recurse = TRUE)
fs::dir_create(fs::path_dir(pdf_file), recurse = TRUE)

ragg::agg_png(
  filename = png_file,
  width = width_mm,
  height = height_mm,
```

```r
  units = "mm",
  res = dpi,
  background = bg
)
draw_graph(new_page = TRUE)
grDevices::dev.off()

if (capabilities("cairo")) {
  grDevices::cairo_pdf(
    filename = pdf_file,
    width = width_in,
    height = height_in,
    bg = bg
  )
} else {
  grDevices::pdf(
    file = pdf_file,
    width = width_in,
    height = height_in,
    bg = bg
  )
}
draw_graph(new_page = FALSE)
grDevices::dev.off()

if (!is.null(preview_file)) {
  fs::dir_create(fs::path_dir(preview_file), recurse = TRUE)
  ragg::agg_png(
    filename = preview_file,
    width = width_mm,
    height = height_mm,
    units = "mm",
    res = preview_dpi,
    background = bg
  )
  draw_graph(new_page = TRUE)
```

```r
    grDevices::dev.off()
  }

  stopifnot(file.exists(png_file), file.exists(pdf_file))
  if (!is.null(preview_file)) stopifnot(file.exists(preview_file))
  if (requireNamespace("pdftools", quietly = TRUE)) {
    pdf_pages <- tryCatch(pdftools::pdf_info(pdf_file)$pages, error = function(e) NA_integer_)
    if (is.finite(pdf_pages) && pdf_pages != 1L) {
      stop("Cohort flow PDF export should be 1 page, got ", pdf_pages, ".")
    }
  }
  invisible(list(png = png_file, pdf = pdf_file, preview = preview_file))
}

cohort_flow_input <- read_cohort_flow_input(cohort_flow_input_path)
cohort_flow_counts <- compute_cohort_flow_counts(cohort_flow_input)
rm(cohort_flow_input)
```

```r
# Purpose: manuscript Figure 1 cohort flow figure.
flow_png <- results_path("cohort_flow.png")
flow_pdf <- results_path("cohort_flow.pdf")

top_box <- paste0(
  "Extracted 2022 encounter cohort\n",
  "(n = ", format_count(cohort_flow_counts$total_extracted), ")"
)

screening_side <- paste0(
  "Ambulatory encounters excluded\n",
  "(n = ", format_count(cohort_flow_counts$excluded_ambulatory), ")"
)

analytic_base_box <- paste0(
  "Analytic cohort: ED and Inpatient Encounters\n",
  "(n = ", format_count(cohort_flow_counts$analytic_ed_inpatient), ")"
)
```

```r
split_boxes <- c(
  paste0(
    "ABG-tested encounters\n",
    "(n = ", format_count(cohort_flow_counts$abg_tested), ")"
  ),
  paste0(
    "VBG-tested encounters\n",
    "(n = ", format_count(cohort_flow_counts$vbg_tested), ")"
  )
)

weighted_box <- paste0(
  "ABG-weighted sample\n",
  "standardized to analytic cohort"
)
weighted_boxes <- c(
  weighted_box,
  paste0(
    "VBG-weighted sample\n",
    "standardized to analytic cohort"
  )
)

target_cohort_box_gp <- grid::gpar(fill = "grey96", col = "grey25", lwd = 1)

cohort_flow_graph <- consort::add_box(txt = top_box, text_width = 34) |>
  consort::add_side_box(txt = screening_side, text_width = 30) |>
  consort::add_box(
    txt = analytic_base_box,
    text_width = 34,
    box_gp = target_cohort_box_gp
  ) |>
  consort::add_split(txt = split_boxes, text_width = 24) |>
  consort::add_box(
    txt = weighted_boxes,
    text_width = 24,
```

```r
    box_gp = target_cohort_box_gp
  )

weighted_nodes <- attr(cohort_flow_graph, "nodes.current")
stopifnot(length(weighted_nodes) == 2L)

save_cohort_flow_outputs(
  cohort_flow_graph,
  flow_png,
  flow_pdf,
  decorate_fn = function(built_graph) {
    decorate_weighted_comparability(
      built_graph,
      weighted_nodes = weighted_nodes,
      label_text = "Conditionally comparable after propensity weighting"
    )
  }
)
invisible(flow_png)
```

```r
# Purpose: setup run config.
stopifnot(!is.null(params$run_mode))
stopifnot(!is.null(params$pilot_frac))
stopifnot(!is.null(params$sample_seed))

RUN_MODE <- match.arg(tolower(params$run_mode), c("pilot", "full"))
PILOT_FRAC <- as.numeric(params$pilot_frac)
stopifnot(is.finite(PILOT_FRAC), PILOT_FRAC > 0, PILOT_FRAC <= 1)
IS_PILOT <- identical(RUN_MODE, "pilot")
if (IS_PILOT) {
  if (!(PILOT_FRAC > 0 && PILOT_FRAC < 1)) {
    stop("RUN_MODE='pilot' requires 0 < PILOT_FRAC < 1.")
  }
} else {
  if (!isTRUE(all.equal(PILOT_FRAC, 1))) {
    warning("RUN_MODE='full' ignores pilot_frac and forces PILOT_FRAC = 1.", call. = FALSE)
  }
```

```r
  PILOT_FRAC <- 1
}
SAMPLE_SEED <- as.integer(params$sample_seed)
stopifnot(is.finite(SAMPLE_SEED))

pilot_frac <- PILOT_FRAC
FULL_RUN <- identical(RUN_MODE, "full")
MI_BATCH_THRESHOLD_PILOT <- 5000L
MI_BATCH_THRESHOLD_FULL <- 200000L
MI_BATCH_THRESHOLD <- if (IS_PILOT) MI_BATCH_THRESHOLD_PILOT else MI_BATCH_THRESHOLD_FULL
stopifnot(is.finite(MI_BATCH_THRESHOLD), MI_BATCH_THRESHOLD > 0)
PLOT_DROP_POLICY <- "warn"
PROB_EPS <- 1e-6
MAX_SHAPE_LEVELS <- 6L
SPLINE_GRID_N <- 200L
OR_XLIM <- c(0.25, 16)
TAIL_N_FINITE <- 5L
TAIL_WINDOW_ITERS <- 10L
BAL_XLIM_MAX <- 1.0
MC_ERR_RATIO_THRESH <- 0.10
MAX_LEVELS_GBM <- 50L
GBM_MM_EST_BYTES_WARN <- 6e9
MAX_GBM_MM_COLS_WARN <- 200L
MAX_GBM_MM_COLS_STOP <- 400L
MAX_GBM_FACTOR_LEVELS_WARN <- 30L
MAX_GBM_FACTOR_LEVELS_STOP <- 60L

message(
  "CONFIG: RUN_MODE=", RUN_MODE,
  " | PILOT_FRAC=", PILOT_FRAC,
  " | SAMPLE_SEED=", SAMPLE_SEED
)
```

```r
# Purpose: setup diagnostics output.
diag_run_id <- format(Sys.time(), "%Y%m%d_%H%M%S")
diag_run_ts <- as.character(Sys.time())
run_id <- diag_run_id
```

```r
run_ts <- diag_run_ts
runtime_run_id <- diag_run_id
options(run_id = run_id, run_ts = run_ts)
runtime_log <- data.frame()
mi_warn_log <- data.frame(
  time = character(), stage = character(), component = character(),
  analysis_variant = character(), model_type = character(),
  group = character(), outcome = character(), imputation = integer(),
  batch = integer(), message = character(), stringsAsFactors = FALSE
)
mi_info_log <- mi_warn_log
mi_outcome_diag <- data.frame()
memory_snapshots <- data.frame()
gbm_preflight_design_dims <- data.frame()
gbm_preflight_warnings <- data.frame()
factor_diag_written <- character()
plot_drop_log_df <- data.frame(
  plot = character(), reason = character(), n_dropped = integer(),
  n_before = integer(), n_after = integer(),
  stage = character(), extra = character(),
  stringsAsFactors = FALSE
)

# helper: clear diagnostics from prior renders so each notebook run writes a
# fresh, internally consistent set of audit outputs.
cleanup_diagnostics_outputs <- function() {
  diag_files <- c(
    "runtime_log.csv", "runtime_summary.csv", "runtime_summary_top15.csv",
    "warnings_log.csv", "mi_warnings_log.csv", "mi_info_log.csv",
    "diagnostics_summary.csv", "diagnostics_missingness.csv",
    "diagnostics_missingness-by-strata.csv",
    "balance_target_imp_summary.csv", "balance_target_by_imp.csv",
    "balance_target_worst_rows.csv", "balance_max_smd_by_imp.csv",
    "balance_worst_terms.csv", "balance_worst10.csv", "balance_table.csv",
    "weight_summary.csv", "ps_overlap_summary.csv",
    "model_fit_diagnostics.csv", "mi_outcome_fit_diagnostics.csv",
```

```
  "mi_fit_issue_summary.csv", "mi_m_stability.csv", "mi_maxit_sensitivity.csv",
  "mi_obs_vs_imp_summary.csv", "mi_spline_curve_abg.csv",
  "mi_spline_curve_vbg.csv",
  "mi_spline_prob_curve_abg.csv", "mi_spline_prob_curve_vbg.csv",
  "mi_spline_coef_abg.csv",
  "mi_spline_coef_vbg.csv", "diag-ps-shap-stability.csv",
  "mice_smoketest.log", "mice_batches_log.csv", "mice_chain_diagnostics.csv",
  "mice_pred_width_preflight.csv", "mice_logged_events_raw.csv",
  "mice_logged_events_summary.csv", "mice_spec.rds",
  "missingness-by-strata.csv", "missingness-drivers.csv",
  "missingness-pattern.csv", "plot_drop_log.csv",
  "plot_registry.csv",
  "mi_logistic_ps_covariate_types.csv",
  "mi_logistic_ps_abg_list.rds",
  "mi_logistic_ps_vbg_list.rds"
)
diag_patterns <- c(
  "^mice_.*\\.(csv|log|rds|txt)$",
  "^diagnostics_.*\\.csv$",
  "^runtime_.*\\.csv$",
  "^warnings_.*\\.csv$",
  "^balance_.*\\.csv$",
  "^weight_summary\\.csv$",
  "^ps_overlap_summary\\.csv$",
  "^model_fit_diagnostics\\.csv$",
  "^plot_drop_log\\.csv$",
  "^diag-.*\\.csv$",
  "^mi_logistic_ps_.*\\.(csv|rds)$"
)
files <- list.files(results_dir, full.names = TRUE)
base <- basename(files)
match_files <- base %in% diag_files
if (length(diag_patterns)) {
  match_files <- match_files | grepl(paste(diag_patterns, collapse = "|"), base)
}
to_delete <- files[match_files]
```

```r
  if (length(to_delete)) {
    safe_root <- normalizePath(results_dir, winslash = "/", mustWork = FALSE)
    del_paths <- normalizePath(to_delete, winslash = "/", mustWork = FALSE)
    stopifnot(all(startsWith(del_paths, safe_root)))
  }
  fs::file_delete(to_delete)

  diag_figs <- list.files(fig_dir, full.names = TRUE)
  diag_figs <- diag_figs[grepl("^(diag-|or-plot|ps-)|-report-preview\\.png$", basename(diag_figs))]
  if (length(diag_figs)) {
    safe_root <- normalizePath(results_dir, winslash = "/", mustWork = FALSE)
    del_paths <- normalizePath(diag_figs, winslash = "/", mustWork = FALSE)
    stopifnot(all(startsWith(del_paths, safe_root)))
  }
  fs::file_delete(diag_figs)
  {
    d <- fs::path(results_dir, "mice_batches")
    if (dir.exists(d)) fs::dir_delete(d)
  }
  {
    d <- fs::path(results_dir, "diagnostics_audit_snippets")
    if (dir.exists(d)) fs::dir_delete(d)
  }
  {
    f <- fs::path(results_dir, "mi_partial_mids.rds")
    if (file.exists(f)) fs::file_delete(f)
  }
}

# helper: remove heavyweight MI scratch artifacts after successful renders so
# routine notebook runs do not silently accumulate stale intermediate objects.
cleanup_mi_transient_outputs <- function() {

  transient_files <- c(
    results_path("mi_abg_vbg_mids.rds"),
    results_path("subset_data_pre_mi.rds"),
```

```
    results_path("mi_logistic_ps_abg_list.rds"),
    results_path("mi_logistic_ps_vbg_list.rds")
  )
  transient_dirs <- c(
    results_path("mi_weights")
  )

  existing_files <- transient_files[file.exists(transient_files)]
  if (length(existing_files)) {
    safe_root <- normalizePath(results_dir, winslash = "/", mustWork = FALSE)
    del_paths <- normalizePath(existing_files, winslash = "/", mustWork = FALSE)
    stopifnot(all(startsWith(del_paths, safe_root)))
    fs::file_delete(existing_files)
  }

  existing_dirs <- transient_dirs[dir.exists(transient_dirs)]
  if (length(existing_dirs)) {
    safe_root <- normalizePath(results_dir, winslash = "/", mustWork = FALSE)
    del_paths <- normalizePath(existing_dirs, winslash = "/", mustWork = FALSE)
    stopifnot(all(startsWith(del_paths, safe_root)))
    fs::dir_delete(existing_dirs)
  }

  invisible(length(existing_files) + length(existing_dirs) > 0L)
}

cleanup_diagnostics_outputs()
cleanup_mi_transient_outputs()
# Ensure any stale Results artifacts from prior runs are removed
stale_patterns <- c(
  "^missingness_by_strata\\.csv$",
  "^missingness-drivers\\.csv$",
  "^missingness_by_strata_preview\\.csv$",
  "^missingness_drivers_top20\\.csv$",
  "^missingness_top10\\.csv$",
  "^outcome_counts_by_cohort\\.csv$",
```

```
  "^weighting_diagnostics_non_mi\\.csv$",
  "^mi_specification\\.csv$",
  "^memory_snapshots\\.csv$",
  "^gbm_preflight_design_dims\\.csv$",
  "^gbm_preflight_factor_levels_.*\\.csv$",
  "^diag_mi_shap_errors\\.csv$",
  "^diag_mi_shap_method_used\\.csv$",
  "^mi_shap_imp_diagnostics\\.csv$",
  "^mi_logistic_ps_factor_levels_.*\\.csv$",
  "^mi_logistic_ps_.*\\.(csv|rds)$",
  "^get_imp_usage\\.csv$",
  "^diagnostics_audit\\.md$",
  "^pdf_page_text_stats\\.csv$",
  "^pdf_blank_pages\\.json$",
  "^pdf_hygiene_scan\\.csv$"
)
stale_files <- list.files(results_dir, full.names = TRUE)
stale_base <- basename(stale_files)
stale_files <- stale_files[grepl(paste(stale_patterns, collapse = "|"), stale_base)]
if (length(stale_files)) fs::file_delete(stale_files)
```

1.1 Helper functions for model diagnostics

```
# Diagnostics helper functions for MI/IPW pipeline

# helper: fail early if an analysis stage expects complete covariates but the
# current data still contains missing values in those columns.
assert_no_na_covars <- function(df, covars, context = "") {
  missing <- setdiff(covars, names(df))
  stopifnot(length(missing) == 0)
  na_counts <- vapply(df[, covars, drop = FALSE], function(x) sum(is.na(x)), numeric(1))
  if (any(na_counts > 0)) {
    msg <- paste0(
      "NA values found in covariates (", context, "): ",
      paste(names(na_counts)[na_counts > 0], na_counts[na_counts > 0], collapse = ", ")
    )
```

```r
    stop(msg)
  }
  invisible(TRUE)
}

# helper: collapse the raw `mice` logged-events table into counts that are
# easier to review in diagnostics CSVs and notebook output.
summarize_logged_events <- function(mids_obj) {
  ev <- mids_obj$loggedEvents
  if (is.null(ev) || nrow(ev) == 0L) {
    return(data.frame(
      variable = character(),
      method = character(),
      event = character(),
      n = integer(),
      pct = numeric(),
      stringsAsFactors = FALSE
    ))
  }
  stopifnot(all(c("dep", "meth", "out") %in% names(ev)))
  tbl <- as.data.frame(table(ev$dep, ev$meth, ev$out), stringsAsFactors = FALSE)
  names(tbl) <- c("variable", "method", "event", "n")
  tbl <- tbl[tbl$n > 0, , drop = FALSE]
  tbl <- tbl[order(-tbl$n), , drop = FALSE]
  tbl$pct <- tbl$n / sum(tbl$n)
  tbl
}

# helper: consistently pull propensity scores out of WeightIt objects.
extract_weightit_ps <- function(w) {
  stopifnot(!is.null(w$ps))
  as.numeric(w$ps)
}

# helper: effective sample size for a vector of weights.
ess <- function(w) {
```

```r
  w <- w[is.finite(w)]
  stopifnot(length(w) > 0)
  sum(w)^2 / sum(w^2)
}

# helper: quantify how much total weight is concentrated in the largest weights.
weight_concentration <- function(w, top_p = 0.01) {
  w <- w[is.finite(w)]
  stopifnot(length(w) > 0)
  ord <- sort(w, decreasing = TRUE)
  n_top <- max(1L, floor(length(ord) * top_p))
  sum(ord[seq_len(n_top)]) / sum(ord)
}

# helper: summarize one set of inverse-probability weights and their matching
# propensity scores for later diagnostics tables.
weight_summary <- function(w, ps, ps_floor, truncated) {
  w_ok <- w[is.finite(w)]
  stopifnot(length(w_ok) > 0)
  ps_ok <- ps[is.finite(ps)]
  stopifnot(length(ps_ok) > 0)
  out <- data.frame(
    n = length(w_ok),
    mean = mean(w_ok, na.rm = TRUE),
    sd = stats::sd(w_ok, na.rm = TRUE),
    min = min(w_ok, na.rm = TRUE),
    p01 = stats::quantile(w_ok, 0.01, na.rm = TRUE),
    p05 = stats::quantile(w_ok, 0.05, na.rm = TRUE),
    p95 = stats::quantile(w_ok, 0.95, na.rm = TRUE),
    p99 = stats::quantile(w_ok, 0.99, na.rm = TRUE),
    max = max(w_ok, na.rm = TRUE),
    sum_w = sum(w_ok, na.rm = TRUE),
    ess = ess(w_ok),
    top01_weight_share = weight_concentration(w_ok, top_p = 0.01),
    ps_floor = ps_floor,
    trunc_rate = mean(truncated, na.rm = TRUE),
```

```r
    ps_min = min(ps_ok, na.rm = TRUE),
    ps_p01 = stats::quantile(ps_ok, 0.01, na.rm = TRUE),
    ps_p05 = stats::quantile(ps_ok, 0.05, na.rm = TRUE),
    ps_p95 = stats::quantile(ps_ok, 0.95, na.rm = TRUE),
    ps_max = max(ps_ok, na.rm = TRUE),
    stringsAsFactors = FALSE
  )
  out
}

# helper: convert fitted propensities into the project-specific one-sided IPSW
# weights used to standardize the tested cohorts back to the analytic cohort.
compute_ipow_weights <- function(w, treat, ps_floor_quantile = 0.01,
                                 stabilize = TRUE) {
  # One-sided IPSW for this project:
  # outcome models are fit only within the tested cohort, so only tested rows
  # receive inverse-propensity weights. The target is the full analytic cohort,
  # not the untested subgroup. We truncate extremely small propensities to avoid
  # unstable huge weights, then optionally stabilize so treated weights average ~1.
  ps <- extract_weightit_ps(w)
  if (length(ps) != length(treat)) stop("Propensity length mismatch.")
  treat <- as.integer(treat)
  if (!all(treat %in% c(0L, 1L))) stop("Treatment indicator must be 0/1.")

  ps_obs <- ps[treat == 1L & is.finite(ps)]
  if (!length(ps_obs)) stop("No treated observations with finite propensity.")
  ps_floor <- as.numeric(stats::quantile(ps_obs, probs = ps_floor_quantile, na.rm = TRUE))
  if (!is.finite(ps_floor) || ps_floor <= 0) stop("Invalid propensity floor.")

  w_raw <- rep(NA_real_, length(ps))
  w_raw[treat == 1L] <- 1 / ps[treat == 1L]

  cap <- 1 / ps_floor
  truncated <- treat == 1L & is.finite(ps) & ps < ps_floor
  w_trunc <- w_raw
  w_trunc[truncated] <- cap
```

```r
  if (stabilize) {
    w_trunc <- w_trunc / mean(w_trunc[treat == 1L], na.rm = TRUE)
  }

  list(
    weights   = w_trunc,
    ps        = ps,
    ps_floor  = ps_floor,
    cap       = cap,
    truncated = truncated
  )
}

# helper: stop immediately if a weighting step produces NaN/Inf values.
assert_finite_weights <- function(w, name = "weights") {
  if (any(!is.finite(w))) {
    stop("Non-finite values detected in ", name, ".")
  }
  invisible(TRUE)
}

# helper: time an expression and append one standardized row to the runtime log.
runtime_logger <- function(step_name, expr, notes = NA_character_) {
  start_time <- Sys.time()
  result <- eval(expr)
  end_time <- Sys.time()
  sec <- as.numeric(difftime(end_time, start_time, units = "secs"))
  row <- data.frame(
    step_name = step_name,
    seconds = sec,
    start_time = as.character(start_time),
    end_time = as.character(end_time),
    notes = notes,
    run_id = runtime_run_id,
    run_mode = RUN_MODE,
```

```r
    n_subset = subset_n,
    stringsAsFactors = FALSE
  )
  runtime_log <<- dplyr::bind_rows(runtime_log, row)
  result
}

# helper: create a consistently labeled context object so warnings and model
# diagnostics can be traced back to the exact analysis stage that produced them.
make_context <- function(stage, component,
                         analysis_variant = NA_character_,
                         model_type = NA_character_,
                         group = NA_character_,
                         outcome = NA_character_,
                         imputation = NA_integer_,
                         batch = NA_integer_) {
  list(
    stage = stage,
    component = component,
    analysis_variant = analysis_variant,
    model_type = model_type,
    group = group,
    outcome = outcome,
    imputation = imputation,
    batch = batch
  )
}

# helper: evaluate an expression while intercepting warnings into a structured
# data frame instead of letting them vanish into console output.
capture_warnings <- function(expr, context) {
  warn_rows <- list()
  stopifnot(all(c("stage", "component", "analysis_variant", "model_type",
                  "group", "outcome", "imputation", "batch") %in% names(context)))
  ctx <- list(
    stage = as.character(context$stage),
```

```
  component = as.character(context$component),
  analysis_variant = as.character(context$analysis_variant),
  model_type = as.character(context$model_type),
  group = as.character(context$group),
  outcome = as.character(context$outcome),
  imputation = as.integer(context$imputation),
  batch = as.integer(context$batch)
)
val <- withCallingHandlers(
  expr,
  warning = function(w) {
    warn_rows <<- append(warn_rows, list(data.frame(
      time = as.character(Sys.time()),
      stage = ctx$stage,
      component = ctx$component,
      analysis_variant = ctx$analysis_variant,
      model_type = ctx$model_type,
      group = ctx$group,
      outcome = ctx$outcome,
      imputation = ctx$imputation,
      batch = ctx$batch,
      message = conditionMessage(w),
      stringsAsFactors = FALSE
    )))
    invokeRestart("muffleWarning")
  }
)
warnings_df <- if (length(warn_rows)) {
  dplyr::bind_rows(warn_rows)
} else {
  data.frame(
    time = character(), stage = character(), component = character(),
    analysis_variant = character(), model_type = character(),
    group = character(), outcome = character(), imputation = integer(),
    batch = integer(), message = character(), stringsAsFactors = FALSE
  )
```

```r
  }
  list(value = val, warnings = warnings_df)
}

# helper: append captured warnings to the global MI warning logs, while
# separating informational `mice` messages from actionable warnings.
append_warnings <- function(wlist) {
  stopifnot(is.data.frame(wlist) || is.list(wlist))
  df <- if (is.data.frame(wlist)) wlist else dplyr::bind_rows(wlist)
  stopifnot(is.data.frame(df))
  stopifnot("message" %in% names(df))
  info_rows <- df[grepl("^Number of logged events", df$message), , drop = FALSE]
  if (nrow(info_rows)) {
    mi_info_log <<- dplyr::bind_rows(mi_info_log, info_rows)
  }
  df <- df[!grepl("^Number of logged events", df$message), , drop = FALSE]
  if (nrow(df) == 0L) return(invisible(FALSE))
  mi_warn_log <<- dplyr::bind_rows(mi_warn_log, df)
  invisible(TRUE)
}

# helper: resolve the active notebook path across Quarto/knitr working-directory
# changes so later audits can always locate the source file.
resolve_current_qmd <- function() {
  # Resolve qmd path robustly across render/knit working directories.
  candidates <- c(
    tryCatch(knitr::current_input(), error = function(e) NA_character_),
    tryCatch(knitr::current_input(dir = TRUE), error = function(e) NA_character_),
    here::here("Code Drafts", "ABG-VBG analysis 2026-4-21.qmd"),
    "Code Drafts/ABG-VBG analysis 2026-4-21.qmd",
    "ABG-VBG analysis 2026-4-21.qmd",
    here::here("Code Drafts", "ABG-VBG analysis 2025-12-11.qmd"),
    "Code Drafts/ABG-VBG analysis 2025-12-11.qmd",
    "ABG-VBG analysis 2025-12-11.qmd"
  )
  candidates <- candidates[!is.na(candidates) & nzchar(candidates)]
```

```r
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) {
    stop("Could not resolve qmd source path for audit checks.")
  }
  normalizePath(existing[[1]], winslash = "/", mustWork = TRUE)
}

# helper: pull warning tables out of lists of fit results and append them to
# the notebook-wide warning logs in one call.
collect_warnings_from_list <- function(xlist) {
  warn_list <- lapply(xlist, function(x) {
    if (is.list(x) && !is.null(x$warnings)) x$warnings else NULL
  })
  warn_list <- warn_list[!vapply(warn_list, is.null, logical(1))]
  append_warnings(warn_list)
  invisible(TRUE)
}

# helper: accumulate per-model outcome diagnostics into one project-wide table.
append_outcome_diag <- function(df) {
  stopifnot(is.data.frame(df))
  if (nrow(df) == 0L) return(invisible(FALSE))
  mi_outcome_diag <<- dplyr::bind_rows(mi_outcome_diag, df)
  invisible(TRUE)
}

# helper: collapse the warnings log to a concise count table for review.
summarize_warnings_log <- function(path = results_path("warnings_log.csv")) {
  warn_df <- read.csv(path, stringsAsFactors = FALSE)
  if (nrow(warn_df) == 0L) return(data.frame())
  warn_df |>
    dplyr::count(stage, component, analysis_variant, model_type, message, sort = TRUE)
}

# helper: write CSV outputs with run identifiers attached so later audits can
# tell which render produced a given artifact.
```

```r
write_csv_safely <- function(df, path, row_names = FALSE, required_cols = NULL) {
  run_id <- diag_run_id
  run_ts <- diag_run_ts
  stopifnot(!is.null(df))
  df <- as.data.frame(df)
  # Normalize pillar/vctrs numeric classes to base numeric for CSV export
  df <- dplyr::mutate(
    df,
    dplyr::across(
      where(~ inherits(.x, "pillar_num")),
      ~ suppressWarnings(as.numeric(.x))
    )
  )
  if (!is.null(required_cols)) {
    missing <- setdiff(required_cols, names(df))
    stopifnot(length(missing) == 0)
  }
  if (nrow(df) == 0L) {
    if (!("empty" %in% names(df))) df$empty <- logical(0)
  }
  df$run_id <- rep(run_id, nrow(df))
  df$run_ts <- rep(run_ts, nrow(df))
  utils::write.csv(df, path, row.names = row_names)
  invisible(TRUE)
}

# helper: write line-based diagnostics with the same run metadata header used
# for CSV artifacts.
write_diag_lines <- function(lines, path) {
  run_id <- diag_run_id
  run_ts <- diag_run_ts
  header <- paste0("run_id: ", run_id, " run_ts: ", run_ts)
  writeLines(c(header, lines), con = path)
  invisible(TRUE)
}
```

```r
# helper: drop unused factor levels after filtering so model frames and tables
# do not silently carry dead categories.
droplevels_all <- function(df, vars = NULL) {
  if (is.null(vars)) {
    vars <- names(df)[vapply(df, is.factor, logical(1))]
  } else {
    vars <- intersect(vars, names(df))
    vars <- vars[vapply(df[vars], is.factor, logical(1))]
  }
  for (nm in vars) df[[nm]] <- droplevels(df[[nm]])
  df
}

# helper: append one GC-based memory snapshot for a named stage. This makes it
# easier to identify where long renders begin to consume disproportionate RAM.
append_mem_snapshot <- function(stage_id, context_id = NA_character_, when = "pre") {
  g <- gc()
  cn <- colnames(g)
  used_col  <- if ("used" %in% cn) "used" else cn[grepl("used", cn)][1]
  trig_col  <- if ("gc trigger" %in% cn) "gc trigger" else cn[grepl("trigger", cn)][1]
  max_col   <- if ("max used" %in% cn) "max used" else cn[grepl("max", cn)][1]
  n_used <- as.numeric(g["Ncells", used_col])
  v_used <- as.numeric(g["Vcells", used_col])
  v_trig <- as.numeric(g["Vcells", trig_col])
  v_max  <- as.numeric(g["Vcells", max_col])
  mem_max <- tryCatch(mem.maxVSize(), error = function(e) NA_real_)
  row <- data.frame(
    run_id = diag_run_id,
    run_ts = diag_run_ts,
    stage_id = stage_id,
    context_id = context_id,
    when = when,
    time = as.character(Sys.time()),
    Ncells_used = n_used,
    Vcells_used = v_used,
    Vcells_gc_trigger = v_trig,
```

```r
    Vcells_max_used = v_max,
    mem_max_vsize = mem_max,
    stringsAsFactors = FALSE
  )
  memory_snapshots <<- dplyr::bind_rows(memory_snapshots, row)
  invisible(TRUE)
}

# helper: write a compact CSV describing factor cardinality before expensive
# GBM fits, since high-cardinality factors are a common failure mode.
write_factor_levels_diag <- function(df, vars, context_id, file_prefix = "gbm_preflight_factor_levels") {
  key <- paste0(file_prefix, "::", context_id)
  if (key %in% factor_diag_written) {
    return(invisible(FALSE))
  }
  vars <- intersect(vars, names(df))
  if (!length(vars)) {
    return(invisible(FALSE))
  }
  diag <- lapply(vars, function(v) {
    x <- df[[v]]
    if (!is.factor(x)) return(NULL)
    tab <- table(x, useNA = "ifany")
    n_unused <- sum(tab == 0)
    top <- utils::head(sort(tab, decreasing = TRUE), 5)
    data.frame(
      variable = v,
      class = class(x)[1],
      nlevels = nlevels(x),
      n_unused_levels = n_unused,
      top_levels = paste(names(top), as.integer(top), sep = "=", collapse = "; "),
      n_missing = sum(is.na(x)),
      context_id = context_id,
      stringsAsFactors = FALSE
    )
  })
```

```r
  diag <- dplyr::bind_rows(diag)
  if (nrow(diag) == 0L) return(invisible(FALSE))
  out_path <- results_path(paste0(file_prefix, "_", context_id, ".csv"))
  write_csv_safely(diag, out_path, row_names = FALSE)
  factor_diag_written <<- c(factor_diag_written, key)
  invisible(TRUE)
}

# helper: estimate sampled design-matrix width so large GBM model matrices can
# be anticipated before the full fit is attempted.
log_design_dims <- function(df, covars, context_id, sample_n = 5000L) {
  covars <- intersect(covars, names(df))
  if (!length(covars)) return(invisible(FALSE))
  n_full <- nrow(df)
  n_samp <- min(sample_n, n_full)
  set.seed(20251206)
  idx <- sample.int(n_full, n_samp)
  df_s <- df[idx, covars, drop = FALSE]
  fml <- stats::as.formula(paste("~", paste(covars, collapse = " + ")))
  mm <- stats::model.matrix(fml, data = df_s)
  n_factor <- sum(vapply(df_s, is.factor, logical(1)))
  max_nlevels <- if (n_factor > 0) max(vapply(df_s[vapply(df_s, is.factor, logical(1))], nlevels, integer(1))) else 0L
  row <- data.frame(
    run_id = diag_run_id,
    run_ts = diag_run_ts,
    context_id = context_id,
    nrow_full = n_full,
    nrow_sample = n_samp,
    ncol_model_matrix = ncol(mm),
    n_factor_vars = n_factor,
    max_nlevels_factor = max_nlevels,
    stringsAsFactors = FALSE
  )
  gbm_preflight_design_dims <<- dplyr::bind_rows(gbm_preflight_design_dims, row)
  invisible(TRUE)
}
```

```r
# helper: run the full GBM preflight suite: drop empty levels, estimate design
# size, log warnings, and stop when the model matrix looks infeasible.
gbm_preflight <- function(df, covars, context_id, sample_n = 50000L) {
  df <- droplevels_all(df)
  write_factor_levels_diag(df, covars, context_id, file_prefix = "gbm_preflight_factor_levels")
  covars <- intersect(covars, names(df))
  if (!length(covars)) return(invisible(FALSE))
  n_full <- nrow(df)
  n_samp <- min(sample_n, n_full)
  set.seed(20251206)
  idx <- sample.int(n_full, n_samp)
  df_s <- df[idx, covars, drop = FALSE]
  fml <- stats::as.formula(paste("~", paste(covars, collapse = " + ")))
  mm <- stats::model.matrix(fml, data = df_s)
  n_factor <- sum(vapply(df_s, is.factor, logical(1)))
  max_nlevels <- if (n_factor > 0) {
    max(vapply(df_s[vapply(df_s, is.factor, logical(1))], nlevels, integer(1)))
  } else 0L
  est_bytes <- n_full * ncol(mm) * 8
  row <- data.frame(
    run_id = diag_run_id,
    run_ts = diag_run_ts,
    context_id = context_id,
    nrow_full = n_full,
    nrow_sample = n_samp,
    ncol_model_matrix = ncol(mm),
    n_factor_vars = n_factor,
    max_nlevels_factor = max_nlevels,
    est_bytes = est_bytes,
    est_gb = est_bytes / 1024^3,
    stringsAsFactors = FALSE
  )
  gbm_preflight_design_dims <<- dplyr::bind_rows(gbm_preflight_design_dims, row)
  warn_msgs <- character()
  if (is.finite(est_bytes) && est_bytes > GBM_MM_EST_BYTES_WARN) {
```

```r
    warn_msgs <- c(warn_msgs, paste0("est_dense_mm_gb>", round(GBM_MM_EST_BYTES_WARN / 1024^3, 2)))
  }
  if (ncol(mm) > MAX_GBM_MM_COLS_WARN) {
    warn_msgs <- c(warn_msgs, paste0("mm_cols_warn>", MAX_GBM_MM_COLS_WARN))
  }
  if (max_nlevels > MAX_GBM_FACTOR_LEVELS_WARN) {
    warn_msgs <- c(warn_msgs, paste0("factor_levels_warn>", MAX_GBM_FACTOR_LEVELS_WARN))
  }
  if (length(warn_msgs)) {
    gbm_preflight_warnings <<- dplyr::bind_rows(
      gbm_preflight_warnings,
      data.frame(
        run_id = diag_run_id,
        run_ts = diag_run_ts,
        context_id = context_id,
        nrow_full = n_full,
        ncol_model_matrix = ncol(mm),
        max_nlevels_factor = max_nlevels,
        est_gb = est_bytes / 1024^3,
        warning = paste(warn_msgs, collapse = ";"),
        stringsAsFactors = FALSE
      )
    )
  }
  if (ncol(mm) > MAX_GBM_MM_COLS_STOP || max_nlevels > MAX_GBM_FACTOR_LEVELS_STOP) {
    stop(
      "GBM preflight stop: context=", context_id,
      "; ncol_model_matrix=", ncol(mm),
      "; max_nlevels_factor=", max_nlevels,
      ". Consider collapsing factor levels or removing high-cardinality predictors."
    )
  }
  invisible(TRUE)
}

# helper: print a small preview with an optional title during debugging.
```

```r
print_head <- function(df, n = 10, title = NULL) {
  if (!is.null(title)) message(title)
  if (is.null(df) || nrow(df) == 0L) {
    message("No rows to display.")
    return(invisible(FALSE))
  }
  print(utils::head(df, n))
  invisible(TRUE)
}

# helper: assert that an object is data-frame-like before downstream code tries
# to treat it as tabular data.
assert_is_df <- function(x, context = "") {
  if (is.null(x)) {
    stop(context, ": object is NULL")
  }
  if (!inherits(x, "data.frame")) {
    stop(context, ": expected data.frame/tibble; got class=",
         paste(class(x), collapse = ", "), " typeof=", typeof(x))
  }
  invisible(TRUE)
}

# helper: stop with a readable message when required columns are missing.
assert_has_cols <- function(df, cols, context = "") {
  missing <- setdiff(cols, names(df))
  if (length(missing)) {
    stop(context, ": missing required columns: ", paste(missing, collapse = ", "))
  }
  invisible(TRUE)
}

# helper: obtain `nrow()` only after validating that the input is truly tabular.
safe_nrow <- function(df, context = "") {
  assert_is_df(df, context)
  nr <- nrow(df)
```

```r
  if (!is.numeric(nr) || length(nr) != 1L || is.na(nr)) {
    stop(context, ": nrow(df) returned invalid value: ",
         paste0(capture.output(str(nr)), collapse = " "))
  }
  as.integer(nr)
}

# helper: expose the cumulative plot-drop log for debugging and export.
plot_drop_log <- function() {
  plot_drop_log_df
}

# helper: record rows dropped during figure preparation so suspicious plotting
# losses are always audit-able after a render.
log_plot_drop <- function(plot_name, reason, n_dropped,
                          n_before = NA_integer_, n_after = NA_integer_,
                          stage = NA_character_, extra = NA_character_) {
  plot_drop_log_df <<- dplyr::bind_rows(
    plot_drop_log(),
    data.frame(plot = plot_name, reason = reason, n_dropped = n_dropped,
               n_before = n_before, n_after = n_after,
               stage = stage, extra = extra,
               stringsAsFactors = FALSE)
  )
  if (identical(PLOT_DROP_POLICY, "stop")) {
    stop("Plot data dropped in ", plot_name, ": ", reason)
  } else {
    warning("Plot data dropped in ", plot_name, ": ", reason, call. = FALSE)
  }
}

# helper: wrap model fitting so warnings, convergence flags, and simple
# separation diagnostics are captured in one standardized object.
fit_with_diagnostics <- function(fit_fun, context, prob_eps = PROB_EPS) {
  stopifnot(all(c("stage", "component", "analysis_variant", "model_type",
                  "group", "outcome", "imputation", "batch") %in% names(context)))
```

```r
cap <- capture_warnings(
  tryCatch(fit_fun(), error = function(e) e),
  context = context
)
append_warnings(cap$warnings)
fit <- cap$value
warn_msgs <- if (is.data.frame(cap$warnings)) cap$warnings$message else character()
warning_n <- if (is.data.frame(cap$warnings)) nrow(cap$warnings) else 0L
top_warning <- if (warning_n) warn_msgs[1] else NA_character_

if (inherits(fit, "error")) {
  diag <- data.frame(
    stage = as.character(context$stage),
    component = as.character(context$component),
    analysis_variant = as.character(context$analysis_variant),
    model_type = as.character(context$model_type),
    group = as.character(context$group),
    outcome = as.character(context$outcome),
    imputation = as.integer(context$imputation),
    n_used = NA_integer_, events = NA_integer_,
    converged = NA, iter = NA_integer_,
    sep_flag = NA, nonconv_flag = NA,
    min_phat = NA_real_, max_phat = NA_real_,
    warning_n = warning_n, top_warning = top_warning,
    error_message = conditionMessage(fit),
    stringsAsFactors = FALSE
  )
  return(list(fit = NULL, diag = diag, warnings = cap$warnings))
}

phat <- tryCatch(fitted(fit), error = function(e) NA_real_)
min_phat <- if (all(is.na(phat))) NA_real_ else min(phat, na.rm = TRUE)
max_phat <- if (all(is.na(phat))) NA_real_ else max(phat, na.rm = TRUE)
sep_flag <- FALSE
if (is.finite(min_phat) && min_phat < prob_eps) sep_flag <- TRUE
if (is.finite(max_phat) && max_phat > 1 - prob_eps) sep_flag <- TRUE
```

```r
  if (any(grepl("fitted probabilities numerically 0 or 1", warn_msgs, fixed = TRUE))) sep_flag <- TRUE

  conv_val <- if (!is.null(fit$converged)) isTRUE(fit$converged) else NA
  iter_val <- if (!is.null(fit$iter)) fit$iter else NA_integer_
  nonconv_flag <- isFALSE(conv_val) || any(grepl("did not converge", warn_msgs, fixed = TRUE))

  mf <- tryCatch(model.frame(fit), error = function(e) NULL)
  y <- if (!is.null(mf)) tryCatch(model.response(mf), error = function(e) NULL) else NULL
  n_used <- if (!is.null(y)) length(y) else NA_integer_
  events <- if (!is.null(y) && is.numeric(y)) sum(y == 1, na.rm = TRUE) else NA_integer_

  diag <- data.frame(
    stage = as.character(context$stage),
    component = as.character(context$component),
    analysis_variant = as.character(context$analysis_variant),
    model_type = as.character(context$model_type),
    group = as.character(context$group),
    outcome = as.character(context$outcome),
    imputation = as.integer(context$imputation),
    n_used = n_used, events = events,
    converged = conv_val, iter = iter_val,
    sep_flag = sep_flag, nonconv_flag = nonconv_flag,
    min_phat = min_phat, max_phat = max_phat,
    warning_n = warning_n, top_warning = top_warning,
    error_message = NA_character_,
    stringsAsFactors = FALSE
  )
  list(fit = fit, diag = diag, warnings = cap$warnings)
}

# helper: confirm that a pooled MI variance object is finite before it is used
# to generate manuscript-facing confidence intervals.
pooled_mi_vcov_check <- function(pooled) {
  if (is.null(pooled$variance)) stop("MIcombine object missing variance matrix.")
  V <- pooled$variance
  if (any(!is.finite(V))) stop("Non-finite pooled variance detected.")
```

```r
  within <- attr(pooled, "within")
  between <- attr(pooled, "between")
  if (!is.null(within) && any(!is.finite(within))) stop("Non-finite within variance.")
  if (!is.null(between) && any(!is.finite(between))) stop("Non-finite between variance.")
  invisible(TRUE)
}

# helper: one project-wide publication theme so manuscript and supplement
# figures inherit the same typography and guide styling.
theme_pub <- function(base_size = 10, base_family = "", legend_position = "top") {
  ggplot2::theme_minimal(base_size = base_size, base_family = base_family) +
    ggplot2::theme(
      plot.title = ggplot2::element_text(face = "bold", hjust = 0, size = ggplot2::rel(1.05)),
      plot.subtitle = ggplot2::element_text(color = "grey20"),
      plot.caption = ggplot2::element_text(size = ggplot2::rel(0.85), color = "grey25", hjust = 0),
      axis.title = ggplot2::element_text(color = "grey10"),
      axis.text = ggplot2::element_text(color = "grey10"),
      panel.grid.minor = ggplot2::element_blank(),
      panel.grid.major.x = ggplot2::element_blank(),
      panel.grid.major.y = ggplot2::element_line(color = "grey85", linewidth = 0.25),
      axis.line = ggplot2::element_line(color = "grey25", linewidth = 0.25),
      axis.ticks = ggplot2::element_line(color = "grey25", linewidth = 0.25),
      legend.position = legend_position,
      legend.title = ggplot2::element_text(face = "bold"),
      legend.background = ggplot2::element_blank(),
      legend.key = ggplot2::element_blank(),
      strip.background = ggplot2::element_rect(fill = "grey95", color = "grey80", linewidth = 0.25),
      strip.text = ggplot2::element_text(face = "bold")
    )
}

# helper: save a diagnostic plot with explicit dimensions and a vector PDF path
# when appropriate, avoiding IDE-dependent defaults.
save_diag_plot <- function(p, file, width = 8, height = 6, units = "in", dpi = 300, bg = "white") {
  if (grepl("\\.pdf$", file, ignore.case = TRUE)) {
    ggplot2::ggsave(
```

```
      filename = file,
      plot = p,
      width = width,
      height = height,
      units = units,
      device = grDevices::cairo_pdf,
      bg = bg
    )
    return(invisible(file))
  }
  ggplot2::ggsave(
    filename = file,
    plot = p,
    width = width,
    height = height,
    units = units,
    dpi = dpi,
    bg = bg
  )
  invisible(file)
}

plot_registry_path <- results_path("plot_registry.csv")

# helper: load the registry of previously written plots for this render family.
read_plot_registry <- function() {
  if (file.exists(plot_registry_path)) {
    utils::read.csv(plot_registry_path, stringsAsFactors = FALSE)
  } else {
    data.frame(
      run_id = character(), run_ts = character(),
      plot_name = character(), fig_path = character(), md5 = character(),
      stringsAsFactors = FALSE
    )
  }
}
```

```r
# helper: persist the plot registry after each update.
write_plot_registry <- function(df) {
  write_csv_safely(df, plot_registry_path, row_names = FALSE)
}

# helper: add one saved plot file to the registry, or log a duplicate if a
# chunk tries to emit the same named figure twice.
register_plot_file <- function(plot_name, file, run_id = diag_run_id, run_ts = diag_run_ts) {
  md5 <- as.character(tools::md5sum(file))
  reg <- read_plot_registry()
  dup_name <- nrow(reg) > 0 &&
    any(reg$run_id == run_id & reg$plot_name == plot_name, na.rm = TRUE)
  if (dup_name) {
    log_plot_drop(
      plot_name,
      "duplicate_figure",
      n_dropped = 1,
      n_before = NA_integer_,
      n_after = NA_integer_,
      stage = "plot_registry",
      extra = file
    )
    return(FALSE)
  }
  reg <- dplyr::bind_rows(
    reg,
    data.frame(run_id = run_id, run_ts = run_ts,
               plot_name = plot_name, fig_path = file, md5 = md5,
               stringsAsFactors = FALSE)
  )
  write_plot_registry(reg)
  TRUE
}

# helper: save a figure to disk and include it in the notebook only if that
```

```r
# named figure has not already been emitted in the current render.
print_plot_once <- function(p, plot_name, width = 8, height = 6, dpi = 300) {
  file_png <- results_path("figs", paste0(plot_name, ".png"))
  file_pdf <- results_path("figs", paste0(plot_name, ".pdf"))
  file_preview <- results_path("figs", paste0(plot_name, "-report-preview.png"))
  save_diag_plot(p, file_png, width = width, height = height, dpi = dpi)
  save_diag_plot(p, file_pdf, width = width, height = height, dpi = dpi)
  # Use a lighter inline preview while keeping manuscript artifacts high resolution.
  save_diag_plot(p, file_preview, width = width, height = height, dpi = min(dpi, 96))
  if (register_plot_file(plot_name, file_png)) {
    if (isTRUE(VALIDATION_INLINE_ANALYSIS_PREVIEWS)) {
      mark_float_emitted()
      render_inline_graphic(file_preview)
    } else {
      invisible(file_preview)
    }
  } else {
    invisible(NULL)
  }
}

# helper: build the canonical on-disk path for any artifact stub/ext pair.
artifact_file_path <- function(stub, ext, subdir = NULL) {
  if (is.null(stub) || is.na(stub) || !nzchar(stub)) return(NA_character_)
  if (is.null(subdir) || is.na(subdir) || !nzchar(subdir)) {
    return(results_path(paste0(stub, ".", ext)))
  }
  results_path(subdir, paste0(stub, ".", ext))
}

# helper: copy one canonical artifact to one or more manuscript-facing alias
# names without changing the original analysis-oriented filename.
write_artifact_alias <- function(source_stub, alias_stub, exts,
                                 source_subdir = NULL,
                                 alias_subdir = source_subdir) {
  copied <- character(0)
```

```r
  missing <- character(0)

  for (ext in exts) {
    src <- artifact_file_path(source_stub, ext, source_subdir)
    dst <- artifact_file_path(alias_stub, ext, alias_subdir)
    if (!file.exists(src)) {
      missing <- c(missing, src)
      next
    }
    if (fs::path_norm(src) != fs::path_norm(dst)) {
      fs::file_copy(src, dst, overwrite = TRUE)
    }
    copied <- c(copied, dst)
  }

  if (length(missing)) {
    stop(
      "Missing source artifact(s) for alias [",
      alias_stub,
      "]: ",
      paste(vapply(missing, pretty_path, character(1L)), collapse = ", ")
    )
  }
  invisible(copied)
}

# helper: mirror one saved figure into its manuscript alias path.
write_plot_alias <- function(plot_name, alias_name,
                             source_subdir = "figs",
                             alias_subdir = source_subdir) {
  write_artifact_alias(
    source_stub = plot_name,
    alias_stub = alias_name,
    exts = c("png", "pdf"),
    source_subdir = source_subdir,
    alias_subdir = alias_subdir
```

```r
  )
}

# helper: mirror one saved table export into its manuscript alias path.
write_table_alias <- function(file_stub, alias_stub,
                              source_subdir = NULL,
                              alias_subdir = source_subdir) {
  write_artifact_alias(
    source_stub = file_stub,
    alias_stub = alias_stub,
    exts = c("csv", "pdf"),
    source_subdir = source_subdir,
    alias_subdir = alias_subdir
  )
}

# helper: force a figure to appear in the notebook even if a previous chunk
# already emitted the same named artifact.
print_plot_force <- function(p, plot_name, width = 8, height = 6, dpi = 300) {
  # Use this for key publication figures that must always appear in the PDF.
  file_png <- results_path("figs", paste0(plot_name, ".png"))
  file_pdf <- results_path("figs", paste0(plot_name, ".pdf"))
  file_preview <- results_path("figs", paste0(plot_name, "-report-preview.png"))
  save_diag_plot(p, file_png, width = width, height = height, dpi = dpi)
  save_diag_plot(p, file_pdf, width = width, height = height, dpi = dpi)
  save_diag_plot(p, file_preview, width = width, height = height, dpi = min(dpi, 96))
  register_plot_file(plot_name, file_png)
  if (isTRUE(VALIDATION_INLINE_ANALYSIS_PREVIEWS)) {
    mark_float_emitted()
    render_inline_graphic(file_preview)
  } else {
    invisible(file_preview)
  }
}

# helper: map a model-matrix term back to its parent feature so SHAP fallbacks
```

```r
# and coefficient-based summaries stay interpretable.
term_to_parent_feature <- function(term, feature_names) {
  if (is.na(term) || identical(term, "(Intercept)")) return(NA_character_)
  clean_term <- gsub("`", "", term, fixed = TRUE)
  feature_names <- sort(unique(feature_names), decreasing = TRUE)
  hit <- feature_names[vapply(feature_names, function(v) grepl(v, clean_term, fixed = TRUE), logical(1))]
  if (length(hit)) return(hit[[1]])
  NA_character_
}

# helper: subsample rows reproducibly before SHAP-style calculations so
# diagnostics remain stable while keeping runtime bounded.
sample_rows_for_shap <- function(df, n_max = SHAP_SAMPLE_N, seed = 20251206L) {
  stopifnot(is.data.frame(df) || is.matrix(df))
  if (nrow(df) <= n_max) return(df)
  set.seed(seed)
  df[sample.int(nrow(df), size = n_max, replace = FALSE), , drop = FALSE]
}

# helper: refit the GBM weighting model with the exact options needed for the
# downstream feature-contribution diagnostics.
fit_weightit_gbm_for_shap <- function(formula_obj, data_obj) {
  args <- c(
    list(
      formula = formula_obj,
      data = data_obj,
      method = "gbm",
      estimand = "ATE",
      missing = "ind",
      include.obj = TRUE
    ),
    gbm_params
  )
  do.call(WeightIt::weightit, args)
}
```

```r
# helper: compute the top GBM contribution features for one non-MI weighting
# model, with a deterministic fallback to relative influence if `fastshap` fails.
extract_nonmi_gbm_shap_top <- function(weight_obj, x_df, feature_names, group_label,
                                       nsim = SHAP_NSIM, top_n = SHAP_TOP_N) {
  if (!HAS_FASTSHAP) {
    return(
      data.frame(
        group = character(),
        feature = character(),
        mean_abs_shap = numeric(),
        stringsAsFactors = FALSE
      )
    )
  }
  stopifnot(inherits(weight_obj, "weightit"))
  stopifnot(!is.null(weight_obj$obj))
  gbm_obj <- weight_obj$obj
  best_tree <- weight_obj$info$best.tree
  if (is.null(best_tree) || !is.finite(best_tree) || best_tree <= 0) {
    best_tree <- gbm_obj$n.trees
  }
  x_use <- x_df[, feature_names, drop = FALSE]
  char_cols <- names(x_use)[vapply(x_use, is.character, logical(1))]
  if (length(char_cols)) {
    for (nm in char_cols) x_use[[nm]] <- factor(x_use[[nm]])
  }
  x_use <- droplevels_all(x_use)
  x_background <- as.data.frame(
    sample_rows_for_shap(x_use, n_max = max(SHAP_SAMPLE_N, 5000L), seed = 20251206L)
  )
  x_shap <- as.data.frame(
    sample_rows_for_shap(x_use, n_max = SHAP_SAMPLE_N, seed = 20251206L)
  )

  pred_fun <- function(object, newdata) {
    new_use <- newdata[, object$var.names, drop = FALSE]
```

```r
  as.numeric(stats::predict(object, newdata = new_use, n.trees = best_tree, type = "link"))
}

shap_mat <- tryCatch(
  fastshap::explain(
    object = gbm_obj,
    feature_names = feature_names,
    X = x_background,
    newdata = x_shap,
    pred_wrapper = pred_fun,
    nsim = nsim,
    adjust = FALSE
  ),
  error = function(e) {
    warning(
      "fastshap failed for ", group_label,
      "; falling back to GBM relative influence. ",
      conditionMessage(e),
      call. = FALSE
    )
    NULL
  }
)

if (is.null(shap_mat)) {
  gbm_imp <- tryCatch(
    gbm::summary.gbm(gbm_obj, plotit = FALSE),
    error = function(e) data.frame(var = character(), rel.inf = numeric(), stringsAsFactors = FALSE)
  )
  if (!nrow(gbm_imp)) {
    return(data.frame(group = character(), feature = character(), mean_abs_shap = numeric(), stringsAsFactors =
     ↪  FALSE))
  }
  return(
    gbm_imp |>
      dplyr::transmute(
```

```r
          group = group_label,
          feature = as.character(var),
          mean_abs_shap = as.numeric(rel.inf) / 100
        ) |>
        dplyr::arrange(dplyr::desc(mean_abs_shap)) |>
        dplyr::slice_head(n = top_n)
    )
  }

  out <- data.frame(
    group = group_label,
    feature = colnames(shap_mat),
    mean_abs_shap = colMeans(abs(shap_mat), na.rm = TRUE),
    stringsAsFactors = FALSE
  ) |>
    dplyr::arrange(dplyr::desc(mean_abs_shap)) |>
    dplyr::slice_head(n = top_n)
  out
}

# helper: plot the top contribution features side by side for ABG and VBG so
# readers can compare the ordering models directly.
plot_shap_top10_two_panel <- function(df_top, title_text, x_label) {
  # Keep only top-N per cohort and force descending order inside each panel.
  df_top_plot <- df_top |>
    dplyr::group_by(group) |>
    dplyr::arrange(dplyr::desc(mean_abs_shap), .by_group = TRUE) |>
    dplyr::slice_head(n = SHAP_TOP_N) |>
    dplyr::mutate(
      feature_rank = dplyr::row_number(),
      feature_plot = paste0(sprintf("%02d", feature_rank), ". ", feature),
      group = factor(group, levels = c("ABG", "VBG"))
    ) |>
    dplyr::ungroup()

  # Prefixing with rank makes ordering deterministic in each facet.
```

```r
  ggplot2::ggplot(
    df_top_plot,
    ggplot2::aes(
      x = mean_abs_shap,
      y = stats::reorder(feature_plot, mean_abs_shap),
      fill = group
    )
  ) +
    ggplot2::geom_col(show.legend = FALSE) +
    ggplot2::facet_wrap(~group, scales = "free_y", ncol = 2) +
    ggplot2::labs(
      title = title_text,
      x = x_label,
      y = NULL
    ) +
    ggplot2::scale_y_discrete(labels = function(x) sub("^[0-9]+\\.\\s*", "", x)) +
    theme_pub(base_size = 10)
}

# helper: prepend a whole-cohort balance summary row so the love plot reports
# both term-level and overall balance shifts.
add_overall_love_rows <- function(df_in, overall_label = "Overall (mean |SMD|)") {
  # Summarize whole-plot balance for each cohort/sample pair and display it
  # as the top row so readers can quickly judge global balance shift.
  overall <- df_in |>
    dplyr::group_by(group, sample) |>
    dplyr::summarise(
      abs_smd = mean(abs_smd, na.rm = TRUE),
      abs_smd_raw = mean(abs_smd_raw, na.rm = TRUE),
      .groups = "drop"
    ) |>
    dplyr::mutate(term = overall_label)
  dplyr::bind_rows(overall, df_in)
}

# helper: apply one consistent love-plot layout across weighting diagnostics.
```

```r
loveplot_style <- function(df_in, title_text) {
  # Build a classic love-plot view with:
  # 1) a segment from Raw -> IPW for each term, and
  # 2) explicit Raw/IPW points so both samples are always visible.
  # This avoids overplotting where one sample can hide the other.
  df_plot <- df_in |>
    dplyr::mutate(
      sample = factor(sample, levels = c("Raw", "IPW")),
      order_val = dplyr::if_else(
        term == "Overall (mean |SMD|)",
        max(abs_smd_raw, na.rm = TRUE) + 1,
        abs_smd_raw
      )
    )

  df_wide <- df_plot |>
    dplyr::select(group, term, order_val, sample, abs_smd) |>
    tidyr::pivot_wider(names_from = sample, values_from = abs_smd)

  ggplot2::ggplot(df_plot, ggplot2::aes(y = stats::reorder(term, order_val))) +
    ggplot2::geom_segment(
      data = df_wide,
      ggplot2::aes(
        x = Raw,
        xend = IPW,
        y = stats::reorder(term, order_val),
        yend = stats::reorder(term, order_val)
      ),
      inherit.aes = FALSE,
      color = "grey65",
      linewidth = 0.3,
      alpha = 0.8
    ) +
    ggplot2::geom_point(
      data = df_plot |> dplyr::filter(sample == "Raw"),
      ggplot2::aes(x = abs_smd, color = sample, shape = sample, size = term == "Overall (mean |SMD|)"),
```

```r
      alpha = 0.95
    ) +
    ggplot2::geom_point(
      data = df_plot |> dplyr::filter(sample == "IPW"),
      ggplot2::aes(x = abs_smd, color = sample, shape = sample, size = term == "Overall (mean |SMD|)"),
      alpha = 0.95
    ) +
    ggplot2::geom_vline(xintercept = 0.10, linetype = "dashed", color = "grey30") +
    ggplot2::facet_wrap(~group, ncol = 2) +
    ggplot2::scale_color_manual(values = c("Raw" = "#F8766D", "IPW" = "#00BFC4")) +
    ggplot2::scale_shape_manual(values = c("Raw" = 16, "IPW" = 17), guide = "none") +
    ggplot2::scale_size_manual(values = c(`TRUE` = 3.0, `FALSE` = 1.9), guide = "none") +
    ggplot2::labs(
      title = title_text,
      x = "Absolute Standardized Mean Differences",
      y = NULL,
      color = "Sample"
    ) +
    theme_pub(base_size = 10) +
    ggplot2::theme(legend.position = "top")
}

CO2_RATE_OUTCOMES <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
CO2_RATE_OUTCOME_LABELS <- c(
  imv_proc = "IMV",
  niv_proc = "NIV",
  death_60d = "Death (60d)",
  hypercap_resp_failure = "Hypercapnic RF"
)

# helper: bin CO2 values and compute observed outcome rates for exploratory
# shape checks before the fully modeled spline figures are built.
compute_co2_bin_rates <- function(df, co2_var, outcomes, outcome_labels,
                                  weight_var = NULL, q = c(0.02, 0.98),
                                  bin_width = 1, cohort_label = NA_character_) {
  assert_is_df(df, context = paste0("compute_co2_bin_rates(", co2_var, ")"))
```

```r
stopifnot(co2_var %in% names(df))
stopifnot(all(outcomes %in% names(df)))
stopifnot(length(q) == 2L, q[1] < q[2], is.finite(bin_width), bin_width > 0)
if (!is.null(weight_var)) stopifnot(weight_var %in% names(df))

co2_vals <- suppressWarnings(as.numeric(df[[co2_var]]))
keep <- is.finite(co2_vals)
if (!any(keep)) {
  return(tibble::tibble(
    cohort = character(), outcome = character(), outcome_label = character(),
    bin_lo = numeric(), bin_mid = numeric(), n_bin = integer(), rate = numeric()
  ))
}

q_lohi <- stats::quantile(co2_vals[keep], probs = q, na.rm = TRUE, names = FALSE)
keep <- keep & co2_vals >= q_lohi[1] & co2_vals <= q_lohi[2]
if (!is.null(weight_var)) {
  w <- suppressWarnings(as.numeric(df[[weight_var]]))
  keep <- keep & is.finite(w) & !is.na(w) & (w > 0)
}
if (!any(keep)) {
  return(tibble::tibble(
    cohort = character(), outcome = character(), outcome_label = character(),
    bin_lo = numeric(), bin_mid = numeric(), n_bin = integer(), rate = numeric()
  ))
}

d <- df[keep, , drop = FALSE]
d$co2_work <- co2_vals[keep]
d$bin_lo <- floor(d$co2_work / bin_width) * bin_width
d$bin_mid <- d$bin_lo + (bin_width / 2)
if (!is.null(weight_var)) {
  d$w_work <- suppressWarnings(as.numeric(d[[weight_var]]))
}

rate_rows <- lapply(outcomes, function(outcome_nm) {
```

```r
  y <- suppressWarnings(as.numeric(d[[outcome_nm]] == 1))
  y[is.na(d[[outcome_nm]])] <- NA_real_
  tmp <- d[, c("bin_lo", "bin_mid"), drop = FALSE]
  tmp$y <- y
  if (!is.null(weight_var)) tmp$w <- d$w_work
  if (!is.null(weight_var)) {
    tmp <- tmp[is.finite(tmp$w) & !is.na(tmp$w) & !is.na(tmp$y), , drop = FALSE]
  } else {
    tmp <- tmp[!is.na(tmp$y), , drop = FALSE]
  }
  if (!nrow(tmp)) return(NULL)

  if (!is.null(weight_var)) {
    out <- tmp |>
      dplyr::group_by(bin_lo, bin_mid) |>
      dplyr::summarise(
        n_bin = dplyr::n(),
        rate = ifelse(sum(w, na.rm = TRUE) > 0, sum(y * w, na.rm = TRUE) / sum(w, na.rm = TRUE), NA_real_),
        .groups = "drop"
      )
  } else {
    out <- tmp |>
      dplyr::group_by(bin_lo, bin_mid) |>
      dplyr::summarise(
        n_bin = dplyr::n(),
        rate = mean(y, na.rm = TRUE),
        .groups = "drop"
      )
  }

  out |>
    dplyr::mutate(
      cohort = cohort_label,
      outcome = outcome_nm,
      outcome_label = dplyr::if_else(
        outcome_nm %in% names(outcome_labels),
```

```
          as.character(outcome_labels[[outcome_nm]]),
          outcome_nm
        )
      ) |>
      dplyr::select(cohort, outcome, outcome_label, bin_lo, bin_mid, n_bin, rate)
  })

  dplyr::bind_rows(rate_rows)
}

# helper: average binned outcome-rate diagnostics across imputations for a
# quick MI-aware visual check of the empirical shape.
pool_mi_bin_rates <- function(rate_df_all_imps) {
  assert_is_df(rate_df_all_imps, context = "pool_mi_bin_rates")
  stopifnot("imp" %in% names(rate_df_all_imps))
  rate_df_all_imps |>
    dplyr::group_by(cohort, outcome, outcome_label, bin_lo, bin_mid) |>
    dplyr::summarise(
      n_bin = mean(n_bin, na.rm = TRUE),
      rate = mean(rate, na.rm = TRUE),
      n_imp_contrib = sum(is.finite(rate)),
      .groups = "drop"
    )
}

# helper: render the exploratory binned-rate panels with shared styling and an
# explicit reference line at the chosen CO2 anchor.
plot_co2_bin_rate_panel <- function(rate_df, title, x_label, vline_at) {
  assert_is_df(rate_df, context = "plot_co2_bin_rate_panel")
  stopifnot(all(c("bin_mid", "n_bin", "rate", "outcome_label") %in% names(rate_df)))
  lbl_levels <- unname(CO2_RATE_OUTCOME_LABELS[CO2_RATE_OUTCOMES])
  rate_df$outcome_label <- factor(rate_df$outcome_label, levels = lbl_levels)

  ggplot2::ggplot(rate_df, ggplot2::aes(x = bin_mid, y = rate)) +
    ggplot2::geom_point(color = "#2C7FB8", alpha = 0.8, size = 1.2) +
    ggplot2::geom_smooth(
```

```r
      ggplot2::aes(weight = n_bin),
      method = "loess", span = 0.5, se = FALSE,
      color = "#D95F0E", linewidth = 0.8
    ) +
    ggplot2::geom_vline(xintercept = vline_at, linetype = "dashed", color = "grey40") +
    ggplot2::facet_wrap(~ outcome_label, ncol = 2, scales = "free_y") +
    ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
    ggplot2::labs(
      title = title,
      x = x_label,
      y = "Observed outcome rate",
      caption = paste0("Points are 1-mmHg bins; LOESS span = 0.5; dashed line at ", vline_at, " mmHg.")
    ) +
    theme_pub(base_size = 10)
}

# helper: derive log-scale odds-ratio axis limits from the observed confidence
# intervals so related figures stay comparable without manual tuning.
compute_or_axis_spec <- function(df_list, lo_col = "conf.low", hi_col = "conf.high",
                                 min_pow = -6, max_pow = 6,
                                 default_limits = c(0.25, 16)) {
  if (inherits(df_list, "data.frame")) df_list <- list(df_list)
  if (!is.list(df_list)) df_list <- list(df_list)
  vals <- unlist(lapply(df_list, function(df) {
    assert_is_df(df, context = "compute_or_axis_spec")
    stopifnot(all(c(lo_col, hi_col) %in% names(df)))
    lo <- df[[lo_col]]
    hi <- df[[hi_col]]
    c(lo, hi)
  }), use.names = FALSE)
  vals <- vals[is.finite(vals) & vals > 0]
  if (!length(vals)) {
    lo_pow <- floor(log2(default_limits[1]))
    hi_pow <- ceiling(log2(default_limits[2]))
  } else {
    lo_pow <- floor(log2(min(vals)))
```

```r
    hi_pow <- ceiling(log2(max(vals)))
  }
  lo_pow <- max(lo_pow, min_pow)
  hi_pow <- min(hi_pow, max_pow)
  if (lo_pow > hi_pow) hi_pow <- lo_pow
  limits <- 2^c(lo_pow, hi_pow)
  breaks <- 2^(lo_pow:hi_pow)
  list(limits = limits, breaks = breaks)
}

# helper: apply a consistent odds-ratio y-axis specification to spline plots.
or_axis_scale <- function(spec) {
  list(
    ggplot2::scale_y_log10(
      breaks = spec$breaks,
      labels = scales::number_format(accuracy = 0.01)
    ),
    ggplot2::coord_cartesian(ylim = spec$limits)
  )
}

# helper: derive probability-axis limits from the observed confidence bands so
# all probability panels share a stable visual range.
compute_prob_axis_spec <- function(df_list, lo_col = "p_LCL", hi_col = "p_UCL",
                                   floor = 0, ceiling = 1, pad = 0.02) {
  if (inherits(df_list, "data.frame")) df_list <- list(df_list)
  if (!is.list(df_list)) df_list <- list(df_list)
  vals <- unlist(lapply(df_list, function(df) {
    assert_is_df(df, context = "compute_prob_axis_spec")
    stopifnot(all(c(lo_col, hi_col) %in% names(df)))
    c(df[[lo_col]], df[[hi_col]])
  }), use.names = FALSE)
  vals <- vals[is.finite(vals)]
  if (!length(vals)) return(list(limits = c(floor, ceiling)))
  lo <- max(floor, min(vals) - pad)
  hi <- min(ceiling, max(vals) + pad)
```

```r
  if (lo >= hi) {
    lo <- floor
    hi <- ceiling
  }
  list(limits = c(lo, hi))
}

# helper: apply a consistent probability y-axis specification to spline plots.
prob_axis_scale <- function(spec, accuracy = 1) {
  list(
    ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = accuracy)),
    ggplot2::coord_cartesian(ylim = spec$limits)
  )
}

KEY_RESULTS_OUTCOME_LEVELS <- c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")
KEY_RESULTS_OUTCOME_MAP <- c(
  imv_proc = "IMV",
  niv_proc = "NIV",
  death_60d = "Death (60d)",
  hypercap_resp_failure = "Hypercapnic RF",
  IMV = "IMV",
  NIV = "NIV",
  Death60d = "Death (60d)",
  HCRF = "Hypercapnic RF",
  "Death (60d)" = "Death (60d)",
  "Hypercapnic RF" = "Hypercapnic RF"
)
KEY_RESULTS_GROUP_LEVELS <- c("ABG", "VBG")
KEY_RESULTS_GROUP_COLORS <- c("ABG" = "#F8766D", "VBG" = "#00BFC4")
KEY_RESULTS_GROUP_LINETYPES <- c("ABG" = "solid", "VBG" = "longdash")
KEY_RESULTS_FIG_WIDTH <- 10.5
KEY_RESULTS_FIG_HEIGHT <- 11

# helper: map raw outcome names from multiple upstream pipelines onto one
# ordered set of manuscript-facing labels for the spline key-results figures.
```

```r
normalize_key_results_outcome <- function(x, context = "key-results") {
  out_chr <- as.character(x)
  mapped <- dplyr::recode(out_chr, !!!KEY_RESULTS_OUTCOME_MAP, .default = NA_character_)
  bad <- is.na(mapped)
  if (any(bad)) {
    stop(
      context, ": unmapped outcome labels: ",
      paste(sort(unique(out_chr[bad])), collapse = ", ")
    )
  }
  factor(mapped, levels = KEY_RESULTS_OUTCOME_LEVELS)
}

# helper: coerce cohort labels onto one stable ABG/VBG factor order.
normalize_key_results_group <- function(x, context = "key-results") {
  grp <- factor(as.character(x), levels = KEY_RESULTS_GROUP_LEVELS)
  if (any(is.na(grp))) {
    stop(
      context, ": unmapped group labels: ",
      paste(sort(unique(as.character(x)[is.na(grp)])), collapse = ", ")
    )
  }
  grp
}

# helper: standardize spline odds-ratio curve objects onto one common schema
# before they are combined into manuscript-facing figures.
normalize_key_results_or_df <- function(df, group = NULL, co2_col = "co2",
                                        outcome_col = "outcome",
                                        or_col = "OR", lcl_col = "LCL", ucl_col = "UCL",
                                        context = "key-results OR") {
  assert_is_df(df, context = context)
  stopifnot(outcome_col %in% names(df), or_col %in% names(df),
            lcl_col %in% names(df), ucl_col %in% names(df))
  if (!co2_col %in% names(df)) stop(context, ": missing CO2 column ", co2_col)
  group_vec <- if ("group" %in% names(df)) df$group else rep(group, nrow(df))
```

```r
  if (is.null(group_vec)) stop(context, ": group must be supplied or present in data.")

  out <- tibble::tibble(
    group = normalize_key_results_group(group_vec, context = context),
    outcome = normalize_key_results_outcome(df[[outcome_col]], context = context),
    co2 = suppressWarnings(as.numeric(df[[co2_col]])),
    OR = suppressWarnings(as.numeric(df[[or_col]])),
    LCL = suppressWarnings(as.numeric(df[[lcl_col]])),
    UCL = suppressWarnings(as.numeric(df[[ucl_col]]))
  ) |>
    dplyr::filter(is.finite(co2), is.finite(OR), is.finite(LCL), is.finite(UCL),
                  OR > 0, LCL > 0, UCL > 0)

  if (!nrow(out)) stop(context, ": no finite OR rows after filtering.")
  out
}

# helper: standardize spline probability curves onto one common schema before
# the ABG/VBG overlays are assembled.
normalize_key_results_prob_df <- function(df, group = NULL, co2_col = "co2",
                                          outcome_col = "outcome",
                                          p_col = "p", lcl_col = "p_LCL", ucl_col = "p_UCL",
                                          context = "key-results probability") {
  assert_is_df(df, context = context)
  stopifnot(outcome_col %in% names(df), p_col %in% names(df),
            lcl_col %in% names(df), ucl_col %in% names(df))
  if (!co2_col %in% names(df)) stop(context, ": missing CO2 column ", co2_col)
  group_vec <- if ("group" %in% names(df)) df$group else rep(group, nrow(df))
  if (is.null(group_vec)) stop(context, ": group must be supplied or present in data.")

  out <- tibble::tibble(
    group = normalize_key_results_group(group_vec, context = context),
    outcome = normalize_key_results_outcome(df[[outcome_col]], context = context),
    co2 = suppressWarnings(as.numeric(df[[co2_col]])),
    p = suppressWarnings(as.numeric(df[[p_col]])),
    p_LCL = suppressWarnings(as.numeric(df[[lcl_col]])),
```

```
    p_UCL = suppressWarnings(as.numeric(df[[ucl_col]]))
  ) |>
    dplyr::filter(
      is.finite(co2), is.finite(p), is.finite(p_LCL), is.finite(p_UCL),
      dplyr::between(p, 0, 1), dplyr::between(p_LCL, 0, 1), dplyr::between(p_UCL, 0, 1)
    )

  if (!nrow(out)) stop(context, ": no finite probability rows after filtering.")
  out
}

# helper: stack one named list of odds-ratio curves into a single tidy frame
# keyed by outcome for the combined spline figure.
bind_key_results_or_curves <- function(curve_list, group, co2_col, context = "key-results OR") {
  stopifnot(is.list(curve_list), length(curve_list) > 0L, !is.null(names(curve_list)))
  dplyr::bind_rows(lapply(names(curve_list), function(outcome_nm) {
    curve_df <- curve_list[[outcome_nm]]
    curve_df$outcome <- outcome_nm
    normalize_key_results_or_df(
      curve_df,
      group = group,
      co2_col = co2_col,
      context = context
    )
  }))
}

# helper: stack one named list of predicted-probability curves into a single
# tidy frame keyed by outcome for the combined spline figure.
bind_key_results_prob_curves <- function(curve_list, group, co2_col,
                                         context = "key-results probability") {
  stopifnot(is.list(curve_list), length(curve_list) > 0L, !is.null(names(curve_list)))
  dplyr::bind_rows(lapply(names(curve_list), function(outcome_nm) {
    curve_df <- curve_list[[outcome_nm]]
    curve_df$outcome <- outcome_nm
    normalize_key_results_prob_df(
```

```r
      curve_df,
      group = group,
      co2_col = co2_col,
      context = context
    )
  }))
}

# helper: build the odds-ratio half of the 4x2 spline key-results figure.
build_key_results_or_column <- function(or_df, column_title = "Odds ratio",
                                        x_label = "CO2 (mmHg)",
                                        show_row_labels = TRUE) {
  axis_spec <- compute_or_axis_spec(list(or_df), lo_col = "LCL", hi_col = "UCL")
  ggplot2::ggplot(
    or_df,
    ggplot2::aes(x = co2, y = OR, color = group, fill = group, linetype = group)
  ) +
    ggplot2::geom_ribbon(
      ggplot2::aes(ymin = LCL, ymax = UCL),
      alpha = 0.15,
      color = NA,
      show.legend = FALSE
    ) +
    ggplot2::geom_line(linewidth = 0.6) +
    ggplot2::geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
    ggplot2::facet_grid(outcome ~ ., scales = "free_x", switch = "y") +
    ggplot2::scale_color_manual(values = KEY_RESULTS_GROUP_COLORS) +
    ggplot2::scale_fill_manual(values = KEY_RESULTS_GROUP_COLORS) +
    ggplot2::scale_linetype_manual(values = KEY_RESULTS_GROUP_LINETYPES) +
    or_axis_scale(axis_spec) +
    ggplot2::labs(
      title = column_title,
      x = x_label,
      y = "Odds ratio (log scale)",
      color = NULL,
      linetype = NULL
```

```r
    ) +
    theme_pub(base_size = 10) +
    ggplot2::theme(
      legend.position = "top",
      plot.title = ggplot2::element_text(hjust = 0.5, face = "bold"),
      strip.placement = "outside",
      strip.background = ggplot2::element_rect(fill = "grey95", color = NA),
      strip.text.y.left = if (show_row_labels) {
        ggplot2::element_text(angle = 0, face = "bold")
      } else {
        ggplot2::element_blank()
      },
      strip.text.y = ggplot2::element_blank()
    )
}

# helper: build the predicted-probability half of the 4x2 spline key-results
# figure using the same grouping and facet structure as the OR column.
build_key_results_prob_column <- function(prob_df, column_title = "Predicted probability",
                                          x_label = "CO2 (mmHg)",
                                          show_row_labels = FALSE) {
  axis_spec <- compute_prob_axis_spec(list(prob_df), lo_col = "p_LCL", hi_col = "p_UCL")
  ggplot2::ggplot(
    prob_df,
    ggplot2::aes(x = co2, y = p, color = group, fill = group, linetype = group)
  ) +
    ggplot2::geom_ribbon(
      ggplot2::aes(ymin = p_LCL, ymax = p_UCL),
      alpha = 0.15,
      color = NA,
      show.legend = FALSE
    ) +
    ggplot2::geom_line(linewidth = 0.6) +
    ggplot2::facet_grid(outcome ~ ., scales = "free_x", switch = "y") +
    ggplot2::scale_color_manual(values = KEY_RESULTS_GROUP_COLORS) +
    ggplot2::scale_fill_manual(values = KEY_RESULTS_GROUP_COLORS) +
```

```r
    ggplot2::scale_linetype_manual(values = KEY_RESULTS_GROUP_LINETYPES) +
    prob_axis_scale(axis_spec) +
    ggplot2::labs(
      title = column_title,
      x = x_label,
      y = "Predicted probability",
      color = NULL,
      linetype = NULL
    ) +
    theme_pub(base_size = 10) +
    ggplot2::theme(
      legend.position = "top",
      plot.title = ggplot2::element_text(hjust = 0.5, face = "bold"),
      strip.placement = "outside",
      strip.background = ggplot2::element_blank(),
      strip.text.y.left = if (show_row_labels) {
        ggplot2::element_text(angle = 0, face = "bold")
      } else {
        ggplot2::element_blank()
      },
      strip.text.y = ggplot2::element_blank()
    )
}

# helper: assemble the full spline key-results figure with shared legend and
# caption once both columns have been built.
build_key_results_figure <- function(or_df, prob_df, figure_title, figure_caption) {
  or_plot <- build_key_results_or_column(or_df, show_row_labels = TRUE)
  prob_plot <- build_key_results_prob_column(prob_df, show_row_labels = FALSE)
  annotation_args <- list(title = figure_title)
  if (!is.null(figure_caption) && nzchar(figure_caption)) {
    annotation_args$caption <- figure_caption
  }
  (or_plot | prob_plot) +
    patchwork::plot_layout(guides = "collect", widths = c(1, 1)) +
    do.call(
```

```r
      patchwork::plot_annotation,
      c(
        annotation_args,
        list(theme = ggplot2::theme(
          plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),
          plot.caption = ggplot2::element_text(hjust = 0)
        ))
      )
    ) &
    ggplot2::theme(legend.position = "top")
}

# helper: apply the shared odds-ratio axis specification to horizontal forest
# plots rather than vertical spline plots.
or_axis_scale_x <- function(spec) {
  list(
    ggplot2::scale_x_log10(
      breaks = spec$breaks,
      labels = scales::number_format(accuracy = 0.01)
    ),
    ggplot2::coord_cartesian(xlim = spec$limits)
  )
}

# helper: apply the shared probability axis specification to horizontal forest
# plots rather than vertical spline plots.
prob_axis_scale_x <- function(spec, accuracy = 1) {
  list(
    ggplot2::scale_x_continuous(labels = scales::percent_format(accuracy = accuracy)),
    ggplot2::coord_cartesian(xlim = spec$limits)
  )
}

CAT3_KEY_RESULTS_METHOD_LEVELS <- c(
  "Crude",
  "Adjusted",
```

```
  "Imputed",
  "Weighted and adjusted",
  "GBM-weighted"
)
CAT3_KEY_RESULTS_CONTRAST_LEVELS <- c("Low vs normal", "High vs normal")
CAT3_KEY_RESULTS_CATEGORY_LEVELS <- c("Low", "Normal", "High")
CAT3_KEY_RESULTS_GROUP_SHAPES <- c("ABG" = 16, "VBG" = 17)
CAT3_KEY_RESULTS_FIG_WIDTH <- 10.5
CAT3_KEY_RESULTS_FIG_HEIGHT <- 11
CAT3_FOREST_DODGE_OFFSET <- 0.16

# helper: coerce analysis-track labels onto one stable order for the
# categorical key-results tables and figures.
normalize_cat3_key_results_method <- function(x, context = "categorical key-results") {
  method <- factor(as.character(x), levels = CAT3_KEY_RESULTS_METHOD_LEVELS)
  if (any(is.na(method))) {
    stop(
      context, ": unmapped methods: ",
      paste(sort(unique(as.character(x)[is.na(method)])), collapse = ", ")
    )
  }
  method
}

# helper: coerce low/high contrasts onto one stable manuscript order.
normalize_cat3_key_results_contrast <- function(x, context = "categorical key-results") {
  contrast <- factor(as.character(x), levels = CAT3_KEY_RESULTS_CONTRAST_LEVELS)
  if (any(is.na(contrast))) {
    stop(
      context, ": unmapped contrasts: ",
      paste(sort(unique(as.character(x)[is.na(contrast)])), collapse = ", ")
    )
  }
  contrast
}
```

```r
# helper: coerce low/normal/high category labels onto one stable order.
normalize_cat3_key_results_category <- function(x, context = "categorical key-results") {
  category <- factor(as.character(x), levels = CAT3_KEY_RESULTS_CATEGORY_LEVELS)
  if (any(is.na(category))) {
    stop(
      context, ": unmapped categories: ",
      paste(sort(unique(as.character(x)[is.na(category)])), collapse = ", ")
    )
  }
  category
}

# helper: translate raw model-term names into the displayed categorical
# contrasts used in tables and forest plots.
map_cat3_term_contrast <- function(term) {
  term_chr <- gsub("`", "", as.character(term))
  exposure <- dplyr::case_when(
    grepl("^pco2_cat_abg", term_chr) ~ sub("^pco2_cat_abg", "", term_chr),
    grepl("^pco2_cat_vbg", term_chr) ~ sub("^pco2_cat_vbg", "", term_chr),
    grepl("^co2_cat", term_chr) ~ sub("^co2_cat", "", term_chr),
    TRUE ~ term_chr
  )
  dplyr::case_when(
    grepl("Low$", exposure) ~ "Low vs normal",
    grepl("High$", exposure) ~ "High vs normal",
    TRUE ~ NA_character_
  )
}

# helper: reduce a fitted categorical model to the coefficient/vcov pieces
# needed for reusable predicted-probability calculations.
compact_cat3_fit <- function(fit_obj, context = "categorical probability prediction") {
  if (inherits(fit_obj, "glm") || inherits(fit_obj, "svyglm")) {
    return(list(coef = stats::coef(fit_obj), vcov = stats::vcov(fit_obj)))
  }
  if (is.list(fit_obj) && !is.null(fit_obj$coef) && !is.null(fit_obj$vcov)) {
```

```r
    return(fit_obj)
  }
  stop(context, ": unsupported fit object class: ", paste(class(fit_obj), collapse = ", "))
}

# helper: standardize categorical odds-ratio outputs from the different model
# tracks into one shared export/plot schema.
normalize_cat3_key_results_or_df <- function(df, method,
                                             outcome_col = "outcome",
                                             group_col = "group",
                                             term_col = "term",
                                             or_col = "OR",
                                             lcl_col = "LCL",
                                             ucl_col = "UCL",
                                             context = "categorical key-results OR") {
  assert_is_df(df, context = context)
  assert_has_cols(df, c(outcome_col, group_col, term_col, or_col, lcl_col, ucl_col), context = context)

  out <- tibble::tibble(
    method = normalize_cat3_key_results_method(method, context = context),
    group = normalize_key_results_group(df[[group_col]], context = context),
    outcome = normalize_key_results_outcome(df[[outcome_col]], context = context),
    contrast = normalize_cat3_key_results_contrast(
      map_cat3_term_contrast(df[[term_col]]),
      context = context
    ),
    OR = suppressWarnings(as.numeric(df[[or_col]])),
    LCL = suppressWarnings(as.numeric(df[[lcl_col]])),
    UCL = suppressWarnings(as.numeric(df[[ucl_col]]))
  ) |>
    dplyr::filter(is.finite(OR), is.finite(LCL), is.finite(UCL), OR > 0, LCL > 0, UCL > 0)

  if (!nrow(out)) stop(context, ": no finite OR rows after filtering.")
  out
}
```

```r
# helper: standardize categorical predicted-probability outputs from the
# different model tracks into one shared export/plot schema.
normalize_cat3_key_results_prob_df <- function(df, method,
                                               outcome_col = "outcome",
                                               group_col = "group",
                                               category_col = "category",
                                               p_col = "p",
                                               lcl_col = "p_LCL",
                                               ucl_col = "p_UCL",
                                               context = "categorical key-results probability") {
  assert_is_df(df, context = context)
  assert_has_cols(df, c(outcome_col, group_col, category_col, p_col, lcl_col, ucl_col), context = context)

  out <- tibble::tibble(
    method = normalize_cat3_key_results_method(method, context = context),
    group = normalize_key_results_group(df[[group_col]], context = context),
    outcome = normalize_key_results_outcome(df[[outcome_col]], context = context),
    category = normalize_cat3_key_results_category(df[[category_col]], context = context),
    p = suppressWarnings(as.numeric(df[[p_col]])),
    p_LCL = suppressWarnings(as.numeric(df[[lcl_col]])),
    p_UCL = suppressWarnings(as.numeric(df[[ucl_col]]))
  ) |>
    dplyr::filter(
      is.finite(p), is.finite(p_LCL), is.finite(p_UCL),
      dplyr::between(p, 0, 1),
      dplyr::between(p_LCL, 0, 1),
      dplyr::between(p_UCL, 0, 1)
    )

  if (!nrow(out)) stop(context, ": no finite probability rows after filtering.")
  out
}

# helper: format one odds ratio and confidence interval for compact table
# display without repeating string-construction logic downstream.
format_or_ci_export <- function(estimate, lcl, ucl, digits = 2) {
```

```r
  sprintf(
    paste0("%.", digits, "f (%.", digits, "f, %.", digits, "f)"),
    estimate, lcl, ucl
  )
}

# helper: turn raw three-level categorical model output into a manuscript-ready
# supplement table with harmonized labels and CI formatting.
build_supplement_threelevel_table <- function(df, method,
                                              outcome_col = "outcome",
                                              group_col = "group",
                                              term_col = "term",
                                              or_col = "estimate",
                                              lcl_col = "conf.low",
                                              ucl_col = "conf.high",
                                              context = "supplement three-level table") {
  normalize_cat3_key_results_or_df(
    df,
    method = method,
    outcome_col = outcome_col,
    group_col = group_col,
    term_col = term_col,
    or_col = or_col,
    lcl_col = lcl_col,
    ucl_col = ucl_col,
    context = context
  ) |>
    dplyr::transmute(
      group = as.character(group),
      outcome = as.character(outcome),
      contrast = as.character(contrast),
      `OR (95% CI)` = format_or_ci_export(OR, LCL, UCL)
    ) |>
    dplyr::arrange(
      factor(group, levels = KEY_RESULTS_GROUP_LEVELS),
      factor(outcome, levels = KEY_RESULTS_OUTCOME_LEVELS),
```

```r
      factor(contrast, levels = CAT3_KEY_RESULTS_CONTRAST_LEVELS)
    )
}

# helper: build the odds-ratio forest-plot column for the categorical
# key-results figure.
build_cat3_or_column <- function(or_df, column_title = "Odds ratio", show_row_labels = TRUE) {
  axis_spec <- compute_or_axis_spec(or_df, lo_col = "LCL", hi_col = "UCL")
  n_contrasts <- length(CAT3_KEY_RESULTS_CONTRAST_LEVELS)

  plot_df <- or_df |>
    dplyr::mutate(
      contrast = normalize_cat3_key_results_contrast(contrast, context = "build_cat3_or_column"),
      y_base = as.numeric(contrast),
      y = y_base + dplyr::if_else(group == "ABG", -CAT3_FOREST_DODGE_OFFSET, CAT3_FOREST_DODGE_OFFSET)
    )

  ggplot2::ggplot(plot_df, ggplot2::aes(x = OR, y = y, color = group, shape = group)) +
    ggplot2::geom_segment(
      ggplot2::aes(x = LCL, xend = UCL, yend = y),
      linewidth = 0.45,
      alpha = 0.9
    ) +
    ggplot2::geom_point(size = 1.8) +
    ggplot2::geom_vline(xintercept = 1, linetype = "dashed", color = "grey40") +
    ggplot2::facet_grid(outcome ~ ., switch = "y") +
    ggplot2::scale_y_continuous(
      breaks = seq_len(n_contrasts),
      labels = CAT3_KEY_RESULTS_CONTRAST_LEVELS,
      limits = c(n_contrasts + 0.5, 0.5)
    ) +
    ggplot2::scale_color_manual(values = KEY_RESULTS_GROUP_COLORS) +
    ggplot2::scale_shape_manual(values = CAT3_KEY_RESULTS_GROUP_SHAPES) +
    or_axis_scale_x(axis_spec) +
    ggplot2::labs(
      title = column_title,
```

```r
      x = "Odds ratio (log scale)",
      y = NULL,
      color = NULL,
      shape = NULL
    ) +
    theme_pub(base_size = 10) +
    ggplot2::theme(
      legend.position = "top",
      plot.title = ggplot2::element_text(hjust = 0.5, face = "bold"),
      strip.placement = "outside",
      strip.background = ggplot2::element_rect(fill = "grey95", color = NA),
      strip.text.y.left = if (show_row_labels) {
        ggplot2::element_text(angle = 0, face = "bold")
      } else {
        ggplot2::element_blank()
      },
      strip.text.y = ggplot2::element_blank()
    )
}

# helper: build the predicted-probability forest-plot column for the
# categorical key-results figure.
build_cat3_prob_column <- function(prob_df, column_title = "Predicted probability",
                                  show_row_labels = FALSE) {
  axis_spec <- compute_prob_axis_spec(prob_df, lo_col = "p_LCL", hi_col = "p_UCL")
  n_categories <- length(CAT3_KEY_RESULTS_CATEGORY_LEVELS)

  plot_df <- prob_df |>
    dplyr::mutate(
      category = normalize_cat3_key_results_category(category, context = "build_cat3_prob_column"),
      y_base = as.numeric(category),
      y = y_base + dplyr::if_else(group == "ABG", -CAT3_FOREST_DODGE_OFFSET, CAT3_FOREST_DODGE_OFFSET)
    )

  ggplot2::ggplot(plot_df, ggplot2::aes(x = p, y = y, color = group, shape = group)) +
    ggplot2::geom_segment(
```

```
      ggplot2::aes(x = p_LCL, xend = p_UCL, yend = y),
      linewidth = 0.45,
      alpha = 0.9
    ) +
    ggplot2::geom_point(size = 1.8) +
    ggplot2::facet_grid(outcome ~ ., switch = "y") +
    ggplot2::scale_y_continuous(
      breaks = seq_len(n_categories),
      labels = CAT3_KEY_RESULTS_CATEGORY_LEVELS,
      limits = c(n_categories + 0.5, 0.5)
    ) +
    ggplot2::scale_color_manual(values = KEY_RESULTS_GROUP_COLORS) +
    ggplot2::scale_shape_manual(values = CAT3_KEY_RESULTS_GROUP_SHAPES) +
    prob_axis_scale_x(axis_spec) +
    ggplot2::labs(
      title = column_title,
      x = "Predicted probability",
      y = NULL,
      color = NULL,
      shape = NULL
    ) +
    theme_pub(base_size = 10) +
    ggplot2::theme(
      legend.position = "top",
      plot.title = ggplot2::element_text(hjust = 0.5, face = "bold"),
      strip.placement = "outside",
      strip.background = ggplot2::element_blank(),
      strip.text.y.left = if (show_row_labels) {
        ggplot2::element_text(angle = 0, face = "bold")
      } else {
        ggplot2::element_blank()
      },
      strip.text.y = ggplot2::element_blank()
    )
}
```

```r
# helper: assemble the full categorical key-results figure with shared legend
# and caption once the OR and probability columns are ready.
assemble_cat3_key_results_figure <- function(or_df, prob_df, figure_title, figure_caption) {
  or_plot <- build_cat3_or_column(or_df, show_row_labels = TRUE)
  prob_plot <- build_cat3_prob_column(prob_df, show_row_labels = FALSE)
  annotation_args <- list(title = figure_title)
  if (!is.null(figure_caption) && nzchar(figure_caption)) {
    annotation_args$caption <- figure_caption
  }
  (or_plot | prob_plot) +
    patchwork::plot_layout(guides = "collect", widths = c(1, 1)) +
    do.call(
      patchwork::plot_annotation,
      c(
        annotation_args,
        list(theme = ggplot2::theme(
          plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),
          plot.caption = ggplot2::element_text(hjust = 0)
        ))
      )
    ) &
    ggplot2::theme(legend.position = "top")
}

# helper: decode categorical exposure terms into plotted low/high CO2
# categories for the legacy OR plots.
map_or_exposure <- function(df, plot_name) {
  assert_is_df(df, context = paste0("map_or_exposure(", plot_name, ")"))
  stopifnot(nrow(df) > 0L)
  stopifnot("term" %in% names(df))
  df$term <- gsub("`", "", df$term)
  df$exposure <- dplyr::case_when(
    grepl("^pco2_cat_abg", df$term) ~ gsub("^pco2_cat_abg", "", df$term),
    grepl("^pco2_cat_vbg", df$term) ~ gsub("^pco2_cat_vbg", "", df$term),
    grepl("^co2_cat", df$term)      ~ gsub("^co2_cat", "", df$term),
    TRUE                            ~ NA_character_
```

```r
  )
  bad <- is.na(df$exposure) | !df$exposure %in% CO2_CAT_CONTRAST_LEVELS
  if (any(bad)) {
    out_path <- results_path("or_term_mapping_failures.csv")
    bad_terms <- data.frame(term = df$term[bad], group = df$group[bad],
                            plot_name = plot_name, stringsAsFactors = FALSE)
    write_csv_safely(bad_terms, out_path, row_names = FALSE)
    stop(paste0("Unmapped OR terms in ", plot_name, ". See ", out_path))
  }
  df$exposure <- factor(df$exposure, levels = CO2_CAT_CONTRAST_LEVELS)
  df
}

# helper: prepare one tidy data frame for the legacy categorical OR figures,
# including row-drop logging when required.
build_or_plot_df <- function(df, plot_name, expected_exposure_levels) {
  df <- tibble::as_tibble(df)
  req_cols <- c("outcome", "group", "estimate", "conf.low", "conf.high")
  req_cols <- c(req_cols, "exposure")
  assert_has_cols(df, unique(req_cols), context = paste0("build_or_plot_df(", plot_name, ")"))
  df <- dplyr::mutate(
    df,
    estimate = as.numeric(estimate),
    conf.low = as.numeric(conf.low),
    conf.high = as.numeric(conf.high)
  )
  if (safe_nrow(df, paste0("build_or_plot_df(", plot_name, ")")) == 0L) {
    stop("Empty OR plot data in ", plot_name)
  }
  drop <- !is.finite(df$estimate) | !is.finite(df$conf.low) | !is.finite(df$conf.high)
  if (any(drop)) {
    log_plot_drop(plot_name, "non-finite estimate/conf", sum(drop),
                  n_before = nrow(df), n_after = nrow(df) - sum(drop),
                  stage = "build_or_plot_df")
    df <- df[!drop, , drop = FALSE]
  }
```

```r
  drop_pos <- (df$estimate <= 0) | (df$conf.low <= 0) | (df$conf.high <= 0)
  if (any(drop_pos, na.rm = TRUE)) {
    log_plot_drop(plot_name, "non-positive estimate/conf", sum(drop_pos, na.rm = TRUE),
                  n_before = nrow(df), n_after = nrow(df) - sum(drop_pos, na.rm = TRUE),
                  stage = "build_or_plot_df")
    df <- df[!drop_pos, , drop = FALSE]
  }
  exp_drop <- is.na(df$exposure) | trimws(as.character(df$exposure)) == ""
  if (any(exp_drop)) {
    log_plot_drop(plot_name, "missing exposure level", sum(exp_drop),
                  n_before = nrow(df), n_after = nrow(df) - sum(exp_drop),
                  stage = "build_or_plot_df")
    df <- df[!exp_drop, , drop = FALSE]
  }
  df$exposure <- factor(df$exposure, levels = expected_exposure_levels)
  bad_exp <- is.na(df$exposure)
  if (any(bad_exp)) {
    bad_df <- df[bad_exp, , drop = FALSE]
    bad_path <- results_path(paste0("or_unmapped_terms_", plot_name, ".csv"))
    write_csv_safely(bad_df, bad_path, row_names = FALSE)
    stop(paste0("Unmapped exposure terms in ", plot_name, ". See ", bad_path))
  }
  missing_levels <- setdiff(expected_exposure_levels, unique(as.character(df$exposure)))
  if (length(missing_levels)) {
    stop(paste0("Missing exposure levels in ", plot_name, ": ", paste(missing_levels, collapse = ", ")))
  }
  if (nrow(df) == 0L) {
    msg <- paste0("OR plot data empty after filtering in ", plot_name)
    stop(msg)
  }
  df
}

# helper: render the legacy categorical OR plots after validating inputs and
# harmonizing the display labels, legend layout, and axis scaling.
plot_or_safe <- function(df, plot_name, axis_spec,
```

```r
                        color_var = "group", shape_var = "exposure",
                        facet_var = NULL, title = NULL, caption = NULL) {
df <- tibble::as_tibble(df)
assert_is_df(df, context = paste0("plot_or_safe(", plot_name, ")"))
nr <- safe_nrow(df, paste0("plot_or_safe(", plot_name, ")"))
if (nr == 0L) {
  stop("plot_or_safe(", plot_name, "): no rows to plot.")
}
assert_has_cols(df, c("outcome", "group", "exposure", "estimate", "conf.low", "conf.high"),
                context = paste0("plot_or_safe(", plot_name, ")"))
if (isTRUE(getOption("ABG_VBG_DEBUG", FALSE))) {
  message("plot_or_safe(", plot_name, "): class=", paste(class(df), collapse = ", "),
          " nrow=", nr, " ncol=", ncol(df))
}
stopifnot(color_var %in% names(df))
stopifnot(shape_var %in% names(df))
stopifnot(!is.null(axis_spec))
df[[shape_var]] <- as.factor(df[[shape_var]])
n_shape <- dplyr::n_distinct(df[[shape_var]])
if (n_shape > MAX_SHAPE_LEVELS) {
  message("OR plot: too many shape levels (", n_shape, ") in ", plot_name,
          ". Dropping shape and faceting by ", shape_var, ".")
  facet_var <- shape_var
  shape_var <- NULL
}

df$.grp <- 1L
if (!is.null(color_var) && !is.null(shape_var)) {
  df$.grp <- interaction(df[[color_var]], df[[shape_var]], drop = TRUE)
} else if (!is.null(color_var)) {
  df$.grp <- df[[color_var]]
} else if (!is.null(shape_var)) {
  df$.grp <- df[[shape_var]]
}

p <- ggplot2::ggplot(df, ggplot2::aes(x = outcome, y = estimate, ymin = conf.low, ymax = conf.high, group = .grp))
```

```
  if (!is.null(color_var)) {
    p <- p + ggplot2::aes(color = .data[[color_var]])
  }
  if (!is.null(shape_var)) {
    p <- p + ggplot2::aes(shape = .data[[shape_var]])
  }

  p <- p +
    ggplot2::geom_errorbar(
  position = ggplot2::position_dodge(width = 0.7),
  width = 0.2,
  linewidth = 0.9
) +
ggplot2::geom_point(
  position = ggplot2::position_dodge(width = 0.7),
  size = 1.8
) +
    ggplot2::geom_hline(yintercept = 1, linetype = "dashed", colour = "grey40") +
    or_axis_scale(axis_spec) +
    ggplot2::labs(
      title = title,
      x = "Outcome",
      y = "Odds Ratio (log scale, 95% CI)",
      color = if (!is.null(color_var)) "Blood-gas type" else NULL,
      shape = if (!is.null(shape_var)) "PCO2 category" else NULL,
      caption = caption
    ) +
    theme_pub(base_size = 9) +
    ggplot2::theme(
      plot.caption = ggplot2::element_text(hjust = 0),
      axis.text.x = ggplot2::element_text(angle = 15, hjust = 1),
      legend.position = "bottom",
      legend.box = "horizontal",
      plot.margin = ggplot2::margin(6, 6, 6, 6)
    )
  if (!is.null(color_var) || !is.null(shape_var)) {
```

```r
    p <- p + ggplot2::guides(
      color = ggplot2::guide_legend(nrow = 1, byrow = TRUE),
      shape = ggplot2::guide_legend(nrow = 1, byrow = TRUE)
    )
  }

  if (!is.null(facet_var)) {
    p <- p + ggplot2::facet_wrap(stats::as.formula(paste("~", facet_var)))
  }
  p
}
```

```r
# Purpose: preflight scan for very long source lines that can stress LaTeX code-block rendering.
qmd_src_scan <- resolve_current_qmd()
qmd_lines <- readLines(qmd_src_scan, warn = FALSE)
risk_threshold <- 220L
line_nchars <- nchar(qmd_lines, type = "chars", allowNA = FALSE, keepNA = FALSE)
risk_idx <- which(line_nchars > risk_threshold)
risk_df <- if (length(risk_idx)) {
  data.frame(
    line_no = risk_idx,
    n_chars = line_nchars[risk_idx],
    line_preview = substr(qmd_lines[risk_idx], 1, 200),
    stringsAsFactors = FALSE
  )
} else {
  data.frame(
    line_no = integer(),
    n_chars = integer(),
    line_preview = character(),
    stringsAsFactors = FALSE
  )
}
write_csv_safely(risk_df, results_path("latex_codeblock_risk_scan.csv"), row_names = FALSE)
```

```r
# Purpose: setup model diagrams.
```

```r
# Toggle to show diagrams inline in the PDF
show_model_diagrams <- FALSE

model_diagram_dir <- fs::path(results_dir, "figs", "model-diagrams")
fs::dir_create(model_diagram_dir)

# helper: turn free-text model names into filesystem-safe stubs for diagram
# export without changing the displayed label.
.make_safe_name <- function(x) {
  x <- gsub("[^A-Za-z0-9]+", "-", x)
  x <- gsub("(^-|-$)", "", x)
  tolower(x)
}

# helper: recover the response and predictor names from a model formula before
# drawing a simple DAG representation.
.extract_model_vars <- function(fml) {
  tt <- stats::terms(fml)
  response <- all.vars(stats::update(fml, . ~ 1))
  response <- if (length(response)) response[1] else NA_character_
  term_labels <- attr(tt, "term.labels")
  preds <- unique(unlist(lapply(term_labels, function(t) {
    all.vars(stats::as.formula(paste("~", t)))
  })))
  preds <- setdiff(preds, response)
  list(response = response, preds = preds)
}

# helper: build a minimal DAG from a model formula so the notebook can export
# lightweight model-structure diagrams alongside the analysis.
.build_model_dag <- function(fml) {
  vars <- .extract_model_vars(fml)
  if (is.na(vars$response) || length(vars$preds) == 0L) {
    stop("Model diagram: formula has no response or predictors: ", deparse(fml))
  }
  edges <- paste(sprintf("%s -> %s", vars$preds, vars$response), collapse = "\n  ")
```

```r
  dagitty::dagitty(sprintf("dag { %s }", edges))
}

# helper: render and save one model diagram with a size that adapts to the
# number of variables in the formula.
save_model_diagram <- function(name, fml, width = NULL, height = NULL) {
  dag <- .build_model_dag(fml)

  vars <- .extract_model_vars(fml)
  n_pred  <- length(vars$preds)
  n_nodes <- n_pred + 1L

  label_size <- if (n_pred >= 35) 1.4 else if (n_pred >= 25) 1.8 else if (n_pred >= 15) 2.2 else 2.8
  point_size <- if (n_pred >= 25) 2.0 else if (n_pred >= 15) 2.5 else 3.0

  base_width  <- if (is.null(width)) 8 else width
  base_height <- if (is.null(height)) 6 else height
  width  <- max(base_width,  min(24, 0.4 * n_nodes + 4))
  height <- max(base_height, min(16, 0.25 * n_nodes + 4))

  layout <- if (n_pred >= 8) "sugiyama" else "nicely"
  dag_data <- ggdag::tidy_dagitty(dag, layout = layout)$data

  p <- ggplot2::ggplot(
    dag_data,
    ggplot2::aes(x = x, y = y, xend = xend, yend = yend)
  ) +
    ggdag::geom_dag_edges() +
    ggdag::geom_dag_point(size = point_size) +
    ggdag::geom_dag_text_repel(
      ggplot2::aes(label = name),
      size = label_size,
      max.overlaps = Inf
    ) +
    ggplot2::scale_x_continuous(expand = ggplot2::expansion(mult = 0.15)) +
    ggplot2::scale_y_continuous(expand = ggplot2::expansion(mult = 0.15)) +
```

```r
    ggplot2::theme_void() +
    ggplot2::labs(title = name) +
    ggplot2::theme(
      plot.title = ggplot2::element_text(size = 10, hjust = 0.5),
      plot.margin = ggplot2::margin(10, 10, 10, 10)
    )

  file <- fs::path(model_diagram_dir, paste0(.make_safe_name(name), ".png"))
  save_diag_plot(file = file, p = p, width = width, height = height, dpi = 300)
  if (isTRUE(show_model_diagrams)) print(p)
  file
}

model_diagrams <- list()
# helper: save one model diagram and register its path for later reuse.
register_model_diagram <- function(name, fml, width = 8, height = 6) {
  file <- save_model_diagram(name, fml, width = width, height = height)
  model_diagrams[[name]] <<- file
  invisible(file)
}

# helper: batch-register a named list of model formulas as diagram exports.
register_model_diagrams <- function(forms, width = 8, height = 6) {
  purrr::iwalk(forms, function(fml, nm) {
    register_model_diagram(nm, fml, width = width, height = height)
  })
}
```

```r
# Purpose: setup shapviz.
# Keep SHAP enabled in both pilot and full so the report has figure parity
# across IPSW sections. Sampling controls below limit runtime/memory.
RUN_SHAP <- TRUE
SHAP_TOP_N <- 10L
SHAP_SAMPLE_N <- 2000L
SHAP_NSIM <- 20L
MI_SHAP_SAMPLE_N_FULL <- 400L
MI_SHAP_SAMPLE_N_PILOT <- 1000L
```

```r
MI_SHAP_SAMPLE_N <- if (RUN_MODE == "full") MI_SHAP_SAMPLE_N_FULL else MI_SHAP_SAMPLE_N_PILOT
MI_SHAP_MAX_TERMS <- 50000L
MI_SHAP_FAIL_OPEN <- TRUE

HAS_FASTSHAP <- requireNamespace("fastshap", quietly = TRUE)
if (RUN_SHAP && !HAS_FASTSHAP) {
  warning(
    "fastshap is not installed; SHAP figures will be skipped. ",
    "Install with renv::install('fastshap').",
    call. = FALSE
  )
}
```

```r
# The canonical report path does not inject per-chunk runtime text into the PDF.
knitr::opts_chunk$set(runtimelog = FALSE)
```

```r
# Purpose: seed escrow.
seed_escrow <- data.frame(
  component = c(
    "Multiple imputation (mice)",
    "ABG propensity GBM (non-MI)",
    "VBG propensity GBM (non-MI)",
    "MI PS (glm RCS) seeds (ABG per imputation)",
    "MI PS (glm RCS) seeds (VBG per imputation)"
  ),
  seed = c(
    "20251206",
    "42",
    "42",
    "20251206 + imputation index",
    "30251206 + imputation index"
  ),
  stringsAsFactors = FALSE
)
render_table_pdf(
  seed_escrow,
  caption = "Seed escrow for MI and GBM runs",
```

```
  file_stub = "seed_escrow",
  digits = 0
)
```

Seed escrow for MI and GBM runs

component	seed
Multiple imputation (mice)	20251206
ABG propensity GBM (non-MI)	42
VBG propensity GBM (non-MI)	42
MI PS (glm RCS) seeds (ABG per imputation)	20251206 + imputation index
MI PS (glm RCS) seeds (VBG per imputation)	30251206 + imputation index

```
DEBUG_SPLINE <- FALSE
```

```
# Make gt tables robust in PDF: full width, caption, small font
gt_pdf <- function(x, title = NULL, subtitle = NULL) {
  out <- x |>
    gt::tab_options(
      table.width              = gt::pct(100),
      table.align              = "left",
      table.font.size          = gt::px(9),
      data_row.padding         = gt::px(1),
      column_labels.font.size  = gt::px(9),
      heading.title.font.size  = gt::px(10),
      heading.subtitle.font.size = gt::px(9)
    ) |>
    gt::opt_align_table_header(align = "left")
  if (!is.null(title))    out <- out |> gt::tab_caption(title)
  if (!is.null(subtitle)) out <- out |> gt::tab_source_note(subtitle)
  out
}
```

Converts the data from a STATA format to rdata if the rdata file does not exist. If it does already exist, it just loads that.

```
# Purpose: stage1 start.
append_mem_snapshot("stage1", "start", "pre")
```

```r
# data_dir_name resolved in setup-packages from params/env
stata_file <- file.path(data_dir_name, "full_db.dta")

stata_data <- read_dta(stata_file)

# Sanitize variable labels for PDF/TOC safety (avoid Unicode subscripts)
sanitize_label_text <- function(x) {
  if (is.null(x)) return(x)
  x <- gsub("\u2080", "0", x, fixed = TRUE)
  x <- gsub("\u2081", "1", x, fixed = TRUE)
  x <- gsub("\u2082", "2", x, fixed = TRUE)
  x <- gsub("\u2083", "3", x, fixed = TRUE)
  x <- gsub("\u2084", "4", x, fixed = TRUE)
  x <- gsub("\u2085", "5", x, fixed = TRUE)
  x <- gsub("\u2086", "6", x, fixed = TRUE)
  x <- gsub("\u2087", "7", x, fixed = TRUE)
  x <- gsub("\u2088", "8", x, fixed = TRUE)
  x <- gsub("\u2089", "9", x, fixed = TRUE)
  x
}
# helper: replace Unicode subscripts in variable labels before PDF rendering so
# gt/kable output stays stable across LaTeX engines.
sanitize_var_labels <- function(df) {
  labs <- labelled::var_label(df)
  labs <- lapply(labs, sanitize_label_text)
  labelled::var_label(df) <- labs
  df
}
stata_data <- sanitize_var_labels(stata_data)

var_labels <- labelled::var_label(stata_data)
value_labels <- lapply(stata_data, function(x) if (is.labelled(x)) val_labels(x))
saveRDS(
  list(var_labels = var_labels, value_labels = value_labels),
  results_path("stata_labels.rds")
)
```

1.1.1 Configuration for the IPW models

```r
# Purpose: propensity config.
drop_vars_ultra_missing <- c("bnp", "spo2")
cat_vars <- c("sex", "race_ethnicity", "location", "encounter_type")
numeric_vars <- c(
  "age_at_encounter", "curr_bmi", "temp_new", "sbp", "dbp", "hr",
  "sodium", "serum_cr", "serum_hco3", "serum_cl", "serum_lac", "serum_k",
  "wbc", "plt", "serum_phos", "serum_ca"
)
co2_vars <- c("paco2", "vbg_co2", "vbg_o2sat")

covars_gbm <- c(
  "age_at_encounter","sex","race_ethnicity","curr_bmi",
  "copd","asthma","osa","chf","acute_nmd","phtn","ckd","dm",
  "location","encounter_type","temp_new","sbp","dbp","hr",
  "sodium","serum_cr","serum_hco3","serum_cl","serum_lac","serum_k",
  "wbc","plt","serum_phos","serum_ca"
)
covars_gbm <- setdiff(covars_gbm, drop_vars_ultra_missing)
covars_ps <- covars_gbm

# Core adjustment set for conditional prognostic models
adj_core <- c("age_at_encounter", "sex", "race_ethnicity", "location", "encounter_type")

gbm_params <- list(
  n.trees           = 800,
  interaction.depth = 3,
  shrinkage         = 0.01,
  bag.fraction      = 0.8,
  n.minobsinnode    = 10,
  cv.folds          = 0,
  stop.method       = "smd.max",
  n.cores           = 1L
)
stopifnot(gbm_params$stop.method == "smd.max")
```

```r
SPLINE_BASIS <- "ns"
SPLINE_DF <- 4L
stopifnot(SPLINE_BASIS %in% c("ns", "rcs"))
# helper: cap GBM parallelism on large renders so pilot/full runs stay within a
# conservative memory budget.
get_gbm_cores <- function() {
  n_rows <- nrow(subset_data)
  if (is.finite(n_rows) && n_rows > 200000L) return(1L)
  gbm_params$n.cores
}
ps_trunc_quantile <- 0.01
stopifnot(ps_trunc_quantile > 0, ps_trunc_quantile < 0.5)

formula_abg      <- reformulate(covars_gbm, response = "has_abg")
formula_vbg      <- reformulate(covars_gbm, response = "has_vbg")

# Model diagrams: propensity models (GBM PS)
register_model_diagram("PS model: ABG test (GBM)", formula_abg, width = 10, height = 7)
register_model_diagram("PS model: VBG test (GBM)", formula_vbg, width = 10, height = 7)

# TODO: consider stop.method = "es.mean" (ATS version) if smd.max remains unstable on full N.
```

```r
# Purpose: build subset data.
# Start from the complete extracted dataset.
subset_data <- stata_data

# In pilot mode, sample a reproducible fraction for faster iteration.
# In full mode, PILOT_FRAC should be 1 and no sampling is applied.
if (IS_PILOT) {
  set.seed(SAMPLE_SEED)
  subset_data <- dplyr::sample_frac(stata_data, size = PILOT_FRAC)
}
sampled_n <- nrow(subset_data)

# Emit and persist row counts so each render is auditable.
message("DATA: full_n=", nrow(stata_data), " | sampled_n=", sampled_n)
if (!IS_PILOT && PILOT_FRAC < 1) {
```

```r
  stop("RUN_MODE='full' but PILOT_FRAC < 1. Set pilot_frac=1 or run_mode='pilot'.")
}

# Remove encounters with legacy code 1 before encounter-type normalization.
subset_data <- subset_data %>%
  filter(encounter_type != 1)
subset_n <- nrow(subset_data)

message("DATA: analysis_subset_n=", subset_n)

run_rowcounts <- data.frame(
  run_id = diag_run_id,
  run_ts = diag_run_ts,
  run_mode = RUN_MODE,
  pilot_frac = PILOT_FRAC,
  full_n = nrow(stata_data),
  sampled_n = sampled_n,
  subset_n = subset_n,
  stringsAsFactors = FALSE
)
write_csv_safely(run_rowcounts, results_path("run_rowcounts.csv"), row_names = FALSE)
```

```r
# Normalize once early and reuse everywhere

# Keep raw copy for missingness reporting
subset_data_raw <- subset_data
encounter_type_raw <- subset_data_raw$encounter_type

# helper: coerce mixed binary encodings onto an explicit {0,1,NA} integer scale
# before factors and models are built.
to01 <- function(x) {
  # Convert mixed encodings (logical/factor/character/numeric labels)
  # to a strict integer indicator in {0, 1, NA}.
  if (inherits(x, "haven_labelled")) x <- unclass(x)
  if (is.logical(x)) return(as.integer(x))
  if (is.factor(x))  x <- as.character(x)
  out <- rep(NA_integer_, length(x))
```

```r
  xs  <- suppressWarnings(as.numeric(x))
  is_num <- !is.na(xs)
  out[is_num & xs %in% c(0, 1)] <- as.integer(xs[is_num & xs %in% c(0, 1)])
  if (any(!is_num)) {
    idx <- which(!is_num)
    s <- trimws(tolower(as.character(x[idx])))
    # Explicit text mappings used in this project.
    # We support both binary labels and testing-status labels.
    out[idx[s %in% c("0","no","false","female","f")]] <- 0L
    out[idx[s %in% c("1","yes","true","male","m")]]   <- 1L
    out[idx[s %in% c("no test", "not tested", "untested", "control", "absent")]] <- 0L
    out[idx[s %in% c("test", "tested", "treated", "present")]] <- 1L
  }
  out
}

# helper: harmonize encounter-type source values onto the canonical
# Emergency/Inpatient factor used throughout the notebook.
normalize_encounter_type <- function(x) {
  # Harmonize encounter type across numeric codes and free-text variants.
  # Output is the canonical factor used in all models/tables:
  # Emergency / Inpatient.
  s_chr <- trimws(tolower(as.character(x)))
  num_from_text <- suppressWarnings(as.numeric(gsub("[^0-9]+", "", s_chr)))
  lab <- rep(NA_character_, length(s_chr))
  lab[!is.na(num_from_text) & num_from_text == 2] <- "Emergency"
  lab[!is.na(num_from_text) & num_from_text == 3] <- "Inpatient"
  is_na <- is.na(lab)
  is_em <- grepl("\\bemerg(?:ency)?\\b", s_chr) |
           grepl("(^|[^a-z])ed([^a-z]|$)", s_chr) |
           grepl("\\ba&e\\b", s_chr) |
           grepl("\\bemergency\\s+dept\\b", s_chr)
  is_ip <- grepl("\\binpatient\\b", s_chr) |
           grepl("\\binpt\\b",     s_chr) |
           grepl("\\binpat\\b",    s_chr) |
           grepl("(^|[^a-z])ip([^a-z]|$)", s_chr)
```

```r
  lab[is_na & is_em] <- "Emergency"
  lab[is_na & is_ip] <- "Inpatient"
  factor(lab, levels = c("Emergency", "Inpatient"))
}

# helper: robustly coerce labelled/factor/character inputs to numeric while
# stripping non-numeric symbols that appear in imported EHR fields.
coerce_num <- function(x) {
  # Robust numeric coercion for labelled/factor/character inputs.
  # Non-numeric symbols are stripped to avoid parse failures.
  if (inherits(x, "haven_labelled")) x <- unclass(x)
  if (is.factor(x)) x <- as.character(x)
  if (is.character(x)) x <- gsub("[^0-9.+-]", "", x)
  suppressWarnings(as.numeric(x))
}

# Reference factor levels are captured once from source data.
# We then reuse them to keep category ordering stable across stages.
levels_ref <- list(
  sex = c("Female", "Male"),
  race_ethnicity = levels(factor(stata_data$race_ethnicity)),
  location = levels(factor(stata_data$location)),
  encounter_type = levels(normalize_encounter_type(stata_data$encounter_type))
)

# helper: perform the one-time global type normalization that the rest of the
# notebook assumes, while preserving row count.
normalize_types <- function(df, levels_ref = NULL) {
  # 1) Drop variables intentionally excluded due to extreme missingness.
  df <- df[, setdiff(names(df), drop_vars_ultra_missing), drop = FALSE]

  # 2) Coerce core numeric/lab/gas variables to numeric once.
  num_vars <- intersect(c(numeric_vars, co2_vars), names(df))
  for (nm in num_vars) df[[nm]] <- coerce_num(df[[nm]])

  # 3) Force sex to the canonical binary factor.
```

```r
  stopifnot("sex" %in% names(df))
  df$sex <- factor(to01(df$sex), levels = c(0L, 1L), labels = c("Female", "Male"))

  # 4) Normalize encounter type into Emergency/Inpatient.
  stopifnot("encounter_type" %in% names(df))
  df$encounter_type <- normalize_encounter_type(df$encounter_type)

  # 5) Convert remaining categorical predictors to factors.
  # If reference levels are supplied, enforce them for consistency.
  for (nm in setdiff(cat_vars, c("sex", "encounter_type"))) {
    stopifnot(nm %in% names(df))
    if (!is.null(levels_ref) && !is.null(levels_ref[[nm]])) {
      df[[nm]] <- factor(df[[nm]], levels = levels_ref[[nm]])
    } else {
      df[[nm]] <- factor(df[[nm]])
    }
  }

  # 6) Ensure treatment indicators are explicit 0/1 integers.
  stopifnot(all(c("has_abg", "has_vbg") %in% names(df)))
  df$has_abg <- to01(df$has_abg)
  df$has_vbg <- to01(df$has_vbg)

  # Keep row count unchanged: this function is for type normalization only.
  df
}

subset_data <- normalize_types(subset_data_raw, levels_ref)
subset_data <- droplevels_all(subset_data)
subset_data$encounter_type <- droplevels(subset_data$encounter_type)
stopifnot(nlevels(subset_data$encounter_type) == 2L)
subset_n <- nrow(subset_data)

# Factor-level diagnostic (CSV + short summary)
factor_diag <- lapply(names(subset_data), function(v) {
  x <- subset_data[[v]]
```

```r
  data.frame(
    variable = v,
    class = class(x)[1],
    nlevels = if (is.factor(x)) nlevels(x) else NA_integer_,
    n_missing = sum(is.na(x)),
    pct_missing = if (nrow(subset_data) > 0) 100 * sum(is.na(x)) / nrow(subset_data) else NA_real_,
    stringsAsFactors = FALSE
  )
})
factor_diag <- dplyr::bind_rows(factor_diag)
write_csv_safely(factor_diag, results_path("factor_levels_diagnostic.csv"), row_names = FALSE)
render_table_pdf_maybe(
  factor_diag,
  caption = "Factor level diagnostics (all variables)",
  file_stub = "factor_levels_diagnostic",
  digits = 1,
  show = FALSE
)

# Factor expansion guard (logs only)
factor_guard <- factor_diag |>
  dplyr::filter(!is.na(nlevels) & nlevels > MAX_LEVELS_GBM)
write_csv_safely(factor_guard, results_path("factor_expansion_guard.csv"), row_names = FALSE)
if (nrow(factor_guard)) {
  warning("GBM factor expansion guard: factors exceeding MAX_LEVELS_GBM detected. See
   ↪  Results/factor_expansion_guard.csv.",
          call. = FALSE)
}

tab_enc <- table(subset_data$encounter_type, useNA = "ifany")
if (sum(!is.na(subset_data$encounter_type)) == 0) {
  message("All encounter_type values are NA after normalization. Showing top raw values:")
  s_raw <- trimws(tolower(as.character(encounter_type_raw)))
  print(utils::head(sort(table(s_raw), decreasing = TRUE), 20))
  stop("normalize_encounter_type produced all NA; extend the synonym map to your raw values.")
}
```

```r
# Reference profile for conditional curves
stopifnot(all(adj_core %in% names(subset_data)))

# helper: choose the modal observed level/value for each adjustment variable so
# predicted curves are conditioned on a plausible reference profile.
mode_level <- function(x) {
  if (is.factor(x)) {
    tab <- table(x, useNA = "no")
    if (!length(tab)) return(factor(NA, levels = levels(x)))
    lev <- names(tab)[which.max(tab)]
    return(factor(lev, levels = levels(x)))
  }
  if (is.character(x)) {
    tab <- table(x, useNA = "no")
    if (!length(tab)) return(NA_character_)
    return(names(tab)[which.max(tab)])
  }
  stats::median(as.numeric(x), na.rm = TRUE)
}

# helper: build a single-row reference profile for conditional predictions by
# taking the median or modal value of each adjustment covariate.
make_ref_profile <- function(data, adj_vars) {
  # Build a representative covariate profile for conditional predictions:
  # medians for numeric variables and most common levels for categorical ones.
  # Spline plots then vary only CO2 while all other predictors stay fixed.
  out <- lapply(adj_vars, function(v) mode_level(data[[v]]))
  names(out) <- adj_vars
  as.data.frame(out, stringsAsFactors = FALSE)
}

# helper: expand a reference profile over a grid of CO2 values for predicted
# probability calculations.
make_co2_grid <- function(co2_var, co2_vals, ref_df) {
  # Repeat the same reference profile across a sequence of CO2 values so the
```

```r
  # plotted curve isolates the modeled effect of CO2 itself.
  grid <- data.frame(co2_vals)
  names(grid) <- co2_var
  stopifnot(!is.null(ref_df))
  stopifnot(nrow(ref_df) > 0L)
  ref_rep <- ref_df[rep(1L, nrow(grid)), , drop = FALSE]
  grid <- cbind(grid, ref_rep)
  grid
}

# helper: expand a reference profile over both a prediction grid and a single
# reference CO2 value for odds-ratio curve calculations.
make_co2_grid_ref <- function(co2_var, co2_vals, ref_df, co2_ref) {
  # The reference grid index defines where the odds-ratio curve is centered,
  # so OR = 1 at the chosen blood-gas-specific reference CO2 value.
  grid_vals <- c(co2_vals, co2_ref)
  grid_vals <- grid_vals[is.finite(grid_vals)]
  grid_vals <- sort(unique(as.numeric(grid_vals)))
  if (length(grid_vals) < 2) {
    fallback_center <- NA_real_
    if (is.finite(co2_ref)) {
      fallback_center <- co2_ref
    } else if (length(co2_vals) && is.finite(co2_vals[1])) {
      fallback_center <- co2_vals[1]
    }
    if (is.finite(fallback_center)) {
      grid_vals <- sort(unique(c(fallback_center - 1, fallback_center, fallback_center + 1)))
      grid_vals <- grid_vals[is.finite(grid_vals)]
    }
  }
  if (length(grid_vals) < 2) stop("CO2 grid is too small for ", co2_var, ".")
  grid <- make_co2_grid(co2_var, grid_vals, ref_df)
  ref_val <- if (is.finite(co2_ref)) co2_ref else stats::median(grid[[co2_var]], na.rm = TRUE)
  ref_idx <- match(ref_val, grid[[co2_var]])
  if (is.na(ref_idx)) {
    ref_idx <- which.min(abs(grid[[co2_var]] - ref_val))
```

```r
  }
  if (any(diff(grid[[co2_var]]) <= 0)) {
    stop("CO2 grid is not strictly increasing for ", co2_var, ".")
  }
  list(grid = grid, ref_idx = ref_idx, co2_ref = grid[[co2_var]][ref_idx])
}

# helper: convert one fitted spline model into a pointwise odds-ratio curve
# relative to the chosen CO2 reference value.
predict_or_curve_from_fit <- function(fit, grid_df, ref_idx, co2_var) {
  # Build a model matrix consistent with the fitted model (handles factor levels)
  tt <- stats::delete.response(stats::terms(fit))
  xlev <- stats::.getXlevels(tt, stats::model.frame(fit))
  mf_new <- stats::model.frame(tt, grid_df, na.action = stats::na.pass, xlev = xlev)
  mm <- stats::model.matrix(tt, mf_new)
  beta <- stats::coef(fit)
  V <- stats::vcov(fit)

  # Align model matrix columns to coefficient names (fill missing with 0, drop extras)
  bn <- names(beta)
  if (is.null(bn) || any(!nzchar(bn))) {
    if (length(beta) == ncol(mm)) {
      bn <- colnames(mm)
      names(beta) <- bn
    } else {
      stop("predict_or_curve_from_fit: coefficient names missing and dimensions do not match.")
    }
  }
  if (!all(bn %in% colnames(mm))) {
    missing_cols <- setdiff(bn, colnames(mm))
    if (length(missing_cols)) {
      mm <- cbind(mm, matrix(0, nrow = nrow(mm), ncol = length(missing_cols),
                             dimnames = list(NULL, missing_cols)))
    }
  }
  mm <- mm[, bn, drop = FALSE]
```

```r
stopifnot(ncol(mm) == length(beta))

eta <- as.numeric(mm %*% beta)
mmV <- mm %*% V
var_eta <- pmax(rowSums(mmV * mm), 0)
se_eta <- sqrt(var_eta)

mm_ref <- mm[ref_idx, , drop = FALSE]
eta_ref <- eta[ref_idx]
var_ref <- as.numeric(mm_ref %*% V %*% t(mm_ref))
cov_ref <- as.numeric(mmV %*% t(mm_ref))

logOR <- eta - eta_ref
var_logOR <- var_eta + var_ref - 2 * cov_ref
var_logOR <- pmax(var_logOR, 0)
logOR[ref_idx] <- 0
var_logOR[ref_idx] <- 0
se <- sqrt(var_logOR)

data.frame(
  grid_df,
  eta = eta,
  var_eta = var_eta,
  SE_eta = se_eta,
  p = plogis(eta),
  p_LCL = plogis(eta - 1.96 * se_eta),
  p_UCL = plogis(eta + 1.96 * se_eta),
  logOR = logOR,
  SE_logOR = se,
  var_logOR = var_logOR,
  OR = exp(logOR),
  LCL = exp(logOR - 1.96 * se),
  UCL = exp(logOR + 1.96 * se),
  co2_ref = grid_df[[co2_var]][ref_idx],
  ref_idx = ref_idx,
  row.names = NULL
```

```r
  )
}

# These reference profiles define the covariates held fixed in the conditional
# spline plots. Predicted probabilities should therefore be read as "for a
# typical tested encounter" rather than as marginal population-average risks.
x_ref_abg <- make_ref_profile(
  subset_data |> dplyr::filter(has_abg == 1, !is.na(paco2)),
  adj_core
)
x_ref_vbg <- make_ref_profile(
  subset_data |> dplyr::filter(has_vbg == 1, !is.na(vbg_co2)),
  adj_core
)
```

```r
# Purpose: detect machine-local execution resources without changing analysis outputs.
parse_positive_int_env <- function(env_name) {
  raw_val <- trimws(Sys.getenv(env_name, unset = ""))
  if (!nzchar(raw_val)) return(NULL)
  parsed_val <- suppressWarnings(as.integer(raw_val))
  if (is.na(parsed_val) || parsed_val <= 0L) {
    stop(env_name, " must be a positive integer when set; got: '", raw_val, "'.")
  }
  parsed_val
}

detect_physical_ram_gb <- function(default_gb = 8L) {
  override_val <- parse_positive_int_env("ABGVBG_MI_RAM_GB")
  if (!is.null(override_val)) {
    return(list(value = override_val, source = "env:ABGVBG_MI_RAM_GB"))
  }

  sys_name <- Sys.info()[["sysname"]]
  ram_bytes <- NA_real_
  if (identical(sys_name, "Darwin")) {
    sysctl_out <- tryCatch(
      system2("sysctl", c("-n", "hw.memsize"), stdout = TRUE, stderr = FALSE),
```

```r
      error = function(e) character()
    )
    if (length(sysctl_out) >= 1L) ram_bytes <- suppressWarnings(as.numeric(sysctl_out[[1L]]))
  } else if (identical(sys_name, "Linux")) {
    meminfo_path <- "/proc/meminfo"
    if (file.exists(meminfo_path)) {
      meminfo_line <- readLines(meminfo_path, n = 1L, warn = FALSE)
      ram_kb <- suppressWarnings(as.numeric(gsub("[^0-9]", "", meminfo_line)))
      if (is.finite(ram_kb)) ram_bytes <- ram_kb * 1024
    }
  }

  if (!is.finite(ram_bytes) || ram_bytes <= 0) {
    return(list(value = as.integer(default_gb), source = "default"))
  }

  ram_gb <- max(1L, as.integer(floor(ram_bytes / (1024^3))))
  list(value = ram_gb, source = paste0("auto:", sys_name))
}

choose_mi_batch_start <- function(ram_gb) {
  if (!is.finite(ram_gb) || ram_gb <= 0) stop("ram_gb must be a positive finite value.")
  if (ram_gb <= 8L) return(1L)
  if (ram_gb <= 16L) return(2L)
  5L
}

mi_ram_info <- detect_physical_ram_gb(default_gb = 8L)
MI_RAM_GB <- mi_ram_info$value
MI_RAM_GB_SOURCE <- mi_ram_info$source
MI_BATCH_START_OVERRIDE <- parse_positive_int_env("ABGVBG_MI_BATCH_START")
if (!is.null(MI_BATCH_START_OVERRIDE)) {
  MI_BATCH_START <- MI_BATCH_START_OVERRIDE
  MI_BATCH_START_SOURCE <- "env:ABGVBG_MI_BATCH_START"
} else {
  MI_BATCH_START <- choose_mi_batch_start(MI_RAM_GB)
```

```r
  MI_BATCH_START_SOURCE <- paste0("auto:", MI_RAM_GB_SOURCE)
}
```

```r
# Purpose: run metadata.
run_meta <- tibble::tibble(
  run_id     = diag_run_id,
  run_mode   = RUN_MODE,
  pilot_frac = PILOT_FRAC,
  mi_ram_gb  = MI_RAM_GB,
  mi_ram_gb_source = MI_RAM_GB_SOURCE,
  mi_batch_threshold = MI_BATCH_THRESHOLD,
  mi_batch_start = MI_BATCH_START,
  mi_batch_start_source = MI_BATCH_START_SOURCE,
  full_n     = nrow(stata_data),
  sampled_n  = sampled_n,
  subset_n   = nrow(subset_data)
)
render_table_pdf(run_meta,
                 "Run metadata (pilot vs full)",
                 "run_metadata",
                 digits = 2)
```

Run metadata (pilot vs full)

run_id	run_mode	pilot_frac	mi_ram_gb	mi_ram_gb_source	mi_batch_threshold
20260421_121048	pilot	0.25	16	auto:Darwin	5000

mi_batch_start	mi_batch_start_source	full_n	sampled_n	subset_n
2	auto:auto:Darwin	833476	208369	128521

```r
# Write run config JSON for portability
json_escape <- function(x) gsub("\"", "\\\\\"", x)
run_cfg <- list(
  run_id = diag_run_id,
  run_mode = RUN_MODE,
  pilot_frac = PILOT_FRAC,
  mi_ram_gb = MI_RAM_GB,
  mi_ram_gb_source = MI_RAM_GB_SOURCE,
  mi_batch_threshold = MI_BATCH_THRESHOLD,
  mi_batch_start = MI_BATCH_START,
  mi_batch_start_source = MI_BATCH_START_SOURCE,
  data_dir = data_dir_name,
  results_dir = results_dir,
  full_n = nrow(stata_data),
  sampled_n = sampled_n,
  subset_n = nrow(subset_data)
)
json_lines <- c(
  "{",
  paste0("  \"run_id\": \"", json_escape(run_cfg$run_id), "\","),
  paste0("  \"run_mode\": \"", json_escape(run_cfg$run_mode), "\","),
  paste0("  \"pilot_frac\": ", run_cfg$pilot_frac, ","),
  paste0("  \"mi_ram_gb\": ", run_cfg$mi_ram_gb, ","),
  paste0("  \"mi_ram_gb_source\": \"", json_escape(run_cfg$mi_ram_gb_source), "\","),
  paste0("  \"mi_batch_threshold\": ", run_cfg$mi_batch_threshold, ","),
  paste0("  \"mi_batch_start\": ", run_cfg$mi_batch_start, ","),
  paste0("  \"mi_batch_start_source\": \"", json_escape(run_cfg$mi_batch_start_source), "\","),
  paste0("  \"data_dir\": \"", json_escape(run_cfg$data_dir), "\","),
  paste0("  \"results_dir\": \"", json_escape(run_cfg$results_dir), "\","),
  paste0("  \"full_n\": ", run_cfg$full_n, ","),
  paste0("  \"sampled_n\": ", run_cfg$sampled_n, ","),
  paste0("  \"subset_n\": ", run_cfg$subset_n),
  "}"
)
writeLines(json_lines, results_path("run_config.json"))
```

Codebook artifact path: Results/codebookr.docx. Generation is skipped in pilot mode to keep notebook validation renders fast.

```r
# Purpose: codebook export full.
if (identical(RUN_MODE, "pilot")) {
  message("Skipping codebookr.docx generation in pilot mode.")
} else {
  study_codebook <- codebookr::codebook(
    stata_data,
    title = "Full TrinetX",
    subtitle = "Dataset Documentation",
    description = "This dataset contains patient-level records from the TrinetX database.
                   It has been processed and converted from the original Stata file."
  )
  codebook_file <- results_path("codebookr.docx")
  print(study_codebook, codebook_file)
}
```

1.1.2 Outcome Variable Creation

```r
# Purpose: derive death 60d.
subset_data <- subset_data %>%
  mutate(
    ## 1. Did the patient die?
    died = if_else(!is.na(death_date), 1L, 0L),

    ## 2. Absolute death date (if death_date is an offset)
    death_abs = if_else(!is.na(death_date),
                        encounter_date + death_date,
                        as.Date(NA)),

    ## 3. Year month (YM) for encounter and death
    enc_ym   = floor_date(encounter_date, unit = "month"),
    death_ym = floor_date(death_abs      , unit = "month"),

    ## 4. Reference censoring date: 1 Jun 2024
    ref_ym = ymd("2024-06-01"),

    ## 5. Months from encounter to death or censoring
```

```r
    months_death_or_cens = case_when(
      !is.na(death_ym) ~ interval(enc_ym, death_ym) %/% months(1),
      TRUE             ~ interval(enc_ym, ref_ym)   %/% months(1)
    ),

    ## 6. Remove impossible values
    months_death_or_cens = if_else(
      months_death_or_cens < 0 | months_death_or_cens > 16,
      NA_integer_, months_death_or_cens
    ),

    ## 7. Death within one or two months
    died_1mo = if_else(died == 1 & months_death_or_cens <  1, 1L, 0L),
    died_2mo = if_else(died == 1 & months_death_or_cens <= 1, 1L, 0L),

    ## 8. Month of death (missing if censored)
    death_time = if_else(died == 1, months_death_or_cens, NA_integer_),

    ## 9. Death within 60 days (new variable)
    death_60d = if_else(died == 1 & death_abs <= (encounter_date + days(60)), 1L, 0L)
  ) %>%
  select(-enc_ym, -death_ym)

subset_data <- subset_data %>%
  mutate(
    death_60d = if_else(died == 1 & death_abs <= (encounter_date + days(60)), 1L, 0L)
  )
```

```r
# Purpose: death 60d summary.
table(subset_data$death_60d, useNA = "ifany")
```

```
     0      1
115047  13474
```

```r
prop.table(table(subset_data$death_60d, useNA = "ifany"))
```

```
        0         1
0.8951611 0.1048389
```

```r
summary(subset_data$death_60d)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.0000  0.0000  0.0000  0.1048  0.0000  1.0000
```

1.2 Baseline tables

```r
# Robust derivation of analysis variables + helper for Table 1 production
# ---------------------------------------------------------------------------

# helper: label binary 0/1 → "No"/"Yes"
bin_lab <- function(x) {
  y <- to01(x)
  if (all(is.na(y))) {
    return(factor(y, levels = c(0, 1), labels = c("No", "Yes")))
  }
  factor(y, levels = c(0, 1), labels = c("No", "Yes"))
}

# helper: preserve labeled factors if already present; otherwise map numeric codes
label_from_codes <- function(x, codes, labels) {
  if (is.factor(x)) {
    lev <- levels(x)
    if (all(lev %in% labels)) {
      return(factor(x, levels = labels))
    }
    lev_num <- suppressWarnings(as.numeric(lev))
    if (all(!is.na(lev_num)) && all(lev_num %in% codes)) {
      return(factor(as.numeric(as.character(x)), levels = codes, labels = labels))
    }
    return(x)
  }
```

```r
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  if (all(is.na(x_num))) return(factor(x, levels = labels))
  if (all(x_num %in% codes, na.rm = TRUE)) {
    return(factor(x_num, levels = codes, labels = labels))
  }
  factor(x)
}

subset_data <- subset_data %>%
  mutate(
    ## ensure 0/1 numerics (avoids factor-level coercion)
    across(c(has_abg, has_vbg),
           ~ to01(.)),

    ## derive ABG / VBG status groups (binary test status only)
    abg_group = case_when(
      has_abg == 0 ~ "No ABG",
      has_abg == 1 ~ "ABG",
      TRUE         ~ "Missing"
    ),
    vbg_group = case_when(
      has_vbg == 0 ~ "No VBG",
      has_vbg == 1 ~ "VBG",
      TRUE         ~ "Missing"
    ),

    ## factorise groups with explicit NA/Missing level
    abg_group = factor(
      abg_group,
      levels = c("No ABG", "ABG", "Missing")
    ),
    vbg_group = factor(
      vbg_group,
      levels = c("No VBG", "VBG", "Missing")
    ),
```

```r
    ## labelled covariates (robust to factor or numeric codes)
    sex_label = label_from_codes(sex, c(0, 1), c("Female", "Male")),
    race_ethnicity_label = label_from_codes(
      race_ethnicity,
      0:6,
      c("White", "Black or African American", "Hispanic",
        "Asian", "American Indian", "Pacific Islander", "Unknown")
    ),
    location_label = label_from_codes(
      location,
      0:3,
      c("South", "Northeast", "Midwest", "West")
    ),
    encounter_type_label = label_from_codes(
      encounter_type,
      c(2, 3),
      c("Emergency", "Inpatient")
    ),
    osa_label      = bin_lab(osa),
    asthma_label   = bin_lab(asthma),
    copd_label     = bin_lab(copd),
    chf_label      = bin_lab(chf),
    nmd_label      = bin_lab(nmd),
    phtn_label     = bin_lab(phtn),
    ckd_label      = bin_lab(ckd),
    diabetes_label = bin_lab(dm)
  )

# variables to summarise
vars <- c(
  "age_at_encounter", "curr_bmi", "sex_label", "race_ethnicity_label", "location_label",
  "osa_label", "asthma_label", "copd_label", "chf_label", "nmd_label",
  "phtn_label", "ckd_label", "diabetes_label", "encounter_type_label", "vbg_co2", "paco2"
)
vars_baseline <- setdiff(vars, c("vbg_co2", "paco2"))
vars_abg <- c(vars_baseline, "paco2")
```

```r
vars_vbg <- c(vars_baseline, "vbg_co2")

# Table 1 constructor
make_table1 <- function(data, group_var, caption = "") {
  group_sym <- rlang::sym(group_var)

  df <- data %>%
    filter(!is.na(!!group_sym),                    # drop explicit NA
           !!group_sym != "Missing") %>%           # drop "Missing" cohort
    mutate(!!group_sym := droplevels(!!group_sym)) %>%  # only drop group levels
    select(all_of(c(group_var, vars_baseline)))

  empty_fac <- names(which(vapply(df, function(z) is.factor(z) && length(levels(z)) == 0L, logical(1))))
  if (length(empty_fac) > 0) {
    warning("0-level factor columns detected: ", paste(empty_fac, collapse = ", "),
            ". Converting to character to prevent gtsummary failure.", call. = FALSE)
    df[empty_fac] <- lapply(df[empty_fac], as.character)
  }

  df %>%
  gtsummary::tbl_summary(
    by   = !!group_sym,
    statistic = list(
      gtsummary::all_continuous()  ~ "{mean} ± {sd}; {N_miss}/{N_obs} missing ({p_miss}%)",
      gtsummary::all_categorical() ~ "{n} ({p}%)"
    ),
    digits  = list(gtsummary::all_continuous() ~ 1),
    missing = "no",
    label = list(
      age_at_encounter     ~ "Age",
      curr_bmi             ~ "BMI",
      sex_label            ~ "Sex",
      race_ethnicity_label ~ "Race/Ethnicity",
      location_label       ~ "Location",
      osa_label            ~ "OSA",
      asthma_label         ~ "Asthma",
```

```r
      copd_label           ~ "COPD",
      chf_label            ~ "CHF",
      nmd_label            ~ "NMD",
      phtn_label           ~ "PHTN",
      ckd_label            ~ "CKD",
      diabetes_label       ~ "Diabetes",
      encounter_type_label ~ "Encounter Type"
    )
  ) %>%
  gtsummary::add_overall(last = FALSE) %>%
  gtsummary::modify_header(label = "Variable") %>%
  gtsummary::modify_caption(caption)
}

if (sum(!is.na(subset_data$sex_label)) == 0L || length(levels(subset_data$sex_label)) == 0L) {
  warning("sex_label is all NA or has zero levels; check sex normalization/mapping.", call. = FALSE)
  stopifnot("sex" %in% names(subset_data))
}

# build tables
table1A <- make_table1(subset_data, "abg_group", caption = "Table 1A: ABG cohorts")
table1B <- make_table1(subset_data, "vbg_group", caption = "Table 1B: VBG cohorts")

tbl1a_pdf <- to_pdf_table(table1A, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                          longtable = TRUE)
tbl1b_pdf <- to_pdf_table(table1B, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                          longtable = TRUE)
tbl1a_pdf
```

ABG cohorts

Variable	**Overall** N = 128,521	**No ABG** N = 81,868	**ABG** N = 46,653
Age	59.2 ± 17.7; 0.0/128,521.0 missing (0.0%)	58.1 ± 18.1; 0.0/81,868.0 missing (0.0%)	61.2 ± 16.9; 0.0/46,653.0 missing (0.0%)
BMI	31.1 ± 8.3; 73,272.0/128,521.0 missing (57.0%)	32.2 ± 8.7; 46,078.0/81,868.0 missing (56.3%)	29.0 ± 7.2; 27,194.0/46,653.0 missing (58.3%)
Sex			
Female	63,461 (49%)	42,242 (52%)	21,219 (45%)
Male	65,060 (51%)	39,626 (48%)	25,434 (55%)

(continued)

Variable	**Overall** N = 128,521	**No ABG** N = 81,868	**ABG** N = 46,653
Race/Ethnicity			
White	80,152 (62%)	49,867 (61%)	30,285 (65%)
Black or African American	22,349 (17%)	15,542 (19%)	6,807 (15%)
Hispanic	8,479 (6.6%)	5,992 (7.3%)	2,487 (5.3%)
Asian	2,085 (1.6%)	1,250 (1.5%)	835 (1.8%)
American Indian	968 (0.8%)	442 (0.5%)	526 (1.1%)
Pacific Islander	184 (0.1%)	122 (0.1%)	62 (0.1%)
Unknown	14,304 (11%)	8,653 (11%)	5,651 (12%)
Location			
South	60,251 (47%)	34,356 (42%)	25,895 (56%)
Northeast	32,438 (25%)	23,477 (29%)	8,961 (19%)
Midwest	9,646 (7.5%)	5,732 (7.0%)	3,914 (8.4%)
West	26,186 (20%)	18,303 (22%)	7,883 (17%)
OSA	22,584 (18%)	15,183 (19%)	7,401 (16%)
Asthma	17,372 (14%)	12,083 (15%)	5,289 (11%)
COPD	24,996 (19%)	15,107 (18%)	9,889 (21%)
CHF	25,458 (20%)	15,053 (18%)	10,405 (22%)
NMD	5,122 (4.0%)	2,965 (3.6%)	2,157 (4.6%)
PHTN	10,542 (8.2%)	6,108 (7.5%)	4,434 (9.5%)
CKD	22,968 (18%)	13,740 (17%)	9,228 (20%)
Diabetes	37,231 (29%)	23,327 (28%)	13,904 (30%)
Encounter Type			
Emergency	42,859 (33%)	35,638 (44%)	7,221 (15%)
Inpatient	85,662 (67%)	46,230 (56%)	39,432 (85%)

```
tbl1b_pdf
```

VBG cohorts

Variable	**Overall** N = 128,521	**No VBG** N = 91,138	**VBG** N = 37,383
Age	59.2 ± 17.7; 0.0/128,521.0 missing (0.0%)	59.4 ± 17.8; 0.0/91,138.0 missing (0.0%)	58.9 ± 17.6; 0.0/37,383.0 missing (0.0%)
BMI	31.1 ± 8.3; 73,272.0/128,521.0 missing (57.0%)	31.7 ± 8.5; 48,047.0/91,138.0 missing (52.7%)	28.8 ± 7.4; 25,225.0/37,383.0 missing (67.5%)
Sex			
Female	63,461 (49%)	46,152 (51%)	17,309 (46%)
Male	65,060 (51%)	44,986 (49%)	20,074 (54%)
Race/Ethnicity			
White	80,152 (62%)	60,106 (66%)	20,046 (54%)
Black or African American	22,349 (17%)	15,339 (17%)	7,010 (19%)
Hispanic	8,479 (6.6%)	5,795 (6.4%)	2,684 (7.2%)
Asian	2,085 (1.6%)	1,367 (1.5%)	718 (1.9%)
American Indian	968 (0.8%)	492 (0.5%)	476 (1.3%)
Pacific Islander	184 (0.1%)	150 (0.2%)	34 (<0.1%)

(continued)

Variable	**Overall** N = 128,521	**No VBG** N = 91,138	**VBG** N = 37,383
Unknown	14,304 (11%)	7,889 (8.7%)	6,415 (17%)
Location			
South	60,251 (47%)	48,895 (54%)	11,356 (30%)
Northeast	32,438 (25%)	16,441 (18%)	15,997 (43%)
Midwest	9,646 (7.5%)	6,223 (6.8%)	3,423 (9.2%)
West	26,186 (20%)	19,579 (21%)	6,607 (18%)
OSA	22,584 (18%)	16,508 (18%)	6,076 (16%)
Asthma	17,372 (14%)	12,437 (14%)	4,935 (13%)
COPD	24,996 (19%)	17,656 (19%)	7,340 (20%)
CHF	25,458 (20%)	17,459 (19%)	7,999 (21%)
NMD	5,122 (4.0%)	3,762 (4.1%)	1,360 (3.6%)
PHTN	10,542 (8.2%)	7,009 (7.7%)	3,533 (9.5%)
CKD	22,968 (18%)	15,382 (17%)	7,586 (20%)
Diabetes	37,231 (29%)	25,356 (28%)	11,875 (32%)
Encounter Type			
Emergency	42,859 (33%)	31,009 (34%)	11,850 (32%)
Inpatient	85,662 (67%)	60,129 (66%)	25,533 (68%)

```r
# Purpose: float barrier table1abg.
float_barrier("section")
```

```r
# Purpose: export table1a table1b word.
ft_table1A <- as_flex_table(table1A)
ft_table1B <- as_flex_table(table1B)

doc <- read_docx() %>%
  body_add_par("Table 1A. Baseline Characteristics by ABG Group", style = "heading 1") %>%
  body_add_flextable(ft_table1A) %>%
  body_add_par("Table 1B. Baseline Characteristics by VBG Group", style = "heading 1") %>%
  body_add_flextable(ft_table1B)

print(doc, target = results_path("Table1_ABG_VBG.docx"))
```

```r
# Status factors (column labels are taken from factor levels)
subset_data <- subset_data %>%
  mutate(
    abg_status = factor(has_abg, levels = c(0, 1),
                        labels = c("Did not get ABG", "Did get ABG")),
    vbg_status = factor(has_vbg, levels = c(0, 1),
```

```
                          labels = c("Did not get VBG", "Did get VBG"))
  )

# ABG table with "Everyone" column first
tbl1_abg <- subset_data %>%
  select(all_of(vars_baseline), abg_status) %>%
  gtsummary::tbl_summary(
    by = abg_status,
    statistic = list(
      gtsummary::all_continuous()  ~ "{mean} ± {sd}; {N_miss}/{N_obs} missing ({p_miss}%)",
      gtsummary::all_categorical() ~ "{n} ({p}%)"
    ),
    digits  = list(gtsummary::all_continuous() ~ 1),
    missing = "no",
    label = list(
      age_at_encounter     ~ "Age",
      curr_bmi             ~ "BMI",
      sex_label            ~ "Sex",
      race_ethnicity_label ~ "Race/Ethnicity",
      location_label       ~ "Location",
      osa_label            ~ "OSA",
      asthma_label         ~ "Asthma",
      copd_label           ~ "COPD",
      chf_label            ~ "CHF",
      nmd_label            ~ "NMD",
      phtn_label           ~ "PHTN",
      ckd_label            ~ "CKD",
      diabetes_label       ~ "Diabetes",
      encounter_type_label ~ "Encounter Type"
    )
  ) %>%
  gtsummary::add_overall(last = FALSE, col_label = "Everyone") %>%
  gtsummary::bold_labels() %>%
  gtsummary::modify_header(label = "**Variable**")

# VBG table with "Everyone" column first
```

```r
tbl1_vbg <- subset_data %>%
  select(all_of(vars_baseline), vbg_status) %>%
  gtsummary::tbl_summary(
    by = vbg_status,
    statistic = list(
      gtsummary::all_continuous()  ~ "{mean} ± {sd}; {N_miss}/{N_obs} missing ({p_miss}%)",
      gtsummary::all_categorical() ~ "{n} ({p}%)"
    ),
    digits  = list(gtsummary::all_continuous() ~ 1),
    missing = "no",
    label = list(
      age_at_encounter     ~ "Age",
      curr_bmi             ~ "BMI",
      sex_label            ~ "Sex",
      race_ethnicity_label ~ "Race/Ethnicity",
      location_label       ~ "Location",
      osa_label            ~ "OSA",
      asthma_label         ~ "Asthma",
      copd_label           ~ "COPD",
      chf_label            ~ "CHF",
      nmd_label            ~ "NMD",
      phtn_label           ~ "PHTN",
      ckd_label            ~ "CKD",
      diabetes_label       ~ "Diabetes",
      encounter_type_label ~ "Encounter Type"
    )
  ) %>%
  gtsummary::add_overall(last = FALSE, col_label = "Everyone") %>%
  gtsummary::bold_labels() %>%
  gtsummary::modify_header(label = "**Characteristic**")

tbl1_abg <- tbl1_abg %>%
  modify_caption(strip_manual_table_number("**Table 1A. Baseline summary: Everyone and ABG status**"))
tbl1_vbg <- tbl1_vbg %>%
  modify_caption(strip_manual_table_number("**Table 1B. Baseline summary: Everyone and VBG status**"))
```

```r
# merged table for Word export (Table 1: Everyone + ABG/VBG status)
tbl1 <- gtsummary::tbl_merge(
  list(tbl1_abg, tbl1_vbg),
  tab_spanner = c("**ABG status**", "**VBG status**")
)

# Export machine-readable Table 1 artifacts from the same gtsummary objects
# used for manuscript rendering, so the notebook remains the single source
# of truth for final table outputs.
# gtsummary is convenient for display, but manuscript drafting and QA also need
# a plain rectangular file. We therefore reconstruct the displayed statistics
# from the hidden gtsummary body/header objects and align the ABG/VBG panels on
# their shared row structure before exporting a combined CSV/PDF version.
clean_gts_text <- function(x) {
  x <- as.character(x)
  x <- gsub("\\*\\*", "", x)
  x <- gsub("\\s*<br\\s*/?>\\s*", " ", x, perl = TRUE)
  x <- gsub("\\s*\\n\\s*", " ", x, perl = TRUE)
  trimws(x)
}

# helper: reconstruct a rectangular export table from one gtsummary object so
# manuscript tables can be audited outside the PDF.
extract_tbl_summary_csv <- function(tbl, panel_name) {
  body <- tbl$table_body |>
    dplyr::select(
      dplyr::any_of(c("variable", "var_type", "row_type", "var_label", "label")),
      dplyr::starts_with("stat_")
    ) |>
    dplyr::mutate(row_id = dplyr::row_number())

  header_map <- tbl$table_styling$header |>
    dplyr::filter(grepl("^stat_", .data$column)) |>
    dplyr::transmute(
      column = .data$column,
      group_label = clean_gts_text(.data$label)
```

```r
    )

  body |>
    tidyr::pivot_longer(
      cols = dplyr::starts_with("stat_"),
      names_to = "column",
      values_to = "stat"
    ) |>
    dplyr::left_join(header_map, by = "column") |>
    dplyr::mutate(
      panel = panel_name,
      var_label = clean_gts_text(.data$var_label),
      label = clean_gts_text(.data$label)
    ) |>
    dplyr::select(
      .data$row_id, .data$panel, .data$variable, .data$var_type,
      .data$row_type, .data$var_label, .data$label,
      .data$group_label, .data$stat
    ) |>
    tidyr::pivot_wider(names_from = .data$group_label, values_from = .data$stat) |>
    dplyr::arrange(.data$row_id)
}

tbl1_abg_csv <- extract_tbl_summary_csv(tbl1_abg, "ABG status")
tbl1_vbg_csv <- extract_tbl_summary_csv(tbl1_vbg, "VBG status")

table1_key_cols <- c("row_id", "variable", "var_type", "row_type", "var_label", "label")
stopifnot(identical(
  tbl1_abg_csv[, table1_key_cols, drop = FALSE],
  tbl1_vbg_csv[, table1_key_cols, drop = FALSE]
))
stopifnot(identical(tbl1_abg_csv$Everyone, tbl1_vbg_csv$Everyone))

abg_group_cols <- setdiff(names(tbl1_abg_csv), c(table1_key_cols, "panel", "Everyone"))
vbg_group_cols <- setdiff(names(tbl1_vbg_csv), c(table1_key_cols, "panel", "Everyone"))
stopifnot(length(abg_group_cols) == 2L, length(vbg_group_cols) == 2L)
```

```r
tbl1_combined_internal <- tbl1_abg_csv |>
  dplyr::select(
    .data$row_id, .data$panel, .data$variable, .data$var_type, .data$row_type,
    .data$var_label, .data$label, .data$Everyone, dplyr::all_of(abg_group_cols)
  ) |>
  dplyr::left_join(
    tbl1_vbg_csv |>
      dplyr::select(.data$row_id, dplyr::all_of(vbg_group_cols)),
    by = "row_id"
  ) |>
  dplyr::select(-.data$row_id)

tbl1_combined_csv <- tbl1_combined_internal |>
  dplyr::transmute(
    Characteristic = dplyr::if_else(
      .data$row_type == "level",
      paste0("  ", .data$label),
      .data$label
    ),
    Everyone = .data$Everyone,
    `Did not get ABG` = .data[[abg_group_cols[[1L]]]],
    `Did get ABG` = .data[[abg_group_cols[[2L]]]],
    `Did not get VBG` = .data[[vbg_group_cols[[1L]]]],
    `Did get VBG` = .data[[vbg_group_cols[[2L]]]]
  ) |>
  dplyr::mutate(dplyr::across(dplyr::everything(), ~ dplyr::if_else(is.na(.x), "", as.character(.x))))

write_csv_safely(tbl1_abg_csv, results_path("table1_abg_status.csv"), row_names = FALSE)
write_csv_safely(tbl1_vbg_csv, results_path("table1_vbg_status.csv"), row_names = FALSE)
write_csv_safely(tbl1_combined_internal, results_path("table1_combined_internal.csv"), row_names = FALSE)
write_csv_safely(tbl1_combined_csv, results_path("table1_combined.csv"), row_names = FALSE)

write_standalone_table_pdf(
  tbl1_combined_csv,
```

```r
  caption = "Baseline characteristics of the analytic cohort overall and by receipt of arterial and venous blood gas
   ↪  testing",
  file_stub = "table1_combined",
  wide = TRUE,
  digits = 2,
  max_cols = 6,
  landscape = TRUE,
  display_label = "Table 1"
)

tbl1_abg_pdf <- to_pdf_table(tbl1_abg, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                             longtable = TRUE)
tbl1_vbg_pdf <- to_pdf_table(tbl1_vbg, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                             longtable = TRUE)
tbl1_abg_pdf
```

Baseline summary: Everyone and ABG status

Variable	Everyone	**Did not get ABG** N = 81,868	**Did get ABG** N = 46,653
___Age___	59.2 ± 17.7; 0.0/128,521.0 missing (0.0%)	58.1 ± 18.1; 0.0/81,868.0 missing (0.0%)	61.2 ± 16.9; 0.0/46,653.0 missing (0.0%)
___BMI___	31.1 ± 8.3; 73,272.0/128,521.0 missing (57.0%)	32.2 ± 8.7; 46,078.0/81,868.0 missing (56.3%)	29.0 ± 7.2; 27,194.0/46,653.0 missing (58.3%)
___Sex___			
Female	63,461 (49%)	42,242 (52%)	21,219 (45%)
Male	65,060 (51%)	39,626 (48%)	25,434 (55%)
___Race/Ethnicity___			
White	80,152 (62%)	49,867 (61%)	30,285 (65%)
Black or African American	22,349 (17%)	15,542 (19%)	6,807 (15%)
Hispanic	8,479 (6.6%)	5,992 (7.3%)	2,487 (5.3%)
Asian	2,085 (1.6%)	1,250 (1.5%)	835 (1.8%)
American Indian	968 (0.8%)	442 (0.5%)	526 (1.1%)
Pacific Islander	184 (0.1%)	122 (0.1%)	62 (0.1%)
Unknown	14,304 (11%)	8,653 (11%)	5,651 (12%)
___Location___			
South	60,251 (47%)	34,356 (42%)	25,895 (56%)
Northeast	32,438 (25%)	23,477 (29%)	8,961 (19%)
Midwest	9,646 (7.5%)	5,732 (7.0%)	3,914 (8.4%)
West	26,186 (20%)	18,303 (22%)	7,883 (17%)
___OSA___	22,584 (18%)	15,183 (19%)	7,401 (16%)
___Asthma___	17,372 (14%)	12,083 (15%)	5,289 (11%)
___COPD___	24,996 (19%)	15,107 (18%)	9,889 (21%)
___CHF___	25,458 (20%)	15,053 (18%)	10,405 (22%)

(continued)

Variable	Everyone	**Did not get ABG** N = 81,868	**Did get ABG** N = 46,653
___NMD___	5,122 (4.0%)	2,965 (3.6%)	2,157 (4.6%)
___PHTN___	10,542 (8.2%)	6,108 (7.5%)	4,434 (9.5%)
___CKD___	22,968 (18%)	13,740 (17%)	9,228 (20%)
___Diabetes___	37,231 (29%)	23,327 (28%)	13,904 (30%)
___Encounter Type___			
Emergency	42,859 (33%)	35,638 (44%)	7,221 (15%)
Inpatient	85,662 (67%)	46,230 (56%)	39,432 (85%)

```
tbl1_vbg_pdf
```

Baseline summary: Everyone and VBG status

Characteristic	Everyone	**Did not get VBG** N = 91,138	**Did get VBG** N = 37,383
___Age___	59.2 ± 17.7; 0.0/128,521.0 missing (0.0%)	59.4 ± 17.8; 0.0/91,138.0 missing (0.0%)	58.9 ± 17.6; 0.0/37,383.0 missing (0.0%)
___BMI___	31.1 ± 8.3; 73,272.0/128,521.0 missing (57.0%)	31.7 ± 8.5; 48,047.0/91,138.0 missing (52.7%)	28.8 ± 7.4; 25,225.0/37,383.0 missing (67.5%)
___Sex___			
Female	63,461 (49%)	46,152 (51%)	17,309 (46%)
Male	65,060 (51%)	44,986 (49%)	20,074 (54%)
___Race/Ethnicity___			
White	80,152 (62%)	60,106 (66%)	20,046 (54%)
Black or African American	22,349 (17%)	15,339 (17%)	7,010 (19%)
Hispanic	8,479 (6.6%)	5,795 (6.4%)	2,684 (7.2%)
Asian	2,085 (1.6%)	1,367 (1.5%)	718 (1.9%)
American Indian	968 (0.8%)	492 (0.5%)	476 (1.3%)
Pacific Islander	184 (0.1%)	150 (0.2%)	34 (<0.1%)
Unknown	14,304 (11%)	7,889 (8.7%)	6,415 (17%)
___Location___			
South	60,251 (47%)	48,895 (54%)	11,356 (30%)
Northeast	32,438 (25%)	16,441 (18%)	15,997 (43%)
Midwest	9,646 (7.5%)	6,223 (6.8%)	3,423 (9.2%)
West	26,186 (20%)	19,579 (21%)	6,607 (18%)
___OSA___	22,584 (18%)	16,508 (18%)	6,076 (16%)
___Asthma___	17,372 (14%)	12,437 (14%)	4,935 (13%)
___COPD___	24,996 (19%)	17,656 (19%)	7,340 (20%)
___CHF___	25,458 (20%)	17,459 (19%)	7,999 (21%)
___NMD___	5,122 (4.0%)	3,762 (4.1%)	1,360 (3.6%)
___PHTN___	10,542 (8.2%)	7,009 (7.7%)	3,533 (9.5%)
___CKD___	22,968 (18%)	15,382 (17%)	7,586 (20%)
___Diabetes___	37,231 (29%)	25,356 (28%)	11,875 (32%)
___Encounter Type___			
Emergency	42,859 (33%)	31,009 (34%)	11,850 (32%)
Inpatient	85,662 (67%)	60,129 (66%)	25,533 (68%)

```r
# Purpose: float barrier table1 everyone.
float_barrier("section")
```

```r
# Purpose: co2 category definitions.
ABG_CO2_VAR <- "paco2"
VBG_CO2_VAR <- "vbg_co2"
ABG_CO2_LOW <- 35
ABG_CO2_HIGH <- 45
VBG_CO2_LOW <- 40
VBG_CO2_HIGH <- 50
CO2_SPEC <- list(
  ABG = list(var = ABG_CO2_VAR, normal_lo = ABG_CO2_LOW, normal_hi = ABG_CO2_HIGH),
  VBG = list(var = VBG_CO2_VAR, normal_lo = VBG_CO2_LOW, normal_hi = VBG_CO2_HIGH)
)
CO2_SPEC$ABG$ref <- (CO2_SPEC$ABG$normal_lo + CO2_SPEC$ABG$normal_hi) / 2
CO2_SPEC$VBG$ref <- (CO2_SPEC$VBG$normal_lo + CO2_SPEC$VBG$normal_hi) / 2
# Reference values for spline OR curves (midpoint of normal range)
ABG_CO2_REF <- CO2_SPEC$ABG$ref
VBG_CO2_REF <- CO2_SPEC$VBG$ref
CO2_CAT_LEVELS <- c("Normal", "Low", "High")
CO2_CAT_CONTRAST_LEVELS <- setdiff(CO2_CAT_LEVELS, "Normal")

# helper: bucket a continuous CO2 value into the manuscript's low/normal/high
# three-level exposure definition.
make_co2_cat3 <- function(x, low_cut, high_cut) {
  x <- suppressWarnings(as.numeric(x))
  out <- dplyr::case_when(
    is.na(x)       ~ NA_character_,
    x < low_cut    ~ "Low",
    x > high_cut   ~ "High",
    TRUE           ~ "Normal"
  )
  factor(out, levels = CO2_CAT_LEVELS)
}

subset_data <- subset_data %>%
```

```r
  mutate(
    pco2_cat_abg = make_co2_cat3(.data[[ABG_CO2_VAR]], ABG_CO2_LOW, ABG_CO2_HIGH),
    pco2_cat_vbg = make_co2_cat3(.data[[VBG_CO2_VAR]], VBG_CO2_LOW, VBG_CO2_HIGH)
  )

stopifnot("Normal" %in% levels(subset_data$pco2_cat_abg),
          "Normal" %in% levels(subset_data$pco2_cat_vbg))

# helper: warn when a tested-cohort category is sparse enough to make later
# categorical-model estimates unstable.
warn_low_counts <- function(cat, label) {
  tab <- table(cat, useNA = "no")
  if (length(tab) && any(tab < 10)) {
    warning(label, ": low counts in CO2 categories: ",
            paste(names(tab), tab, collapse = ", "), call. = FALSE)
  }
}
warn_low_counts(subset_data$pco2_cat_abg[subset_data$has_abg == 1 & !is.na(subset_data$paco2)],
                "ABG")
warn_low_counts(subset_data$pco2_cat_vbg[subset_data$has_vbg == 1 & !is.na(subset_data$vbg_co2)],
                "VBG")
```

```r
# Fail-fast if any binary CO2 indicator references remain in this QMD
qmd_path <- resolve_current_qmd()
txt <- readLines(qmd_path, warn = FALSE)
pat_var <- paste0("hypercap", "_on_")
if (any(grepl(pat_var, txt))) {
  stop("Binary CO2 indicator variable references remain in the QMD.")
}
```

```r
# ABG cohort (has_abg == 1)
tbl2_abg <- subset_data %>%
  filter(has_abg == 1) %>%
  select(all_of(vars_abg), pco2_cat_abg) %>%
  gtsummary::tbl_summary(
    by = pco2_cat_abg,
    statistic = list(
```

```r
      gtsummary::all_continuous()  ~ "{mean} ± {sd}; {N_miss}/{N_obs} missing ({p_miss}%)",
      gtsummary::all_categorical() ~ "{n} ({p}%)"
    ),
    digits  = list(gtsummary::all_continuous() ~ 1),
    missing = "no",
    label = list(
      age_at_encounter     ~ "Age",
      curr_bmi             ~ "BMI",
      sex_label            ~ "Sex",
      race_ethnicity_label ~ "Race/Ethnicity",
      location_label       ~ "Location",
      osa_label            ~ "OSA",
      asthma_label         ~ "Asthma",
      copd_label           ~ "COPD",
      chf_label            ~ "CHF",
      nmd_label            ~ "NMD",
      phtn_label           ~ "PHTN",
      ckd_label            ~ "CKD",
      diabetes_label       ~ "Diabetes",
      encounter_type_label ~ "Encounter Type",
      paco2                ~ "PaCO2 (mmHg)"
    )
  ) %>%
  gtsummary::modify_header(label = "**Variable**") %>%
  gtsummary::bold_labels() %>%
  gtsummary::modify_caption("Baseline summary by CO2 category within ABG cohort")

# VBG cohort (has_vbg == 1)
tbl2_vbg <- subset_data %>%
  filter(has_vbg == 1) %>%
  select(all_of(vars_vbg), pco2_cat_vbg) %>%
  gtsummary::tbl_summary(
    by = pco2_cat_vbg,
    statistic = list(
      gtsummary::all_continuous()  ~ "{mean} ± {sd}; {N_miss}/{N_obs} missing ({p_miss}%)",
      gtsummary::all_categorical() ~ "{n} ({p}%)"
```

```r
    ),
    digits  = list(gtsummary::all_continuous() ~ 1),
    missing = "no",
    label = list(
      age_at_encounter     ~ "Age",
      curr_bmi             ~ "BMI",
      sex_label            ~ "Sex",
      race_ethnicity_label ~ "Race/Ethnicity",
      location_label       ~ "Location",
      osa_label            ~ "OSA",
      asthma_label         ~ "Asthma",
      copd_label           ~ "COPD",
      chf_label            ~ "CHF",
      nmd_label            ~ "NMD",
      phtn_label           ~ "PHTN",
      ckd_label            ~ "CKD",
      diabetes_label       ~ "Diabetes",
      encounter_type_label ~ "Encounter Type",
      vbg_co2              ~ "PvCO2 (mmHg)"
    )
  ) %>%
  gtsummary::modify_header(label = "**Variable**") %>%
  gtsummary::bold_labels() %>%
  gtsummary::modify_caption("Baseline summary by CO2 category within VBG cohort")


# Render ABG and VBG as separate PDF tables to avoid clipped multi-spanner output.
tbl2_abg_pdf <- to_pdf_table(tbl2_abg, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                             longtable = TRUE)
tbl2_abg_pdf
```

Baseline summary by CO2 category within ABG cohort

Variable	**Normal** N = 20,881	**Low** N = 12,666	**High** N = 13,106
___Age___	60.9 ± 16.9; 0.0/20,881.0 missing (0.0%)	60.8 ± 17.4; 0.0/12,666.0 missing (0.0%)	62.2 ± 16.3; 0.0/13,106.0 missing (0.0%)
___BMI___	29.0 ± 6.9; 12,026.0/20,881.0 missing (57.6%)	28.1 ± 6.7; 7,537.0/12,666.0 missing (59.5%)	29.8 ± 7.8; 7,631.0/13,106.0 missing (58.2%)

(continued)

Variable	**Normal** N = 20,881	**Low** N = 12,666	**High** N = 13,106
___Sex___			
Female	9,176 (44%)	5,924 (47%)	6,119 (47%)
Male	11,705 (56%)	6,742 (53%)	6,987 (53%)
___Race/Ethnicity___			
White	13,595 (65%)	7,644 (60%)	9,046 (69%)
Black or African American	2,871 (14%)	2,051 (16%)	1,885 (14%)
Hispanic	1,141 (5.5%)	736 (5.8%)	610 (4.7%)
Asian	378 (1.8%)	281 (2.2%)	176 (1.3%)
American Indian	244 (1.2%)	207 (1.6%)	75 (0.6%)
Pacific Islander	23 (0.1%)	23 (0.2%)	16 (0.1%)
Unknown	2,629 (13%)	1,724 (14%)	1,298 (9.9%)
___Location___			
South	11,411 (55%)	7,130 (56%)	7,354 (56%)
Northeast	3,903 (19%)	2,054 (16%)	3,004 (23%)
Midwest	1,876 (9.0%)	928 (7.3%)	1,110 (8.5%)
West	3,691 (18%)	2,554 (20%)	1,638 (12%)
___OSA___	3,118 (15%)	1,506 (12%)	2,777 (21%)
___Asthma___	2,240 (11%)	1,162 (9.2%)	1,887 (14%)
___COPD___	3,679 (18%)	1,847 (15%)	4,363 (33%)
___CHF___	4,188 (20%)	2,477 (20%)	3,740 (29%)
___NMD___	1,058 (5.1%)	517 (4.1%)	582 (4.4%)
___PHTN___	1,735 (8.3%)	1,016 (8.0%)	1,683 (13%)
___CKD___	3,955 (19%)	2,595 (20%)	2,678 (20%)
___Diabetes___	5,973 (29%)	3,707 (29%)	4,224 (32%)
___Encounter Type___			
Emergency	3,065 (15%)	1,930 (15%)	2,226 (17%)
Inpatient	17,816 (85%)	10,736 (85%)	10,880 (83%)
___PaCO			
textsubscript2 (mmHg)___	39.7 ± 3.0; 0.0/20,881.0 missing (0.0%)	29.2 ± 4.6; 0.0/12,666.0 missing (0.0%)	59.3 ± 20.0; 0.0/13,106.0 missing (0.0%)

```
tbl2_vbg_pdf <- to_pdf_table(tbl2_vbg, font_size = 7, landscape = FALSE, label_col_width = "2.0in",
                             longtable = TRUE)
tbl2_vbg_pdf
```

Baseline summary by CO2 category within VBG cohort

Variable	**Normal** N = 17,296	**Low** N = 10,162	**High** N = 9,925
___Age___	58.0 ± 18.0; 0.0/17,296.0 missing (0.0%)	58.0 ± 17.7; 0.0/10,162.0 missing (0.0%)	61.2 ± 16.6; 0.0/9,925.0 missing (0.0%)
___BMI___	28.7 ± 7.2; 11,941.0/17,296.0 missing (69.0%)	28.6 ± 7.2; 6,299.0/10,162.0 missing (62.0%)	29.2 ± 8.0; 6,985.0/9,925.0 missing (70.4%)
___Sex___			
Female	8,030 (46%)	4,720 (46%)	4,559 (46%)
Male	9,266 (54%)	5,442 (54%)	5,366 (54%)

(continued)

Variable	**Normal** N = 17,296	**Low** N = 10,162	**High** N = 9,925
___Race/Ethnicity___			
White	9,276 (54%)	5,122 (50%)	5,648 (57%)
Black or African American	3,215 (19%)	1,803 (18%)	1,992 (20%)
Hispanic	1,239 (7.2%)	886 (8.7%)	559 (5.6%)
Asian	354 (2.0%)	208 (2.0%)	156 (1.6%)
American Indian	171 (1.0%)	254 (2.5%)	51 (0.5%)
Pacific Islander	14 (<0.1%)	10 (<0.1%)	10 (0.1%)
Unknown	3,027 (18%)	1,879 (18%)	1,509 (15%)
___Location___			
South	5,421 (31%)	2,584 (25%)	3,351 (34%)
Northeast	7,787 (45%)	3,838 (38%)	4,372 (44%)
Midwest	1,564 (9.0%)	812 (8.0%)	1,047 (11%)
West	2,524 (15%)	2,928 (29%)	1,155 (12%)
___OSA___	2,516 (15%)	1,509 (15%)	2,051 (21%)
___Asthma___	2,122 (12%)	1,321 (13%)	1,492 (15%)
___COPD___	2,764 (16%)	1,560 (15%)	3,016 (30%)
___CHF___	3,224 (19%)	2,030 (20%)	2,745 (28%)
___NMD___	584 (3.4%)	399 (3.9%)	377 (3.8%)
___PHTN___	1,361 (7.9%)	871 (8.6%)	1,301 (13%)
___CKD___	3,275 (19%)	2,313 (23%)	1,998 (20%)
___Diabetes___	5,279 (31%)	3,428 (34%)	3,168 (32%)
___Encounter Type___			
Emergency	6,061 (35%)	2,982 (29%)	2,807 (28%)
Inpatient	11,235 (65%)	7,180 (71%)	7,118 (72%)
___PvCO			
textsubscript2 (mmHg)___	44.6 ± 3.0; 0.0/17,296.0 missing (0.0%)	33.4 ± 5.0; 0.0/10,162.0 missing (0.0%)	61.3 ± 12.9; 0.0/9,925.0 missing (0.0%)

```r
# Keep merged object for Word export only.
tbl2 <- gtsummary::tbl_merge(
  tbls = list(tbl2_abg, tbl2_vbg),
  tab_spanner = c("**ABG (PaCO2)**", "**VBG (PvCO2)**")
) %>%
  gtsummary::modify_caption(strip_manual_table_number("**Table 2. Baseline summary by CO2 category within ABG and VBG
↪  cohorts**"))
```

```r
# Purpose: float barrier table2.
float_barrier("section")
```

```r
# Purpose: tbl2 outcome panel.
outcome_vars <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
outcome_labels <- c(
```

```
  imv_proc = "IMV",
  niv_proc = "NIV",
  death_60d = "Death (60d)",
  hypercap_resp_failure = "Hypercapnic RF"
)

rate_panel <- bind_rows(
  subset_data %>%
    filter(has_abg == 1) %>%
    select(all_of(outcome_vars), pco2_cat_abg) %>%
    tidyr::pivot_longer(all_of(outcome_vars), names_to = "outcome", values_to = "event") %>%
    group_by(pco2_cat_abg, outcome) %>%
    summarise(
      events = sum(event == 1, na.rm = TRUE),
      denom  = sum(!is.na(event)),
      rate   = events / denom,
      .groups = "drop_last"
    ) %>%
    mutate(group = "ABG", co2_group = as.character(pco2_cat_abg)),
  subset_data %>%
    filter(has_vbg == 1) %>%
    select(all_of(outcome_vars), pco2_cat_vbg) %>%
    tidyr::pivot_longer(all_of(outcome_vars), names_to = "outcome", values_to = "event") %>%
    group_by(pco2_cat_vbg, outcome) %>%
    summarise(
      events = sum(event == 1, na.rm = TRUE),
      denom  = sum(!is.na(event)),
      rate   = events / denom,
      .groups = "drop_last"
    ) %>%
    mutate(group = "VBG", co2_group = as.character(pco2_cat_vbg))
) %>%
  ungroup() %>%
  mutate(
    outcome      = recode(outcome, !!!outcome_labels),
    rate_fmt     = sprintf("%s/%s (%.1f%%)", events, denom, 100 * rate),
```

```r
    co2_group    = factor(co2_group, levels = CO2_CAT_LEVELS)
  ) %>%
  select(group, outcome, co2_group, rate_fmt) %>%
  tidyr::pivot_wider(names_from = co2_group, values_from = rate_fmt) %>%
  arrange(factor(group, levels = c("ABG", "VBG")),
          factor(outcome, levels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")))

kableExtra::kbl(
  rate_panel,
  format    = "latex",
  booktabs  = TRUE,
  longtable = FALSE,
  col.names = c("Cohort", "Outcome", "Normal", "Low", "High"),
  caption   = "Table 2a. Crude outcomes by CO2 category",
  label     = NA
) |>
  kableExtra::kable_styling(
    latex_options = c("hold_position", "scale_down"),
    font_size     = 8
  )
```

Table 7: Table 2a. Crude outcomes by CO2 category

Cohort	Outcome	Normal	Low	High
ABG	IMV	4798/20881 (23.0%)	3655/12666 (28.9%)	3555/13106 (27.1%)
ABG	NIV	1561/20881 (7.5%)	995/12666 (7.9%)	2134/13106 (16.3%)
ABG	Death (60d)	2869/20881 (13.7%)	2941/12666 (23.2%)	2439/13106 (18.6%)
ABG	Hypercapnic RF	1106/20881 (5.3%)	643/12666 (5.1%)	3241/13106 (24.7%)
VBG	IMV	2109/17296 (12.2%)	1785/10162 (17.6%)	1894/9925 (19.1%)
VBG	NIV	849/17296 (4.9%)	576/10162 (5.7%)	1232/9925 (12.4%)
VBG	Death (60d)	1829/17296 (10.6%)	1717/10162 (16.9%)	1541/9925 (15.5%)
VBG	Hypercapnic RF	627/17296 (3.6%)	395/10162 (3.9%)	2301/9925 (23.2%)

```r
# Purpose: export table1 tbl2 word.

# gtsummary objects (example: tbl1, tbl2)
ft1 <- as_flex_table(tbl1)
ft2 <- as_flex_table(tbl2)
```

```r
doc1 <- read_docx() %>%
  body_add_par("Table 1. Baseline summary: Everyone, ABG status, and VBG status",
               style = "heading 1") %>%
  body_add_flextable(ft1)
print(doc1, target = results_path("Table1.docx"))

doc2 <- read_docx() %>%
  body_add_par("Table 2. Baseline summary by CO2 category within ABG and VBG cohorts",
               style = "heading 1") %>%
  body_add_flextable(ft2)
print(doc2, target = results_path("Table2.docx"))
```

1.3 Three-level PCO2 categories (unweighted)

Three groups using low/normal/high CO2 categories

```r
# Purpose: or data three level unweighted.
stopifnot(all(c("pco2_cat_abg", "pco2_cat_vbg") %in% names(subset_data)))


# helper: fit one three-level categorical outcome model, optionally restricted
# to the tested cohort and optionally using sampling weights.
fit_cat3_model <- function(data, outcome, cat_var, group_name,
                           adj_vars = NULL,
                           analysis_variant = "unweighted",
                           weight_var = NULL,
                           treat_var = NULL) {
  stopifnot(outcome %in% names(data), cat_var %in% names(data))
  if (!is.null(weight_var)) stopifnot(weight_var %in% names(data))
  if (!is.null(treat_var)) stopifnot(treat_var %in% names(data))

  dat <- data
  if (!is.null(treat_var)) {
    dat <- dat |>
      dplyr::filter(.data[[treat_var]] == 1)
```

```r
}

keep_vars <- unique(c(outcome, cat_var, adj_vars, weight_var))
dat <- dat |>
  dplyr::select(dplyr::all_of(keep_vars)) |>
  dplyr::mutate(
    !!cat_var := stats::relevel(
      factor(.data[[cat_var]], levels = CO2_CAT_LEVELS),
      ref = "Normal"
    ),
    !!outcome := to01(.data[[outcome]])
  ) |>
  dplyr::filter(!is.na(.data[[cat_var]]), !is.na(.data[[outcome]]))

if (nrow(dat) == 0L || dplyr::n_distinct(dat[[outcome]]) < 2L) {
  stop("fit_cat3_model: insufficient outcome variation after filtering for ", group_name, " / ", outcome)
}

rhs_terms <- c(cat_var, adj_vars)
f <- stats::reformulate(rhs_terms, response = outcome)
fit_call <- if (is.null(weight_var)) {
  function() glm(f, data = dat, family = binomial, control = glm.control(maxit = 50))
} else {
  dat <- dat |>
    dplyr::mutate(.wt_cat3 = suppressWarnings(as.numeric(.data[[weight_var]]))) |>
    dplyr::filter(is.finite(.wt_cat3), .wt_cat3 > 0)
  if (nrow(dat) == 0L) {
    stop("fit_cat3_model: no rows with finite positive weights for ", group_name, " / ", outcome)
  }
  function() {
    design <- survey::svydesign(ids = ~1, weights = ~.wt_cat3, data = dat)
    survey::svyglm(f, design = design, family = quasibinomial(),
                   control = glm.control(maxit = 50))
  }
}
```

```r
  fit_res <- fit_with_diagnostics(
    fit_call,
    context = make_context(
      stage = "outcome",
      component = "cat3",
      analysis_variant = analysis_variant,
      model_type = "cat3",
      group = group_name,
      outcome = outcome,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_outcome_diag(fit_res$diag)
  if (is.null(fit_res$fit)) {
    stop("fit_cat3_model: model fit failed for outcome=", outcome,
         " cat_var=", cat_var, " group=", group_name)
  }
  fit_res$fit
}

# helper: pull the low/high-versus-normal odds-ratio rows out of one fitted
# three-level categorical model.
extract_cat3_or_terms <- function(fit_obj, outcome, cat_var, group_name, model_type) {
  broom::tidy(fit_obj, exponentiate = TRUE, conf.int = TRUE) |>
    dplyr::filter(term != "(Intercept)", startsWith(term, cat_var)) |>
    dplyr::mutate(
      outcome = outcome,
      group = group_name,
      model = model_type
    )
}

# helper: thin convenience wrapper retained for legacy unweighted categorical
# OR outputs built from the shared fitting helper above.
run_logit <- function(data, outcome, exposure, group_name, adj_vars = NULL, model_type = "Crude") {
```

```r
  fit_obj <- fit_cat3_model(
    data = data,
    outcome = outcome,
    cat_var = exposure,
    group_name = group_name,
    adj_vars = adj_vars,
    analysis_variant = if (length(adj_vars)) "unweighted" else "crude"
  )
  extract_cat3_or_terms(fit_obj, outcome, exposure, group_name, model_type)
}

outcomes_unw <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")

unw_three_level_forms <- list(
  "ABG 3-level: IMV ~ CO2 category + X"      = reformulate(c("pco2_cat_abg", adj_core), response = "imv_proc"),
  "ABG 3-level: NIV ~ CO2 category + X"      = reformulate(c("pco2_cat_abg", adj_core), response = "niv_proc"),
  "ABG 3-level: Death60d ~ CO2 category + X" = reformulate(c("pco2_cat_abg", adj_core), response = "death_60d"),
  "ABG 3-level: HCRF ~ CO2 category + X"     = reformulate(c("pco2_cat_abg", adj_core), response =
   ↪  "hypercap_resp_failure"),
  "VBG 3-level: IMV ~ CO2 category + X"      = reformulate(c("pco2_cat_vbg", adj_core), response = "imv_proc"),
  "VBG 3-level: NIV ~ CO2 category + X"      = reformulate(c("pco2_cat_vbg", adj_core), response = "niv_proc"),
  "VBG 3-level: Death60d ~ CO2 category + X" = reformulate(c("pco2_cat_vbg", adj_core), response = "death_60d"),
  "VBG 3-level: HCRF ~ CO2 category + X"     = reformulate(c("pco2_cat_vbg", adj_core), response =
   ↪  "hypercap_resp_failure")
)
register_model_diagrams(unw_three_level_forms)

crude_cat3_fits_abg <- setNames(lapply(
  outcomes_unw,
  function(o) fit_cat3_model(
    subset_data, o, "pco2_cat_abg", "ABG",
    adj_vars = NULL,
    analysis_variant = "crude"
  )
), outcomes_unw)
crude_cat3_fits_vbg <- setNames(lapply(
```

```r
  outcomes_unw,
  function(o) fit_cat3_model(
    subset_data, o, "pco2_cat_vbg", "VBG",
    adj_vars = NULL,
    analysis_variant = "crude"
  )
), outcomes_unw)

adjusted_cat3_fits_abg <- setNames(lapply(
  outcomes_unw,
  function(o) fit_cat3_model(
    subset_data, o, "pco2_cat_abg", "ABG",
    adj_vars = adj_core,
    analysis_variant = "unweighted"
  )
), outcomes_unw)
adjusted_cat3_fits_vbg <- setNames(lapply(
  outcomes_unw,
  function(o) fit_cat3_model(
    subset_data, o, "pco2_cat_vbg", "VBG",
    adj_vars = adj_core,
    analysis_variant = "unweighted"
  )
), outcomes_unw)

unw_results_crude <- bind_rows(
  lapply(outcomes_unw, function(o) extract_cat3_or_terms(crude_cat3_fits_abg[[o]], o, "pco2_cat_abg", "ABG",
   ↪  "Crude")),
  lapply(outcomes_unw, function(o) extract_cat3_or_terms(crude_cat3_fits_vbg[[o]], o, "pco2_cat_vbg", "VBG", "Crude"))
)
unw_results_adj <- bind_rows(
  lapply(outcomes_unw, function(o) extract_cat3_or_terms(adjusted_cat3_fits_abg[[o]], o, "pco2_cat_abg", "ABG",
   ↪  "Adjusted")),
  lapply(outcomes_unw, function(o) extract_cat3_or_terms(adjusted_cat3_fits_vbg[[o]], o, "pco2_cat_vbg", "VBG",
   ↪  "Adjusted"))
)
```

```r
unw_threelevel_results <- unw_results_adj %>%
  mutate(method = "Unweighted adjusted")
```

```r
# Purpose: standalone Supplementary Table S2 export for crude categorical associations.
table_s2_crude_threelevel <- build_supplement_threelevel_table(
  unw_results_crude,
  method = "Crude",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "estimate",
  lcl_col = "conf.low",
  ucl_col = "conf.high",
  context = "Supplementary Table S2"
)

write_standalone_table_pdf(
  table_s2_crude_threelevel,
  caption = "Crude associations of pCO2 category with clinical outcomes by ABG and VBG",
  file_stub = "table_s2_crude_threelevel",
  wide = TRUE,
  digits = 2,
  display_label = "Table S2"
)

render_table_pdf_maybe(
  table_s2_crude_threelevel,
  caption = "Crude associations of pCO2 category with clinical outcomes by ABG and VBG",
  file_stub = "table_s2_crude_threelevel",
  wide = TRUE,
  digits = 2,
  show = FALSE
)
```

```r
# Purpose: resume unweighted categorical OR assembly after Supplementary Table S2 export.
```

```r
unw_combined_or_df <- unw_results_adj %>%
  mutate(
    outcome = recode(outcome,
                     imv_proc = "Intubation",
                     niv_proc = "NIV",
                     death_60d = "Death (60d)",
                     hypercap_resp_failure = "Hypercapnic RF")
  )
unw_combined_or_df <- map_or_exposure(unw_combined_or_df, "or-plot-three-level-unweighted") |>
  select(outcome, group, exposure, estimate, conf.low, conf.high)
```

```r
# Purpose: or plot three level unweighted.

unw_combined_or_df$group <- factor(
  unw_combined_or_df$group,
  levels = c("ABG", "VBG")
)

unw_plot_df <- build_or_plot_df(unw_combined_or_df, "or-plot-three-level-unweighted",
                            expected_exposure_levels = CO2_CAT_CONTRAST_LEVELS)
stopifnot(is.data.frame(unw_plot_df))
unw_axis_spec <- compute_or_axis_spec(unw_plot_df, lo_col = "conf.low", hi_col = "conf.high",
                                      default_limits = OR_XLIM)
unw_p_or <- plot_or_safe(
  unw_plot_df,
  plot_name = "or-plot-three-level-unweighted",
  axis_spec = unw_axis_spec,
  title  = "Adjusted odds ratios by PCO2 category (ABG, VBG)",
  caption = paste(
    "Adjusted for age, sex, race/ethnicity, location, and encounter type.",
    "Reference = patients in the normal PCO2 range.",
    "Low: <35 mmHg (ABG) or <40 mmHg (VBG); High: >45 mmHg (ABG) or >50 mmHg (VBG).",
    "Because the underlying cohorts differ (ABG, VBG), denominators are not identical across groups.",
    sep = "\n"
  )
)
print_plot_once(unw_p_or, "or-plot-three-level-unweighted", width = 7.5, height = 4.8)
```

1.4 Observed outcome rates by CO2 bin (unweighted)

```
# Purpose: summarize raw event rates in 1-mmHg CO2 bins to inspect local jumps.
co2_rate_unw_abg <- compute_co2_bin_rates(
  df = subset_data |> dplyr::filter(has_abg == 1),
  co2_var = "paco2",
  outcomes = CO2_RATE_OUTCOMES,
  outcome_labels = CO2_RATE_OUTCOME_LABELS,
  weight_var = NULL,
  q = c(0.02, 0.98),
  bin_width = 1,
  cohort_label = "ABG"
)
co2_rate_unw_vbg <- compute_co2_bin_rates(
  df = subset_data |> dplyr::filter(has_vbg == 1),
  co2_var = "vbg_co2",
  outcomes = CO2_RATE_OUTCOMES,
  outcome_labels = CO2_RATE_OUTCOME_LABELS,
  weight_var = NULL,
  q = c(0.02, 0.98),
  bin_width = 1,
  cohort_label = "VBG"
)

write_csv_safely(co2_rate_unw_abg, results_path("co2_binned_rates_unweighted_abg.csv"), row_names = FALSE)
write_csv_safely(co2_rate_unw_vbg, results_path("co2_binned_rates_unweighted_vbg.csv"), row_names = FALSE)

p_co2_rate_unw_abg <- plot_co2_bin_rate_panel(
  co2_rate_unw_abg,
  title = "Observed outcome rates by ABG CO2 bin (unweighted)",
  x_label = "ABG CO2 (mmHg)",
  vline_at = 45
)
p_co2_rate_unw_vbg <- plot_co2_bin_rate_panel(
  co2_rate_unw_vbg,
  title = "Observed outcome rates by VBG CO2 bin (unweighted)",
```

```
  x_label = "VBG CO2 (mmHg)",
  vline_at = 50
)

print_plot_once(p_co2_rate_unw_abg, "co2-binned-rates-unweighted-abg", width = 8.5, height = 6.0)
print_plot_once(p_co2_rate_unw_vbg, "co2-binned-rates-unweighted-vbg", width = 8.5, height = 6.0)
```

1.5 Restricted cubic spline regressions (unweighted)

Spline curves are shown as odds ratios relative to CO2_ref (midpoint of the normal range), holding covariates at the reference profile.

```
# ABG spline dataset
subset_data_abg <- subset_data %>%
  filter(has_abg == 1, !is.na(paco2)) %>%
  select(paco2, imv_proc, niv_proc, death_60d, hypercap_resp_failure, all_of(adj_core)) %>%
  filter(complete.cases(.))
```

1.5.1 Unweighted, Restricted Cubic Spline Regression - ABG by PaCO2

```
# Purpose: rcs abg unweighted models.
make_spline_fml <- function(outcome, co2_var, adj_vars) {
  spline_term <- if (SPLINE_BASIS == "rcs") {
    sprintf("rms::rcs(%s, %d)", co2_var, SPLINE_DF)
  } else {
    sprintf("splines::ns(%s, %d)", co2_var, SPLINE_DF)
  }
  stats::as.formula(
    paste0(outcome, " ~ ", spline_term,
           if (length(adj_vars)) paste0(" + ", paste(adj_vars, collapse = " + ")) else "")
  )
}

#| code-block-title: "Unweighted ABG spline models (adjusted)"
abg_spline_forms <- list(
  "ABG spline (adjusted): IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "paco2", adj_core),
```

```r
  "ABG spline (adjusted): NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "paco2", adj_core),
  "ABG spline (adjusted): Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "paco2", adj_core),
  "ABG spline (adjusted): HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "paco2", adj_core)
)
register_model_diagrams(abg_spline_forms)

co2_seq_abg <- stats::quantile(subset_data_abg$paco2, probs = c(0.02, 0.98), na.rm = TRUE)
grid_abg_info_unw <- make_co2_grid_ref(
  "paco2",
  seq(co2_seq_abg[1], co2_seq_abg[2], length.out = SPLINE_GRID_N),
  x_ref_abg,
  ABG_CO2_REF
)
grid_abg_unw <- grid_abg_info_unw$grid
ref_idx_abg_unw <- grid_abg_info_unw$ref_idx
co2_ref_abg_unw <- grid_abg_info_unw$co2_ref

# helper: fit one adjusted spline logistic model for the complete-case
# unweighted analyses and record standardized diagnostics.
fit_spline_glm <- function(outcome, co2_var, data, group_label,
                           adj_vars = adj_core,
                           analysis_variant = "unweighted") {
  fit_res <- fit_with_diagnostics(
    function() glm(make_spline_fml(outcome, co2_var, adj_vars),
                   data = data, family = binomial,
                   control = glm.control(maxit = 50)),
    context = make_context(
      stage = "outcome",
      component = "spline",
      analysis_variant = analysis_variant,
      model_type = "spline",
      group = group_label,
      outcome = outcome,
      imputation = NA_integer_,
      batch = NA_integer_
    )
```

```r
  )
  append_outcome_diag(fit_res$diag)
  fit_res$fit
}

fit_imv <- fit_spline_glm("imv_proc", "paco2", subset_data_abg, "ABG", adj_vars = adj_core)
fit_niv <- fit_spline_glm("niv_proc", "paco2", subset_data_abg, "ABG", adj_vars = adj_core)
fit_death <- fit_spline_glm("death_60d", "paco2", subset_data_abg, "ABG", adj_vars = adj_core)
fit_hcrf <- fit_spline_glm("hypercap_resp_failure", "paco2", subset_data_abg, "ABG", adj_vars = adj_core)
if (any(vapply(list(fit_imv, fit_niv, fit_death, fit_hcrf), is.null, logical(1)))) {
  stop("Unweighted ABG spline fits failed; see model_fit_diagnostics.csv.")
}

pred_imv <- predict_or_curve_from_fit(fit_imv, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_niv <- predict_or_curve_from_fit(fit_niv, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_death <- predict_or_curve_from_fit(fit_death, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_hcrf <- predict_or_curve_from_fit(fit_hcrf, grid_abg_unw, ref_idx_abg_unw, "paco2")
## Plotting deferred until VBG curves are computed so axes can be shared.
```

1.5.2 Unweighted, Restricted Cubic Spline - VBG

```r
# --- VBG dataset ---
subset_data_vbg <- subset_data %>%
  dplyr::filter(has_vbg == 1, !is.na(vbg_co2)) %>%
  dplyr::select(vbg_co2, imv_proc, niv_proc, death_60d, hypercap_resp_failure, all_of(adj_core)) %>%
  dplyr::filter(complete.cases(.))
```

```r
# Purpose: rcs vbg unweighted models.
vbg_spline_forms <- list(
  "VBG spline (adjusted): IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "vbg_co2", adj_core),
  "VBG spline (adjusted): NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "vbg_co2", adj_core),
  "VBG spline (adjusted): Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "vbg_co2", adj_core),
  "VBG spline (adjusted): HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "vbg_co2", adj_core)
)
register_model_diagrams(vbg_spline_forms)
```

```r
co2_seq_vbg <- stats::quantile(subset_data_vbg$vbg_co2, probs = c(0.02, 0.98), na.rm = TRUE)
grid_vbg_info_unw <- make_co2_grid_ref(
  "vbg_co2",
  seq(co2_seq_vbg[1], co2_seq_vbg[2], length.out = SPLINE_GRID_N),
  x_ref_vbg,
  VBG_CO2_REF
)
grid_vbg_unw <- grid_vbg_info_unw$grid
ref_idx_vbg_unw <- grid_vbg_info_unw$ref_idx
co2_ref_vbg_unw <- grid_vbg_info_unw$co2_ref

fit_imv_vbg <- fit_spline_glm("imv_proc", "vbg_co2", subset_data_vbg, "VBG", adj_vars = adj_core)
fit_niv_vbg <- fit_spline_glm("niv_proc", "vbg_co2", subset_data_vbg, "VBG", adj_vars = adj_core)
fit_death_vbg <- fit_spline_glm("death_60d", "vbg_co2", subset_data_vbg, "VBG", adj_vars = adj_core)
fit_hcrf_vbg <- fit_spline_glm("hypercap_resp_failure", "vbg_co2", subset_data_vbg, "VBG", adj_vars = adj_core)
if (any(vapply(list(fit_imv_vbg, fit_niv_vbg, fit_death_vbg, fit_hcrf_vbg), is.null, logical(1)))) {
  stop("Unweighted VBG spline fits failed; see model_fit_diagnostics.csv.")
}

pred_imv_vbg <- predict_or_curve_from_fit(fit_imv_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_niv_vbg <- predict_or_curve_from_fit(fit_niv_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_death_vbg <- predict_or_curve_from_fit(fit_death_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_hcrf_vbg <- predict_or_curve_from_fit(fit_hcrf_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
axis_unw_common <- compute_or_axis_spec(
  list(pred_imv, pred_niv, pred_death, pred_hcrf,
       pred_imv_vbg, pred_niv_vbg, pred_death_vbg, pred_hcrf_vbg),
  lo_col = "LCL", hi_col = "UCL"
)

plot_imv <- ggplot(pred_imv, aes(x = paco2, y = OR)) +
  geom_line(color = "blue", linewidth = 1.2) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "blue", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "Intubation (adjusted)", x = "PaCO2 (mmHg)",
       y = paste0("Odds ratio (ref PaCO2 = ", co2_ref_abg_unw, "; log scale)")) +
```

```r
  theme_pub()

plot_niv <- ggplot(pred_niv, aes(x = paco2, y = OR)) +
  geom_line(color = "green", linewidth = 1.2) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "green", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "NIV (adjusted)", x = "PaCO2 (mmHg)",
       y = paste0("Odds ratio (ref PaCO2 = ", co2_ref_abg_unw, "; log scale)")) +
  theme_pub()

plot_death <- ggplot(pred_death, aes(x = paco2, y = OR)) +
  geom_line(color = "red", linewidth = 1.2) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "red", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "Death (60d, adjusted)", x = "PaCO2 (mmHg)",
       y = paste0("Odds ratio (ref PaCO2 = ", co2_ref_abg_unw, "; log scale)")) +
  theme_pub()

plot_hcrf <- ggplot(pred_hcrf, aes(x = paco2, y = OR)) +
  geom_line(color = "purple", linewidth = 1.2) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "purple", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "Hypercapnic RF (adjusted)", x = "PaCO2 (mmHg)",
       y = paste0("Odds ratio (ref PaCO2 = ", co2_ref_abg_unw, "; log scale)")) +
  theme_pub()

plot_imv_vbg <- ggplot(pred_imv_vbg, aes(x = vbg_co2, y = OR)) +
  geom_line(color = "blue") +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "blue", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "IMV (adjusted)", x = "VBG CO2 (mmHg)",
       y = paste0("Odds ratio (ref VBG CO2 = ", co2_ref_vbg_unw, "; log scale)")) +
```

```
  theme_pub()

plot_niv_vbg <- ggplot(pred_niv_vbg, aes(x = vbg_co2, y = OR)) +
  geom_line(color = "green") +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "green", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "NIV (adjusted)", x = "VBG CO2 (mmHg)",
       y = paste0("Odds ratio (ref VBG CO2 = ", co2_ref_vbg_unw, "; log scale)")) +
  theme_pub()

plot_death_vbg <- ggplot(pred_death_vbg, aes(x = vbg_co2, y = OR)) +
  geom_line(color = "red") +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "red", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "Death (60d, adjusted)", x = "VBG CO2 (mmHg)",
       y = paste0("Odds ratio (ref VBG CO2 = ", co2_ref_vbg_unw, "; log scale)")) +
  theme_pub()

plot_hcrf_vbg <- ggplot(pred_hcrf_vbg, aes(x = vbg_co2, y = OR)) +
  geom_line(color = "purple") +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), fill = "purple", alpha = 0.2) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_unw_common) +
  labs(title = "Hypercapnic RF (adjusted)", x = "VBG CO2 (mmHg)",
       y = paste0("Odds ratio (ref VBG CO2 = ", co2_ref_vbg_unw, "; log scale)")) +
  theme_pub()

unw_abg_panel <- (plot_imv | plot_niv) / (plot_death | plot_hcrf) +
  plot_annotation(caption = paste0("Reference PaCO2 = ", co2_ref_abg_unw, " mmHg"))

unw_vbg_panel <- ((plot_imv_vbg | plot_niv_vbg) /
  (plot_death_vbg | plot_hcrf_vbg)) +
  plot_annotation(
    title = paste0("Adjusted odds ratios by VBG CO2 (ref = ", co2_ref_vbg_unw, ")"),
```

```r
    caption = paste0("Reference VBG CO2 = ", co2_ref_vbg_unw, " mmHg")
  )

print_plot_once(unw_abg_panel, "spline-unweighted-abg", width = 8.5, height = 6)
print_plot_once(unw_vbg_panel, "spline-unweighted-vbg", width = 8.5, height = 6)

axis_unw_prob <- compute_prob_axis_spec(
  list(pred_imv, pred_niv, pred_death, pred_hcrf,
       pred_imv_vbg, pred_niv_vbg, pred_death_vbg, pred_hcrf_vbg),
  lo_col = "p_LCL", hi_col = "p_UCL"
)

# helper: plot one unweighted ABG predicted-probability spline panel using the
# shared probability axis computed across outcomes.
plot_prob_abg <- function(dat, title_txt) {
  ggplot(dat, aes(x = paco2, y = p)) +
    geom_line(color = "#1f77b4", linewidth = 1.0) +
    geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), fill = "#1f77b4", alpha = 0.2) +
    prob_axis_scale(axis_unw_prob) +
    labs(
      title = title_txt,
      x = "PaCO2 (mmHg)",
      y = "Predicted probability",
      caption = paste0("Conditional on reference covariate profile X_ref; ref PaCO2 = ", co2_ref_abg_unw, " mmHg")
    ) +
    theme_pub()
}

# helper: plot one unweighted VBG predicted-probability spline panel using the
# shared probability axis computed across outcomes.
plot_prob_vbg <- function(dat, title_txt) {
  ggplot(dat, aes(x = vbg_co2, y = p)) +
    geom_line(color = "#d95f02", linewidth = 1.0) +
    geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), fill = "#d95f02", alpha = 0.2) +
    prob_axis_scale(axis_unw_prob) +
    labs(
```

```r
      title = title_txt,
      x = "VBG CO2 (mmHg)",
      y = "Predicted probability",
      caption = paste0("Conditional on reference covariate profile X_ref; ref VBG CO2 = ", co2_ref_vbg_unw, " mmHg")
    ) +
    theme_pub()
}

unw_abg_prob_panel <- (plot_prob_abg(pred_imv, "IMV (adjusted)") |
  plot_prob_abg(pred_niv, "NIV (adjusted)")) /
  (plot_prob_abg(pred_death, "Death (60d, adjusted)") |
     plot_prob_abg(pred_hcrf, "Hypercapnic RF (adjusted)")) +
  plot_annotation(
    title = paste0("Adjusted predicted probabilities by ABG CO2 (ref = ", co2_ref_abg_unw, ")"),
    caption = paste0("Probabilities are conditional on X_ref; reference PaCO2 = ", co2_ref_abg_unw, " mmHg")
  )

unw_vbg_prob_panel <- (plot_prob_vbg(pred_imv_vbg, "IMV (adjusted)") |
  plot_prob_vbg(pred_niv_vbg, "NIV (adjusted)")) /
  (plot_prob_vbg(pred_death_vbg, "Death (60d, adjusted)") |
     plot_prob_vbg(pred_hcrf_vbg, "Hypercapnic RF (adjusted)")) +
  plot_annotation(
    title = paste0("Adjusted predicted probabilities by VBG CO2 (ref = ", co2_ref_vbg_unw, ")"),
    caption = paste0("Probabilities are conditional on X_ref; reference VBG CO2 = ", co2_ref_vbg_unw, " mmHg")
  )

print_plot_once(unw_abg_prob_panel, "spline-unweighted-abg-prob", width = 8.5, height = 6)
print_plot_once(unw_vbg_prob_panel, "spline-unweighted-vbg-prob", width = 8.5, height = 6)
```

```r
# Purpose: float barrier spline unweighted.
float_barrier("section")
```

2 Inverse Propensity Weighting (GBM)

IPW done using Gradient Boosting Methods (GBM) - a type of decision-tree based machine learning. "***Random forests and GBM are designed to automatically include relevant interactions for variables included in the model.*** As such, using a GBM to estimate the PS model, can reduce model misspecification, since ***the analyst is not required to identify relevant interactions or nonlinearities.***" from this citation: PMID: 39947224. PMCID: PMC11825193.

Current propensity score uses `covars_gbm` (demographics, comorbidities, encounter type, vitals, labs) as defined above; in this block only `encounter_type` is explicitly factored before weighting.

Note: for all these, I suggested new GBM adjustments that accomplish the following:

1. Smaller GBM & balance-based stopping (stop.method = "smd.max") → faster fit, avoids over-fitting, lighter tails (which lead to extreme weights that are problematic).
2. Target balance compares weighted treated cohort to the full sample; aim for $|\text{SMD}| < 0.1$.
3. Weight stabilization (divide by mean) mitigates a few huge weights. We use one-sided truncation at very small propensities (caps large weights only).
4. Uses robust variance estimation (e.g. allows the variances to change by PaCO2) for IP-weighted GLM; works with splines via rcs(). This is a bit nuanced but I think good to change even though it adds complexity
5. Deterministic seed ensures result replication.

2.0.1 ABG IPW weighting and diagnostics

```
# Already normalized globally; just drop unused levels
subset_data$encounter_type <- droplevels(subset_data$encounter_type)
```

GBM tuning is shared across ABG and VBG via `gbm_params` to keep symmetry; update there if needed.

```
#   1. fit GBM propensity model, ABG
# Build a minimal modeling frame (treatment + covariates only).
# Including has_vbg preserves shared normalization checks used elsewhere.
set.seed(42)
gbm_df_abg <- subset_data[, c("has_abg", "has_vbg", covars_gbm), drop = FALSE]

# Normalize data types before fitting GBM so predictors are consistent.
```

```r
gbm_df_abg <- normalize_types(gbm_df_abg, levels_ref)
gbm_df_abg <- droplevels_all(gbm_df_abg)

# Log potential design-size problems and memory usage around model fit.
gbm_preflight(gbm_df_abg, covars_gbm, "unimp_abg")
append_mem_snapshot("gbm_unimp", "unimp_abg", "pre")

weight_model <- do.call(
  weightit,
  c(
    list(
      formula_abg,
      data        = gbm_df_abg,
      method      = "gbm",
      estimand    = "ATE",
      missing    = "ind",
      include.obj = FALSE
    ),
    gbm_params
  )
)
append_mem_snapshot("gbm_unimp", "unimp_abg", "post")

#   2. One-sided IPSW (ABG observed only) + truncation of small propensities
# `compute_ipow_weights()` applies the project's one-sided weighting rule:
# treated rows receive inverse propensity weights, non-treated rows are anchors.
ipow_abg <- compute_ipow_weights(
  weight_model,
  treat = gbm_df_abg$has_abg,
  ps_floor_quantile = ps_trunc_quantile,
  stabilize = TRUE
)
w_abg <- ipow_abg$weights
ps_floor_abg <- ipow_abg$ps_floor
subset_data$trunc_abg <- ipow_abg$truncated
subset_data$ps_abg <- ipow_abg$ps
```

```r
subset_data$w_abg  <- w_abg

# Sanity-check: treated weights must be finite before outcome modeling.
assert_finite_weights(w_abg[subset_data$has_abg == 1], "w_abg")
rm(weight_model, gbm_df_abg)
invisible(gc())

# Balance diagnostics and treated-only outcome models are handled later.
```

Inverse Propensity-Weighted Logistic Regressions with CO2 predictor represented as a restricted cubic spline.

These are covariate-adjusted outcome models (outcome ~ spline(CO2) + X), fit separately for ABG and VBG cohorts using survey::svyglm with robust (design-based) SEs. Spline curves are shown as odds ratios relative to CO2_ref (midpoint of the normal range).

2.0.2 ABG IPW spline models

```r
# set.seed(42)  # reproducible GBM fit
#
# #   1. inverse-probability weights for receiving an ABG
#
# # done in the last block, so not needed
#

# Model diagrams: IPW ABG spline models
ipw_abg_rcs_forms <- list(
  "ABG IPW spline (adjusted): IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "paco2", adj_core),
  "ABG IPW spline (adjusted): NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "paco2", adj_core),
  "ABG IPW spline (adjusted): Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "paco2", adj_core),
  "ABG IPW spline (adjusted): HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "paco2", adj_core)
)
register_model_diagrams(ipw_abg_rcs_forms)

#   2. analysis sample: rows with a measured PaCO2
subset_data_abg <- subset_data %>%
  filter(!is.na(paco2)) %>%                    # implies has_abg == 1
  select(paco2, imv_proc, niv_proc, death_60d,
         hypercap_resp_failure, w_abg, all_of(adj_core)) %>%
```

```r
  filter(complete.cases(.))


#   3. weighted logistic spline models with robust SEs
# helper: fit one ABG IPW spline model with robust survey variance and attach
# standardized diagnostics.
fitfun <- function(formula, outcome) {
  fit_res <- fit_with_diagnostics(
    function() svyglm(
      formula,
      design = svydesign(ids = ~1, weights = ~w_abg, data = subset_data_abg),
      family = quasibinomial(),
      control = glm.control(maxit = 50)
    ),
    context = make_context(
      stage = "outcome",
      component = "spline",
      analysis_variant = "ipw",
      model_type = "spline",
      group = "ABG",
      outcome = outcome,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_outcome_diag(fit_res$diag)
  fit_res$fit
}

fit_imv_abg   <- fitfun(make_spline_fml("imv_proc", "paco2", adj_core), "imv_proc")
fit_niv_abg   <- fitfun(make_spline_fml("niv_proc", "paco2", adj_core), "niv_proc")
fit_death_abg <- fitfun(make_spline_fml("death_60d", "paco2", adj_core), "death_60d")
fit_hcrf_abg  <- fitfun(make_spline_fml("hypercap_resp_failure", "paco2", adj_core),
                        "hypercap_resp_failure")
if (any(vapply(list(fit_imv_abg, fit_niv_abg, fit_death_abg, fit_hcrf_abg), is.null, logical(1)))) {
  stop("IPW ABG spline fits failed; see model_fit_diagnostics.csv.")
```

```r
}

#   4. prediction helper
# helper: generate one ABG IPW spline odds-ratio curve over the observed CO2
# range while holding the reference profile fixed.
mkpred <- function(fit, data_ref, co2_var, ref_df, co2_ref) {
  co2_seq <- stats::quantile(data_ref[[co2_var]], probs = c(0.02, 0.98), na.rm = TRUE)
  grid_info <- make_co2_grid_ref(
    co2_var,
    seq(co2_seq[1], co2_seq[2], length.out = SPLINE_GRID_N),
    ref_df,
    co2_ref
  )
  predict_or_curve_from_fit(fit, grid_info$grid, grid_info$ref_idx, co2_var)
}

pred_imv_abg   <- mkpred(fit_imv_abg,   subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_niv_abg   <- mkpred(fit_niv_abg,   subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_death_abg <- mkpred(fit_death_abg, subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_hcrf_abg  <- mkpred(fit_hcrf_abg,  subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
axis_abg_ipw_trim <- compute_or_axis_spec(
  list(pred_imv_abg, pred_niv_abg, pred_death_abg, pred_hcrf_abg),
  lo_col = "LCL", hi_col = "UCL"
)
#   5. plotting
# Plotting deferred until VBG curves are computed so axes can be shared.
```

Restricting plots bewtween 0.02 and 0.98

2.0.3 ABG IPW spline models (2–98th percentile)

```r
# Purpose: ipw abg rcs trimmed.
subset_data_abg <- subset_data %>%
  filter(!is.na(paco2)) %>%                    # implies has_abg == 1
  select(paco2, imv_proc, niv_proc, death_60d,
         hypercap_resp_failure, w_abg, all_of(adj_core)) %>%
```

```r
  filter(complete.cases(.))

# helper: refit the ABG IPW spline models on the trimmed 2nd-98th percentile
# plotting range used for the manuscript-friendly panels.
fitfun <- function(formula, outcome) {
  fit_res <- fit_with_diagnostics(
    function() svyglm(
      formula,
      design = svydesign(ids = ~1, weights = ~w_abg, data = subset_data_abg),
      family = quasibinomial(),
      control = glm.control(maxit = 50)
    ),
    context = make_context(
      stage = "outcome",
      component = "spline",
      analysis_variant = "ipw",
      model_type = "spline",
      group = "ABG",
      outcome = outcome,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_outcome_diag(fit_res$diag)
  fit_res$fit
}

fit_imv_abg   <- fitfun(make_spline_fml("imv_proc", "paco2", adj_core), "imv_proc")
fit_niv_abg   <- fitfun(make_spline_fml("niv_proc", "paco2", adj_core), "niv_proc")
fit_death_abg <- fitfun(make_spline_fml("death_60d", "paco2", adj_core), "death_60d")
fit_hcrf_abg  <- fitfun(make_spline_fml("hypercap_resp_failure", "paco2", adj_core),
                        "hypercap_resp_failure")
if (any(vapply(list(fit_imv_abg, fit_niv_abg, fit_death_abg, fit_hcrf_abg), is.null, logical(1)))) {
  stop("IPW ABG spline fits (trimmed) failed; see model_fit_diagnostics.csv.")
}
```

```r
#   4. prediction helper
# helper: generate one trimmed-range ABG IPW spline odds-ratio curve from a
# fitted survey-weighted model.
mkpred <- function(fit, data_ref, co2_var, ref_df, co2_ref) {
  q <- stats::quantile(data_ref[[co2_var]], probs = c(0.02, 0.98), na.rm = TRUE)
  grid_info <- make_co2_grid_ref(
    co2_var,
    seq(q[1], q[2], length.out = SPLINE_GRID_N),
    ref_df,
    co2_ref
  )
  predict_or_curve_from_fit(fit, grid_info$grid, grid_info$ref_idx, co2_var)
}

pred_imv_abg   <- mkpred(fit_imv_abg,   subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_niv_abg   <- mkpred(fit_niv_abg,   subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_death_abg <- mkpred(fit_death_abg, subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)
pred_hcrf_abg  <- mkpred(fit_hcrf_abg,  subset_data_abg, "paco2", x_ref_abg, ABG_CO2_REF)

#   5. plotting
xlab <- expression(paste("ABG CO"[2], " (mmHg)"))

# helper: draw one trimmed-range ABG IPW odds-ratio panel.
# helper: draw one VBG IPW odds-ratio panel using the ABG/VBG-shared axis.
plt <- function(dat, title)
  ggplot(dat, aes(paco2, OR)) +
    geom_line() +
    geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.2) +
    geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
    or_axis_scale(axis_abg_ipw_trim) +
    labs(title = title, x = xlab,
         y = paste0("Odds ratio (ref = ", ABG_CO2_REF, "; log scale)")) +
    theme_pub()

ipw_abg_panel <- (patchwork::wrap_plots(
  plt(pred_imv_abg,   "IMV"),
```

```
  plt(pred_niv_abg,   "NIV"),
  plt(pred_death_abg, "Death"),
  plt(pred_hcrf_abg,  "Hypercapnic RF"),
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted odds ratios by ABG CO2 (ref = ",
                   ABG_CO2_REF, "; conditional on X; 2-98% range)"),
    caption = paste0("Reference PaCO2 = ", ABG_CO2_REF, " mmHg")
  )

print_plot_once(ipw_abg_panel, "spline-ipw-abg-trimmed", width = 8.5, height = 6)

axis_abg_ipw_trim_prob <- compute_prob_axis_spec(
  list(pred_imv_abg, pred_niv_abg, pred_death_abg, pred_hcrf_abg),
  lo_col = "p_LCL", hi_col = "p_UCL"
)

# helper: draw one trimmed-range ABG IPW predicted-probability panel.
plt_prob <- function(dat, title)
  ggplot(dat, aes(paco2, p)) +
    geom_line(color = "#1f77b4") +
    geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), alpha = 0.2, fill = "#1f77b4") +
    prob_axis_scale(axis_abg_ipw_trim_prob) +
    labs(
      title = title,
      x = xlab,
      y = "Predicted probability"
    ) +
    theme_pub()

ipw_abg_prob_panel <- (patchwork::wrap_plots(
  plt_prob(pred_imv_abg,   "IMV"),
  plt_prob(pred_niv_abg,   "NIV"),
  plt_prob(pred_death_abg, "Death"),
  plt_prob(pred_hcrf_abg,  "Hypercapnic RF"),
```

```
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted predicted probabilities by ABG CO2 (ref = ",
                   ABG_CO2_REF, "; conditional on X; 2-98% range)"),
    caption = paste0("Probabilities are conditional on X_ref; reference PaCO2 = ", ABG_CO2_REF, " mmHg")
  )

print_plot_once(ipw_abg_prob_panel, "spline-ipw-abg-trimmed-prob", width = 8.5, height = 6)
```

VBG uses the same GBM tuning as ABG (shared `gbm_params`).

2.0.4 VBG IPW weighting and spline models

```
#  Inverse-propensity weighting & outcome modelling for **VBG** cohort
#    - mirrored 1-to-1 to the validated ABG workflow

set.seed(42)

# 1. IPW for VBG ---------------------------------------------------------------
set.seed(42)
# Mirror ABG logic: create a minimal design frame and normalize types first.
gbm_df_vbg <- subset_data[, c("has_abg", "has_vbg", covars_gbm), drop = FALSE]
gbm_df_vbg <- normalize_types(gbm_df_vbg, levels_ref)
gbm_df_vbg <- droplevels_all(gbm_df_vbg)

# Capture model-size and memory telemetry before/after fit.
gbm_preflight(gbm_df_vbg, covars_gbm, "unimp_vbg")
append_mem_snapshot("gbm_unimp", "unimp_vbg", "pre")
w_vbg <- do.call(
  weightit,
  c(
    list(
      formula_vbg,
      data        = gbm_df_vbg,
      method      = "gbm",
```

```
      estimand    = "ATE",
      missing     = "ind",
      include.obj = FALSE
    ),
    gbm_params
  )
)
append_mem_snapshot("gbm_unimp", "unimp_vbg", "post")

# One-sided IPSW (VBG observed only) + truncation of small propensities
# Same weighting convention as ABG, but for VBG testing indicator.
ipow_vbg <- compute_ipow_weights(
  w_vbg,
  treat = gbm_df_vbg$has_vbg,
  ps_floor_quantile = ps_trunc_quantile,
  stabilize = TRUE
)
w_vbg_ipow <- ipow_vbg$weights
ps_floor_vbg <- ipow_vbg$ps_floor
subset_data$trunc_vbg <- ipow_vbg$truncated
subset_data$ps_vbg <- ipow_vbg$ps
subset_data$w_vbg  <- w_vbg_ipow

# Guard against non-finite treated weights before survey models.
assert_finite_weights(w_vbg_ipow[subset_data$has_vbg == 1], "w_vbg")
rm(w_vbg, gbm_df_vbg)
invisible(gc())

# Balance diagnostics are handled later.

# 2. Analysis set (VBG only) ----------------------------------------------------
subset_data_vbg <- subset_data %>%
  filter(!is.na(vbg_co2)) %>%
  select(vbg_co2, imv_proc, niv_proc, death_60d,
         hypercap_resp_failure, w_vbg, all_of(adj_core)) %>%
  filter(complete.cases(.))
```

```r
# helper: fit one VBG IPW spline model with robust survey variance and attach
# standardized diagnostics.
fitfun <- function(formula, outcome) {
  fit_res <- fit_with_diagnostics(
    function() svyglm(
      formula,
      design = svydesign(ids = ~1, weights = ~w_vbg, data = subset_data_vbg),
      family = quasibinomial(),
      control = glm.control(maxit = 50)
    ),
    context = make_context(
      stage = "outcome",
      component = "spline",
      analysis_variant = "ipw",
      model_type = "spline",
      group = "VBG",
      outcome = outcome,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_outcome_diag(fit_res$diag)
  fit_res$fit
}

# Model diagrams: IPW VBG spline models
ipw_vbg_rcs_forms <- list(
  "VBG IPW spline (adjusted): IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "vbg_co2", adj_core),
  "VBG IPW spline (adjusted): NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "vbg_co2", adj_core),
  "VBG IPW spline (adjusted): Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "vbg_co2", adj_core),
  "VBG IPW spline (adjusted): HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "vbg_co2",
   ↪  adj_core)
)
register_model_diagrams(ipw_vbg_rcs_forms)
```

```r
fit_imv_vbg   <- fitfun(make_spline_fml("imv_proc", "vbg_co2", adj_core), "imv_proc")
fit_niv_vbg   <- fitfun(make_spline_fml("niv_proc", "vbg_co2", adj_core), "niv_proc")
fit_death_vbg <- fitfun(make_spline_fml("death_60d", "vbg_co2", adj_core), "death_60d")
fit_hcrf_vbg  <- fitfun(make_spline_fml("hypercap_resp_failure", "vbg_co2", adj_core),
                        "hypercap_resp_failure")
if (any(vapply(list(fit_imv_vbg, fit_niv_vbg, fit_death_vbg, fit_hcrf_vbg), is.null, logical(1)))) {
  stop("IPW VBG spline fits failed; see model_fit_diagnostics.csv.")
}

# 4. Prediction helper ---------------------------------------------------------
# helper: generate one VBG IPW spline odds-ratio curve over the observed CO2
# range while holding the reference profile fixed.
mkpred <- function(fit, data_ref, co2_var, ref_df, co2_ref) {
  co2_seq <- stats::quantile(data_ref[[co2_var]], probs = c(0.02, 0.98), na.rm = TRUE)
  grid_info <- make_co2_grid_ref(
    co2_var,
    seq(co2_seq[1], co2_seq[2], length.out = SPLINE_GRID_N),
    ref_df,
    co2_ref
  )
  predict_or_curve_from_fit(fit, grid_info$grid, grid_info$ref_idx, co2_var)
}

pred_imv_vbg   <- mkpred(fit_imv_vbg,   subset_data_vbg, "vbg_co2", x_ref_vbg, VBG_CO2_REF)
pred_niv_vbg   <- mkpred(fit_niv_vbg,   subset_data_vbg, "vbg_co2", x_ref_vbg, VBG_CO2_REF)
pred_death_vbg <- mkpred(fit_death_vbg, subset_data_vbg, "vbg_co2", x_ref_vbg, VBG_CO2_REF)
pred_hcrf_vbg  <- mkpred(fit_hcrf_vbg,  subset_data_vbg, "vbg_co2", x_ref_vbg, VBG_CO2_REF)
axis_ipw_common <- compute_or_axis_spec(
  list(pred_imv_abg, pred_niv_abg, pred_death_abg, pred_hcrf_abg,
       pred_imv_vbg, pred_niv_vbg, pred_death_vbg, pred_hcrf_vbg),
  lo_col = "LCL", hi_col = "UCL"
)
axis_ipw_common_prob <- compute_prob_axis_spec(
  list(pred_imv_abg, pred_niv_abg, pred_death_abg, pred_hcrf_abg,
       pred_imv_vbg, pred_niv_vbg, pred_death_vbg, pred_hcrf_vbg),
  lo_col = "p_LCL", hi_col = "p_UCL"
```

```r
)

# 5. Plotting (gray scheme) ----------------------------------------------------
xlab <- expression(paste("VBG CO"[2], " (mmHg)"))

# helper: render one VBG spline panel with a consistent odds-ratio axis and
# publication theme so the four outcomes can be compared side by side.
plt <- function(dat, title)
  ggplot(dat, aes(vbg_co2, OR)) +
    geom_line() +
    geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.2) +
    geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
    or_axis_scale(axis_ipw_common) +
    labs(title = title, x = xlab,
         y = paste0("Odds ratio (ref = ", VBG_CO2_REF, "; log scale)")) +
    theme_pub()

ipw_vbg_panel <- (patchwork::wrap_plots(
  plt(pred_imv_vbg,   "IMV"),
  plt(pred_niv_vbg,   "NIV"),
  plt(pred_death_vbg, "Death"),
  plt(pred_hcrf_vbg,  "Hypercapnic RF"),
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted odds ratios by VBG CO2 (ref = ",
                   VBG_CO2_REF, "; conditional on X)"),
    caption = paste0("Reference VBG CO2 = ", VBG_CO2_REF, " mmHg")
  )

print_plot_once(ipw_vbg_panel, "spline-ipw-vbg", width = 8.5, height = 6)

# helper: draw one VBG IPW predicted-probability panel using the ABG/VBG-shared
# probability axis.
plt_prob_vbg <- function(dat, title)
  ggplot(dat, aes(vbg_co2, p)) +
```

```r
    geom_line(color = "#d95f02") +
    geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), alpha = 0.2, fill = "#d95f02") +
    prob_axis_scale(axis_ipw_common_prob) +
    labs(title = title, x = xlab, y = "Predicted probability") +
    theme_pub()

ipw_vbg_prob_panel <- (patchwork::wrap_plots(
  plt_prob_vbg(pred_imv_vbg,   "IMV"),
  plt_prob_vbg(pred_niv_vbg,   "NIV"),
  plt_prob_vbg(pred_death_vbg, "Death"),
  plt_prob_vbg(pred_hcrf_vbg,  "Hypercapnic RF"),
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted predicted probabilities by VBG CO2 (ref = ",
                   VBG_CO2_REF, "; conditional on X)"),
    caption = paste0("Probabilities are conditional on X_ref; reference VBG CO2 = ", VBG_CO2_REF, " mmHg")
  )

print_plot_once(ipw_vbg_prob_panel, "spline-ipw-vbg-prob", width = 8.5, height = 6)

# ABG plots with the same axis (shared with VBG)
xlab_abg <- expression(paste("ABG CO"[2], " (mmHg)"))

# helper: redraw the ABG IPW odds-ratio panels on the same axis used for VBG so
# the overlaid comparison figures remain visually comparable.
plt_abg <- function(dat, title)
  ggplot(dat, aes(paco2, OR)) +
    geom_line() +
    geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.2) +
    geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
    or_axis_scale(axis_ipw_common) +
    labs(title = title, x = xlab_abg,
         y = paste0("Odds ratio (ref = ", ABG_CO2_REF, "; log scale)")) +
    theme_pub()
```

```r
ipw_abg_shared_panel <- (patchwork::wrap_plots(
  plt_abg(pred_imv_abg,   "IMV"),
  plt_abg(pred_niv_abg,   "NIV"),
  plt_abg(pred_death_abg, "Death"),
  plt_abg(pred_hcrf_abg,  "Hypercapnic RF"),
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted odds ratios by ABG CO2 (ref = ",
                   ABG_CO2_REF, "; conditional on X)"),
    caption = paste0("Reference PaCO2 = ", ABG_CO2_REF, " mmHg")
  )

print_plot_once(ipw_abg_shared_panel, "spline-ipw-abg-shared", width = 8.5, height = 6)

# helper: redraw the ABG IPW predicted-probability panels on the same axis used
# for VBG so the overlaid comparison figures remain visually comparable.
plt_prob_abg <- function(dat, title)
  ggplot(dat, aes(paco2, p)) +
    geom_line(color = "#1f77b4") +
    geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), alpha = 0.2, fill = "#1f77b4") +
    prob_axis_scale(axis_ipw_common_prob) +
    labs(title = title, x = xlab_abg, y = "Predicted probability") +
    theme_pub()

ipw_abg_shared_prob_panel <- (patchwork::wrap_plots(
  plt_prob_abg(pred_imv_abg,   "IMV"),
  plt_prob_abg(pred_niv_abg,   "NIV"),
  plt_prob_abg(pred_death_abg, "Death"),
  plt_prob_abg(pred_hcrf_abg,  "Hypercapnic RF"),
  ncol = 2
)) +
  plot_annotation(
    title = paste0("GBM-IPSW-adjusted predicted probabilities by ABG CO2 (ref = ",
                   ABG_CO2_REF, "; conditional on X)"),
    caption = paste0("Probabilities are conditional on X_ref; reference PaCO2 = ", ABG_CO2_REF, " mmHg")
```

```r
  )

print_plot_once(ipw_abg_shared_prob_panel, "spline-ipw-abg-shared-prob", width = 8.5, height = 6)

# ABG + VBG overlaid spline panel (non-MI IPSW), matching MI-style comparison
ipw_curve_abg <- dplyr::bind_rows(
  pred_imv_abg   |> dplyr::mutate(outcome = "IMV"),
  pred_niv_abg   |> dplyr::mutate(outcome = "NIV"),
  pred_death_abg |> dplyr::mutate(outcome = "Death (60d)"),
  pred_hcrf_abg  |> dplyr::mutate(outcome = "Hypercapnic RF")
) |>
  dplyr::mutate(group = "ABG", co2 = paco2) |>
  dplyr::select(group, outcome, co2, OR, LCL, UCL)

ipw_curve_vbg <- dplyr::bind_rows(
  pred_imv_vbg   |> dplyr::mutate(outcome = "IMV"),
  pred_niv_vbg   |> dplyr::mutate(outcome = "NIV"),
  pred_death_vbg |> dplyr::mutate(outcome = "Death (60d)"),
  pred_hcrf_vbg  |> dplyr::mutate(outcome = "Hypercapnic RF")
) |>
  dplyr::mutate(group = "VBG", co2 = vbg_co2) |>
  dplyr::select(group, outcome, co2, OR, LCL, UCL)

ipw_curve_overlay <- dplyr::bind_rows(ipw_curve_abg, ipw_curve_vbg) |>
  dplyr::mutate(
    group = factor(group, levels = c("ABG", "VBG")),
    outcome = factor(outcome, levels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF"))
  )

ipw_overlay_plot <- ggplot(ipw_curve_overlay, aes(x = co2, y = OR, color = group, fill = group)) +
  geom_line(linewidth = 0.6) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.15, color = NA) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_ipw_common) +
  facet_wrap(~ outcome, scales = "free_x") +
  labs(
```

```
    title = expression(
      paste("GBM-IPSW-adjusted spline odds ratios: ABG vs VBG CO"[2], " (non-MI)")
    ),
    x = expression(CO[2]~"(mmHg)"),
    y = paste0("Odds ratio (ref: ABG ", ABG_CO2_REF, ", VBG ", VBG_CO2_REF, "; log scale)"),
    caption = paste0("Reference CO2: ABG ", ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF, " mmHg")
  ) +
  theme_pub(base_size = 10)

print_plot_once(ipw_overlay_plot, "spline-ipw-overlay-abg-vbg", width = 8.5, height = 6.5)

ipw_curve_abg_prob <- dplyr::bind_rows(
  pred_imv_abg   |> dplyr::mutate(outcome = "IMV"),
  pred_niv_abg   |> dplyr::mutate(outcome = "NIV"),
  pred_death_abg |> dplyr::mutate(outcome = "Death (60d)"),
  pred_hcrf_abg  |> dplyr::mutate(outcome = "Hypercapnic RF")
) |>
  dplyr::mutate(group = "ABG", co2 = paco2) |>
  dplyr::select(group, outcome, co2, p, p_LCL, p_UCL)

ipw_curve_vbg_prob <- dplyr::bind_rows(
  pred_imv_vbg   |> dplyr::mutate(outcome = "IMV"),
  pred_niv_vbg   |> dplyr::mutate(outcome = "NIV"),
  pred_death_vbg |> dplyr::mutate(outcome = "Death (60d)"),
  pred_hcrf_vbg  |> dplyr::mutate(outcome = "Hypercapnic RF")
) |>
  dplyr::mutate(group = "VBG", co2 = vbg_co2) |>
  dplyr::select(group, outcome, co2, p, p_LCL, p_UCL)

ipw_curve_overlay_prob <- dplyr::bind_rows(ipw_curve_abg_prob, ipw_curve_vbg_prob) |>
  dplyr::mutate(
    group = factor(group, levels = c("ABG", "VBG")),
    outcome = factor(outcome, levels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF"))
  )

ipw_overlay_prob_plot <- ggplot(ipw_curve_overlay_prob, aes(x = co2, y = p, color = group, fill = group)) +
```

```r
  geom_line(linewidth = 0.6) +
  geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), alpha = 0.15, color = NA) +
  prob_axis_scale(axis_ipw_common_prob) +
  facet_wrap(~ outcome, scales = "free_x") +
  labs(
    title = expression(
      paste("GBM-IPSW-adjusted spline predicted probabilities: ABG vs VBG CO"[2], " (non-MI)")
    ),
    x = expression(CO[2]~"(mmHg)"),
    y = "Predicted probability",
    caption = paste0(
      "Conditional on reference profiles X_ref; reference CO2: ABG ",
      ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF, " mmHg"
    )
  ) +
  theme_pub(base_size = 10)

print_plot_once(ipw_overlay_prob_plot, "spline-ipw-overlay-abg-vbg-prob", width = 8.5, height = 6.5)
```

2.0.5 Three-level PCO2 categories (weighted; ABG, VBG)

Three groups with weights and covariate adjustment

```r
# Purpose: ipw three level pco2 all.

#   1. Ensure PCO2 categories are present
stopifnot(all(c("pco2_cat_abg", "pco2_cat_vbg") %in% names(subset_data)))

#   2. Function: weighted logistic regression & OR extraction
# helper: thin wrapper retained for the legacy GBM-weighted categorical OR
# figures while reusing the shared weighted categorical fitting helper.
run_weighted_or <- function(data, outcome, cat_var, weight_var, group_name,
                            treat_var, adj_vars) {
  fit_obj <- fit_cat3_model(
    data = data,
    outcome = outcome,
```

```
    cat_var = cat_var,
    group_name = group_name,
    adj_vars = adj_vars,
    analysis_variant = "ipw",
    weight_var = weight_var,
    treat_var = treat_var
  )

  extract_cat3_or_terms(fit_obj, outcome, cat_var, group_name, "GBM-weighted")
}

#   3. Run across outcomes & cohorts
outcomes_ipw <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")

ipw_three_level_forms <- list(
  "ABG IPW 3-level: IMV ~ CO2 category + X"      = reformulate(c("pco2_cat_abg", adj_core), response = "imv_proc"),
  "ABG IPW 3-level: NIV ~ CO2 category + X"      = reformulate(c("pco2_cat_abg", adj_core), response = "niv_proc"),
  "ABG IPW 3-level: Death60d ~ CO2 category + X" = reformulate(c("pco2_cat_abg", adj_core), response = "death_60d"),
  "ABG IPW 3-level: HCRF ~ CO2 category + X"     = reformulate(c("pco2_cat_abg", adj_core), response =
   ↪  "hypercap_resp_failure"),
  "VBG IPW 3-level: IMV ~ CO2 category + X"      = reformulate(c("pco2_cat_vbg", adj_core), response = "imv_proc"),
  "VBG IPW 3-level: NIV ~ CO2 category + X"      = reformulate(c("pco2_cat_vbg", adj_core), response = "niv_proc"),
  "VBG IPW 3-level: Death60d ~ CO2 category + X" = reformulate(c("pco2_cat_vbg", adj_core), response = "death_60d"),
  "VBG IPW 3-level: HCRF ~ CO2 category + X"     = reformulate(c("pco2_cat_vbg", adj_core), response =
   ↪  "hypercap_resp_failure")
)
register_model_diagrams(ipw_three_level_forms)

gbm_cat3_fits_abg <- setNames(lapply(
  outcomes_ipw,
  function(outcome) fit_cat3_model(
    subset_data,
    outcome,
    "pco2_cat_abg",
    "ABG",
    adj_vars = adj_core,
```

```r
    analysis_variant = "ipw",
    weight_var = "w_abg",
    treat_var = "has_abg"
  )
), outcomes_ipw)
gbm_cat3_fits_vbg <- setNames(lapply(
  outcomes_ipw,
  function(outcome) fit_cat3_model(
    subset_data,
    outcome,
    "pco2_cat_vbg",
    "VBG",
    adj_vars = adj_core,
    analysis_variant = "ipw",
    weight_var = "w_vbg",
    treat_var = "has_vbg"
  )
), outcomes_ipw)

ipw_combined_or_df <- bind_rows(
  lapply(outcomes_ipw, function(outcome)
    extract_cat3_or_terms(gbm_cat3_fits_abg[[outcome]], outcome, "pco2_cat_abg", "ABG", "GBM-weighted")),
  lapply(outcomes_ipw, function(outcome)
    extract_cat3_or_terms(gbm_cat3_fits_vbg[[outcome]], outcome, "pco2_cat_vbg", "VBG", "GBM-weighted"))
)

ipw_threelevel_results <- ipw_combined_or_df %>%
  mutate(method = "IPW adjusted")
```

```r
# Purpose: standalone Supplementary Table S3 export for GBM-weighted categorical associations.
table_s3_gbm_threelevel <- build_supplement_threelevel_table(
  ipw_threelevel_results,
  method = "GBM-weighted",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "estimate",
```

```
  lcl_col = "conf.low",
  ucl_col = "conf.high",
  context = "Supplementary Table S3"
)

write_standalone_table_pdf(
  table_s3_gbm_threelevel,
  caption = "GBM IPSW-weighted associations of pCO2 category with clinical outcomes by ABG and VBG",
  file_stub = "table_s3_gbm_threelevel",
  wide = TRUE,
  digits = 2,
  display_label = "Table S3"
)

render_table_pdf_maybe(
  table_s3_gbm_threelevel,
  caption = "GBM IPSW-weighted associations of pCO2 category with clinical outcomes by ABG and VBG",
  file_stub = "table_s3_gbm_threelevel",
  wide = TRUE,
  digits = 2,
  show = FALSE
)
```

```
# Purpose: resume GBM-weighted categorical OR assembly after Supplementary Table S3 export.

ipw_combined_or_df <- map_or_exposure(ipw_combined_or_df, "or-plot-three-level-weighted")
ipw_combined_or_df$group <- factor(ipw_combined_or_df$group, levels = c("ABG", "VBG"))

#   4. Plot weighted odds ratios
ipw_plot_df <- build_or_plot_df(ipw_combined_or_df, "or-plot-three-level-weighted",
                            expected_exposure_levels = CO2_CAT_CONTRAST_LEVELS)
ipw_axis_spec <- compute_or_axis_spec(ipw_plot_df, lo_col = "conf.low", hi_col = "conf.high",
                                      default_limits = OR_XLIM)
ipw_p_or <- plot_or_safe(
  ipw_plot_df,
  plot_name = "or-plot-three-level-weighted",
  axis_spec = ipw_axis_spec,
```

```
  title  = "GBM-IPSW-adjusted weighted odds ratios by PCO2 category (ABG, VBG)"
)
print_plot_once(ipw_p_or, "or-plot-three-level-weighted", width = 7.5, height = 4.8)
```

2.0.6 Observed outcome rates by CO2 bin (GBM IPSW, non-MI)

```
# Purpose: summarize weighted event rates in 1-mmHg bins for the non-MI IPSW analyses.
stopifnot(all(c("w_abg", "w_vbg") %in% names(subset_data)))

co2_rate_ipw_abg <- compute_co2_bin_rates(
  df = subset_data |> dplyr::filter(has_abg == 1),
  co2_var = "paco2",
  outcomes = CO2_RATE_OUTCOMES,
  outcome_labels = CO2_RATE_OUTCOME_LABELS,
  weight_var = "w_abg",
  q = c(0.02, 0.98),
  bin_width = 1,
  cohort_label = "ABG"
)
co2_rate_ipw_vbg <- compute_co2_bin_rates(
  df = subset_data |> dplyr::filter(has_vbg == 1),
  co2_var = "vbg_co2",
  outcomes = CO2_RATE_OUTCOMES,
  outcome_labels = CO2_RATE_OUTCOME_LABELS,
  weight_var = "w_vbg",
  q = c(0.02, 0.98),
  bin_width = 1,
  cohort_label = "VBG"
)

write_csv_safely(co2_rate_ipw_abg, results_path("co2_binned_rates_ipw_abg.csv"), row_names = FALSE)
write_csv_safely(co2_rate_ipw_vbg, results_path("co2_binned_rates_ipw_vbg.csv"), row_names = FALSE)

p_co2_rate_ipw_abg <- plot_co2_bin_rate_panel(
  co2_rate_ipw_abg,
  title = "Observed outcome rates by ABG CO2 bin (GBM IPSW, non-MI)",
```

```
  x_label = "ABG CO2 (mmHg)",
  vline_at = 45
)
p_co2_rate_ipw_vbg <- plot_co2_bin_rate_panel(
  co2_rate_ipw_vbg,
  title = "Observed outcome rates by VBG CO2 bin (GBM IPSW, non-MI)",
  x_label = "VBG CO2 (mmHg)",
  vline_at = 50
)

print_plot_once(p_co2_rate_ipw_abg, "co2-binned-rates-ipw-abg", width = 8.5, height = 6.0)
print_plot_once(p_co2_rate_ipw_vbg, "co2-binned-rates-ipw-vbg", width = 8.5, height = 6.0)
```

2.1 Propensity score diagnostics

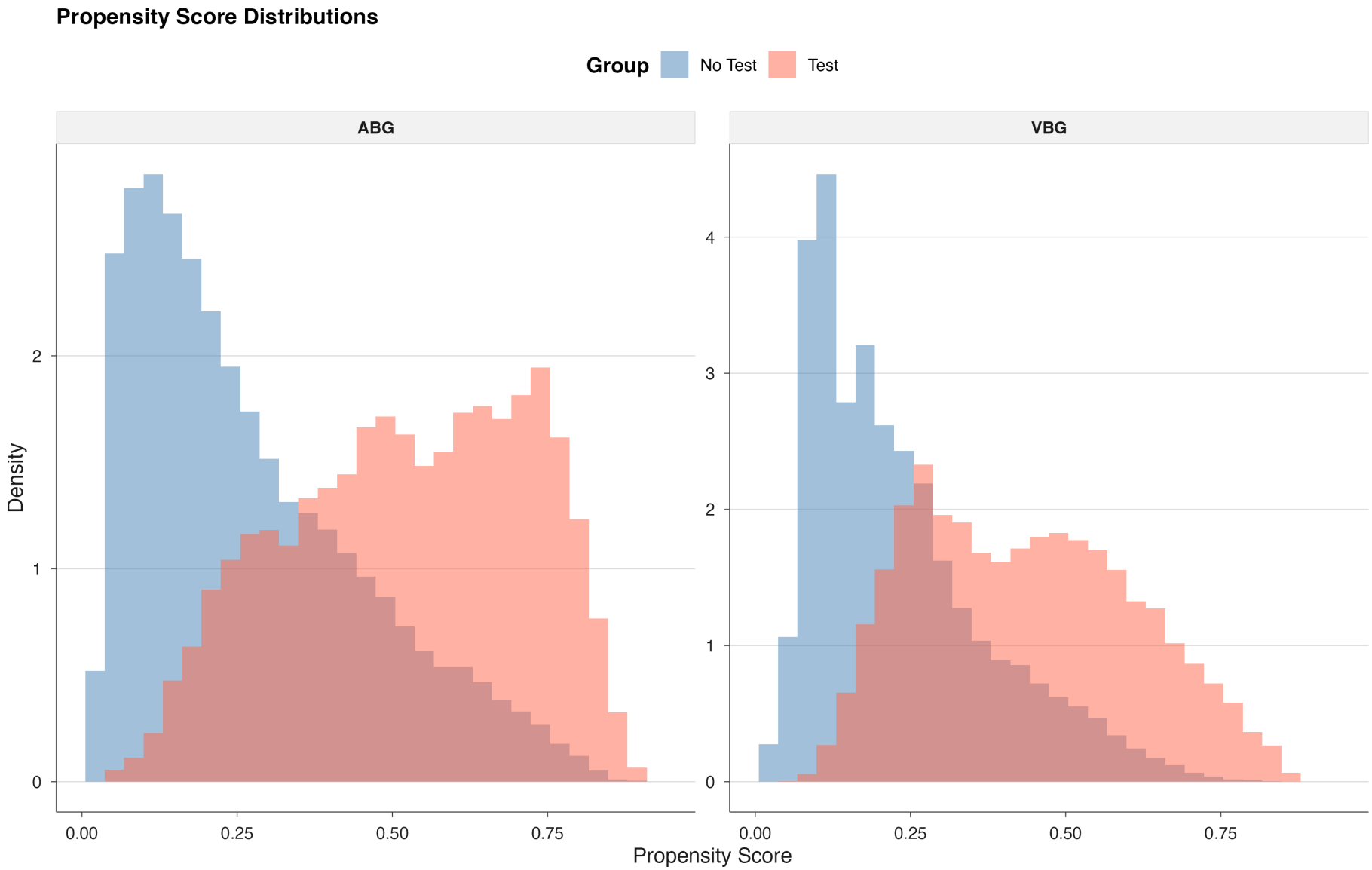
Plotting propensity scores

```
# --- Propensity score histograms (ABG / VBG) ---------------------------------
# ABG = arterial blood gas; VBG = venous blood gas


stopifnot("has_abg" %in% names(subset_data))
stopifnot("has_vbg" %in% names(subset_data))
stopifnot(all(c("ps_abg", "ps_vbg") %in% names(subset_data)))

# Build list of per-cohort PS data frames conditionally (so missing cohorts don't error)
ps_dfs_cond <- list(
  ABG = data.frame(
    ps        = subset_data$ps_abg,
    treat     = subset_data$has_abg,
    ScoreType = "ABG"
  ),
  VBG = data.frame(
    ps        = subset_data$ps_vbg,
    treat     = subset_data$has_vbg,
    ScoreType = "VBG"
```

```r
  )
)

# Bind, clean, and factorize for plotting
df_ps_cond <- bind_rows(ps_dfs_cond) %>%
  filter(!is.na(ps), !is.na(treat)) %>%
  mutate(
    treat     = factor(treat, levels = c(0, 1), labels = c("Untested", "Tested")),
    ScoreType = factor(ScoreType, levels = c("ABG", "VBG"))
  )

# Plot
p_ps_cond <- ggplot(df_ps_cond, aes(x = ps, fill = treat)) +
  geom_histogram(aes(y = after_stat(density)), alpha = 0.5,
                 position = "identity", bins = 30) +
  scale_fill_manual(values = c("Untested" = "steelblue", "Tested" = "tomato")) +
  facet_wrap(~ScoreType, scales = "free_y") +
  coord_cartesian(xlim = c(0, 1)) +
  labs(
    title = "Propensity-score overlap for gradient-boosted ABG and VBG test-ordering models",
    x     = "Propensity score",
    y     = "Density",
    fill  = "Group"
  ) +
  theme_pub(base_size = 12)
print_plot_once(p_ps_cond, "propensity-histograms-conditional", width = 8.5, height = 5)
```

```r
# Purpose: propensity histograms all.
stopifnot(all(c("ps_abg", "ps_vbg") %in% names(subset_data)))

ps_dfs_all <- list(
  ABG = data.frame(
    ps        = subset_data$ps_abg,
    treat     = subset_data$has_abg,
    ScoreType = "ABG"
  ),
  VBG = data.frame(
```

```r
    ps        = subset_data$ps_vbg,
    treat     = subset_data$has_vbg,
    ScoreType = "VBG"
  )
)

df_ps_all <- bind_rows(ps_dfs_all) %>%
  mutate(
    treat     = factor(treat, levels = c(0,1), labels = c("No Test", "Test")),
    ScoreType = factor(ScoreType, levels = c("ABG","VBG"))
  )

ggplot(df_ps_all, aes(x = ps, fill = treat)) +
  geom_histogram(aes(y = after_stat(density)), alpha = 0.5,
                 position = "identity", bins = 30) +
  scale_fill_manual(values = c("No Test" = "steelblue", "Test" = "tomato")) +
  facet_wrap(~ScoreType, scales = "free_y") +
  labs(
    title = "Propensity Score Distributions",
    x = "Propensity Score",
    y = "Density",
    fill = "Group"
  ) +
  theme_pub(base_size = 12)
```




```
# Purpose: shap top10 ipw gbm.
# This chunk fits lightweight explainability GBM models for ABG/VBG
# and reports top predictors by mean absolute SHAP value.
nonmi_shap_abg_file <- results_path("shap_top10_ipw_gbm_abg.csv")
```

```r
nonmi_shap_vbg_file <- results_path("shap_top10_ipw_gbm_vbg.csv")

if (RUN_SHAP && HAS_FASTSHAP) {
  # Build ABG/VBG design frames using the same covariates as propensity models.
  gbm_shap_df_abg <- subset_data[, c("has_abg", "has_vbg", covars_gbm), drop = FALSE]
  gbm_shap_df_abg <- normalize_types(gbm_shap_df_abg, levels_ref)
  gbm_shap_df_abg <- droplevels_all(gbm_shap_df_abg)

  gbm_shap_df_vbg <- subset_data[, c("has_abg", "has_vbg", covars_gbm), drop = FALSE]
  gbm_shap_df_vbg <- normalize_types(gbm_shap_df_vbg, levels_ref)
  gbm_shap_df_vbg <- droplevels_all(gbm_shap_df_vbg)

  # Fit separate GBM explainers for ABG and VBG testing models.
  set.seed(20251206)
  shap_fit_abg <- fit_weightit_gbm_for_shap(formula_abg, gbm_shap_df_abg)
  set.seed(30251206)
  shap_fit_vbg <- fit_weightit_gbm_for_shap(formula_vbg, gbm_shap_df_vbg)

  # Compute and rank mean |SHAP| values (top N per cohort).
  shap_top_abg <- extract_nonmi_gbm_shap_top(
    shap_fit_abg, gbm_shap_df_abg, covars_gbm, "ABG",
    nsim = SHAP_NSIM, top_n = SHAP_TOP_N
  )
  shap_top_vbg <- extract_nonmi_gbm_shap_top(
    shap_fit_vbg, gbm_shap_df_vbg, covars_gbm, "VBG",
    nsim = SHAP_NSIM, top_n = SHAP_TOP_N
  )

  write_csv_safely(shap_top_abg, nonmi_shap_abg_file, row_names = FALSE)
  write_csv_safely(shap_top_vbg, nonmi_shap_vbg_file, row_names = FALSE)

  # Draw publication figure: ABG panel on left, VBG panel on right.
  shap_top_nonmi <- dplyr::bind_rows(shap_top_abg, shap_top_vbg)
  if (nrow(shap_top_nonmi) > 0) {
    p_shap_nonmi <- plot_shap_top10_two_panel(
      shap_top_nonmi,
```

```
      "SHAP-style contribution summaries for gradient-boosted ABG and VBG test-ordering models",
      "Mean |SHAP value|"
    )
    # Save + register; display occurs after love plot for narrative flow.
    shap_file <- results_path("figs", "shap-top10-ipw-gbm-abg-vbg.png")
    save_diag_plot(p_shap_nonmi, shap_file, width = 9, height = 5, dpi = 300)
    register_plot_file("shap-top10-ipw-gbm-abg-vbg", shap_file)
  }

  rm(
    gbm_shap_df_abg, gbm_shap_df_vbg, shap_fit_abg, shap_fit_vbg,
    shap_top_abg, shap_top_vbg, shap_top_nonmi
  )
  invisible(gc())
} else {
  # Keep output schema stable even when SHAP is disabled/unavailable.
  write_csv_safely(data.frame(), nonmi_shap_abg_file, row_names = FALSE)
  write_csv_safely(data.frame(), nonmi_shap_vbg_file, row_names = FALSE)
}
```

```
# Purpose: manuscript Figure S6 love plot for GBM weighting.
# Produce a standard love-plot style balance display:
# raw |SMD| vs weighted |SMD| for ABG and VBG cohorts.
covars_balance <- intersect(covars_ps, names(subset_data))
# helper: convert one cobalt balance table into the long format used by the
# manuscript love plot.
love_data_from_bal <- function(data, treat_var, weights_vec, covars, group_label) {
  id_vars <- intersect(c("has_abg", "has_vbg"), names(data))
  bal_cols <- unique(c(id_vars, treat_var, covars))
  bal_df <- data[, bal_cols, drop = FALSE]
  weights_vec <- ifelse(is.finite(weights_vec) & !is.na(weights_vec), weights_vec, 1)
  if (nrow(bal_df) == 0L) {
    return(data.frame(group = character(), term = character(),
                      abs_smd_raw = numeric(), abs_smd_ipw = numeric(),
                      stringsAsFactors = FALSE))
  }
  treat_vals <- unique(stats::na.omit(bal_df[[treat_var]]))
```

```r
  if (length(treat_vals) < 2L) {
    return(data.frame(group = character(), term = character(),
                      abs_smd_raw = numeric(), abs_smd_ipw = numeric(),
                      stringsAsFactors = FALSE))
  }
  bal_df <- normalize_types(bal_df, levels_ref)
  bal_df <- droplevels_all(bal_df)
  bal_formula <- stats::as.formula(paste(treat_var, "~", paste(covars, collapse = " + ")))
  bal_obj <- cobalt::bal.tab(
    bal_formula,
    data = bal_df,
    weights = weights_vec,
    un = TRUE,
    method = "weighting",
    estimand = "ATE",
    quick = FALSE
  )
  bal_tbl <- as.data.frame(bal_obj$Balance, stringsAsFactors = FALSE)
  bal_tbl$term <- rownames(bal_tbl)
  if (!("Diff.Un" %in% names(bal_tbl))) bal_tbl$Diff.Un <- NA_real_
  if (!("Diff.Adj" %in% names(bal_tbl))) bal_tbl$Diff.Adj <- NA_real_
  bal_tbl |>
    dplyr::transmute(
      group = group_label,
      term = term,
      abs_smd_raw = abs(.data$Diff.Un),
      abs_smd_ipw = abs(.data$Diff.Adj)
    )
}

lov_nonmi <- dplyr::bind_rows(
  love_data_from_bal(subset_data, "has_abg", subset_data$w_abg, covars_balance, "ABG"),
  love_data_from_bal(subset_data, "has_vbg", subset_data$w_vbg, covars_balance, "VBG")
)

lov_nonmi_long <- dplyr::bind_rows(
```

```r
  lov_nonmi |>
    dplyr::transmute(group, term, sample = "Raw", abs_smd = abs_smd_raw, abs_smd_raw),
  lov_nonmi |>
    dplyr::transmute(group, term, sample = "IPW", abs_smd = abs_smd_ipw, abs_smd_raw)
)
lov_nonmi_long <- add_overall_love_rows(lov_nonmi_long)

write_csv_safely(lov_nonmi_long, results_path("loveplot_ipw_gbm_data.csv"), row_names = FALSE)
p_lov_nonmi <- loveplot_style(
  lov_nonmi_long,
  "Covariate balance after gradient-boosted propensity weighting for ABG and VBG test-ordering models"
)
print_plot_once(p_lov_nonmi, "loveplot-ipw-gbm-abg-vbg", width = 9, height = 6)
```

```r
# Purpose: manuscript Figure S8 SHAP summary for GBM weighting.
# Read precomputed top-10 SHAP values and re-plot deterministically.
shap_abg_file <- results_path("shap_top10_ipw_gbm_abg.csv")
shap_vbg_file <- results_path("shap_top10_ipw_gbm_vbg.csv")

shap_abg <- utils::read.csv(shap_abg_file, stringsAsFactors = FALSE)
shap_vbg <- utils::read.csv(shap_vbg_file, stringsAsFactors = FALSE)
shap_abg$group <- "ABG"
shap_vbg$group <- "VBG"

shap_plot_df <- dplyr::bind_rows(shap_abg, shap_vbg) |>
  dplyr::filter(is.finite(mean_abs_shap)) |>
  dplyr::group_by(group) |>
  dplyr::arrange(dplyr::desc(mean_abs_shap), .by_group = TRUE) |>
  dplyr::slice_head(n = 10L) |>
  dplyr::ungroup()

stopifnot(nrow(shap_plot_df) > 0)

p_shap <- plot_shap_top10_two_panel(
  shap_plot_df,
  "GBM IPSW test-ordering models: SHAP-style contribution summaries",
  "Mean |SHAP value|"
```

```r
)
print_plot_force(p_shap, "shap-top10-ipw-gbm-abg-vbg", width = 9, height = 5)
```

```r
# Purpose: stage2 cleanup.
append_mem_snapshot("stage2", "end", "post")
stage2_rm_required <- c(
  "ipow_abg", "ipow_vbg", "w_abg", "w_vbg_ipow",
  "fit_imv_abg", "fit_niv_abg", "fit_death_abg", "fit_hcrf_abg",
  "fit_imv_vbg", "fit_niv_vbg", "fit_death_vbg", "fit_hcrf_vbg",
  "plt", "mkpred",
  "ipw_combined_or_df", "ipw_plot_df", "ipw_p_or", "ipw_axis_spec", "outcomes_ipw",
  "df_ps_cond", "ps_dfs_cond", "p_ps_cond",
  "df_ps_all", "ps_dfs_all"
)
stage2_rm_optional <- c(
  "covars_balance", "lov_abg", "lov_vbg", "lov_nonmi", "lov_nonmi_long", "p_lov_nonmi",
  "nonmi_shap_abg_file", "nonmi_shap_vbg_file", "p_shap_nonmi",
  "axis_unw_prob", "axis_abg_ipw_trim_prob", "axis_ipw_common_prob",
  "plot_prob_abg", "plot_prob_vbg", "plt_prob", "plt_prob_abg", "plt_prob_vbg",
  "unw_abg_prob_panel", "unw_vbg_prob_panel",
  "ipw_abg_prob_panel", "ipw_vbg_prob_panel", "ipw_abg_shared_prob_panel",
  "ipw_curve_abg_prob", "ipw_curve_vbg_prob", "ipw_overlay_prob_plot"
)
missing_stage2 <- setdiff(stage2_rm_required, ls())
stopifnot(length(missing_stage2) == 0)
rm(list = c(stage2_rm_required, intersect(stage2_rm_optional, ls())))
invisible(gc())
append_mem_snapshot("stage2", "cleanup", "post")
```

3 Multiple Imputation (and logistic IPSW)

added 12/6/2025

```r
# Packages are centrally loaded in setup-packages.
stopifnot(exists("ensure_packages_loaded"))
ensure_packages_loaded(c(
```

```r
  "mice", "miceadds", "naniar", "visdat", "skimr",
  "WeightIt", "gbm", "survey", "cobalt", "broom",
  "dplyr", "ggplot2", "mitools", "parallel",
  "future", "future.apply", "progressr"
))

mi_mids_file    <- results_path("mi_abg_vbg_mids.rds")
mi_logistic_ps_abg_file   <- results_path("mi_logistic_ps_abg_list.rds")
mi_logistic_ps_vbg_file   <- results_path("mi_logistic_ps_vbg_list.rds")
mi_pooled_file  <- results_path("mi_pooled_results.rds")

# Use sequential futures to avoid PSOCK cluster startup failures during render
future::plan(sequential)

# choose a handler, but DO NOT make it global inside a knitted document
progressr::handlers(progressr::handler_rstudio)   # or handler_txtprogressbar
options(future.rng.onMisuse = "error")            # safer RNG with futures

set.seed(20251206)

# ensure a writable figure dir + stable device on macOS
fs::dir_create(fig_dir, recurse = TRUE)
knitr::opts_chunk$set(fig.path = fig_path, dev = "png", dpi = 300)
options(bitmapType = "cairo")  # prevents device issues on macOS
```

```r
# Purpose: mi settings.
M_IMP    <- if (IS_PILOT) 20L else 80L
MAXIT_MI <- if (IS_PILOT) 5L else 20L
MI_SEED  <- 20251206
DEBUG_SPLINE <- FALSE
M_IMP_TARGET <- if (IS_PILOT) 20L else 80L
M_IMP_MIN <- if (IS_PILOT) 20L else 50L
M_IMP_STEP <- 10
MCERR_RATIO_TARGET <- 0.10
ALLOW_M_IMP_EARLY_STOP <- FALSE

# MI propensity model (MI-only)
```

```r
MI_PS_METHOD    <- "glm_rcs4"
MI_PS_SPLINE_K  <- 4L
MI_GLM_MAXIT    <- 25L

MINCOR_QUICKPRED <- 0.05
MINPUC_QUICKPRED <- 0.25
MAX_PRED_PER_VAR <- 40L
MAX_MM_COLS <- 300L
MAX_LEVELS_PRED <- 100L
COR_SAMPLE_N <- 50000L
MI_MAX_BYTES <- 8e9

stopifnot(exists("MI_RAM_GB"), exists("MI_BATCH_START"))
stopifnot(is.finite(MI_RAM_GB), MI_RAM_GB > 0, MI_BATCH_START >= 1L)
MI_BATCH_MIN <- 1L
MI_BATCH_SEED_STRIDE <- 100000L
MI_GC_EVERY_BATCH <- TRUE
MI_PREEMPTIVE_BATCH_REDUCE <- TRUE
MI_VCELLS_FRAC_THRESHOLD <- 0.80
MI_SMOKE_TEST <- TRUE
MI_DEBUG_PRINTFLAG <- FALSE
MI_MEMORY_HYGIENE <- TRUE
FORCE_MI_BATCHED <- FALSE

stopifnot(exists("RUN_MODE"), exists("FULL_RUN"))

M_IMP <- M_IMP_TARGET
M_IMP_MIN <- max(M_IMP_MIN, if (IS_PILOT) 20L else 50L)
if (isTRUE(DEBUG_SPLINE)) {
  stop("DEBUG_SPLINE must be FALSE; pilot and full runs must use the same model workflow.")
}
stopifnot(M_IMP >= M_IMP_MIN, M_IMP <= 100)
stopifnot(MI_PS_METHOD %in% c("glm_rcs4"))
```

```r
# Key covariates + outcomes used in MI (plus report-only BNP/Spo2)
extra_miss_vars <- intersect(c("bnp", "spo2"), names(subset_data_raw))
miss_vars <- unique(c(
```

```r
  covars_ps,
  extra_miss_vars,
  "paco2", "vbg_co2", "vbg_o2sat",
  "has_abg", "has_vbg",
  "imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"
))
miss_vars <- intersect(miss_vars, names(subset_data_raw))

miss_tbl <- subset_data_raw |>
  dplyr::select(dplyr::all_of(miss_vars)) |>
  dplyr::summarise(dplyr::across(dplyr::everything(), ~ sum(is.na(.)))) |>
  tidyr::pivot_longer(dplyr::everything(), names_to = "variable", values_to = "n_missing") |>
  dplyr::mutate(
    total_n     = nrow(subset_data_raw),
    pct_missing = 100 * n_missing / total_n
  ) |>
  dplyr::arrange(dplyr::desc(pct_missing))

miss_tbl_disp <- miss_tbl |>
  dplyr::mutate(
    n_missing   = scales::comma(n_missing),
    pct_missing = sprintf("%.1f%%", pct_missing)
  ) |>
  dplyr::select(variable, n_missing, pct_missing)

# Keep the full table in artifacts; suppress this low-value, very long display in PDF.
write_csv_safely(miss_tbl, results_path("missingness_preimputation_full.csv"), row_names = FALSE)
```

```r
# MI propensity model helpers (glm with RCS on continuous covariates)
ps_is_continuous <- function(v, df) {
  x <- df[[v]]
  if (!is.numeric(x)) return(FALSE)
  x <- x[is.finite(x)]
  if (length(x) < 10) return(FALSE)
  if (length(unique(x)) < 10) return(FALSE)
  TRUE
}
```

```r
# helper: build the MI logistic propensity-score formula with spline terms for
# continuous predictors and plain terms for categorical predictors.
build_mi_ps_formula <- function(treat, covars, df, k = 4L, basis = "rcs") {
  cont <- covars[vapply(covars, ps_is_continuous, logical(1), df = df)]
  catv <- setdiff(covars, cont)
  term_cont <- character()
  if (length(cont)) {
    term_cont <- if (basis == "rcs") {
      paste0("rms::rcs(`", cont, "`, ", k, ")")
    } else {
      paste0("splines::ns(`", cont, "`, ", k, ")")
    }
  }
  term_cat <- if (length(catv)) paste0("`", catv, "`") else character()
  rhs <- c(term_cont, term_cat)
  if (!length(rhs)) stop("No covariates available for MI PS model.")
  stats::as.formula(paste0("`", treat, "` ~ ", paste(rhs, collapse = " + ")))
}

# helper: fit one MI logistic propensity model and return both propensities and
# enough metadata to audit convergence and pooling inputs.
fit_mi_ps_glm <- function(df, treat, covars, k = 4L, maxit = 25L, context = NULL) {
  # Primary MI weighting uses logistic propensity models with flexible spline
  # terms for continuous predictors. This keeps the PS model transparent and
  # straightforward to pool across imputations.
  stopifnot(all(c(treat, covars) %in% names(df)))
  df <- droplevels_all(df)
  basis <- if (requireNamespace("rms", quietly = TRUE)) "rcs" else "ns"
  if (is.null(context)) {
    context <- make_context(
      stage = "MI", component = "mi_ps_glm",
      analysis_variant = "weighted_imputed",
      model_type = "ps",
      group = NA_character_,
      outcome = NA_character_,
```

```r
    imputation = NA_integer_,
    batch = NA_integer_
  )
}

fit_once <- function(basis_type) {
  # Prefer rms::rcs() when available for continuity with prior analyses, but
  # fall back to natural splines if the rcs path is unavailable or unstable.
  if (basis_type == "rcs") {
    dd <- rms::datadist(df)
    old_opt <- options(datadist = ".__dd_ps__")
    assign(".__dd_ps__", dd, envir = .GlobalEnv)
    on.exit({
      options(old_opt)
      rm(list = ".__dd_ps__", envir = .GlobalEnv)
    }, add = TRUE)
  }
  form <- build_mi_ps_formula(treat, covars, df, k = k, basis = basis_type)
  cap <- capture_warnings(
    tryCatch(
      stats::glm(form, data = df, family = stats::binomial(),
                 control = stats::glm.control(maxit = maxit),
                 model = FALSE, x = FALSE, y = FALSE),
      error = function(e) e
    ),
    context = context
  )
  append_warnings(cap$warnings)
  list(fit = cap$value, formula = form, basis = basis_type)
}

res <- fit_once(basis)
if (inherits(res$fit, "error") && basis == "rcs") {
  res <- fit_once("ns")
}
if (inherits(res$fit, "error")) {
```

```r
    return(list(error = conditionMessage(res$fit), method = paste0("glm_", res$basis)))
  }

  ps <- as.numeric(res$fit$fitted.values)
  ps <- pmin(pmax(ps, 1e-8), 1 - 1e-8)
  vc <- tryCatch(stats::vcov(res$fit), error = function(e) NULL)
  list(
    ps = ps,
    fit_ok = TRUE,
    converged = if (!is.null(res$fit$converged)) isTRUE(res$fit$converged) else NA,
    formula = paste(deparse(res$formula), collapse = " "),
    n = nrow(df),
    p = length(res$fit$coefficients),
    method = paste0("glm_", res$basis),
    basis = res$basis,
    coef = stats::coef(res$fit),
    vcov_diag = if (is.null(vc)) NULL else diag(vc)
  )
}
```

3.0.1 Missingness structure and drivers

```r
# Purpose: mi missing structure.

# Use raw data for missingness rates; normalized data for model-based drivers
miss_data <- subset_data_raw
model_data <- subset_data

# Focus on key variables (same as MI set) for consistency
extra_miss_vars <- intersect(c("bnp", "spo2"), names(miss_data))
miss_vars <- unique(c(
  covars_ps,
  extra_miss_vars,
  "paco2", "vbg_co2", "vbg_o2sat",
  "has_abg", "has_vbg",
  "imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"
```

```r
))
miss_vars <- intersect(miss_vars, names(miss_data))
stopifnot(length(miss_vars) > 0)

# Strata to compare missingness across
strata_vars <- c("has_abg", "has_vbg",
                 "imv_proc", "death_60d", "encounter_type", "location")
strata_vars <- intersect(strata_vars, names(miss_data))

# Overall missingness %
miss_overall <- miss_data |>
  summarise(across(all_of(miss_vars), ~ mean(is.na(.)) * 100)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing_overall")

# Missingness % on model_data (used for driver models)
miss_overall_model <- model_data |>
  summarise(across(all_of(intersect(miss_vars, names(model_data))), ~ mean(is.na(.)) * 100)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing_overall_model")

# Missingness by strata (percent within each level)
miss_by_strata <- purrr::map_dfr(strata_vars, function(sv) {
  # drop the current grouping variable from targets
  vars_here <- setdiff(intersect(miss_vars, names(miss_data)), sv)
  stopifnot(length(vars_here) > 0L)
  stopifnot(sv %in% names(miss_data))

  miss_data |>
    group_by(.data[[sv]]) |>
    summarise(across(all_of(vars_here), ~ mean(is.na(.)) * 100), .groups = "drop") |>
    pivot_longer(-all_of(sv), names_to = "variable", values_to = "pct_missing") |>
    rename(level = !!sym(sv)) |>
    mutate(
      stratum = sv,
      level   = as.character(level)
    )
})
```

```r
# Combine overall + strata (save full table; display full table in PDF)
miss_panel_full <- miss_by_strata |>
  left_join(miss_overall, by = "variable") |>
  arrange(desc(pct_missing_overall), stratum, level)

miss_panel_file <- results_path("missingness-by-strata.csv")
write_csv_safely(miss_panel_full, miss_panel_file, row_names = FALSE)

render_table_pdf_maybe(
  miss_panel_full,
  caption = "Missingness by key strata (pre-imputation).",
  file_stub = "missingness_by_strata",
  digits = 1,
  show = FALSE
)
```

```r
# Purpose: float barrier missingness.
float_barrier("section")
```

```r
# --- Drivers of missingness (logit I(NA) on observed covariates) --------------
# Candidate predictors (observed covariates only)
driver_covars <- intersect(
  c("age_at_encounter", "sex", "encounter_type", "location", "curr_bmi",
    "has_abg", "has_vbg", "imv_proc", "death_60d"),
  names(model_data)
)

# Fit models for variables with any missingness
vars_to_model <- miss_overall_model |>
  filter(pct_missing_overall_model > 0) |>
  pull(variable)
vars_to_model <- intersect(vars_to_model, names(model_data))

skip_log <- list()
```

```r
model_results <- purrr::map_dfr(vars_to_model, function(v) {
  df <- model_data |>
    select(all_of(c(v, driver_covars))) |>
    mutate(miss = as.integer(is.na(.data[[v]])))

  # use only rows with observed predictors and varying miss indicator
  df <- df[stats::complete.cases(df[driver_covars]), , drop = FALSE]
  if (nrow(df) == 0L) {
    skip_log[[length(skip_log) + 1L]] <<- data.frame(
      variable = v,
      reason = "no_complete_cases",
      stringsAsFactors = FALSE
    )
    return(tibble::tibble())
  }
  if (dplyr::n_distinct(df$miss) < 2L) {
    skip_log[[length(skip_log) + 1L]] <<- data.frame(
      variable = v,
      reason = "miss_indicator_constant",
      stringsAsFactors = FALSE
    )
    return(tibble::tibble())
  }

  fml <- as.formula(paste0("miss ~ ", paste(driver_covars, collapse = " + ")))
  if (length(driver_covars) > 0L) {
    diag_outcome <- paste0("missing_", make.names(v))
    diag_terms <- paste0("`", driver_covars, "`", collapse = " + ")
    diag_fml <- stats::as.formula(paste0(diag_outcome, " ~ ", diag_terms))
    register_model_diagram(paste("Missingness model:", v), diag_fml, width = 10, height = 6)
  }
  fit <- tryCatch(
    suppressWarnings(glm(fml, data = df, family = binomial())),
    error = function(e) e
  )
  if (inherits(fit, "error")) {
```

```r
    stop("Missingness driver: glm failed for variable ", v)
  }

  broom::tidy(fit, conf.int = FALSE, exponentiate = FALSE) |>
    filter(term != "(Intercept)") |>
    mutate(
      OR  = exp(estimate),
      LCL = exp(estimate - 1.96 * std.error),
      UCL = exp(estimate + 1.96 * std.error)
    ) |>
    transmute(
      variable = v,
      term,
      OR,
      LCL,
      UCL,
      p.value
    )
})

skip_tbl <- dplyr::bind_rows(skip_log)
write_csv_safely(skip_tbl, results_path("missingness_driver_skips.csv"), row_names = FALSE)

if (nrow(model_results) > 0) {
  model_results_file <- results_path("missingness-drivers.csv")
  model_results_out <- model_results |>
    dplyr::mutate(run_id = diag_run_id)
  write_csv_safely(model_results_out, model_results_file, row_names = FALSE)

  model_results_disp <- model_results_out |>
    dplyr::select(-run_id) |>
    arrange(p.value) |>
    mutate(
      OR  = round(OR, 2),
      LCL = round(LCL, 2),
      UCL = round(UCL, 2),
```

```
      p.value = signif(p.value, 3)
    )

  render_table_pdf_maybe(
    model_results_disp,
    caption = "Predictors of missingness (logit OR).",
    file_stub = "missingness-drivers",
    digits = 2,
    show = FALSE
  )
} else {
  model_results_stub <- data.frame(
    variable = character(),
    term = character(),
    OR = numeric(),
    LCL = numeric(),
    UCL = numeric(),
    p.value = numeric(),
    stringsAsFactors = FALSE
  )
  write_csv_safely(model_results_stub, results_path("missingness-drivers.csv"), row_names = FALSE)
  message("No modelable missingness signals (all complete or no variation).")
}
```

3.0.2 Monte Carlo error check after MI

3.1 Pre-imputation data prep (consistent types & predictors)

Why: MI models need coherent types; using exactly the same covariates as the propensity score models avoids model drift.

```
# Types are normalized in the schema-normalize block.
```

```
# Quick invariant checks after type normalization (fail fast before MI/weighting)
req_outcomes <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
missing_out  <- setdiff(req_outcomes, names(subset_data))
if (length(missing_out)) stop("Missing outcome columns: ", paste(missing_out, collapse = ", "))
```

```r
stopifnot(
  is.factor(subset_data$sex),
  nlevels(droplevels(subset_data$sex)) >= 2,
  is.factor(subset_data$race_ethnicity),
  nlevels(droplevels(subset_data$race_ethnicity)) >= 2,
  is.factor(subset_data$encounter_type),
  nlevels(droplevels(subset_data$encounter_type)) == 2,
  is.numeric(subset_data$has_abg),
  is.numeric(subset_data$has_vbg)
)

for (col in req_outcomes) {
  vals <- subset_data[[col]]
  if (!is.numeric(vals)) stop(col, " is not numeric")
  bad <- !is.na(vals) & !vals %in% c(0, 1)
  if (any(bad)) stop(col, " has non-binary values")
}
```

3.2 Imputation model specification (MICE)

3.2.1 Predictor matrix & methods. Run MICE (moderate settings for scale)

```r
# --- variables for propensity score model (kept identical to main analysis) ---
# ---- MICE setup: include PaCO2/VBG CO2 as predictors but do not impute ----

# --- add analysis targets and CO2 measures explicitly -------------------------
mi_vars <- setdiff(unique(c(
  covars_ps,
  "has_abg","has_vbg",                                  # treatments (NOT imputed)
  "imv_proc","niv_proc","death_60d","hypercap_resp_failure",  # outcomes (NOT imputed)
  co2_vars
)), drop_vars_ultra_missing)

mi_df <- subset_data[, mi_vars, drop = FALSE]
mi_df <- normalize_types(mi_df, levels_ref)
```

```r
mi_df_size <- utils::object.size(mi_df)
message("MI data size (bytes): ", format(mi_df_size, units = "auto"))

# Rough memory preflight based on total missing cells (imputed values only).
miss_counts <- vapply(mi_df, function(x) sum(is.na(x)), numeric(1))
miss_total <- sum(miss_counts)
if (is.finite(miss_total) && miss_total > 0) {
  est_bytes <- miss_total * M_IMP * 8
  message("MI imputation storage estimate: ", format(structure(est_bytes, class = "object_size"), units = "auto"))
  if (est_bytes > MI_MAX_BYTES) {
    m_max <- floor(MI_MAX_BYTES / (miss_total * 8))
    m_target <- max(50L, m_max)
    if (m_target < 50L) {
      stop("Estimated MI storage exceeds memory (m=", M_IMP,
           ", estimated=", format(structure(est_bytes, class = "object_size"), units = "auto"),
           "). Reduce missingness or MI scope, or increase available memory.")
    }
    if (m_target < M_IMP) {
      message("Reducing M_IMP from ", M_IMP, " to ", m_target,
              " to stay within MI_MAX_BYTES.")
      M_IMP <- m_target
    }
  }
}

# Make binary comorbids factors so "logreg" is used (and stays binary)
bin_covars <- c("copd","asthma","osa","chf","acute_nmd","phtn","ckd","dm")
missing_bin <- setdiff(bin_covars, names(mi_df))
stopifnot(length(missing_bin) == 0)
mi_df[bin_covars] <- lapply(mi_df[bin_covars], function(z) {
  if (is.factor(z)) return(droplevels(z))
  zz <- suppressWarnings(as.integer(z))
  factor(zz, levels = c(0L,1L), labels = c("0","1"))
})
```

```r
# For MICE: convert any remaining characters → factors
mi_df <- dplyr::mutate(mi_df, across(where(is.character), ~ factor(.x)))

# Guardrail: high-cardinality factors can blow up MICE model matrices.
# Exclude them from predictorMatrix and (if missing) make "Missing" explicit.
high_card <- names(which(vapply(mi_df, function(x) is.factor(x) && nlevels(x) > MAX_LEVELS_PRED, logical(1))))
if (length(high_card)) {
  message("MICE: high-cardinality factors detected (nlevels > ", MAX_LEVELS_PRED, "): ",
          paste(high_card, collapse = ", "))
  for (v in high_card) {
    if (any(is.na(mi_df[[v]]))) {
      lv <- levels(mi_df[[v]])
      tmp <- as.character(mi_df[[v]])
      tmp[is.na(tmp)] <- "Missing"
      mi_df[[v]] <- factor(tmp, levels = unique(c(lv, "Missing")))
      if (!is.null(levels_ref) && !is.null(levels_ref[[v]]) &&
          !"Missing" %in% levels_ref[[v]]) {
        levels_ref[[v]] <- c(levels_ref[[v]], "Missing")
      }
    }
  }
}

# --- methods & predictor matrix aligned to *mi_df* ----------------------------
meth <- mice::make.method(mi_df)

is_fac      <- vapply(mi_df, is.factor,  logical(1))
is_num      <- vapply(mi_df, is.numeric, logical(1))
is_bin_fac  <- vapply(mi_df, function(x) is.factor(x)  && nlevels(x) == 2, logical(1))
is_multicat <- vapply(mi_df, function(x) is.factor(x)  && nlevels(x) >  2, logical(1))

# robust defaults
meth[is_num]      <- "pmm"      # numerics: predictive mean matching
meth[is_multicat] <- "polyreg"  # unordered multicategory
meth[is_bin_fac]  <- "logreg"   # binary factors: logistic regression
```

```r
# never impute treatments, outcomes, or CO2 exposures
no_imp <- c("has_abg","has_vbg","imv_proc","niv_proc","death_60d","hypercap_resp_failure",
            "paco2","vbg_co2")
if (length(high_card)) no_imp <- unique(c(no_imp, high_card))
meth[intersect(names(meth), no_imp)] <- ""

# predictor matrix; force has_abg/has_vbg as predictors, but do not impute no_imp
pred <- mice::quickpred(mi_df, mincor = MINCOR_QUICKPRED, minpuc = MINPUC_QUICKPRED)
for (nm in intersect(c("has_abg", "has_vbg"), colnames(pred))) {
  pred[, nm] <- 1
}
pred[intersect(rownames(pred), no_imp), ] <- 0

if (length(high_card)) {
  pred[, intersect(high_card, colnames(pred))] <- 0
  pred[intersect(high_card, rownames(pred)), ] <- 0
}

# Ensure dropped covariates do not appear as predictors
drop_covars <- intersect(drop_vars_ultra_missing, colnames(pred))
if (length(drop_covars)) {
  pred[, drop_covars] <- 0
}

# Non-imputed variables with missingness should NOT be predictors
no_imp_with_missing <- intersect(no_imp, names(mi_df))
no_imp_with_missing <- no_imp_with_missing[
  vapply(mi_df[no_imp_with_missing], function(x) any(is.na(x)), logical(1))
]
no_imp_with_missing <- setdiff(no_imp_with_missing, c("has_abg", "has_vbg"))

pred_cols_check <- intersect(c("paco2", "vbg_co2"), colnames(pred))
if (length(pred_cols_check)) {
  message(
    "MICE: predictor column sums (pre-exclude) for ",
    paste(pred_cols_check, collapse = ", "),
```

```
    ": ",
    paste(colSums(pred[, pred_cols_check, drop = FALSE]), collapse = ", ")
  )
}
if (length(no_imp_with_missing)) {
  pred[, intersect(no_imp_with_missing, colnames(pred))] <- 0
  message("MICE: excluded non-imputed missing predictors: ",
          paste(no_imp_with_missing, collapse = ", "))
}
if (length(pred_cols_check)) {
  message(
    "MICE: predictor column sums (post-exclude) for ",
    paste(pred_cols_check, collapse = ", "),
    ": ",
    paste(colSums(pred[, pred_cols_check, drop = FALSE]), collapse = ", ")
  )
}

# Ensure key covariates with missingness have predictors (avoid zero-row pred)
core_preds <- intersect(
  c("age_at_encounter", "sex", "race_ethnicity", "location", "encounter_type",
    "has_abg", "has_vbg", "imv_proc", "niv_proc", "death_60d",
    "hypercap_resp_failure", "paco2", "vbg_co2"),
  colnames(pred)
)
core_preds <- setdiff(core_preds, no_imp_with_missing)
vars_need_pred <- intersect(covars_ps, rownames(pred))
vars_need_pred <- setdiff(vars_need_pred, no_imp)
vars_need_pred <- vars_need_pred[vapply(mi_df[vars_need_pred], function(x) any(is.na(x)), logical(1))]
zero_pred <- vars_need_pred[rowSums(pred[vars_need_pred, , drop = FALSE] != 0, na.rm = TRUE) == 0]
if (length(zero_pred)) {
  message("MICE predictor rows empty for: ", paste(zero_pred, collapse = ", "),
          ". Adding core predictors.")
  pred[zero_pred, core_preds] <- 1
}
pred[intersect(rownames(pred), no_imp), ] <- 0
```

```r
pred[c(zero_pred), c(zero_pred)] <- 0

# --- Preflight and cap predictor rows to avoid huge model matrices -------------

n_mm_cols <- function(pred_row, df) {
  preds <- names(which(pred_row != 0))
  cols <- 1L
  for (p in preds) {
    x <- df[[p]]
    if (is.factor(x)) {
      lv <- nlevels(x)
      cols <- cols + max(1L, lv - 1L)
    } else {
      cols <- cols + 1L
    }
  }
  cols
}

# helper: summarize the effective width of each MICE predictor row before and
# after capping so memory pressure is visible in exported diagnostics.
pred_width_preflight <- function(pred, df, meth) {
  vars <- names(meth)[meth != ""]
  vars <- vars[vapply(df[vars], function(x) any(is.na(x)), logical(1))]
  rows <- lapply(vars, function(v) {
    pred_row <- pred[v, ]
    n_pred <- sum(pred_row != 0, na.rm = TRUE)
    mm_cols <- n_mm_cols(pred_row, df)
    x <- df[[v]]
    nlevels_v <- if (is.factor(x)) nlevels(x) else NA_integer_
    miss_n <- sum(is.na(x))
    data.frame(
      variable = v,
      method = meth[[v]],
      n_pred = n_pred,
      mm_cols = mm_cols,
```

```
      nlevels_v = nlevels_v,
      miss_n = miss_n,
      stringsAsFactors = FALSE
    )
  })
  if (!length(rows)) {
    return(data.frame(variable = character(), method = character(), n_pred = integer(),
                      mm_cols = integer(), nlevels_v = integer(), miss_n = integer()))
  }
  dplyr::bind_rows(rows)
}

preflight_pre <- pred_width_preflight(pred, mi_df, meth) |>
  dplyr::mutate(stage = "pre")

core_preds <- intersect(
  c("age_at_encounter", "sex", "race_ethnicity", "location", "encounter_type",
    "has_abg", "has_vbg", "imv_proc", "niv_proc", "death_60d",
    "hypercap_resp_failure", "paco2", "vbg_co2"),
  colnames(pred)
)
core_preds <- setdiff(core_preds, no_imp_with_missing)

set.seed(MI_SEED + 100)
idx <- sample.int(nrow(mi_df), min(COR_SAMPLE_N, nrow(mi_df)))

vars_cap <- names(meth)[meth != ""]
vars_cap <- vars_cap[vapply(mi_df[vars_cap], function(x) any(is.na(x)), logical(1))]

# helper: count the number of effective dummy-expanded levels a predictor could
# contribute to a MICE model matrix.
pred_nlevels <- function(x) {
  if (is.factor(x)) nlevels(x) else 1L
}

for (v in vars_cap) {
```

```r
cand <- names(which(pred[v, ] != 0))
if (!length(cand)) next
keep <- intersect(core_preds, cand)
rem  <- setdiff(cand, keep)

if (length(rem)) {
  y_raw <- mi_df[[v]][idx]
  y <- if (is.factor(y_raw)) as.integer(y_raw) else suppressWarnings(as.numeric(y_raw))
  scores <- vapply(rem, function(r) {
    x_raw <- mi_df[[r]][idx]
    x <- if (is.factor(x_raw)) as.integer(x_raw) else suppressWarnings(as.numeric(x_raw))
    suppressWarnings(abs(stats::cor(y, x, use = "pairwise.complete.obs")))
  }, numeric(1))
  ord <- order(is.na(scores), -scores)
  rem_keep <- rem[ord]
  rem_keep <- rem_keep[seq_len(min(length(rem_keep), max(0L, MAX_PRED_PER_VAR - length(keep))))]
} else {
  rem_keep <- character()
}

keep_all <- unique(c(keep, rem_keep))
pred[v, ] <- 0
pred[v, keep_all] <- 1
pred[v, v] <- 0

mm_cols <- n_mm_cols(pred[v, ], mi_df)
if (mm_cols > MAX_MM_COLS) {
  drop_pool <- setdiff(keep_all, keep)
  if (!length(drop_pool)) drop_pool <- keep
  drop_order <- drop_pool[order(vapply(drop_pool, function(p) pred_nlevels(mi_df[[p]]), integer(1)),
                                decreasing = TRUE)]
  for (p in drop_order) {
    if (p %in% keep && length(keep) == 1) break
    pred[v, p] <- 0
    mm_cols <- n_mm_cols(pred[v, ], mi_df)
    if (mm_cols <= MAX_MM_COLS) break
```

```r
    }
  }
}

preflight_post <- pred_width_preflight(pred, mi_df, meth) |>
  dplyr::mutate(stage = "post")
preflight_all <- dplyr::bind_rows(preflight_pre, preflight_post)
write_csv_safely(preflight_all, results_path("mice_pred_width_preflight.csv"), row_names = FALSE)

# MI integrity: treatments/outcomes excluded; binaries use logreg
stopifnot(all(meth[no_imp] == ""))
stopifnot(all(rowSums(pred[intersect(rownames(pred), no_imp), , drop = FALSE]) == 0))
bin_fac <- names(which(vapply(mi_df, function(x) is.factor(x) && nlevels(x) == 2, logical(1))))
bin_fac <- setdiff(bin_fac, no_imp)
stopifnot(all(meth[bin_fac] == "logreg"))

# integrity checks
stopifnot(
  ncol(pred) == ncol(mi_df),
  nrow(pred) == ncol(mi_df),
  length(meth) == ncol(mi_df),
  identical(names(meth), colnames(mi_df))
)

# --- run MICE ------------------------------------------------------------
mi_df_run <- mi_df
M_IMP_RUN <- M_IMP
MAXIT_RUN <- MAXIT_MI
FORCE_MI_BATCHED <- nrow(mi_df_run) > MI_BATCH_THRESHOLD

if (!requireNamespace("digest", quietly = TRUE)) {
  stop("Package 'digest' is required for MI checkpoint signatures.")
}

# helper: hash the MI input schema so cached/imputed objects can be checked
# against the exact data structure used to create them.
```

```r
make_mi_signature <- function(df) {
  classes <- vapply(df, function(x) class(x)[1], character(1))
  na_counts <- vapply(df, function(x) sum(is.na(x)), integer(1))
  nlevels <- vapply(df, function(x) if (is.factor(x)) nlevels(x) else NA_integer_, integer(1))
  lvl_hash <- vapply(df, function(x) {
    if (!is.factor(x)) return(NA_character_)
    digest::digest(levels(x), algo = "xxhash64")
  }, character(1))
  col_sig <- data.frame(
    name = names(df),
    class = classes,
    na = na_counts,
    nlevels = nlevels,
    lvl_hash = lvl_hash,
    stringsAsFactors = FALSE
  )
  hash <- digest::digest(
    list(dim = dim(df), col_sig = col_sig),
    algo = "xxhash64"
  )
  list(
    hash = hash,
    nrow = nrow(df),
    ncol = ncol(df),
    run_mode = RUN_MODE,
    pilot_frac = PILOT_FRAC,
    mi_pilot_mode = RUN_MODE,
    sample_seed = SAMPLE_SEED,
    mi_seed = MI_SEED,
    col_sig = col_sig
  )
}

mi_smoke_log_path <- results_path("mice_smoketest.log")
# helper: append one deterministic plain-text log for the MICE smoke test and
# preflight memory checks.
```

```r
write_smoke_log <- function(lines) {
  write_diag_lines(lines, mi_smoke_log_path)
}

mem_max <- tryCatch(mem.maxVSize(), error = function(e) NA_real_)
mem_env <- Sys.getenv("R_MAX_VSIZE", unset = NA_character_)
gc_pre <- utils::capture.output(gc())
write_smoke_log(c(
  paste0("MI smoke test log: ", Sys.time()),
  paste0("mem.maxVSize: ", ifelse(is.na(mem_max), "NA", format(mem_max, scientific = FALSE))),
  paste0("R_MAX_VSIZE env: ", ifelse(nchar(mem_env), mem_env, "NA")),
  paste0("mi_df size: ", format(utils::object.size(mi_df), units = "auto")),
  paste0("mi_df_run size: ", format(utils::object.size(mi_df_run), units = "auto")),
  "gc() pre:",
  gc_pre
))
if (RUN_MODE == "full" && is.finite(mem_max) && mem_max < 2.2e10) {
  message("Full run on ~16GB memory: consider running MI/weights on a >32GB machine for speed.")
}

subset_data_saved <- FALSE
subset_data_path <- results_path("subset_data_pre_mi.rds")
if (MI_MEMORY_HYGIENE) {
  saveRDS(subset_data, subset_data_path)
  rm(subset_data)
  subset_data_saved <- TRUE
  invisible(gc())
}

# helper: central wrapper for a single `mice()` call so smoke tests, batched
# runs, and full runs all use the same arguments.
run_mice_call <- function(m_val, maxit, seed_val, print_flag = FALSE) {
  mice::mice(
    data            = mi_df_run,
    m               = m_val,
    maxit           = maxit,
```

```
    predictorMatrix = pred,
    method          = meth,
    printFlag       = print_flag,
    seed            = seed_val
  )
}

if (MI_SMOKE_TEST) {
  smoke_con <- file(mi_smoke_log_path, open = "at")
  sink(smoke_con, type = "output")

  cap_smoke <- capture_warnings(
    tryCatch(
      run_mice_call(m_val = 1L, maxit = 1L, seed_val = MI_SEED, print_flag = TRUE),
      error = function(e) e
    ),
    context = make_context(
      stage = "MI",
      component = "mice_smoketest",
      analysis_variant = "mi",
      model_type = "mice",
      group = NA_character_,
      outcome = NA_character_,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_warnings(cap_smoke$warnings)
  sink(type = "output")
  close(smoke_con)

  if (inherits(cap_smoke$value, "error")) {
    smoke_msg <- conditionMessage(cap_smoke$value)
    message("MICE smoke test failed: ", smoke_msg)
    write_smoke_log(c(
      "Smoke test failed.",
```

```
      paste0("Error: ", smoke_msg)
    ))
    if (grepl("vector memory limit", smoke_msg, fixed = TRUE)) {
      MI_DEBUG_PRINTFLAG <- TRUE
    }
    if (subset_data_saved) {
      subset_data <- readRDS(subset_data_path)
    }
    if (MI_DEBUG_PRINTFLAG) {
      debug_path <- results_path("mice_debug_print.txt")
      dbg_con <- file(debug_path, open = "wt")
      sink(dbg_con, type = "output")
      message("Debug MI: running m=1, maxit=1 with printFlag=TRUE.")
      tryCatch(
        run_mice_call(m_val = 1L, maxit = 1L, seed_val = MI_SEED, print_flag = TRUE),
        error = function(e) e
      )
      sink(type = "output")
      close(dbg_con)
    }
    stop("MICE smoke test failed; see ", mi_smoke_log_path, " and Results/mice_debug_print.txt.")
  } else {
    message("MICE smoke test succeeded (m=1, maxit=1).")
  }
}

# helper: summarize current R heap pressure from `gc()` output so batching logic
# can react before a long MI run exhausts memory.
get_vcells_stats <- function() {
  g <- gc()
  cn <- colnames(g)
  mb_cols <- cn[grepl("\\(Mb\\)", cn)]
  used_mb_col <- if (length(mb_cols)) mb_cols[1] else NA_character_
  limit_col <- cn[grepl("limit", cn, ignore.case = TRUE)]
  limit_col <- if (length(limit_col)) limit_col[1] else NA_character_
  trig_col <- cn[grepl("trigger", cn, ignore.case = TRUE)]
```

```r
  trig_col <- if (length(trig_col)) trig_col[1] else NA_character_
  max_col <- cn[grepl("max", cn, ignore.case = TRUE)]
  max_col <- if (length(max_col)) max_col[1] else NA_character_

  used_mb <- if (!is.na(used_mb_col)) as.numeric(g["Vcells", used_mb_col]) else NA_real_
  limit_mb <- if (!is.na(limit_col)) as.numeric(g["Vcells", limit_col]) else NA_real_
  trig_mb <- if (!is.na(trig_col)) as.numeric(g["Vcells", trig_col]) else NA_real_
  max_mb <- if (!is.na(max_col)) as.numeric(g["Vcells", max_col]) else NA_real_

  if (!is.finite(limit_mb) || limit_mb <= 0) {
    mem_lim <- tryCatch(mem.maxVSize(), error = function(e) NA_real_)
    if (is.finite(mem_lim) && mem_lim > 0) {
      limit_mb <- if (mem_lim < 1e8) mem_lim else mem_lim / 1024^2
    }
  }

  data.frame(
    gc_vcells_used_mb = used_mb,
    gc_vcells_limit_mb = limit_mb,
    gc_vcells_frac = ifelse(is.finite(limit_mb) && limit_mb > 0, used_mb / limit_mb, NA_real_),
    gc_vcells_trigger_mb = trig_mb,
    gc_vcells_max_used_mb = max_mb,
    stringsAsFactors = FALSE
  )
}

get_rss_mb <- function(pid = Sys.getpid()) {
  rss_txt <- tryCatch(
    system2("ps", c("-o", "rss=", "-p", as.character(pid)), stdout = TRUE, stderr = FALSE),
    error = function(e) character()
  )
  rss_kb <- suppressWarnings(as.numeric(trimws(rss_txt[[1]])))
  if (!length(rss_kb) || !is.finite(rss_kb) || rss_kb <= 0) return(NA_real_)
  rss_kb / 1024
}
```

```r
safe_object_size_bytes <- function(x) {
  if (is.null(x)) return(NA_real_)
  as.numeric(utils::object.size(x))
}

set.seed(MI_SEED)
# helper: run one non-batched MICE job while routing timing through the shared
# runtime logger.
run_mice_single <- function(m_val) {
  runtime_logger(
    "mice_imputation",
    run_mice_call(m_val = m_val, maxit = MAXIT_RUN, seed_val = MI_SEED, print_flag = FALSE),
    notes = paste0("m=", m_val, "; maxit=", MAXIT_RUN)
  )
}

mc_progress <- list()
sentinel_specs <- list(
  list(name = "abg_imv", outcome = "imv_proc", treat = "has_abg", co2_var = "paco2",
       low = ABG_CO2_LOW, high = ABG_CO2_HIGH),
  list(name = "vbg_imv", outcome = "imv_proc", treat = "has_vbg", co2_var = "vbg_co2",
       low = VBG_CO2_LOW, high = VBG_CO2_HIGH)
)

# helper: estimate a Monte Carlo error to standard-error ratio for one sentinel
# outcome/CO2 specification during optional early-stopping checks.
mcerr_ratio_for_spec <- function(imp_obj, spec) {
  fits <- lapply(seq_len(imp_obj$m), function(i) {
    d <- mice::complete(imp_obj, action = i)
    d <- d[, c(spec$outcome, spec$treat, spec$co2_var), drop = FALSE]
    d <- d[d[[spec$treat]] == 1 & is.finite(d[[spec$co2_var]]), , drop = FALSE]
    if (nrow(d) == 0L) return(NULL)
    d$co2_cat <- make_co2_cat3(d[[spec$co2_var]], spec$low, spec$high)
    d$co2_cat <- stats::relevel(base::droplevels(d$co2_cat), ref = "Normal")
    if (dplyr::n_distinct(d[[spec$outcome]]) < 2L) return(NULL)
    fit <- tryCatch(
```

```
      stats::glm(stats::reformulate("co2_cat", response = spec$outcome),
                 data = d, family = binomial(), x = FALSE, y = FALSE, model = FALSE),
      error = function(e) NULL
    )
    if (is.null(fit)) return(NULL)
    list(coef = stats::coef(fit), vcov = stats::vcov(fit))
  })
  fits <- fits[!vapply(fits, is.null, logical(1))]
  if (length(fits) < 2L) return(NA_real_)
  results <- lapply(fits, `[[`, "coef")
  variances <- lapply(fits, `[[`, "vcov")
  pooled <- mitools::MIcombine(results = results, variances = variances)
  est <- as.numeric(stats::coef(pooled))
  names(est) <- names(stats::coef(pooled))
  se <- sqrt(diag(pooled$variance))
  names(se) <- names(stats::coef(pooled))
  coef_mat <- do.call(cbind, lapply(results, function(v) v[names(est)]))
  B <- apply(coef_mat, 1, stats::var, na.rm = TRUE)
  mcerr <- sqrt(B / length(results))
  ratio <- mcerr / se
  idx <- grepl("^co2_cat", names(ratio))
  if (!any(idx)) return(max(ratio, na.rm = TRUE))
  max(ratio[idx], na.rm = TRUE)
}

# helper: run MICE in smaller batches while logging enough stage detail to
# distinguish a hard stop inside `mice()` from a later combine or checkpoint
# failure when a full-scale run dies abruptly.
run_mice_batched <- function(m_total, m_batch_start, maxit, base_seed) {
  imp_acc <- NULL
  m_done <- 0L
  batch_attempt_idx <- 0L
  m_batch <- m_batch_start
  batch_log <- list()
  combine_log <- list()
  logged_events_acc <- list()
```

```r
checkpoint_dir <- results_path("mi_batch_checkpoints")
batch_context_path <- results_path("mi_batch_context.rds")

dir.create(checkpoint_dir, recursive = TRUE, showWarnings = FALSE)
old_checkpoints <- list.files(checkpoint_dir, full.names = TRUE, recursive = TRUE)
if (length(old_checkpoints)) unlink(old_checkpoints, recursive = TRUE, force = TRUE)
if (file.exists(batch_context_path)) unlink(batch_context_path, force = TRUE)

saveRDS(
  list(
    mi_df_run = mi_df_run,
    predictorMatrix = pred,
    method = meth,
    maxit = maxit,
    m_total = m_total,
    m_batch_start = m_batch_start,
    base_seed = base_seed,
    run_mode = RUN_MODE,
    pilot_frac = PILOT_FRAC,
    mi_seed = MI_SEED,
    run_id = if (exists("RUN_ID")) RUN_ID else NA_character_,
    run_ts = if (exists("RUN_TS")) RUN_TS else NA_character_
  ),
  batch_context_path
)

empty_batch_log <- data.frame(
  batch = integer(),
  stage = character(),
  status = character(),
  failure_stage = character(),
  m_batch = integer(),
  seed = integer(),
  ok = logical(),
  error_message = character(),
  started_at = character(),
```

```
  finished_at = character(),
  m_done_before = integer(),
  m_done_after = integer(),
  seconds = numeric(),
  gc_vcells_used_mb_pre = numeric(),
  gc_vcells_limit_mb_pre = numeric(),
  gc_vcells_frac_pre = numeric(),
  gc_vcells_used_mb_post = numeric(),
  gc_vcells_limit_mb_post = numeric(),
  gc_vcells_frac_post = numeric(),
  rss_mb = numeric(),
  imp_b_size_bytes = numeric(),
  imp_acc_size_bytes = numeric(),
  checkpoint_path = character(),
  notes = character(),
  stringsAsFactors = FALSE
)

empty_combine_log <- data.frame(
  batch = integer(),
  combine_stage = character(),
  left_m = integer(),
  right_m = integer(),
  left_size_bytes = numeric(),
  right_size_bytes = numeric(),
  rss_mb = numeric(),
  gc_vcells_frac = numeric(),
  started_at = character(),
  finished_at = character(),
  ok = logical(),
  error_message = character(),
  stringsAsFactors = FALSE
)

write_batch_log <- function(extra_rows = NULL) {
  rows_out <- batch_log
```

```r
  if (!is.null(extra_rows)) rows_out[[length(rows_out) + 1L]] <- extra_rows
  if (length(rows_out) == 0L) {
    write_csv_safely(empty_batch_log, results_path("mice_batches_log.csv"), row_names = FALSE)
  } else {
    write_csv_safely(dplyr::bind_rows(rows_out), results_path("mice_batches_log.csv"), row_names = FALSE)
  }
}

append_batch_log <- function(row) {
  batch_log[[length(batch_log) + 1L]] <<- row
  write_batch_log()
}

write_combine_log <- function(extra_rows = NULL) {
  rows_out <- combine_log
  if (!is.null(extra_rows)) rows_out[[length(rows_out) + 1L]] <- extra_rows
  if (length(rows_out) == 0L) {
    write_csv_safely(empty_combine_log, results_path("mice_combine_log.csv"), row_names = FALSE)
  } else {
    write_csv_safely(dplyr::bind_rows(rows_out), results_path("mice_combine_log.csv"), row_names = FALSE)
  }
}

append_combine_log <- function(row) {
  combine_log[[length(combine_log) + 1L]] <<- row
  write_combine_log()
}

make_batch_row <- function(batch, stage, status, failure_stage = NA_character_,
                           m_batch, seed, ok, error_message = NA_character_,
                           started_at, finished_at = NA_character_,
                           m_done_before, m_done_after = NA_integer_,
                           seconds = NA_real_,
                           gc_pre = get_vcells_stats(),
                           gc_post = NULL,
                           rss_mb = get_rss_mb(),
```

```
                           imp_b_size_bytes = NA_real_,
                           imp_acc_size_bytes = NA_real_,
                           checkpoint_path = NA_character_,
                           notes = NA_character_) {
  if (is.null(gc_post)) {
    gc_post <- data.frame(
      gc_vcells_used_mb = NA_real_,
      gc_vcells_limit_mb = NA_real_,
      gc_vcells_frac = NA_real_,
      stringsAsFactors = FALSE
    )
  }
  data.frame(
    batch = batch,
    stage = stage,
    status = status,
    failure_stage = failure_stage,
    m_batch = m_batch,
    seed = seed,
    ok = ok,
    error_message = error_message,
    started_at = as.character(started_at),
    finished_at = as.character(finished_at),
    m_done_before = m_done_before,
    m_done_after = m_done_after,
    seconds = seconds,
    gc_vcells_used_mb_pre = gc_pre$gc_vcells_used_mb,
    gc_vcells_limit_mb_pre = gc_pre$gc_vcells_limit_mb,
    gc_vcells_frac_pre = gc_pre$gc_vcells_frac,
    gc_vcells_used_mb_post = gc_post$gc_vcells_used_mb,
    gc_vcells_limit_mb_post = gc_post$gc_vcells_limit_mb,
    gc_vcells_frac_post = gc_post$gc_vcells_frac,
    rss_mb = rss_mb,
    imp_b_size_bytes = imp_b_size_bytes,
    imp_acc_size_bytes = imp_acc_size_bytes,
    checkpoint_path = checkpoint_path,
```

```
    notes = notes,
    stringsAsFactors = FALSE
  )
}

make_combine_row <- function(batch, combine_stage, left_m, right_m,
                             left_size_bytes, right_size_bytes,
                             started_at, finished_at = NA_character_,
                             ok = NA, error_message = NA_character_,
                             gc_vcells_frac = get_vcells_stats()$gc_vcells_frac,
                             rss_mb = get_rss_mb()) {
  data.frame(
    batch = batch,
    combine_stage = combine_stage,
    left_m = left_m,
    right_m = right_m,
    left_size_bytes = left_size_bytes,
    right_size_bytes = right_size_bytes,
    rss_mb = rss_mb,
    gc_vcells_frac = gc_vcells_frac,
    started_at = as.character(started_at),
    finished_at = as.character(finished_at),
    ok = ok,
    error_message = error_message,
    stringsAsFactors = FALSE
  )
}

write_batch_log()
write_combine_log()

while (m_done < m_total) {
  if (MI_GC_EVERY_BATCH) invisible(gc())
  mem_stats <- get_vcells_stats()
  if (MI_PREEMPTIVE_BATCH_REDUCE &&
      is.finite(mem_stats$gc_vcells_frac) &&
```

```
    mem_stats$gc_vcells_frac > MI_VCELLS_FRAC_THRESHOLD &&
    m_batch > MI_BATCH_MIN) {
  m_batch <- max(MI_BATCH_MIN, floor(m_batch / 2))
  message("Preemptively reducing MI batch size to ", m_batch,
          " (Vcells pressure: ", round(mem_stats$gc_vcells_frac, 2), ").")
}

m_b <- min(m_batch, m_total - m_done)
batch_attempt_idx <- batch_attempt_idx + 1L
seed_b <- base_seed + batch_attempt_idx * MI_BATCH_SEED_STRIDE
message("MICE batch ", batch_attempt_idx, " (m=", m_b, ", seed=", seed_b, ")")

batch_id <- sprintf("%02d", batch_attempt_idx)
imp_b_path <- file.path(checkpoint_dir, paste0("imp_b_batch_", batch_id, ".rds"))
imp_acc_path <- file.path(checkpoint_dir, paste0("imp_acc_after_batch_", batch_id, ".rds"))
t0 <- Sys.time()
m_done_before <- m_done
mem_pre <- get_vcells_stats()
append_batch_log(
  make_batch_row(
    batch = batch_attempt_idx,
    stage = "batch_started",
    status = "started",
    m_batch = m_b,
    seed = seed_b,
    ok = NA,
    started_at = t0,
    finished_at = NA_character_,
    m_done_before = m_done_before,
    m_done_after = NA_integer_,
    gc_pre = mem_pre,
    gc_post = NULL,
    rss_mb = get_rss_mb(),
    imp_b_size_bytes = NA_real_,
    imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
    checkpoint_path = NA_character_,
```

```
      notes = "batch entry"
    )
  )

  mice_started_at <- Sys.time()
  append_batch_log(
    make_batch_row(
      batch = batch_attempt_idx,
      stage = "mice_call_entered",
      status = "started",
      m_batch = m_b,
      seed = seed_b,
      ok = NA,
      started_at = mice_started_at,
      finished_at = NA_character_,
      m_done_before = m_done_before,
      m_done_after = m_done_before,
      gc_pre = mem_pre,
      gc_post = NULL,
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = NA_real_,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = NA_character_,
      notes = "about to call mice::mice()"
    )
  )
  cap <- capture_warnings(
    tryCatch(
      runtime_logger(
        paste0("mice_batch_", batch_attempt_idx),
        mice::mice(
          data            = mi_df_run,
          m               = m_b,
          maxit           = maxit,
          predictorMatrix = pred,
          method          = meth,
```

```
        printFlag       = FALSE,
        seed            = seed_b
      ),
      notes = paste0("batch=", batch_attempt_idx, "; m=", m_b, "; maxit=", maxit)
    ),
    error = function(e) e
  ),
  context = make_context(
    stage = "MI",
    component = "mice_batch",
    analysis_variant = "mi",
    model_type = "mice",
    group = NA_character_,
    outcome = NA_character_,
    imputation = NA_integer_,
    batch = batch_attempt_idx
  )
)
append_warnings(cap$warnings)

imp_b <- cap$value
if (inherits(imp_b, "error")) {
  err_msg <- conditionMessage(imp_b)
  message("MICE batch ", batch_attempt_idx, " failed: ", err_msg)
  mem_stats <- get_vcells_stats()
  append_batch_log(
    make_batch_row(
      batch = batch_attempt_idx,
      stage = "batch_failed",
      status = "failed",
      failure_stage = "mice",
      m_batch = m_b,
      seed = seed_b,
      ok = FALSE,
      error_message = err_msg,
      started_at = mice_started_at,
```

```
      finished_at = Sys.time(),
      m_done_before = m_done_before,
      m_done_after = m_done_before,
      seconds = as.numeric(difftime(Sys.time(), mice_started_at, units = "secs")),
      gc_pre = mem_pre,
      gc_post = mem_stats,
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = NA_real_,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = NA_character_,
      notes = "mice() returned a caught error"
    )
  )
  if (grepl("vector memory limit", err_msg, fixed = TRUE) && m_batch > MI_BATCH_MIN) {
    m_batch <- max(MI_BATCH_MIN, floor(m_batch / 2))
    message("Reducing MI batch size to ", m_batch, " and retrying.")
    invisible(gc())
    next
  }
  stop("MICE batch ", batch_attempt_idx, " failed; see log: ", results_path("mice_batches_log.csv"))
}

mice_finished_at <- Sys.time()
mem_after_mice <- get_vcells_stats()
imp_b_size_bytes <- safe_object_size_bytes(imp_b)
append_batch_log(
  make_batch_row(
    batch = batch_attempt_idx,
    stage = "mice_returned",
    status = "ok",
    m_batch = m_b,
    seed = seed_b,
    ok = TRUE,
    started_at = mice_started_at,
    finished_at = mice_finished_at,
    m_done_before = m_done_before,
```

```
      m_done_after = m_done_before,
      seconds = as.numeric(difftime(mice_finished_at, mice_started_at, units = "secs")),
      gc_pre = mem_pre,
      gc_post = mem_after_mice,
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = imp_b_size_bytes,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = NA_character_,
      notes = "mice() returned successfully"
    )
  )

  imp_b_saved <- tryCatch({
    saveRDS(imp_b, imp_b_path)
    TRUE
  }, error = function(e) e)
  if (inherits(imp_b_saved, "error")) {
    err_msg <- conditionMessage(imp_b_saved)
    mem_stats <- get_vcells_stats()
    append_batch_log(
      make_batch_row(
        batch = batch_attempt_idx,
        stage = "batch_failed",
        status = "failed",
        failure_stage = "imp_b_checkpoint_save",
        m_batch = m_b,
        seed = seed_b,
        ok = FALSE,
        error_message = err_msg,
        started_at = mice_finished_at,
        finished_at = Sys.time(),
        m_done_before = m_done_before,
        m_done_after = m_done_before,
        seconds = as.numeric(difftime(Sys.time(), mice_finished_at, units = "secs")),
        gc_pre = mem_after_mice,
        gc_post = mem_stats,
```

```
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = imp_b_size_bytes,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = imp_b_path,
      notes = "failed while saving imp_b checkpoint"
    )
  )
  stop("Failed to save imp_b checkpoint for batch ", batch_attempt_idx, ": ", err_msg, call. = FALSE)
}

ibind_started_at <- Sys.time()
mem_ibind_start <- get_vcells_stats()
left_m <- if (is.null(imp_acc)) 0L else imp_acc$m
left_size_bytes <- safe_object_size_bytes(imp_acc)
right_m <- imp_b$m
append_batch_log(
  make_batch_row(
    batch = batch_attempt_idx,
    stage = "ibind_started",
    status = "started",
    m_batch = m_b,
    seed = seed_b,
    ok = NA,
    started_at = ibind_started_at,
    finished_at = NA_character_,
    m_done_before = m_done_before,
    m_done_after = m_done_before,
    gc_pre = mem_ibind_start,
    gc_post = NULL,
    rss_mb = get_rss_mb(),
    imp_b_size_bytes = imp_b_size_bytes,
    imp_acc_size_bytes = left_size_bytes,
    checkpoint_path = imp_b_path,
    notes = if (is.null(imp_acc)) "initial accumulator assignment" else "about to call mice::ibind()"
  )
)
```

```r
  append_combine_log(
    make_combine_row(
      batch = batch_attempt_idx,
      combine_stage = "ibind_started",
      left_m = left_m,
      right_m = right_m,
      left_size_bytes = left_size_bytes,
      right_size_bytes = imp_b_size_bytes,
      started_at = ibind_started_at,
      finished_at = NA_character_,
      ok = NA,
      error_message = NA_character_,
      gc_vcells_frac = mem_ibind_start$gc_vcells_frac,
      rss_mb = get_rss_mb()
    )
  )

  ibind_result <- tryCatch(
    if (is.null(imp_acc)) imp_b else mice::ibind(imp_acc, imp_b),
    error = function(e) e
  )
  if (inherits(ibind_result, "error")) {
    err_msg <- conditionMessage(ibind_result)
    mem_stats <- get_vcells_stats()
    append_combine_log(
      make_combine_row(
        batch = batch_attempt_idx,
        combine_stage = "ibind_failed",
        left_m = left_m,
        right_m = right_m,
        left_size_bytes = left_size_bytes,
        right_size_bytes = imp_b_size_bytes,
        started_at = ibind_started_at,
        finished_at = Sys.time(),
        ok = FALSE,
        error_message = err_msg,
```

```
        gc_vcells_frac = mem_stats$gc_vcells_frac,
        rss_mb = get_rss_mb()
      )
    )
    append_batch_log(
      make_batch_row(
        batch = batch_attempt_idx,
        stage = "batch_failed",
        status = "failed",
        failure_stage = "ibind",
        m_batch = m_b,
        seed = seed_b,
        ok = FALSE,
        error_message = err_msg,
        started_at = ibind_started_at,
        finished_at = Sys.time(),
        m_done_before = m_done_before,
        m_done_after = m_done_before,
        seconds = as.numeric(difftime(Sys.time(), ibind_started_at, units = "secs")),
        gc_pre = mem_ibind_start,
        gc_post = mem_stats,
        rss_mb = get_rss_mb(),
        imp_b_size_bytes = imp_b_size_bytes,
        imp_acc_size_bytes = left_size_bytes,
        checkpoint_path = imp_b_path,
        notes = "mice::ibind() returned a caught error"
      )
    )
    stop("MICE batch ", batch_attempt_idx, " failed during ibind(); see log: ",
     ↪  results_path("mice_batches_log.csv"))
  }

  imp_acc <- ibind_result
  ibind_finished_at <- Sys.time()
  mem_after_ibind <- get_vcells_stats()
  append_combine_log(
```

```r
    make_combine_row(
      batch = batch_attempt_idx,
      combine_stage = "ibind_completed",
      left_m = left_m,
      right_m = right_m,
      left_size_bytes = left_size_bytes,
      right_size_bytes = imp_b_size_bytes,
      started_at = ibind_started_at,
      finished_at = ibind_finished_at,
      ok = TRUE,
      error_message = NA_character_,
      gc_vcells_frac = mem_after_ibind$gc_vcells_frac,
      rss_mb = get_rss_mb()
    )
  )
  append_batch_log(
    make_batch_row(
      batch = batch_attempt_idx,
      stage = "ibind_completed",
      status = "ok",
      m_batch = m_b,
      seed = seed_b,
      ok = TRUE,
      started_at = ibind_started_at,
      finished_at = ibind_finished_at,
      m_done_before = m_done_before,
      m_done_after = imp_acc$m,
      seconds = as.numeric(difftime(ibind_finished_at, ibind_started_at, units = "secs")),
      gc_pre = mem_ibind_start,
      gc_post = mem_after_ibind,
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = imp_b_size_bytes,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = imp_b_path,
      notes = if (left_m == 0L) "initialized accumulator" else "mice::ibind() completed"
    )
```

```
)

le_b <- imp_b$loggedEvents
le_b <- if (is.null(le_b)) data.frame() else as.data.frame(le_b)
if (NROW(le_b) > 0) {
  le_b <- le_b |>
    dplyr::mutate(
      batch = batch_attempt_idx,
      seed = seed_b,
      m_global_start = m_done + 1L
    )
  logged_events_acc[[length(logged_events_acc) + 1L]] <- le_b
}

m_done <- imp_acc$m
checkpoint_started_at <- Sys.time()
append_batch_log(
  make_batch_row(
    batch = batch_attempt_idx,
    stage = "checkpoint_started",
    status = "started",
    m_batch = m_b,
    seed = seed_b,
    ok = NA,
    started_at = checkpoint_started_at,
    finished_at = NA_character_,
    m_done_before = m_done_before,
    m_done_after = m_done,
    gc_pre = mem_after_ibind,
    gc_post = NULL,
    rss_mb = get_rss_mb(),
    imp_b_size_bytes = imp_b_size_bytes,
    imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
    checkpoint_path = imp_acc_path,
    notes = "about to save accumulated mids checkpoint"
  )
```

```
  )
  imp_acc_saved <- tryCatch({
    saveRDS(imp_acc, imp_acc_path)
    TRUE
  }, error = function(e) e)
  if (inherits(imp_acc_saved, "error")) {
    err_msg <- conditionMessage(imp_acc_saved)
    mem_stats <- get_vcells_stats()
    append_batch_log(
      make_batch_row(
        batch = batch_attempt_idx,
        stage = "batch_failed",
        status = "failed",
        failure_stage = "imp_acc_checkpoint_save",
        m_batch = m_b,
        seed = seed_b,
        ok = FALSE,
        error_message = err_msg,
        started_at = checkpoint_started_at,
        finished_at = Sys.time(),
        m_done_before = m_done_before,
        m_done_after = m_done,
        seconds = as.numeric(difftime(Sys.time(), checkpoint_started_at, units = "secs")),
        gc_pre = mem_after_ibind,
        gc_post = mem_stats,
        rss_mb = get_rss_mb(),
        imp_b_size_bytes = imp_b_size_bytes,
        imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
        checkpoint_path = imp_acc_path,
        notes = "failed while saving imp_acc checkpoint"
      )
    )
    stop("Failed to save imp_acc checkpoint for batch ", batch_attempt_idx, ": ", err_msg, call. = FALSE)
  }
  checkpoint_finished_at <- Sys.time()
  mem_after_checkpoint <- get_vcells_stats()
```

```
  append_batch_log(
    make_batch_row(
      batch = batch_attempt_idx,
      stage = "checkpoint_saved",
      status = "completed",
      m_batch = m_b,
      seed = seed_b,
      ok = TRUE,
      started_at = checkpoint_started_at,
      finished_at = checkpoint_finished_at,
      m_done_before = m_done_before,
      m_done_after = m_done,
      seconds = as.numeric(difftime(checkpoint_finished_at, checkpoint_started_at, units = "secs")),
      gc_pre = mem_after_ibind,
      gc_post = mem_after_checkpoint,
      rss_mb = get_rss_mb(),
      imp_b_size_bytes = imp_b_size_bytes,
      imp_acc_size_bytes = safe_object_size_bytes(imp_acc),
      checkpoint_path = imp_acc_path,
      notes = "saved accumulated mids checkpoint"
    )
  )
  rm(imp_b)
  if (MI_GC_EVERY_BATCH) invisible(gc())

  if (ALLOW_M_IMP_EARLY_STOP &&
      m_done >= M_IMP_MIN &&
      (m_done %% M_IMP_STEP == 0 || m_done == m_total)) {
    ratios <- vapply(sentinel_specs, function(s) mcerr_ratio_for_spec(imp_acc, s), numeric(1))
    mc_progress[[length(mc_progress) + 1L]] <<- data.frame(
      m = m_done,
      abg_ratio = ratios[["abg_imv"]],
      vbg_ratio = ratios[["vbg_imv"]],
      max_ratio = max(ratios, na.rm = TRUE),
      stringsAsFactors = FALSE
    )
```

```r
      write_csv_safely(dplyr::bind_rows(mc_progress),
                       results_path("mi_mcerr_progress.csv"),
                       row_names = FALSE)
      if (all(is.finite(ratios)) && max(ratios, na.rm = TRUE) <= MCERR_RATIO_TARGET) {
        message("MC error criterion met at m=", m_done,
                " (max MCerr/SE=", round(max(ratios, na.rm = TRUE), 3), "). Stopping early.")
        break
      }
    }
  }

  write_batch_log()
  write_combine_log()
  log_events_raw_batched <- dplyr::bind_rows(logged_events_acc)
  attr(imp_acc, "logged_events_batched") <- log_events_raw_batched
  imp_acc
}


imp <- NULL
use_batched <- isTRUE(FORCE_MI_BATCHED)
if (!use_batched) {
  cap_mice <- capture_warnings(
    tryCatch(run_mice_single(M_IMP_RUN), error = function(e) e),
    context = make_context(
      stage = "MI",
      component = "mice",
      analysis_variant = "mi",
      model_type = "mice",
      group = NA_character_,
      outcome = NA_character_,
      imputation = NA_integer_,
      batch = NA_integer_
    )
  )
  append_warnings(cap_mice$warnings)
```

```r
  imp <- cap_mice$value
  if (inherits(imp, "error") && grepl("vector memory limit", conditionMessage(imp), fixed = TRUE)) {
    message("MICE memory limit hit; switching to batched mode.")
    use_batched <- TRUE
  }
}
if (use_batched) {
  message("MICE: running in batches (start=", MI_BATCH_START, ").")
  imp <- run_mice_batched(M_IMP_RUN, MI_BATCH_START, MAXIT_RUN, MI_SEED)
} else {
  write_csv_safely(data.frame(), results_path("mice_batches_log.csv"), row_names = FALSE)
  write_csv_safely(data.frame(), results_path("mice_combine_log.csv"), row_names = FALSE)
}
if (inherits(imp, "error")) stop(imp)
stopifnot(inherits(imp, "mids"))
if (ALLOW_M_IMP_EARLY_STOP && imp$m < M_IMP_RUN) {
  message("Early stop: stopping at m=", imp$m, " (target ", M_IMP_RUN, ").")
  M_IMP_RUN <- imp$m
} else {
  stopifnot(imp$m == M_IMP_RUN)
}
if (M_IMP != M_IMP_RUN) M_IMP <- M_IMP_RUN
if (MAXIT_MI != MAXIT_RUN) MAXIT_MI <- MAXIT_RUN
write_csv_safely(dplyr::bind_rows(mc_progress), results_path("mi_mcerr_progress.csv"), row_names = FALSE)
saveRDS(imp, file = mi_mids_file)

# Save MICE spec for reproducibility
saveRDS(
  list(method = imp$method, predictorMatrix = imp$predictorMatrix),
  results_path("mice_spec.rds")
)
if (use_batched) {
  message("Multiple imputation was run in batches and combined via mice::ibind().")
}
if (subset_data_saved) {
  subset_data <- readRDS(subset_data_path)
```

```r
}

# Logged events: raw + summary (by dep/out/meth)
log_events_raw <- as.data.frame(imp$loggedEvents)
log_events_batched <- as.data.frame(attr(imp, "logged_events_batched"))
log_events_raw <- dplyr::bind_rows(log_events_raw, log_events_batched)

if (nrow(log_events_raw)) {
  write_csv_safely(log_events_raw, results_path("mice_logged_events_raw.csv"), row_names = FALSE,
                   required_cols = c("dep", "out", "meth"))
  log_events_summary <- log_events_raw |>
    dplyr::count(dep, out, meth, name = "n") |>
    dplyr::mutate(variable = dep) |>
    dplyr::arrange(dplyr::desc(n)) |>
    dplyr::mutate(pct = n / sum(n))
  write_csv_safely(log_events_summary, results_path("mice_logged_events_summary.csv"), row_names = FALSE,
                   required_cols = c("variable", "n", "pct"))
} else {
  log_events_raw_empty <- data.frame(
    dep = character(), out = character(), meth = character(),
    stringsAsFactors = FALSE
  )
  log_events_summary_empty <- data.frame(
    variable = character(), n = integer(), pct = numeric(),
    stringsAsFactors = FALSE
  )
  write_csv_safely(log_events_raw_empty, results_path("mice_logged_events_raw.csv"), row_names = FALSE,
                   required_cols = c("dep", "out", "meth"))
  write_csv_safely(log_events_summary_empty, results_path("mice_logged_events_summary.csv"), row_names = FALSE,
                   required_cols = c("variable", "n", "pct"))
  log_events_summary <- log_events_summary_empty
}
if (nrow(mi_info_log)) {
  warns_events <- mi_info_log |>
    dplyr::filter(stage == "MI", component %in% c("mice", "mice_batch"))
  if (nrow(warns_events) && nrow(log_events_raw) == 0L) {
```

```r
    warning("Mismatch: main MI run reported logged events but loggedEvents table is empty; ",
            "check batch capture and loggedEvents exports.", call. = FALSE)
  }
}

# Chain diagnostics (lightweight; no complete("long"))
chain_diag <- data.frame()
chain_diag_stats <- list(
  n_imputed_vars = 0L,
  n_with_chainMean = 0L,
  n_with_drift_tail = 0L,
  drift_tail_na_frac = NA_real_,
  tail_window_na_mean = NA_real_
)
stopifnot(!is.null(imp$method))
impute_vars <- names(imp$method)[imp$method != ""]
impute_vars <- intersect(impute_vars, names(imp$data))
if (length(impute_vars)) {
  impute_vars <- impute_vars[vapply(imp$data[impute_vars], function(x) any(is.na(x)), logical(1))]
}
stopifnot(!is.null(imp$chainMean))
{
  cm <- imp$chainMean
  dims <- dim(cm)
  dn <- dimnames(cm)
  iter_candidates <- unique(c(imp$iteration, MAXIT_MI, MAXIT_MI + 1L))
  iter_candidates <- iter_candidates[is.finite(iter_candidates) & iter_candidates > 0]
  imp_m <- imp$m

  extract_chain_mean <- function(cm_obj, dims_obj, dn_obj, imp_m_val, iter_cand) {
    mean_chain <- NULL
    var_names <- NULL
    iter_dim <- NA_integer_
    var_dim <- NA_integer_
    m_dim <- NA_integer_
    if (!is.null(dims_obj) && length(dims_obj) == 2) {
```

```r
    iter_dim <- which(dims_obj %in% iter_cand)[1]
    if (length(iter_dim) == 0) iter_dim <- 1L
    var_dim <- setdiff(seq_along(dims_obj), iter_dim)[1]
    mean_chain <- cm_obj
    if (iter_dim == 2L) mean_chain <- t(mean_chain)
  } else if (!is.null(dims_obj) && length(dims_obj) == 3) {
    m_dim <- which(dims_obj == imp_m_val)
    if (length(m_dim)) {
      m_dim <- m_dim[1]
    } else {
      m_dim <- 3L
    }
    iter_dim <- setdiff(which(dims_obj %in% iter_cand), m_dim)[1]
    if (length(iter_dim) == 0) iter_dim <- setdiff(1:3, c(m_dim, NA_integer_))[1]
    var_dim <- setdiff(1:3, c(iter_dim, m_dim))[1]
    if (all(is.finite(c(iter_dim, var_dim, m_dim)))) {
      cm_std <- aperm(cm_obj, c(iter_dim, var_dim, m_dim))
      mean_chain <- apply(cm_std, c(1, 2), mean, na.rm = TRUE)
      if (!is.null(dimnames(cm_std))) {
        if (!is.null(dimnames(cm_std)[[2]])) {
          var_names <- dimnames(cm_std)[[2]]
        }
        if (!is.null(dimnames(cm_std)[[1]])) {
          rownames(mean_chain) <- dimnames(cm_std)[[1]]
        }
      }
    }
  }
  if (is.null(mean_chain)) {
    return(list(mean_chain = NULL, var_names = NULL, iter_dim = iter_dim, var_dim = var_dim, m_dim = m_dim))
  }
  if (is.null(var_names) && !is.null(dn_obj) && length(dn_obj) >= var_dim) {
    var_names <- dn_obj[[var_dim]]
  }
  if (is.null(var_names)) {
    var_names <- colnames(mean_chain)
```

```r
  }
  list(mean_chain = mean_chain, var_names = var_names, iter_dim = iter_dim, var_dim = var_dim, m_dim = m_dim)
}

res_chain <- extract_chain_mean(cm, dims, dn, imp_m, iter_candidates)
mean_chain <- res_chain$mean_chain
var_names <- res_chain$var_names
if (is.null(mean_chain)) {
  stop("Chain diagnostics: unable to construct mean_chain from imp$chainMean.")
}

{
  iter_idx <- seq_len(nrow(mean_chain))
  numeric_names <- !is.null(var_names) && all(grepl("^\\d+$", var_names))
  if (is.null(var_names) || numeric_names) {
    if (length(impute_vars) && length(impute_vars) == ncol(mean_chain)) {
      var_names <- impute_vars
      numeric_names <- FALSE
    }
  }
  if (is.null(var_names)) {
    stop("Chain diagnostics: variable names missing and could not be matched to imp$method.")
  }
  colnames(mean_chain) <- var_names

  if (numeric_names) {
    warn_df <- data.frame(
      time = as.character(Sys.time()),
      stage = "MI",
      component = "chain_diagnostics",
      analysis_variant = NA_character_,
      model_type = NA_character_,
      group = NA_character_,
      outcome = NA_character_,
      imputation = NA_integer_,
      batch = NA_integer_,
```

```
    message = "Chain diagnostics variable name mapping failed; using numeric/fallback names.",
    stringsAsFactors = FALSE
  )
  append_warnings(warn_df)
}

vars_imputed <- impute_vars
chain_diag_stats$n_imputed_vars <- length(vars_imputed)
keep_vars <- intersect(vars_imputed, var_names)
missing_in_chain <- setdiff(vars_imputed, var_names)
if (length(missing_in_chain)) {
  missing_df <- data.frame(
    variable = missing_in_chain,
    stringsAsFactors = FALSE
  )
  write_csv_safely(missing_df, results_path("mice_chain_diagnostics_missing_vars.csv"), row_names = FALSE)
}
if (length(vars_imputed) && length(keep_vars) == 0L) {
  warn_df <- data.frame(
    time = as.character(Sys.time()),
    stage = "MI",
    component = "chain_diagnostics",
    analysis_variant = NA_character_,
    model_type = NA_character_,
    group = NA_character_,
    outcome = NA_character_,
    imputation = NA_integer_,
    batch = NA_integer_,
    message = "Chain diagnostics: no imputed variables matched chainMean names; skipping drift metrics.",
    stringsAsFactors = FALSE
  )
  append_warnings(warn_df)
  mean_chain <- mean_chain[, 0, drop = FALSE]
  var_names <- character()
} else if (length(keep_vars)) {
  mean_chain <- mean_chain[, keep_vars, drop = FALSE]
```

```r
  var_names <- keep_vars
}

safe_sd <- function(x) if (sum(is.finite(x)) < 2) NA_real_ else stats::sd(x, na.rm = TRUE)
safe_maxdiff <- function(x) {
  x <- x[is.finite(x)]
  if (length(x) < 2) return(NA_real_)
  max(abs(diff(x)))
}
safe_slope <- function(x, iter) {
  ok <- is.finite(x)
  if (sum(ok) < 2) return(NA_real_)
  coef(stats::lm(x[ok] ~ iter[ok]))[2]
}
tail_finite <- function(x, k) tail(x[is.finite(x)], k)
safe_maxdiff_tail <- function(x, k) {
  xf <- tail_finite(x, k)
  if (length(xf) < 2) return(NA_real_)
  max(abs(diff(xf)))
}

n_vars <- ncol(mean_chain)
n_finite <- integer(n_vars)
drift_all <- numeric(n_vars)
drift_tail <- numeric(n_vars)
tail_n_finite <- integer(n_vars)
tail_window_na_frac <- numeric(n_vars)
slope <- numeric(n_vars)
sd_chain <- numeric(n_vars)
overall_reason <- character(n_vars)
tail_reason <- character(n_vars)
flag_reason <- character(n_vars)
flag <- logical(n_vars)
diagnostic_available <- logical(n_vars)
sd_obs <- rep(NA_real_, n_vars)
```

```r
chain_src_df <- mi_df_run
stopifnot(is.data.frame(chain_src_df))
sd_obs <- vapply(var_names, function(v) {
  stopifnot(v %in% names(chain_src_df))
  x <- chain_src_df[[v]]
  if (inherits(x, "haven_labelled")) x <- suppressWarnings(as.numeric(x))
  if (!is.numeric(x)) return(NA_real_)
  safe_sd(x)
}, numeric(1))

for (j in seq_len(n_vars)) {
  x <- mean_chain[, j]
  n_finite[j] <- sum(is.finite(x))
  drift_all[j] <- safe_maxdiff(x)
  slope[j] <- safe_slope(x, iter_idx)
  sd_chain[j] <- safe_sd(x)
  tail_vals <- tail_finite(x, TAIL_N_FINITE)
  tail_n_finite[j] <- length(tail_vals)
  drift_tail[j] <- safe_maxdiff_tail(x, TAIL_N_FINITE)
  tail_window <- tail(x, min(TAIL_WINDOW_ITERS, length(x)))
  tail_window_na_frac[j] <- mean(!is.finite(tail_window))
  overall_reason[j] <- if (n_finite[j] >= 2) "ok" else "insufficient_finite_overall"
  tail_reason[j] <- if (tail_n_finite[j] >= 2) "ok" else "insufficient_finite_tail"
}

drift_all_scaled <- drift_all / sd_obs
drift_tail_scaled <- drift_tail / sd_obs
slope_scaled <- slope / sd_obs

for (j in seq_len(n_vars)) {
  if (!is.finite(sd_obs[j]) || sd_obs[j] <= 0) {
    flag[j] <- NA
    flag_reason[j] <- "missing_scale"
    diagnostic_available[j] <- FALSE
    next
  }
```

```r
    if (tail_n_finite[j] < 2) {
      flag[j] <- NA
      flag_reason[j] <- "tail_insufficient"
      diagnostic_available[j] <- FALSE
      next
    }
    if (!is.finite(drift_tail_scaled[j]) || !is.finite(slope_scaled[j])) {
      flag[j] <- NA
      flag_reason[j] <- "insufficient_data"
      diagnostic_available[j] <- FALSE
      next
    }
    flag_tail <- drift_tail_scaled[j] > 0.01
    flag_slope <- abs(slope_scaled[j]) > 0.001
    flag[j] <- flag_tail | flag_slope
    flag_reason[j] <- if (flag_tail & flag_slope) {
      "both"
    } else if (flag_tail) {
      "tail_drift"
    } else if (flag_slope) {
      "slope"
    } else {
      "none"
    }
    diagnostic_available[j] <- TRUE
  }

  chain_diag <- data.frame(
    variable = var_names,
    method = imp$method[var_names],
    n_finite = n_finite,
    drift_all = drift_all,
    drift_tail = drift_tail,
    drift_all_scaled = drift_all_scaled,
    drift_tail_scaled = drift_tail_scaled,
    tail_n_finite = tail_n_finite,
```

```r
      tail_window_na_frac = tail_window_na_frac,
      slope = slope,
      slope_scaled = slope_scaled,
      sd_chain = sd_chain,
      sd_obs = sd_obs,
      overall_reason = overall_reason,
      tail_reason = tail_reason,
      flag_reason = flag_reason,
      diagnostic_available = diagnostic_available,
      flag = flag,
      stringsAsFactors = FALSE
    )

    chain_diag_stats$n_with_chainMean <- nrow(chain_diag)
    chain_diag_stats$n_with_drift_tail <- sum(is.finite(chain_diag$drift_tail))
    if (chain_diag_stats$n_with_chainMean > 0) {
      chain_diag_stats$drift_tail_na_frac <- 1 - (chain_diag_stats$n_with_drift_tail /
                                                  chain_diag_stats$n_with_chainMean)
      chain_diag_stats$tail_window_na_mean <- mean(chain_diag$tail_window_na_frac, na.rm = TRUE)
    }
    write_csv_safely(chain_diag, results_path("mice_chain_diagnostics.csv"), row_names = FALSE)
    if (any(isTRUE(chain_diag$flag), na.rm = TRUE)) {
      frac_flag <- mean(chain_diag$flag, na.rm = TRUE)
      if (is.finite(frac_flag) && frac_flag < 0.05) {
        message("Chain diagnostics: low drift flags (", round(frac_flag * 100, 1),
                "%). MAXIT_MI may be reduced safely (consider 10 or 5).")
      }
    }
  }
}

# quick sanity: these must exist and be numeric in completed data
imp_n <- imp$m
get_imp_stats <- list(count = 0L, seconds = 0)
# helper: materialize one completed imputation with the same normalization used
# everywhere else in the notebook, while timing repeated extraction cost.
```

```r
get_imp <- function(i, imp_obj = imp) {
  t0 <- Sys.time()
  d <- normalize_types(mice::complete(imp_obj, action = i), levels_ref)
  get_imp_stats$count <<- get_imp_stats$count + 1L
  get_imp_stats$seconds <<- get_imp_stats$seconds +
    as.numeric(difftime(Sys.time(), t0, units = "secs"))
  d
}
d1 <- get_imp(1)
stopifnot(all(c("paco2","vbg_co2") %in% names(d1)))
stopifnot(is.numeric(d1$paco2), is.numeric(d1$vbg_co2))

# post-MICE sanity: no remaining NA in covars_ps
covars_check <- intersect(covars_ps, names(d1))
na_counts <- vapply(d1[, covars_check, drop = FALSE], function(x) sum(is.na(x)), numeric(1))
na_counts <- na_counts[na_counts > 0]
if (length(na_counts)) {
  message("Post-MICE NA counts (covars_ps): ",
          paste(names(na_counts), na_counts, collapse = ", "))
  ev_sum <- summarize_logged_events(imp)
  if (nrow(ev_sum)) {
    ev_sub <- ev_sum[ev_sum$variable %in% names(na_counts), , drop = FALSE]
    if (nrow(ev_sub)) {
      print(utils::head(ev_sub, 10))
    } else {
      message("No loggedEvents entries for covars_ps with NA.")
    }
  } else {
    message("No loggedEvents recorded.")
  }
  stop("Post-MICE check failed: remaining NA in covars_ps. See loggedEvents summary above.")
}
```

```r
# Purpose: mi mcerror.
stopifnot(exists("imp"))

# Representative logistic model (unweighted) to assess Monte Carlo error
```

```r
mc_diag_fml <- imv_proc ~ has_abg + age_at_encounter + curr_bmi + sex + encounter_type
register_model_diagram("MI MC error: IMV ~ ABG + covariates", mc_diag_fml, width = 10, height = 6)
cap_mc <- capture_warnings(
  tryCatch(
    with(
      imp,
      glm(imv_proc ~ has_abg + age_at_encounter + curr_bmi + sex + encounter_type,
          family = binomial(), x = FALSE, y = FALSE, model = FALSE)
    ),
    error = function(e) e
  ),
  context = make_context(
    stage = "diagnostics",
    component = "mi_mcerror_glm",
    analysis_variant = "mi",
    model_type = "glm",
    group = NA_character_,
    outcome = "imv_proc",
    imputation = NA_integer_,
    batch = NA_integer_
  )
)
append_warnings(cap_mc$warnings)
mc_fit <- cap_mc$value

use_compact <- inherits(mc_fit, "error")
if (use_compact) {
  message("MC error: fallback to compact per-imputation fits (", conditionMessage(mc_fit), ").")
}

mc_pool <- NULL
mc_sum <- NULL
mc_results <- NULL
mc_variances <- NULL
if (!use_compact) {
  mc_pool <- pool(mc_fit)
```

```r
  mc_sum  <- summary(mc_pool, conf.int = TRUE)
}

# helper: compute Monte Carlo error from the full `mira` fit object returned by
# the standard `with(imp, ...)` workflow.
compute_mc_error <- function(mc_pool, mc_fit, m) {
  stopifnot(!is.null(mc_fit$analyses), length(mc_fit$analyses) >= 2)
  coefs <- lapply(mc_fit$analyses, coef)
  common_terms <- Reduce(intersect, lapply(coefs, names))
  stopifnot(length(common_terms) > 0)
  Q <- do.call(cbind, lapply(coefs, function(v) v[common_terms]))
  B <- apply(Q, 1, stats::var, na.rm = TRUE)
  out <- sqrt(B / ncol(Q))
  names(out) <- common_terms
  out
}

# helper: compute Monte Carlo error from compact coefficient lists when the
# standard `with(imp, ...)` path fails and the notebook falls back to manual fits.
compute_mc_error_from_results <- function(results_list) {
  coefs <- results_list
  common_terms <- Reduce(intersect, lapply(coefs, names))
  if (!length(common_terms)) return(NA_real_)
  Q <- do.call(cbind, lapply(coefs, function(v) v[common_terms]))
  B <- apply(Q, 1, stats::var, na.rm = TRUE)
  out <- sqrt(B / ncol(Q))
  names(out) <- common_terms
  out
}

if (use_compact) {
  mc_terms <- all.vars(mc_diag_fml)
  mc_fit_list <- lapply(seq_len(imp$m), function(i) {
    d <- mice::complete(imp, action = i)
    d <- d[, mc_terms, drop = FALSE]
    cap_i <- capture_warnings(
```

```r
    tryCatch(
      glm(mc_diag_fml, data = d, family = binomial(), x = FALSE, y = FALSE, model = FALSE),
      error = function(e) e
    ),
    context = make_context(
      stage = "diagnostics",
      component = "mi_mcerror_glm",
      analysis_variant = "mi",
      model_type = "glm",
      group = NA_character_,
      outcome = "imv_proc",
      imputation = i,
      batch = NA_integer_
    )
  )
  append_warnings(cap_i$warnings)
  fit_i <- cap_i$value
  if (inherits(fit_i, "error")) return(list(error = conditionMessage(fit_i)))
  list(coef = coef(fit_i), vcov = vcov(fit_i))
})

ok <- vapply(mc_fit_list, function(x) is.list(x) && is.null(x$error), logical(1))
mc_fit_list <- mc_fit_list[ok]
if (!length(mc_fit_list)) stop("MC error fallback failed: no successful fits.")

mc_results <- lapply(mc_fit_list, function(x) x$coef)
mc_variances <- lapply(mc_fit_list, function(x) x$vcov)

mc_pool <- mitools::MIcombine(results = mc_results, variances = mc_variances)
est <- as.numeric(coef(mc_pool))
se  <- sqrt(diag(mc_pool$variance))
mc_sum <- data.frame(
  term = names(coef(mc_pool)),
  estimate = est,
  std.error = se,
  conf.low = est - 1.96 * se,
```

```r
    conf.high = est + 1.96 * se,
    stringsAsFactors = FALSE
  )
}

m_mc <- if (!use_compact && !is.null(mc_fit$analyses)) length(mc_fit$analyses) else imp$m
mc_err_vec <- if (!use_compact) {
  compute_mc_error(mc_pool, mc_fit, m_mc)
} else {
  compute_mc_error_from_results(mc_results)
}

if (is.null(names(mc_err_vec)) || all(names(mc_err_vec) == "")) {
  names(mc_err_vec) <- mc_sum$term[seq_len(min(length(mc_err_vec), nrow(mc_sum)))]
}
mc_err_aligned <- mc_err_vec[mc_sum$term]
if (all(is.na(mc_err_aligned))) {
  message("MC error not computed (all NA). mice version: ", as.character(packageVersion("mice")))
  if (!is.null(mc_pool$pooled)) {
    message("mc_pool$pooled columns: ", paste(names(mc_pool$pooled), collapse = ", "))
  }
}

stopifnot(all(c("conf.low", "conf.high") %in% names(mc_sum)))

mc_tab <- mc_sum |>
  mutate(
    mc_error = mc_err_aligned,
    mc_err_over_se = mc_error / std.error
  ) |>
  select(term, estimate, std.error, mc_error, mc_err_over_se, conf.low, conf.high)

mc_model_desc <- "Diagnostic model: imv_proc ~ has_abg + age_at_encounter + curr_bmi + sex + encounter_type
 ↪  (unweighted)."
knitr::asis_output(paste0("**MC error diagnostic model:** ", mc_model_desc, "\n"))
```

```r
gt::gt(mc_tab) |>
  gt::tab_header(title = "Monte Carlo error vs SE (diagnostic only)") |>
  gt::cols_label(
    term           = "Term",
    estimate       = "Estimate",
    std.error      = "SE",
    mc_error       = "MC error",
    mc_err_over_se = "MC error / SE",
    conf.low       = "2.5%",
    conf.high      = "97.5%"
  ) |>
  gt::fmt_number(columns = c(estimate, std.error, mc_error, mc_err_over_se), decimals = 3) |>
  gt::fmt_missing(columns = gt::everything(), missing_text = "-")

# QC flag for key estimands only (diagnostic, not pass/fail for all coefficients)
mc_key_terms <- intersect(c("has_abg"), mc_tab$term)
if (length(mc_key_terms)) {
  mc_key <- mc_tab |>
    dplyr::filter(term %in% mc_key_terms) |>
    dplyr::mutate(qc_flag = mc_err_over_se > MC_ERR_RATIO_THRESH)
  render_table_pdf_maybe(
    mc_key,
    caption = paste0("MC error QC (key terms only; threshold = ", MC_ERR_RATIO_THRESH, ")"),
    file_stub = "mc_error_qc_key_terms",
    digits = 3,
    show = FALSE
  )
  if (any(mc_key$qc_flag, na.rm = TRUE)) {
    knitr::asis_output(
      paste0(
        "**QC note:** MC error/SE exceeds ",
        MC_ERR_RATIO_THRESH,
        " for at least one key term; consider larger m or report as a limitation."
      )
    )
  }
```

```
}
```

```
# Purpose: mi diagnostics.
stopifnot(exists("imp"))
imp_n <- imp$m
# helper: extract one completed dataset for MI diagnostics while preserving the
# same normalization rules used in the modeling sections.
get_imp <- function(i, imp_obj = imp) {
  normalize_types(mice::complete(imp_obj, action = i), levels_ref)
}

# Guard against memory blow-up from complete("long")
N <- nrow(imp$data)
M <- imp$m
long_rows <- (M + 1L) * N
LONG_ROWS_MAX <- 2e6
imp_size <- as.numeric(utils::object.size(imp))
if (long_rows > LONG_ROWS_MAX || imp_size > 1e9) {
  message("Skipping mice::densityplot/stripplot (mids -> complete('long')) due to size; using memory-safe
   ↪ diagnostics.")
}

# helper: extract observed and imputed values for one variable without
# constructing the full long-format completed dataset.
get_obs_imp_vectors <- function(imp_obj, var) {
  obs <- imp_obj$data[[var]]
  obs <- obs[!is.na(obs)]
  impv <- imp_obj$imp[[var]]
  stopifnot(!is.null(impv))
  imp_vals <- unlist(impv, use.names = FALSE)
  if (is.numeric(obs)) obs <- obs[is.finite(obs)]
  if (is.numeric(imp_vals)) imp_vals <- imp_vals[is.finite(imp_vals)]
  list(obs = obs, imp = imp_vals)
}

# helper: generate lightweight observed-vs-imputed diagnostics and the matching
# summary rows for one variable.
```

```r
plot_obs_imp <- function(imp_obj, var, fig_dir, n_obs = 50000, n_imp = 50000) {
  vecs <- get_obs_imp_vectors(imp_obj, var)
  obs <- vecs$obs
  imp_vals <- vecs$imp
  stopifnot(length(obs) + length(imp_vals) > 0)

  is_cat <- is.factor(obs) || is.character(obs) ||
    (is.numeric(obs) && length(unique(obs[!is.na(obs)])) <= 5)

  if (!is_cat) {
    set.seed(MI_SEED)
    if (length(obs) > n_obs) obs <- sample(obs, n_obs)
    if (length(imp_vals) > n_imp) imp_vals <- sample(imp_vals, n_imp)

    df <- dplyr::tibble(
      value = c(obs, imp_vals),
      status = c(rep("Observed", length(obs)), rep("Imputed", length(imp_vals)))
    )
    p <- ggplot2::ggplot(df, ggplot2::aes(x = value, color = status, fill = status)) +
      ggplot2::geom_density(alpha = 0.2, na.rm = TRUE) +
      theme_pub(base_size = 10) +
      ggplot2::labs(title = paste("Observed vs imputed:", var), x = NULL, y = "Density")
    out_file <- fs::path(fig_dir, paste0("diag-mi-density-", .make_safe_name(var), ".png"))
    save_diag_plot(file = out_file, p = p, width = 7, height = 5, dpi = 300)

    stat_names <- c("mean", "sd", "p10", "p50", "p90", "n")
    obs_stats <- c(
      mean = mean(obs, na.rm = TRUE),
      sd = stats::sd(obs, na.rm = TRUE),
      p10 = stats::quantile(obs, 0.10, na.rm = TRUE),
      p50 = stats::quantile(obs, 0.50, na.rm = TRUE),
      p90 = stats::quantile(obs, 0.90, na.rm = TRUE),
      n = length(obs)
    )
    imp_stats <- c(
      mean = if (length(imp_vals)) mean(imp_vals, na.rm = TRUE) else NA_real_,
```

```r
    sd = if (length(imp_vals)) stats::sd(imp_vals, na.rm = TRUE) else NA_real_,
    p10 = if (length(imp_vals)) stats::quantile(imp_vals, 0.10, na.rm = TRUE) else NA_real_,
    p50 = if (length(imp_vals)) stats::quantile(imp_vals, 0.50, na.rm = TRUE) else NA_real_,
    p90 = if (length(imp_vals)) stats::quantile(imp_vals, 0.90, na.rm = TRUE) else NA_real_,
    n = length(imp_vals)
  )
  return(dplyr::tibble(
    variable = var,
    type = "numeric",
    stat = stat_names,
    observed = unname(obs_stats),
    imputed = unname(imp_stats)
  ))
}

obs_chr <- as.character(obs)
imp_chr <- as.character(imp_vals)
levels_all <- sort(unique(c(obs_chr, imp_chr)))
prop_obs <- if (length(obs_chr)) {
  as.numeric(table(factor(obs_chr, levels = levels_all))) / length(obs_chr)
} else {
  rep(0, length(levels_all))
}
prop_imp <- if (length(imp_chr)) {
  as.numeric(table(factor(imp_chr, levels = levels_all))) / length(imp_chr)
} else {
  rep(0, length(levels_all))
}
prop_df <- dplyr::tibble(
  level = levels_all,
  observed = prop_obs,
  imputed = prop_imp
)
miss_plot_df <- tidyr::pivot_longer(prop_df, c(observed, imputed),
                                    names_to = "status", values_to = "prop")
p <- ggplot2::ggplot(miss_plot_df, ggplot2::aes(x = level, y = prop, fill = status)) +
```

```r
    ggplot2::geom_col(position = "dodge", width = 0.7) +
    theme_pub(base_size = 10) +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 20, hjust = 1)) +
    ggplot2::labs(title = paste("Observed vs imputed:", var), x = NULL, y = "Proportion")
  out_file <- fs::path(fig_dir, paste0("diag-mi-bar-", .make_safe_name(var), ".png"))
  save_diag_plot(file = out_file, p = p, width = 7, height = 5, dpi = 300)

  return(prop_df |>
           tidyr::pivot_longer(c(observed, imputed), names_to = "status", values_to = "prop") |>
           dplyr::mutate(variable = var, type = "categorical", stat = "prop"))
}

# Throttle diagnostics to avoid memory blow-up
trace_vars <- intersect(c("curr_bmi", "serum_hco3", "hr"), names(imp$imp))
if (length(trace_vars)) {
  trace_list <- lapply(trace_vars, function(v) {
    imp_mat <- imp$imp[[v]]
    stopifnot(!is.null(imp_mat))
    means <- colMeans(imp_mat, na.rm = TRUE)
    dplyr::tibble(variable = v, imputation = seq_along(means), mean_imputed = means)
  })
  trace_df <- dplyr::bind_rows(trace_list)
  if (nrow(trace_df)) {
    p_trace <- ggplot2::ggplot(trace_df, ggplot2::aes(x = imputation, y = mean_imputed)) +
      ggplot2::geom_line() +
      ggplot2::geom_point(size = 0.6) +
      ggplot2::facet_wrap(~variable, scales = "free_y") +
      theme_pub(base_size = 10) +
      ggplot2::labs(title = "Mean imputed value by imputation", x = "Imputation", y = "Mean")
    save_diag_plot(p_trace, results_path("figs", "diag-mi-trace-means.png"), width = 8, height = 5, dpi = 300)
    print(p_trace)
  }

  vars_show <- trace_vars[seq_len(min(2, length(trace_vars)))]
  dens_list <- lapply(vars_show, function(v) {
    obs <- imp$data[[v]]
```

```r
    imp1 <- get_imp(1)[[v]]
    obs_vals <- obs[!is.na(obs)]
    imp_vals <- imp1[is.na(obs)]
    if (length(obs_vals) > 5000) obs_vals <- sample(obs_vals, 5000)
    if (length(imp_vals) > 5000) imp_vals <- sample(imp_vals, 5000)
    dplyr::tibble(
      variable = v,
      value = c(obs_vals, imp_vals),
      status = c(rep("Observed", length(obs_vals)),
                 rep("Imputed (imp1)", length(imp_vals)))
    )
  })
  dens_df <- dplyr::bind_rows(dens_list)
  p_dens <- ggplot2::ggplot(dens_df, ggplot2::aes(x = value, color = status, fill = status)) +
    ggplot2::geom_density(alpha = 0.2, na.rm = TRUE) +
    ggplot2::facet_wrap(~variable, scales = "free") +
    theme_pub(base_size = 10) +
    ggplot2::labs(title = "Observed vs imputed distributions (imp1)", x = NULL, y = "Density")
  save_diag_plot(p_dens, results_path("figs", "diag-mi-density-imp1.png"), width = 8, height = 5, dpi = 300)
  print(p_dens)

  p_box <- ggplot2::ggplot(dens_df, ggplot2::aes(x = status, y = value, fill = status)) +
    ggplot2::geom_boxplot(outlier.size = 0.5, na.rm = TRUE) +
    ggplot2::facet_wrap(~variable, scales = "free") +
    theme_pub(base_size = 10) +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 20, hjust = 1)) +
    ggplot2::labs(title = "Observed vs imputed (imp1)", x = NULL, y = NULL)
  save_diag_plot(p_box, results_path("figs", "diag-mi-box-imp1.png"), width = 8, height = 5, dpi = 300)
  print(p_box)
}
```

```r
# Observed vs imputed summaries (memory-safe)
diag_vars <- intersect(
  c("age_at_encounter", "curr_bmi", "temp_new", "sbp", "dbp", "hr",
    "sodium", "serum_cr", "serum_hco3", "serum_lac", "wbc", "plt",
    "sex", "race_ethnicity", "location", "encounter_type",
    "copd", "asthma", "chf", "dm"),
```

```r
  names(imp$data)
)
diag_vars <- intersect(diag_vars, names(imp$imp))
if (length(diag_vars)) {
  miss_counts <- colSums(is.na(imp$data[diag_vars]))
  diag_vars <- diag_vars[miss_counts > 0]
}

mi_obs_imp_summary <- dplyr::bind_rows(lapply(diag_vars, function(v) {
  out <- plot_obs_imp(imp, v, results_path("figs"))
  invisible(gc())
  out
}))
mi_obs_imp_summary_file <- results_path("mi_obs_vs_imp_summary.csv")
if (nrow(mi_obs_imp_summary)) {
  write_csv_safely(mi_obs_imp_summary, mi_obs_imp_summary_file, row_names = FALSE)
  render_table_pdf_maybe(
    mi_obs_imp_summary,
    caption = "Observed vs imputed summaries",
    file_stub = "mi_obs_vs_imp_summary",
    digits = 3,
    show = FALSE
  )
} else {
  mi_obs_imp_stub <- data.frame(
    variable = character(),
    n_obs = numeric(),
    mean_obs = numeric(),
    sd_obs = numeric(),
    mean_imp = numeric(),
    sd_imp = numeric(),
    stringsAsFactors = FALSE
  )
  write_csv_safely(mi_obs_imp_stub, mi_obs_imp_summary_file, row_names = FALSE)
  message("No variables with missingness available for observed vs imputed summaries.")
}
```

```r
# Purpose: mi missing pattern.
md_pat <- NULL
invisible(utils::capture.output(
  md_pat <- mice::md.pattern(imp$data, plot = FALSE)
))

md_pat_file <- results_path("missingness-pattern.csv")
write_csv_safely(md_pat, md_pat_file, row_names = TRUE)

md_fig_file <- results_path("figs", "missingness-pattern.png")
grDevices::png(md_fig_file, width = 1800, height = 1200, res = 200)
mice::md.pattern(imp$data, plot = TRUE)
grDevices::dev.off()
```

```r
# Purpose: mi imputation diagnostics.
stopifnot(exists("imp"))
imp_n <- imp$m
# helper: lightweight imputation extractor reused inside the more visual
# imputation-diagnostics section below.
get_imp <- function(i, imp_obj = imp) { normalize_types(mice::complete(imp_obj, action = i), levels_ref) }

# Choose a manageable set of incomplete variables
miss_overall <- naniar::miss_var_summary(subset_data) %>% arrange(desc(pct_miss))
vars_incomplete <- miss_overall$variable[miss_overall$n_miss > 0]
per_imp_n <- min(500, nrow(imp$data))
dat_obs <- imp$data %>%
  dplyr::slice_sample(n = min(per_imp_n, nrow(imp$data))) %>%
  dplyr::mutate(.imp = 0L, .imp_label = "Observed")
dat_imp <- purrr::map_dfr(seq_len(imp_n), function(i) {
  di <- get_imp(i)
  di <- dplyr::slice_sample(di, n = min(per_imp_n, nrow(di)))
  di$.imp <- i
  di$.imp_label <- "Imputed"
  di
})
dat_long <- dplyr::bind_rows(dat_obs, dat_imp)
vars_show <- head(intersect(vars_incomplete, names(dat_long)), 6)  # limit for plotting & ensure present in imp
```

```r
# Density plots by imputation status
if (length(vars_show)) {
  strata_candidates <- intersect(c("has_abg", "has_vbg", "imv_proc", "death_60d"), names(dat_long))
  printed_counts <- FALSE

  for (v in vars_show) {
    if (is.numeric(dat_long[[v]])) {
      df_plot <- dat_long |>
        dplyr::filter(is.finite(.data[[v]]))
      p <- ggplot(df_plot, aes(x = .data[[v]], fill = .imp_label)) +
        geom_density(alpha = 0.4, adjust = 1, na.rm = TRUE) +
        labs(title = paste("Observed vs imputed density:", v), x = v, fill = "Source") +
        theme_pub()
    } else {
      df_plot <- dat_long |>
        dplyr::filter(!is.na(.data[[v]]))
      p <- ggplot(df_plot, aes(x = .data[[v]], fill = .imp_label)) +
        geom_bar(position = "fill", na.rm = TRUE) +
        labs(title = paste("Observed vs imputed proportions:", v), x = v, y = "Proportion", fill = "Source") +
        theme_pub() +
        coord_flip()
    }
    save_diag_plot(
      p,
      results_path("figs", paste0("diag-mi-obs-imp-", .make_safe_name(v), ".png")),
      width = 7,
      height = 5,
      dpi = 300
    )
    print(p)
  }

  # Box/violin plots by strata (numeric vars only)
  if (length(strata_candidates)) {
    pd <- position_dodge(width = 0.8)
```

```r
for (sv in strata_candidates) {
  for (v in vars_show) {
    if (!is.numeric(dat_long[[v]])) next
    df_plot <- dat_long |>
      dplyr::filter(!is.na(.data[[sv]]), is.finite(.data[[v]])) |>
      dplyr::mutate(.strata = factor(.data[[sv]]))
    if (nrow(df_plot) == 0L) next

    if (!printed_counts) {
      invisible(table(strata = df_plot$.strata, source = df_plot$.imp_label))
      printed_counts <- TRUE
    }

    if (all(levels(df_plot$.strata) %in% c("0", "1"))) {
      x_scale <- ggplot2::scale_x_discrete(labels = c("0" = "No", "1" = "Yes"))
    } else {
      x_scale <- ggplot2::scale_x_discrete(drop = FALSE)
    }

    p <- ggplot(df_plot, aes(
      x = .strata,
      y = .data[[v]],
      fill = .imp_label,
      group = interaction(.imp_label, .strata)
    )) +
      geom_violin(alpha = 0.5, scale = "width", trim = TRUE, position = pd, na.rm = TRUE) +
      geom_boxplot(width = 0.2, outlier.size = 0.6, position = pd, na.rm = TRUE) +
      x_scale +
      labs(title = paste("Observed vs imputed:", v, "by", sv),
           x = sv, y = v, fill = "Source") +
      theme_pub()
    print(p)
    out_file <- results_path(
      "figs",
      paste0("diag-mi-obs-imp-by-", .make_safe_name(sv), "-", .make_safe_name(v), ".png")
    )
```

```
        save_diag_plot(p, out_file, width = 7, height = 5)
      }
    }
  }
}
```

```
# Purpose: mi mcerr progress.
mc_file <- results_path("mi_mcerr_progress.csv")
stopifnot(file.exists(mc_file))
mc_prog <- utils::read.csv(mc_file)
render_table_pdf_maybe(
  mc_prog,
  caption = "MC error progress by m (early-stop diagnostic)",
  file_stub = "mi_mcerr_progress",
  digits = 3,
  show = FALSE
)
```

```
# --- Lean missingness audit (memory-safe) ---------------------------------

stopifnot(exists("imp"))

dat_imp <- mice::complete(imp, action = 1, include = FALSE)

# 1) Tabular summary on completed data (should be near 0% by design)
miss_tbl <- naniar::miss_var_summary(dat_imp) %>% arrange(desc(pct_miss))
render_table_pdf_maybe(
  miss_tbl,
  caption = "Missingness after MI (completed data)",
  file_stub = "missingness_after_mi",
  digits = 2,
  show = FALSE
)

# 2) Bar plot of top-K (mostly zeros after imputation)
K <- 40
top_vars <- miss_tbl$variable[seq_len(min(K, nrow(miss_tbl)))]
```

```r
p_top <- ggplot(miss_tbl[miss_tbl$variable %in% top_vars, ],
                aes(x = reorder(variable, pct_miss), y = pct_miss)) +
  geom_col() +
  coord_flip() +
  labs(title = "Top missing variables (after MI)", x = NULL, y = "% missing") +
  theme_pub()
save_diag_plot(p_top, results_path("figs", "missingness-after-mi-top-vars.png"), width = 8, height = 7, dpi = 300)
print(p_top)
```

```r
# 3) Small heatmap on imputed data (rows/cols sampled)
M <- 60
R <- min(1500, nrow(dat_imp))
cols_heat <- head(top_vars, M)
rows_heat <- dplyr::slice_sample(dat_imp, n = R)

p_heat <- naniar::vis_miss(rows_heat[, cols_heat, drop = FALSE]) +
  labs(title = sprintf("Missingness heatmap on imputed data (%d rows × %d cols)", R, length(cols_heat)))
save_diag_plot(p_heat, results_path("figs", "missingness-after-mi-heatmap.png"), width = 10, height = 7, dpi = 300)
print(p_heat)
```

```r
# 4) Optional, but often heavy: UpSet of co-missingness - skip by default.
# If you really want it, do it for top 6-10 variables only:
# naniar::gg_miss_upset(dat_imp[, head(top_vars, 8), drop = FALSE])
```

3.3 Refit propensity models within each imputation

MI propensity scores use logistic regression with restricted cubic splines (`rms::rcs`, 4 knots by default) for continuous covariates; the same covariate set used in non-MI models is reused here (`covars_ps`). IPSW truncation rules are unchanged.

Note: the MI computations below run in a single pass per imputation (weights, balance, cat3, spline). Subsequent MI sections reuse those outputs and will stop if they are missing.

3.3.1 FAIL-FAST CHECKS

```r
# Purpose: fail fast checks.
# These checks intentionally stop early if MI state is not valid.
```

```r
# Failing here is cheaper than failing deep in long modeling loops.
mi_min_required <- if (IS_PILOT) 20L else 50L
if (M_IMP < mi_min_required) {
  stop("M_IMP must be >= ", mi_min_required, ". Current M_IMP = ", M_IMP)
}

stopifnot(exists("imp"))

if (imp$m < mi_min_required) {
  stop("imp$m must be >= ", mi_min_required, ". Current imp$m = ", imp$m)
}

# Ensure no missing covariates in each imputation for WeightIt
for (i in seq_len(imp$m)) {
  di <- get_imp(i)
  # Weighting models require complete covariates after MI.
  assert_no_na_covars(di, covars_ps, context = paste0("imputation ", i))
}
```

```r
# This single-pass loop computes MI weights, target balance, 3-level outcomes,
# and spline outcomes in one traversal of imputations to avoid repeated
# mice::complete() calls and re-normalization.

stopifnot(exists("imp"))
imp_n <- imp$m
mi_single_pass_t0 <- Sys.time()

# --- Helper: fit ABG/VBG weights for a single imputation ----------------------
fit_abg_one <- function(d, imp_index) {
  # Build PS design frame for this imputation.
  d_ps <- d[, c("has_abg", "has_vbg", covars_ps), drop = FALSE]
  d_ps <- normalize_types(d_ps, levels_ref)
  d_ps <- droplevels_all(d_ps)

  # Write one factor-level diagnostic file (first imputation only).
  if (imp_index == 1L) {
    write_factor_levels_diag(d_ps, covars_ps, "abg", file_prefix = "mi_logistic_ps_factor_levels")
```

```r
}

# Guardrail: covariates must be complete before fitting propensity model.
assert_no_na_covars(d_ps, covars_ps, context = "ABG MI PS (glm)")

# Fit MI logistic propensity model with spline terms for continuous predictors.
ps_fit <- fit_mi_ps_glm(
  d_ps, "has_abg", covars_ps,
  k = MI_PS_SPLINE_K, maxit = MI_GLM_MAXIT,
  context = make_context(
    stage = "MI", component = "mi_ps_glm",
    analysis_variant = "weighted_imputed",
    model_type = "ps",
    group = "ABG",
    outcome = NA_character_,
    imputation = imp_index,
    batch = NA_integer_
  )
)
if (!is.null(ps_fit$error)) {
  stop("MI PS model failed (ABG, imp ", imp_index, "): ", ps_fit$error)
}

# Convert propensity scores to one-sided IPSW according to project rules.
ipow <- compute_ipow_weights(
  ps_fit,
  treat = to01(d_ps$has_abg),
  ps_floor_quantile = ps_trunc_quantile,
  stabilize = TRUE
)
assert_finite_weights(ipow$weights[d_ps$has_abg == 1], "w_abg")

# Return compact metadata only (no large model object retained).
list(
  weights = ipow$weights,
  ps = ipow$ps,
```

```r
    ipow_info = list(
      ps_floor = ipow$ps_floor,
      cap      = ipow$cap,
      trunc_rate = mean(ipow$truncated, na.rm = TRUE)
    ),
    method = ps_fit$method,
    fit_ok = isTRUE(ps_fit$fit_ok),
    converged = isTRUE(ps_fit$converged),
    n = ps_fit$n,
    p = ps_fit$p,
    formula = ps_fit$formula,
    basis = ps_fit$basis,
    coef = ps_fit$coef,
    vcov_diag = ps_fit$vcov_diag
  )
}

# helper: fit the VBG logistic propensity model and convert it to one-sided
# IPSW weights for a single imputation.
fit_vbg_one <- function(d, imp_index) {
  # Same logic as ABG helper, using has_vbg as treatment indicator.
  d_ps <- d[, c("has_abg", "has_vbg", covars_ps), drop = FALSE]
  d_ps <- normalize_types(d_ps, levels_ref)
  d_ps <- droplevels_all(d_ps)
  if (imp_index == 1L) {
    write_factor_levels_diag(d_ps, covars_ps, "vbg", file_prefix = "mi_logistic_ps_factor_levels")
  }
  assert_no_na_covars(d_ps, covars_ps, context = "VBG MI PS (glm)")
  ps_fit <- fit_mi_ps_glm(
    d_ps, "has_vbg", covars_ps,
    k = MI_PS_SPLINE_K, maxit = MI_GLM_MAXIT,
    context = make_context(
      stage = "MI", component = "mi_ps_glm",
      analysis_variant = "weighted_imputed",
      model_type = "ps",
      group = "VBG",
```

```r
    outcome = NA_character_,
    imputation = imp_index,
    batch = NA_integer_
  )
)
if (!is.null(ps_fit$error)) {
  stop("MI PS model failed (VBG, imp ", imp_index, "): ", ps_fit$error)
}

# Apply one-sided IPSW transformation to VBG propensity scores.
ipow <- compute_ipow_weights(
  ps_fit,
  treat = to01(d_ps$has_vbg),
  ps_floor_quantile = ps_trunc_quantile,
  stabilize = TRUE
)
assert_finite_weights(ipow$weights[d_ps$has_vbg == 1], "w_vbg")

# Keep only lightweight fields needed downstream.
list(
  weights = ipow$weights,
  ps = ipow$ps,
  ipow_info = list(
    ps_floor = ipow$ps_floor,
    cap      = ipow$cap,
    trunc_rate = mean(ipow$truncated, na.rm = TRUE)
  ),
  method = ps_fit$method,
  fit_ok = isTRUE(ps_fit$fit_ok),
  converged = isTRUE(ps_fit$converged),
  n = ps_fit$n,
  p = ps_fit$p,
  formula = ps_fit$formula,
  basis = ps_fit$basis,
  coef = ps_fit$coef,
  vcov_diag = ps_fit$vcov_diag
```

```r
  )
}

# --- Helper: target balance (treated vs target) -------------------------------
target_balance_table <- function(data, treat_var, weights, covars, levels_ref = NULL) {
  # These SMDs compare the tested cohort to the target analytic cohort, not to
  # the untested cohort. That is the balance target implied by this one-sided
  # weighting design, because we want tested encounters to resemble the full
  # analytic population after weighting.
  stopifnot(length(weights) == nrow(data))
  treat <- to01(data[[treat_var]]) == 1L
  treat[is.na(treat)] <- FALSE
  w <- as.numeric(weights)

  w_mean <- function(x, wts) {
    # Small weighted-mean helper that tolerates missing/non-finite values.
    ok <- is.finite(x) & is.finite(wts)
    if (!any(ok)) return(NA_real_)
    sum(wts[ok] * x[ok]) / sum(wts[ok])
  }

  out <- lapply(covars, function(v) {
    x <- data[[v]]
    if (is.character(x)) {
      if (!is.null(levels_ref) && !is.null(levels_ref[[v]])) {
        x <- factor(x, levels = levels_ref[[v]])
      } else {
        x <- factor(x)
      }
    }

    if (is.factor(x)) {
      # Factor covariates: evaluate SMD at each level indicator.
      levs <- levels(x)
      lapply(levs, function(lv) {
        ind <- as.integer(x == lv)
```

```r
      p_target <- mean(ind, na.rm = TRUE)
      sd_target <- sqrt(p_target * (1 - p_target))
      ind_treat <- ind[treat]
      w_treat <- w[treat]
      pre <- mean(ind_treat, na.rm = TRUE)
      post <- w_mean(ind_treat, w_treat)
      smd_pre <- if (is.finite(sd_target) && sd_target > 0) (pre - p_target) / sd_target else NA_real_
      smd_post <- if (is.finite(sd_target) && sd_target > 0) (post - p_target) / sd_target else NA_real_
      data.frame(
        variable = v,
        level = lv,
        type = "factor",
        smd_pre = smd_pre,
        smd_post = smd_post,
        stringsAsFactors = FALSE
      )
    }) |> dplyr::bind_rows()
  } else {
    # Numeric covariates: compare means scaled by target SD.
    x_num <- suppressWarnings(as.numeric(x))
    mean_target <- mean(x_num, na.rm = TRUE)
    sd_target <- stats::sd(x_num, na.rm = TRUE)
    x_treat <- x_num[treat]
    w_treat <- w[treat]
    mean_pre <- mean(x_treat, na.rm = TRUE)
    mean_post <- w_mean(x_treat, w_treat)
    smd_pre <- if (is.finite(sd_target) && sd_target > 0) (mean_pre - mean_target) / sd_target else NA_real_
    smd_post <- if (is.finite(sd_target) && sd_target > 0) (mean_post - mean_target) / sd_target else NA_real_
    data.frame(
      variable = v,
      level = NA_character_,
      type = "numeric",
      smd_pre = smd_pre,
      smd_post = smd_post,
      stringsAsFactors = FALSE
    )
```

```r
    }
  })
  dplyr::bind_rows(out)
}

# helper: keep only model metadata needed for prediction without retaining the
# fitting function frame through terms/formula environments.
strip_terms_environment <- function(tt) {
  stopifnot(inherits(tt, "terms"))
  attr(tt, ".Environment") <- baseenv()
  stopifnot(identical(attr(tt, ".Environment"), baseenv()))
  tt
}

compact_model_metadata <- function(fit) {
  fit_terms <- strip_terms_environment(stats::delete.response(stats::terms(fit)))
  fit_frame <- stats::model.frame(fit)
  list(
    terms = fit_terms,
    xlevels = stats::.getXlevels(fit_terms, fit_frame),
    contrasts = tryCatch(
      attr(stats::model.matrix(fit), "contrasts"),
      error = function(e) NULL
    )
  )
}

# --- Helper: fit 3-level outcome per imputation -------------------------------
fit_cat3_imp <- function(d, weights, outcome_var, co2_var, treat_var,
                         low_cut, high_cut, group_label, imp_index,
                         adj_vars = adj_core,
                         analysis_variant = "mi_ipw") {
  # Fit one categorical outcome model for one imputation within the tested
  # cohort only. This restriction is intentional because measured CO2 is only
  # defined for encounters that actually had the corresponding blood gas.
  # The Normal CO2 category is the reference for Low/High contrasts.
```

```r
stopifnot(co2_var %in% names(d))
d[[co2_var]] <- coerce_num(d[[co2_var]])

# Restrict to treated cohort with observed CO2.
g <- d[[treat_var]] == 1 & is.finite(d[[co2_var]])
if (!any(g)) {
  return(list(error = "No treated rows with finite CO2"))
}

d2 <- d[g, , drop = FALSE]
w <- weights[g]
w[!is.finite(w)] <- NA_real_
ok <- is.finite(w)
if (!all(ok)) {
  d2 <- d2[ok, , drop = FALSE]
  w  <- w[ok]
  if (nrow(d2) == 0L) return(list(error = "All weights non-finite"))
}

d2$co2_cat <- make_co2_cat3(d2[[co2_var]], low_cut, high_cut)
d2$co2_cat <- stats::relevel(base::droplevels(d2$co2_cat), ref = "Normal")
if (nlevels(d2$co2_cat) < 2) {
  return(list(error = "co2_cat has <2 levels"))
}

d2[[outcome_var]] <- to01(d2[[outcome_var]])
des <- survey::svydesign(ids = ~1, weights = ~w, data = d2)
fml <- stats::reformulate(c("co2_cat", adj_vars), response = outcome_var)

# Fit model with diagnostics wrapper so warnings/errors are tracked in logs.
fit_res <- fit_with_diagnostics(
  function() survey::svyglm(fml, design = des, family = quasibinomial(),
                            control = stats::glm.control(maxit = 50)),
  context = make_context(
    stage = "outcome",
    component = "cat3",
```

```r
      analysis_variant = analysis_variant,
      model_type = "cat3",
      group = group_label,
      outcome = outcome_var,
      imputation = imp_index,
      batch = NA_integer_
    )
  )
  diag <- fit_res$diag
  if (!is.null(diag) && nrow(diag)) {
    diag$n_used <- nrow(d2)
    diag$events <- sum(d2[[outcome_var]] == 1, na.rm = TRUE)
  }
  fit <- fit_res$fit
  fit_compact <- NULL
  if (!is.null(fit) && inherits(fit, "svyglm")) {
    # Store only lightweight model metadata needed for reusable predictions.
    fit_compact <- c(
      list(
        coef = stats::coef(fit),
        vcov = stats::vcov(fit)
      ),
      compact_model_metadata(fit)
    )
  }
  list(fit = fit_compact, diag = diag, warnings = fit_res$warnings)
}

# --- Pool terms across imputations (robust svyglm) ----------------------------
pool_terms <- function(fits, term_prefix = NULL, term_pattern = NULL, min_ok_frac = 0.9) {
  fit_compact <- lapply(fits, function(x) {
    if (inherits(x, "svyglm")) {
      return(list(coef = stats::coef(x), vcov = stats::vcov(x)))
    }
    if (is.list(x) && !is.null(x$coef) && !is.null(x$vcov)) {
      return(list(coef = x$coef, vcov = x$vcov))
```

```r
  }
  NULL
})
ok <- !vapply(fit_compact, is.null, logical(1))
fit_compact <- fit_compact[ok]
m_tot <- length(ok)
m_ok <- length(fit_compact)
if (m_ok == 0L) {
  stop("pool_terms: no successful fits; check per-imputation diagnostics.")
}
if (m_ok < ceiling(min_ok_frac * m_tot)) {
  warning("pool_terms: only ", m_ok, " / ", m_tot,
          " fits succeeded (below min_ok_frac).", call. = FALSE)
}
results <- lapply(fit_compact, `[[`, "coef")
variances <- lapply(fit_compact, `[[`, "vcov")
pooled <- mitools::MIcombine(results = results, variances = variances)
pooled_mi_vcov_check(pooled)
est <- as.numeric(coef(pooled))
se  <- sqrt(diag(pooled$variance))
term <- names(coef(pooled))
out <- data.frame(
  term = term,
  logOR = est,
  estimate = exp(est),
  SE = se,
  LCL = exp(est - 1.96 * se),
  UCL = exp(est + 1.96 * se),
  stringsAsFactors = FALSE
)
if (!is.null(term_prefix)) {
  out <- out[grepl(paste0("^", term_prefix), out$term), , drop = FALSE]
}
if (!is.null(term_pattern)) {
  out <- out[grepl(term_pattern, out$term), , drop = FALSE]
}
```

```r
  out
}

# --- Fit spline model on treated cohort only (IPSW) ---------------------------
fit_spline_imp <- function(data, weights, outcome, co2_var, treat_var,
                           adj_vars = NULL,
                           spline_df = SPLINE_DF,
                           spline_basis = SPLINE_BASIS,
                           grid_df = NULL,
                           ref_idx = NULL,
                           imp_index = NA_integer_,
                           analysis_variant = "mi_ipw") {
  # Fit one spline outcome model for one imputation within the tested cohort.
  # If grid_df/ref_idx are supplied, also return pointwise log-OR and linear-
  # predictor quantities so Rubin pooling can happen later without storing large
  # model objects for every imputation/outcome combination.
  spline_basis <- match.arg(spline_basis, c("ns", "rcs"))
  stopifnot(is.data.frame(data))
  stopifnot(length(weights) == nrow(data))
  stopifnot(all(c(outcome, co2_var, treat_var) %in% names(data)))
  if (!is.null(adj_vars) && length(adj_vars)) {
    missing_adj <- setdiff(adj_vars, names(data))
    if (length(missing_adj)) {
      return(list(error = paste0("Missing adj_core vars: ", paste(missing_adj, collapse = ", "))))
    }
  }

  data[[treat_var]] <- to01(data[[treat_var]])
  data[[outcome]]   <- to01(data[[outcome]])
  data[[co2_var]]   <- coerce_num(data[[co2_var]])
  w <- suppressWarnings(as.numeric(weights))

  keep <- (data[[treat_var]] == 1L) &
    is.finite(data[[co2_var]]) &
    !is.na(data[[outcome]]) &
    is.finite(w) & (w > 0)
```

```r
if (!any(keep)) {
  return(list(error = "No eligible treated rows after filtering (treated==1, finite CO2, non-missing outcome, finite
  ↪  positive weights).")) 
}

d2 <- data[keep, , drop = FALSE]
w2 <- w[keep]
d2$w <- w2

if (length(unique(d2[[outcome]])) < 2L) {
  return(list(error = paste0("Outcome has one level in treated sample after filtering: outcome=",
                             outcome, ", n=", nrow(d2),
                             ", events=", sum(d2[[outcome]] == 1, na.rm = TRUE))))
}

des <- survey::svydesign(ids = ~1, weights = ~w, data = d2)
if (spline_basis == "ns") {
  # Natural spline basis for CO2 plus fixed adjustment covariates.
  rhs_terms <- c(sprintf("splines::ns(%s, %d)", co2_var, spline_df), adj_vars)
  fml <- stats::as.formula(paste(outcome, "~", paste(rhs_terms, collapse = " + ")))
} else {
  # rms::rcs option retained for compatibility with previous runs.
  dd <- rms::datadist(d2)
  old_opt <- options(datadist = ".__dd_tmp__")
  assign(".__dd_tmp__", dd, envir = .GlobalEnv)
  on.exit({
    options(old_opt)
    rm(list = ".__dd_tmp__", envir = .GlobalEnv)
  }, add = TRUE)
  rhs_terms <- c(sprintf("rms::rcs(%s, %d)", co2_var, spline_df), adj_vars)
  fml <- stats::as.formula(paste(outcome, "~", paste(rhs_terms, collapse = " + ")))
}

group_label <- if (identical(treat_var, "has_abg")) "ABG" else if (identical(treat_var, "has_vbg")) "VBG" else
↪  treat_var
```

```r
fit_res <- fit_with_diagnostics(
  function() survey::svyglm(fml, design = des, family = quasibinomial(),
                            control = stats::glm.control(maxit = 50)),
  context = make_context(
    stage = "outcome",
    component = "spline",
    analysis_variant = analysis_variant,
    model_type = "spline",
    group = group_label,
    outcome = outcome,
    imputation = imp_index,
    batch = NA_integer_
  )
)
fit <- fit_res$fit
warns <- fit_res$warnings
diag <- fit_res$diag
if (!is.null(diag) && nrow(diag)) {
  diag$n_used <- nrow(d2)
  diag$events <- sum(d2[[outcome]] == 1, na.rm = TRUE)
}
if (is.null(fit)) {
  err_msg <- if (!is.null(diag$error_message) && is.finite(nchar(diag$error_message))) diag$error_message else
↪ "svyglm error"
  return(list(error = err_msg,
              warnings = warns,
              diag = diag,
              n_used = nrow(d2),
              events = sum(d2[[outcome]] == 1, na.rm = TRUE)))
}

fit_compact <- list(
  # Compact storage for Rubin pooling and post-hoc diagnostic predictions.
  coef = stats::coef(fit),
  vcov = stats::vcov(fit),
  formula_text = paste(deparse(fml, width.cutoff = 500L), collapse = " "),
```

```r
  outcome = outcome,
  co2_var = co2_var,
  adj_vars = adj_vars,
  spline_df = spline_df,
  spline_basis = spline_basis
)
fit_compact <- c(fit_compact, compact_model_metadata(fit))

if (!is.null(grid_df)) {
  if (is.null(ref_idx)) {
    return(list(error = "ref_idx is required when grid_df is provided.",
                warnings = warns, diag = diag))
  }
  pr <- tryCatch(
    predict_or_curve_from_fit(fit, grid_df, ref_idx, co2_var),
    error = function(e) list(error = paste0("predict error: ", conditionMessage(e)))
  )
  if (!is.null(pr$error)) return(list(error = pr$error,
                                      warnings = warns,
                                      diag = diag,
                                      n_used = nrow(d2),
                                      events = sum(d2[[outcome]] == 1, na.rm = TRUE)))
  return(list(
    fit = fit_compact,
    eta = pr$eta,
    var_eta = pr$var_eta,
    logOR = pr$logOR,
    var_logOR = pr$var_logOR,
    warnings = warns,
    diag = diag,
    n_used = nrow(d2),
    events = sum(d2[[outcome]] == 1, na.rm = TRUE)
  ))
}

list(fit = fit_compact, warnings = warns, diag = diag,
```

```r
      n_used = nrow(d2),
      events = sum(d2[[outcome]] == 1, na.rm = TRUE))
}

# --- Pool spline coefficients across imputations ------------------------------
pool_spline_coefs <- function(fit_list, min_ok_frac = 0.9) {
  fit_compact <- lapply(fit_list, function(x) {
    if (is.list(x) && !is.null(x$fit) && is.list(x$fit) &&
        !is.null(x$fit$coef) && !is.null(x$fit$vcov)) {
      return(x$fit)
    }
    if (is.list(x) && !is.null(x$coef) && !is.null(x$vcov)) {
      return(x)
    }
    if (inherits(x, "svyglm")) {
      return(list(coef = stats::coef(x), vcov = stats::vcov(x)))
    }
    NULL
  })
  ok <- !vapply(fit_compact, is.null, logical(1))
  m_tot <- length(ok)
  m_ok  <- sum(ok)
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed spline fits: m_ok=", m_ok, " / m_total=", m_tot)
  }
  fit_compact <- fit_compact[ok]
  results <- lapply(fit_compact, `[[`, "coef")
  variances <- lapply(fit_compact, `[[`, "vcov")
  pooled <- mitools::MIcombine(results = results, variances = variances)
  pooled_mi_vcov_check(pooled)
  est <- as.numeric(coef(pooled))
  se  <- sqrt(diag(pooled$variance))
  data.frame(
    term = names(coef(pooled)),
    estimate = est,
    SE = se,
```

```r
    LCL = est - 1.96 * se,
    UCL = est + 1.96 * se,
    m_used = m_ok,
    m_total = m_tot,
    row.names = NULL
  )
}

# --- Pool spline curves (pointwise Rubin pooling on log-OR scale) -------------
pool_spline_curve <- function(fit_list, grid_df, ref_idx, co2_ref, min_ok_frac = 0.9) {
  # Pool on the log-OR scale, then exponentiate. Pooling already-transformed ORs
  # directly would distort both the mean curve and its uncertainty interval.
  n_grid <- nrow(grid_df)
  ok <- vapply(fit_list, function(x) {
    is.list(x) && is.null(x$error) &&
      !is.null(x$logOR) && !is.null(x$var_logOR) &&
      length(x$logOR) == n_grid && length(x$var_logOR) == n_grid
  }, logical(1))
  m_tot <- length(ok)
  m_ok  <- sum(ok)
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed spline fits: m_ok=", m_ok, " / m_total=", m_tot)
  }

  logOR_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  var_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  ok_idx <- which(ok)
  for (i in seq_along(ok_idx)) {
    fit_i <- fit_list[[ok_idx[i]]]
    logOR_mat[, i] <- fit_i$logOR
    var_mat[, i] <- fit_i$var_logOR
  }
  Qbar <- rowMeans(logOR_mat, na.rm = TRUE)
  Ubar <- rowMeans(var_mat, na.rm = TRUE)
  B    <- apply(logOR_mat, 1, stats::var, na.rm = TRUE)
  Tvar <- Ubar + (1 + 1 / m_ok) * B
```

```r
  se   <- sqrt(Tvar)
  if (!is.na(ref_idx) && is.finite(ref_idx)) {
    if (abs(exp(Qbar[ref_idx]) - 1) > 1e-6) {
      warning("Pooled spline OR at reference differs from 1 (check ref_idx/co2_ref).", call. = FALSE)
    }
  }
  data.frame(
    grid_df,
    logOR = Qbar,
    SE_logOR  = se,
    OR = exp(Qbar),
    LCL = exp(Qbar - 1.96 * se),
    UCL = exp(Qbar + 1.96 * se),
    co2_ref = co2_ref,
    m_used = m_ok,
    m_total = m_tot,
    row.names = NULL
  )
}

# --- Pool spline predicted probabilities (pointwise Rubin pooling on eta) -----
pool_spline_prob_curve <- function(fit_list, grid_df, co2_ref, min_ok_frac = 0.9) {
  # Pool predicted probabilities on the linear predictor (eta) scale, then
  # convert back with plogis(). This is more stable than averaging probabilities.
  n_grid <- nrow(grid_df)
  ok <- vapply(fit_list, function(x) {
    is.list(x) && is.null(x$error) &&
      !is.null(x$eta) && !is.null(x$var_eta) &&
      length(x$eta) == n_grid && length(x$var_eta) == n_grid
  }, logical(1))
  m_tot <- length(ok)
  m_ok  <- sum(ok)
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed spline fits for probability pooling: m_ok=", m_ok, " / m_total=", m_tot)
  }
```

```r
  eta_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  var_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  ok_idx <- which(ok)
  for (i in seq_along(ok_idx)) {
    fit_i <- fit_list[[ok_idx[i]]]
    eta_mat[, i] <- fit_i$eta
    var_mat[, i] <- fit_i$var_eta
  }

  Qbar <- rowMeans(eta_mat, na.rm = TRUE)
  Ubar <- rowMeans(var_mat, na.rm = TRUE)
  B <- apply(eta_mat, 1, stats::var, na.rm = TRUE)
  Tvar <- Ubar + (1 + 1 / m_ok) * B
  se_eta <- sqrt(pmax(Tvar, 0))

  data.frame(
    grid_df,
    eta = Qbar,
    SE_eta = se_eta,
    var_eta = Tvar,
    p = plogis(Qbar),
    p_LCL = plogis(Qbar - 1.96 * se_eta),
    p_UCL = plogis(Qbar + 1.96 * se_eta),
    co2_ref = co2_ref,
    m_used = m_ok,
    m_total = m_tot,
    row.names = NULL
  )
}

# --- Pool 3-level category probabilities (Rubin pooling on eta) ----------------
make_cat3_ref_grid <- function(ref_df = data.frame(.ref = 1), cat_var = "co2_cat") {
  stopifnot(is.data.frame(ref_df), nrow(ref_df) > 0L, length(cat_var) == 1L, nzchar(cat_var))
  grid <- ref_df[rep(1L, length(CO2_CAT_LEVELS)), , drop = FALSE]
  grid[[cat_var]] <- factor(CO2_CAT_LEVELS, levels = CO2_CAT_LEVELS)
  grid
```

```r
}

compact_spline_fit <- function(x) {
  if (is.list(x) && !is.null(x$fit) && is.list(x$fit) &&
      !is.null(x$fit$coef) && !is.null(x$fit$vcov)) {
    return(x$fit)
  }
  if (is.list(x) && !is.null(x$coef) && !is.null(x$vcov)) {
    return(x)
  }
  NULL
}

compact_model_matrix <- function(fit_compact, new_data, fallback_formula,
                                 context = "compact model prediction") {
  stopifnot(is.list(fit_compact), is.data.frame(new_data))
  if (!is.null(fit_compact$terms)) {
    tt <- fit_compact$terms
    if (!identical(attr(tt, ".Environment"), baseenv())) {
      stop(context, ": compact terms retain a non-base environment.")
    }
    mf_new <- stats::model.frame(
      tt,
      data = new_data,
      na.action = stats::na.pass,
      xlev = fit_compact$xlevels
    )
    return(stats::model.matrix(tt, mf_new, contrasts.arg = fit_compact$contrasts))
  }
  tt <- stats::delete.response(stats::terms(fallback_formula))
  stats::model.matrix(tt, data = new_data)
}

align_compact_vcov <- function(vcov_mat, coef_names, context = "compact model prediction") {
  if (is.null(vcov_mat)) stop(context, ": missing vcov matrix.")
  if (!is.null(rownames(vcov_mat)) && !is.null(colnames(vcov_mat)) &&
```

```r
      all(coef_names %in% rownames(vcov_mat)) && all(coef_names %in% colnames(vcov_mat))) {
    return(vcov_mat[coef_names, coef_names, drop = FALSE])
  }
  vcov_mat
}

predict_prob_from_compact <- function(fit_compact, new_data, fallback_formula,
                                      context = "compact model prediction") {
  stopifnot(is.list(fit_compact), !is.null(fit_compact$coef), !is.null(fit_compact$vcov))
  beta <- fit_compact$coef
  beta_names <- names(beta)
  if (is.null(beta_names) || any(!nzchar(beta_names))) {
    stop(context, ": coefficient names are required.")
  }
  mm <- compact_model_matrix(fit_compact, new_data, fallback_formula, context = context)
  missing_cols <- setdiff(beta_names, colnames(mm))
  if (length(missing_cols)) {
    mm <- cbind(
      mm,
      matrix(
        0,
        nrow = nrow(mm),
        ncol = length(missing_cols),
        dimnames = list(NULL, missing_cols)
      )
    )
  }
  mm <- mm[, beta_names, drop = FALSE]
  vcov_mat <- align_compact_vcov(fit_compact$vcov, beta_names, context = context)
  if (!identical(dim(vcov_mat), c(length(beta), length(beta)))) {
    stop(context, ": vcov dimensions do not match coefficient vector.")
  }
  eta <- as.numeric(mm %*% beta)
  mm_v <- mm %*% vcov_mat
  var_eta <- pmax(rowSums(mm_v * mm), 0)
  tibble::as_tibble(new_data) |>
```

```r
    dplyr::mutate(
      eta = eta,
      SE_eta = sqrt(var_eta),
      var_eta = var_eta,
      p = stats::plogis(eta),
      p_LCL = stats::plogis(eta - 1.96 * sqrt(var_eta)),
      p_UCL = stats::plogis(eta + 1.96 * sqrt(var_eta))
    )
}

predict_marginal_prob_from_compact <- function(fit_compact, new_data, fallback_formula,
                                               context = "marginal standardized prediction") {
  stopifnot(is.list(fit_compact), !is.null(fit_compact$coef), !is.null(fit_compact$vcov))
  beta <- fit_compact$coef
  beta_names <- names(beta)
  if (is.null(beta_names) || any(!nzchar(beta_names))) {
    stop(context, ": coefficient names are required.")
  }
  mm <- compact_model_matrix(fit_compact, new_data, fallback_formula, context = context)
  missing_cols <- setdiff(beta_names, colnames(mm))
  if (length(missing_cols)) {
    mm <- cbind(
      mm,
      matrix(
        0,
        nrow = nrow(mm),
        ncol = length(missing_cols),
        dimnames = list(NULL, missing_cols)
      )
    )
  }
  mm <- mm[, beta_names, drop = FALSE]
  vcov_mat <- align_compact_vcov(fit_compact$vcov, beta_names, context = context)
  eta <- as.numeric(mm %*% beta)
  p_i <- stats::plogis(eta)
  p_bar <- mean(p_i, na.rm = TRUE)
```

```r
  p_safe <- min(max(p_bar, 1e-8), 1 - 1e-8)
  dp_dbeta <- colMeans(sweep(mm, 1L, p_i * (1 - p_i), `*`), na.rm = TRUE)
  var_p <- as.numeric(t(dp_dbeta) %*% vcov_mat %*% dp_dbeta)
  var_p <- max(var_p, 0)
  var_eta <- var_p / ((p_safe * (1 - p_safe))^2)
  tibble::tibble(
    eta_imp = stats::qlogis(p_safe),
    var_eta_imp = var_eta,
    p_imp = p_bar,
    var_p_imp = var_p,
    n_standardized = nrow(new_data)
  )
}

# helper: compute category-level predicted probabilities from a compact
# coefficient/vcov representation so pooled MI fits need not keep full model objects.
predict_cat3_prob_from_compact <- function(fit_compact, grid_df, outcome_var,
                                           cat_var = "co2_cat",
                                           adj_vars = adj_core) {
  stopifnot(is.list(fit_compact), !is.null(fit_compact$coef), !is.null(fit_compact$vcov))
  stopifnot(cat_var %in% names(grid_df))
  fml <- stats::reformulate(c(cat_var, adj_vars), response = outcome_var)
  predict_prob_from_compact(
    fit_compact,
    grid_df,
    fallback_formula = fml,
    context = paste0("predict_cat3_prob_from_compact(", outcome_var, ")")
  ) |>
    as.data.frame()
}

predict_spline_prob_from_compact <- function(fit_compact, grid_df,
                                             outcome_var,
                                             co2_var,
                                             adj_vars = adj_core) {
  stopifnot(is.list(fit_compact), !is.null(fit_compact$coef), !is.null(fit_compact$vcov))
```

```r
  stopifnot(is.data.frame(grid_df), co2_var %in% names(grid_df))
  rhs_terms <- c(sprintf("splines::ns(%s, %d)", co2_var, SPLINE_DF), adj_vars)
  fallback_fml <- stats::as.formula(paste(outcome_var, "~", paste(rhs_terms, collapse = " + ")))
  predict_prob_from_compact(
    fit_compact,
    grid_df,
    fallback_formula = fallback_fml,
    context = paste0("predict_spline_prob_from_compact(", outcome_var, ")")
  )
}

pool_spline_prob_curve_from_compact <- function(fit_list, grid_df,
                                                outcome_var,
                                                co2_var,
                                                co2_ref,
                                                min_ok_frac = 0.9) {
  pred_list <- lapply(fit_list, function(x) {
    fit_compact <- compact_spline_fit(x)
    if (is.null(fit_compact)) return(NULL)
    tryCatch(
      predict_spline_prob_from_compact(
        fit_compact,
        grid_df = grid_df,
        outcome_var = outcome_var,
        co2_var = co2_var
      ),
      error = function(e) NULL
    )
  })

  n_grid <- nrow(grid_df)
  ok <- vapply(pred_list, function(x) {
    is.data.frame(x) &&
      nrow(x) == n_grid &&
      all(c("eta", "var_eta") %in% names(x))
  }, logical(1L))
```

```r
  m_tot <- length(ok)
  m_ok <- sum(ok)
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed compact spline predictions: m_ok=", m_ok, " / m_total=", m_tot)
  }

  eta_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  var_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  ok_idx <- which(ok)
  for (j in seq_along(ok_idx)) {
    pred_j <- pred_list[[ok_idx[[j]]]]
    eta_mat[, j] <- pred_j$eta
    var_mat[, j] <- pred_j$var_eta
  }

  qbar <- rowMeans(eta_mat, na.rm = TRUE)
  ubar <- rowMeans(var_mat, na.rm = TRUE)
  bvar <- apply(eta_mat, 1, stats::var, na.rm = TRUE)
  tvar <- ubar + (1 + 1 / m_ok) * bvar
  se_eta <- sqrt(pmax(tvar, 0))

  tibble::as_tibble(grid_df) |>
    dplyr::mutate(
      eta = qbar,
      SE_eta = se_eta,
      var_eta = tvar,
      p = stats::plogis(qbar),
      p_LCL = stats::plogis(qbar - 1.96 * se_eta),
      p_UCL = stats::plogis(qbar + 1.96 * se_eta),
      co2_ref = co2_ref,
      m_used = m_ok,
      m_total = m_tot
    )
}

# helper: pool categorical predicted probabilities across imputations on the
```

```r
# linear-predictor scale and return low/normal/high estimates with CIs.
pool_cat3_prob_curve <- function(fit_list, ref_df, outcome_var,
                                 cat_var = "co2_cat",
                                 adj_vars = adj_core, min_ok_frac = 0.9) {
  grid_df <- make_cat3_ref_grid(ref_df, cat_var = cat_var)
  n_grid <- nrow(grid_df)

  pred_list <- lapply(fit_list, function(x) {
    fit_compact <- NULL
    if (inherits(x, "svyglm")) {
      fit_compact <- list(coef = stats::coef(x), vcov = stats::vcov(x))
    } else if (is.list(x) && !is.null(x$coef) && !is.null(x$vcov)) {
      fit_compact <- x
    }
    if (is.null(fit_compact)) return(NULL)
    tryCatch(
      predict_cat3_prob_from_compact(
        fit_compact,
        grid_df,
        outcome_var,
        cat_var = cat_var,
        adj_vars = adj_vars
      ),
      error = function(e) NULL
    )
  })

  ok <- vapply(pred_list, function(x) {
    is.data.frame(x) &&
      nrow(x) == n_grid &&
      all(c("eta", "var_eta") %in% names(x)) &&
      length(x$eta) == n_grid &&
      length(x$var_eta) == n_grid
  }, logical(1))
  m_tot <- length(ok)
  m_ok <- sum(ok)
```

```r
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed category-probability fits: m_ok=", m_ok, " / m_total=", m_tot)
  }

  eta_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  var_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  ok_idx <- which(ok)
  for (i in seq_along(ok_idx)) {
    pred_i <- pred_list[[ok_idx[i]]]
    eta_mat[, i] <- pred_i$eta
    var_mat[, i] <- pred_i$var_eta
  }

  Qbar <- rowMeans(eta_mat, na.rm = TRUE)
  Ubar <- rowMeans(var_mat, na.rm = TRUE)
  B <- apply(eta_mat, 1, stats::var, na.rm = TRUE)
  Tvar <- Ubar + (1 + 1 / m_ok) * B
  se_eta <- sqrt(pmax(Tvar, 0))

  data.frame(
    category = as.character(grid_df[[cat_var]]),
    co2_cat = as.character(grid_df[[cat_var]]),
    eta = Qbar,
    SE_eta = se_eta,
    var_eta = Tvar,
    p = plogis(Qbar),
    p_LCL = plogis(Qbar - 1.96 * se_eta),
    p_UCL = plogis(Qbar + 1.96 * se_eta),
    m_used = m_ok,
    m_total = m_tot,
    row.names = NULL
  )
}

common_std_sample_cap <- if (!IS_PILOT) {
  20000L
```

```r
} else if (is.finite(PILOT_FRAC) && PILOT_FRAC >= 0.25) {
  5000L
} else {
  1000L
}
common_std_min_ok_frac <- 0.9

build_common_std_source_df <- function(d, imp_index,
                                       adj_vars = adj_core,
                                       sample_cap = common_std_sample_cap) {
  stopifnot(is.data.frame(d), length(adj_vars) > 0L)
  missing_adj <- setdiff(adj_vars, names(d))
  if (length(missing_adj)) {
    stop("Common standardization source missing adjustment variables: ",
         paste(missing_adj, collapse = ", "), call. = FALSE)
  }
  source_df <- d[, adj_vars, drop = FALSE]
  source_df <- source_df[stats::complete.cases(source_df), , drop = FALSE]
  n_available <- nrow(source_df)
  if (!n_available) {
    stop("Common standardization source has zero complete rows in imputation ", imp_index, call. = FALSE)
  }
  if (is.finite(sample_cap) && n_available > sample_cap) {
    set.seed(8100L + as.integer(imp_index))
    source_df <- source_df[sample.int(n_available, size = as.integer(sample_cap), replace = FALSE), , drop = FALSE]
  }
  attr(source_df, "n_available") <- n_available
  attr(source_df, "sample_cap") <- as.integer(sample_cap)
  source_df
}

predict_prob_curve_marginal_standardized <- function(fit_compact, source_df,
                                                     co2_var, co2_values,
                                                     outcome_var,
                                                     adj_vars = adj_core) {
  rhs_terms <- c(sprintf("splines::ns(%s, %d)", co2_var, SPLINE_DF), adj_vars)
```

```
  fallback_fml <- stats::as.formula(paste(outcome_var, "~", paste(rhs_terms, collapse = " + ")))
  purrr::map_dfr(co2_values, function(co2_value) {
    nd <- source_df
    nd[[co2_var]] <- co2_value
    predict_marginal_prob_from_compact(
      fit_compact,
      nd,
      fallback_formula = fallback_fml,
      context = paste0("predict_prob_curve_marginal_standardized(", outcome_var, ")")
    ) |>
      dplyr::mutate(
        co2_value = co2_value,
        n_available = attr(source_df, "n_available"),
        standardization_sample_cap = attr(source_df, "sample_cap")
      )
  })
}

predict_prob_cat3_marginal_standardized <- function(fit_compact, source_df,
                                                    outcome_var,
                                                    cat_var = "co2_cat",
                                                    adj_vars = adj_core) {
  fallback_fml <- stats::reformulate(c(cat_var, adj_vars), response = outcome_var)
  purrr::map_dfr(CO2_CAT_LEVELS, function(category_value) {
    nd <- source_df
    nd[[cat_var]] <- factor(category_value, levels = CO2_CAT_LEVELS)
    predict_marginal_prob_from_compact(
      fit_compact,
      nd,
      fallback_formula = fallback_fml,
      context = paste0("predict_prob_cat3_marginal_standardized(", outcome_var, ")")
    ) |>
      dplyr::mutate(
        category = category_value,
        co2_cat = category_value,
        n_available = attr(source_df, "n_available"),
```

```r
        standardization_sample_cap = attr(source_df, "sample_cap")
      )
  })
}

pool_marginal_standardized_predictions <- function(pred_df, group_vars,
                                                   m_total = imp_n,
                                                   min_ok_frac = common_std_min_ok_frac,
                                                   context = "marginal standardized predictions") {
  assert_is_df(pred_df, context = context)
  assert_has_cols(pred_df, c(group_vars, "imp", "eta_imp", "var_eta_imp", "p_imp"), context = context)
  min_required <- ceiling(min_ok_frac * m_total)
  pooled_df <- pred_df |>
    dplyr::group_by(dplyr::across(dplyr::all_of(group_vars))) |>
    dplyr::summarise(
      eta = mean(.data$eta_imp, na.rm = TRUE),
      ubar = mean(.data$var_eta_imp, na.rm = TRUE),
      bvar = stats::var(.data$eta_imp, na.rm = TRUE),
      p_imp_mean = mean(.data$p_imp, na.rm = TRUE),
      m_used = dplyr::n_distinct(.data$imp),
      m_total = m_total,
      n_available = stats::median(.data$n_available, na.rm = TRUE),
      n_standardized = stats::median(.data$n_standardized, na.rm = TRUE),
      standardization_sample_cap = stats::median(.data$standardization_sample_cap, na.rm = TRUE),
      .groups = "drop"
    ) |>
    dplyr::mutate(
      bvar = dplyr::if_else(is.na(.data$bvar), 0, .data$bvar),
      var_eta = .data$ubar + (1 + 1 / pmax(.data$m_used, 1)) * .data$bvar,
      SE_eta = sqrt(pmax(.data$var_eta, 0)),
      p = stats::plogis(.data$eta),
      p_LCL = stats::plogis(.data$eta - 1.96 * .data$SE_eta),
      p_UCL = stats::plogis(.data$eta + 1.96 * .data$SE_eta),
      passed_threshold = .data$m_used >= min_required,
      min_required = min_required
    )
```

```r
  failed <- pooled_df |>
    dplyr::filter(!.data$passed_threshold)
  if (nrow(failed)) {
    stop(
      context, " failed completion threshold for ",
      paste(apply(failed[, group_vars, drop = FALSE], 1, paste, collapse = "/"), collapse = "; "),
      call. = FALSE
    )
  }
  pooled_df
}

# helper: convenience wrapper for non-MI categorical probability predictions
# from a full fitted model object.
predict_cat3_prob_from_fit <- function(fit_obj, ref_df, outcome_var,
                                       cat_var = "co2_cat",
                                       adj_vars = adj_core) {
  fit_compact <- compact_cat3_fit(
    fit_obj,
    context = paste0("predict_cat3_prob_from_fit(", outcome_var, ")")
  )
  grid_df <- make_cat3_ref_grid(ref_df, cat_var = cat_var)
  pred_df <- predict_cat3_prob_from_compact(
    fit_compact,
    grid_df,
    outcome_var,
    cat_var = cat_var,
    adj_vars = adj_vars
  )
  pred_df |>
    dplyr::mutate(
      category = as.character(.data[[cat_var]]),
      co2_cat = as.character(.data[[cat_var]])
    )
}
```

```r
# --- Single-pass loop -------------------------------------------------------
cat3_outcomes <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
cat3_labels <- c(
  imv_proc = "IMV",
  niv_proc = "NIV",
  death_60d = "Death60d",
  hypercap_resp_failure = "HCRF"
)

abg_co2_obs <- subset_data$paco2[subset_data$has_abg == 1 & !is.na(subset_data$paco2)]
if (length(abg_co2_obs) < 10) stop("ABG spline: not enough observed PaCO2 values.")
q_abg <- stats::quantile(abg_co2_obs, probs = c(0.02, 0.98), na.rm = TRUE)
grid_abg_info_mi <- make_co2_grid_ref(
  "paco2",
  seq(q_abg[1], q_abg[2], length.out = SPLINE_GRID_N),
  x_ref_abg,
  ABG_CO2_REF
)
grid_abg_mi <- grid_abg_info_mi$grid
ref_idx_abg_mi <- grid_abg_info_mi$ref_idx
co2_ref_abg_mi <- grid_abg_info_mi$co2_ref

vbg_co2_obs <- subset_data$vbg_co2[subset_data$has_vbg == 1 & !is.na(subset_data$vbg_co2)]
if (length(vbg_co2_obs) < 10) stop("VBG spline: not enough observed VBG CO2 values.")
q_vbg <- stats::quantile(vbg_co2_obs, probs = c(0.02, 0.98), na.rm = TRUE)
grid_vbg_info_mi <- make_co2_grid_ref(
  "vbg_co2",
  seq(q_vbg[1], q_vbg[2], length.out = SPLINE_GRID_N),
  x_ref_vbg,
  VBG_CO2_REF
)
grid_vbg_mi <- grid_vbg_info_mi$grid
ref_idx_vbg_mi <- grid_vbg_info_mi$ref_idx
co2_ref_vbg_mi <- grid_vbg_info_mi$co2_ref

covars_use <- intersect(covars_ps, names(subset_data))
```

```r
covars_use_abg <- covars_use
covars_use_vbg <- covars_use

# MI PS covariate typing (continuous vs categorical) for transparency
mi_logistic_ps_type_file <- results_path("mi_logistic_ps_covariate_types.csv")
d_ps_tmp <- get_imp(1)
d_ps_tmp <- normalize_types(d_ps_tmp, levels_ref)
d_ps_tmp <- d_ps_tmp[, c("has_abg", "has_vbg", covars_ps), drop = FALSE]
d_ps_tmp <- droplevels_all(d_ps_tmp)
cont_vars <- covars_ps[vapply(covars_ps, ps_is_continuous, logical(1), df = d_ps_tmp)]
cat_vars_ps  <- setdiff(covars_ps, cont_vars)
mi_logistic_ps_types <- dplyr::bind_rows(
  data.frame(variable = cont_vars, type = "continuous", stringsAsFactors = FALSE),
  data.frame(variable = cat_vars_ps,  type = "categorical", stringsAsFactors = FALSE)
)
write_csv_safely(mi_logistic_ps_types, mi_logistic_ps_type_file, row_names = FALSE)
rm(d_ps_tmp, cont_vars, cat_vars_ps, mi_logistic_ps_types)
gc()
```

```
            used   (Mb) gc trigger   (Mb) limit (Mb)   max used   (Mb)
Ncells   7202493  384.7   12317788  657.9         NA   12317788  657.9
Vcells 808964705 6172.0 1209504017 9227.8      16384 1209497606 9227.8
```

```r
mi_logistic_ps_abg_list <- vector("list", imp_n)
mi_logistic_ps_vbg_list <- vector("list", imp_n)
mi_weight_dir <- results_path("mi_weights")
# These cached per-imputation weight files are scratch outputs used later in
# the render and for post-hoc debugging. Create the directory explicitly with
# fs so long Quarto runs do not depend on a silent base-R dir.create() result.
fs::dir_create(mi_weight_dir, recurse = TRUE)
if (!dir.exists(mi_weight_dir)) {
  stop("Failed to create MI weight cache directory: ", mi_weight_dir)
}
bal_rows_abg <- vector("list", imp_n)
bal_rows_vbg <- vector("list", imp_n)
cat3_fits_abg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
```

```r
cat3_fits_vbg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
mi_unweighted_cat3_fits_abg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
mi_unweighted_cat3_fits_vbg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
spline_fits_abg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
spline_fits_vbg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
mi_unweighted_spline_fits_abg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)
mi_unweighted_spline_fits_vbg <- setNames(lapply(cat3_outcomes, function(x) vector("list", imp_n)), cat3_outcomes)

cat3_diag_rows <- list()
spline_diag_rows <- list()
weight_diag_abg <- vector("list", imp_n)
weight_diag_vbg <- vector("list", imp_n)
mi_imp_perf <- vector("list", imp_n)

message("MI single-pass: running ", imp_n, " imputations sequentially.")
for (i in seq_len(imp_n)) {
  # MI single-pass workflow:
  # 1) complete one imputed dataset,
  # 2) fit ABG and VBG propensity models,
  # 3) derive one-sided IPSW weights and target-balance summaries,
  # 4) fit categorical and spline outcome models, and
  # 5) save only compact outputs needed for later pooling and reporting.
  # Per-imputation telemetry (time + memory) supports full-run troubleshooting.
  imp_label <- sprintf("imp_%03d", i)
  append_mem_snapshot("mi_single_pass", imp_label, "pre")
  mem_pre <- get_vcells_stats()
  imp_t0 <- Sys.time()
  d <- get_imp(i)

  # Fit ABG and VBG propensity models for this imputation.
  set.seed(20251206 + i)
  fit_abg <- fit_abg_one(d, i)
  set.seed(30251206 + i)
  fit_vbg <- fit_vbg_one(d, i)

  w_abg_i <- fit_abg$weights
```

```r
w_vbg_i <- fit_vbg$weights
ps_abg_i <- fit_abg$ps
ps_vbg_i <- fit_vbg$ps
w_path_abg <- file.path(mi_weight_dir, sprintf("w_abg_imp_%03d.rds", i))
w_path_vbg <- file.path(mi_weight_dir, sprintf("w_vbg_imp_%03d.rds", i))
ps_path_abg <- file.path(mi_weight_dir, sprintf("ps_abg_imp_%03d.rds", i))
ps_path_vbg <- file.path(mi_weight_dir, sprintf("ps_vbg_imp_%03d.rds", i))
# Reassert the scratch directory at write time because this chunk is long
# and we want an informative failure if the cache path disappears mid-render.
fs::dir_create(mi_weight_dir, recurse = TRUE)
if (!dir.exists(mi_weight_dir)) {
  stop("MI weight cache directory is unavailable during imputation ", i, ": ", mi_weight_dir)
}
saveRDS(w_abg_i, w_path_abg)
saveRDS(w_vbg_i, w_path_vbg)
saveRDS(ps_abg_i, ps_path_abg)
saveRDS(ps_vbg_i, ps_path_vbg)
# Unit weights intentionally reuse the same modeling code path for the
# imputed-but-unweighted analyses, making weighted vs unweighted contrasts
# differ only in the supplied weights rather than in helper logic.
w_unweighted_i <- rep(1, nrow(d))

# Persist compact PS metadata so downstream chunks can reload on demand.
# Keep an in-memory cache as well so late chunks do not depend on scratch
# files surviving the full render. The RDS files are still written because
# they are useful for diagnostics and post-hoc inspection outside the live
# notebook session.
mi_logistic_ps_abg_list[[i]] <- list(
  weights_path = w_path_abg,
  ps_path = ps_path_abg,
  weights_cache = w_abg_i,
  ps_cache = ps_abg_i,
  ipow_info = fit_abg$ipow_info,
  method = fit_abg$method,
  formula = fit_abg$formula,
  basis = fit_abg$basis,
```

```
  coef = fit_abg$coef,
  vcov_diag = fit_abg$vcov_diag,
  fit_ok = fit_abg$fit_ok,
  converged = fit_abg$converged,
  n = fit_abg$n,
  p = fit_abg$p
)
mi_logistic_ps_vbg_list[[i]] <- list(
  weights_path = w_path_vbg,
  ps_path = ps_path_vbg,
  weights_cache = w_vbg_i,
  ps_cache = ps_vbg_i,
  ipow_info = fit_vbg$ipow_info,
  method = fit_vbg$method,
  formula = fit_vbg$formula,
  basis = fit_vbg$basis,
  coef = fit_vbg$coef,
  vcov_diag = fit_vbg$vcov_diag,
  fit_ok = fit_vbg$fit_ok,
  converged = fit_vbg$converged,
  n = fit_vbg$n,
  p = fit_vbg$p
)

# Target-balance summaries for this imputation and group.
bal_rows_abg[[i]] <- target_balance_table(
  d, "has_abg", w_abg_i, covars_use_abg, levels_ref
) |>
  dplyr::mutate(group = "ABG", imp = i)
bal_rows_vbg[[i]] <- target_balance_table(
  d, "has_vbg", w_vbg_i, covars_use_vbg, levels_ref
) |>
  dplyr::mutate(group = "VBG", imp = i)

# 3-level CO2 outcome models for each endpoint (ABG and VBG).
for (outcome_var in cat3_outcomes) {
```

```r
fit_abg_cat <- fit_cat3_imp(
  d, w_abg_i, outcome_var, "paco2", "has_abg",
  ABG_CO2_LOW, ABG_CO2_HIGH, "ABG", i,
  adj_vars = adj_core,
  analysis_variant = "mi_ipw"
)
cat3_fits_abg[[outcome_var]][[i]] <- fit_abg_cat$fit
if (!is.null(fit_abg_cat$diag)) cat3_diag_rows[[length(cat3_diag_rows) + 1L]] <- fit_abg_cat$diag

fit_vbg_cat <- fit_cat3_imp(
  d, w_vbg_i, outcome_var, "vbg_co2", "has_vbg",
  VBG_CO2_LOW, VBG_CO2_HIGH, "VBG", i,
  adj_vars = adj_core,
  analysis_variant = "mi_ipw"
)
cat3_fits_vbg[[outcome_var]][[i]] <- fit_vbg_cat$fit
if (!is.null(fit_vbg_cat$diag)) cat3_diag_rows[[length(cat3_diag_rows) + 1L]] <- fit_vbg_cat$diag

fit_abg_cat_unweighted <- fit_cat3_imp(
  d, w_unweighted_i, outcome_var, "paco2", "has_abg",
  ABG_CO2_LOW, ABG_CO2_HIGH, "ABG", i,
  adj_vars = adj_core,
  analysis_variant = "mi_unweighted"
)
mi_unweighted_cat3_fits_abg[[outcome_var]][[i]] <- fit_abg_cat_unweighted$fit
if (!is.null(fit_abg_cat_unweighted$diag)) {
  cat3_diag_rows[[length(cat3_diag_rows) + 1L]] <- fit_abg_cat_unweighted$diag
}

fit_vbg_cat_unweighted <- fit_cat3_imp(
  d, w_unweighted_i, outcome_var, "vbg_co2", "has_vbg",
  VBG_CO2_LOW, VBG_CO2_HIGH, "VBG", i,
  adj_vars = adj_core,
  analysis_variant = "mi_unweighted"
)
mi_unweighted_cat3_fits_vbg[[outcome_var]][[i]] <- fit_vbg_cat_unweighted$fit
```

```
  if (!is.null(fit_vbg_cat_unweighted$diag)) {
    cat3_diag_rows[[length(cat3_diag_rows) + 1L]] <- fit_vbg_cat_unweighted$diag
  }
}

# Spline outcome models for each endpoint (ABG and VBG).
for (outcome_var in cat3_outcomes) {
  spline_abg <- fit_spline_imp(
    d, w_abg_i, outcome_var, "paco2", "has_abg",
    adj_vars = adj_core, spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS,
    grid_df = grid_abg_mi, ref_idx = ref_idx_abg_mi, imp_index = i
  )
  spline_fits_abg[[outcome_var]][[i]] <- spline_abg
  if (!is.null(spline_abg$diag)) spline_diag_rows[[length(spline_diag_rows) + 1L]] <- spline_abg$diag

  spline_vbg <- fit_spline_imp(
    d, w_vbg_i, outcome_var, "vbg_co2", "has_vbg",
    adj_vars = adj_core, spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS,
    grid_df = grid_vbg_mi, ref_idx = ref_idx_vbg_mi, imp_index = i
  )
  spline_fits_vbg[[outcome_var]][[i]] <- spline_vbg
  if (!is.null(spline_vbg$diag)) spline_diag_rows[[length(spline_diag_rows) + 1L]] <- spline_vbg$diag

  spline_abg_mi_unweighted <- fit_spline_imp(
    d, w_unweighted_i, outcome_var, "paco2", "has_abg",
    adj_vars = adj_core, spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS,
    grid_df = grid_abg_mi, ref_idx = ref_idx_abg_mi, imp_index = i,
    analysis_variant = "mi_unweighted"
  )
  mi_unweighted_spline_fits_abg[[outcome_var]][[i]] <- spline_abg_mi_unweighted
  if (!is.null(spline_abg_mi_unweighted$diag)) {
    spline_diag_rows[[length(spline_diag_rows) + 1L]] <- spline_abg_mi_unweighted$diag
  }

  spline_vbg_mi_unweighted <- fit_spline_imp(
    d, w_unweighted_i, outcome_var, "vbg_co2", "has_vbg",
```

```r
    adj_vars = adj_core, spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS,
    grid_df = grid_vbg_mi, ref_idx = ref_idx_vbg_mi, imp_index = i,
    analysis_variant = "mi_unweighted"
  )
  mi_unweighted_spline_fits_vbg[[outcome_var]][[i]] <- spline_vbg_mi_unweighted
  if (!is.null(spline_vbg_mi_unweighted$diag)) {
    spline_diag_rows[[length(spline_diag_rows) + 1L]] <- spline_vbg_mi_unweighted$diag
  }
}

# Store compact weight diagnostics used in report tables.
weight_diag_abg[[i]] <- {
  w <- w_abg_i
  t <- d$has_abg
  w <- w[t == 1]
  c(
    n = length(w),
    min = min(w, na.rm = TRUE),
    p99 = stats::quantile(w, 0.99, na.rm = TRUE),
    max = max(w, na.rm = TRUE),
    ess = sum(w)^2 / sum(w^2)
  )
}
weight_diag_vbg[[i]] <- {
  w <- w_vbg_i
  t <- d$has_vbg
  w <- w[t == 1]
  c(
    n = length(w),
    min = min(w, na.rm = TRUE),
    p99 = stats::quantile(w, 0.99, na.rm = TRUE),
    max = max(w, na.rm = TRUE),
    ess = sum(w)^2 / sum(w^2)
  )
}
```

```r
  # Save per-imputation performance diagnostics.
  mem_post <- get_vcells_stats()
  imp_t1 <- Sys.time()
  mi_imp_perf[[i]] <- data.frame(
    imp = i,
    seconds = as.numeric(difftime(imp_t1, imp_t0, units = "secs")),
    gc_vcells_used_mb_pre = mem_pre$gc_vcells_used_mb,
    gc_vcells_frac_pre = mem_pre$gc_vcells_frac,
    gc_vcells_used_mb_post = mem_post$gc_vcells_used_mb,
    gc_vcells_frac_post = mem_post$gc_vcells_frac,
    run_id = diag_run_id,
    run_ts = as.character(Sys.time()),
    stringsAsFactors = FALSE
  )
  append_mem_snapshot("mi_single_pass", imp_label, "post")

  # Aggressive cleanup inside loop to reduce peak memory pressure.
  rm(d, fit_abg, fit_vbg, w_abg_i, w_vbg_i, w_unweighted_i, ps_abg_i, ps_vbg_i,
     w_path_abg, w_path_vbg, ps_path_abg, ps_path_vbg,
     mem_pre, mem_post, imp_t0, imp_t1, imp_label)
  invisible(gc())
}

mi_imp_perf_df <- dplyr::bind_rows(mi_imp_perf)
write_csv_safely(mi_imp_perf_df, results_path("mi_single_pass_imp_diagnostics.csv"), row_names = FALSE)
saveRDS(mi_logistic_ps_abg_list, mi_logistic_ps_abg_file)
saveRDS(mi_logistic_ps_vbg_list, mi_logistic_ps_vbg_file)

# helper: retrieve one cached or on-disk MI weight vector for the requested
# cohort and imputation without rebuilding anything.
get_mi_weight <- function(group, i) {
  stopifnot(i >= 1L, i <= imp_n)
  grp <- toupper(group)
  meta <- if (grp == "ABG") {
    mi_logistic_ps_abg_list[[i]]
  } else if (grp == "VBG") {
```

```r
    mi_logistic_ps_vbg_list[[i]]
  } else {
    stop("Unknown group for get_mi_weight: ", group)
  }
  if (!is.null(meta$weights_cache)) {
    return(meta$weights_cache)
  }
  stopifnot(!is.null(meta$weights_path), file.exists(meta$weights_path))
  readRDS(meta$weights_path)
}

# helper: retrieve one cached or on-disk MI propensity-score vector for the
# requested cohort and imputation.
get_mi_ps <- function(group, i) {
  stopifnot(i >= 1L, i <= imp_n)
  grp <- toupper(group)
  meta <- if (grp == "ABG") {
    mi_logistic_ps_abg_list[[i]]
  } else if (grp == "VBG") {
    mi_logistic_ps_vbg_list[[i]]
  } else {
    stop("Unknown group for get_mi_ps: ", group)
  }
  if (!is.null(meta$ps_cache)) {
    return(meta$ps_cache)
  }
  stopifnot(!is.null(meta$ps_path), file.exists(meta$ps_path))
  readRDS(meta$ps_path)
}

append_outcome_diag(dplyr::bind_rows(cat3_diag_rows, spline_diag_rows))

bal_imp_abg <- list(
  bal_long = dplyr::bind_rows(bal_rows_abg)
)
bal_imp_abg$bal_imp_summary <- bal_imp_abg$bal_long |>
```

```r
  dplyr::group_by(group, imp) |>
  dplyr::summarise(
    max_abs_post = max(abs(smd_post), na.rm = TRUE),
    mean_abs_post = mean(abs(smd_post), na.rm = TRUE),
    .groups = "drop"
  )
bal_imp_abg$worst_rows_overall <- bal_imp_abg$bal_long |>
  dplyr::mutate(abs_post = abs(smd_post)) |>
  dplyr::arrange(desc(abs_post)) |>
  dplyr::ungroup()
bal_imp_abg$worst_by_imp <- bal_imp_abg$bal_long |>
  dplyr::mutate(term = ifelse(is.na(level), variable, paste0(variable, ":", level)),
                abs_post = abs(smd_post)) |>
  dplyr::group_by(group, imp) |>
  dplyr::slice_max(abs_post, n = 1, with_ties = FALSE) |>
  dplyr::ungroup() |>
  dplyr::select(group, imp, term, smd_pre, smd_post, abs_post)
bal_imp_abg$worst_terms_by_imp <- bal_imp_abg$bal_long |>
  dplyr::mutate(term = ifelse(is.na(level), variable, paste0(variable, ":", level)),
                abs_post = abs(smd_post)) |>
  dplyr::group_by(group, imp) |>
  dplyr::slice_max(abs_post, n = 10, with_ties = FALSE) |>
  dplyr::ungroup() |>
  dplyr::select(group, imp, term, smd_pre, smd_post, abs_post)

bal_imp_vbg <- list(
  bal_long = dplyr::bind_rows(bal_rows_vbg)
)
bal_imp_vbg$bal_imp_summary <- bal_imp_vbg$bal_long |>
  dplyr::group_by(group, imp) |>
  dplyr::summarise(
    max_abs_post = max(abs(smd_post), na.rm = TRUE),
    mean_abs_post = mean(abs(smd_post), na.rm = TRUE),
    .groups = "drop"
  )
bal_imp_vbg$worst_rows_overall <- bal_imp_vbg$bal_long |>
```

```
  dplyr::mutate(abs_post = abs(smd_post)) |>
  dplyr::arrange(desc(abs_post)) |>
  dplyr::ungroup()
bal_imp_vbg$worst_by_imp <- bal_imp_vbg$bal_long |>
  dplyr::mutate(term = ifelse(is.na(level), variable, paste0(variable, ":", level)),
                abs_post = abs(smd_post)) |>
  dplyr::group_by(group, imp) |>
  dplyr::slice_max(abs_post, n = 1, with_ties = FALSE) |>
  dplyr::ungroup() |>
  dplyr::select(group, imp, term, smd_pre, smd_post, abs_post)
bal_imp_vbg$worst_terms_by_imp <- bal_imp_vbg$bal_long |>
  dplyr::mutate(term = ifelse(is.na(level), variable, paste0(variable, ":", level)),
                abs_post = abs(smd_post)) |>
  dplyr::group_by(group, imp) |>
  dplyr::slice_max(abs_post, n = 10, with_ties = FALSE) |>
  dplyr::ungroup() |>
  dplyr::select(group, imp, term, smd_pre, smd_post, abs_post)

mi_weight_diag_abg <- dplyr::bind_rows(lapply(weight_diag_abg, function(x) {
  as.data.frame(as.list(x), stringsAsFactors = FALSE)
}))
mi_weight_diag_vbg <- dplyr::bind_rows(lapply(weight_diag_vbg, function(x) {
  as.data.frame(as.list(x), stringsAsFactors = FALSE)
}))

abg_cat_results <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_terms(cat3_fits_abg[[outcome_var]], term_prefix = "co2_cat") |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "ABG",
      OR = estimate
    )
}))
vbg_cat_results <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_terms(cat3_fits_vbg[[outcome_var]], term_prefix = "co2_cat") |>
    dplyr::mutate(
```

```r
      outcome = cat3_labels[[outcome_var]],
      group = "VBG",
      OR = estimate
    )
}))
mi_unweighted_abg_cat_results <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_terms(mi_unweighted_cat3_fits_abg[[outcome_var]], term_prefix = "co2_cat") |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "ABG",
      OR = estimate
    )
}))
mi_unweighted_vbg_cat_results <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_terms(mi_unweighted_cat3_fits_vbg[[outcome_var]], term_prefix = "co2_cat") |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "VBG",
      OR = estimate
    )
}))

abg_cat_prob_curves <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_cat3_prob_curve(cat3_fits_abg[[outcome_var]], x_ref_abg, outcome_var) |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "ABG"
    )
}))
vbg_cat_prob_curves <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_cat3_prob_curve(cat3_fits_vbg[[outcome_var]], x_ref_vbg, outcome_var) |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "VBG"
    )
}))
```

```r
mi_unweighted_abg_cat_prob_curves <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_cat3_prob_curve(mi_unweighted_cat3_fits_abg[[outcome_var]], x_ref_abg, outcome_var) |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "ABG"
    )
}))
mi_unweighted_vbg_cat_prob_curves <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_cat3_prob_curve(mi_unweighted_cat3_fits_vbg[[outcome_var]], x_ref_vbg, outcome_var) |>
    dplyr::mutate(
      outcome = cat3_labels[[outcome_var]],
      group = "VBG"
    )
}))

abg_spline <- list(
  curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_curve(spline_fits_abg[[outcome_var]], grid_abg_mi, ref_idx_abg_mi, co2_ref_abg_mi) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  })),
  prob_curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_prob_curve(spline_fits_abg[[outcome_var]], grid_abg_mi, co2_ref_abg_mi) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  })),
  coefs = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_coefs(spline_fits_abg[[outcome_var]]) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  }))
)
vbg_spline <- list(
  curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_curve(spline_fits_vbg[[outcome_var]], grid_vbg_mi, ref_idx_vbg_mi, co2_ref_vbg_mi) |>
      dplyr::mutate(outcome = outcome_var, group = "VBG")
  })),
  prob_curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_prob_curve(spline_fits_vbg[[outcome_var]], grid_vbg_mi, co2_ref_vbg_mi) |>
```

```
      dplyr::mutate(outcome = outcome_var, group = "VBG")
  })),
  coefs = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_coefs(spline_fits_vbg[[outcome_var]]) |>
      dplyr::mutate(outcome = outcome_var, group = "VBG")
  }))
)

mi_unweighted_abg_spline <- list(
  curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_curve(
      mi_unweighted_spline_fits_abg[[outcome_var]],
      grid_abg_mi,
      ref_idx_abg_mi,
      co2_ref_abg_mi
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  })),
  prob_curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_prob_curve(
      mi_unweighted_spline_fits_abg[[outcome_var]],
      grid_abg_mi,
      co2_ref_abg_mi
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  }))
)
mi_unweighted_vbg_spline <- list(
  curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_curve(
      mi_unweighted_spline_fits_vbg[[outcome_var]],
      grid_vbg_mi,
      ref_idx_vbg_mi,
      co2_ref_vbg_mi
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "VBG")
```

```
  })),
  prob_curves = dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
    pool_spline_prob_curve(
      mi_unweighted_spline_fits_vbg[[outcome_var]],
      grid_vbg_mi,
      co2_ref_vbg_mi
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "VBG")
  }))
)

abg_curves <- abg_spline$curves
abg_prob_curves <- abg_spline$prob_curves
abg_coefs  <- abg_spline$coefs
vbg_curves <- vbg_spline$curves
vbg_prob_curves <- vbg_spline$prob_curves
vbg_coefs  <- vbg_spline$coefs
mi_unweighted_abg_curves <- mi_unweighted_abg_spline$curves
mi_unweighted_abg_prob_curves <- mi_unweighted_abg_spline$prob_curves
mi_unweighted_vbg_curves <- mi_unweighted_vbg_spline$curves
mi_unweighted_vbg_prob_curves <- mi_unweighted_vbg_spline$prob_curves

abg_prob_curves_conditional_xref <- abg_prob_curves
vbg_prob_curves_conditional_xref <- vbg_prob_curves
abg_cat_prob_curves_conditional_xref <- abg_cat_prob_curves
vbg_cat_prob_curves_conditional_xref <- vbg_cat_prob_curves

standardization_validation_rows <- list()
record_standardization_status <- function(component, status, reason, artifact_path = "") {
  standardization_validation_rows[[length(standardization_validation_rows) + 1L]] <<- tibble::tibble(
    component = component,
    status = status,
    reason = reason,
    artifact_path = artifact_path,
    run_mode = RUN_MODE,
    pilot_frac = PILOT_FRAC,
```

```
    run_id = diag_run_id,
    run_ts = diag_run_ts
  )
  invisible(TRUE)
}

probability_generation_inventory <- tibble::tribble(
  ~artifact, ~analysis_family, ~current_method, ~uses_x_ref_abg, ~uses_x_ref_vbg, ~manuscript_facing,
   ↪  ~needs_conversion,
  "Figure 2", "primary_mi_ipw_spline", "conditional_modality_specific_xref", TRUE, TRUE, TRUE, TRUE,
  "Table 2", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", TRUE, TRUE, TRUE, TRUE,
  "Figure S4", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", TRUE, TRUE, TRUE, TRUE,
  "cat3_key_results_long.csv", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", TRUE, TRUE, TRUE, TRUE,
  "table_summary_key_results_cat3.csv", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", TRUE, TRUE, TRUE,
   ↪  TRUE,
  "mi_spline_prob_curve_abg.csv", "primary_mi_ipw_spline", "conditional_modality_specific_xref", TRUE, FALSE, TRUE,
   ↪  TRUE,
  "mi_spline_prob_curve_vbg.csv", "primary_mi_ipw_spline", "conditional_modality_specific_xref", FALSE, TRUE, TRUE,
   ↪  TRUE,
  "mi_cat3_prob_curve_abg.csv", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", TRUE, FALSE, TRUE, TRUE,
  "mi_cat3_prob_curve_vbg.csv", "primary_mi_ipw_cat3", "conditional_modality_specific_xref", FALSE, TRUE, TRUE, TRUE,
  "discordance_current_summary.csv", "discordance_diagnostic", "conditional_modality_specific_xref", TRUE, TRUE,
   ↪  FALSE, FALSE,
  "probability_method_comparison_summary.csv", "standardization_diagnostic", "conditional_vs_common_vs_marginal",
   ↪  TRUE, TRUE, FALSE, FALSE
)
write_csv_safely(
  probability_generation_inventory,
  results_path("probability_generation_inventory.csv"),
  row_names = FALSE
)
record_standardization_status(
  "probability_generation_inventory",
  "completed",
  "Inventory identified manuscript-facing probability outputs requiring common-source marginal standardization.",
  "Results/probability_generation_inventory.csv"
```

```
)

common_ref_profile <- make_ref_profile(subset_data, adj_core)
grid_abg_common_ref_info <- make_co2_grid_ref(
  "paco2",
  grid_abg_mi$paco2,
  common_ref_profile,
  co2_ref_abg_mi
)
grid_vbg_common_ref_info <- make_co2_grid_ref(
  "vbg_co2",
  grid_vbg_mi$vbg_co2,
  common_ref_profile,
  co2_ref_vbg_mi
)

prob_curve_common_xref_abg <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_spline_prob_curve_from_compact(
    spline_fits_abg[[outcome_var]],
    grid_df = grid_abg_common_ref_info$grid,
    outcome_var = outcome_var,
    co2_var = "paco2",
    co2_ref = co2_ref_abg_mi
  ) |>
    dplyr::mutate(outcome = outcome_var, group = "ABG", probability_method = "common_reference_profile")
}))
prob_curve_common_xref_vbg <- dplyr::bind_rows(lapply(cat3_outcomes, function(outcome_var) {
  pool_spline_prob_curve_from_compact(
    spline_fits_vbg[[outcome_var]],
    grid_df = grid_vbg_common_ref_info$grid,
    outcome_var = outcome_var,
    co2_var = "vbg_co2",
    co2_ref = co2_ref_vbg_mi
  ) |>
    dplyr::mutate(outcome = outcome_var, group = "VBG", probability_method = "common_reference_profile")
}))
```

```r
standardization_prediction_status_rows <- list()
record_standardization_prediction_status <- function(imp_index, group, outcome_var,
                                                     probability_family, status,
                                                     error_message = "",
                                                     n_available = NA_integer_,
                                                     n_standardized = NA_integer_,
                                                     n_grid = NA_integer_) {
  standardization_prediction_status_rows[[length(standardization_prediction_status_rows) + 1L]] <<- tibble::tibble(
    imp = imp_index,
    group = group,
    outcome = outcome_var,
    probability_family = probability_family,
    status = status,
    error_message = error_message,
    n_available = as.integer(n_available),
    n_standardized = as.integer(n_standardized),
    n_grid = as.integer(n_grid)
  )
  invisible(TRUE)
}

std_spline_imp_rows <- list()
std_cat3_imp_rows <- list()
for (imp_index in seq_len(imp_n)) {
  d_imp <- get_imp(imp_index)
  std_source_df <- build_common_std_source_df(d_imp, imp_index)
  n_available_i <- attr(std_source_df, "n_available")
  n_standardized_i <- nrow(std_source_df)

  for (group_label in c("ABG", "VBG")) {
    co2_var <- if (identical(group_label, "ABG")) "paco2" else "vbg_co2"
    co2_values <- if (identical(group_label, "ABG")) grid_abg_mi$paco2 else grid_vbg_mi$vbg_co2
    spline_bank <- if (identical(group_label, "ABG")) spline_fits_abg else spline_fits_vbg
    cat3_bank <- if (identical(group_label, "ABG")) cat3_fits_abg else cat3_fits_vbg
```

```r
for (outcome_var in cat3_outcomes) {
  spline_fit <- compact_spline_fit(spline_bank[[outcome_var]][[imp_index]])
  if (is.null(spline_fit)) {
    record_standardization_prediction_status(
      imp_index, group_label, outcome_var, "spline",
      "skipped_no_compact_fit",
      "compact spline fit unavailable",
      n_available_i, n_standardized_i, length(co2_values)
    )
  } else {
    spline_try <- tryCatch(
      list(
        value = predict_prob_curve_marginal_standardized(
          spline_fit,
          source_df = std_source_df,
          co2_var = co2_var,
          co2_values = co2_values,
          outcome_var = outcome_var
        ),
        error = NULL
      ),
      error = function(e) list(value = NULL, error = conditionMessage(e))
    )
    if (is.null(spline_try$error) && is.data.frame(spline_try$value) && nrow(spline_try$value)) {
      std_spline_imp_rows[[length(std_spline_imp_rows) + 1L]] <- spline_try$value |>
        dplyr::mutate(
          imp = imp_index,
          group = group_label,
          outcome = outcome_var,
          co2_scale = co2_var,
          probability_method = "common_source_marginal_standardized"
        )
      record_standardization_prediction_status(
        imp_index, group_label, outcome_var, "spline",
        "completed", "", n_available_i, n_standardized_i, length(co2_values)
      )
```

```
    } else {
    record_standardization_prediction_status(
      imp_index, group_label, outcome_var, "spline",
      "failed_prediction",
      if (is.null(spline_try$error)) "prediction returned no rows" else spline_try$error,
      n_available_i, n_standardized_i, length(co2_values)
    )
  }
  }

  cat3_fit <- cat3_bank[[outcome_var]][[imp_index]]
  if (is.null(cat3_fit) || is.null(cat3_fit$coef) || is.null(cat3_fit$vcov)) {
    record_standardization_prediction_status(
      imp_index, group_label, outcome_var, "cat3",
      "skipped_no_compact_fit",
      "compact categorical fit unavailable",
      n_available_i, n_standardized_i, length(CO2_CAT_LEVELS)
    )
  } else {
    cat3_try <- tryCatch(
      list(
        value = predict_prob_cat3_marginal_standardized(
          cat3_fit,
          source_df = std_source_df,
          outcome_var = outcome_var
        ),
        error = NULL
      ),
      error = function(e) list(value = NULL, error = conditionMessage(e))
    )
    if (is.null(cat3_try$error) && is.data.frame(cat3_try$value) && nrow(cat3_try$value)) {
      std_cat3_imp_rows[[length(std_cat3_imp_rows) + 1L]] <- cat3_try$value |>
        dplyr::mutate(
          imp = imp_index,
          group = group_label,
          outcome = outcome_var,
```

```
              probability_method = "common_source_marginal_standardized"
            )
          record_standardization_prediction_status(
            imp_index, group_label, outcome_var, "cat3",
            "completed", "", n_available_i, n_standardized_i, length(CO2_CAT_LEVELS)
          )
        } else {
        record_standardization_prediction_status(
          imp_index, group_label, outcome_var, "cat3",
          "failed_prediction",
          if (is.null(cat3_try$error)) "prediction returned no rows" else cat3_try$error,
          n_available_i, n_standardized_i, length(CO2_CAT_LEVELS)
        )
      }
      }
    }
  }

  rm(d_imp, std_source_df)
  invisible(gc(FALSE))
}

standardization_prediction_status <- dplyr::bind_rows(standardization_prediction_status_rows)
standardization_prediction_summary <- standardization_prediction_status |>
  dplyr::group_by(probability_family, group, outcome) |>
  dplyr::summarise(
    m_used = sum(.data$status == "completed", na.rm = TRUE),
    m_total = imp_n,
    min_required = ceiling(common_std_min_ok_frac * imp_n),
    m_failed = sum(.data$status == "failed_prediction", na.rm = TRUE),
    m_skipped = sum(.data$status == "skipped_no_compact_fit", na.rm = TRUE),
    first_error = {
      errors <- .data$error_message[nzchar(.data$error_message)]
      if (length(errors)) errors[[1L]] else ""
    },
    passed_threshold = .data$m_used >= .data$min_required,
```

```r
    .groups = "drop"
  )
standardization_prediction_status <- standardization_prediction_status |>
    dplyr::left_join(
    standardization_prediction_summary |>
      dplyr::select(probability_family, group, outcome,
                    m_used, m_total, min_required, passed_threshold),
    by = c("probability_family", "group", "outcome")
  )
write_csv_safely(
  standardization_prediction_status,
  results_path("standardization_prediction_status.csv"),
  row_names = FALSE
)
write_csv_safely(
  standardization_prediction_summary,
  results_path("standardization_prediction_summary.csv"),
  row_names = FALSE
)

standardization_failures <- standardization_prediction_summary |>
  dplyr::filter(!.data$passed_threshold)
if (nrow(standardization_failures)) {
  record_standardization_status(
    "marginal_prediction_completion",
    "failed",
    paste0("Common-source marginal standardization failed completion threshold for ",
           paste(paste0(standardization_failures$probability_family, "/",
                        standardization_failures$group, "/",
                        standardization_failures$outcome, " ",
                        standardization_failures$m_used, "/",
                        standardization_failures$m_total), collapse = "; ")),
    "Results/standardization_prediction_status.csv"
  )
  write_csv_safely(
    dplyr::bind_rows(standardization_validation_rows),
```

```
    results_path("standardization_validation_status.csv"),
    row_names = FALSE
  )
  stop("Common-source marginal standardization failed; see Results/standardization_prediction_status.csv")
}
record_standardization_status(
  "marginal_prediction_completion",
  "completed",
  paste0("All primary spline and categorical marginal predictions met min_ok_frac = ",
         common_std_min_ok_frac, "."),
  "Results/standardization_prediction_status.csv"
)

std_spline_imp <- dplyr::bind_rows(std_spline_imp_rows)
std_cat3_imp <- dplyr::bind_rows(std_cat3_imp_rows)
prob_curve_marginal_standardized_all <- pool_marginal_standardized_predictions(
  std_spline_imp,
  group_vars = c("group", "outcome", "co2_scale", "co2_value", "probability_method"),
  context = "primary spline common-source marginal probabilities"
) |>
  dplyr::mutate(
    co2_ref = dplyr::if_else(.data$group == "ABG", co2_ref_abg_mi, co2_ref_vbg_mi)
  )
prob_cat3_marginal_standardized_all <- pool_marginal_standardized_predictions(
  std_cat3_imp,
  group_vars = c("group", "outcome", "category", "co2_cat", "probability_method"),
  context = "primary categorical common-source marginal probabilities"
)

prob_curve_marginal_standardized_abg <- prob_curve_marginal_standardized_all |>
  dplyr::filter(.data$group == "ABG") |>
  dplyr::mutate(paco2 = .data$co2_value) |>
  dplyr::relocate(paco2, .after = co2_value)
prob_curve_marginal_standardized_vbg <- prob_curve_marginal_standardized_all |>
  dplyr::filter(.data$group == "VBG") |>
  dplyr::mutate(vbg_co2 = .data$co2_value) |>
```

```
  dplyr::relocate(vbg_co2, .after = co2_value)
prob_cat3_marginal_standardized_abg <- prob_cat3_marginal_standardized_all |>
  dplyr::filter(.data$group == "ABG")
prob_cat3_marginal_standardized_vbg <- prob_cat3_marginal_standardized_all |>
  dplyr::filter(.data$group == "VBG")

write_csv_safely(
  prob_curve_marginal_standardized_abg,
  results_path("prob_curve_marginal_standardized_abg.csv"),
  row_names = FALSE
)
write_csv_safely(
  prob_curve_marginal_standardized_vbg,
  results_path("prob_curve_marginal_standardized_vbg.csv"),
  row_names = FALSE
)
write_csv_safely(
  prob_cat3_marginal_standardized_abg,
  results_path("prob_cat3_marginal_standardized_abg.csv"),
  row_names = FALSE
)
write_csv_safely(
  prob_cat3_marginal_standardized_vbg,
  results_path("prob_cat3_marginal_standardized_vbg.csv"),
  row_names = FALSE
)
write_csv_safely(
  abg_prob_curves_conditional_xref |>
    dplyr::mutate(probability_method = "conditional_modality_specific_xref"),
  results_path("prob_curve_conditional_xref_abg.csv"),
  row_names = FALSE
)
write_csv_safely(
  vbg_prob_curves_conditional_xref |>
    dplyr::mutate(probability_method = "conditional_modality_specific_xref"),
  results_path("prob_curve_conditional_xref_vbg.csv"),
```

```
  row_names = FALSE
)
write_csv_safely(
  abg_cat_prob_curves_conditional_xref |>
    dplyr::mutate(probability_method = "conditional_modality_specific_xref"),
  results_path("prob_cat3_conditional_xref_abg.csv"),
  row_names = FALSE
)
write_csv_safely(
  vbg_cat_prob_curves_conditional_xref |>
    dplyr::mutate(probability_method = "conditional_modality_specific_xref"),
  results_path("prob_cat3_conditional_xref_vbg.csv"),
  row_names = FALSE
)

abg_prob_curves <- prob_curve_marginal_standardized_abg |>
  dplyr::select(-dplyr::any_of(c("probability_method"))) |>
  dplyr::mutate(probability_method = "common_source_marginal_standardized")
vbg_prob_curves <- prob_curve_marginal_standardized_vbg |>
  dplyr::select(-dplyr::any_of(c("probability_method"))) |>
  dplyr::mutate(probability_method = "common_source_marginal_standardized")
abg_cat_prob_curves <- prob_cat3_marginal_standardized_abg |>
  dplyr::select(-dplyr::any_of(c("probability_method"))) |>
  dplyr::mutate(probability_method = "common_source_marginal_standardized")
vbg_cat_prob_curves <- prob_cat3_marginal_standardized_vbg |>
  dplyr::select(-dplyr::any_of(c("probability_method"))) |>
  dplyr::mutate(probability_method = "common_source_marginal_standardized")

record_standardization_status(
  "marginal_standardized_outputs",
  "completed",
  paste0(
    "Generated common-source marginal standardized probability outputs with sample_cap = ",
    common_std_sample_cap,
    " and common eligible source-population covariate distribution."
  ),
```

```r
  "Results/prob_curve_marginal_standardized_abg.csv; Results/prob_curve_marginal_standardized_vbg.csv;
   ↪  Results/prob_cat3_marginal_standardized_abg.csv; Results/prob_cat3_marginal_standardized_vbg.csv"
)

probability_method_comparison_curves <- dplyr::bind_rows(
  abg_prob_curves_conditional_xref |>
    dplyr::mutate(co2_value = .data$paco2, probability_method = "conditional_modality_specific_xref"),
  vbg_prob_curves_conditional_xref |>
    dplyr::mutate(co2_value = .data$vbg_co2, probability_method = "conditional_modality_specific_xref"),
  prob_curve_common_xref_abg |>
    dplyr::mutate(co2_value = .data$paco2),
  prob_curve_common_xref_vbg |>
    dplyr::mutate(co2_value = .data$vbg_co2),
  prob_curve_marginal_standardized_abg |>
    dplyr::mutate(co2_value = .data$paco2),
  prob_curve_marginal_standardized_vbg |>
    dplyr::mutate(co2_value = .data$vbg_co2)
) |>
  dplyr::mutate(
    probability_method = factor(
      .data$probability_method,
      levels = c(
        "conditional_modality_specific_xref",
        "common_reference_profile",
        "common_source_marginal_standardized"
      ),
      labels = c(
        "Conditional modality-specific X_ref",
        "Common reference profile",
        "Common-source marginal standardized"
      )
    ),
    outcome_label = normalize_key_results_outcome(.data$outcome, context = "probability method comparison"),
    group = normalize_key_results_group(.data$group, context = "probability method comparison")
  )
write_csv_safely(
```

```r
  probability_method_comparison_curves,
  results_path("probability_method_comparison_curves.csv"),
  row_names = FALSE
)

comparison_nearest_probability <- function(method_label, group_label, outcome_value, target_value) {
  curve_df <- probability_method_comparison_curves |>
    dplyr::filter(
      as.character(.data$probability_method) == method_label,
      as.character(.data$group) == group_label,
      .data$outcome == outcome_value
    )
  if (!nrow(curve_df) || !is.finite(target_value)) return(NA_real_)
  x <- suppressWarnings(as.numeric(curve_df$co2_value))
  ok <- is.finite(x)
  if (!any(ok)) return(NA_real_)
  idx <- which(ok)[which.min(abs(x[ok] - target_value))]
  suppressWarnings(as.numeric(curve_df$p[[idx]]))
}

comparison_relative_gap <- function(abg_value, vbg_value) {
  ifelse(
    is.finite(vbg_value) & abs(vbg_value) > 0,
    (abg_value - vbg_value) / abs(vbg_value),
    NA_real_
  )
}

probability_method_comparison_summary <- tidyr::crossing(
  outcome = cat3_outcomes,
  level = c("Low", "Normal", "High"),
  probability_method = levels(probability_method_comparison_curves$probability_method)
) |>
  dplyr::rowwise() |>
  dplyr::mutate(
    abg_target = dplyr::case_when(
```

```
      level == "Low" ~ ABG_CO2_LOW,
      level == "Normal" ~ co2_ref_abg_mi,
      TRUE ~ ABG_CO2_HIGH
    ),
    vbg_target = dplyr::case_when(
      level == "Low" ~ VBG_CO2_LOW,
      level == "Normal" ~ co2_ref_vbg_mi,
      TRUE ~ VBG_CO2_HIGH
    ),
    abg_probability = comparison_nearest_probability(probability_method, "ABG", outcome, abg_target),
    vbg_probability = comparison_nearest_probability(probability_method, "VBG", outcome, vbg_target),
    absolute_gap = abg_probability - vbg_probability,
    relative_gap = comparison_relative_gap(abg_probability, vbg_probability),
    outcome_label = as.character(normalize_key_results_outcome(outcome, context = "probability comparison summary"))
  ) |>
  dplyr::ungroup() |>
  dplyr::select(
    outcome, outcome_label, level, probability_method,
    abg_target, vbg_target, abg_probability, vbg_probability,
    absolute_gap, relative_gap
  )
write_csv_safely(
  probability_method_comparison_summary,
  results_path("probability_method_comparison_summary.csv"),
  row_names = FALSE
)

probability_method_comparison_plot_df <- probability_method_comparison_curves |>
  dplyr::filter(.data$outcome %in% c("imv_proc", "niv_proc")) |>
  dplyr::mutate(
    outcome_label = factor(as.character(.data$outcome_label), levels = c("IMV", "NIV")),
    group = factor(as.character(.data$group), levels = c("ABG", "VBG"))
  )
probability_method_comparison_plot <- ggplot2::ggplot(
  probability_method_comparison_plot_df,
  ggplot2::aes(x = .data$co2_value, y = .data$p, color = .data$probability_method,
```

```r
               linetype = .data$probability_method)
) +
  ggplot2::geom_line(linewidth = 0.65) +
  ggplot2::facet_grid(outcome_label ~ group, scales = "free_x") +
  ggplot2::scale_color_brewer(palette = "Dark2") +
  ggplot2::labs(
    title = "Predicted probability methods for NIV and IMV",
    subtitle = "Conditional reference profiles vs common reference profile vs common-source marginal standardization",
    x = expression(CO[2]~"(mmHg)"),
    y = "Predicted probability",
    color = "Method",
    linetype = "Method"
  ) +
  theme_pub(base_size = 10) +
  ggplot2::theme(legend.position = "bottom")
probability_method_comparison_png <- results_path("figs", "probability_method_comparison.png")
probability_method_comparison_pdf <- results_path("figs", "probability_method_comparison.pdf")
save_diag_plot(probability_method_comparison_plot, probability_method_comparison_png, width = 8.5, height = 5.6)
save_diag_plot(probability_method_comparison_plot, probability_method_comparison_pdf, width = 8.5, height = 5.6)
register_plot_file("probability_method_comparison", probability_method_comparison_png)
```

```
[1] TRUE
```

```r
record_standardization_status(
  "probability_method_comparison",
  "completed",
  "Generated comparison curves and gap summary for conditional X_ref, common reference, and common-source marginal
   ↪ standardized probabilities.",
  "Results/probability_method_comparison_summary.csv; Results/figs/probability_method_comparison.png"
)

manuscript_probability_output_map <- probability_generation_inventory |>
  dplyr::mutate(
    old_probability_method = .data$current_method,
    new_probability_method = dplyr::if_else(
      .data$manuscript_facing & .data$needs_conversion,
```

```r
      "common_source_marginal_standardized",
      .data$current_method
    ),
    updated = .data$manuscript_facing & .data$needs_conversion
  ) |>
  dplyr::select(
    artifact, analysis_family,
    old_probability_method, new_probability_method,
    manuscript_facing, updated
  )
write_csv_safely(
  manuscript_probability_output_map,
  results_path("manuscript_probability_output_map.csv"),
  row_names = FALSE
)
record_standardization_status(
  "manuscript_probability_output_map",
  "completed",
  "Manuscript-facing primary probability outputs now map to common-source marginal standardized predictions.",
  "Results/manuscript_probability_output_map.csv"
)

write_csv_safely(abg_curves, results_path("mi_spline_curve_abg.csv"), row_names = FALSE)
write_csv_safely(abg_prob_curves, results_path("mi_spline_prob_curve_abg.csv"), row_names = FALSE)
write_csv_safely(abg_coefs,  results_path("mi_spline_coef_abg.csv"), row_names = FALSE)
write_csv_safely(vbg_curves, results_path("mi_spline_curve_vbg.csv"), row_names = FALSE)
write_csv_safely(vbg_prob_curves, results_path("mi_spline_prob_curve_vbg.csv"), row_names = FALSE)
write_csv_safely(vbg_coefs,  results_path("mi_spline_coef_vbg.csv"), row_names = FALSE)
write_csv_safely(abg_cat_prob_curves, results_path("mi_cat3_prob_curve_abg.csv"), row_names = FALSE)
write_csv_safely(vbg_cat_prob_curves, results_path("mi_cat3_prob_curve_vbg.csv"), row_names = FALSE)
write_csv_safely(
  dplyr::bind_rows(standardization_validation_rows),
  results_path("standardization_validation_status.csv"),
  row_names = FALSE
)
```

```r
mi_single_pass_t1 <- Sys.time()
stopifnot(exists("runtime_log"))
runtime_log <- dplyr::bind_rows(
  runtime_log,
  data.frame(
    step_name = "mi_single_pass",
    seconds = as.numeric(difftime(mi_single_pass_t1, mi_single_pass_t0, units = "secs")),
    start_time = as.character(mi_single_pass_t0),
    end_time = as.character(mi_single_pass_t1),
    notes = paste0("m=", imp_n),
    run_id = runtime_run_id,
    run_mode = RUN_MODE,
    n_subset = nrow(subset_data),
    stringsAsFactors = FALSE
  )
)

mi_single_pass_done <- TRUE
```

3.3.2 ABG propensity (has_abg)

```r
# Purpose: mi propensity abg.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("mi_logistic_ps_abg_list"))
message("ABG MI weights were computed in the single-pass MI loop above.")
```

```r
# Weight diagnostics for ABG IPW across imputations
stopifnot(exists("mi_weight_diag_abg"))
mi_weight_diag_abg_df <- dplyr::as_tibble(mi_weight_diag_abg)
mi_weight_diag_abg_df <- dplyr::mutate(
  mi_weight_diag_abg_df,
  dplyr::across(dplyr::everything(), ~ suppressWarnings(as.numeric(.x)))
)
mi_weight_diag_abg_sum <- mi_weight_diag_abg_df |>
  dplyr::summarise(dplyr::across(where(is.numeric), ~ stats::median(.x, na.rm = TRUE)))
render_table_pdf_maybe(
```

```r
  round(mi_weight_diag_abg_sum, 3),
  caption = "ABG MI-logistic IPSW weight diagnostics (median across imputations)",
  file_stub = "mi_weight_diag_abg_summary",
  digits = 3,
  show = FALSE
)
stopifnot(all(is.finite(mi_weight_diag_abg_df$ess)))
```

3.3.3 Balance diagnostics across imputations

```r
# Purpose: target balance helpers.
stopifnot(exists("target_balance_table"))

# Use a fixed x-axis max so ABG/VBG balance plots are directly comparable.
```

```r
# Purpose: mi balance abg.
stopifnot(exists("bal_imp_abg"))

bal_abg_pooled <- bal_imp_abg$bal_long |>
  mutate(label = ifelse(is.na(level), variable, paste0(variable, ":", level))) |>
  group_by(label) |>
  summarise(
    pre_med = median(abs(smd_pre), na.rm = TRUE),
    post_med = median(abs(smd_post), na.rm = TRUE),
    pre_mean = mean(abs(smd_pre), na.rm = TRUE),
    post_mean = mean(abs(smd_post), na.rm = TRUE),
    post_max = max(abs(smd_post), na.rm = TRUE),
    .groups = "drop"
  )

bal_abg_plot <- bal_abg_pooled |>
  mutate(label = factor(label, levels = label[order(post_med, decreasing = TRUE)])) |>
  pivot_longer(c(pre_med, post_med), names_to = "type", values_to = "smd") |>
  mutate(type = recode(type, pre_med = "Pre", post_med = "Post"))

p_abg <- ggplot(bal_abg_plot, aes(x = smd, y = label, shape = type)) +
```

```
  geom_vline(xintercept = 0.1, linetype = 2, linewidth = 0.3) +
  geom_point(size = 1.2) +
  labs(title = "MI target balance (ABG): pooled |SMD|", x = "|Target SMD|", y = NULL, shape = "Stage") +
  scale_x_continuous(limits = c(0, BAL_XLIM_MAX),
                     expand = expansion(mult = c(0, 0.02))) +
  theme_pub(base_size = 10)
save_diag_plot(p_abg, results_path("figs", "diag-mi-balance-pooled-abg.png"), width = 7, height = 6)

render_table_pdf_maybe(
  bal_imp_abg$worst_rows_overall,
  caption = "ABG MI-logistic IPSW target SMD rows across imputations (sorted by |SMD|)",
  file_stub = "abg_worst_target_smd_rows",
  digits = 3,
  show = FALSE
)

abg_imp_summary <- bal_imp_abg$bal_imp_summary |>
  summarise(
    med = median(max_abs_post, na.rm = TRUE),
    p90 = quantile(max_abs_post, 0.9, na.rm = TRUE),
    max = max(max_abs_post, na.rm = TRUE)
  )
render_table_pdf_maybe(
  abg_imp_summary,
  caption = "ABG MI-logistic IPSW max |Target SMD| summary across imputations",
  file_stub = "abg_target_smd_summary",
  digits = 3,
  show = FALSE
)
```

3.3.4 VBG propensity (has_vbg)

```
# Purpose: mi propensity vbg.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("mi_logistic_ps_vbg_list"))
message("VBG MI weights were computed in the single-pass MI loop above.")
```

```r
# Weight diagnostics for VBG IPW across imputations
stopifnot(exists("mi_weight_diag_vbg"))
mi_weight_diag_vbg_df <- dplyr::as_tibble(mi_weight_diag_vbg)
mi_weight_diag_vbg_df <- dplyr::mutate(
  mi_weight_diag_vbg_df,
  dplyr::across(dplyr::everything(), ~ suppressWarnings(as.numeric(.x)))
)
mi_weight_diag_vbg_sum <- mi_weight_diag_vbg_df |>
  dplyr::summarise(dplyr::across(where(is.numeric), ~ stats::median(.x, na.rm = TRUE)))
render_table_pdf_maybe(
  round(mi_weight_diag_vbg_sum, 3),
  caption = "VBG MI-logistic IPSW weight diagnostics (median across imputations)",
  file_stub = "mi_weight_diag_vbg_summary",
  digits = 3,
  show = FALSE
)
stopifnot(all(is.finite(mi_weight_diag_vbg_df$ess)))
```

3.3.5 VBG balance

```r
# Purpose: mi balance vbg.
stopifnot(exists("bal_imp_vbg"))

bal_vbg_pooled <- bal_imp_vbg$bal_long |>
  mutate(label = ifelse(is.na(level), variable, paste0(variable, ":", level))) |>
  group_by(label) |>
  summarise(
    pre_med = median(abs(smd_pre), na.rm = TRUE),
    post_med = median(abs(smd_post), na.rm = TRUE),
    pre_mean = mean(abs(smd_pre), na.rm = TRUE),
    post_mean = mean(abs(smd_post), na.rm = TRUE),
    post_max = max(abs(smd_post), na.rm = TRUE),
    .groups = "drop"
  )

bal_vbg_plot <- bal_vbg_pooled |>
```

```r
  mutate(label = factor(label, levels = label[order(post_med, decreasing = TRUE)])) |>
  pivot_longer(c(pre_med, post_med), names_to = "type", values_to = "smd") |>
  mutate(type = recode(type, pre_med = "Pre", post_med = "Post"))

p_vbg <- ggplot(bal_vbg_plot, aes(x = smd, y = label, shape = type)) +
  geom_vline(xintercept = 0.1, linetype = 2, linewidth = 0.3) +
  geom_point(size = 1.2) +
  labs(title = "MI target balance (VBG): pooled |SMD|", x = "|Target SMD|", y = NULL, shape = "Stage") +
  scale_x_continuous(limits = c(0, BAL_XLIM_MAX),
                     expand = expansion(mult = c(0, 0.02))) +
  theme_pub(base_size = 10)
save_diag_plot(p_vbg, results_path("figs", "diag-mi-balance-pooled-vbg.png"), width = 7, height = 6)

render_table_pdf_maybe(
  bal_imp_vbg$worst_rows_overall,
  caption = "VBG MI-logistic IPSW target SMD rows across imputations (sorted by |SMD|)",
  file_stub = "vbg_worst_target_smd_rows",
  digits = 3,
  show = FALSE
)

vbg_imp_summary <- bal_imp_vbg$bal_imp_summary |>
  summarise(
    med = median(max_abs_post, na.rm = TRUE),
    p90 = quantile(max_abs_post, 0.9, na.rm = TRUE),
    max = max(max_abs_post, na.rm = TRUE)
  )
render_table_pdf_maybe(
  vbg_imp_summary,
  caption = "VBG MI-logistic IPSW max |Target SMD| summary across imputations",
  file_stub = "vbg_target_smd_summary",
  digits = 3,
  show = FALSE
)
```

3.4 Weighted outcome models within each imputation + pooling

Within each imputation, fit covariate-adjusted CO2 spline outcome models **only in the measured cohort** (has_abg==1 for PaCO2; has_vbg==1 for VBG CO2), using IPSW weights to address nonrandom testing. Curves are pooled pointwise across imputations (Rubin's rules on the log-OR scale) and displayed as odds ratios relative to CO2_ref at a reference covariate profile.

3.4.1 Helper: fit + extract log-OR and SE from svyglm

```
# Purpose: mi pool helpers.
stopifnot(exists("fit_spline_imp"), exists("pool_spline_curve"),
          exists("pool_spline_prob_curve"),
          exists("pool_spline_coefs"), exists("pool_terms"))
```

```
# Purpose: mi abg outcomes.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("abg_spline"), exists("abg_curves"), exists("abg_prob_curves"), exists("abg_coefs"))
message("ABG MI spline results were computed in the single-pass MI loop above.")
```

3.4.2 VBG: MI pooled spline models (treated cohort only)

```
# Purpose: mi vbg outcomes.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("vbg_spline"), exists("vbg_curves"), exists("vbg_prob_curves"), exists("vbg_coefs"))
message("VBG MI spline results were computed in the single-pass MI loop above.")
```

3.5 Explainability

```
# Purpose: shap top10 mi logistic.
# MI logistic models are linear on the log-odds scale, so we compute
# contribution magnitudes from centered design-matrix terms and aggregate
# to parent features.
stopifnot(exists("mi_logistic_ps_abg_list"), exists("mi_logistic_ps_vbg_list"))

mi_shap_abg_file <- results_path("shap_top10_mi_logistic_abg.csv")
mi_shap_vbg_file <- results_path("shap_top10_mi_logistic_vbg.csv")
```

```r
mi_shap_stability_file <- results_path("diag-ps-shap-stability.csv")
mi_shap_error_file <- results_path("diag_mi_shap_errors.csv")
mi_shap_method_file <- results_path("diag_mi_shap_method_used.csv")
mi_shap_diag_file <- results_path("mi_shap_imp_diagnostics.csv")

# helper: return an empty MI SHAP summary with the same schema as the populated
# outputs so downstream code can stay simple.
make_empty_mi_shap <- function() {
  data.frame(feature = character(), mean_abs_shap = numeric(), stringsAsFactors = FALSE)
}

# helper: approximate per-feature contribution magnitudes for one MI logistic
# propensity model using a sampled model matrix on the link scale.
calc_mi_glm_contrib <- function(meta, d_ps, feature_names, imp_index, group_label,
                                sample_n = MI_SHAP_SAMPLE_N,
                                max_terms = MI_SHAP_MAX_TERMS) {
  # Memory-safe contribution approximation for GLM PS:
  # sample rows first, build sparse model matrix, then aggregate |contribution|
  # by feature without constructing a full-N dense matrix.
  diag_row <- data.frame(
    imp = imp_index,
    group = group_label,
    sample_n = NA_integer_,
    n_terms_used = NA_integer_,
    seconds = NA_real_,
    vcells_pre_mb = NA_real_,
    vcells_post_mb = NA_real_,
    method_used = "sparse_sampled_glm_link",
    ok = FALSE,
    error_msg = NA_character_,
    stringsAsFactors = FALSE
  )
  t0 <- Sys.time()
  mem_pre <- get_vcells_stats()
  diag_row$vcells_pre_mb <- mem_pre$gc_vcells_used_mb
```

```r
err_msg <- NA_character_
out <- tryCatch({
  if (is.null(meta$formula) || is.null(meta$coef) || !length(meta$coef)) {
    stop("Missing formula/coefficients in MI PS metadata.")
  }

  n_all <- nrow(d_ps)
  n_use <- min(as.integer(sample_n), n_all)
  if (!is.finite(n_use) || n_use < 2L) {
    stop("Insufficient rows available for MI SHAP sample.")
  }
  seed_offset <- if (toupper(group_label) == "ABG") 10000L else 20000L
  set.seed(20251206L + as.integer(imp_index) + seed_offset)
  idx <- if (n_all > n_use) sample.int(n_all, size = n_use, replace = FALSE) else seq_len(n_all)
  d_sample <- d_ps[idx, , drop = FALSE]
  diag_row$sample_n <- n_use

  form <- stats::as.formula(meta$formula)
  mm_model <- NULL
  mm_mode <- "dense_model_matrix"
  if (requireNamespace("Matrix", quietly = TRUE)) {
    mm_sparse_try <- tryCatch(
      Matrix::sparse.model.matrix(form, data = d_sample),
      error = function(e) NULL
    )
    if (!is.null(mm_sparse_try)) {
      mm_model <- mm_sparse_try
      mm_mode <- "sparse_sampled_glm_link"
    }
  }
  if (is.null(mm_model)) {
    mm_model <- stats::model.matrix(form, data = d_sample)
    mm_mode <- "dense_sampled_glm_link"
  }
  diag_row$method_used <- mm_mode
```

```r
coef_vec <- meta$coef
canon_term <- function(x) {
  x <- gsub("`", "", x, fixed = TRUE)
  gsub("\\s+", "", x, perl = TRUE)
}
mm_terms <- colnames(mm_model)
coef_terms <- names(coef_vec)
mm_key <- canon_term(mm_terms)
coef_key <- canon_term(coef_terms)
coef_pos <- setNames(seq_along(coef_terms), coef_key)
keep_mm <- which(mm_key %in% names(coef_pos) & mm_terms != "(Intercept)")
if (!length(keep_mm)) {
  stop("No overlapping terms between sparse design matrix and coefficient vector.")
}

mm_model <- mm_model[, keep_mm, drop = FALSE]
diag_row$n_terms_used <- ncol(mm_model)
if (is.finite(max_terms) && ncol(mm_model) > as.integer(max_terms)) {
  stop(
    "MI SHAP term count (", ncol(mm_model), ") exceeded MI_SHAP_MAX_TERMS (",
    as.integer(max_terms), ")."
  )
}

mm_terms_kept <- colnames(mm_model)
mm_keys_kept <- canon_term(mm_terms_kept)
coef_idx <- coef_pos[mm_keys_kept]
beta <- as.numeric(coef_vec[coef_idx])
names(beta) <- mm_terms_kept
mu <- colMeans(mm_model, na.rm = TRUE)
abs_beta <- abs(beta)
term_abs <- vapply(
  seq_len(ncol(mm_model)),
  function(j) {
    xj <- mm_model[, j]
    if (!is.numeric(xj)) xj <- as.numeric(xj)
```

```
      mean(abs(xj - mu[j]), na.rm = TRUE) * abs_beta[j]
    },
    numeric(1)
  )
  names(term_abs) <- colnames(mm_model)
  rm(mm_model, mu, abs_beta)
  invisible(gc())

  out_tbl <- data.frame(
    term = names(term_abs),
    mean_abs = as.numeric(term_abs),
    stringsAsFactors = FALSE
  ) |>
    dplyr::mutate(feature = vapply(term, term_to_parent_feature, character(1), feature_names = feature_names)) |>
    dplyr::filter(!is.na(feature)) |>
    dplyr::group_by(feature) |>
    dplyr::summarise(mean_abs_shap = sum(mean_abs, na.rm = TRUE), .groups = "drop")
  if (!nrow(out_tbl)) {
    stop("No feature-mapped terms remained after contribution aggregation.")
  }
  out_tbl
}, error = function(e) {
  err_msg <<- conditionMessage(e)
  make_empty_mi_shap()
})

mem_post <- get_vcells_stats()
diag_row$vcells_post_mb <- mem_post$gc_vcells_used_mb
diag_row$seconds <- as.numeric(difftime(Sys.time(), t0, units = "secs"))
diag_row$ok <- nrow(out) > 0
if (!isTRUE(diag_row$ok) && is.na(diag_row$error_msg)) {
  diag_row$error_msg <- err_msg
}

list(data = out, diag = diag_row)
}
```

```r
# helper: fall back to absolute pooled coefficient magnitudes when sampled
# contribution calculations fail or are intentionally skipped.
calc_mi_coef_fallback <- function(meta_list, feature_names, group_label, top_n = SHAP_TOP_N) {
  # Fallback ranking using absolute coefficients if contribution matrix
  # cannot be computed (keeps figure/table generation robust).
  coef_rows <- lapply(seq_along(meta_list), function(i) {
    coef_vec <- meta_list[[i]]$coef
    if (is.null(coef_vec) || !length(coef_vec)) return(NULL)
    data.frame(
      imp = i,
      term = names(coef_vec),
      mean_abs = abs(as.numeric(coef_vec)),
      stringsAsFactors = FALSE
    )
  })
  coef_df <- dplyr::bind_rows(coef_rows)
  if (!nrow(coef_df)) {
    return(data.frame(feature = character(), mean_abs_shap = numeric(), group = character(), stringsAsFactors =
     ↪  FALSE))
  }
  coef_df |>
    dplyr::filter(term != "(Intercept)") |>
    dplyr::mutate(feature = vapply(term, term_to_parent_feature, character(1), feature_names = feature_names)) |>
    dplyr::filter(!is.na(feature)) |>
    dplyr::group_by(feature) |>
    dplyr::summarise(mean_abs_shap = mean(mean_abs, na.rm = TRUE), .groups = "drop") |>
    dplyr::arrange(dplyr::desc(mean_abs_shap)) |>
    dplyr::slice_head(n = top_n) |>
    dplyr::mutate(group = group_label)
}

if (RUN_SHAP) {
  # Streaming accumulator avoids storing per-imputation SHAP tables in memory.
  update_feature_accum <- function(accum, contrib_df) {
    if (!nrow(contrib_df)) return(accum)
```

```r
  vals <- stats::setNames(contrib_df$mean_abs_shap, contrib_df$feature)
  feats <- names(vals)
  new_feats <- setdiff(feats, names(accum$sum))
  if (length(new_feats)) {
    accum$sum[new_feats] <- 0
    accum$count[new_feats] <- 0
  }
  accum$sum[feats] <- accum$sum[feats] + vals
  accum$count[feats] <- accum$count[feats] + 1L
  accum
}
finalize_feature_accum <- function(accum, group_label) {
  if (!length(accum$sum)) {
    return(data.frame(feature = character(), mean_abs_shap = numeric(), group = character(), stringsAsFactors =
     ↪  FALSE))
  }
  keep <- accum$count > 0
  data.frame(
    feature = names(accum$sum)[keep],
    mean_abs_shap = as.numeric(accum$sum[keep] / accum$count[keep]),
    group = group_label,
    stringsAsFactors = FALSE
  ) |>
    dplyr::arrange(dplyr::desc(mean_abs_shap)) |>
    dplyr::slice_head(n = SHAP_TOP_N)
}

accum_abg <- list(sum = numeric(0), count = numeric(0))
accum_vbg <- list(sum = numeric(0), count = numeric(0))
mi_shap_diag_rows <- list()
mi_shap_error_rows <- list()

for (i in seq_len(imp$m)) {
  d_i <- get_imp(i)
  d_ps <- d_i[, c("has_abg", "has_vbg", covars_ps), drop = FALSE]
  d_ps <- normalize_types(d_ps, levels_ref)
```

```r
  d_ps <- droplevels_all(d_ps)

  abg_res <- calc_mi_glm_contrib(
    mi_logistic_ps_abg_list[[i]], d_ps, covars_ps,
    imp_index = i, group_label = "ABG",
    sample_n = MI_SHAP_SAMPLE_N, max_terms = MI_SHAP_MAX_TERMS
  )
  mi_shap_diag_rows[[length(mi_shap_diag_rows) + 1L]] <- abg_res$diag
  if (!isTRUE(abg_res$diag$ok)) {
    mi_shap_error_rows[[length(mi_shap_error_rows) + 1L]] <- data.frame(
      imp = i, group = "ABG", error_msg = abg_res$diag$error_msg,
      stringsAsFactors = FALSE
    )
  }
  accum_abg <- update_feature_accum(accum_abg, abg_res$data)

  vbg_res <- calc_mi_glm_contrib(
    mi_logistic_ps_vbg_list[[i]], d_ps, covars_ps,
    imp_index = i, group_label = "VBG",
    sample_n = MI_SHAP_SAMPLE_N, max_terms = MI_SHAP_MAX_TERMS
  )
  mi_shap_diag_rows[[length(mi_shap_diag_rows) + 1L]] <- vbg_res$diag
  if (!isTRUE(vbg_res$diag$ok)) {
    mi_shap_error_rows[[length(mi_shap_error_rows) + 1L]] <- data.frame(
      imp = i, group = "VBG", error_msg = vbg_res$diag$error_msg,
      stringsAsFactors = FALSE
    )
  }
  accum_vbg <- update_feature_accum(accum_vbg, vbg_res$data)

  rm(d_i, d_ps, abg_res, vbg_res)
  invisible(gc())
}

mi_shap_top_abg <- finalize_feature_accum(accum_abg, "ABG")
mi_shap_top_vbg <- finalize_feature_accum(accum_vbg, "VBG")
```

```r
used_method_abg <- "sparse_sampled_glm_link"
used_method_vbg <- "sparse_sampled_glm_link"
if (nrow(mi_shap_top_abg) == 0L && isTRUE(MI_SHAP_FAIL_OPEN)) {
  mi_shap_top_abg <- calc_mi_coef_fallback(mi_logistic_ps_abg_list, covars_ps, "ABG")
  used_method_abg <- "coef_fallback"
}
if (nrow(mi_shap_top_vbg) == 0L && isTRUE(MI_SHAP_FAIL_OPEN)) {
  mi_shap_top_vbg <- calc_mi_coef_fallback(mi_logistic_ps_vbg_list, covars_ps, "VBG")
  used_method_vbg <- "coef_fallback"
}

mi_shap_diag_df <- dplyr::bind_rows(mi_shap_diag_rows)
mi_shap_error_df <- dplyr::bind_rows(mi_shap_error_rows)
infer_primary_method <- function(diag_df, grp, fallback_method) {
  if (identical(fallback_method, "coef_fallback")) return("coef_fallback")
  diag_grp <- diag_df |>
    dplyr::filter(group == grp, ok %in% TRUE, !is.na(method_used), nzchar(method_used))
  if (!nrow(diag_grp)) return(fallback_method)
  names(sort(table(diag_grp$method_used), decreasing = TRUE))[1]
}
used_method_abg <- infer_primary_method(mi_shap_diag_df, "ABG", used_method_abg)
used_method_vbg <- infer_primary_method(mi_shap_diag_df, "VBG", used_method_vbg)
mi_shap_method_df <- data.frame(
  group = c("ABG", "VBG"),
  method_used = c(used_method_abg, used_method_vbg),
  n_success = c(
    sum(mi_shap_diag_df$group == "ABG" & (mi_shap_diag_df$ok %in% TRUE), na.rm = TRUE),
    sum(mi_shap_diag_df$group == "VBG" & (mi_shap_diag_df$ok %in% TRUE), na.rm = TRUE)
  ),
  fallback_used = c(used_method_abg == "coef_fallback", used_method_vbg == "coef_fallback"),
  stringsAsFactors = FALSE
)

write_csv_safely(mi_shap_diag_df, mi_shap_diag_file, row_names = FALSE)
write_csv_safely(mi_shap_error_df, mi_shap_error_file, row_names = FALSE)
```

```r
  write_csv_safely(mi_shap_method_df, mi_shap_method_file, row_names = FALSE)
  write_csv_safely(mi_shap_top_abg, mi_shap_abg_file, row_names = FALSE)
  write_csv_safely(mi_shap_top_vbg, mi_shap_vbg_file, row_names = FALSE)

  mi_shap_stability <- dplyr::bind_rows(mi_shap_top_abg, mi_shap_top_vbg) |>
    dplyr::transmute(
      cohort = group,
      feature = feature,
      mean_abs_seed1 = mean_abs_shap,
      mean_abs_seed2 = mean_abs_shap,
      run_id = diag_run_id
    )
  write_csv_safely(mi_shap_stability, mi_shap_stability_file, row_names = FALSE)

  mi_shap_plot_df <- dplyr::bind_rows(mi_shap_top_abg, mi_shap_top_vbg)
  if (nrow(mi_shap_plot_df) > 0) {
    p_shap_mi <- plot_shap_top10_two_panel(
      mi_shap_plot_df,
      "MI logistic IPSW: top 10 features by mean absolute contribution",
      "Mean absolute contribution on link scale"
    )
    shap_file <- results_path("figs", "shap-top10-mi-logistic-abg-vbg.png")
    save_diag_plot(p_shap_mi, shap_file, width = 9, height = 5, dpi = 300)
    register_plot_file("shap-top10-mi-logistic-abg-vbg", shap_file)
  }

  rm(
    accum_abg, accum_vbg, mi_shap_diag_rows, mi_shap_error_rows,
    mi_shap_diag_df, mi_shap_error_df, mi_shap_method_df,
    mi_shap_plot_df, mi_shap_stability, mi_shap_top_abg, mi_shap_top_vbg
  )
  invisible(gc())
} else {
  write_csv_safely(data.frame(), mi_shap_diag_file, row_names = FALSE)
  write_csv_safely(data.frame(), mi_shap_error_file, row_names = FALSE)
  write_csv_safely(data.frame(), mi_shap_method_file, row_names = FALSE)
```

```r
  write_csv_safely(data.frame(), mi_shap_abg_file, row_names = FALSE)
  write_csv_safely(data.frame(), mi_shap_vbg_file, row_names = FALSE)
  write_csv_safely(data.frame(), mi_shap_stability_file, row_names = FALSE)
}
```

3.6 MI three-level PCO2 helpers and checks

```r
# Purpose: chunk.
stopifnot(exists("pool_terms"))
message("MI 3-level helpers defined in the single-pass MI section.")
```

```r
# Three-level CO2 category sanity: per-imputation overlap for ABG/VBG
stopifnot(exists("imp"))
stopifnot(exists("mi_logistic_ps_abg_list"), exists("mi_logistic_ps_vbg_list"))
imp_n <- imp$m

# helper: verify that each imputation still has at least two observed CO2
# categories within the tested cohort before categorical modeling proceeds.
check_cat <- function(d, co2, grp, low_cut, high_cut) {
  x   <- suppressWarnings(as.numeric(d[[co2]]))
  fac <- make_co2_cat3(x, low_cut, high_cut)
  tab <- table(fac, useNA = "ifany")
  message(grp, ": ", paste(names(tab), tab, collapse = " | "))
  obs_levels <- names(tab)[!is.na(names(tab))]
  stopifnot(sum(tab[obs_levels] > 0) >= 2)
}

invisible(lapply(seq_len(imp_n), function(i) {
  di <- get_imp(i)
  stopifnot("paco2" %in% names(di))
  check_cat(di[di$has_abg == 1, , drop = FALSE], "paco2",
            sprintf("ABG imp %d", i), low_cut = ABG_CO2_LOW, high_cut = ABG_CO2_HIGH)
}))

invisible(lapply(seq_len(imp_n), function(i) {
  di <- get_imp(i)
```

```
  stopifnot("vbg_co2" %in% names(di))
  check_cat(di[di$has_vbg == 1, , drop = FALSE], "vbg_co2",
            sprintf("VBG imp %d", i), low_cut = VBG_CO2_LOW, high_cut = VBG_CO2_HIGH)
}))
```

3.7 MI + IPW three-level PCO2 (ABG & VBG)

3.7.1 ABG: MI + IPW, three-level PCO2 outcomes

```
# Purpose: chunk.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("abg_cat_results"))

mi_abg_cat_forms <- list(
  "MI IPW ABG 3-level: IMV ~ CO2 category + X"      = reformulate(c("co2_cat", adj_core), response = "imv_proc"),
  "MI IPW ABG 3-level: NIV ~ CO2 category + X"      = reformulate(c("co2_cat", adj_core), response = "niv_proc"),
  "MI IPW ABG 3-level: Death60d ~ CO2 category + X" = reformulate(c("co2_cat", adj_core), response = "death_60d"),
  "MI IPW ABG 3-level: HCRF ~ CO2 category + X"     = reformulate(c("co2_cat", adj_core), response =
   ↪  "hypercap_resp_failure")
)
register_model_diagrams(mi_abg_cat_forms)
message("ABG MI 3-level results were computed in the single-pass MI loop above.")
```

3.7.2 VBG: MI + IPW, three-level PCO2 outcomes

```
# Purpose: chunk.
stopifnot(exists("mi_single_pass_done"), isTRUE(mi_single_pass_done))
stopifnot(exists("vbg_cat_results"))

mi_vbg_cat_forms <- list(
  "MI IPW VBG 3-level: IMV ~ CO2 category + X"      = reformulate(c("co2_cat", adj_core), response = "imv_proc"),
  "MI IPW VBG 3-level: NIV ~ CO2 category + X"      = reformulate(c("co2_cat", adj_core), response = "niv_proc"),
  "MI IPW VBG 3-level: Death60d ~ CO2 category + X" = reformulate(c("co2_cat", adj_core), response = "death_60d"),
  "MI IPW VBG 3-level: HCRF ~ CO2 category + X"     = reformulate(c("co2_cat", adj_core), response =
   ↪  "hypercap_resp_failure")
```

```
)
register_model_diagrams(mi_vbg_cat_forms)
message("VBG MI 3-level results were computed in the single-pass MI loop above.")
```

3.7.3 MI-pooled IPW associations (3-level CO2)

```
# Purpose: table3 ipw mi co2.
stopifnot(exists("abg_cat_results"), exists("vbg_cat_results"))

mi_threelevel_results <- dplyr::bind_rows(
  dplyr::mutate(abg_cat_results, group = "ABG"),
  dplyr::mutate(vbg_cat_results, group = "VBG")
) |>
  dplyr::mutate(method = "IPW + MI adjusted")
write_csv_safely(mi_threelevel_results, results_path("mi_threelevel_results.csv"), row_names = FALSE)

table3_df <- dplyr::bind_rows(
  dplyr::mutate(abg_cat_results, group = "ABG"),
  dplyr::mutate(vbg_cat_results, group = "VBG")
) |>
  dplyr::mutate(
    exposure = gsub("^co2_cat", "", term),
    contrast = dplyr::recode(exposure,
                             Low  = "Low vs normal",
                             High = "High vs normal",
                             .default    = exposure),
    outcome_label = dplyr::recode(
      outcome,
      IMV      = "IMV",
      NIV      = "NIV",
      Death60d = "Death (60d)",
      HCRF     = "Hypercapnic RF"
    ),
    or_ci = sprintf("%.2f (%.2f, %.2f)", OR, LCL, UCL)
  ) |>
  dplyr::select(group, outcome_label, contrast, or_ci)
```

```r
table3_wide <- table3_df |>
  tidyr::pivot_wider(names_from = contrast, values_from = or_ci) |>
  dplyr::arrange(
    factor(group, levels = c("ABG", "VBG")),
    factor(outcome_label, levels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF"))
  )

gt::gt(table3_wide) |>
  gt::tab_header(title = "MI-pooled MI-logistic IPSW associations between CO2 category and outcomes (adjusted)") |>
  gt::cols_label(
    group           = "Cohort",
    outcome_label   = "Outcome",
    `Low vs normal` = "Low vs normal OR (95% CI)",
    `High vs normal`= "High vs normal OR (95% CI)"
  ) |>
  gt::fmt_missing(columns = gt::everything(), missing_text = "-") |>
  gt::tab_source_note(
    paste0(
      "Weighted survey GLMs adjusted for baseline covariates; weights = MI-specific GLM (RCS) IPW; m = ", M_IMP,
      " imputations (seed ", MI_SEED, "); reference = Normal."
    )
  )
```

3.7.4 Summary: adjusted CO2-category associations across analysis tracks

```r
# Purpose: table summary adjusted threelevel.
# Build the canonical categorical key-results table across all requested tracks,
# while retaining the narrower adjusted-only legacy summary outputs.
label_outcome <- function(x) {
  dplyr::recode(
    x,
    imv_proc = "IMV",
    niv_proc = "NIV",
    death_60d = "Death (60d)",
    hypercap_resp_failure = "Hypercapnic RF",
```

MI-pooled MI-logistic IPSW associations between CO2 category and outcomes (adjusted)

Cohort	Outcome	Low vs normal OR (95% CI)	High vs normal OR (95% CI)
ABG	IMV	1.38 (1.30, 1.46)	1.33 (1.25, 1.41)
ABG	NIV	0.98 (0.89, 1.09)	2.62 (2.41, 2.85)
ABG	Death (60d)	1.90 (1.77, 2.03)	1.38 (1.28, 1.48)
ABG	Hypercapnic RF	0.95 (0.84, 1.07)	6.39 (5.86, 6.97)
VBG	IMV	1.39 (1.28, 1.52)	1.56 (1.43, 1.70)
VBG	NIV	0.97 (0.84, 1.11)	2.85 (2.55, 3.19)
VBG	Death (60d)	1.74 (1.59, 1.91)	1.37 (1.25, 1.50)
VBG	Hypercapnic RF	0.88 (0.75, 1.03)	8.13 (7.23, 9.14)

Weighted survey GLMs adjusted for baseline covariates; weights = MI-specific GLM (RCS) IPW; m = 20 imputations (seed 20251206); reference = Normal.

```
    IMV = "IMV",
    NIV = "NIV",
    Death60d = "Death (60d)",
    HCRF = "Hypercapnic RF"
  )
}

# helper: format one categorical OR and confidence interval for compact table
# display.
format_cat3_ci <- function(estimate, lcl, ucl) {
  sprintf("%.2f (%.2f, %.2f)", estimate, lcl, ucl)
}

# helper: count the effective modeled sample and number of events for one
# complete-case categorical model specification.
count_model <- function(data, outcome, exposure, adj_vars = NULL,
                        treat_var = NULL, weight_var = NULL) {
  d <- data
```

```r
  if (!is.null(treat_var)) d <- d[d[[treat_var]] == 1, , drop = FALSE]
  vars <- unique(c(outcome, exposure, adj_vars, weight_var))
  if (!is.null(weight_var)) vars <- c(vars, weight_var)
  d <- d[, vars, drop = FALSE]
  d <- d[complete.cases(d), , drop = FALSE]
  if (!is.null(weight_var)) {
    w <- suppressWarnings(as.numeric(d[[weight_var]]))
    d <- d[is.finite(w) & w > 0, , drop = FALSE]
  }
  if (nrow(d) == 0L) {
    return(tibble::tibble(n_model = 0L, events = 0L))
  }
  d[[outcome]] <- to01(d[[outcome]])
  tibble::tibble(
    n_model = nrow(d),
    events = sum(d[[outcome]] == 1, na.rm = TRUE)
  )
}

# helper: count complete-case modeled rows/events for the non-MI categorical
# tracks so the summary table reports the right denominator for each cohort.
count_group <- function(group, method) {
  exp_var   <- if (group == "ABG") "pco2_cat_abg" else "pco2_cat_vbg"
  treat_var <- if (group == "ABG") "has_abg" else "has_vbg"
  adj_vars <- switch(
    method,
    "Crude" = NULL,
    "Adjusted" = adj_core,
    "GBM-weighted" = adj_core,
    stop("Unsupported non-MI count method: ", method)
  )
  weight_var <- switch(
    method,
    "Crude" = NULL,
    "Adjusted" = NULL,
    "GBM-weighted" = if (group == "ABG") "w_abg" else "w_vbg",
```

```
    stop("Unsupported non-MI count method: ", method)
  )
  outcomes <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
  purrr::map_dfr(outcomes, function(outcome) {
    cnt <- count_model(subset_data, outcome, exp_var, adj_vars,
                       treat_var = treat_var, weight_var = weight_var)
    dplyr::mutate(cnt,
                  method = method,
                  group = group,
                  outcome = label_outcome(outcome))
  })
}

# helper: return zero-row MI count summaries for one cohort when an imputation
# contributes no analyzable tested encounters after the required filtering.
empty_mi_count_rows <- function(group, outcomes) {
  tidyr::expand_grid(
    method = c("Imputed", "Weighted and adjusted"),
    outcome = label_outcome(outcomes)
  ) |>
    dplyr::mutate(
      group = group,
      n_model = 0L,
      events = 0L
    ) |>
    dplyr::relocate(method, group, outcome, n_model, events)
}

# helper: compute MI modeled counts for both the imputed-only and weighted MI
# categorical tracks in one pass for one cohort within one imputed dataset.
# This avoids reloading the same full imputation hundreds of times inside the
# manuscript table summary chunk.
compute_mi_count_group_all_methods <- function(d, group, weights, outcomes) {
  if (group == "ABG") {
    co2_var <- "paco2"
    treat_var <- "has_abg"
```

```r
  low_cut <- ABG_CO2_LOW
  high_cut <- ABG_CO2_HIGH
} else {
  co2_var <- "vbg_co2"
  treat_var <- "has_vbg"
  low_cut <- VBG_CO2_LOW
  high_cut <- VBG_CO2_HIGH
}

d[[co2_var]] <- suppressWarnings(as.numeric(d[[co2_var]]))
keep <- d[[treat_var]] == 1 & is.finite(d[[co2_var]])
if (!any(keep)) {
  return(empty_mi_count_rows(group, outcomes))
}

d2 <- d[keep, c(outcomes, adj_core, co2_var), drop = FALSE]
w2 <- weights[keep]
d2$co2_cat <- make_co2_cat3(d2[[co2_var]], low_cut, high_cut)

keep2 <- !is.na(d2$co2_cat)
d2 <- d2[keep2, , drop = FALSE]
w2 <- w2[keep2]
if (nrow(d2) == 0L) {
  return(empty_mi_count_rows(group, outcomes))
}

purrr::map_dfr(outcomes, function(outcome) {
  outcome_complete <- complete.cases(d2[, c(outcome, adj_core, "co2_cat"), drop = FALSE])
  outcome_vals <- to01(d2[[outcome]])
  weighted_keep <- outcome_complete & is.finite(w2) & w2 > 0

  tibble::tibble(
    method = c("Imputed", "Weighted and adjusted"),
    group = group,
    outcome = label_outcome(outcome),
    n_model = c(
```

```
        sum(outcome_complete),
        sum(weighted_keep)
      ),
      events = c(
        sum(outcome_vals[outcome_complete] == 1, na.rm = TRUE),
        sum(outcome_vals[weighted_keep] == 1, na.rm = TRUE)
      )
    )
  })
}

# helper: summarize MI modeled sample size and events across imputations for all
# categorical outcomes and both cohorts using a single pass over each imputed
# dataset. This preserves the original median-across-imputations definition
# while removing a major full-run bottleneck.
build_mi_counts_cat3 <- function() {
  outcomes <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
  per_imp_counts <- vector("list", imp$m)

  for (i in seq_len(imp$m)) {
    d <- get_imp(i)
    w_abg <- get_mi_weight("ABG", i)
    w_vbg <- get_mi_weight("VBG", i)

    per_imp_counts[[i]] <- dplyr::bind_rows(
      compute_mi_count_group_all_methods(d, "ABG", w_abg, outcomes),
      compute_mi_count_group_all_methods(d, "VBG", w_vbg, outcomes)
    ) |>
      dplyr::mutate(imp = i)

    rm(d, w_abg, w_vbg)
    invisible(gc(FALSE))
  }

  dplyr::bind_rows(per_imp_counts) |>
    dplyr::group_by(method, group, outcome) |>
```

```
    dplyr::summarise(
      n_model = round(stats::median(n_model, na.rm = TRUE)),
      events = round(stats::median(events, na.rm = TRUE)),
      .groups = "drop"
    )
}

# helper: build one harmonized predicted-probability export for a full
# categorical analysis track across both ABG and VBG cohorts.
predict_cat3_track_probs <- function(fit_list_abg, fit_list_vbg, method,
                                     ref_abg, ref_vbg,
                                     adj_vars = adj_core,
                                     cat_var_abg = "pco2_cat_abg",
                                     cat_var_vbg = "pco2_cat_vbg") {
  prob_abg <- dplyr::bind_rows(lapply(names(fit_list_abg), function(outcome_var) {
    predict_cat3_prob_from_fit(
      fit_list_abg[[outcome_var]],
      ref_df = ref_abg,
      outcome_var = outcome_var,
      cat_var = cat_var_abg,
      adj_vars = adj_vars
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "ABG")
  }))
  prob_vbg <- dplyr::bind_rows(lapply(names(fit_list_vbg), function(outcome_var) {
    predict_cat3_prob_from_fit(
      fit_list_vbg[[outcome_var]],
      ref_df = ref_vbg,
      outcome_var = outcome_var,
      cat_var = cat_var_vbg,
      adj_vars = adj_vars
    ) |>
      dplyr::mutate(outcome = outcome_var, group = "VBG")
  }))

  normalize_cat3_key_results_prob_df(
```

```
    dplyr::bind_rows(prob_abg, prob_vbg),
    method = method,
    outcome_col = "outcome",
    group_col = "group",
    category_col = "category",
    p_col = "p",
    lcl_col = "p_LCL",
    ucl_col = "p_UCL",
    context = paste0("categorical track probabilities: ", method)
  )
}

crude_cat3_results <- normalize_cat3_key_results_or_df(
  unw_results_crude,
  method = "Crude",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "estimate",
  lcl_col = "conf.low",
  ucl_col = "conf.high",
  context = "categorical crude ORs"
)
adjusted_cat3_results <- normalize_cat3_key_results_or_df(
  unw_results_adj,
  method = "Adjusted",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "estimate",
  lcl_col = "conf.low",
  ucl_col = "conf.high",
  context = "categorical adjusted ORs"
)
gbm_cat3_results <- normalize_cat3_key_results_or_df(
  ipw_threelevel_results,
```

```
  method = "GBM-weighted",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "estimate",
  lcl_col = "conf.low",
  ucl_col = "conf.high",
  context = "categorical GBM-weighted ORs"
)
main_cat3_results <- normalize_cat3_key_results_or_df(
  mi_threelevel_results,
  method = "Weighted and adjusted",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "OR",
  lcl_col = "LCL",
  ucl_col = "UCL",
  context = "categorical main ORs"
)
imputed_cat3_results <- normalize_cat3_key_results_or_df(
  dplyr::bind_rows(mi_unweighted_abg_cat_results, mi_unweighted_vbg_cat_results),
  method = "Imputed",
  outcome_col = "outcome",
  group_col = "group",
  term_col = "term",
  or_col = "OR",
  lcl_col = "LCL",
  ucl_col = "UCL",
  context = "categorical imputed ORs"
)

crude_cat3_probs <- predict_cat3_track_probs(
  crude_cat3_fits_abg,
  crude_cat3_fits_vbg,
  method = "Crude",
```

```
  ref_abg = data.frame(.ref = 1),
  ref_vbg = data.frame(.ref = 1),
  adj_vars = NULL
)
adjusted_cat3_probs <- predict_cat3_track_probs(
  adjusted_cat3_fits_abg,
  adjusted_cat3_fits_vbg,
  method = "Adjusted",
  ref_abg = x_ref_abg,
  ref_vbg = x_ref_vbg,
  adj_vars = adj_core
)
gbm_cat3_probs <- predict_cat3_track_probs(
  gbm_cat3_fits_abg,
  gbm_cat3_fits_vbg,
  method = "GBM-weighted",
  ref_abg = x_ref_abg,
  ref_vbg = x_ref_vbg,
  adj_vars = adj_core
)
main_cat3_probs <- normalize_cat3_key_results_prob_df(
  dplyr::bind_rows(abg_cat_prob_curves, vbg_cat_prob_curves),
  method = "Weighted and adjusted",
  outcome_col = "outcome",
  group_col = "group",
  category_col = "category",
  p_col = "p",
  lcl_col = "p_LCL",
  ucl_col = "p_UCL",
  context = "categorical main probabilities"
)
main_cat3_probs_conditional_xref <- normalize_cat3_key_results_prob_df(
  dplyr::bind_rows(abg_cat_prob_curves_conditional_xref, vbg_cat_prob_curves_conditional_xref),
  method = "Weighted and adjusted",
  outcome_col = "outcome",
  group_col = "group",
```

```
  category_col = "category",
  p_col = "p",
  lcl_col = "p_LCL",
  ucl_col = "p_UCL",
  context = "categorical main conditional X_ref probabilities"
)
imputed_cat3_probs <- normalize_cat3_key_results_prob_df(
  dplyr::bind_rows(mi_unweighted_abg_cat_prob_curves, mi_unweighted_vbg_cat_prob_curves),
  method = "Imputed",
  outcome_col = "outcome",
  group_col = "group",
  category_col = "category",
  p_col = "p",
  lcl_col = "p_LCL",
  ucl_col = "p_UCL",
  context = "categorical imputed probabilities"
)

mi_counts_cat3_all <- build_mi_counts_cat3()

counts_tbl_cat3 <- dplyr::bind_rows(
  count_group("ABG", "Crude"),
  count_group("VBG", "Crude"),
  count_group("ABG", "Adjusted"),
  count_group("VBG", "Adjusted"),
  count_group("ABG", "GBM-weighted"),
  count_group("VBG", "GBM-weighted"),
  dplyr::filter(mi_counts_cat3_all, method == "Imputed"),
  dplyr::filter(mi_counts_cat3_all, method == "Weighted and adjusted")
)

cat3_or_all <- dplyr::bind_rows(
  crude_cat3_results,
  adjusted_cat3_results,
  imputed_cat3_results,
  main_cat3_results,
```

```
  gbm_cat3_results
) |>
  dplyr::mutate(outcome = as.character(outcome)) |>
  dplyr::left_join(counts_tbl_cat3, by = c("method", "group", "outcome"))

cat3_prob_all <- dplyr::bind_rows(
  crude_cat3_probs,
  adjusted_cat3_probs,
  imputed_cat3_probs,
  main_cat3_probs,
  gbm_cat3_probs
) |>
  dplyr::mutate(outcome = as.character(outcome)) |>
  dplyr::left_join(counts_tbl_cat3, by = c("method", "group", "outcome"))

cat3_key_results_long <- dplyr::bind_rows(
  cat3_or_all |>
    dplyr::transmute(
      method,
      group,
      outcome,
      metric_type = "OR",
      level = as.character(contrast),
      estimate = OR,
      lcl = LCL,
      ucl = UCL,
      n_model,
      events
    ),
  cat3_prob_all |>
    dplyr::transmute(
      method,
      group,
      outcome,
      metric_type = "Probability",
      level = as.character(category),
```

```
      estimate = p,
      lcl = p_LCL,
      ucl = p_UCL,
      n_model,
      events
    )
)
write_csv_safely(cat3_key_results_long, results_path("cat3_key_results_long.csv"), row_names = FALSE)

cat3_or_display <- cat3_or_all |>
  dplyr::mutate(display = format_cat3_ci(OR, LCL, UCL)) |>
  dplyr::select(method, group, outcome, n_model, events, contrast, display) |>
  tidyr::pivot_wider(
    names_from = contrast,
    values_from = display,
    names_glue = "{contrast} OR (95% CI)"
  )

cat3_prob_display <- cat3_prob_all |>
  dplyr::mutate(display = format_cat3_ci(p, p_LCL, p_UCL)) |>
  dplyr::select(method, group, outcome, category, display) |>
  tidyr::pivot_wider(
    names_from = category,
    values_from = display,
    names_glue = "{category} probability (95% CI)"
  )

table_summary_key_results_cat3 <- counts_tbl_cat3 |>
  dplyr::left_join(
    dplyr::select(cat3_or_display, -n_model, -events),
    by = c("method", "group", "outcome")
  ) |>
  dplyr::left_join(cat3_prob_display, by = c("method", "group", "outcome")) |>
  dplyr::arrange(
    factor(method, levels = CAT3_KEY_RESULTS_METHOD_LEVELS),
    factor(group, levels = c("ABG", "VBG")),
```

```r
    factor(outcome, levels = KEY_RESULTS_OUTCOME_LEVELS)
  )

write_csv_safely(
  table_summary_key_results_cat3,
  results_path("table_summary_key_results_cat3.csv"),
  row_names = FALSE
)

table_2_weighted_categorical_wide <- table_summary_key_results_cat3 |>
  dplyr::filter(method == "Weighted and adjusted") |>
  dplyr::select(
    group,
    outcome,
    n_model,
    events,
    `Low vs normal OR (95% CI)`,
    `High vs normal OR (95% CI)`,
    `Normal probability (95% CI)`,
    `Low probability (95% CI)`,
    `High probability (95% CI)`
  )

table_2_weighted_or_panel <- table_2_weighted_categorical_wide |>
  dplyr::select(
    group,
    outcome,
    n_model,
    events,
    `Low vs normal OR (95% CI)`,
    `High vs normal OR (95% CI)`
  )

table_2_weighted_probability_panel <- table_2_weighted_categorical_wide |>
  dplyr::select(
    group,
```

```r
    outcome,
    `Normal probability (95% CI)`,
    `Low probability (95% CI)`,
    `High probability (95% CI)`
  )

table_2_weighted_categorical_outcomes <- dplyr::bind_rows(
  table_2_weighted_or_panel |>
    tidyr::pivot_longer(
      cols = dplyr::ends_with("OR (95% CI)"),
      names_to = "measure",
      values_to = "estimate"
    ) |>
    dplyr::mutate(panel = "Odds ratios", .before = 1L),
  table_2_weighted_probability_panel |>
    tidyr::pivot_longer(
      cols = dplyr::ends_with("probability (95% CI)"),
      names_to = "measure",
      values_to = "estimate"
    ) |>
    dplyr::mutate(
      panel = "Predicted probabilities",
      n_model = NA_integer_,
      events = NA_integer_,
      .before = 1L
    ) |>
    dplyr::select(panel, group, outcome, n_model, events, measure, estimate)
)

write_standalone_panel_table_pdf(
  list(
    "Odds ratios" = table_2_weighted_or_panel,
    "Predicted probabilities" = table_2_weighted_probability_panel
  ),
  caption = paste0(
    "MI-pooled, MI-logistic IPSW-weighted 3-level categorical results for ABG and VBG; ",
```

```r
    "predicted probabilities are marginally standardized to the common eligible source-population covariate
     ↪ distribution."
  ),
  file_stub = "table_2_weighted_categorical_outcomes",
  digits = 2,
  landscape = TRUE,
  display_label = "Table 2",
  csv_df = table_2_weighted_categorical_outcomes
)

table_summary_key_results_cat3_abg <- table_summary_key_results_cat3 |>
  dplyr::filter(group == "ABG")
table_summary_key_results_cat3_vbg <- table_summary_key_results_cat3 |>
  dplyr::filter(group == "VBG")

render_table_pdf_maybe(
  table_summary_key_results_cat3_abg,
  caption = paste0(
    "Categorical key results for the ABG cohort across crude, unweighted adjusted, imputed non-weighted, ",
    "MI-logistic IPSW-weighted, and GBM IPSW-weighted analyses; primary weighted probabilities are ",
    "marginally standardized to the common eligible source-population covariate distribution, adjusted/imputed ",
    "non-weighted/GBM probabilities are conditional on X_ref, and crude probabilities are crude model-implied."
  ),
  file_stub = "table_summary_key_results_cat3_abg",
  wide = TRUE,
  digits = 2,
  show = FALSE
)

render_table_pdf_maybe(
  table_summary_key_results_cat3_vbg,
  caption = paste0(
    "Categorical key results for the VBG cohort across crude, unweighted adjusted, imputed non-weighted, ",
    "MI-logistic IPSW-weighted, and GBM IPSW-weighted analyses; primary weighted probabilities are ",
    "marginally standardized to the common eligible source-population covariate distribution, adjusted/imputed ",
    "non-weighted/GBM probabilities are conditional on X_ref, and crude probabilities are crude model-implied."
```

```
  ),
  file_stub = "table_summary_key_results_cat3_vbg",
  wide = TRUE,
  digits = 2,
  show = FALSE
)

legacy_counts_tbl <- dplyr::bind_rows(
  counts_tbl_cat3 |>
    dplyr::filter(method == "Adjusted") |>
    dplyr::mutate(method = "Unweighted adjusted"),
  counts_tbl_cat3 |>
    dplyr::filter(method == "GBM-weighted") |>
    dplyr::mutate(method = "IPW adjusted"),
  counts_tbl_cat3 |>
    dplyr::filter(method == "Weighted and adjusted") |>
    dplyr::mutate(method = "IPW + MI adjusted")
)

table_summary <- dplyr::bind_rows(
  adjusted_cat3_results |>
    dplyr::mutate(method = "Unweighted adjusted", outcome = as.character(outcome)),
  gbm_cat3_results |>
    dplyr::mutate(method = "IPW adjusted", outcome = as.character(outcome)),
  main_cat3_results |>
    dplyr::mutate(method = "IPW + MI adjusted", outcome = as.character(outcome))
) |>
  dplyr::left_join(legacy_counts_tbl, by = c("method", "group", "outcome")) |>
  dplyr::mutate(or_ci = format_cat3_ci(OR, LCL, UCL)) |>
  dplyr::select(method, group, outcome, n_model, events, contrast, or_ci) |>
  tidyr::pivot_wider(names_from = contrast, values_from = or_ci) |>
  dplyr::arrange(
    factor(method, levels = c("Unweighted adjusted", "IPW adjusted", "IPW + MI adjusted")),
    factor(group, levels = c("ABG", "VBG")),
    factor(outcome, levels = KEY_RESULTS_OUTCOME_LEVELS)
  )
```

```r
write_csv_safely(table_summary, results_path("table_summary_adjusted_threelevel.csv"), row_names = FALSE)

table_summary_abg <- table_summary |>
  dplyr::filter(group == "ABG")
table_summary_vbg <- table_summary |>
  dplyr::filter(group == "VBG")

render_table_pdf_maybe(
  table_summary_abg,
  caption = "Adjusted odds ratios (low/high vs normal) across unweighted adjusted, GBM IPSW-weighted, and MI-logistic
   ↪ IPSW-weighted analysis tracks for the ABG cohort; n/events reflect model sample size (median across imputations
   ↪ for MI-based tracks).",
  file_stub = "table_summary_adjusted_threelevel_abg",
  digits = 2,
  show = FALSE
)

render_table_pdf_maybe(
  table_summary_vbg,
  caption = "Adjusted odds ratios (low/high vs normal) across unweighted adjusted, GBM IPSW-weighted, and MI-logistic
   ↪ IPSW-weighted analysis tracks for the VBG cohort; n/events reflect model sample size (median across imputations
   ↪ for MI-based tracks).",
  file_stub = "table_summary_adjusted_threelevel_vbg",
  digits = 2,
  show = FALSE
)
```

3.8 Canonical Categorical Key-Results Figures Across Analysis Tracks

```r
# Purpose: prepare canonical categorical key-results figure objects across analysis tracks.
key_results_cat3_caption_crude <- paste0(
  "Crude, unadjusted categorical odds ratios use the normal CO2 category as the reference. ",
  "Predicted probabilities are crude model-implied category probabilities within the tested cohorts."
)
key_results_cat3_caption_adjusted <- paste0(
```

```r
  "Odds ratios use the normal CO2 category as the reference. ",
  "Predicted probabilities are conditional on the reference covariate profile X_ref within each test-specific cohort."
)
key_results_cat3_caption_imputed <- paste0(
  "MI-pooled categorical odds ratios use the normal CO2 category as the reference. ",
  "Predicted probabilities are pooled on the link scale and are conditional on X_ref within each test-specific
   ↪  cohort."
)
key_results_cat3_caption_main <- paste0(
  "MI-pooled, MI-logistic IPSW-adjusted categorical odds ratios use the normal CO2 category as the reference. ",
  "Predicted probabilities are pooled on the link scale and marginally standardized to the common eligible
   ↪  source-population covariate distribution."
)
key_results_cat3_caption_gbm <- paste0(
  "GBM-IPSW-adjusted categorical odds ratios use the normal CO2 category as the reference. ",
  "Predicted probabilities are conditional on X_ref within each test-specific cohort."
)

key_results_cat3_crude_plot <- assemble_cat3_key_results_figure(
  crude_cat3_results,
  crude_cat3_probs,
  figure_title = "Crude categorical associations: ABG vs VBG CO2 category",
  figure_caption = key_results_cat3_caption_crude
)
key_results_cat3_adjusted_plot <- assemble_cat3_key_results_figure(
  adjusted_cat3_results,
  adjusted_cat3_probs,
  figure_title = "Adjusted categorical associations: ABG vs VBG CO2 category",
  figure_caption = key_results_cat3_caption_adjusted
)
key_results_cat3_imputed_plot <- assemble_cat3_key_results_figure(
  imputed_cat3_results,
  imputed_cat3_probs,
  figure_title = "MI-pooled categorical associations (non-weighted): ABG vs VBG CO2 category",
  figure_caption = key_results_cat3_caption_imputed
)
```

```r
key_results_cat3_main_plot <- assemble_cat3_key_results_figure(
  main_cat3_results,
  main_cat3_probs,
  figure_title = "MI-logistic IPSW-weighted categorical associations of pCO2 category with clinical outcomes by ABG
   ↪  and VBG",
  figure_caption = NULL
)
key_results_cat3_gbm_plot <- assemble_cat3_key_results_figure(
  gbm_cat3_results,
  gbm_cat3_probs,
  figure_title = "GBM-IPSW-adjusted categorical associations: ABG vs VBG CO2 category",
  figure_caption = key_results_cat3_caption_gbm
)
```

```r
# Purpose: canonical categorical key-results figure for crude models.
print_plot_once(
  key_results_cat3_crude_plot,
  "key-results-cat3-crude-abg-vbg",
  width = CAT3_KEY_RESULTS_FIG_WIDTH,
  height = CAT3_KEY_RESULTS_FIG_HEIGHT
)
```

```r
# Purpose: canonical categorical key-results figure for adjusted models.
print_plot_once(
  key_results_cat3_adjusted_plot,
  "key-results-cat3-adjusted-abg-vbg",
  width = CAT3_KEY_RESULTS_FIG_WIDTH,
  height = CAT3_KEY_RESULTS_FIG_HEIGHT
)
```

```r
# Purpose: canonical categorical key-results figure for imputed non-weighted models.
print_plot_once(
  key_results_cat3_imputed_plot,
  "key-results-cat3-imputed-abg-vbg",
  width = CAT3_KEY_RESULTS_FIG_WIDTH,
  height = CAT3_KEY_RESULTS_FIG_HEIGHT
)
```

```r
# Purpose: manuscript Figure S4 weighted categorical associations for the primary MI-weighted analysis.
print_plot_once(
  key_results_cat3_main_plot,
  "key-results-cat3-main-mi-ipw-abg-vbg",
  width = CAT3_KEY_RESULTS_FIG_WIDTH,
  height = CAT3_KEY_RESULTS_FIG_HEIGHT
)
```

```r
# Purpose: canonical categorical key-results figure for GBM-weighted models.
print_plot_once(
  key_results_cat3_gbm_plot,
  "key-results-cat3-gbm-weighted-abg-vbg",
  width = CAT3_KEY_RESULTS_FIG_WIDTH,
  height = CAT3_KEY_RESULTS_FIG_HEIGHT
)
```

```r
# Purpose: stage1 cleanup.
append_mem_snapshot("stage1", "end", "post")
stage1_rm <- c(
  "unw_results_crude", "unw_results_adj", "unw_threelevel_results",
  "unw_combined_or_df", "unw_plot_df", "unw_p_or", "unw_axis_spec", "outcomes_unw"
)
missing_stage1 <- setdiff(stage1_rm, ls())
stopifnot(length(missing_stage1) == 0)
rm(list = stage1_rm)
invisible(gc())
append_mem_snapshot("stage1", "cleanup", "post")
```

3.9 Observed outcome rates by CO2 bin (MI-logistic IPSW)

```r
# Purpose: pool weighted 1-mmHg bin rates across imputations using summary-only objects.
stopifnot(exists("imp"), imp$m > 0)
stopifnot(exists("get_imp"), exists("get_mi_weight"))

co2_rate_mi_rows_abg <- vector("list", imp$m)
co2_rate_mi_rows_vbg <- vector("list", imp$m)
```

```r
for (i in seq_len(imp$m)) {
  d_i <- get_imp(i)
  w_abg_i <- get_mi_weight("ABG", i)
  w_vbg_i <- get_mi_weight("VBG", i)

  d_abg_i <- d_i[, c("has_abg", "paco2", CO2_RATE_OUTCOMES), drop = FALSE]
  d_abg_i$w_rate <- w_abg_i
  d_vbg_i <- d_i[, c("has_vbg", "vbg_co2", CO2_RATE_OUTCOMES), drop = FALSE]
  d_vbg_i$w_rate <- w_vbg_i

  co2_rate_mi_rows_abg[[i]] <- compute_co2_bin_rates(
    df = d_abg_i |> dplyr::filter(has_abg == 1),
    co2_var = "paco2",
    outcomes = CO2_RATE_OUTCOMES,
    outcome_labels = CO2_RATE_OUTCOME_LABELS,
    weight_var = "w_rate",
    q = c(0.02, 0.98),
    bin_width = 1,
    cohort_label = "ABG"
  ) |> dplyr::mutate(imp = i)

  co2_rate_mi_rows_vbg[[i]] <- compute_co2_bin_rates(
    df = d_vbg_i |> dplyr::filter(has_vbg == 1),
    co2_var = "vbg_co2",
    outcomes = CO2_RATE_OUTCOMES,
    outcome_labels = CO2_RATE_OUTCOME_LABELS,
    weight_var = "w_rate",
    q = c(0.02, 0.98),
    bin_width = 1,
    cohort_label = "VBG"
  ) |> dplyr::mutate(imp = i)

  rm(d_i, d_abg_i, d_vbg_i, w_abg_i, w_vbg_i)
  invisible(gc(FALSE))
}
```

```r
co2_rate_mi_abg <- pool_mi_bin_rates(dplyr::bind_rows(co2_rate_mi_rows_abg))
co2_rate_mi_vbg <- pool_mi_bin_rates(dplyr::bind_rows(co2_rate_mi_rows_vbg))

write_csv_safely(co2_rate_mi_abg, results_path("co2_binned_rates_mi_ipw_abg.csv"), row_names = FALSE)
write_csv_safely(co2_rate_mi_vbg, results_path("co2_binned_rates_mi_ipw_vbg.csv"), row_names = FALSE)

p_co2_rate_mi_abg <- plot_co2_bin_rate_panel(
  co2_rate_mi_abg,
  title = paste0("Observed outcome rates by ABG CO2 bin (MI-logistic IPSW, m = ", imp$m, ")"),
  x_label = "ABG CO2 (mmHg)",
  vline_at = 45
)
p_co2_rate_mi_vbg <- plot_co2_bin_rate_panel(
  co2_rate_mi_vbg,
  title = paste0("Observed outcome rates by VBG CO2 bin (MI-logistic IPSW, m = ", imp$m, ")"),
  x_label = "VBG CO2 (mmHg)",
  vline_at = 50
)

print_plot_once(p_co2_rate_mi_abg, "co2-binned-rates-mi-ipw-abg", width = 8.5, height = 6.0)
print_plot_once(p_co2_rate_mi_vbg, "co2-binned-rates-mi-ipw-vbg", width = 8.5, height = 6.0)

rm(co2_rate_mi_rows_abg, co2_rate_mi_rows_vbg, co2_rate_mi_abg, co2_rate_mi_vbg,
   p_co2_rate_mi_abg, p_co2_rate_mi_vbg)
invisible(gc(FALSE))
```

3.10 Manuscript outputs summary

```r
# Use full-cohort flow counts here so manuscript-facing sample sizes remain
# stable in pilot renders and stay synchronized with Figure 1.
flow_tbl <- tibble::tibble(
  metric = c(
    "Extracted encounter cohort",
    "Ambulatory encounters excluded",
    "Analytic ED/inpatient cohort",
```

```
    "ABG-tested encounters",
    "VBG-tested encounters",
    "Both ABG and VBG",
    "ABG only",
    "VBG only",
    "Neither ABG nor VBG",
    "ABG with observed PaCO2",
    "VBG with observed CO2"
  ),
  n = c(
    cohort_flow_counts$total_extracted,
    cohort_flow_counts$excluded_ambulatory,
    cohort_flow_counts$analytic_ed_inpatient,
    cohort_flow_counts$abg_tested,
    cohort_flow_counts$vbg_tested,
    cohort_flow_counts$both_abg_vbg,
    cohort_flow_counts$abg_only,
    cohort_flow_counts$vbg_only,
    cohort_flow_counts$neither_abg_nor_vbg,
    cohort_flow_counts$abg_with_paco2,
    cohort_flow_counts$vbg_with_vbg_co2
  )
)
render_table_pdf_maybe(flow_tbl, "Cohort flow summary", "cohort_flow_summary",
                       digits = 0, show = FALSE)

# Event counts by cohort
outcome_vars <- c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
outcome_labels <- c(
  imv_proc = "IMV",
  niv_proc = "NIV",
  death_60d = "Death (60d)",
  hypercap_resp_failure = "Hypercapnic RF"
)
event_tbl <- dplyr::bind_rows(
  lapply(outcome_vars, function(o) {
```

```r
    dplyr::tibble(
      outcome = outcome_labels[[o]],
      group = "ABG",
      n = sum(subset_data$has_abg == 1 & !is.na(subset_data[[o]]), na.rm = TRUE),
      events = sum(subset_data$has_abg == 1 & subset_data[[o]] == 1, na.rm = TRUE)
    )
  }),
  lapply(outcome_vars, function(o) {
    dplyr::tibble(
      outcome = outcome_labels[[o]],
      group = "VBG",
      n = sum(subset_data$has_vbg == 1 & !is.na(subset_data[[o]]), na.rm = TRUE),
      events = sum(subset_data$has_vbg == 1 & subset_data[[o]] == 1, na.rm = TRUE)
    )
  })
)
event_tbl <- event_tbl |>
  dplyr::mutate(pct = ifelse(n > 0, 100 * events / n, NA_real_))
render_table_pdf_maybe(event_tbl,
                       "Outcome counts by cohort (ABG/VBG tested)",
                       "outcome_counts_by_cohort",
                       digits = 1, show = FALSE)

# Weighting diagnostics (non-MI weights)
stopifnot(all(c("w_abg", "w_vbg", "ps_abg", "ps_vbg") %in% names(subset_data)))
wt_abg <- subset_data$w_abg[subset_data$has_abg == 1]
wt_vbg <- subset_data$w_vbg[subset_data$has_vbg == 1]
ps_abg <- subset_data$ps_abg[subset_data$has_abg == 1]
ps_vbg <- subset_data$ps_vbg[subset_data$has_vbg == 1]
trunc_abg <- subset_data$trunc_abg[subset_data$has_abg == 1]
trunc_vbg <- subset_data$trunc_vbg[subset_data$has_vbg == 1]

wt_sum <- dplyr::bind_rows(
  weight_summary(wt_abg, ps = ps_abg, ps_floor = ps_floor_abg,
                 truncated = trunc_abg) |>
    dplyr::mutate(group = "ABG"),
```

```
  weight_summary(wt_vbg, ps = ps_vbg, ps_floor = ps_floor_vbg,
                 truncated = trunc_vbg) |>
    dplyr::mutate(group = "VBG")
)
wt_sum_display <- wt_sum |>
  dplyr::select(group, n, ess, min, p99, max, trunc_rate, ps_p01) |>
  dplyr::rename(
    trunc = trunc_rate,
    p01_ps = ps_p01
  )
render_table_pdf_maybe(wt_sum_display,
                       "GBM IPSW weighting diagnostics summary (non-MI)",
                       "weighting_diagnostics_non_mi",
                       wide = TRUE,
                       digits = 3,
                       show = FALSE)

# Missingness + MI spec summary
miss_vars <- c(covars_ps, "paco2", "vbg_co2",
               "imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure")
miss_vars <- intersect(miss_vars, names(subset_data_raw))
miss_tbl <- subset_data_raw |>
  dplyr::summarise(dplyr::across(dplyr::all_of(miss_vars), ~ mean(is.na(.)) * 100)) |>
  tidyr::pivot_longer(dplyr::everything(), names_to = "variable", values_to = "pct_missing") |>
  dplyr::arrange(dplyr::desc(pct_missing))
render_table_pdf_maybe(miss_tbl,
                       "Variables by missingness (pre-imputation)",
                       "missingness_all",
                       digits = 1,
                       show = FALSE)

mi_spec_tbl <- tibble::tibble(
  m = imp$m,
  maxit = MAXIT_MI,
  methods = paste(unique(imp$method[imp$method != ""]), collapse = ", "),
  ps_model = MI_PS_METHOD,
```

```r
  ps_spline_k = MI_PS_SPLINE_K,
  ps_glm_maxit = MI_GLM_MAXIT
)
render_table_pdf_maybe(mi_spec_tbl,
                       "MI specification (methods used)",
                       "mi_specification",
                       show = FALSE)
```

```r
# Purpose: chunk.
stopifnot(exists("imp"))
imp_n <- imp$m
# helper: pull one completed imputation back into the normalized type schema used
# by downstream assertions so diagnostics are checking the same structure as the
# analysis code.
get_imp <- function(i, imp_obj = imp) { normalize_types(mice::complete(imp_obj, action = i), levels_ref) }

# 1) must exist and be numeric
d1 <- get_imp(1)
stopifnot(all(c("paco2","vbg_co2") %in% names(d1)))
stopifnot(is.numeric(d1$paco2), is.numeric(d1$vbg_co2))

# 2) confirm at least two PaCO2 levels among those with ABG in each imputation
table(vapply(seq_len(imp_n), function(i) {
  d <- get_imp(i)
  dplyr::n_distinct(d$paco2[d$has_abg == 1 & is.finite(d$paco2)])
}, integer(1)) > 1)
```

```
TRUE
  20
```

```r
# 3) light sanity check: ensure pooled MI cat3 results exist
stopifnot(exists("abg_cat_results"), nrow(abg_cat_results) > 0)
```

3.10.1 Visualization: pooled three-level ORs

```r
# Purpose: chunk.
```

```r
stopifnot(exists("abg_cat_results"), exists("vbg_cat_results"))

mi_combined_or_df <- dplyr::bind_rows(
  dplyr::mutate(abg_cat_results, group = "ABG"),
  dplyr::mutate(vbg_cat_results, group = "VBG")
) |>
  dplyr::mutate(
    outcome = factor(outcome,
                     levels = c("IMV", "NIV", "Death60d", "HCRF"),
                     labels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")),
    group = factor(group, levels = c("ABG", "VBG"))
  )

mi_combined_or_df <- map_or_exposure(mi_combined_or_df, "or-plot-mi-weighted")
mi_plot_df <- mi_combined_or_df |>
  dplyr::mutate(estimate = OR, conf.low = LCL, conf.high = UCL) |>
  dplyr::select(-OR, -LCL, -UCL)

mi_plot_df <- build_or_plot_df(
  mi_plot_df,
  "or-plot-mi-weighted",
  expected_exposure_levels = CO2_CAT_CONTRAST_LEVELS
)
mi_axis_spec <- compute_or_axis_spec(list(mi_plot_df), lo_col = "conf.low", hi_col = "conf.high")

mi_p_or <- plot_or_safe(
  mi_plot_df,
  plot_name = "or-plot-mi-weighted",
  axis_spec = mi_axis_spec,
  title = "MI-pooled, MI-logistic IPSW-adjusted odds ratios by PCO2 category (ABG vs VBG)"
)
print_plot_once(mi_p_or, "or-plot-mi-weighted", width = 7.5, height = 4.8)
```

3.10.2 Visualization: pooled three-level predicted probabilities

```r
# Purpose: plot MI-pooled adjusted probabilities by categorical CO2 exposure.
```

```r
stopifnot(exists("abg_cat_prob_curves"), exists("vbg_cat_prob_curves"))

mi_cat_prob_plot_df <- dplyr::bind_rows(
  abg_cat_prob_curves,
  vbg_cat_prob_curves
) |>
  dplyr::mutate(
    group = factor(group, levels = c("ABG", "VBG")),
    co2_cat = factor(co2_cat, levels = CO2_CAT_LEVELS),
    outcome = factor(
      outcome,
      levels = c("IMV", "NIV", "Death60d", "HCRF"),
      labels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")
    )
  )

axis_mi_cat_prob <- compute_prob_axis_spec(list(mi_cat_prob_plot_df), lo_col = "p_LCL", hi_col = "p_UCL")
dodge_cat_prob <- ggplot2::position_dodge(width = 0.45)

mi_cat_prob_plot <- ggplot(
  mi_cat_prob_plot_df,
  aes(x = co2_cat, y = p, color = group)
) +
  geom_pointrange(
    aes(ymin = p_LCL, ymax = p_UCL),
    position = dodge_cat_prob,
    fatten = 1.2,
    linewidth = 0.4
  ) +
  prob_axis_scale(axis_mi_cat_prob) +
  facet_wrap(~ outcome) +
  labs(
    title = "MI-pooled, MI-logistic IPSW-adjusted predicted probabilities by PCO2 category (ABG vs VBG)",
    x = "PCO2 category",
    y = "Predicted probability",
    caption = "Marginally standardized to the common eligible source-population covariate distribution."
```

```r
  ) +
  theme_pub(base_size = 10)

print_plot_once(mi_cat_prob_plot, "ipw-three-level-mi-abg-vbg-prob", width = 7.8, height = 4.9)
```

```r
# Purpose: manuscript Figure S2 propensity-score overlap under the MI logistic IPSW models.
# Build an imputation-averaged propensity histogram for ABG and VBG.
# This mirrors the non-MI propensity display while respecting MI variability.
stopifnot(exists("imp"))
stopifnot(exists("mi_logistic_ps_abg_list"), exists("mi_logistic_ps_vbg_list"))

bin_edges <- seq(0, 1, length.out = 31)
bin_width <- diff(bin_edges)[1]
bin_centers <- head(bin_edges, -1) + bin_width / 2
treat_abg_all <- to01(imp$data$has_abg)
treat_vbg_all <- to01(imp$data$has_vbg)

# helper: collapse one imputation's propensity scores into comparable histogram
# bins so the manuscript figure can average densities across imputations without
# overplotting dozens of raw curves.
mi_ps_hist_one <- function(ps_vec, treat_vec, score_type, imp_index) {
  # Use integer bin IDs to avoid floating-point matching issues during complete().
  df <- data.frame(
    ps = as.numeric(ps_vec),
    treat = as.integer(treat_vec),
    stringsAsFactors = FALSE
  ) |>
    dplyr::filter(is.finite(ps), !is.na(treat), ps >= 0, ps <= 1) |>
    dplyr::mutate(
      treat = factor(treat, levels = c(0L, 1L), labels = c("No Test", "Test")),
      bin_id = cut(
        ps,
        breaks = bin_edges,
        include.lowest = TRUE,
        right = FALSE,
        labels = FALSE
      )
```

```r
    )

  if (nrow(df) == 0 || all(is.na(df$bin_id))) return(data.frame())

  df |>
    dplyr::filter(!is.na(bin_id)) |>
    dplyr::count(treat, bin_id, .drop = FALSE, name = "n") |>
    tidyr::complete(treat, bin_id = seq_along(bin_centers), fill = list(n = 0L)) |>
    dplyr::group_by(treat) |>
    dplyr::mutate(
      n_total = sum(n, na.rm = TRUE),
      density = dplyr::if_else(n_total > 0, n / (n_total * bin_width), 0)
    ) |>
    dplyr::ungroup() |>
    dplyr::mutate(
      ScoreType = score_type,
      imp = imp_index,
      bin_mid = bin_centers[bin_id]
    ) |>
    dplyr::select(ScoreType, imp, treat, bin_mid, density)
}

mi_ps_hist_rows <- vector("list", imp$m * 2L)
row_idx <- 0L
for (i in seq_len(imp$m)) {
  # Aggregate ABG and VBG bin-level densities for each imputation.
  row_idx <- row_idx + 1L
  mi_ps_hist_rows[[row_idx]] <- mi_ps_hist_one(get_mi_ps("ABG", i), treat_abg_all, "ABG", i)
  row_idx <- row_idx + 1L
  mi_ps_hist_rows[[row_idx]] <- mi_ps_hist_one(get_mi_ps("VBG", i), treat_vbg_all, "VBG", i)
}

# Average densities across imputations to avoid overplotting 80+ curves.
mi_ps_hist <- dplyr::bind_rows(mi_ps_hist_rows) |>
  dplyr::group_by(ScoreType, treat, bin_mid) |>
  dplyr::summarise(density = mean(density, na.rm = TRUE), .groups = "drop") |>
```

```r
  dplyr::mutate(ScoreType = factor(ScoreType, levels = c("ABG", "VBG")))

if (nrow(mi_ps_hist) > 0 && any(is.finite(mi_ps_hist$density) & mi_ps_hist$density > 0)) {
  # Render side-by-side ABG/VBG density histograms with shared x-range [0,1].
  mi_ps_hist_plot <- ggplot(mi_ps_hist, aes(x = bin_mid, y = density, fill = treat)) +
    geom_col(position = "identity", alpha = 0.5, width = bin_width * 0.95) +
    facet_wrap(~ ScoreType, scales = "free_y") +
    coord_cartesian(xlim = c(0, 1)) +
    scale_fill_manual(values = c("No Test" = "steelblue", "Test" = "tomato")) +
    labs(
      title = "Propensity-score overlap for MI logistic ABG and VBG test-ordering models",
      subtitle = paste0("Averaged across ", imp$m, " imputations"),
      x = "Propensity score",
      y = "Density",
      fill = "Group"
    ) +
    theme_pub(base_size = 12)

  print_plot_once(mi_ps_hist_plot, "propensity-histograms-mi-logistic", width = 8.5, height = 5)
} else {
  warning("MI propensity histogram has no positive density values; inspect mi_logistic_ps_* artifacts.", call. =
   ↪  FALSE)
}
```

```r
# Purpose: manuscript Figure S1 covariate balance after the primary MI logistic IPSW analysis.
# Build pooled (median across imputations) raw vs weighted |SMD|
# so MI balance can be read with the same visual grammar as non-MI.
stopifnot(exists("bal_imp_abg"), exists("bal_imp_vbg"))

lov_mi <- dplyr::bind_rows(bal_imp_abg$bal_long, bal_imp_vbg$bal_long) |>
  dplyr::mutate(
    term = ifelse(is.na(level), variable, paste0(variable, ":", level)),
    abs_smd_raw = abs(smd_pre),
    abs_smd_ipw = abs(smd_post)
  ) |>
  dplyr::group_by(group, term) |>
  dplyr::summarise(
```

```r
    abs_smd_raw = median(abs_smd_raw, na.rm = TRUE),
    abs_smd_ipw = median(abs_smd_ipw, na.rm = TRUE),
    .groups = "drop"
  )

lov_mi_long <- dplyr::bind_rows(
  lov_mi |>
    dplyr::transmute(group, term, sample = "Raw", abs_smd = abs_smd_raw, abs_smd_raw),
  lov_mi |>
    dplyr::transmute(group, term, sample = "IPW", abs_smd = abs_smd_ipw, abs_smd_raw)
)
lov_mi_long <- add_overall_love_rows(lov_mi_long)

# Export plotting data and then draw the two-panel love plot.
write_csv_safely(lov_mi_long, results_path("loveplot_mi_logistic_data.csv"), row_names = FALSE)
p_lov_mi <- loveplot_style(
  lov_mi_long,
  "Covariate balance after MI logistic inverse-probability weighting for ABG and VBG test-ordering models"
)
print_plot_once(p_lov_mi, "loveplot-mi-logistic-abg-vbg", width = 9, height = 6)
```

```r
# Purpose: manuscript Figure S3 SHAP-style contribution summaries for the MI logistic IPSW models.
# Read precomputed top-10 SHAP values and re-plot deterministically.
shap_abg_file <- results_path("shap_top10_mi_logistic_abg.csv")
shap_vbg_file <- results_path("shap_top10_mi_logistic_vbg.csv")

shap_abg <- utils::read.csv(shap_abg_file, stringsAsFactors = FALSE)
shap_vbg <- utils::read.csv(shap_vbg_file, stringsAsFactors = FALSE)
shap_abg$group <- "ABG"
shap_vbg$group <- "VBG"

shap_plot_df <- dplyr::bind_rows(shap_abg, shap_vbg) |>
  dplyr::filter(is.finite(mean_abs_shap)) |>
  dplyr::group_by(group) |>
  dplyr::arrange(dplyr::desc(mean_abs_shap), .by_group = TRUE) |>
  dplyr::slice_head(n = 10L) |>
  dplyr::ungroup()
```

```r
stopifnot(nrow(shap_plot_df) > 0)

p_shap <- plot_shap_top10_two_panel(
  shap_plot_df,
  "SHAP-style contribution summaries for MI logistic ABG and VBG test-ordering models",
  "Mean |SHAP value|"
)
print_plot_force(p_shap, "shap-top10-mi-logistic-abg-vbg", width = 9, height = 5)
```

```r
# Purpose: float barrier mi or plot.
float_barrier("section")
```

3.10.3 Visualization

```r
# Purpose: chunk.

mi_ipw_rcs_forms <- list(
  "MI IPW spline (adjusted) ABG: IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "paco2", adj_core),
  "MI IPW spline (adjusted) ABG: NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "paco2", adj_core),
  "MI IPW spline (adjusted) ABG: Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "paco2", adj_core),
  "MI IPW spline (adjusted) ABG: HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "paco2",
   ↪  adj_core),
  "MI IPW spline (adjusted) VBG: IMV ~ CO2 spline + X"      = make_spline_fml("imv_proc", "vbg_co2", adj_core),
  "MI IPW spline (adjusted) VBG: NIV ~ CO2 spline + X"      = make_spline_fml("niv_proc", "vbg_co2", adj_core),
  "MI IPW spline (adjusted) VBG: Death60d ~ CO2 spline + X" = make_spline_fml("death_60d", "vbg_co2", adj_core),
  "MI IPW spline (adjusted) VBG: HCRF ~ CO2 spline + X"     = make_spline_fml("hypercap_resp_failure", "vbg_co2",
   ↪  adj_core)
)
register_model_diagrams(mi_ipw_rcs_forms)

stopifnot(exists("abg_curves"), exists("vbg_curves"))

curve_abg <- abg_curves |>
  mutate(co2 = paco2) |>
```

```r
  select(-paco2)
curve_vbg <- vbg_curves |>
  mutate(co2 = vbg_co2) |>
  select(-vbg_co2)
curve_all <- bind_rows(curve_abg, curve_vbg) |>
  mutate(outcome = factor(outcome,
                          levels = c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"),
                          labels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")))
axis_mi_ipw <- compute_or_axis_spec(list(curve_abg, curve_vbg), lo_col = "LCL", hi_col = "UCL")

mi_or_overlay <- ggplot(curve_all, aes(x = co2, y = OR, color = group, fill = group)) +
  geom_line(linewidth = 0.6) +
  geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.15, color = NA) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
  or_axis_scale(axis_mi_ipw) +
  facet_wrap(~ outcome, scales = "free_x") +
  labs(
    title = expression(
      paste("MI-pooled, MI-logistic IPSW-adjusted spline odds ratios: ABG vs VBG CO"[2])
    ),
    x = expression(CO[2]~"(mmHg)"),
    y = paste0("Odds ratio (ref: ABG ", ABG_CO2_REF, ", VBG ", VBG_CO2_REF, "; log scale)"),
    caption = paste0("Reference CO2: ABG ", ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF, " mmHg")
  ) +
  theme_pub(base_size = 10)

print_plot_once(mi_or_overlay, "ipw-rcs-overlay-mi-abg-vbg", width = 8.5, height = 6.5)
```

```r
# Purpose: MI pooled IPSW spline probability overlay.
stopifnot(exists("abg_prob_curves"), exists("vbg_prob_curves"))

curve_abg_prob <- abg_prob_curves |>
  mutate(co2 = paco2) |>
  select(-paco2)
curve_vbg_prob <- vbg_prob_curves |>
  mutate(co2 = vbg_co2) |>
  select(-vbg_co2)
```

```r
curve_all_prob <- bind_rows(curve_abg_prob, curve_vbg_prob) |>
  mutate(outcome = factor(outcome,
                          levels = c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"),
                          labels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")))
axis_mi_ipw_prob <- compute_prob_axis_spec(
  list(curve_abg_prob, curve_vbg_prob),
  lo_col = "p_LCL", hi_col = "p_UCL"
)

mi_prob_overlay <- ggplot(curve_all_prob, aes(x = co2, y = p, color = group, fill = group)) +
  geom_line(linewidth = 0.6) +
  geom_ribbon(aes(ymin = p_LCL, ymax = p_UCL), alpha = 0.15, color = NA) +
  prob_axis_scale(axis_mi_ipw_prob) +
  facet_wrap(~ outcome, scales = "free_x") +
  labs(
    title = expression(
      paste("MI-pooled, MI-logistic IPSW-adjusted spline predicted probabilities: ABG vs VBG CO"[2])
    ),
    x = expression(CO[2]~"(mmHg)"),
    y = "Predicted probability",
    caption = "Marginally standardized to the common eligible source-population covariate distribution."
  ) +
  theme_pub(base_size = 10)

print_plot_once(mi_prob_overlay, "ipw-rcs-overlay-mi-abg-vbg-prob", width = 8.5, height = 6.5)
```

3.11 Canonical Key-Results Figures Across Analysis Tracks

The following 4×2 figures are the canonical manuscript-facing spline displays across crude, adjusted, imputed, and weighted analyses.

```r
# Purpose: prepare canonical key-results figure data across analysis tracks.
stopifnot(
  exists("fit_spline_glm"),
  exists("subset_data_abg"), exists("subset_data_vbg"),
  exists("grid_abg_unw"), exists("grid_vbg_unw"),
  exists("ref_idx_abg_unw"), exists("ref_idx_vbg_unw"),
```

```
  exists("pred_imv"), exists("pred_niv"), exists("pred_death"), exists("pred_hcrf"),
  exists("pred_imv_vbg"), exists("pred_niv_vbg"), exists("pred_death_vbg"), exists("pred_hcrf_vbg"),
  exists("ipw_curve_overlay"), exists("ipw_curve_overlay_prob"),
  exists("abg_curves"), exists("vbg_curves"),
  exists("abg_prob_curves"), exists("vbg_prob_curves"),
  exists("mi_unweighted_abg_curves"), exists("mi_unweighted_vbg_curves"),
  exists("mi_unweighted_abg_prob_curves"), exists("mi_unweighted_vbg_prob_curves")
)

crude_spline_forms <- list(
  "ABG spline (crude): IMV ~ CO2 spline"      = make_spline_fml("imv_proc", "paco2", NULL),
  "ABG spline (crude): NIV ~ CO2 spline"      = make_spline_fml("niv_proc", "paco2", NULL),
  "ABG spline (crude): Death60d ~ CO2 spline" = make_spline_fml("death_60d", "paco2", NULL),
  "ABG spline (crude): HCRF ~ CO2 spline"     = make_spline_fml("hypercap_resp_failure", "paco2", NULL),
  "VBG spline (crude): IMV ~ CO2 spline"      = make_spline_fml("imv_proc", "vbg_co2", NULL),
  "VBG spline (crude): NIV ~ CO2 spline"      = make_spline_fml("niv_proc", "vbg_co2", NULL),
  "VBG spline (crude): Death60d ~ CO2 spline" = make_spline_fml("death_60d", "vbg_co2", NULL),
  "VBG spline (crude): HCRF ~ CO2 spline"     = make_spline_fml("hypercap_resp_failure", "vbg_co2", NULL)
)
register_model_diagrams(crude_spline_forms)

fit_imv_crude_abg <- fit_spline_glm("imv_proc", "paco2", subset_data_abg, "ABG",
                                    adj_vars = NULL, analysis_variant = "crude")
fit_niv_crude_abg <- fit_spline_glm("niv_proc", "paco2", subset_data_abg, "ABG",
                                    adj_vars = NULL, analysis_variant = "crude")
fit_death_crude_abg <- fit_spline_glm("death_60d", "paco2", subset_data_abg, "ABG",
                                      adj_vars = NULL, analysis_variant = "crude")
fit_hcrf_crude_abg <- fit_spline_glm("hypercap_resp_failure", "paco2", subset_data_abg, "ABG",
                                     adj_vars = NULL, analysis_variant = "crude")
fit_imv_crude_vbg <- fit_spline_glm("imv_proc", "vbg_co2", subset_data_vbg, "VBG",
                                    adj_vars = NULL, analysis_variant = "crude")
fit_niv_crude_vbg <- fit_spline_glm("niv_proc", "vbg_co2", subset_data_vbg, "VBG",
                                    adj_vars = NULL, analysis_variant = "crude")
fit_death_crude_vbg <- fit_spline_glm("death_60d", "vbg_co2", subset_data_vbg, "VBG",
                                      adj_vars = NULL, analysis_variant = "crude")
fit_hcrf_crude_vbg <- fit_spline_glm("hypercap_resp_failure", "vbg_co2", subset_data_vbg, "VBG",
```

```r
                                    adj_vars = NULL, analysis_variant = "crude")

if (any(vapply(
  list(
    fit_imv_crude_abg, fit_niv_crude_abg, fit_death_crude_abg, fit_hcrf_crude_abg,
    fit_imv_crude_vbg, fit_niv_crude_vbg, fit_death_crude_vbg, fit_hcrf_crude_vbg
  ),
  is.null,
  logical(1)
))) {
  stop("Crude key-results spline fits failed; see model_fit_diagnostics.csv.")
}

pred_imv_crude_abg <- predict_or_curve_from_fit(fit_imv_crude_abg, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_niv_crude_abg <- predict_or_curve_from_fit(fit_niv_crude_abg, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_death_crude_abg <- predict_or_curve_from_fit(fit_death_crude_abg, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_hcrf_crude_abg <- predict_or_curve_from_fit(fit_hcrf_crude_abg, grid_abg_unw, ref_idx_abg_unw, "paco2")
pred_imv_crude_vbg <- predict_or_curve_from_fit(fit_imv_crude_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_niv_crude_vbg <- predict_or_curve_from_fit(fit_niv_crude_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_death_crude_vbg <- predict_or_curve_from_fit(fit_death_crude_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")
pred_hcrf_crude_vbg <- predict_or_curve_from_fit(fit_hcrf_crude_vbg, grid_vbg_unw, ref_idx_vbg_unw, "vbg_co2")

key_results_crude_or_df <- dplyr::bind_rows(
  bind_key_results_or_curves(
    list(
      imv_proc = pred_imv_crude_abg,
      niv_proc = pred_niv_crude_abg,
      death_60d = pred_death_crude_abg,
      hypercap_resp_failure = pred_hcrf_crude_abg
    ),
    group = "ABG",
    co2_col = "paco2",
    context = "key-results crude OR"
  ),
  bind_key_results_or_curves(
    list(
```

```
      imv_proc = pred_imv_crude_vbg,
      niv_proc = pred_niv_crude_vbg,
      death_60d = pred_death_crude_vbg,
      hypercap_resp_failure = pred_hcrf_crude_vbg
    ),
    group = "VBG",
    co2_col = "vbg_co2",
    context = "key-results crude OR"
  )
)
key_results_crude_prob_df <- dplyr::bind_rows(
  bind_key_results_prob_curves(
    list(
      imv_proc = pred_imv_crude_abg,
      niv_proc = pred_niv_crude_abg,
      death_60d = pred_death_crude_abg,
      hypercap_resp_failure = pred_hcrf_crude_abg
    ),
    group = "ABG",
    co2_col = "paco2",
    context = "key-results crude probability"
  ),
  bind_key_results_prob_curves(
    list(
      imv_proc = pred_imv_crude_vbg,
      niv_proc = pred_niv_crude_vbg,
      death_60d = pred_death_crude_vbg,
      hypercap_resp_failure = pred_hcrf_crude_vbg
    ),
    group = "VBG",
    co2_col = "vbg_co2",
    context = "key-results crude probability"
  )
)

key_results_adjusted_or_df <- dplyr::bind_rows(
```

```
  bind_key_results_or_curves(
    list(
      imv_proc = pred_imv,
      niv_proc = pred_niv,
      death_60d = pred_death,
      hypercap_resp_failure = pred_hcrf
    ),
    group = "ABG",
    co2_col = "paco2",
    context = "key-results adjusted OR"
  ),
  bind_key_results_or_curves(
    list(
      imv_proc = pred_imv_vbg,
      niv_proc = pred_niv_vbg,
      death_60d = pred_death_vbg,
      hypercap_resp_failure = pred_hcrf_vbg
    ),
    group = "VBG",
    co2_col = "vbg_co2",
    context = "key-results adjusted OR"
  )
)
key_results_adjusted_prob_df <- dplyr::bind_rows(
  bind_key_results_prob_curves(
    list(
      imv_proc = pred_imv,
      niv_proc = pred_niv,
      death_60d = pred_death,
      hypercap_resp_failure = pred_hcrf
    ),
    group = "ABG",
    co2_col = "paco2",
    context = "key-results adjusted probability"
  ),
  bind_key_results_prob_curves(
```

```
    list(
      imv_proc = pred_imv_vbg,
      niv_proc = pred_niv_vbg,
      death_60d = pred_death_vbg,
      hypercap_resp_failure = pred_hcrf_vbg
    ),
    group = "VBG",
    co2_col = "vbg_co2",
    context = "key-results adjusted probability"
  )
)

key_results_imputed_or_df <- dplyr::bind_rows(
  normalize_key_results_or_df(
    mi_unweighted_abg_curves,
    co2_col = "paco2",
    context = "key-results imputed OR"
  ),
  normalize_key_results_or_df(
    mi_unweighted_vbg_curves,
    co2_col = "vbg_co2",
    context = "key-results imputed OR"
  )
)
key_results_imputed_prob_df <- dplyr::bind_rows(
  normalize_key_results_prob_df(
    mi_unweighted_abg_prob_curves,
    co2_col = "paco2",
    context = "key-results imputed probability"
  ),
  normalize_key_results_prob_df(
    mi_unweighted_vbg_prob_curves,
    co2_col = "vbg_co2",
    context = "key-results imputed probability"
  )
)
```

```r
key_results_main_mi_ipw_or_df <- dplyr::bind_rows(
  normalize_key_results_or_df(abg_curves, co2_col = "paco2", context = "key-results main MI IPW OR"),
  normalize_key_results_or_df(vbg_curves, co2_col = "vbg_co2", context = "key-results main MI IPW OR")
)
key_results_main_mi_ipw_prob_df <- dplyr::bind_rows(
  normalize_key_results_prob_df(abg_prob_curves, co2_col = "paco2", context = "key-results main MI IPW probability"),
  normalize_key_results_prob_df(vbg_prob_curves, co2_col = "vbg_co2", context = "key-results main MI IPW probability")
)

key_results_gbm_weighted_or_df <- normalize_key_results_or_df(
  ipw_curve_overlay,
  context = "key-results GBM-weighted OR"
)
key_results_gbm_weighted_prob_df <- normalize_key_results_prob_df(
  ipw_curve_overlay_prob,
  context = "key-results GBM-weighted probability"
)

key_results_caption_crude <- paste0(
  "Odds ratio curves are relative to the test-specific reference CO2 values (ABG ",
  ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF,
  " mmHg). Predicted probability curves are unadjusted model-based probabilities within the tested cohorts."
)
key_results_caption_adjusted <- paste0(
  "Odds ratio curves are relative to the test-specific reference CO2 values (ABG ",
  ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF,
  " mmHg). Predicted probability curves are conditional on the reference covariate profile X_ref within each
  ↪ test-specific cohort."
)
key_results_caption_imputed <- paste0(
  "Odds ratio curves are relative to the test-specific reference CO2 values (ABG ",
  ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF,
  " mmHg). Predicted probability curves are conditional on the reference covariate profile X_ref within each
  ↪ test-specific cohort."
)
```

```r
key_results_caption_main <- paste0(
  "These curves come from the primary MI-logistic IPSW analysis. ",
  "Odds ratio curves are relative to the test-specific reference CO2 values (ABG ",
  ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF,
  " mmHg). Predicted probability curves are marginally standardized to the common eligible source-population covariate
   ↪ distribution."
)
key_results_caption_gbm <- paste0(
  "These curves come from the non-MI GBM IPSW sensitivity analysis. ",
  "Odds ratio curves are relative to the test-specific reference CO2 values (ABG ",
  ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF,
  " mmHg). Predicted probability curves are conditional on the reference covariate profile X_ref within each
   ↪ test-specific cohort."
)

key_results_crude_plot <- build_key_results_figure(
  key_results_crude_or_df,
  key_results_crude_prob_df,
  figure_title = "Crude spline associations: ABG vs VBG CO2",
  figure_caption = key_results_caption_crude
)
key_results_adjusted_plot <- build_key_results_figure(
  key_results_adjusted_or_df,
  key_results_adjusted_prob_df,
  figure_title = "Unweighted covariate-adjusted spline associations of pCO2 with clinical outcomes by ABG and VBG",
  figure_caption = NULL
)
key_results_imputed_plot <- build_key_results_figure(
  key_results_imputed_or_df,
  key_results_imputed_prob_df,
  figure_title = "MI-pooled adjusted spline associations (non-weighted): ABG vs VBG CO2",
  figure_caption = key_results_caption_imputed
)
key_results_main_mi_ipw_plot <- build_key_results_figure(
  key_results_main_mi_ipw_or_df,
  key_results_main_mi_ipw_prob_df,
```

```
  figure_title = "Primary MI-logistic IPSW-weighted spline associations of pCO2 with clinical outcomes by ABG and
   ↪  VBG",
  figure_caption = NULL
)
key_results_gbm_weighted_plot <- build_key_results_figure(
  key_results_gbm_weighted_or_df,
  key_results_gbm_weighted_prob_df,
  figure_title = "GBM-IPSW-adjusted spline associations (non-MI): ABG vs VBG CO2",
  figure_caption = key_results_caption_gbm
)
```

```
# Purpose: canonical key-results figure for crude spline models.
print_plot_once(
  key_results_crude_plot,
  "key-results-spline-crude-abg-vbg",
  width = KEY_RESULTS_FIG_WIDTH,
  height = KEY_RESULTS_FIG_HEIGHT
)
```

```
# Purpose: manuscript Figure S5 unweighted covariate-adjusted spline associations by ABG and VBG.
print_plot_once(
  key_results_adjusted_plot,
  "key-results-spline-adjusted-abg-vbg",
  width = KEY_RESULTS_FIG_WIDTH,
  height = KEY_RESULTS_FIG_HEIGHT
)
```

```
# Purpose: canonical key-results figure for imputed non-weighted spline models.
print_plot_once(
  key_results_imputed_plot,
  "key-results-spline-imputed-abg-vbg",
  width = KEY_RESULTS_FIG_WIDTH,
  height = KEY_RESULTS_FIG_HEIGHT
)
```

```
# Purpose: manuscript Figure 2 primary weighted spline associations.
print_plot_once(
  key_results_main_mi_ipw_plot,
```

```
  "key-results-spline-main-mi-ipw-abg-vbg",
  width = KEY_RESULTS_FIG_WIDTH,
  height = KEY_RESULTS_FIG_HEIGHT
)
```

```
# Purpose: canonical key-results figure for GBM-weighted spline models.
print_plot_once(
  key_results_gbm_weighted_plot,
  "key-results-spline-gbm-weighted-abg-vbg",
  width = KEY_RESULTS_FIG_WIDTH,
  height = KEY_RESULTS_FIG_HEIGHT
)
```

```
# Purpose: float barrier mi spline.
float_barrier("section")
```

```
# Purpose: ipw output parity.
non_mi_ipw_outputs <- tibble::tribble(
  ~parity_key, ~artifact_type, ~artifact_name,
  "threelevel_or", "figure", "or-plot-three-level-weighted.png",
  "key_results_figure", "figure", "key-results-spline-gbm-weighted-abg-vbg.png",
  "categorical_key_results_figure", "figure", "key-results-cat3-gbm-weighted-abg-vbg.png",
  "abg_spline_panel", "figure", "spline-ipw-abg-trimmed.png",
  "vbg_spline_panel", "figure", "spline-ipw-vbg.png",
  "abg_shared_spline", "figure", "spline-ipw-abg-shared.png",
  "overlay_spline", "figure", "spline-ipw-overlay-abg-vbg.png",
  "abg_spline_panel_prob", "figure", "spline-ipw-abg-trimmed-prob.png",
  "vbg_spline_panel_prob", "figure", "spline-ipw-vbg-prob.png",
  "abg_shared_spline_prob", "figure", "spline-ipw-abg-shared-prob.png",
  "overlay_spline_prob", "figure", "spline-ipw-overlay-abg-vbg-prob.png",
  "propensity_hist_tested", "figure", "propensity-histograms-conditional.png",
  "shap_top10_plot", "figure", "shap-top10-ipw-gbm-abg-vbg.png",
  "shap_top10_abg", "table_csv", "shap_top10_ipw_gbm_abg.csv",
  "shap_top10_vbg", "table_csv", "shap_top10_ipw_gbm_vbg.csv",
  "love_plot", "figure", "loveplot-ipw-gbm-abg-vbg.png",
  "love_plot_data", "table_csv", "loveplot_ipw_gbm_data.csv",
  "threelevel_table", "table_csv", "table_summary_adjusted_threelevel.csv",
```

```r
  "categorical_key_results_table", "table_csv", "table_summary_key_results_cat3.csv",
  "weighting_diag", "table_csv", "weighting_diagnostics_non_mi.csv"
)

mi_ipw_outputs <- tibble::tribble(
  ~parity_key, ~artifact_type, ~artifact_name,
  "threelevel_or", "figure", "or-plot-mi-weighted.png",
  "threelevel_prob", "figure", "ipw-three-level-mi-abg-vbg-prob.png",
  "key_results_figure", "figure", "key-results-spline-main-mi-ipw-abg-vbg.png",
  "categorical_key_results_figure", "figure", "key-results-cat3-main-mi-ipw-abg-vbg.png",
  "overlay_spline", "figure", "ipw-rcs-overlay-mi-abg-vbg.png",
  "overlay_spline_prob", "figure", "ipw-rcs-overlay-mi-abg-vbg-prob.png",
  "mi_cat3_prob_curve_abg", "table_csv", "mi_cat3_prob_curve_abg.csv",
  "mi_cat3_prob_curve_vbg", "table_csv", "mi_cat3_prob_curve_vbg.csv",
  "mi_spline_curve_abg", "table_csv", "mi_spline_curve_abg.csv",
  "mi_spline_curve_vbg", "table_csv", "mi_spline_curve_vbg.csv",
  "mi_spline_prob_curve_abg", "table_csv", "mi_spline_prob_curve_abg.csv",
  "mi_spline_prob_curve_vbg", "table_csv", "mi_spline_prob_curve_vbg.csv",
  "propensity_hist_tested", "figure", "propensity-histograms-mi-logistic.png",
  "shap_top10_plot", "figure", "shap-top10-mi-logistic-abg-vbg.png",
  "shap_top10_abg", "table_csv", "shap_top10_mi_logistic_abg.csv",
  "shap_top10_vbg", "table_csv", "shap_top10_mi_logistic_vbg.csv",
  "love_plot", "figure", "loveplot-mi-logistic-abg-vbg.png",
  "love_plot_data", "table_csv", "loveplot_mi_logistic_data.csv",
  "threelevel_table", "table_csv", "mi_threelevel_results.csv",
  "categorical_key_results_table", "table_csv", "table_summary_key_results_cat3.csv",
  "ps_model_store_abg", "diagnostic_rds", "mi_logistic_ps_abg_list.rds",
  "ps_model_store_vbg", "diagnostic_rds", "mi_logistic_ps_vbg_list.rds"
)

# helper: test whether one parity-audit artifact is present in the expected
# figures or results directory.
artifact_exists <- function(name, type) {
  if (is.na(name) || !nzchar(name)) return(FALSE)
  if (type == "figure") {
    fig_path <- results_path("figs", name)
```

```r
    if (grepl("[*?\\[]", name)) return(length(Sys.glob(fig_path)) > 0)
    return(file.exists(fig_path))
  }
  file.exists(results_path(name))
}

parity_tbl <- dplyr::full_join(
  dplyr::rename(non_mi_ipw_outputs, non_mi_artifact = artifact_name, artifact_type_non_mi = artifact_type),
  dplyr::rename(mi_ipw_outputs, mi_artifact = artifact_name, artifact_type_mi = artifact_type),
  by = "parity_key"
) |>
  dplyr::mutate(
    artifact_type = dplyr::coalesce(artifact_type_non_mi, artifact_type_mi),
    present_non_mi = mapply(artifact_exists, non_mi_artifact, artifact_type, USE.NAMES = FALSE),
    present_mi = mapply(artifact_exists, mi_artifact, artifact_type, USE.NAMES = FALSE),
    parity_status = dplyr::case_when(
      present_non_mi & present_mi ~ "both",
      present_non_mi & !present_mi ~ "non_mi_only",
      !present_non_mi & present_mi ~ "mi_only",
      TRUE ~ "unknown"
    ),
    recommended_action = dplyr::case_when(
      parity_status == "both" ~ "none",
      parity_status == "non_mi_only" ~ "add MI equivalent output",
      parity_status == "mi_only" ~ "add non-MI equivalent output",
      TRUE ~ "review manually"
    )
  ) |>
  dplyr::arrange(artifact_type, parity_key) |>
  dplyr::select(parity_key, artifact_type, non_mi_artifact, mi_artifact,
                present_non_mi, present_mi, parity_status, recommended_action)

write_csv_safely(parity_tbl, results_path("ipw_output_parity.csv"), row_names = FALSE)

parity_unknown <- parity_tbl |>
  dplyr::filter(parity_status == "unknown")
```

```r
write_csv_safely(parity_unknown, results_path("ipw_output_parity_unknown.csv"), row_names = FALSE)
if (RUN_MODE == "full" && nrow(parity_unknown) > 0) {
  stop("Parity audit has unknown rows; see ", results_path("ipw_output_parity_unknown.csv"))
}

parity_one_sided <- parity_tbl |>
  dplyr::filter(parity_status != "both")

write_csv_safely(parity_one_sided, results_path("ipw_output_parity_diffs.csv"), row_names = FALSE)

render_table_pdf_maybe(
  parity_one_sided,
  caption = "GBM IPSW vs MI-logistic IPSW output parity (artifacts present in only one track)",
  file_stub = "ipw_output_parity_one_sided",
  digits = 0,
  show = FALSE
)
```

```r
# Purpose: verify the additional canonical key-results artifacts across spline and categorical tracks.
key_results_outputs <- tibble::tribble(
  ~family, ~track, ~artifact_type, ~artifact_name,
  "Spline", "Crude", "figure", "key-results-spline-crude-abg-vbg.png",
  "Spline", "Adjusted", "figure", "key-results-spline-adjusted-abg-vbg.png",
  "Spline", "Imputed", "figure", "key-results-spline-imputed-abg-vbg.png",
  "Categorical", "Crude", "figure", "key-results-cat3-crude-abg-vbg.png",
  "Categorical", "Adjusted", "figure", "key-results-cat3-adjusted-abg-vbg.png",
  "Categorical", "Imputed", "figure", "key-results-cat3-imputed-abg-vbg.png",
  "Categorical", "Weighted and adjusted", "figure", "key-results-cat3-main-mi-ipw-abg-vbg.png",
  "Categorical", "GBM-weighted", "figure", "key-results-cat3-gbm-weighted-abg-vbg.png",
  "Categorical", "Master table", "table_csv", "table_summary_key_results_cat3.csv",
  "Categorical", "ABG table", "table_csv", "table_summary_key_results_cat3_abg.csv",
  "Categorical", "VBG table", "table_csv", "table_summary_key_results_cat3_vbg.csv",
  "Categorical", "Long-form extract", "table_csv", "cat3_key_results_long.csv"
) |>
  dplyr::mutate(
    artifact_file = dplyr::if_else(
      artifact_type == "figure",
```

```r
      results_path("figs", artifact_name),
      results_path(artifact_name)
    ),
    artifact_path = pretty_path(artifact_file),
    exists = file.exists(artifact_file)
  )

key_results_outputs |>
  dplyr::select(-artifact_file)
```

family	track	artifact_type	artifact_name	artifact_path	exists
Spline	Crude	figure	key-results-spline-crude-abg-vbg.png	Results/figs/key-results-spline-crude-abg-vbg.png	TRUE
Spline	Adjusted	figure	key-results-spline-adjusted-abg-vbg.png	Results/figs/key-results-spline-adjusted-abg-vbg.png	TRUE
Spline	Imputed	figure	key-results-spline-imputed-abg-vbg.png	Results/figs/key-results-spline-imputed-abg-vbg.png	TRUE
Categorical	Crude	figure	key-results-cat3-crude-abg-vbg.png	Results/figs/key-results-cat3-crude-abg-vbg.png	TRUE
Categorical	Adjusted	figure	key-results-cat3-adjusted-abg-vbg.png	Results/figs/key-results-cat3-adjusted-abg-vbg.png	TRUE
Categorical	Imputed	figure	key-results-cat3-imputed-abg-vbg.png	Results/figs/key-results-cat3-imputed-abg-vbg.png	TRUE
Categorical	Weighted and adjusted	figure	key-results-cat3-main-mi-ipw-abg-vbg.png	Results/figs/key-results-cat3-main-mi-ipw-abg-vbg.png	TRUE
Categorical	GBM-weighted	figure	key-results-cat3-gbm-weighted-abg-vbg.png	Results/figs/key-results-cat3-gbm-weighted-abg-vbg.png	TRUE
Categorical	Master table	table_csv	table_summary_key_results_cat3.csv	Results/table_summary_key_results_cat3.csv	TRUE
Categorical	ABG table	table_csv	table_summary_key_results_cat3_abg.csv	Results/table_summary_key_results_cat3_abg.csv	TRUE
Categorical	VBG table	table_csv	table_summary_key_results_cat3_vbg.csv	Results/table_summary_key_results_cat3_vbg.csv	TRUE
Categorical	Long-form extract	table_csv	cat3_key_results_long.csv	Results/cat3_key_results_long.csv	TRUE

```r
# Purpose: generate Supplementary Tables S1, S4, and S5 from notebook state.
table_s1_inclusion_criteria <- tibble::tibble(
  step = seq_len(5L),
  criterion = c(
    "Source extract",
    "ED/inpatient encounters",
    "ABG-tested cohort",
    "VBG-tested cohort",
    "Target population"
  ),
  `Source field / rule` = c(
    "TriNetX-derived 2022 extract",
    "encounter_type != ambulatory",
    "has_abg == 1",
    "has_vbg == 1",
    "ED/inpatient analytic cohort"
  ),
  `Excluded, n` = scales::comma(c(
    0L,
    cohort_flow_counts$excluded_ambulatory,
    cohort_flow_counts$analytic_ed_inpatient - cohort_flow_counts$abg_tested,
    cohort_flow_counts$analytic_ed_inpatient - cohort_flow_counts$vbg_tested,
    0L
  )),
  `Retained, n` = scales::comma(c(
    cohort_flow_counts$total_extracted,
    cohort_flow_counts$analytic_ed_inpatient,
    cohort_flow_counts$abg_tested,
    cohort_flow_counts$vbg_tested,
    cohort_flow_counts$analytic_ed_inpatient
  )),
  notes = c(
    "Initial local source cohort.",
    "Matches Figure 1 cohort-flow restriction.",
    "Measured arterial CO2 outcome models.",
    "Measured venous CO2 outcome models.",
```

```
    "Target for IPSW standardization."
  )
)

write_standalone_table_pdf(
  table_s1_inclusion_criteria,
  caption = "Inclusion criteria into the enriched sample",
  file_stub = "table_s1_inclusion_criteria",
  wide = TRUE,
  digits = 0,
  max_cols = 6,
  landscape = TRUE,
  display_label = "Table S1"
)

baseline_role_vars <- intersect(covars_ps, names(subset_data_raw))
baseline_role_labels <- c(
  age_at_encounter = "Age",
  curr_bmi = "BMI",
  sex = "Sex",
  race_ethnicity = "Race/ethnicity",
  location = "Location",
  encounter_type = "Encounter type",
  copd = "COPD",
  asthma = "Asthma",
  osa = "OSA",
  chf = "CHF",
  acute_nmd = "Neuromuscular disease",
  phtn = "Pulmonary hypertension",
  ckd = "Chronic kidney disease",
  dm = "Diabetes",
  temp_new = "Temperature",
  sbp = "Systolic blood pressure",
  dbp = "Diastolic blood pressure",
  hr = "Heart rate",
  sodium = "Sodium",
```

```r
  serum_cr = "Serum creatinine",
  serum_hco3 = "Serum bicarbonate",
  serum_cl = "Serum chloride",
  serum_lac = "Serum lactate",
  serum_k = "Serum potassium",
  wbc = "White blood cell count",
  plt = "Platelet count",
  serum_phos = "Serum phosphate",
  serum_ca = "Serum calcium"
)

role_for_var <- function(v) {
  roles <- character()
  if (v %in% adj_core) roles <- c(roles, "direct adjustment")
  if (v %in% covars_ps) roles <- c(roles, "PS model")
  if (!length(roles)) roles <- "auxiliary"
  paste(unique(roles), collapse = "; ")
}

missing_lookup <- miss_tbl |>
  dplyr::select(.data$variable, .data$pct_missing)

table_s4_missingness_primary_analysis <- tibble::tibble(variable = baseline_role_vars) |>
  dplyr::left_join(missing_lookup, by = "variable") |>
  dplyr::mutate(
    variable = dplyr::recode(.data$variable, !!!baseline_role_labels, .default = .data$variable),
    `Analysis role` = vapply(baseline_role_vars, role_for_var, character(1L)),
    `% missing` = sprintf("%.1f%%", .data$pct_missing),
    `Imputed in primary MI` = dplyr::if_else(
      dplyr::coalesce(meth[baseline_role_vars], "") != "",
      "Yes",
      "No"
    )
  ) |>
  dplyr::select(.data$variable, .data$`Analysis role`, .data$`% missing`, .data$`Imputed in primary MI`)
```

```r
write_standalone_table_pdf(
  table_s4_missingness_primary_analysis,
  caption = "Missingness of baseline covariates used in the primary analysis",
  file_stub = "table_s4_missingness_primary_analysis",
  wide = FALSE,
  digits = 1,
  max_cols = 4,
  landscape = FALSE,
  display_label = "Table S4"
)

diag_df_for_s5 <- mi_outcome_diag
diag_df_for_s5$sep_warn <- grepl("fitted probabilities numerically 0 or 1",
                                 diag_df_for_s5$top_warning, fixed = TRUE) |
  grepl("separat", diag_df_for_s5$top_warning, ignore.case = TRUE)
diag_df_for_s5$phat_extreme <- (diag_df_for_s5$min_phat < PROB_EPS) |
  (diag_df_for_s5$max_phat > 1 - PROB_EPS)
diag_df_for_s5$sep_flag <- dplyr::coalesce(as.logical(diag_df_for_s5$sep_flag), FALSE)
diag_df_for_s5$nonconv_flag <- dplyr::coalesce(as.logical(diag_df_for_s5$nonconv_flag), FALSE) |
  dplyr::coalesce(!as.logical(diag_df_for_s5$converged), FALSE)

mi_variants_for_s5 <- intersect(c("mi_ipw", "weighted_imputed"), unique(diag_df_for_s5$analysis_variant))
if (!length(mi_variants_for_s5)) mi_variants_for_s5 <- unique(diag_df_for_s5$analysis_variant)

issue_summary <- diag_df_for_s5 |>
  dplyr::filter(analysis_variant %in% mi_variants_for_s5) |>
  dplyr::group_by(group, outcome, component, analysis_variant) |>
  dplyr::summarise(
    n_fits = dplyr::n(),
    n_nonconv = sum(nonconv_flag %in% TRUE, na.rm = TRUE),
    n_sep_warn = sum(sep_warn %in% TRUE, na.rm = TRUE),
    n_phat_extreme = sum(phat_extreme %in% TRUE, na.rm = TRUE),
    n_sep_flag = sum(sep_flag %in% TRUE, na.rm = TRUE),
    .groups = "drop"
  )
write_csv_safely(issue_summary, results_path("mi_fit_issue_summary.csv"), row_names = FALSE)
```

```r
safe_max <- function(x) {
  x <- x[is.finite(x)]
  if (!length(x)) return(NA_real_)
  max(x)
}

table_s5_mi_diagnostic_summary <- tibble::tibble(
  domain = c(
    "Imputation run",
    "Imputation run",
    "Imputation run",
    "Monte Carlo error",
    "Chain diagnostics",
    "Chain diagnostics",
    "MICE logged events",
    "Captured warnings",
    "Outcome fit diagnostics",
    "Outcome fit diagnostics",
    "Outcome fit diagnostics"
  ),
  metric = c(
    "Completed imputations",
    "Iterations per imputation",
    "Variables imputed",
    "Maximum MC error / SE in diagnostic model",
    "Variables with available chain diagnostics",
    "Variables flagged for drift",
    "Total logged events",
    "Captured warning rows",
    "MI model fits audited",
    "Fits with nonconvergence flags",
    "Fits with separation/extreme-probability flags"
  ),
  value = c(
    as.character(imp$m),
```

```r
    as.character(MAXIT_RUN),
    as.character(length(impute_vars)),
    ifelse(is.finite(safe_max(mc_tab$mc_err_over_se)), sprintf("%.3f", safe_max(mc_tab$mc_err_over_se)), "Not
     ↪  estimable"),
    as.character(nrow(chain_diag)),
    as.character(sum(chain_diag$flag %in% TRUE, na.rm = TRUE)),
    as.character(sum(log_events_summary$n, na.rm = TRUE)),
    as.character(nrow(mi_warn_log)),
    as.character(nrow(diag_df_for_s5)),
    as.character(sum(diag_df_for_s5$nonconv_flag %in% TRUE, na.rm = TRUE)),
    as.character(sum(diag_df_for_s5$sep_flag %in% TRUE | diag_df_for_s5$phat_extreme %in% TRUE, na.rm = TRUE))
  ),
  interpretation = c(
    "Primary analysis imputation count used in this render.",
    "Configured MICE iteration count for this render.",
    "Variables with non-empty MICE imputation methods and observed missingness.",
    "Diagnostic model ratio; lower values indicate less simulation noise relative to model SE.",
    "Imputed variables with traceable chain-mean diagnostics.",
    "Variables exceeding drift/slope screening thresholds.",
    "Rows reported by mice::loggedEvents across batches.",
    "Warnings captured by notebook-level diagnostics wrappers.",
    "Outcome-model diagnostic rows captured for MI analyses.",
    "Potential convergence problems requiring review if nonzero.",
    "Known separation/extreme-probability screen; interpret affected estimates cautiously."
  )
)

write_standalone_table_pdf(
  table_s5_mi_diagnostic_summary,
  caption = "Multiple-imputation diagnostic summary",
  file_stub = "table_s5_mi_diagnostic_summary",
  wide = TRUE,
  digits = 3,
  max_cols = 4,
  landscape = TRUE,
  display_label = "Table S5"
```

```
)

# Purpose: freeze manuscript-facing numbering, paths, aliases, and targeted duplicate cleanup.
# helper: delete only the known duplicate spill files for mapped supplement
# artifacts, leaving unrelated historical files untouched.
cleanup_targeted_supplement_duplicates <- function() {
  duplicate_map <- tibble::tribble(
    ~canonical_file, ~duplicate_file,
    results_path("figs", "propensity-histograms-conditional.png"), results_path("figs",
     ↪  "propensity-histograms-conditional-1.png"),
    results_path("figs", "shap-top10-ipw-gbm-abg-vbg.png"), results_path("figs", "shap-top10-ipw-gbm-display-1.png")
  )

  duplicate_map |>
    dplyr::rowwise() |>
    dplyr::mutate(
      remove_ok = file.exists(canonical_file) && file.exists(duplicate_file)
    ) |>
    dplyr::ungroup() |>
    dplyr::filter(remove_ok) |>
    dplyr::pull(duplicate_file) |>
    unique() |>
    (\(paths) {
      if (length(paths)) fs::file_delete(paths)
      invisible(paths)
    })()
}

cleanup_targeted_supplement_duplicates()

# helper: convert an internal absolute artifact path into the shorter manifest
# path used in CSV audits and drafting.
format_manifest_path <- function(path) {
  if (is.null(path) || is.na(path) || !nzchar(path)) return("")
  pretty_path(path)
}
```

```r
# helper: split comma-separated extension declarations into a stable vector.
split_expected_extensions <- function(x) {
  if (is.null(x) || is.na(x) || !nzchar(x)) return(character(0))
  trimws(strsplit(x, ",", fixed = TRUE)[[1]]) |>
    (\(vals) vals[nzchar(vals)])()
}

# helper: write markdown reports without prepending the diagnostics log header.
write_markdown_report <- function(lines, path) {
  writeLines(lines, con = path)
  invisible(TRUE)
}

# helper: collapse repeated strings for compact audit detail columns.
collapse_unique_text <- function(x) {
  x <- unique(as.character(x))
  x <- x[!is.na(x) & nzchar(x)]
  paste(x, collapse = "; ")
}

# Canonical artifact names stay analysis-oriented for code stability, while
# alias artifact names are manuscript-facing and numbered/titled for drafting.
# All provenance/check outputs are derived from this one registry so numbering,
# paths, and validation display cannot silently drift apart.
manuscript_asset_manifest_internal <- tibble::tribble(
  ~manuscript_label, ~numbering_slot, ~title, ~kind, ~section, ~chunk_label, ~status, ~source_type,
   ↪  ~required_in_validation, ~expected_extensions, ~pdf_display, ~deprecated_replacement,
   ↪  ~missing_artifact_severity, ~notes, ~analysis_family, ~canonical_subdir, ~canonical_stub, ~alias_subdir,
   ↪  ~alias_stub, ~primary_ext,
  "Figure 1", "Figure 1", "Cohort assembly and target population for inverse probability-of-sampling weighting",
   ↪  "figure", "main", "cohort-flow-figure", "generated", "notebook_generated", TRUE, "png,pdf", TRUE, "", "fail",
   ↪  "Primary cohort-flow display; .pdf sidecar also generated.", "main", "", "cohort_flow", "figs",
   ↪  "figure_1_cohort_assembly_target_population_ipsw", "png",
```

```
"Figure 2", "Figure 2", "Primary MI-logistic IPSW-weighted spline associations of pCO2 with clinical outcomes by ABG
 ↪  and VBG", "figure", "main", "key-results-spline-main-mi-ipw-abg-vbg", "generated", "notebook_generated", TRUE,
 ↪  "png,pdf", TRUE, "", "fail", "Combined manuscript-facing weighted spline figure from the primary MI-logistic
 ↪  IPSW analysis; predicted probabilities are common-source marginal standardized; .pdf sidecar also generated.",
 ↪  "main_mi_ipw_spline", "figs", "key-results-spline-main-mi-ipw-abg-vbg", "figs",
 ↪  "figure_2_primary_weighted_spline_associations_abg_vbg", "png",
"Table 1", "Table 1", "Baseline characteristics of the analytic cohort overall and by receipt of arterial and venous
 ↪  blood gas testing", "table", "main", "table1-everyone-abg-vbg", "generated", "notebook_generated", TRUE,
 ↪  "csv,pdf", TRUE, "", "fail", "Rendered from the canonical tbl1/tbl1_combined_csv objects; paired CSV also
 ↪  generated.", "baseline", "", "table1_combined", "", "table_1_baseline_characteristics_analytic_cohort", "pdf",
"Table 2", "Table 2", "MI-pooled, MI-logistic IPSW-weighted 3-level categorical results for ABG and VBG", "table",
 ↪  "main", "table2-weighted-categorical-outcomes", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "",
 ↪  "fail", "Main manuscript table for weighted categorical ORs and common-source marginal standardized predicted
 ↪  probabilities across ABG and VBG.", "main_mi_weighted_categorical", "", "table_2_weighted_categorical_outcomes",
 ↪  "", "table_2_weighted_categorical_outcomes", "pdf",
"Figure S1", "Figure S1", "Covariate balance after MI logistic inverse-probability weighting for ABG and VBG
 ↪  test-ordering models", "figure", "supplement", "loveplot-mi-logistic", "generated", "notebook_generated", TRUE,
 ↪  "png,pdf", TRUE, "", "fail", "Primary balance diagnostic for the MI logistic IPSW analysis.",
 ↪  "mi_logistic_primary", "figs", "loveplot-mi-logistic-abg-vbg", "figs",
 ↪  "figure_s1_covariate_balance_mi_logistic_ipsw", "png",
"Figure S2", "Figure S2", "Propensity-score overlap for MI logistic ABG and VBG test-ordering models", "figure",
 ↪  "supplement", "mi-propensity-histograms-logistic", "generated", "notebook_generated", TRUE, "png,pdf", TRUE, "",
 ↪  "fail", "Imputation-averaged tested-versus-untested propensity-score distributions under the MI logistic IPSW
 ↪  models.", "mi_logistic_primary", "figs", "propensity-histograms-mi-logistic", "figs",
 ↪  "figure_s2_propensity_score_overlap_mi_logistic", "png",
"Figure S3", "Figure S3", "SHAP-style contribution summaries for MI logistic ABG and VBG test-ordering models",
 ↪  "figure", "supplement", "shap-top10-mi-logistic-display", "generated", "notebook_generated", TRUE, "png,pdf",
 ↪  TRUE, "", "fail", "Mean absolute contribution summaries from the MI logistic IPSW test-ordering models.",
 ↪  "mi_logistic_primary", "figs", "shap-top10-mi-logistic-abg-vbg", "figs",
 ↪  "figure_s3_shap_style_contributions_mi_logistic", "png",
"Figure S4", "Figure S4", "MI-logistic IPSW-weighted categorical associations of pCO2 category with clinical
 ↪  outcomes by ABG and VBG", "figure", "supplement", "key-results-cat3-main-mi-ipw-abg-vbg", "generated",
 ↪  "notebook_generated", TRUE, "png,pdf", TRUE, "Figure 3", "fail", "Weighted categorical display from the primary
 ↪  MI-logistic IPSW analysis with common-source marginal standardized predicted probabilities, intentionally
 ↪  harmonized as Figure S4 rather than a main-text Figure 3.", "mi_weighted_categorical", "figs",
 ↪  "key-results-cat3-main-mi-ipw-abg-vbg", "figs", "figure_s4_weighted_categorical_associations_abg_vbg", "png",
```

```
"Figure S5", "Figure S5", "Unweighted covariate-adjusted spline associations of pCO2 with clinical outcomes by ABG
 ↪  and VBG", "figure", "supplement", "key-results-spline-adjusted-abg-vbg", "generated", "notebook_generated",
 ↪  TRUE, "png,pdf", TRUE, "", "fail", "Combined adjusted complete-case spline figure; .pdf sidecar also
 ↪  generated.", "unweighted_adjusted", "figs", "key-results-spline-adjusted-abg-vbg", "figs",
 ↪  "figure_s5_unweighted_adjusted_spline_abg_vbg", "png",
"Table S2", "Table S2", "Crude associations of pCO2 category with clinical outcomes by ABG and VBG", "table",
 ↪  "supplement", "supplement-table-s2-crude", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "", "fail",
 ↪  "Standalone crude categorical odds-ratio table; paired CSV also generated.", "crude", "",
 ↪  "table_s2_crude_threelevel", "", "table_s2_crude_threelevel", "pdf",
"Table S3", "Table S3", "GBM IPSW-weighted associations of pCO2 category with clinical outcomes by ABG and VBG",
 ↪  "table", "supplement", "supplement-table-s3-gbm", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "",
 ↪  "fail", "Standalone GBM IPSW-weighted categorical odds-ratio table; paired CSV also generated.",
 ↪  "gbm_sensitivity", "", "table_s3_gbm_threelevel", "", "table_s3_gbm_threelevel", "pdf",
"Figure S6", "Figure S6", "Covariate balance after gradient-boosted propensity weighting for ABG and VBG
 ↪  test-ordering models", "figure", "supplement", "loveplot-ipw-gbm", "generated", "notebook_generated", TRUE,
 ↪  "png,pdf", TRUE, "", "fail", "Raw-versus-weighted absolute standardized mean differences; dashed line marks
 ↪  |SMD| = 0.1.", "gbm_sensitivity", "figs", "loveplot-ipw-gbm-abg-vbg", "figs",
 ↪  "figure_s6_covariate_balance_gbm_weighting", "png",
"Figure S7", "Figure S7", "Propensity-score overlap for gradient-boosted ABG and VBG test-ordering models",
 ↪  "figure", "supplement", "propensity-histograms-conditional", "generated", "notebook_generated", TRUE, "png,pdf",
 ↪  TRUE, "", "fail", "Tested-versus-untested propensity-score distributions under the GBM sensitivity models.",
 ↪  "gbm_sensitivity", "figs", "propensity-histograms-conditional", "figs",
 ↪  "figure_s7_propensity_score_overlap_gbm", "png",
"Figure S8", "Figure S8", "SHAP-style contribution summaries for gradient-boosted ABG and VBG test-ordering models",
 ↪  "figure", "supplement", "shap-top10-ipw-gbm-display", "generated", "notebook_generated", TRUE, "png,pdf", TRUE,
 ↪  "", "fail", "Mean absolute SHAP value summaries; not interpreted as independent predictors.", "gbm_sensitivity",
 ↪  "figs", "shap-top10-ipw-gbm-abg-vbg", "figs", "figure_s8_shap_style_contributions_gbm", "png",
"Table S1", "Table S1", "Inclusion criteria into the enriched sample", "table", "supplement",
 ↪  "manuscript-supplement-tables-additional", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "", "fail",
 ↪  "Notebook-generated inclusion/restriction table derived from the same cohort-flow logic as Figure 1.",
 ↪  "inclusion", "", "table_s1_inclusion_criteria", "", "table_s1_inclusion_criteria", "pdf",
"Table S4", "Table S4", "Missingness of baseline covariates used in the primary analysis", "table", "supplement",
 ↪  "manuscript-supplement-tables-additional", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "", "fail",
 ↪  "Notebook-generated missingness table for baseline covariates used in the primary PS/direct-adjustment
 ↪  analysis.", "missingness", "", "table_s4_missingness_primary_analysis", "",
 ↪  "table_s4_missingness_primary_analysis", "pdf",
```

```r
  "Table S5", "Table S5", "Multiple-imputation diagnostic summary", "table", "supplement",
   ↪  "manuscript-supplement-tables-additional", "generated", "notebook_generated", TRUE, "csv,pdf", TRUE, "", "fail",
   ↪  "Compact manuscript-facing summary of MI execution, chain, Monte Carlo error, logged-event, warning, and
   ↪  outcome-fit diagnostics.", "mi_diagnostics", "", "table_s5_mi_diagnostic_summary", "",
   ↪  "table_s5_mi_diagnostic_summary", "pdf",
  "Figure 3", "Figure 3", "Deprecated draft-only reference for weighted categorical results", "figure", "draft_only",
   ↪  "", "draft_only", "suppressed/deprecated", FALSE, "", FALSE, "Figure S4", "suppressed", "Weighted categorical
   ↪  display is intentionally harmonized as Figure S4; no Figure 3 alias is generated.", "deprecated", "", "", "",
   ↪  "", ""
) |>
  dplyr::mutate(
    manuscript_label_safe = safe_manuscript_text(manuscript_label),
    title_safe = safe_manuscript_text(title),
    caption_safe = manuscript_caption(manuscript_label, title),
    expected_extension_list = purrr::map(expected_extensions, split_expected_extensions),
    canonical_artifact_file = mapply(
      artifact_file_path,
      stub = canonical_stub,
      ext = primary_ext,
      subdir = canonical_subdir,
      USE.NAMES = FALSE
    ),
    alias_artifact_file = mapply(
      artifact_file_path,
      stub = alias_stub,
      ext = primary_ext,
      subdir = alias_subdir,
      USE.NAMES = FALSE
    )
  )

# helper: write the manuscript-facing alias file(s) for one registered asset.
materialize_manuscript_alias <- function(kind, canonical_stub, alias_stub,
                                         canonical_subdir, alias_subdir) {
  if (kind == "figure") {
    return(write_plot_alias(canonical_stub, alias_stub, canonical_subdir, alias_subdir))
```

```r
  }
  if (kind == "table") {
    return(write_table_alias(canonical_stub, alias_stub, canonical_subdir, alias_subdir))
  }
  stop("Unsupported artifact kind for manuscript alias: ", kind)
}

manuscript_asset_manifest_internal <- manuscript_asset_manifest_internal |>
  dplyr::rowwise() |>
  dplyr::mutate(
    alias_written = list(
      if (
        identical(status, "generated") &&
        !is.na(alias_stub) && nzchar(alias_stub) &&
        !is.na(canonical_stub) && nzchar(canonical_stub)
      ) {
        materialize_manuscript_alias(kind, canonical_stub, alias_stub, canonical_subdir, alias_subdir)
      } else {
        character(0)
      }
    )
  ) |>
  dplyr::ungroup() |>
  dplyr::mutate(
    canonical_artifact_path = vapply(canonical_artifact_file, format_manifest_path, character(1L)),
    alias_artifact_path = vapply(alias_artifact_file, format_manifest_path, character(1L))
  )

active_manuscript_assets <- manuscript_asset_manifest_internal |>
  dplyr::filter(status != "draft_only")

generated_manuscript_assets <- active_manuscript_assets |>
  dplyr::filter(status == "generated")

numbering_conflicts <- active_manuscript_assets |>
  dplyr::count(numbering_slot, name = "n_assets") |>
```

```r
  dplyr::filter(n_assets > 1L)

alias_conflicts <- generated_manuscript_assets |>
  dplyr::filter(!is.na(alias_stub), nzchar(alias_stub)) |>
  dplyr::count(alias_subdir, alias_stub, name = "n_assets") |>
  dplyr::filter(n_assets > 1L)

canonical_conflicts <- generated_manuscript_assets |>
  dplyr::filter(!is.na(canonical_stub), nzchar(canonical_stub)) |>
  dplyr::count(canonical_subdir, canonical_stub, name = "n_assets") |>
  dplyr::filter(n_assets > 1L)

deprecated_numbering_conflicts <- active_manuscript_assets |>
  dplyr::filter(numbering_slot == "Figure 3")

plot_registry_current <- read_plot_registry() |>
  dplyr::filter(run_id == diag_run_id)

plot_registry_conflicts <- plot_registry_current |>
  dplyr::count(plot_name, name = "n_rows") |>
  dplyr::filter(n_rows > 1L)

duplicate_asset_audit <- dplyr::bind_rows(
  tibble::tibble(
    audit_scope = "numbering_slot",
    audit_key = if (nrow(numbering_conflicts)) numbering_conflicts$numbering_slot else "all_active_slots",
    status = if (nrow(numbering_conflicts)) "conflict" else "pass",
    severity = if (nrow(numbering_conflicts)) "high" else "none",
    n_rows = if (nrow(numbering_conflicts)) numbering_conflicts$n_assets else 0L,
    detail = if (nrow(numbering_conflicts)) "Multiple active assets share one manuscript numbering slot." else "No
     ↪  contradictory active numbering slots detected."
  ),
  tibble::tibble(
    audit_scope = "alias_stub",
    audit_key = if (nrow(alias_conflicts)) paste0(alias_conflicts$alias_subdir, "::", alias_conflicts$alias_stub) else
     ↪  "all_aliases",
```

```
    status = if (nrow(alias_conflicts)) "conflict" else "pass",
    severity = if (nrow(alias_conflicts)) "high" else "none",
    n_rows = if (nrow(alias_conflicts)) alias_conflicts$n_assets else 0L,
    detail = if (nrow(alias_conflicts)) "Multiple active assets share one manuscript-facing alias." else "No
     ↪  conflicting manuscript alias stubs detected."
  ),
  tibble::tibble(
    audit_scope = "canonical_stub",
    audit_key = if (nrow(canonical_conflicts)) paste0(canonical_conflicts$canonical_subdir, "::",
     ↪  canonical_conflicts$canonical_stub) else "all_canonical_stubs",
    status = if (nrow(canonical_conflicts)) "conflict" else "pass",
    severity = if (nrow(canonical_conflicts)) "high" else "none",
    n_rows = if (nrow(canonical_conflicts)) canonical_conflicts$n_assets else 0L,
    detail = if (nrow(canonical_conflicts)) "Multiple active assets share one canonical artifact stub." else "No
     ↪  conflicting canonical artifact stubs detected."
  ),
  tibble::tibble(
    audit_scope = "deprecated_slot",
    audit_key = "Figure 3",
    status = if (nrow(deprecated_numbering_conflicts)) "conflict" else "pass",
    severity = if (nrow(deprecated_numbering_conflicts)) "high" else "none",
    n_rows = nrow(deprecated_numbering_conflicts),
    detail = if (nrow(deprecated_numbering_conflicts)) "Deprecated Figure 3 is still active in validation outputs."
     ↪  else "Deprecated Figure 3 numbering is suppressed."
  ),
  tibble::tibble(
    audit_scope = "plot_registry",
    audit_key = if (nrow(plot_registry_conflicts)) plot_registry_conflicts$plot_name else "plot_registry_run",
    status = if (nrow(plot_registry_conflicts)) "conflict" else "pass",
    severity = if (nrow(plot_registry_conflicts)) "high" else "none",
    n_rows = if (nrow(plot_registry_conflicts)) plot_registry_conflicts$n_rows else nrow(plot_registry_current),
    detail = if (nrow(plot_registry_conflicts)) "plot_registry.csv contains duplicate figure names for the current
     ↪  run." else "plot_registry.csv has unique figure names for the current run."
  )
)
```

```r
write_csv_safely(duplicate_asset_audit, results_path("duplicate_asset_audit.csv"), row_names = FALSE)

glyph_audit <- dplyr::bind_rows(
  active_manuscript_assets |>
    dplyr::transmute(
      manuscript_label = manuscript_label_safe,
      field_name = "caption",
      raw_text = paste0(manuscript_label, ". ", title),
      safe_text = caption_safe
    ),
  tibble::tibble(
    manuscript_label = "validation",
    field_name = "section_heading",
    raw_text = "Validation build summary",
    safe_text = safe_manuscript_text("Validation build summary")
  ),
  tibble::tibble(
    manuscript_label = "validation",
    field_name = "section_heading",
    raw_text = "Diagnostics audit summary",
    safe_text = safe_manuscript_text("Diagnostics audit summary")
  )
) |>
  dplyr::mutate(
    raw_is_ascii = is_ascii_text(raw_text),
    safe_is_ascii = is_ascii_text(safe_text),
    normalized = raw_text != safe_text,
    glyph_status = dplyr::if_else(safe_is_ascii, "pass", "fail"),
    severity = dplyr::if_else(safe_is_ascii, "none", "high")
  )

write_csv_safely(glyph_audit, results_path("glyph_audit.csv"), row_names = FALSE)

build_artifact_check_rows <- function(row) {
  row <- as.list(row)
  if (identical(row$status, "generated")) {
```

```r
  roles <- list(
    canonical = list(stub = row$canonical_stub, subdir = row$canonical_subdir),
    alias = list(stub = row$alias_stub, subdir = row$alias_subdir)
  )
  return(
    purrr::imap_dfr(roles, function(info, artifact_role) {
      if (is.null(info$stub) || is.na(info$stub) || !nzchar(info$stub)) {
        return(tibble::tibble())
      }
      purrr::map_dfr(row$expected_extension_list[[1]], function(ext) {
        artifact_path <- artifact_file_path(info$stub, ext, info$subdir)
        exists_flag <- file.exists(artifact_path)
        tibble::tibble(
          manuscript_label = row$manuscript_label_safe,
          numbering_slot = row$numbering_slot,
          kind = row$kind,
          section = row$section,
          artifact_role = artifact_role,
          ext = ext,
          artifact_path = format_manifest_path(artifact_path),
          exists = exists_flag,
          source_type = row$source_type,
          required_in_validation = row$required_in_validation,
          pdf_display = row$pdf_display,
          allowed_missing = FALSE,
          check_status = dplyr::if_else(exists_flag, "present", "missing_required"),
          severity = dplyr::if_else(exists_flag, "none", row$missing_artifact_severity),
          notes = row$notes
        )
      })
    })
  )
}
if (identical(row$status, "external/manual")) {
  return(
    tibble::tibble(
```

```
      manuscript_label = row$manuscript_label_safe,
      numbering_slot = row$numbering_slot,
      kind = row$kind,
      section = row$section,
      artifact_role = "external/manual",
      ext = "",
      artifact_path = "",
      exists = NA,
      source_type = row$source_type,
      required_in_validation = row$required_in_validation,
      pdf_display = row$pdf_display,
      allowed_missing = TRUE,
      check_status = "external_manual_declared",
      severity = row$missing_artifact_severity,
      notes = row$notes
    )
  )
}
tibble::tibble(
  manuscript_label = row$manuscript_label_safe,
  numbering_slot = row$numbering_slot,
  kind = row$kind,
  section = row$section,
  artifact_role = "suppressed",
  ext = "",
  artifact_path = "",
  exists = NA,
  source_type = row$source_type,
  required_in_validation = row$required_in_validation,
  pdf_display = row$pdf_display,
  allowed_missing = TRUE,
  check_status = "suppressed_draft_only",
  severity = "none",
  notes = row$notes
)
}
```

```r
artifact_check_status <- purrr::map_dfr(
  seq_len(nrow(manuscript_asset_manifest_internal)),
  ~ build_artifact_check_rows(manuscript_asset_manifest_internal[.x, , drop = FALSE])
)

artifact_check_missing <- artifact_check_status |>
  dplyr::filter(check_status %in% c("missing_required", "external_manual_declared"))

write_csv_safely(artifact_check_status, results_path("artifact_check_status.csv"), row_names = FALSE)
write_csv_safely(artifact_check_missing, results_path("artifact_check_missing.csv"), row_names = FALSE)

manuscript_asset_audit <- manuscript_asset_manifest_internal |>
  dplyr::mutate(
    existence_status = dplyr::case_when(
      status == "generated" &
        file.exists(canonical_artifact_file) &
        file.exists(alias_artifact_file) ~ "generated",
      status == "external/manual" ~ "external/manual",
      status == "draft_only" ~ "draft_only",
      status == "generated" ~ "missing",
      TRUE ~ status
    )
  ) |>
  dplyr::transmute(
    manuscript_label = manuscript_label_safe,
    title = title_safe,
    canonical_artifact_path,
    alias_artifact_path,
    existence_status
  )

write_csv_safely(manuscript_asset_audit, results_path("manuscript_asset_audit.csv"), row_names = FALSE)

artifact_provenance_manifest <- manuscript_asset_manifest_internal |>
  dplyr::transmute(
```

```
    manuscript_label = manuscript_label_safe,
    numbering_slot,
    title = title_safe,
    caption = caption_safe,
    kind,
    section,
    chunk_label,
    status,
    source_type,
    required_in_validation,
    expected_extensions,
    pdf_display,
    deprecated_replacement,
    missing_artifact_severity,
    canonical_stub,
    canonical_artifact_path,
    alias_stub,
    alias_artifact_path,
    analysis_family,
    notes
  )

canonical_asset_registry <- active_manuscript_assets |>
  dplyr::transmute(
    numbering_slot,
    manuscript_label = manuscript_label_safe,
    title = title_safe,
    kind,
    section,
    status,
    source_type,
    required_in_validation,
    pdf_display,
    canonical_stub,
    canonical_artifact_path,
    alias_stub,
```

```r
    alias_artifact_path,
    deprecated_replacement,
    analysis_family
  )

write_csv_safely(artifact_provenance_manifest, results_path("artifact_provenance_manifest.csv"), row_names = FALSE)
write_csv_safely(canonical_asset_registry, results_path("canonical_asset_registry.csv"), row_names = FALSE)

write_csv_safely(
  manuscript_asset_manifest_internal |>
    dplyr::transmute(
      manuscript_label = manuscript_label_safe,
      title = title_safe,
      kind,
      section,
      chunk_label,
      canonical_artifact_path,
      alias_artifact_path,
      status,
      source_type,
      required_in_validation,
      pdf_display,
      notes
    ),
  results_path("manuscript_asset_manifest.csv"),
  row_names = FALSE
)

supplement_manifest_internal <- manuscript_asset_manifest_internal |>
  dplyr::filter(section == "supplement") |>
  dplyr::transmute(
    supp_label = manuscript_label_safe,
    title = title_safe,
    analysis_family = analysis_family,
    artifact_path = alias_artifact_path,
    chunk_name = chunk_label,
```

```r
    notes = notes,
    artifact_file = alias_artifact_file
  )

write_csv_safely(
  supplement_manifest_internal |>
    dplyr::select(-artifact_file),
  results_path("supplement_manifest.csv"),
  row_names = FALSE
)

manuscript_sync_lines <- c(
  "# Manuscript sync report",
  "",
  paste0("- Run ID: ", diag_run_id),
  paste0("- Active validation assets: ", nrow(active_manuscript_assets)),
  paste0("- Generated validation assets: ", nrow(generated_manuscript_assets)),
  paste0("- External/manual assets: ", sum(active_manuscript_assets$status == "external/manual")),
  paste0("- Draft-only assets suppressed: ", sum(manuscript_asset_manifest_internal$status == "draft_only")),
  paste0("- Missing required generated artifacts: ", sum(artifact_check_status$check_status == "missing_required",
   ↪  na.rm = TRUE)),
  paste0("- Allowed external/manual declarations: ", sum(artifact_check_status$check_status ==
   ↪  "external_manual_declared", na.rm = TRUE)),
  paste0("- Glyph audit failures: ", sum(glyph_audit$glyph_status == "fail", na.rm = TRUE)),
  paste0("- Duplicate/numbering conflicts: ", sum(duplicate_asset_audit$status == "conflict", na.rm = TRUE)),
  "",
  "## Active numbering map",
  ""
)

active_numbering_lines <- active_manuscript_assets |>
  dplyr::transmute(line = paste0("- ", numbering_slot, ": ", title_safe, " [", kind, "; ", status, "]")) |>
  dplyr::pull(line)

write_markdown_report(
  c(manuscript_sync_lines, active_numbering_lines),
```

```
  results_path("manuscript_sync_report.md")
)

validation_static_summary <- tibble::tibble(
  component = c("artifact_registry", "external_manual_assets", "glyph_audit", "duplicate_audit"),
  status = c(
    if (any(artifact_check_status$check_status == "missing_required")) "FAIL" else "PASS",
    if (any(artifact_check_status$check_status == "external_manual_declared")) "PASS_WITH_WARNINGS" else "PASS",
    if (any(glyph_audit$glyph_status == "fail")) "FAIL" else "PASS",
    if (any(duplicate_asset_audit$status == "conflict")) "FAIL" else "PASS"
  ),
  detail = c(
    paste0(sum(artifact_check_status$check_status == "missing_required"), " missing required generated artifact
     ↪  rows."),
    paste0(sum(artifact_check_status$check_status == "external_manual_declared"), " declared external/manual rows."),
    paste0(sum(glyph_audit$glyph_status == "fail"), " unsafe manuscript glyph rows."),
    paste0(sum(duplicate_asset_audit$status == "conflict"), " conflicting duplicate/numbering rows.")
  )
)

validation_static_failures <- c(
  if (nrow(numbering_conflicts)) paste0("Conflicting active numbering slots: ",
   ↪  collapse_unique_text(numbering_conflicts$numbering_slot)),
  if (nrow(alias_conflicts)) paste0("Conflicting manuscript alias stubs: ",
   ↪  collapse_unique_text(alias_conflicts$alias_stub)),
  if (nrow(canonical_conflicts)) paste0("Conflicting canonical artifact stubs: ",
   ↪  collapse_unique_text(canonical_conflicts$canonical_stub)),
  if (nrow(deprecated_numbering_conflicts)) "Deprecated Figure 3 remained active in the validation registry.",
  if (any(glyph_audit$glyph_status == "fail")) "Unsafe manuscript glyphs remained after normalization.",
  if (any(artifact_check_status$check_status == "missing_required")) paste0(
    "Missing required generated artifacts: ",
    collapse_unique_text(artifact_check_status$artifact_path[artifact_check_status$check_status ==
     ↪  "missing_required"])
  ),
  if (any(duplicate_asset_audit$status == "conflict")) paste0(
    "Duplicate audit conflicts: ",
```

```
    collapse_unique_text(duplicate_asset_audit$audit_key[duplicate_asset_audit$status == "conflict"])
  )
)

if (length(validation_static_failures)) {
  stop(
    "Validation registry checks failed: ",
    paste(validation_static_failures, collapse = " | ")
  )
}
```

```
# Purpose: define the canonical manuscript-facing display helpers used later in the validation section.
manifest_row <- function(label) {
  row <- manuscript_asset_manifest_internal |>
    dplyr::filter(manuscript_label_safe == safe_manuscript_text(label))
  if (nrow(row) != 1L) stop("Expected exactly one manifest row for ", label)
  row
}

# helper: resolve the display-ready figure path for one manifest label.
# LaTeX/PDF renders must fail fast if the manuscript PDF artifact is missing.
manifest_figure_path <- function(label) {
  row <- manifest_row(label)
  ext <- if (knitr::is_latex_output()) "pdf" else "png"
  stub <- if (!is.na(row$alias_stub) && nzchar(row$alias_stub)) row$alias_stub else row$canonical_stub
  subdir <- if (!is.na(row$alias_subdir) && nzchar(row$alias_subdir)) row$alias_subdir else row$canonical_subdir
  path <- artifact_file_path(stub, ext, subdir)
  if (knitr::is_latex_output() && !file.exists(path)) {
    stop("Missing PDF display artifact for ", label, ": ", path)
  }
  if (!file.exists(path)) stop("Missing display artifact for ", label, ": ", path)
  path
}

manuscript_figure_notes <- list(
  "Figure 2" = key_results_caption_main,
  "Figure S4" = key_results_cat3_caption_main,
```

```
  "Figure S5" = key_results_caption_adjusted
)

# helper: render explanatory manuscript notes as Quarto text below figures
# instead of embedding long text inside the plot canvas.
render_manuscript_note <- function(label) {
  note <- manuscript_figure_notes[[label]]
  if (is.null(note) || !nzchar(note)) return("")
  note <- paste0("Note: ", note)
  if (knitr::is_latex_output()) {
    return(paste0(
      "\\par\\vspace{0.5em}\\noindent\\footnotesize ",
      latex_safe_table_text(escape_latex_text(note)),
      "\\normalsize"
    ))
  }
  paste0("*", note, "*")
}

# helper: insert one manifest-registered figure into the rendered notebook in
# official manuscript order, independent of generation chunk order.
render_manuscript_figure <- function(label, width = "0.88\\linewidth") {
  row <- manifest_row(label)
  fig_path <- manifest_figure_path(label)
  caption_text <- row$caption_safe[[1]]
  if (knitr::is_latex_output()) {
    latex_caption <- escape_latex_text(caption_text)
    latex_path <- display_path(fig_path)
    out <- c(
      "\\clearpage",
      "\\begin{center}",
      "\\begin{minipage}{\\linewidth}",
      "\\centering",
      sprintf("\\small %s\\par", latex_caption),
      "\\vspace{0.5em}",
      sprintf("\\includegraphics[width=%s,height=0.82\\textheight,keepaspectratio]{%s}", width, latex_path),
```

```
      render_manuscript_note(label),
      "\\end{minipage}",
      "\\end{center}"
    )
    knitr::asis_output(paste(out, collapse = "\n"))
  } else {
    knitr::asis_output(
      paste0("![](", display_path(fig_path), ")\n\n*", caption_text, "*\n\n", render_manuscript_note(label), "\n")
    )
  }
}

# helper: render one manifest-registered table with its manuscript label and
# title while reusing the standard standalone-table path.
render_manifest_table <- function(label, df, file_stub, wide = FALSE, digits = 2,
                                  max_cols = 6, landscape = NULL) {
  row <- manifest_row(label)
  render_table_pdf(
    df,
    caption = row$title_safe[[1]],
    file_stub = file_stub,
    wide = wide,
    digits = digits,
    max_cols = max_cols,
    landscape = landscape,
    display_label = label,
    caption_mode = "plain",
    page_break_before = TRUE
  )
}

render_manifest_panel_table <- function(label, panels, file_stub, digits = 2,
                                        landscape = TRUE, csv_df = NULL) {
  row <- manifest_row(label)
  render_panel_table_pdf(
    panels,
```

```
    caption = row$title_safe[[1]],
    file_stub = file_stub,
    digits = digits,
    display_label = label,
    landscape = landscape,
    page_break_before = TRUE,
    csv_df = csv_df
  )
}

render_validation_manuscript_assets <- function() {
  as_asis_text <- function(x) {
    if (is.null(x)) return("")
    paste(as.character(x), collapse = "\n")
  }

  blocks <- list(
    render_manuscript_figure("Figure 1"),
    render_manuscript_figure("Figure 2"),
    render_manifest_table(
      "Table 1",
      tbl1_combined_csv,
      file_stub = "table1_combined",
      wide = TRUE,
      digits = 2,
      max_cols = 8,
      landscape = TRUE
    ),
    render_manifest_panel_table(
      "Table 2",
      list(
        "Odds ratios" = table_2_weighted_or_panel,
        "Predicted probabilities" = table_2_weighted_probability_panel
      ),
      file_stub = "table_2_weighted_categorical_outcomes",
      digits = 2,
```

```
    landscape = TRUE,
    csv_df = table_2_weighted_categorical_outcomes
  ),
  render_manifest_table(
    "Table S1",
    table_s1_inclusion_criteria,
    file_stub = "table_s1_inclusion_criteria",
    wide = TRUE,
    digits = 0,
    max_cols = 6,
    landscape = TRUE
  ),
  render_manuscript_figure("Figure S1"),
  render_manuscript_figure("Figure S2"),
  render_manuscript_figure("Figure S3"),
  render_manuscript_figure("Figure S4"),
  render_manuscript_figure("Figure S5"),
  render_manifest_table(
    "Table S2",
    table_s2_crude_threelevel,
    file_stub = "table_s2_crude_threelevel",
    wide = TRUE,
    digits = 2
  ),
  render_manifest_table(
    "Table S3",
    table_s3_gbm_threelevel,
    file_stub = "table_s3_gbm_threelevel",
    wide = TRUE,
    digits = 2
  ),
  render_manuscript_figure("Figure S6"),
  render_manuscript_figure("Figure S7"),
  render_manuscript_figure("Figure S8"),
  render_manifest_table(
    "Table S4",
```

```r
      table_s4_missingness_primary_analysis,
      file_stub = "table_s4_missingness_primary_analysis",
      wide = FALSE,
      digits = 1,
      max_cols = 4,
      landscape = FALSE
    ),
    render_manifest_table(
      "Table S5",
      table_s5_mi_diagnostic_summary,
      file_stub = "table_s5_mi_diagnostic_summary",
      wide = TRUE,
      digits = 3,
      max_cols = 4,
      landscape = TRUE
    )
  )

  block_text <- vapply(blocks, as_asis_text, character(1L))
  block_text <- block_text[nzchar(trimws(block_text))]
  knitr::asis_output(paste(block_text, collapse = "\n\n"))
}
```

```r
# Purpose: keep the compact manuscript-facing asset audit available for the final validation section.
invisible(manuscript_asset_audit)
```

```r
# Purpose: verify manuscript-facing artifacts are present and numbering stays harmonized.
required_manifest_files <- c(
  results_path("manuscript_asset_manifest.csv"),
  results_path("supplement_manifest.csv"),
  results_path("artifact_provenance_manifest.csv"),
  results_path("artifact_check_status.csv"),
  results_path("artifact_check_missing.csv"),
  results_path("canonical_asset_registry.csv"),
  results_path("manuscript_sync_report.md"),
  results_path("glyph_audit.csv"),
  results_path("duplicate_asset_audit.csv")
```

```r
)

generated_manifest_files <- manuscript_asset_manifest_internal |>
  dplyr::filter(status == "generated") |>
  (\(df) c(df$canonical_artifact_file, df$alias_artifact_file))() |>
  (\(paths) paths[!is.na(paths) & nzchar(paths)])() |>
  unique()

missing_manifest_files <- c(
  required_manifest_files[!file.exists(required_manifest_files)],
  generated_manifest_files[!file.exists(generated_manifest_files)]
) |>
  unique()

supplement_manifest_expected <- manuscript_asset_manifest_internal |>
  dplyr::filter(section == "supplement") |>
  dplyr::transmute(
    supp_label = manuscript_label_safe,
    title = title_safe,
    analysis_family = analysis_family,
    artifact_path = alias_artifact_path,
    chunk_name = chunk_label,
    notes = notes
  )

supplement_manifest_actual <- utils::read.csv(
  results_path("supplement_manifest.csv"),
  stringsAsFactors = FALSE,
  na.strings = character()
)
supplement_manifest_compare_cols <- c(
  "supp_label", "title", "artifact_path", "chunk_name", "notes"
)
# helper: coerce manifest tables into a stable all-character form before
# comparison so NA handling and tibble/data.frame differences do not create
# false assertion failures.
```

```r
normalize_manifest_strings <- function(df) {
  tibble::as_tibble(df) |>
    dplyr::mutate(
      dplyr::across(
        dplyr::everything(),
        ~ dplyr::if_else(is.na(.x), "", as.character(.x))
      )
    )
}
supplement_manifest_analysis_family_ok <- "analysis_family" %in% names(supplement_manifest_actual) &&
  all(nzchar(as.character(supplement_manifest_actual$analysis_family)))
supplement_manifest_matches <- supplement_manifest_analysis_family_ok && all(
  supplement_manifest_compare_cols %in% names(supplement_manifest_actual)
) && identical(
  normalize_manifest_strings(dplyr::select(supplement_manifest_expected,
   ↪  dplyr::all_of(supplement_manifest_compare_cols))),
  normalize_manifest_strings(supplement_manifest_actual) |>
    dplyr::select(dplyr::all_of(supplement_manifest_compare_cols))
)

figure_3_aliases <- list.files(
  results_path("figs"),
  pattern = "^figure_3([._]|$)",
  full.names = TRUE
)

has_generated_s1 <- any(
  manuscript_asset_manifest_internal$manuscript_label_safe == "Table S1" &
    manuscript_asset_manifest_internal$status == "generated" &
    manuscript_asset_manifest_internal$pdf_display
)

forbidden_duplicate_files <- c(results_path("figs", "propensity-histograms-conditional-1.png"))
existing_forbidden_duplicates <- forbidden_duplicate_files[file.exists(forbidden_duplicate_files)]

if (
```

```r
  length(missing_manifest_files) ||
  !supplement_manifest_matches ||
  length(figure_3_aliases) ||
  !has_generated_s1 ||
  length(existing_forbidden_duplicates)
) {
  missing_pretty <- vapply(missing_manifest_files, pretty_path, character(1L))
  duplicate_pretty <- vapply(existing_forbidden_duplicates, pretty_path, character(1L))
  figure3_pretty <- vapply(figure_3_aliases, pretty_path, character(1L))
  stop(
    "Manuscript asset check failed: missing files [",
    paste(missing_pretty, collapse = ", "),
    "]; supplement manifest matches [", supplement_manifest_matches,
    "]; Figure 3 aliases [", paste(figure3_pretty, collapse = ", "),
    "]; Table S1 generated [", has_generated_s1,
    "]; forbidden duplicates [", paste(duplicate_pretty, collapse = ", "),
    "]."
  )
}
```

4 Diagnostics and Exports

```r
# Purpose: diagnostics inputs.
stopifnot(gbm_params$stop.method == "smd.max")
stopifnot(exists("M_IMP"), exists("MAXIT_MI"), exists("MI_SEED"))
stopifnot(exists("imp"))
stopifnot(exists("mi_logistic_ps_abg_list"), exists("mi_logistic_ps_vbg_list"))
meth <- imp$method

diag_fig_dir <- results_path("figs")
fs::dir_create(diag_fig_dir)
```

```r
# Purpose: diagnostics missingness.

extra_miss_vars <- intersect(c("bnp", "spo2"), names(subset_data_raw))
```

```r
miss_vars <- intersect(
  c(covars_ps, extra_miss_vars, "paco2", "vbg_co2", "vbg_o2sat",
    "imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"),
  names(subset_data_raw)
)

miss_overall <- subset_data_raw |>
  summarise(across(all_of(miss_vars), ~ mean(is.na(.)) * 100)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing_overall") |>
  arrange(desc(pct_missing_overall))

miss_overall_file <- results_path("diagnostics_missingness.csv")
write_csv_safely(miss_overall, miss_overall_file, row_names = FALSE)

miss_by_strata <- bind_rows(
  subset_data_raw |>
    group_by(has_abg) |>
    summarise(across(all_of(miss_vars), ~ mean(is.na(.)) * 100), .groups = "drop") |>
    pivot_longer(-has_abg, names_to = "variable", values_to = "pct_missing") |>
    mutate(stratum = "has_abg", level = as.character(has_abg)) |>
    select(stratum, level, variable, pct_missing),
  subset_data_raw |>
    group_by(has_vbg) |>
    summarise(across(all_of(miss_vars), ~ mean(is.na(.)) * 100), .groups = "drop") |>
    pivot_longer(-has_vbg, names_to = "variable", values_to = "pct_missing") |>
    mutate(stratum = "has_vbg", level = as.character(has_vbg)) |>
    select(stratum, level, variable, pct_missing)
)

miss_by_strata_file <- results_path("diagnostics_missingness-by-strata.csv")
write_csv_safely(miss_by_strata, miss_by_strata_file, row_names = FALSE)

md_fig_file <- results_path("figs", "diag-mi-missingness-pattern.png")
grDevices::png(md_fig_file, width = 1800, height = 1200, res = 200)
mice::md.pattern(imp$data, plot = TRUE)
grDevices::dev.off()
```

```r
if (nrow(miss_overall) > 0) {
  top_vars <- miss_overall |>
    slice_head(n = min(20, nrow(miss_overall))) |>
    pull(variable)
  miss_sample <- subset_data_raw |>
    select(all_of(top_vars)) |>
    slice_sample(n = min(1000, nrow(subset_data_raw)))

  p_miss <- naniar::vis_miss(miss_sample, sort_miss = TRUE) +
    theme_pub(base_size = 10) +
    labs(title = "Missingness heatmap (top 20 variables)")
  save_diag_plot(p_miss, results_path("figs", "diag-mi-missingness-heatmap.png"),
                 width = 10, height = 6)
}
```

4.0.1 MI convergence and mixing

```r
# Purpose: diagnostics mi convergence.
stopifnot(exists("imp"))

log_events_summary_file <- results_path("mice_logged_events_summary.csv")
stopifnot(file.exists(log_events_summary_file))
log_events_summary <- tryCatch(
  utils::read.csv(log_events_summary_file, stringsAsFactors = FALSE),
  error = function(e) data.frame()
)
if (nrow(log_events_summary) && !("empty" %in% names(log_events_summary) && all(log_events_summary$empty))) {
  render_table_pdf_maybe(
    log_events_summary,
    caption = "Logged events (MI)",
    file_stub = "mice_logged_events_summary",
    digits = 0,
    show = FALSE
  )
```

```r
  stopifnot(all(c("variable", "n") %in% names(log_events_summary)))
  total_events <- sum(log_events_summary$n, na.rm = TRUE)
  if (is.finite(total_events) && total_events > 1000) {
    warning("MI loggedEvents count is high (", total_events, "). Review mice_logged_events_summary.csv.", call. =
     ↪  FALSE)
  }
  by_var <- log_events_summary |>
    dplyr::group_by(variable) |>
    dplyr::summarise(n = sum(n), .groups = "drop") |>
    dplyr::arrange(dplyr::desc(n))
  if (nrow(by_var) && is.finite(total_events) && total_events > 0) {
    top_share <- by_var$n[1] / total_events
    if (top_share > 0.5) {
      warning("MI loggedEvents dominated by variable '", by_var$variable[1],
              "' (", round(100 * top_share, 1), "% of events).", call. = FALSE)
    }
  }
} else {
  message("No logged events to summarize.")
}

stopifnot(exists("chain_diag"))
stopifnot(nrow(chain_diag) > 0)
n_imputed <- chain_diag_stats$n_imputed_vars
n_with_tail <- chain_diag_stats$n_with_drift_tail
message("Chain diagnostics: drift_tail available for ", n_with_tail, " / ", n_imputed, " imputed variables.")
stopifnot("drift_tail_scaled" %in% names(chain_diag))
frac_flagged <- mean(chain_diag$flag %in% TRUE, na.rm = TRUE)
med_scaled <- stats::median(chain_diag$drift_tail_scaled, na.rm = TRUE)
max_scaled <- max(chain_diag$drift_tail_scaled, na.rm = TRUE)
message("Chain diagnostics (scaled): flagged=", round(frac_flagged, 3),
        "; median drift_tail_scaled=", signif(med_scaled, 3),
        "; max drift_tail_scaled=", signif(max_scaled, 3))

top_flagged <- chain_diag |>
  dplyr::filter(flag %in% TRUE) |>
```

```r
  dplyr::arrange(dplyr::desc(abs(drift_tail_scaled)))
if (nrow(top_flagged)) {
  render_table_pdf_maybe(
    top_flagged |>
      dplyr::select(variable, method, drift_tail, drift_tail_scaled, slope, slope_scaled,
                    tail_n_finite, tail_window_na_frac, flag_reason),
    caption = "MICE chain diagnostics (flagged drift)",
    file_stub = "mice_chain_diagnostics_flagged",
    digits = 3,
    show = FALSE
  )
} else {
  message("No flagged drift in chain diagnostics.")
}

top_tail_na <- chain_diag |>
  dplyr::arrange(dplyr::desc(tail_window_na_frac))
if (nrow(top_tail_na)) {
  render_table_pdf_maybe(
    top_tail_na,
    caption = "MICE chain diagnostics (tail NA fraction)",
    file_stub = "mice_chain_diagnostics_tail_na",
    digits = 3,
    show = FALSE
  )
}

trace_vars <- intersect(c("curr_bmi", "serum_hco3", "hr", "sodium", "serum_cr"),
                        names(imp$data))
imp_trace <- imp
if (imp$m > 10) {
  set.seed(MI_SEED)
  idx <- sort(sample(seq_len(imp$m), 10))
  imp_trace$imp <- lapply(imp$imp, function(x) x[, idx, drop = FALSE])
  imp_trace$m <- length(idx)
}
```

```r
if (length(trace_vars)) {
  tr_file <- results_path("figs", "diag-mi-trace-selected.png")
  grDevices::png(tr_file, width = 1800, height = 1200, res = 200)
  plot(imp_trace, trace_vars)
  grDevices::dev.off()
}
```

```
png
  2
```

```r
message("Using memory-safe observed vs imputed plots from mi-diagnostics (no mids densityplot/stripplot).")
```

4.0.2 MI stability across m

```r
# Purpose: diagnostics mi stability.
stopifnot(exists("imp"))
stopifnot(exists("mi_logistic_ps_abg_list"), exists("mi_logistic_ps_vbg_list"))

# helper: trim a `mids` object down to its first `m_keep` imputations for
# stability diagnostics across different values of m.
subset_mids <- function(imp_obj, m_keep) {
  if (imp_obj$m == m_keep) return(imp_obj)
  idx <- seq_len(m_keep)
  imp_new <- imp_obj
  imp_new$imp <- lapply(imp_obj$imp, function(x) x[, idx, drop = FALSE])
  imp_new$m <- m_keep
  imp_new
}

m_vals <- sort(unique(c(20, 50, M_IMP)))
m_vals <- m_vals[m_vals <= imp$m]

med_abg <- median(subset_data$paco2[subset_data$has_abg == 1 & !is.na(subset_data$paco2)], na.rm = TRUE)
med_vbg <- median(subset_data$vbg_co2[subset_data$has_vbg == 1 & !is.na(subset_data$vbg_co2)], na.rm = TRUE)
if (!is.finite(med_abg)) med_abg <- ABG_CO2_REF
```

```r
if (!is.finite(med_vbg)) med_vbg <- VBG_CO2_REF
grid_abg_info_m <- make_co2_grid_ref("paco2", c(med_abg, ABG_CO2_REF), x_ref_abg, ABG_CO2_REF)
grid_vbg_info_m <- make_co2_grid_ref("vbg_co2", c(med_vbg, VBG_CO2_REF), x_ref_vbg, VBG_CO2_REF)
grid_abg_m <- grid_abg_info_m$grid
grid_vbg_m <- grid_vbg_info_m$grid
ref_idx_abg_m <- grid_abg_info_m$ref_idx
ref_idx_vbg_m <- grid_vbg_info_m$ref_idx
med_idx_abg <- match(med_abg, grid_abg_m$paco2)
med_idx_vbg <- match(med_vbg, grid_vbg_m$vbg_co2)
if (is.na(med_idx_abg)) med_idx_abg <- which.min(abs(grid_abg_m$paco2 - med_abg))
if (is.na(med_idx_vbg)) med_idx_vbg <- which.min(abs(grid_vbg_m$vbg_co2 - med_vbg))

stab_rows <- lapply(m_vals, function(mv) {
  imp_m <- subset_mids(imp, mv)
  get_imp_m <- function(i) normalize_types(mice::complete(imp_m, action = i), levels_ref)

  fits_abg <- lapply(seq_len(mv), function(i) {
    d <- get_imp_m(i)
    d <- d[, c("imv_proc", "has_abg", "paco2", adj_core), drop = FALSE]
    tryCatch(
      fit_spline_imp(
        d, get_mi_weight("ABG", i), "imv_proc", "paco2", "has_abg",
        adj_vars = adj_core,
        spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS, grid_df = grid_abg_m,
        ref_idx = ref_idx_abg_m,
        imp_index = i
      ),
      error = function(e) list(error = conditionMessage(e))
    )
  })
  collect_warnings_from_list(fits_abg)
  curve_abg <- pool_spline_curve(fits_abg, grid_abg_m, ref_idx_abg_m, ABG_CO2_REF,
                                 min_ok_frac = 0.9)

  fits_vbg <- lapply(seq_len(mv), function(i) {
    d <- get_imp_m(i)
```

```r
    d <- d[, c("imv_proc", "has_vbg", "vbg_co2", adj_core), drop = FALSE]
    tryCatch(
      fit_spline_imp(
        d, get_mi_weight("VBG", i), "imv_proc", "vbg_co2", "has_vbg",
        adj_vars = adj_core,
        spline_df = SPLINE_DF, spline_basis = SPLINE_BASIS, grid_df = grid_vbg_m,
        ref_idx = ref_idx_vbg_m,
        imp_index = i
      ),
      error = function(e) list(error = conditionMessage(e))
    )
  })
  collect_warnings_from_list(fits_vbg)
  curve_vbg <- pool_spline_curve(fits_vbg, grid_vbg_m, ref_idx_vbg_m, VBG_CO2_REF,
                                 min_ok_frac = 0.9)

  bind_rows(
    tibble::tibble(group = "ABG", m = mv, OR = curve_abg$OR[med_idx_abg]),
    tibble::tibble(group = "VBG", m = mv, OR = curve_vbg$OR[med_idx_vbg])
  )
})

stab_df <- bind_rows(stab_rows)
ref <- stab_df |> filter(m == max(m_vals)) |> select(group, OR) |> rename(OR_ref = OR)
stab_df <- stab_df |>
  left_join(ref, by = "group") |>
  mutate(abs_diff = OR - OR_ref,
         pct_diff = 100 * (OR - OR_ref) / OR_ref)

stab_file <- results_path("mi_m_stability.csv")
write_csv_safely(stab_df, stab_file, row_names = FALSE)

# FMI / relative efficiency for a representative unweighted model
append_mem_snapshot("diagnostics", "mi_fmi_glm", "pre")
# This diagnostic previously used with(imp, glm(...)) which can materialize large
# completed-data objects. We pool a single coefficient via pool.scalar() instead.
```

```r
fmi_formula <- imv_proc ~ has_abg + age_at_encounter + sex
fmi_vars <- all.vars(fmi_formula)
qhat <- rep(NA_real_, imp$m)
uhat <- rep(NA_real_, imp$m)
for (i in seq_len(imp$m)) {
  d_i <- mice::complete(imp, action = i)
  d_i <- d_i[, intersect(fmi_vars, names(d_i)), drop = FALSE]
  if (!all(c("imv_proc", "has_abg", "age_at_encounter", "sex") %in% names(d_i))) {
    next
  }
  cap_fit <- capture_warnings(
    glm(
      formula = fmi_formula,
      data = d_i,
      family = binomial(),
      model = FALSE,
      x = FALSE,
      y = FALSE
    ),
    context = make_context(
      stage = "diagnostics",
      component = "mi_fmi_glm",
      analysis_variant = "mi",
      model_type = "glm",
      group = NA_character_,
      outcome = "imv_proc",
      imputation = i,
      batch = NA_integer_
    )
  )
  append_warnings(cap_fit$warnings)
  fit_i <- cap_fit$value
  if (!is.null(fit_i)) {
    coef_i <- stats::coef(fit_i)
    vcov_i <- tryCatch(stats::vcov(fit_i), error = function(e) NULL)
    if (
```

```r
      "has_abg" %in% names(coef_i) &&
      !is.null(vcov_i) &&
      "has_abg" %in% rownames(vcov_i) &&
      "has_abg" %in% colnames(vcov_i)
    ) {
      qhat[i] <- unname(coef_i[["has_abg"]])
      uhat[i] <- unname(vcov_i["has_abg", "has_abg"])
    }
  }
  rm(list = intersect(c("d_i", "cap_fit", "fit_i", "coef_i", "vcov_i"), ls()))
  invisible(gc(FALSE))
}
ok <- is.finite(qhat) & is.finite(uhat) & uhat > 0
if (sum(ok) >= 2) {
  ps <- mice::pool.scalar(
    Q = qhat[ok],
    U = uhat[ok],
    n = sum(subset_data$has_abg == 1, na.rm = TRUE),
    k = 1
  )
  fmi_abg <- tibble::tibble(
    term = "has_abg",
    estimate = ps$qbar,
    std.error = sqrt(ps$t),
    fmi = ps$fmi,
    m_used = sum(ok)
  )
} else {
  warning("FMI not available from pooled scalar diagnostic; leaving FMI as NA.", call. = FALSE)
  fmi_abg <- tibble::tibble(
    term = "has_abg",
    estimate = NA_real_,
    std.error = NA_real_,
    fmi = NA_real_,
    m_used = sum(ok)
  )
```

```r
}
fmi_abg$rel_eff <- if (is.finite(fmi_abg$fmi[1])) 1 / (1 + fmi_abg$fmi[1] / M_IMP) else NA_real_
append_mem_snapshot("diagnostics", "mi_fmi_glm", "post")
```

4.0.3 MI maxit sensitivity (sampled)

```r
# Purpose: diagnostics mi maxit.
run_maxit_sensitivity <- TRUE
if (run_maxit_sensitivity) {
  stopifnot(exists("mi_df"), exists("meth"), exists("pred"))
  set.seed(MI_SEED)
  idx <- sample(seq_len(nrow(mi_df)), min(2000, nrow(mi_df)))
  mi_df_sens <- mi_df[idx, , drop = FALSE]

  m_sens <- min(20, M_IMP)
  maxit_short <- max(5, floor(MAXIT_MI / 2))

  cap_short <- capture_warnings(
    mice::mice(
      data            = mi_df_sens,
      m               = m_sens,
      maxit           = maxit_short,
      predictorMatrix = pred,
      method          = meth,
      printFlag       = FALSE,
      seed            = MI_SEED
    ),
    context = make_context(
      stage = "MI",
      component = "mice_maxit_short",
      analysis_variant = "mi",
      model_type = "mice",
      group = NA_character_,
      outcome = NA_character_,
      imputation = NA_integer_,
      batch = NA_integer_
```

```r
  )
)
append_warnings(cap_short$warnings)
imp_short <- cap_short$value
le_short <- as.data.frame(imp_short$loggedEvents)
if (nrow(le_short)) {
  le_short$run_type <- "maxit_short"
}
write_csv_safely(le_short, results_path("mice_logged_events_maxit_short.csv"), row_names = FALSE)

cap_long <- capture_warnings(
  mice::mice(
    data            = mi_df_sens,
    m               = m_sens,
    maxit           = MAXIT_MI,
    predictorMatrix = pred,
    method          = meth,
    printFlag       = FALSE,
    seed            = MI_SEED + 1
  ),
  context = make_context(
    stage = "MI",
    component = "mice_maxit_long",
    analysis_variant = "mi",
    model_type = "mice",
    group = NA_character_,
    outcome = NA_character_,
    imputation = NA_integer_,
    batch = NA_integer_
  )
)
append_warnings(cap_long$warnings)
imp_long <- cap_long$value
le_long <- as.data.frame(imp_long$loggedEvents)
if (nrow(le_long)) {
  le_long$run_type <- "maxit_long"
```

```r
  }
  write_csv_safely(le_long, results_path("mice_logged_events_maxit_long.csv"), row_names = FALSE)

  mean_across_imps <- function(imp_obj, var) {
    means <- vapply(seq_len(imp_obj$m), function(i) {
      d <- mice::complete(imp_obj, action = i)
      mean(d[[var]], na.rm = TRUE)
    }, numeric(1))
    mean(means, na.rm = TRUE)
  }

  key_vars <- intersect(c("curr_bmi", "serum_hco3", "sodium", "serum_cr"), names(mi_df_sens))
  sens_df <- lapply(key_vars, function(v) {
    m_short <- mean_across_imps(imp_short, v)
    m_long  <- mean_across_imps(imp_long, v)
    data.frame(
      variable = v,
      mean_short = m_short,
      mean_long  = m_long,
      diff       = m_long - m_short,
      stringsAsFactors = FALSE
    )
  }) |> bind_rows()

  sens_file <- results_path("mi_maxit_sensitivity.csv")
  write_csv_safely(sens_df, sens_file, row_names = FALSE)
}
```

```r
# Purpose: diagnostics weights.
stopifnot(all(c("w_abg", "w_vbg") %in% names(subset_data)))

wt_abg <- subset_data$w_abg[subset_data$has_abg == 1]
wt_vbg <- subset_data$w_vbg[subset_data$has_vbg == 1]
ps_abg <- subset_data$ps_abg[subset_data$has_abg == 1]
ps_vbg <- subset_data$ps_vbg[subset_data$has_vbg == 1]
trunc_abg <- subset_data$trunc_abg[subset_data$has_abg == 1]
trunc_vbg <- subset_data$trunc_vbg[subset_data$has_vbg == 1]
```

```r
wt_sum <- bind_rows(
  weight_summary(wt_abg, ps = ps_abg, ps_floor = ps_floor_abg,
                 truncated = trunc_abg) |>
    mutate(group = "ABG"),
  weight_summary(wt_vbg, ps = ps_vbg, ps_floor = ps_floor_vbg,
                 truncated = trunc_vbg) |>
    mutate(group = "VBG")
)
wt_sum_file <- results_path("weight_summary.csv")
write_csv_safely(wt_sum, wt_sum_file, row_names = FALSE)

# helper: summarize propensity-score overlap numerically for one tested cohort.
ps_stat <- function(ps, group) {
  ps_ok <- ps[is.finite(ps)]
  if (!length(ps_ok)) {
    return(data.frame(
      group = group,
      ps_min = NA_real_,
      ps_p01 = NA_real_,
      ps_p50 = NA_real_,
      ps_p99 = NA_real_,
      ps_max = NA_real_,
      stringsAsFactors = FALSE
    ))
  }
  data.frame(
    group = group,
    ps_min = min(ps_ok, na.rm = TRUE),
    ps_p01 = stats::quantile(ps_ok, 0.01, na.rm = TRUE),
    ps_p50 = stats::median(ps_ok, na.rm = TRUE),
    ps_p99 = stats::quantile(ps_ok, 0.99, na.rm = TRUE),
    ps_max = max(ps_ok, na.rm = TRUE),
    stringsAsFactors = FALSE
  )
}
```

```r
ps_summary <- dplyr::bind_rows(
  ps_stat(ps_abg, "ABG"),
  ps_stat(ps_vbg, "VBG")
)
write_csv_safely(ps_summary, results_path("ps_overlap_summary.csv"), row_names = FALSE)
render_table_pdf_maybe(
  ps_summary,
  caption = "GBM IPSW propensity-score overlap summary (tested cohort)",
  file_stub = "ps_overlap_summary",
  digits = 3,
  show = FALSE
)

wt_df <- bind_rows(
  tibble::tibble(group = "ABG", ps = ps_abg, weight = wt_abg),
  tibble::tibble(group = "VBG", ps = ps_vbg, weight = wt_vbg)
) |>
  filter(is.finite(weight), is.finite(ps))

# helper: draw a simple histogram of stabilized weights for one cohort.
plot_hist <- function(df, title) {
  ggplot(df, aes(x = weight)) +
    geom_histogram(bins = 40, fill = "grey70", color = "white") +
    labs(title = title, x = "Weight", y = "Count") +
    theme_pub(base_size = 10)
}

p_hist_abg <- plot_hist(filter(wt_df, group == "ABG"), "ABG weight distribution")
p_hist_vbg <- plot_hist(filter(wt_df, group == "VBG"), "VBG weight distribution")
save_diag_plot(p_hist_abg, results_path("figs", "diag-wt-weights-hist-abg.png"), width = 7, height = 5)
save_diag_plot(p_hist_vbg, results_path("figs", "diag-wt-weights-hist-vbg.png"), width = 7, height = 5)

# helper: visualize the relationship between propensities and resulting weights
# while highlighting the most extreme weights.
plot_scatter <- function(df, title) {
```

```r
  top_wt <- df |>
    slice_max(order_by = weight, n = min(20, nrow(df)))
  ggplot(df, aes(x = ps, y = weight)) +
    geom_point(alpha = 0.3, size = 0.7) +
    geom_point(data = top_wt, color = "red", size = 1.2) +
    scale_y_log10() +
    labs(title = title, x = "Propensity score", y = "Weight (log10)") +
    theme_pub(base_size = 10)
}

p_scatter_abg <- plot_scatter(filter(wt_df, group == "ABG"),
                              "ABG weights vs propensity (top 20 highlighted)")
p_scatter_vbg <- plot_scatter(filter(wt_df, group == "VBG"),
                              "VBG weights vs propensity (top 20 highlighted)")
save_diag_plot(p_scatter_abg, results_path("figs", "diag-wt-weights-vs-ps-abg.png"), width = 7, height = 5)
save_diag_plot(p_scatter_vbg, results_path("figs", "diag-wt-weights-vs-ps-vbg.png"), width = 7, height = 5)
```

4.0.4 Balance diagnostics

```r
# Purpose: diagnostics balance.
stopifnot(exists("target_balance_table"))

covars_use <- intersect(covars_ps, names(subset_data))

bal_target_abg <- target_balance_table(subset_data, "has_abg", subset_data$w_abg, covars_use, levels_ref) |>
  mutate(group = "ABG")
bal_target_vbg <- target_balance_table(subset_data, "has_vbg", subset_data$w_vbg, covars_use, levels_ref) |>
  mutate(group = "VBG")

bal_target <- bind_rows(bal_target_abg, bal_target_vbg)
write_csv_safely(bal_target, results_path("balance_table.csv"), row_names = FALSE)

bal_target_sum <- bal_target |>
  group_by(group, variable) |>
  summarise(
    max_abs_pre = max(abs(smd_pre), na.rm = TRUE),
```

```r
    max_abs_post = max(abs(smd_post), na.rm = TRUE),
    .groups = "drop"
  )
bal_worst <- bal_target_sum |>
  group_by(group) |>
  arrange(desc(max_abs_post)) |>
  ungroup()
write_csv_safely(bal_worst, results_path("balance_worst10.csv"), row_names = FALSE)
render_table_pdf_maybe(
  bal_worst,
  caption = "GBM IPSW target balance by covariate (sorted by max |SMD|)",
  file_stub = "balance_worst",
  digits = 3,
  show = FALSE
)

bal_plot_df <- bal_target |>
  mutate(label = ifelse(is.na(level), variable, paste0(variable, ":", level))) |>
  tidyr::pivot_longer(
    cols = c(smd_pre, smd_post),
    names_to = "stage",
    values_to = "smd"
  ) |>
  mutate(stage = recode(stage, smd_pre = "Pre", smd_post = "Post"))

p_bal_abg <- ggplot(filter(bal_plot_df, group == "ABG"),
                    aes(x = reorder(label, abs(smd)), y = smd, color = stage, shape = stage)) +
  geom_point(size = 1) +
  geom_hline(yintercept = c(-0.1, 0.1), linetype = 2, linewidth = 0.3) +
  geom_hline(yintercept = c(-0.05, 0.05), linetype = 3, linewidth = 0.3) +
  coord_flip() +
  labs(x = NULL, y = "Target SMD", title = "ABG target balance") +
  theme_pub(base_size = 10)

p_bal_vbg <- ggplot(filter(bal_plot_df, group == "VBG"),
                    aes(x = reorder(label, abs(smd)), y = smd, color = stage, shape = stage)) +
```

```r
  geom_point(size = 1) +
  geom_hline(yintercept = c(-0.1, 0.1), linetype = 2, linewidth = 0.3) +
  geom_hline(yintercept = c(-0.05, 0.05), linetype = 3, linewidth = 0.3) +
  coord_flip() +
  labs(x = NULL, y = "Target SMD", title = "VBG target balance") +
  theme_pub(base_size = 10)

save_diag_plot(p_bal_abg, results_path("figs", "diag-balance-loveplot-abg.png"), width = 9, height = 7)
save_diag_plot(p_bal_vbg, results_path("figs", "diag-balance-loveplot-vbg.png"), width = 9, height = 7)

# MI target balance summaries across imputations
stopifnot(exists("bal_imp_abg"), exists("bal_imp_vbg"))
bal_imp <- bind_rows(bal_imp_abg$bal_long, bal_imp_vbg$bal_long)
bal_imp_summary <- bind_rows(bal_imp_abg$bal_imp_summary, bal_imp_vbg$bal_imp_summary)
worst_rows <- bind_rows(bal_imp_abg$worst_rows_overall, bal_imp_vbg$worst_rows_overall)
worst_by_imp <- bind_rows(bal_imp_abg$worst_by_imp, bal_imp_vbg$worst_by_imp)
worst_terms_by_imp <- bind_rows(bal_imp_abg$worst_terms_by_imp, bal_imp_vbg$worst_terms_by_imp)

write_csv_safely(bal_imp, results_path("balance_target_by_imp.csv"), row_names = FALSE)
write_csv_safely(bal_imp_summary, results_path("balance_target_imp_summary.csv"), row_names = FALSE)
write_csv_safely(worst_rows, results_path("balance_target_worst_rows.csv"), row_names = FALSE)
render_table_pdf_maybe(
  worst_rows,
  caption = "MI-logistic IPSW target SMD rows across imputations (sorted by |SMD|)",
  file_stub = "balance_target_worst_rows",
  digits = 3,
  show = FALSE
)
write_csv_safely(worst_by_imp, results_path("balance_max_smd_by_imp.csv"), row_names = FALSE)
write_csv_safely(worst_terms_by_imp, results_path("balance_worst_terms.csv"), row_names = FALSE)
if (any(bal_imp_summary$max_abs_post > 0.10, na.rm = TRUE)) {
  warning("Target balance: max |SMD| > 0.10 in at least one imputation.", call. = FALSE)
}
if (nrow(bal_imp_summary)) {
  dist_tbl <- bal_imp_summary |>
    dplyr::group_by(group) |>
```

```r
    dplyr::summarise(
      med = median(max_abs_post, na.rm = TRUE),
      iqr = IQR(max_abs_post, na.rm = TRUE),
      max = max(max_abs_post, na.rm = TRUE),
      .groups = "drop"
    )
  render_table_pdf_maybe(
    dist_tbl,
    caption = "Distribution of max |Target SMD| across MI-logistic IPSW imputations",
    file_stub = "balance_target_max_smd_distribution",
    digits = 3,
    show = FALSE
  )
}
if (nrow(worst_terms_by_imp)) {
  worst_freq <- worst_terms_by_imp |>
    dplyr::count(group, term, sort = TRUE) |>
    dplyr::ungroup()
  render_table_pdf_maybe(
    worst_freq,
    caption = "Most frequent worst-balance terms under MI-logistic IPSW",
    file_stub = "balance_worst_terms_freq",
    digits = 0,
    show = FALSE
  )
}

n_abg <- sum(subset_data$has_abg == 1, na.rm = TRUE)
n_vbg <- sum(subset_data$has_vbg == 1, na.rm = TRUE)

stopifnot(exists("wt_sum"))
ess_abg <- wt_sum$ess[wt_sum$group == "ABG"]
ess_vbg <- wt_sum$ess[wt_sum$group == "VBG"]

if (is.finite(ess_abg) && is.finite(n_abg) && ess_abg < 0.2 * n_abg) {
  warning("ABG balance: ESS < 0.2 * n_abg (", round(ess_abg, 1), " vs ", n_abg, ").", call. = FALSE)
```

```r
}
if (is.finite(ess_vbg) && is.finite(n_vbg) && ess_vbg < 0.2 * n_vbg) {
  warning("VBG balance: ESS < 0.2 * n_vbg (", round(ess_vbg, 1), " vs ", n_vbg, ").", call. = FALSE)
}
```

4.0.5 Outcome diagnostics

```r
# Purpose: diagnostics outcome.
stopifnot(exists("abg_curves"), exists("vbg_curves"))
  curve_abg <- abg_curves |>
    mutate(co2 = paco2) |>
    select(-paco2)
  curve_vbg <- vbg_curves |>
    mutate(co2 = vbg_co2) |>
    select(-vbg_co2)
  curve_all <- bind_rows(curve_abg, curve_vbg) |>
    mutate(outcome = factor(outcome,
                            levels = c("imv_proc", "niv_proc", "death_60d", "hypercap_resp_failure"),
                            labels = c("IMV", "NIV", "Death (60d)", "Hypercapnic RF")))
  axis_mi_outcome <- compute_or_axis_spec(list(curve_abg, curve_vbg), lo_col = "LCL", hi_col = "UCL")

  p_outcome <- ggplot(curve_all, aes(x = co2, y = OR, color = group, fill = group)) +
    geom_line(linewidth = 0.6) +
    geom_ribbon(aes(ymin = LCL, ymax = UCL), alpha = 0.15, color = NA) +
    geom_hline(yintercept = 1, linetype = "dashed", color = "grey40") +
    or_axis_scale(axis_mi_outcome) +
    facet_wrap(~ outcome, scales = "free_x") +
    labs(
      title = "MI-pooled MI-logistic IPSW spline odds ratios: ABG vs VBG",
      x = expression(CO[2]~"(mmHg)"),
      y = paste0("Odds ratio (ref: ABG ", ABG_CO2_REF, ", VBG ", VBG_CO2_REF, "; log scale)"),
      caption = paste0("Reference CO2: ABG ", ABG_CO2_REF, " mmHg; VBG ", VBG_CO2_REF, " mmHg")
    ) +
    theme_pub(base_size = 10)
  save_diag_plot(p_outcome, results_path("figs", "diag-outcome-or-vs-paco2.png"),
                 width = 10, height = 8)
```

4.0.6 Diagnostics summary and audit

```
# Purpose: diagnostics summary.
stopifnot(exists("bal_imp_summary"), exists("wt_sum"), exists("chain_diag_stats"))
target_abg_med <- median(bal_imp_summary$max_abs_post[bal_imp_summary$group == "ABG"], na.rm = TRUE)
target_abg_max <- max(bal_imp_summary$max_abs_post[bal_imp_summary$group == "ABG"], na.rm = TRUE)
target_vbg_med <- median(bal_imp_summary$max_abs_post[bal_imp_summary$group == "VBG"], na.rm = TRUE)
target_vbg_max <- max(bal_imp_summary$max_abs_post[bal_imp_summary$group == "VBG"], na.rm = TRUE)

runtime_proj_total_hrs <- NA_real_
if (is.finite(PILOT_FRAC) && PILOT_FRAC > 0 && PILOT_FRAC < 1) {
  runtime_log_curr <- runtime_log
  runtime_log_curr <- runtime_log_curr[runtime_log_curr$run_id == runtime_run_id, , drop = FALSE]
  scalable_steps <- c("mice_imputation", "mi_single_pass")
  scalable_secs <- runtime_log_curr |>
    dplyr::filter(step_name %in% scalable_steps) |>
    dplyr::summarise(total = sum(seconds, na.rm = TRUE)) |>
    dplyr::pull(total)
  if (length(scalable_secs) && is.finite(scalable_secs)) {
    runtime_proj_total_hrs <- scalable_secs / PILOT_FRAC / 3600
  }
}

# helper: pull one group's summary row out of the weight diagnostics table and
# fail loudly if it is missing.
get_wt_row <- function(group) {
  row <- wt_sum[wt_sum$group == group, , drop = FALSE]
  if (nrow(row) == 0L) {
    stop("Weight summary missing for group: ", group)
  }
  row[, c("ps_floor", "p01", "p05", "p95", "p99", "max", "sum_w", "ess",
          "top01_weight_share", "trunc_rate"), drop = FALSE]
}

wt_abg_row <- get_wt_row("ABG")
wt_vbg_row <- get_wt_row("VBG")
```

```r
diag_summary <- bind_rows(
  tibble::tibble(
    block = "ABG weights",
    run_mode = RUN_MODE,
    m = M_IMP,
    maxit = MAXIT_MI,
    pilot_frac = PILOT_FRAC,
    stop_method = gbm_params$stop.method,
    ps_floor_quantile = ps_trunc_quantile,
    ps_floor = wt_abg_row$ps_floor,
    weight_p01 = wt_abg_row$p01,
    weight_p05 = wt_abg_row$p05,
    weight_p95 = wt_abg_row$p95,
    weight_p99 = wt_abg_row$p99,
    weight_max = wt_abg_row$max,
    sum_w = wt_abg_row$sum_w,
    ess = wt_abg_row$ess,
    top1_weight_share = wt_abg_row$top01_weight_share,
    trunc_rate = wt_abg_row$trunc_rate,
    target_max_smd_post_med = target_abg_med,
    target_max_smd_post_max = target_abg_max,
    runtime_proj_total_hrs = runtime_proj_total_hrs,
    chain_diag_n_imputed_vars = chain_diag_stats$n_imputed_vars,
    chain_diag_n_with_chainMean = chain_diag_stats$n_with_chainMean,
    chain_diag_n_with_drift_tail = chain_diag_stats$n_with_drift_tail,
    chain_diag_drift_tail_na_frac = chain_diag_stats$drift_tail_na_frac,
    chain_diag_tail_window_na_mean = chain_diag_stats$tail_window_na_mean
  ),
  tibble::tibble(
    block = "VBG weights",
    run_mode = RUN_MODE,
    m = M_IMP,
    maxit = MAXIT_MI,
    pilot_frac = PILOT_FRAC,
    stop_method = gbm_params$stop.method,
```

```
  ps_floor_quantile = ps_trunc_quantile,
  ps_floor = wt_vbg_row$ps_floor,
  weight_p01 = wt_vbg_row$p01,
  weight_p05 = wt_vbg_row$p05,
  weight_p95 = wt_vbg_row$p95,
  weight_p99 = wt_vbg_row$p99,
  weight_max = wt_vbg_row$max,
  sum_w = wt_vbg_row$sum_w,
  ess = wt_vbg_row$ess,
  top1_weight_share = wt_vbg_row$top01_weight_share,
  trunc_rate = wt_vbg_row$trunc_rate,
  target_max_smd_post_med = target_vbg_med,
  target_max_smd_post_max = target_vbg_max,
  runtime_proj_total_hrs = runtime_proj_total_hrs,
  chain_diag_n_imputed_vars = chain_diag_stats$n_imputed_vars,
  chain_diag_n_with_chainMean = chain_diag_stats$n_with_chainMean,
  chain_diag_n_with_drift_tail = chain_diag_stats$n_with_drift_tail,
  chain_diag_drift_tail_na_frac = chain_diag_stats$drift_tail_na_frac,
  chain_diag_tail_window_na_mean = chain_diag_stats$tail_window_na_mean
),
tibble::tibble(
  block = "ABG outcomes",
  run_mode = RUN_MODE,
  m = M_IMP,
  maxit = MAXIT_MI,
  pilot_frac = PILOT_FRAC,
  stop_method = gbm_params$stop.method,
  ps_floor_quantile = ps_trunc_quantile,
  ps_floor = wt_abg_row$ps_floor,
  weight_p01 = wt_abg_row$p01,
  weight_p05 = wt_abg_row$p05,
  weight_p95 = wt_abg_row$p95,
  weight_p99 = wt_abg_row$p99,
  weight_max = wt_abg_row$max,
  sum_w = wt_abg_row$sum_w,
  ess = wt_abg_row$ess,
```

```
  top1_weight_share = wt_abg_row$top01_weight_share,
  trunc_rate = wt_abg_row$trunc_rate,
  target_max_smd_post_med = target_abg_med,
  target_max_smd_post_max = target_abg_max,
  runtime_proj_total_hrs = runtime_proj_total_hrs,
  chain_diag_n_imputed_vars = chain_diag_stats$n_imputed_vars,
  chain_diag_n_with_chainMean = chain_diag_stats$n_with_chainMean,
  chain_diag_n_with_drift_tail = chain_diag_stats$n_with_drift_tail,
  chain_diag_drift_tail_na_frac = chain_diag_stats$drift_tail_na_frac,
  chain_diag_tail_window_na_mean = chain_diag_stats$tail_window_na_mean
),
tibble::tibble(
  block = "VBG outcomes",
  run_mode = RUN_MODE,
  m = M_IMP,
  maxit = MAXIT_MI,
  pilot_frac = PILOT_FRAC,
  stop_method = gbm_params$stop.method,
  ps_floor_quantile = ps_trunc_quantile,
  ps_floor = wt_vbg_row$ps_floor,
  weight_p01 = wt_vbg_row$p01,
  weight_p05 = wt_vbg_row$p05,
  weight_p95 = wt_vbg_row$p95,
  weight_p99 = wt_vbg_row$p99,
  weight_max = wt_vbg_row$max,
  sum_w = wt_vbg_row$sum_w,
  ess = wt_vbg_row$ess,
  top1_weight_share = wt_vbg_row$top01_weight_share,
  trunc_rate = wt_vbg_row$trunc_rate,
  target_max_smd_post_med = target_vbg_med,
  target_max_smd_post_max = target_vbg_max,
  runtime_proj_total_hrs = runtime_proj_total_hrs,
  chain_diag_n_imputed_vars = chain_diag_stats$n_imputed_vars,
  chain_diag_n_with_chainMean = chain_diag_stats$n_with_chainMean,
  chain_diag_n_with_drift_tail = chain_diag_stats$n_with_drift_tail,
  chain_diag_drift_tail_na_frac = chain_diag_stats$drift_tail_na_frac,
```

```r
    chain_diag_tail_window_na_mean = chain_diag_stats$tail_window_na_mean
  )
)

check_abg <- bal_imp_summary |>
  dplyr::filter(group == "ABG") |>
  dplyr::summarise(x = max(max_abs_post, na.rm = TRUE), .groups = "drop") |>
  dplyr::pull(x)
check_vbg <- bal_imp_summary |>
  dplyr::filter(group == "VBG") |>
  dplyr::summarise(x = max(max_abs_post, na.rm = TRUE), .groups = "drop") |>
  dplyr::pull(x)
diag_abg <- diag_summary$target_max_smd_post_max[diag_summary$block == "ABG weights"]
diag_vbg <- diag_summary$target_max_smd_post_max[diag_summary$block == "VBG weights"]
if (length(diag_abg) && is.finite(check_abg)) {
  stopifnot(isTRUE(all.equal(check_abg, diag_abg, tolerance = 1e-8)))
}
if (length(diag_vbg) && is.finite(check_vbg)) {
  stopifnot(isTRUE(all.equal(check_vbg, diag_vbg, tolerance = 1e-8)))
}
stopifnot(exists("covars_use_abg"), exists("covars_use_vbg"))
stopifnot(setequal(covars_use_abg, covars_use_vbg))

diag_summary_file <- results_path("diagnostics_summary.csv")
write_csv_safely(diag_summary, diag_summary_file, row_names = FALSE)

diag_summary_display <- diag_summary |>
  dplyr::select(
    block,
    m,
    ess,
    trunc_rate,
    target_max_smd_post_med,
    target_max_smd_post_max
  ) |>
  dplyr::rename(
```

```r
    `trunc` = trunc_rate,
    `med_max_smd` = target_max_smd_post_med,
    `max_max_smd` = target_max_smd_post_max
  )
render_table_pdf_maybe(diag_summary_display,
                       "Diagnostics summary (GBM IPSW + MI-logistic IPSW)",
                       "diagnostics_summary_display",
                       wide = TRUE,
                       digits = 3,
                       show = FALSE)

audit_lines <- c(
  "# Diagnostics Audit",
  "",
  "## A1. MI workflow",
  "- Impute -> single-pass per-imputation loop (weights, target balance, 3-level outcomes, spline outcomes) -> pool
   ↪  curves and coefficients with Rubin's rules (chunk: `mi-single-pass`; downstream MI display chunks read those
   ↪  objects).",
  "## A2. MI settings",
  paste0("- m = ", M_IMP, ", maxit = ", MAXIT_MI, ", seed = ", MI_SEED,
         "; treatments/outcomes/PaCO2/VBG CO2 are not imputed but are predictors (`mi-exec`)."),
  "## A3. Propensity weighting",
  "- Unimputed weighting uses WeightIt with method = \"gbm\" and balance-based stopping (stop.method = \"smd.max\");
   ↪  no AUC-based tuning (`propensity-config`, `ipw-abg-weighting`, `ipw-vbg-workflow`).",
  "- MI weighting uses logistic PS with restricted cubic splines (glm + rcs); no SHAP is computed for MI.",
  "## A4. One-sided IPSW + truncation",
  "- Weights are 1/ps for observed tests (ABG or VBG), truncated only for very small propensities (ps floor = 1st
   ↪  percentile), then stabilized by the mean; controls receive weight 1 for balance diagnostics only.",
  "## A5. Robust variance",
  "- Outcome models are survey::svyglm with svydesign (robust SEs), using spline(CO2) + X adjustment; ABG and VBG are
   ↪  fit separately within the measured cohort.",
  "## A6. Pooling",
  "- mitools::MIcombine pools coefficients and robust vcov from svyglm; spline curves are pooled pointwise on log-OR
   ↪  (relative to CO2_ref) and on eta for predicted-probability curves (helpers defined in `mi-single-pass`).",
  "",
  "## Potential mismatches / risks",
```

```
  "- Target balance diagnostics compare weighted treated cohort to the full analytic sample (no treated-vs-control
   ↪  balance).",
  "- MI stability across m uses subsets of the first m imputations from the main mids object (not full re-imputation
   ↪  at each m).",
  "- Unweighted analyses remain earlier in the notebook for context; primary inference is based on weighted spline
   ↪  models."
)
audit_file <- results_path("diagnostics_audit.md")
write_diag_lines(audit_lines, audit_file)
```

4.0.7 Performance / runtime log

```
# Purpose: warnings summary.
stopifnot(exists("mi_warn_log"), exists("mi_info_log"))
warn_df <- mi_warn_log
write_csv_safely(warn_df, results_path("mi_warnings_log.csv"), row_names = FALSE)
write_csv_safely(warn_df, results_path("warnings_log.csv"), row_names = FALSE)
write_csv_safely(mi_info_log, results_path("mi_info_log.csv"), row_names = FALSE)

if (nrow(warn_df)) {
  msg_counts <- warn_df |>
    dplyr::count(stage, component, message, sort = TRUE)
  render_table_pdf_maybe(
    msg_counts,
    caption = "Warning messages by stage/component",
    file_stub = "warnings_by_stage_component",
    digits = 0,
    show = FALSE
  )

  nonconv <- warn_df |>
    dplyr::filter(grepl("glm.fit: algorithm did not converge", message, fixed = TRUE))
  if (nrow(nonconv)) {
    nonconv_ctx <- nonconv |>
      dplyr::count(stage, component, analysis_variant, model_type, outcome, group, imputation, sort = TRUE)
    render_table_pdf_maybe(
```

```r
      nonconv_ctx,
      caption = "Nonconvergence contexts",
      file_stub = "nonconvergence_contexts",
      digits = 0,
      show = FALSE
    )
  }
} else {
  message("No captured warnings.")
}

write_csv_safely(plot_drop_log(), results_path("plot_drop_log.csv"), row_names = FALSE)
```

```r
# Purpose: mi nonconvergence summary.
stopifnot(exists("mi_outcome_diag"))
if (nrow(mi_outcome_diag) == 0L) {
  stop("mi_outcome_diag is empty; outcome diagnostics were not captured.")
}
out_diag_file <- results_path("model_fit_diagnostics.csv")
write_csv_safely(mi_outcome_diag, out_diag_file)
write_csv_safely(mi_outcome_diag, results_path("mi_outcome_fit_diagnostics.csv"))

nonconv <- mi_outcome_diag |>
  dplyr::filter(isTRUE(nonconv_flag) | isFALSE(converged))
sep <- mi_outcome_diag |>
  dplyr::filter(isTRUE(sep_flag))

if (nrow(nonconv)) {
  nonconv_counts <- nonconv |>
    dplyr::count(analysis_variant, group, outcome, model_type, sort = TRUE)
  render_table_pdf_maybe(
    nonconv_counts,
    caption = "Nonconverged fits by variant/group/outcome",
    file_stub = "nonconverged_fits_by_variant",
    digits = 0,
    show = FALSE
  )
```

```r
  worst_imps <- nonconv |>
    dplyr::count(imputation, sort = TRUE)
  render_table_pdf_maybe(
    worst_imps,
    caption = "Imputations with most nonconverged fits",
    file_stub = "nonconverged_imputations",
    digits = 0,
    show = FALSE
  )
} else {
  message("No nonconverged outcome fits recorded.")
}

if (nrow(sep)) {
  sep_counts <- sep |>
    dplyr::count(analysis_variant, group, outcome, model_type, sort = TRUE)
  render_table_pdf_maybe(
    sep_counts,
    caption = "Separation flags by variant/group/outcome",
    file_stub = "separation_flags_by_variant",
    digits = 0,
    show = FALSE
  )
} else {
  message("No separation flags recorded.")
}

  worst_phat <- mi_outcome_diag |>
    dplyr::filter(is.finite(min_phat) | is.finite(max_phat)) |>
    dplyr::mutate(extreme = pmin(min_phat, 1 - max_phat, na.rm = TRUE)) |>
    dplyr::arrange(extreme)
if (nrow(worst_phat)) {
  render_table_pdf_maybe(
    worst_phat,
    caption = "Worst fitted-probability extremes",
```

```r
    file_stub = "worst_phat_extremes",
    digits = 3,
    show = FALSE
  )
}

  diag_df <- mi_outcome_diag
  diag_df$sep_warn <- grepl("fitted probabilities numerically 0 or 1",
                            diag_df$top_warning, fixed = TRUE) |
    grepl("separat", diag_df$top_warning, ignore.case = TRUE)
  diag_df$phat_extreme <- (diag_df$min_phat < PROB_EPS) |
    (diag_df$max_phat > 1 - PROB_EPS)
  diag_df$sep_flag <- dplyr::coalesce(as.logical(diag_df$sep_flag), FALSE)
  diag_df$nonconv_flag <- dplyr::coalesce(as.logical(diag_df$nonconv_flag), FALSE) |
    dplyr::coalesce(!as.logical(diag_df$converged), FALSE)

  mi_variants <- intersect(c("mi_ipw", "weighted_imputed"), unique(diag_df$analysis_variant))
  if (!length(mi_variants)) mi_variants <- unique(diag_df$analysis_variant)

issue_summary <- diag_df |>
  dplyr::filter(analysis_variant %in% mi_variants) |>
  dplyr::group_by(group, outcome, component, analysis_variant) |>
  dplyr::summarise(
    n_fits = dplyr::n(),
    n_nonconv = sum(nonconv_flag %in% TRUE, na.rm = TRUE),
    n_sep_warn = sum(sep_warn %in% TRUE, na.rm = TRUE),
    n_phat_extreme = sum(phat_extreme %in% TRUE, na.rm = TRUE),
    n_sep_flag = sum(sep_flag %in% TRUE, na.rm = TRUE),
    .groups = "drop"
  )
write_csv_safely(issue_summary, results_path("mi_fit_issue_summary.csv"))

# Top warning messages by stage/component
if (nrow(mi_warn_log)) {
  warn_top <- mi_warn_log |>
    dplyr::count(stage, component, analysis_variant, model_type, message, sort = TRUE)
```

```r
  render_table_pdf_maybe(
    warn_top,
    caption = "Warnings by stage/component",
    file_stub = "warnings_by_stage_component_full",
    digits = 0,
    show = FALSE
  )
}

# Top MICE loggedEvents drivers
stopifnot(exists("log_events_summary"))
if (nrow(log_events_summary)) {
  render_table_pdf_maybe(
    log_events_summary,
    caption = "MICE loggedEvents drivers",
    file_stub = "mice_logged_events_drivers",
    digits = 0,
    show = FALSE
  )
}

# Replay debugging is intentionally omitted from the canonical render path.
```

4.0.8 Performance / runtime log

```r
# Purpose: runtime log export.
stopifnot(exists("runtime_log"), nrow(runtime_log) > 0)
runtime_log_curr <- runtime_log
runtime_log_curr <- runtime_log_curr[runtime_log_curr$run_id == runtime_run_id, , drop = FALSE]
runtime_log_file <- results_path("runtime_log.csv")
write_csv_safely(runtime_log_curr, runtime_log_file, row_names = FALSE)
top_steps <- runtime_log_curr |>
  arrange(desc(seconds))
top15_file <- results_path("runtime_summary_top15.csv")
write_csv_safely(top_steps, top15_file, row_names = FALSE)
render_table_pdf_maybe(
```

```r
  top_steps,
  caption = "Runtime steps (seconds)",
  file_stub = "runtime_steps",
  digits = 3,
  show = FALSE
)

total_seconds <- sum(runtime_log_curr$seconds, na.rm = TRUE)
runtime_summary <- bind_rows(
  tibble::tibble(step_name = "TOTAL", seconds = total_seconds),
  runtime_log_curr |>
    arrange(desc(seconds)) |>
    select(step_name, seconds)
)
runtime_summary_file <- results_path("runtime_summary.csv")
write_csv_safely(runtime_summary, runtime_summary_file, row_names = FALSE)
render_table_pdf_maybe(
  runtime_summary,
  caption = "Runtime summary (total + all steps)",
  file_stub = "runtime_summary",
  digits = 3,
  show = FALSE
)

# GBM preflight diagnostics (CSV only)
write_csv_safely(memory_snapshots, results_path("memory_snapshots.csv"), row_names = FALSE)
write_csv_safely(gbm_preflight_design_dims, results_path("gbm_preflight_design_dims.csv"), row_names = FALSE)
write_csv_safely(gbm_preflight_warnings, results_path("gbm_preflight_warnings.csv"), row_names = FALSE)

if (is.finite(PILOT_FRAC) && PILOT_FRAC > 0 && PILOT_FRAC < 1) {
  scalable_steps <- c("mice_imputation", "mi_single_pass")
  proj <- runtime_log_curr |>
    filter(step_name %in% scalable_steps) |>
    mutate(projected_seconds = seconds / PILOT_FRAC,
           projected_hours = projected_seconds / 3600)
  proj_total <- sum(proj$projected_hours, na.rm = TRUE)
```

```r
  render_table_pdf_maybe(
    proj |> select(step_name, seconds, projected_hours),
    caption = paste0("Runtime projection (scalable steps; pilot_frac = ", PILOT_FRAC, ")"),
    file_stub = "runtime_projection",
    digits = 3,
    show = FALSE
  )
  if (is.finite(proj_total) && proj_total > 10) {
    warning("Projected full-run time exceeds 10 hours (", round(proj_total, 1), "h).",
            call. = FALSE)
  }
} else {
  message("Runtime projection skipped because PILOT_FRAC == 1 (full run).")
}

stopifnot(exists("get_imp_stats"))
get_imp_df <- data.frame(
  count = get_imp_stats$count,
  seconds = get_imp_stats$seconds,
  seconds_per_call = if (get_imp_stats$count > 0) get_imp_stats$seconds / get_imp_stats$count else NA_real_,
  run_id = runtime_run_id,
  run_mode = RUN_MODE,
  n_subset = nrow(subset_data),
  stringsAsFactors = FALSE
)
write_csv_safely(get_imp_df, results_path("get_imp_usage.csv"), row_names = FALSE)
```

5 Analysis of the discordance between predicted probabilities and OR for NIV and IMV

This section is diagnostic and additive. It does not change the primary manuscript estimates; instead it reproduces the NIV/IMV probability-OR discordance and checks whether it is explained by reference profiles, probability scale, residual capture/missingness, sentinel procedure capture, support, setting/site, subgroup, or timing structure.

5.1 Pilot validation and scope

```
# Purpose: additive diagnostics for NIV/IMV probability-OR discordance.
discordance_outcome_vars <- c("imv_proc", "niv_proc")
discordance_outcome_labels <- c(imv_proc = "IMV", niv_proc = "NIV")
discordance_groups <- c("ABG", "VBG")

discordance_status_rows <- list()

record_discordance_status <- function(component, status, reason,
                                      artifact_path = "",
                                      optional = FALSE) {
  discordance_status_rows[[length(discordance_status_rows) + 1L]] <<- tibble::tibble(
    component = component,
    status = status,
    reason = reason,
    artifact_path = artifact_path,
    optional = optional,
    run_mode = RUN_MODE,
    pilot_frac = PILOT_FRAC
  )
  invisible(TRUE)
}

write_discordance_status <- function() {
  status_df <- dplyr::bind_rows(discordance_status_rows)
  write_csv_safely(
    status_df,
    results_path("discordance_validation_status.csv"),
    row_names = FALSE
  )
  invisible(status_df)
}

safe_relative_gap <- function(abg_value, vbg_value) {
  ifelse(is.finite(vbg_value) & abs(vbg_value) > 0,
```

```
        (abg_value - vbg_value) / abs(vbg_value),
        NA_real_)
}

weighted_mean_safe <- function(x, w = NULL) {
  x <- suppressWarnings(as.numeric(x))
  if (is.null(w)) {
    return(mean(x, na.rm = TRUE))
  }
  w <- suppressWarnings(as.numeric(w))
  ok <- is.finite(x) & is.finite(w) & w > 0
  if (!any(ok)) return(NA_real_)
  sum(w[ok] * x[ok]) / sum(w[ok])
}

weighted_ess <- function(w) {
  w <- suppressWarnings(as.numeric(w))
  w <- w[is.finite(w) & w > 0]
  if (!length(w)) return(NA_real_)
  sum(w)^2 / sum(w^2)
}

first_existing_col <- function(candidates, data_names) {
  hits <- candidates[candidates %in% data_names]
  if (length(hits)) hits[[1L]] else NA_character_
}

candidate_cols <- function(data_names, pattern, exclude = character(), max_n = Inf) {
  out <- grep(pattern, data_names, value = TRUE, ignore.case = TRUE, perl = TRUE)
  out <- setdiff(out, exclude)
  unique(utils::head(out, max_n))
}

compact_spline_fit <- function(x) {
  if (is.list(x) && !is.null(x$fit) && is.list(x$fit) &&
      !is.null(x$fit$coef) && !is.null(x$fit$vcov)) {
```

```r
    return(x$fit)
  }
  if (is.list(x) && !is.null(x$coef) && !is.null(x$vcov)) {
    return(x)
  }
  NULL
}

predict_spline_prob_from_compact <- function(fit_compact, grid_df,
                                             outcome_var,
                                             co2_var,
                                             adj_vars = adj_core) {
  stopifnot(is.list(fit_compact), !is.null(fit_compact$coef), !is.null(fit_compact$vcov))
  stopifnot(is.data.frame(grid_df), co2_var %in% names(grid_df))

  beta <- fit_compact$coef
  V <- fit_compact$vcov
  bn <- names(beta)
  if (is.null(bn) || any(!nzchar(bn))) {
    stop("predict_spline_prob_from_compact: coefficient names are required.")
  }

  if (!is.null(fit_compact$terms)) {
    tt <- fit_compact$terms
    if (!identical(attr(tt, ".Environment"), baseenv())) {
      stop("predict_spline_prob_from_compact: compact terms retain a non-base environment.")
    }
    mf_new <- stats::model.frame(
      tt,
      data = grid_df,
      na.action = stats::na.pass,
      xlev = fit_compact$xlevels
    )
    mm <- stats::model.matrix(tt, mf_new, contrasts.arg = fit_compact$contrasts)
  } else {
    rhs_terms <- c(sprintf("splines::ns(%s, %d)", co2_var, SPLINE_DF), adj_vars)
```

```r
  fml <- stats::as.formula(paste(outcome_var, "~", paste(rhs_terms, collapse = " + ")))
  tt <- stats::delete.response(stats::terms(fml))
  mm <- stats::model.matrix(tt, data = grid_df)
}

missing_cols <- setdiff(bn, colnames(mm))
if (length(missing_cols)) {
  mm <- cbind(
    mm,
    matrix(
      0,
      nrow = nrow(mm),
      ncol = length(missing_cols),
      dimnames = list(NULL, missing_cols)
    )
  )
}
mm <- mm[, bn, drop = FALSE]

if (!is.null(rownames(V)) && all(bn %in% rownames(V)) && all(bn %in% colnames(V))) {
  V <- V[bn, bn, drop = FALSE]
}

eta <- as.numeric(mm %*% beta)
mmV <- mm %*% V
var_eta <- pmax(rowSums(mmV * mm), 0)
se_eta <- sqrt(var_eta)

tibble::as_tibble(grid_df) |>
  dplyr::mutate(
    eta = eta,
    SE_eta = se_eta,
    var_eta = var_eta,
    p = stats::plogis(eta),
    p_LCL = stats::plogis(eta - 1.96 * se_eta),
    p_UCL = stats::plogis(eta + 1.96 * se_eta)
```

```r
    )
}

pool_spline_prob_curve_from_compact <- function(fit_list, grid_df,
                                                outcome_var,
                                                co2_var,
                                                co2_ref,
                                                min_ok_frac = 0.9) {
  pred_list <- lapply(fit_list, function(x) {
    fit_compact <- compact_spline_fit(x)
    if (is.null(fit_compact)) return(NULL)
    tryCatch(
      predict_spline_prob_from_compact(
        fit_compact,
        grid_df = grid_df,
        outcome_var = outcome_var,
        co2_var = co2_var
      ),
      error = function(e) NULL
    )
  })

  n_grid <- nrow(grid_df)
  ok <- vapply(pred_list, function(x) {
    is.data.frame(x) &&
      nrow(x) == n_grid &&
      all(c("eta", "var_eta") %in% names(x))
  }, logical(1L))
  m_tot <- length(ok)
  m_ok <- sum(ok)
  if (m_ok < ceiling(min_ok_frac * m_tot)) {
    stop("Too many failed compact spline predictions: m_ok=", m_ok, " / m_total=", m_tot)
  }

  eta_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
  var_mat <- matrix(NA_real_, nrow = n_grid, ncol = m_ok)
```

```r
  ok_idx <- which(ok)
  for (j in seq_along(ok_idx)) {
    pred_j <- pred_list[[ok_idx[[j]]]]
    eta_mat[, j] <- pred_j$eta
    var_mat[, j] <- pred_j$var_eta
  }

  qbar <- rowMeans(eta_mat, na.rm = TRUE)
  ubar <- rowMeans(var_mat, na.rm = TRUE)
  bvar <- apply(eta_mat, 1, stats::var, na.rm = TRUE)
  tvar <- ubar + (1 + 1 / m_ok) * bvar
  se_eta <- sqrt(pmax(tvar, 0))

  tibble::as_tibble(grid_df) |>
    dplyr::mutate(
      eta = qbar,
      SE_eta = se_eta,
      var_eta = tvar,
      p = stats::plogis(qbar),
      p_LCL = stats::plogis(qbar - 1.96 * se_eta),
      p_UCL = stats::plogis(qbar + 1.96 * se_eta),
      co2_ref = co2_ref,
      m_used = m_ok,
      m_total = m_tot
    )
}

nearest_curve_value <- function(curve_df, co2_var, target_value, value_col) {
  if (!nrow(curve_df) || !(co2_var %in% names(curve_df)) || !(value_col %in% names(curve_df))) {
    return(NA_real_)
  }
  x <- suppressWarnings(as.numeric(curve_df[[co2_var]]))
  ok <- is.finite(x)
  if (!any(ok) || !is.finite(target_value)) return(NA_real_)
  idx <- which(ok)[which.min(abs(x[ok] - target_value))]
  suppressWarnings(as.numeric(curve_df[[value_col]][[idx]]))
```

```r
}

co2_level_targets <- function(group) {
  if (identical(group, "ABG")) {
    return(c(Low = ABG_CO2_LOW, Normal = co2_ref_abg_mi, High = ABG_CO2_HIGH))
  }
  c(Low = VBG_CO2_LOW, Normal = co2_ref_vbg_mi, High = VBG_CO2_HIGH)
}

cat_prob_value <- function(group, outcome_label, category, value_col = "p") {
  group_i <- group
  outcome_i <- outcome_label
  category_i <- category
  out <- main_cat3_probs_conditional_xref |>
    dplyr::filter(
      .data$group == .env$group_i,
      .data$outcome == .env$outcome_i,
      .data$category == .env$category_i
    ) |>
    dplyr::slice_head(n = 1) |>
    dplyr::pull(dplyr::all_of(value_col))
  if (length(out)) suppressWarnings(as.numeric(out[[1L]])) else NA_real_
}

cat_or_value <- function(group, outcome_label, contrast, value_col = "OR") {
  group_i <- group
  outcome_i <- outcome_label
  contrast_i <- contrast
  out <- main_cat3_results |>
    dplyr::filter(
      .data$group == .env$group_i,
      .data$outcome == .env$outcome_i,
      .data$contrast == .env$contrast_i
    ) |>
    dplyr::slice_head(n = 1) |>
    dplyr::pull(dplyr::all_of(value_col))
```

```r
  if (length(out)) suppressWarnings(as.numeric(out[[1L]])) else NA_real_
}

spline_current_summary_one <- function(group, outcome_var) {
  outcome_label <- discordance_outcome_labels[[outcome_var]]
  co2_var <- if (identical(group, "ABG")) "paco2" else "vbg_co2"
  or_df <- if (identical(group, "ABG")) abg_curves else vbg_curves
  prob_df <- if (identical(group, "ABG")) abg_prob_curves_conditional_xref else vbg_prob_curves_conditional_xref
  or_df <- or_df |>
    dplyr::filter(.data$group == group, .data$outcome == outcome_var)
  prob_df <- prob_df |>
    dplyr::filter(.data$group == group, .data$outcome == outcome_var)
  targets <- co2_level_targets(group)

  tibble::tibble(
    outcome = outcome_label,
    modality = group,
    analysis_family = "MI-logistic IPSW",
    co2_representation = "spline",
    or_low_vs_normal = nearest_curve_value(or_df, co2_var, targets[["Low"]], "OR"),
    or_high_vs_normal = nearest_curve_value(or_df, co2_var, targets[["High"]], "OR"),
    prob_normal = nearest_curve_value(prob_df, co2_var, targets[["Normal"]], "p"),
    prob_low = nearest_curve_value(prob_df, co2_var, targets[["Low"]], "p"),
    prob_high = nearest_curve_value(prob_df, co2_var, targets[["High"]], "p"),
    notes = "Current modality-specific reference profile."
  )
}

cat_current_summary <- tidyr::crossing(
  outcome_var = discordance_outcome_vars,
  modality = discordance_groups
) |>
  dplyr::mutate(outcome = unname(discordance_outcome_labels[outcome_var])) |>
  dplyr::rowwise() |>
  dplyr::mutate(
    analysis_family = "MI-logistic IPSW",
```

```r
    co2_representation = "cat3",
    or_low_vs_normal = cat_or_value(modality, outcome, "Low vs normal"),
    or_high_vs_normal = cat_or_value(modality, outcome, "High vs normal"),
    prob_normal = cat_prob_value(modality, outcome, "Normal"),
    prob_low = cat_prob_value(modality, outcome, "Low"),
    prob_high = cat_prob_value(modality, outcome, "High"),
    notes = "Current categorical ORs and predicted probabilities."
  ) |>
  dplyr::ungroup() |>
  dplyr::select(
    outcome, modality, analysis_family, co2_representation,
    or_low_vs_normal, or_high_vs_normal, prob_normal, prob_low, prob_high, notes
  )

spline_current_summary <- purrr::map_dfr(discordance_groups, function(group) {
  purrr::map_dfr(discordance_outcome_vars, function(outcome_var) {
    spline_current_summary_one(group, outcome_var)
  })
})

discordance_current_summary <- dplyr::bind_rows(
  cat_current_summary,
  spline_current_summary
)
write_csv_safely(
  discordance_current_summary,
  results_path("discordance_current_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "current_discordance",
  "completed",
  "Current categorical and spline summaries were extracted from primary MI outputs.",
  "Results/discordance_current_summary.csv"
)
```

```r
current_metric_long <- dplyr::bind_rows(
  discordance_current_summary |>
    dplyr::transmute(
      outcome,
      co2_representation,
      metric_type = "OR",
      level = "Low vs normal",
      modality,
      value = or_low_vs_normal
    ),
  discordance_current_summary |>
    dplyr::transmute(
      outcome,
      co2_representation,
      metric_type = "OR",
      level = "High vs normal",
      modality,
      value = or_high_vs_normal
    ),
  discordance_current_summary |>
    dplyr::transmute(
      outcome,
      co2_representation,
      metric_type = "probability",
      level = "Normal",
      modality,
      value = prob_normal
    ),
  discordance_current_summary |>
    dplyr::transmute(
      outcome,
      co2_representation,
      metric_type = "probability",
      level = "Low",
      modality,
      value = prob_low
```

```r
    ),
  discordance_current_summary |>
    dplyr::transmute(
      outcome,
      co2_representation,
      metric_type = "probability",
      level = "High",
      modality,
      value = prob_high
    )
)

discordance_gap_metrics <- current_metric_long |>
  tidyr::pivot_wider(names_from = modality, values_from = value) |>
  dplyr::transmute(
    outcome,
    co2_representation,
    metric_type,
    level,
    abg_value = ABG,
    vbg_value = VBG,
    absolute_gap = ABG - VBG,
    relative_gap = safe_relative_gap(ABG, VBG),
    comment = dplyr::case_when(
      metric_type == "OR" ~ "ABG-VBG gap on relative odds scale.",
      metric_type == "probability" ~ "ABG-VBG gap on absolute probability scale.",
      TRUE ~ "ABG-VBG gap."
    )
  )
write_csv_safely(
  discordance_gap_metrics,
  results_path("discordance_gap_metrics.csv"),
  row_names = FALSE
)
record_discordance_status(
  "gap_metrics",
```

```r
  "completed",
  "ABG-VBG gaps were computed for OR and probability summaries.",
  "Results/discordance_gap_metrics.csv"
)

## Reproduction of the current discordance
render_table_pdf(
  discordance_current_summary |>
    dplyr::mutate(
      dplyr::across(
        c(or_low_vs_normal, or_high_vs_normal, prob_normal, prob_low, prob_high),
        ~ round(.x, 3)
      )
    ),
  caption = "Current NIV/IMV OR and predicted-probability summaries",
  file_stub = "discordance_current_summary_display",
  wide = TRUE,
  digits = 3,
  max_cols = 6,
  landscape = TRUE
)
```

Current NIV/IMV OR and predicted-probability summaries

outcome	modality	analysis_family	co2_representation	or_low_vs_normal	or_high_vs_normal
IMV	ABG	MI-logistic IPSW	cat3	1.380	1.332
IMV	VBG	MI-logistic IPSW	cat3	1.391	1.559
NIV	ABG	MI-logistic IPSW	cat3	0.984	2.620
NIV	VBG	MI-logistic IPSW	cat3	0.966	2.852
IMV	ABG	MI-logistic IPSW	spline	1.191	1.046
NIV	ABG	MI-logistic IPSW	spline	1.016	1.378
IMV	VBG	MI-logistic IPSW	spline	1.090	1.103
NIV	VBG	MI-logistic IPSW	spline	0.992	1.392

prob_normal	prob_low	prob_high	notes
0.207	0.265	0.258	Current categorical ORs and predicted probabilities.
0.164	0.214	0.234	Current categorical ORs and predicted probabilities.
0.087	0.086	0.201	Current categorical ORs and predicted probabilities.
0.022	0.021	0.059	Current categorical ORs and predicted probabilities.
0.203	0.233	0.210	Current modality-specific reference profile.
0.084	0.085	0.112	Current modality-specific reference profile.
0.165	0.177	0.179	Current modality-specific reference profile.
0.020	0.020	0.028	Current modality-specific reference profile.

```r
## Common-reference and marginal-standardization checks
common_ref_source <- subset_data |>
  dplyr::filter(
    (.data$has_abg == 1 & is.finite(suppressWarnings(as.numeric(.data$paco2)))) |
      (.data$has_vbg == 1 & is.finite(suppressWarnings(as.numeric(.data$vbg_co2))))
  )
if (!nrow(common_ref_source)) common_ref_source <- subset_data
x_ref_common <- make_ref_profile(common_ref_source, adj_core)

grid_abg_common_info <- make_co2_grid_ref(
  "paco2",
  grid_abg_mi$paco2,
  x_ref_common,
  co2_ref_abg_mi
)
grid_vbg_common_info <- make_co2_grid_ref(
  "vbg_co2",
  grid_vbg_mi$vbg_co2,
  x_ref_common,
  co2_ref_vbg_mi
)

common_ref_curve_one <- function(group, outcome_var) {
  fit_list <- if (identical(group, "ABG")) spline_fits_abg[[outcome_var]] else spline_fits_vbg[[outcome_var]]
  grid_info <- if (identical(group, "ABG")) grid_abg_common_info else grid_vbg_common_info
  co2_var <- if (identical(group, "ABG")) "paco2" else "vbg_co2"
  co2_ref <- if (identical(group, "ABG")) co2_ref_abg_mi else co2_ref_vbg_mi
  pool_spline_prob_curve_from_compact(
    fit_list,
    grid_df = grid_info$grid,
    outcome_var = outcome_var,
    co2_var = co2_var,
    co2_ref = co2_ref
  ) |>
    dplyr::mutate(
      outcome = discordance_outcome_labels[[outcome_var]],
```

```
      outcome_var = outcome_var,
      group = group,
      strategy = "common_reference",
      co2_scale = co2_var,
      co2_value = .data[[co2_var]]
    )
}

discordance_xref_common_prob_curves <- purrr::map_dfr(discordance_groups, function(group) {
  purrr::map_dfr(discordance_outcome_vars, function(outcome_var) {
    common_ref_curve_one(group, outcome_var)
  })
})
write_csv_safely(
  discordance_xref_common_prob_curves,
  results_path("discordance_xref_common_prob_curves.csv"),
  row_names = FALSE
)

standardization_sample_cap <- if (IS_PILOT) 1000L else 20000L
min_marginal_ok_frac <- 0.9
min_marginal_required <- ceiling(min_marginal_ok_frac * imp_n)
standardization_source_rows <- function(d, imp_index) {
  d <- d[, intersect(c(adj_core, "has_abg", "has_vbg"), names(d)), drop = FALSE]
  d <- d[stats::complete.cases(d[, adj_core, drop = FALSE]), , drop = FALSE]
  if (nrow(d) <= standardization_sample_cap) return(d)
  set.seed(7100L + imp_index)
  d[sample.int(nrow(d), standardization_sample_cap), , drop = FALSE]
}

marginal_predict_compact <- function(fit_compact, source_df, co2_var,
                                     co2_values, outcome_var) {
  purrr::map_dfr(co2_values, function(co2_value) {
    nd <- source_df
    nd[[co2_var]] <- co2_value
    pred <- predict_spline_prob_from_compact(
```

```
      fit_compact,
      grid_df = nd,
      outcome_var = outcome_var,
      co2_var = co2_var
    )
    tibble::tibble(
      co2_value = co2_value,
      p_imp = mean(pred$p, na.rm = TRUE),
      n_standardized = nrow(nd)
    )
  })
}

marginal_rows <- list()
marginal_status_rows <- list()

record_marginal_status <- function(imp_index, group, outcome_var, co2_var,
                                   status, error_message = "",
                                   n_standardized = NA_integer_,
                                   n_grid = NA_integer_) {
  marginal_status_rows[[length(marginal_status_rows) + 1L]] <<- tibble::tibble(
    imp = imp_index,
    group = group,
    outcome = discordance_outcome_labels[[outcome_var]],
    outcome_var = outcome_var,
    co2_scale = co2_var,
    status = status,
    error_message = error_message,
    n_standardized = as.integer(n_standardized),
    n_grid = as.integer(n_grid)
  )
  invisible(TRUE)
}

for (imp_index in seq_len(imp_n)) {
  d_imp <- get_imp(imp_index)
```

```r
source_imp <- standardization_source_rows(d_imp, imp_index)
for (group in discordance_groups) {
  co2_var <- if (identical(group, "ABG")) "paco2" else "vbg_co2"
  co2_values <- if (identical(group, "ABG")) grid_abg_mi$paco2 else grid_vbg_mi$vbg_co2
  fit_bank <- if (identical(group, "ABG")) spline_fits_abg else spline_fits_vbg
  for (outcome_var in discordance_outcome_vars) {
    fit_compact <- compact_spline_fit(fit_bank[[outcome_var]][[imp_index]])
    if (is.null(fit_compact)) {
      record_marginal_status(
        imp_index,
        group,
        outcome_var,
        co2_var,
        status = "skipped_no_compact_fit",
        error_message = "compact spline fit unavailable",
        n_standardized = nrow(source_imp),
        n_grid = length(co2_values)
      )
      next
    }
    pred_try <- tryCatch(
      list(
        value = marginal_predict_compact(
          fit_compact,
          source_df = source_imp,
          co2_var = co2_var,
          co2_values = co2_values,
          outcome_var = outcome_var
        ),
        error = NULL
      ),
      error = function(e) list(value = NULL, error = conditionMessage(e))
    )
    if (!is.null(pred_try$error) || is.null(pred_try$value) || !nrow(pred_try$value)) {
      error_message <- if (is.null(pred_try$error)) {
        "prediction returned no rows"
```

```
      } else {
        pred_try$error
      }
      record_marginal_status(
        imp_index,
        group,
        outcome_var,
        co2_var,
        status = "failed_prediction",
        error_message = error_message,
        n_standardized = nrow(source_imp),
        n_grid = length(co2_values)
      )
      next
    }
    pred_imp <- pred_try$value
    record_marginal_status(
      imp_index,
      group,
      outcome_var,
      co2_var,
      status = "completed",
      n_standardized = nrow(source_imp),
      n_grid = nrow(pred_imp)
    )
    marginal_rows[[length(marginal_rows) + 1L]] <- pred_imp |>
      dplyr::mutate(
        imp = imp_index,
        outcome = discordance_outcome_labels[[outcome_var]],
        outcome_var = outcome_var,
        group = group,
        co2_scale = co2_var,
        strategy = "marginal_standardized"
      )
  }
}
```

```r
  rm(d_imp, source_imp)
  invisible(gc(FALSE))
}

marginal_status_raw <- dplyr::bind_rows(marginal_status_rows)
required_marginal_combos <- tidyr::crossing(
  group = discordance_groups,
  outcome_var = discordance_outcome_vars
) |>
  dplyr::mutate(
    outcome = unname(discordance_outcome_labels[outcome_var]),
    m_total = imp_n,
    min_required = min_marginal_required
  )
marginal_completion_summary <- marginal_status_raw |>
  dplyr::group_by(group, outcome_var, outcome) |>
  dplyr::summarise(
    m_used = sum(.data$status == "completed", na.rm = TRUE),
    m_failed = sum(.data$status == "failed_prediction", na.rm = TRUE),
    m_skipped_no_compact_fit = sum(.data$status == "skipped_no_compact_fit", na.rm = TRUE),
    m_attempted = dplyr::n(),
    first_error = {
      errors <- .data$error_message[nzchar(.data$error_message)]
      if (length(errors)) errors[[1L]] else ""
    },
    .groups = "drop"
  ) |>
  dplyr::right_join(
    required_marginal_combos,
    by = c("group", "outcome_var", "outcome")
  ) |>
  dplyr::mutate(
    dplyr::across(
      c(m_used, m_failed, m_skipped_no_compact_fit, m_attempted),
      ~ tidyr::replace_na(.x, 0L)
    ),
```

```r
    first_error = tidyr::replace_na(.data$first_error, ""),
    passed_threshold = .data$m_used >= .data$min_required
  )
marginal_status_df <- marginal_status_raw |>
  dplyr::left_join(
    marginal_completion_summary |>
      dplyr::select(
        group, outcome_var, m_used, m_total, min_required,
        passed_threshold
      ),
    by = c("group", "outcome_var")
  )
write_csv_safely(
  marginal_status_df,
  results_path("discordance_marginal_standardization_status.csv"),
  row_names = FALSE
)

marginal_threshold_failures <- marginal_completion_summary |>
  dplyr::filter(!.data$passed_threshold)
if (nrow(marginal_threshold_failures)) {
  record_discordance_status(
    "marginal_standardization",
    "failed",
    paste0(
      "Marginal standardization did not meet min_marginal_ok_frac = ",
      min_marginal_ok_frac,
      " for: ",
      paste(
        paste0(
          marginal_threshold_failures$group,
          "/",
          marginal_threshold_failures$outcome,
          " (",
          marginal_threshold_failures$m_used,
          "/",
```

```
          marginal_threshold_failures$m_total,
          ")"
        ),
        collapse = "; "
      )
    ),
    "Results/discordance_marginal_standardization_status.csv"
  )
  write_discordance_status()
  stop(
    "Discordance marginal standardization failed the completion threshold: ",
    paste(
      paste0(
        marginal_threshold_failures$group,
        "/",
        marginal_threshold_failures$outcome,
        "=",
        marginal_threshold_failures$m_used,
        "/",
        marginal_threshold_failures$m_total
      ),
      collapse = ", "
    )
  )
}
record_discordance_status(
  "marginal_standardization",
  "completed",
  paste0(
    "Marginal standardization met min_marginal_ok_frac = ",
    min_marginal_ok_frac,
    " for all NIV/IMV group/outcome combinations."
  ),
  "Results/discordance_marginal_standardization_status.csv"
)
```

```r
marginal_imp_curves <- dplyr::bind_rows(marginal_rows)
if (!nrow(marginal_imp_curves)) {
  stop("Discordance marginal standardization produced no predictions.")
}

discordance_marginal_standardized_prob_curves <- marginal_imp_curves |>
  dplyr::group_by(outcome, outcome_var, group, co2_scale, strategy, co2_value) |>
  dplyr::summarise(
    p = mean(p_imp, na.rm = TRUE),
    p_LCL = stats::quantile(p_imp, 0.025, na.rm = TRUE, names = FALSE),
    p_UCL = stats::quantile(p_imp, 0.975, na.rm = TRUE, names = FALSE),
    m_used = dplyr::n_distinct(imp),
    n_standardized = stats::median(n_standardized, na.rm = TRUE),
    standardization_sample_cap = standardization_sample_cap,
    .groups = "drop"
  )
write_csv_safely(
  discordance_marginal_standardized_prob_curves,
  results_path("discordance_marginal_standardized_prob_curves.csv"),
  row_names = FALSE
)

current_spline_prob_curves_for_plot <- dplyr::bind_rows(
  abg_prob_curves_conditional_xref |>
    dplyr::filter(.data$outcome %in% discordance_outcome_vars) |>
    dplyr::mutate(
      outcome_var = .data$outcome,
      outcome = unname(discordance_outcome_labels[.data$outcome]),
      strategy = "current_modality_reference",
      co2_scale = "paco2",
      co2_value = .data$paco2
    ),
  vbg_prob_curves_conditional_xref |>
    dplyr::filter(.data$outcome %in% discordance_outcome_vars) |>
    dplyr::mutate(
      outcome_var = .data$outcome,
```

```r
      outcome = unname(discordance_outcome_labels[.data$outcome]),
      strategy = "current_modality_reference",
      co2_scale = "vbg_co2",
      co2_value = .data$vbg_co2
    )
) |>
  dplyr::select(outcome, outcome_var, group, strategy, co2_scale, co2_value, p, p_LCL, p_UCL)

standardization_curves_for_plot <- dplyr::bind_rows(
  current_spline_prob_curves_for_plot,
  discordance_xref_common_prob_curves |>
    dplyr::select(outcome, outcome_var, group, strategy, co2_scale, co2_value, p, p_LCL, p_UCL),
  discordance_marginal_standardized_prob_curves |>
    dplyr::select(outcome, outcome_var, group, strategy, co2_scale, co2_value, p, p_LCL, p_UCL)
) |>
  dplyr::mutate(
    strategy = factor(
      strategy,
      levels = c("current_modality_reference", "common_reference", "marginal_standardized"),
      labels = c("Current reference", "Common reference", "Marginal standardized")
    )
  )

representative_prob <- function(curves, strategy_name, group, outcome_var, level) {
  strategy_i <- strategy_name
  group_i <- group
  outcome_i <- outcome_var
  co2_var <- if (identical(group, "ABG")) "paco2" else "vbg_co2"
  target <- co2_level_targets(group)[[level]]
  df <- curves |>
    dplyr::filter(
      as.character(.data$strategy) == .env$strategy_i,
      .data$group == .env$group_i,
      .data$outcome_var == .env$outcome_i
    )
  nearest_curve_value(df, "co2_value", target, "p")
```

```r
}

standardization_summary_rows <- tidyr::crossing(
  outcome_var = discordance_outcome_vars,
  co2_level = c("Low", "Normal", "High")
) |>
  dplyr::mutate(outcome = unname(discordance_outcome_labels[outcome_var])) |>
  dplyr::rowwise() |>
  dplyr::mutate(
    gap_current_ref = representative_prob(
      standardization_curves_for_plot,
      "Current reference",
      "ABG",
      outcome_var,
      co2_level
    ) - representative_prob(
      standardization_curves_for_plot,
      "Current reference",
      "VBG",
      outcome_var,
      co2_level
    ),
    gap_common_ref = representative_prob(
      standardization_curves_for_plot,
      "Common reference",
      "ABG",
      outcome_var,
      co2_level
    ) - representative_prob(
      standardization_curves_for_plot,
      "Common reference",
      "VBG",
      outcome_var,
      co2_level
    ),
    gap_marginal_standardized = representative_prob(
```

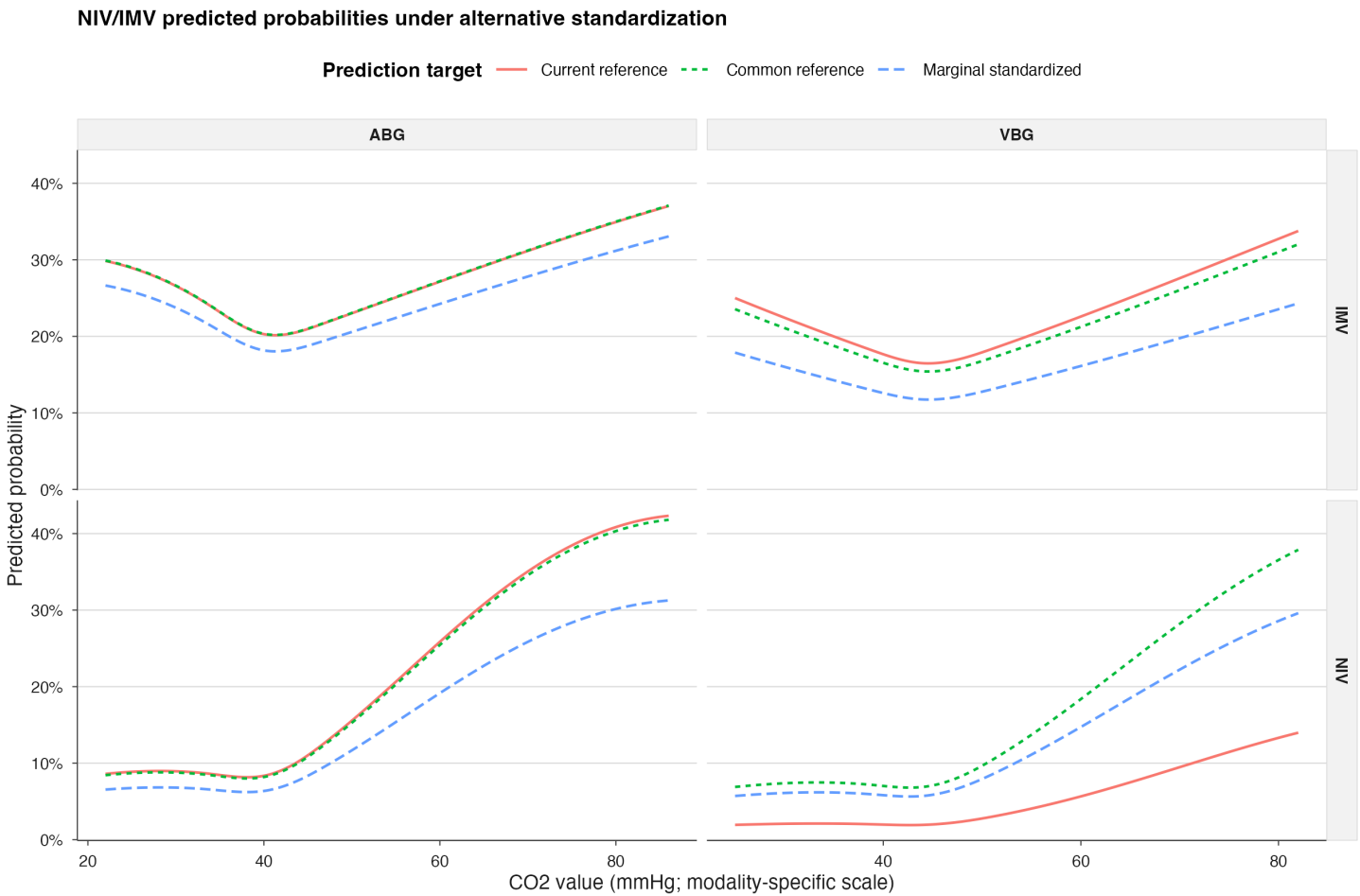
```
    standardization_curves_for_plot,
    "Marginal standardized",
    "ABG",
    outcome_var,
    co2_level
  ) - representative_prob(
    standardization_curves_for_plot,
    "Marginal standardized",
    "VBG",
    outcome_var,
    co2_level
  ),
  standardization_sample_cap = standardization_sample_cap,
  interpretation = dplyr::case_when(
    is.finite(gap_current_ref) &
      is.finite(gap_common_ref) &
      abs(gap_common_ref) <= 0.75 * abs(gap_current_ref) ~ "Common reference shrinks the current probability gap.",
    is.finite(gap_current_ref) &
      is.finite(gap_marginal_standardized) &
      abs(gap_marginal_standardized) <= 0.75 * abs(gap_current_ref) ~ "Marginal standardization shrinks the current
       ↪  probability gap.",
    TRUE ~ "Standardization did not materially shrink the current probability gap."
  )
) |>
dplyr::ungroup() |>
dplyr::left_join(
  marginal_completion_summary |>
    dplyr::group_by(outcome_var, outcome) |>
    dplyr::summarise(
      m_used = min(.data$m_used, na.rm = TRUE),
      m_total = max(.data$m_total, na.rm = TRUE),
      min_required = max(.data$min_required, na.rm = TRUE),
      .groups = "drop"
    ),
  by = c("outcome_var", "outcome")
) |>
```

```r
  dplyr::select(
    outcome, co2_level, gap_current_ref, gap_common_ref,
    gap_marginal_standardized, standardization_sample_cap,
    m_used, m_total, min_required, interpretation
  )

discordance_standardization_summary <- standardization_summary_rows
write_csv_safely(
  discordance_standardization_summary,
  results_path("discordance_standardization_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "standardization",
  "completed",
  paste0(
    "Common-reference and marginal-standardized curves computed; marginal cap = ",
    standardization_sample_cap,
    "; min_marginal_ok_frac = ",
    min_marginal_ok_frac,
    "."
  ),
  "Results/discordance_xref_common_prob_curves.csv; Results/discordance_marginal_standardized_prob_curves.csv;
   ↪  Results/discordance_standardization_summary.csv; Results/discordance_marginal_standardization_status.csv"
)

discordance_probability_standardization_plot <- standardization_curves_for_plot |>
  ggplot2::ggplot(ggplot2::aes(x = co2_value, y = p, color = strategy, linetype = strategy)) +
  ggplot2::geom_line(linewidth = 0.55) +
  ggplot2::facet_grid(outcome ~ group, scales = "free_x") +
  ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  ggplot2::labs(
    title = "NIV/IMV predicted probabilities under alternative standardization",
    x = "CO2 value (mmHg; modality-specific scale)",
    y = "Predicted probability",
    color = "Prediction target",
```

```r
    linetype = "Prediction target"
  ) +
  theme_pub(base_size = 9)

discordance_standardization_png <- results_path(
  "figs",
  "discordance_probability_standardization_niv_imv.png"
)
discordance_standardization_pdf <- results_path(
  "figs",
  "discordance_probability_standardization_niv_imv.pdf"
)
save_diag_plot(discordance_probability_standardization_plot, discordance_standardization_png, width = 8.5, height =
 ↪  5.6)
save_diag_plot(discordance_probability_standardization_plot, discordance_standardization_pdf, width = 8.5, height =
 ↪  5.6)
register_plot_file("discordance_probability_standardization_niv_imv", discordance_standardization_png)
```

[1] TRUE

```r
render_inline_graphic(discordance_standardization_png)
```




```r
## Noncollapsibility and scale checks
discordance_scale_summary <- tidyr::crossing(
  outcome_var = discordance_outcome_vars,
  contrast = c("Low vs normal", "High vs normal")
) |>
  dplyr::mutate(
    outcome = unname(discordance_outcome_labels[outcome_var]),
    category = dplyr::if_else(contrast == "Low vs normal", "Low", "High")
  ) |>
```

```r
  dplyr::rowwise() |>
  dplyr::mutate(
    abg_or = cat_or_value("ABG", outcome, contrast),
    vbg_or = cat_or_value("VBG", outcome, contrast),
    abg_prob = cat_prob_value("ABG", outcome, category),
    vbg_prob = cat_prob_value("VBG", outcome, category),
    abg_prob_ref = cat_prob_value("ABG", outcome, "Normal"),
    vbg_prob_ref = cat_prob_value("VBG", outcome, "Normal"),
    abg_rd = abg_prob - abg_prob_ref,
    vbg_rd = vbg_prob - vbg_prob_ref,
    abg_rr = ifelse(is.finite(abg_prob_ref) & abg_prob_ref > 0, abg_prob / abg_prob_ref, NA_real_),
    vbg_rr = ifelse(is.finite(vbg_prob_ref) & vbg_prob_ref > 0, vbg_prob / vbg_prob_ref, NA_real_),
    comment = dplyr::case_when(
      is.finite(abg_or) & is.finite(vbg_or) &
        abs(log(abg_or) - log(vbg_or)) < abs(abg_rd - vbg_rd) ~
        "Relative odds are closer than absolute risk differences.",
      TRUE ~ "Scale check does not show clearly closer ORs."
    )
  ) |>
  dplyr::ungroup() |>
  dplyr::select(
    outcome, contrast, abg_or, vbg_or, abg_prob, vbg_prob,
    abg_rd, vbg_rd, abg_rr, vbg_rr, comment
  )
write_csv_safely(
  discordance_scale_summary,
  results_path("discordance_scale_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "scale_noncollapsibility",
  "completed",
  "OR, risk-difference, risk-ratio, and predicted-probability scale summaries were computed.",
  "Results/discordance_scale_summary.csv"
)
```

```r
## Missingness and procedure-capture diagnostics
outcome_source_fields <- unique(c(
  "imv_proc",
  "niv_proc",
  candidate_cols(
    names(subset_data_raw),
    "(imv|niv).*(date|time|timestamp|source|code|proc|procedure)",
    exclude = character(),
    max_n = 30
  )
))
outcome_source_fields <- outcome_source_fields[outcome_source_fields %in% names(subset_data_raw)]

if (!length(outcome_source_fields)) {
  discordance_missingness_outcome_fields <- tibble::tibble(
    field = NA_character_,
    component_status = "skipped",
    reason = "unavailable_field"
  )
  record_discordance_status(
    "outcome_field_missingness",
    "skipped",
    "No NIV/IMV procedure source fields were available.",
    "Results/discordance_missingness_outcome_fields.csv",
    optional = TRUE
  )
} else {
  discordance_missingness_outcome_fields <- purrr::map_dfr(outcome_source_fields, function(field) {
    x <- subset_data_raw[[field]]
    tibble::tibble(
      field = field,
      missing_overall = mean(is.na(x)),
      missing_abg_tested = mean(is.na(x[subset_data$has_abg == 1])),
      missing_vbg_tested = mean(is.na(x[subset_data$has_vbg == 1])),
      n_overall = length(x),
      n_abg_tested = sum(subset_data$has_abg == 1, na.rm = TRUE),
```

```
      n_vbg_tested = sum(subset_data$has_vbg == 1, na.rm = TRUE),
      component_status = "completed",
      reason = "available_field"
    )
  })
  record_discordance_status(
    "outcome_field_missingness",
    "completed",
    "NIV/IMV outcome-source field missingness was summarized.",
    "Results/discordance_missingness_outcome_fields.csv"
  )
}
write_csv_safely(
  discordance_missingness_outcome_fields,
  results_path("discordance_missingness_outcome_fields.csv"),
  row_names = FALSE
)

proxy_fields <- unique(c(
  "imv_proc",
  "niv_proc",
  candidate_cols(
    names(subset_data),
    "(procedure|proc|cpt|icd|code|dx|charge|document|claim)",
    exclude = c("has_abg", "has_vbg"),
    max_n = 20
  )
))
proxy_fields <- proxy_fields[proxy_fields %in% names(subset_data)]

capture_rows <- list()
for (imp_index in seq_len(imp_n)) {
  d_imp <- get_imp(imp_index)
  for (group in discordance_groups) {
    treat_var <- if (identical(group, "ABG")) "has_abg" else "has_vbg"
    weights_i <- get_mi_weight(group, imp_index)
```

```r
    tested <- d_imp[[treat_var]] == 1
    for (field in proxy_fields) {
      x <- d_imp[[field]]
      if (is.factor(x) || is.character(x)) {
        proxy_value <- as.integer(!is.na(x) & nzchar(trimws(as.character(x))))
      } else {
        proxy_value <- suppressWarnings(as.numeric(x))
      }
      capture_rows[[length(capture_rows) + 1L]] <- tibble::tibble(
        imp = imp_index,
        group = group,
        proxy_field = field,
        proxy_type = ifelse(field %in% c("imv_proc", "niv_proc"), "binary_outcome", "coding_density_proxy"),
        unweighted_mean_tested = mean(proxy_value[tested], na.rm = TRUE),
        weighted_mean_tested = weighted_mean_safe(proxy_value[tested], weights_i[tested]),
        missing_fraction_tested = mean(is.na(proxy_value[tested])),
        n_tested = sum(tested, na.rm = TRUE)
      )
    }
  }
  rm(d_imp)
  invisible(gc(FALSE))
}

discordance_procedure_capture_proxies <- dplyr::bind_rows(capture_rows) |>
  dplyr::group_by(group, proxy_field, proxy_type) |>
  dplyr::summarise(
    unweighted_mean_tested = stats::median(unweighted_mean_tested, na.rm = TRUE),
    weighted_mean_tested = stats::median(weighted_mean_tested, na.rm = TRUE),
    missing_fraction_tested = stats::median(missing_fraction_tested, na.rm = TRUE),
    n_tested = stats::median(n_tested, na.rm = TRUE),
    m_used = dplyr::n_distinct(imp),
    .groups = "drop"
  )
write_csv_safely(
  discordance_procedure_capture_proxies,
```

```
  results_path("discordance_procedure_capture_proxies.csv"),
  row_names = FALSE
)

missing_indicator_vars <- covars_ps[covars_ps %in% names(subset_data_raw)]
missing_indicator_vars <- missing_indicator_vars[
  vapply(missing_indicator_vars, function(v) any(is.na(subset_data_raw[[v]])), logical(1L))
]

if (!length(missing_indicator_vars)) {
  discordance_missing_indicator_balance <- tibble::tibble(
    variable = NA_character_,
    group = NA_character_,
    smd_pre = NA_real_,
    smd_post = NA_real_,
    component_status = "skipped",
    reason = "no_missing_baseline_indicators"
  )
} else {
  miss_bal_rows <- list()
  target_missing <- lapply(missing_indicator_vars, function(v) as.integer(is.na(subset_data_raw[[v]])))
  names(target_missing) <- missing_indicator_vars
  for (imp_index in seq_len(imp_n)) {
    for (group in discordance_groups) {
      treat_var <- if (identical(group, "ABG")) "has_abg" else "has_vbg"
      treat <- subset_data[[treat_var]] == 1
      weights_i <- get_mi_weight(group, imp_index)
      for (v in missing_indicator_vars) {
        x <- target_missing[[v]]
        target_p <- mean(x, na.rm = TRUE)
        target_sd <- sqrt(target_p * (1 - target_p))
        pre <- mean(x[treat], na.rm = TRUE)
        post <- weighted_mean_safe(x[treat], weights_i[treat])
        miss_bal_rows[[length(miss_bal_rows) + 1L]] <- tibble::tibble(
          imp = imp_index,
          variable = v,
```

```
          group = group,
          missing_target = target_p,
          missing_tested = pre,
          missing_weighted = post,
          smd_pre = ifelse(is.finite(target_sd) & target_sd > 0, (pre - target_p) / target_sd, NA_real_),
          smd_post = ifelse(is.finite(target_sd) & target_sd > 0, (post - target_p) / target_sd, NA_real_)
        )
      }
    }
  }
  discordance_missing_indicator_balance <- dplyr::bind_rows(miss_bal_rows) |>
    dplyr::group_by(variable, group) |>
    dplyr::summarise(
      missing_target = stats::median(missing_target, na.rm = TRUE),
      missing_tested = stats::median(missing_tested, na.rm = TRUE),
      missing_weighted = stats::median(missing_weighted, na.rm = TRUE),
      smd_pre = stats::median(smd_pre, na.rm = TRUE),
      smd_post = stats::median(smd_post, na.rm = TRUE),
      .groups = "drop"
    ) |>
    dplyr::mutate(component_status = "completed", reason = "baseline_missingness_indicators_available")
}
write_csv_safely(
  discordance_missing_indicator_balance,
  results_path("discordance_missing_indicator_balance.csv"),
  row_names = FALSE
)

capture_wide <- discordance_procedure_capture_proxies |>
  dplyr::select(group, proxy_field, weighted_mean_tested) |>
  tidyr::pivot_wider(names_from = group, values_from = weighted_mean_tested)

missingness_capture_summary <- tibble::tibble(
  component = c("outcome_source_missingness", "procedure_capture_proxy", "baseline_missing_indicator_balance"),
  max_abs_abg_vbg_difference = c(
    suppressWarnings(max(abs(
```

```
      discordance_missingness_outcome_fields$missing_abg_tested -
        discordance_missingness_outcome_fields$missing_vbg_tested
    ), na.rm = TRUE)),
    suppressWarnings(max(abs(capture_wide$ABG - capture_wide$VBG), na.rm = TRUE)),
    suppressWarnings(max(abs(discordance_missing_indicator_balance$smd_post), na.rm = TRUE))
  ),
  interpretation = c(
    "Difference in missingness among detected NIV/IMV source fields.",
    "Difference in weighted tested-cohort proxy means.",
    "Largest post-weighting SMD among baseline missingness indicators."
  )
) |>
  dplyr::mutate(
    max_abs_abg_vbg_difference = ifelse(is.infinite(max_abs_abg_vbg_difference), NA_real_, max_abs_abg_vbg_difference)
  )
write_csv_safely(
  missingness_capture_summary,
  results_path("discordance_missingness_capture_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "missingness_capture",
  "completed",
  "Procedure-source missingness, capture proxies, and baseline missingness indicator balance were summarized.",
  "Results/discordance_missingness_outcome_fields.csv; Results/discordance_procedure_capture_proxies.csv;
   ↪ Results/discordance_missing_indicator_balance.csv; Results/discordance_missingness_capture_summary.csv"
)

## Sentinel procedure capture checks
sentinel_specs <- tibble::tribble(
  ~sentinel_concept, ~priority_rank, ~matched_pattern,
  "ECG/EKG", 1L, "(ecg|ekg|electrocard)",
  "Chest radiograph", 2L, "(cxr|chest.*(xray|x_ray|radiograph)|xray.*chest|x_ray.*chest)",
  "Transthoracic echocardiography", 3L, "(tte|transthoracic|echo)",
  "CT chest / CT pulmonary angiography", 4L,
   ↪ "(ctc|ct_chest|ct.*chest|pulmonary.*angio|ctpa|cta.*chest|ct.*pulmonary)",
```

```
  "Central venous catheter placement", 5L, "(central.*ven|central.*line|cvc|cvp)",
  "Urinary catheter placement", 6L, "(urinary.*catheter|foley)",
  "Blood transfusion", 7L, "(transfusion|blood.*transfus|rbc|prbc)",
  "Hemodialysis initiation", 8L, "(hemodialysis|dialysis|renal.*replacement|crrt)",
  "Bronchoscopy", 9L, "(bronch)"
)

sentinel_excluded_fields <- c(
  "imv_proc", "niv_proc", "vent_proc",
  "meas_arterial_gas_proc", "meas_venous_o2_proc"
)
sentinel_candidate_names <- union(names(subset_data), names(subset_data_raw))
sentinel_binary_names <- setdiff(
  sentinel_candidate_names,
  grep("(date|time|timestamp|datetime)|_(first|last)_date", sentinel_candidate_names,
       value = TRUE, ignore.case = TRUE, perl = TRUE)
)
sentinel_binary_names <- setdiff(sentinel_binary_names, sentinel_excluded_fields)
sentinel_binary_names <- sentinel_binary_names[
  !grepl("(abg|vbg|blood_gas|gas|paco2|co2|po2|o2sat|hco3|ph|lactate)",
         sentinel_binary_names, ignore.case = TRUE, perl = TRUE)
]

sentinel_indicator <- function(x) {
  if (is.factor(x) || is.character(x)) {
    raw <- trimws(as.character(x))
    raw_lower <- tolower(raw)
    num <- suppressWarnings(as.numeric(raw))
    if (all(is.na(num[!is.na(raw) & nzchar(raw)]))) {
      return(as.integer(!is.na(raw) & nzchar(raw) &
                        !raw_lower %in% c("0", "no", "false", "absent", "none")))
    }
    return(as.integer(is.finite(num) & num > 0))
  }
  num <- suppressWarnings(as.numeric(x))
  as.integer(is.finite(num) & num > 0)
```

```r
}

sentinel_source_vector <- function(field, d_imp = NULL) {
  if (!is.null(d_imp) && field %in% names(d_imp)) {
    x <- d_imp[[field]]
  } else if (field %in% names(subset_data)) {
    x <- subset_data[[field]]
  } else if (field %in% names(subset_data_raw)) {
    x <- subset_data_raw[[field]]
  } else {
    stop("Sentinel field not found in analysis or raw data: ", field, call. = FALSE)
  }
  if (length(x) != nrow(subset_data)) {
    stop("Sentinel field has unexpected row count: ", field, call. = FALSE)
  }
  x
}

sentinel_date_stems <- function(binary_field) {
  stripped_stem <- sub("(_0|_dur|_duration)$", "", binary_field, perl = TRUE)
  stems <- unique(c(binary_field, stripped_stem))
  stems[nzchar(stems)]
}

sentinel_date_field_for <- function(binary_field, pattern) {
  force(pattern)
  raw_names <- names(subset_data_raw)
  stems <- sentinel_date_stems(binary_field)
  direct_candidates <- unique(unlist(
    lapply(stems, function(stem) {
      c(
        paste0(stem, "_first_date"),
        paste0(stem, "_last_date")
      )
    }),
    use.names = FALSE
```

```
  ))
  date_like_names <- raw_names[
    grepl("(date|time|timestamp|datetime)", raw_names, ignore.case = TRUE, perl = TRUE)
  ]
  stem_timestamp_candidates <- unique(unlist(
    lapply(stems, function(stem) {
      date_like_names[startsWith(date_like_names, paste0(stem, "_"))]
    }),
    use.names = FALSE
  ))
  hits <- unique(c(direct_candidates, stem_timestamp_candidates))
  hits <- setdiff(hits, binary_field)
  hits <- hits[grepl("(date|time|timestamp|datetime)", hits, ignore.case = TRUE, perl = TRUE)]
  hits <- hits[hits %in% raw_names]
  if (length(hits)) hits[[1L]] else NA_character_
}

sentinel_inventory_rows <- purrr::map_dfr(seq_len(nrow(sentinel_specs)), function(idx) {
  spec <- sentinel_specs[idx, ]
  matched_fields <- grep(
    spec$matched_pattern,
    sentinel_binary_names,
    value = TRUE,
    ignore.case = TRUE,
    perl = TRUE
  )
  matched_fields <- unique(matched_fields)
  if (!length(matched_fields)) {
    return(tibble::tibble(
      sentinel_concept = spec$sentinel_concept,
      candidate_field = NA_character_,
      field_role = "binary_indicator",
      matched_pattern = spec$matched_pattern,
      present = FALSE,
      class = NA_character_,
      n_nonmissing = 0L,
```

```
    prevalence_overall = NA_real_,
    prevalence_abg_tested = NA_real_,
    prevalence_vbg_tested = NA_real_,
    events_abg_tested = 0L,
    events_vbg_tested = 0L,
    date_field = NA_character_,
    date_missing_abg_tested = NA_real_,
    date_missing_vbg_tested = NA_real_,
    priority_rank = spec$priority_rank,
    selection_reason = "no_matching_binary_field"
  ))
}
purrr::map_dfr(matched_fields, function(field) {
  source_vec <- sentinel_source_vector(field)
  y <- sentinel_indicator(source_vec)
  date_field <- sentinel_date_field_for(field, spec$matched_pattern)
  date_vec <- if (!is.na(date_field)) subset_data_raw[[date_field]] else rep(NA, nrow(subset_data_raw))
  abg_tested <- subset_data$has_abg == 1
  vbg_tested <- subset_data$has_vbg == 1
  tibble::tibble(
    sentinel_concept = spec$sentinel_concept,
    candidate_field = field,
    field_role = ifelse(field %in% names(subset_data), "analysis_indicator", "raw_indicator"),
    matched_pattern = spec$matched_pattern,
    present = TRUE,
    class = paste(class(source_vec), collapse = "/"),
    n_nonmissing = sum(!is.na(source_vec)),
    prevalence_overall = mean(y == 1, na.rm = TRUE),
    prevalence_abg_tested = mean(y[abg_tested] == 1, na.rm = TRUE),
    prevalence_vbg_tested = mean(y[vbg_tested] == 1, na.rm = TRUE),
    events_abg_tested = sum(y[abg_tested] == 1, na.rm = TRUE),
    events_vbg_tested = sum(y[vbg_tested] == 1, na.rm = TRUE),
    date_field = ifelse(is.na(date_field), "", date_field),
    date_missing_abg_tested = ifelse(
      is.na(date_field) || !any(abg_tested & y == 1, na.rm = TRUE),
      NA_real_,
```

```
        mean(is.na(date_vec[abg_tested & y == 1]))
      ),
      date_missing_vbg_tested = ifelse(
        is.na(date_field) || !any(vbg_tested & y == 1, na.rm = TRUE),
        NA_real_,
        mean(is.na(date_vec[vbg_tested & y == 1]))
      ),
      priority_rank = spec$priority_rank,
      selection_reason = "candidate_detected"
    )
  })
})

sentinel_min_events <- if (IS_PILOT) 20L else 100L
sentinel_selection_scored <- sentinel_inventory_rows |>
  dplyr::rowwise() |>
  dplyr::mutate(
    prevalence_for_selection = suppressWarnings(max(
      c(prevalence_abg_tested, prevalence_vbg_tested),
      na.rm = TRUE
    )),
    event_count_for_selection = suppressWarnings(max(
      c(events_abg_tested, events_vbg_tested),
      na.rm = TRUE
    ))
  ) |>
  dplyr::ungroup() |>
  dplyr::mutate(
    prevalence_for_selection = ifelse(is.infinite(prevalence_for_selection), NA_real_, prevalence_for_selection),
    event_count_for_selection = ifelse(is.infinite(event_count_for_selection), 0L, event_count_for_selection),
    eligible = .data$present &
      is.finite(.data$prevalence_for_selection) &
      .data$prevalence_for_selection >= 0.01 &
      .data$prevalence_for_selection <= 0.60 &
      .data$event_count_for_selection >= sentinel_min_events,
    selection_reason = dplyr::case_when(
```

```
      !.data$present ~ "no_matching_binary_field",
      !is.finite(.data$prevalence_for_selection) ~ "prevalence_unavailable",
      .data$prevalence_for_selection < 0.01 ~ "prevalence_below_1_percent",
      .data$prevalence_for_selection > 0.60 ~ "prevalence_above_60_percent",
      .data$event_count_for_selection < sentinel_min_events ~ "insufficient_events",
      TRUE ~ "eligible"
    ),
    sentinel_key = paste(.data$sentinel_concept, .data$candidate_field, sep = "::")
  )

sentinel_best_by_concept <- sentinel_selection_scored |>
  dplyr::arrange(.data$priority_rank, dplyr::desc(.data$eligible), dplyr::desc(.data$prevalence_for_selection)) |>
  dplyr::group_by(.data$sentinel_concept) |>
  dplyr::slice_head(n = 1) |>
  dplyr::ungroup()

sentinel_selected_keys <- sentinel_best_by_concept |>
  dplyr::filter(.data$eligible) |>
  dplyr::arrange(.data$priority_rank) |>
  dplyr::slice_head(n = 4) |>
  dplyr::pull(.data$sentinel_key)

discordance_sentinel_procedure_inventory <- sentinel_inventory_rows |>
  dplyr::select(
    sentinel_concept, candidate_field, field_role, matched_pattern, present,
    class, n_nonmissing, prevalence_overall, prevalence_abg_tested,
    prevalence_vbg_tested, date_field, date_missing_abg_tested,
    date_missing_vbg_tested, selection_reason, dplyr::everything()
  )
write_csv_safely(
  discordance_sentinel_procedure_inventory,
  results_path("discordance_sentinel_procedure_inventory.csv"),
  row_names = FALSE
)

discordance_sentinel_procedure_selection <- sentinel_best_by_concept |>
```

```
  dplyr::mutate(
    selected = .data$sentinel_key %in% sentinel_selected_keys,
    selection_status = dplyr::case_when(
      .data$selected ~ "selected",
      .data$eligible ~ "not_selected_lower_priority",
      TRUE ~ paste0("not_selected_", .data$selection_reason)
    ),
    binary_field = dplyr::if_else(.data$present, .data$candidate_field, "")
  ) |>
  dplyr::select(
    sentinel_concept, binary_field, date_field, priority_rank, selected,
    prevalence_for_selection, event_count_for_selection,
    selection_status, selection_reason
  )
write_csv_safely(
  discordance_sentinel_procedure_selection,
  results_path("discordance_sentinel_procedure_selection.csv"),
  row_names = FALSE
)

selected_sentinels <- discordance_sentinel_procedure_selection |>
  dplyr::filter(.data$selected)
sentinel_balance_rows <- list()
if (nrow(selected_sentinels)) {
  for (imp_index in seq_len(imp_n)) {
    d_imp <- get_imp(imp_index)
    for (row_idx in seq_len(nrow(selected_sentinels))) {
      sentinel_i <- selected_sentinels[row_idx, ]
      binary_field <- sentinel_i$binary_field
      date_field <- sentinel_i$date_field
      y <- sentinel_indicator(sentinel_source_vector(binary_field, d_imp))
      date_vec <- if (nzchar(date_field) && date_field %in% names(subset_data_raw)) {
        subset_data_raw[[date_field]]
      } else {
        rep(NA, nrow(subset_data_raw))
      }
```

```r
      for (group in discordance_groups) {
        treat_var <- if (identical(group, "ABG")) "has_abg" else "has_vbg"
        weights_i <- get_mi_weight(group, imp_index)
        tested <- d_imp[[treat_var]] == 1
        event_rows <- tested & y == 1
        sentinel_balance_rows[[length(sentinel_balance_rows) + 1L]] <- tibble::tibble(
          sentinel_concept = sentinel_i$sentinel_concept,
          group = group,
          imp = imp_index,
          n_tested = sum(tested, na.rm = TRUE),
          events = sum(event_rows, na.rm = TRUE),
          unweighted_rate = mean(y[tested] == 1, na.rm = TRUE),
          weighted_rate = weighted_mean_safe(y[tested], weights_i[tested]),
          weighted_n = sum(weights_i[tested], na.rm = TRUE),
          ess = weighted_ess(weights_i[tested]),
          date_missing_fraction = ifelse(
            nzchar(date_field) && any(event_rows, na.rm = TRUE),
            mean(is.na(date_vec[event_rows])),
            NA_real_
          )
        )
      }
    }
    rm(d_imp)
    invisible(gc(FALSE))
  }
  discordance_sentinel_procedure_balance <- dplyr::bind_rows(sentinel_balance_rows)
} else {
  discordance_sentinel_procedure_balance <- tibble::tibble(
    sentinel_concept = NA_character_,
    group = NA_character_,
    imp = NA_integer_,
    n_tested = NA_integer_,
    events = NA_integer_,
    unweighted_rate = NA_real_,
    weighted_rate = NA_real_,
```

```r
    weighted_n = NA_real_,
    ess = NA_real_,
    date_missing_fraction = NA_real_,
    component_status = "skipped",
    reason = "no_sentinel_procedure_selected"
  )
}
write_csv_safely(
  discordance_sentinel_procedure_balance,
  results_path("discordance_sentinel_procedure_balance.csv"),
  row_names = FALSE
)

sentinel_metric_summary <- if (nrow(selected_sentinels)) {
  discordance_sentinel_procedure_balance |>
    dplyr::group_by(.data$sentinel_concept, .data$group) |>
    dplyr::summarise(
      weighted_rate = stats::median(.data$weighted_rate, na.rm = TRUE),
      date_missing_fraction = stats::median(.data$date_missing_fraction, na.rm = TRUE),
      m_used = dplyr::n_distinct(.data$imp),
      .groups = "drop"
    ) |>
    tidyr::pivot_wider(
      names_from = group,
      values_from = c(weighted_rate, date_missing_fraction, m_used)
    ) |>
    dplyr::transmute(
      sentinel_concept,
      abg_weighted_rate = .data$weighted_rate_ABG,
      vbg_weighted_rate = .data$weighted_rate_VBG,
      weighted_abs_diff = .data$abg_weighted_rate - .data$vbg_weighted_rate,
      weighted_relative_diff = safe_relative_gap(.data$abg_weighted_rate, .data$vbg_weighted_rate),
      date_missing_abs_diff = abs(.data$date_missing_fraction_ABG - .data$date_missing_fraction_VBG),
      m_used = pmin(.data$m_used_ABG, .data$m_used_VBG, na.rm = TRUE),
      m_used = ifelse(is.infinite(.data$m_used), NA_integer_, as.integer(.data$m_used))
    )
```

```r
} else {
  tibble::tibble(
    sentinel_concept = character(),
    abg_weighted_rate = numeric(),
    vbg_weighted_rate = numeric(),
    weighted_abs_diff = numeric(),
    weighted_relative_diff = numeric(),
    date_missing_abs_diff = numeric(),
    m_used = integer()
  )
}

discordance_sentinel_procedure_summary <- discordance_sentinel_procedure_selection |>
  dplyr::select(sentinel_concept, selected, selection_status, selection_reason) |>
  dplyr::left_join(sentinel_metric_summary, by = "sentinel_concept") |>
  dplyr::mutate(
    interpretation = dplyr::case_when(
      !.data$selected ~ .data$selection_status,
      is.finite(.data$date_missing_abs_diff) & .data$date_missing_abs_diff > 0.05 ~
        "Sentinel procedure timestamp completeness differs by >5 percentage points.",
      is.finite(.data$weighted_abs_diff) & abs(.data$weighted_abs_diff) > 0.05 ~
        "Sentinel procedure weighted capture rates differ by >5 percentage points.",
      TRUE ~ "Selected sentinel does not show a large ABG-VBG capture/completeness gap."
    )
  ) |>
  dplyr::select(
    sentinel_concept, selected, abg_weighted_rate, vbg_weighted_rate,
    weighted_abs_diff, weighted_relative_diff, date_missing_abs_diff,
    m_used, interpretation
  )
write_csv_safely(
  discordance_sentinel_procedure_summary,
  results_path("discordance_sentinel_procedure_summary.csv"),
  row_names = FALSE
)
```

```r
n_sentinel_selected <- sum(discordance_sentinel_procedure_selection$selected %in% TRUE, na.rm = TRUE)
record_discordance_status(
  "sentinel_procedure_capture",
  if (n_sentinel_selected >= 2L) "completed" else if (n_sentinel_selected > 0L) "partial" else "skipped",
  paste0(
    "Selected ",
    n_sentinel_selected,
    " sentinel procedure concept(s) using prevalence and event-count rules."
  ),
  "Results/discordance_sentinel_procedure_inventory.csv; Results/discordance_sentinel_procedure_selection.csv;
   ↪  Results/discordance_sentinel_procedure_balance.csv; Results/discordance_sentinel_procedure_summary.csv",
  optional = n_sentinel_selected < 2L
)

## Support, tail, and positivity diagnostics
make_quantile_bin <- function(x) {
  probs <- c(0, 0.02, 0.25, 0.75, 0.98, 1)
  cuts <- unique(stats::quantile(x, probs = probs, na.rm = TRUE, names = FALSE))
  if (length(cuts) < 3L) return(factor(rep(NA_character_, length(x))))
  cut(x, breaks = cuts, include.lowest = TRUE, ordered_result = TRUE)
}

tail_support_rows <- list()
for (imp_index in seq_len(imp_n)) {
  d_imp <- get_imp(imp_index)
  for (group in discordance_groups) {
    co2_var <- if (identical(group, "ABG")) "paco2" else "vbg_co2"
    treat_var <- if (identical(group, "ABG")) "has_abg" else "has_vbg"
    low_cut <- if (identical(group, "ABG")) ABG_CO2_LOW else VBG_CO2_LOW
    high_cut <- if (identical(group, "ABG")) ABG_CO2_HIGH else VBG_CO2_HIGH
    weights_i <- get_mi_weight(group, imp_index)
    tested <- d_imp[[treat_var]] == 1 & is.finite(suppressWarnings(as.numeric(d_imp[[co2_var]])))
    co2_x <- suppressWarnings(as.numeric(d_imp[[co2_var]]))
    clinical_bin <- make_co2_cat3(co2_x, low_cut, high_cut)
    clinical_bin <- factor(clinical_bin, levels = c("Low", "Normal", "High"), ordered = TRUE)
    quantile_bin <- rep(NA, nrow(d_imp))
```

```r
quantile_bin[tested] <- as.character(make_quantile_bin(co2_x[tested]))
bin_df <- dplyr::bind_rows(
  tibble::tibble(bin_type = "clinical_cat3", bin = as.character(clinical_bin)),
  tibble::tibble(bin_type = "quantile_support", bin = as.character(quantile_bin))
) |>
  dplyr::mutate(row_id = rep(seq_len(nrow(d_imp)), times = 2L))

for (outcome_var in discordance_outcome_vars) {
  y <- to01(d_imp[[outcome_var]])
  for (bin_type_i in unique(bin_df$bin_type)) {
    rows_i <- bin_df |>
      dplyr::filter(.data$bin_type == bin_type_i) |>
      dplyr::mutate(
        tested = tested[row_id],
        y = y[row_id],
        w = weights_i[row_id],
        co2 = co2_x[row_id]
      ) |>
      dplyr::filter(.data$tested, !is.na(.data$bin))
    if (!nrow(rows_i)) next
    tail_support_rows[[length(tail_support_rows) + 1L]] <- rows_i |>
      dplyr::group_by(bin_type, bin) |>
      dplyr::summarise(
        n_unweighted = dplyr::n(),
        co2_min = min(co2, na.rm = TRUE),
        co2_max = max(co2, na.rm = TRUE),
        weighted_n = sum(w, na.rm = TRUE),
        weighted_events = sum(w * y, na.rm = TRUE),
        weighted_event_rate = ifelse(weighted_n > 0, weighted_events / weighted_n, NA_real_),
        ess = weighted_ess(w),
        .groups = "drop"
      ) |>
      dplyr::mutate(
        imp = imp_index,
        group = group,
        outcome = discordance_outcome_labels[[outcome_var]],
```

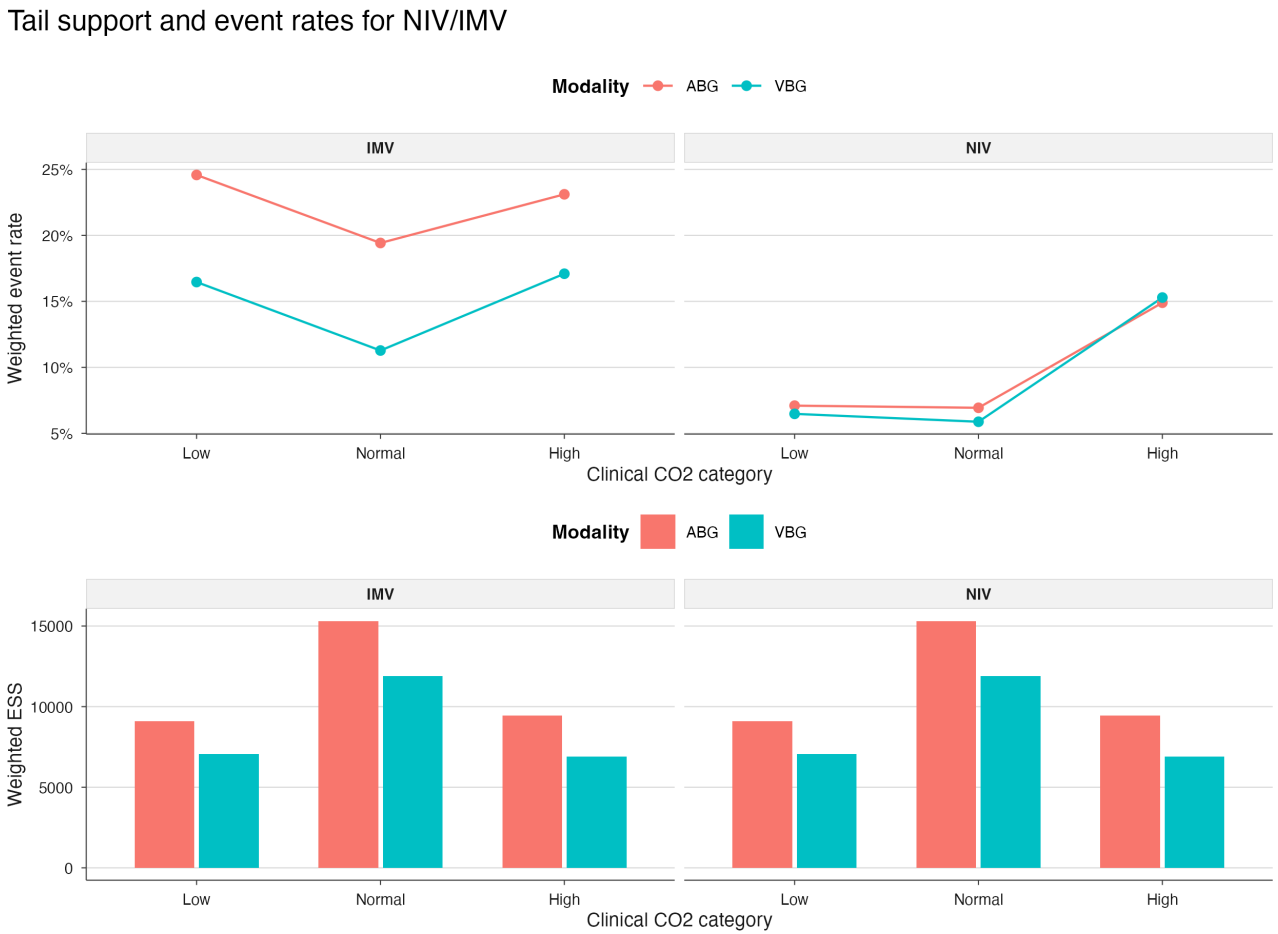
```
            outcome_var = outcome_var
          )
      }
    }
  }
  rm(d_imp)
  invisible(gc(FALSE))
}

discordance_tail_support_by_bin <- dplyr::bind_rows(tail_support_rows) |>
  dplyr::group_by(group, outcome, outcome_var, bin_type, bin) |>
  dplyr::summarise(
    n_unweighted = stats::median(n_unweighted, na.rm = TRUE),
    co2_min = stats::median(co2_min, na.rm = TRUE),
    co2_max = stats::median(co2_max, na.rm = TRUE),
    weighted_n = stats::median(weighted_n, na.rm = TRUE),
    weighted_events = stats::median(weighted_events, na.rm = TRUE),
    weighted_event_rate = stats::median(weighted_event_rate, na.rm = TRUE),
    ess = stats::median(ess, na.rm = TRUE),
    m_used = dplyr::n_distinct(imp),
    .groups = "drop"
  )
write_csv_safely(
  discordance_tail_support_by_bin,
  results_path("discordance_tail_support_by_bin.csv"),
  row_names = FALSE
)

discordance_tail_support_summary <- discordance_tail_support_by_bin |>
  dplyr::group_by(group, outcome, bin_type) |>
  dplyr::summarise(
    min_ess = min(ess, na.rm = TRUE),
    min_weighted_events = min(weighted_events, na.rm = TRUE),
    max_weighted_event_rate = max(weighted_event_rate, na.rm = TRUE),
    n_bins = dplyr::n(),
    support_flag = any(ess < 100 | weighted_events < 10, na.rm = TRUE),
```

```r
    .groups = "drop"
  )
write_csv_safely(
  discordance_tail_support_summary,
  results_path("discordance_tail_support_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "tail_support",
  "completed",
  "Weighted support, events, event rates, and ESS were summarized by clinical and quantile CO2 bins.",
  "Results/discordance_tail_support_summary.csv; Results/discordance_tail_support_by_bin.csv"
)

tail_clinical <- discordance_tail_support_by_bin |>
  dplyr::filter(.data$bin_type == "clinical_cat3") |>
  dplyr::mutate(bin = factor(bin, levels = c("Low", "Normal", "High")))

tail_rate_plot <- tail_clinical |>
  ggplot2::ggplot(ggplot2::aes(x = bin, y = weighted_event_rate, color = group, group = group)) +
  ggplot2::geom_line(linewidth = 0.5) +
  ggplot2::geom_point(size = 1.8) +
  ggplot2::facet_wrap(~ outcome, nrow = 1) +
  ggplot2::scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  ggplot2::labs(x = "Clinical CO2 category", y = "Weighted event rate", color = "Modality") +
  theme_pub(base_size = 9)

tail_ess_plot <- tail_clinical |>
  ggplot2::ggplot(ggplot2::aes(x = bin, y = ess, fill = group)) +
  ggplot2::geom_col(position = ggplot2::position_dodge(width = 0.7), width = 0.65) +
  ggplot2::facet_wrap(~ outcome, nrow = 1) +
  ggplot2::labs(x = "Clinical CO2 category", y = "Weighted ESS", fill = "Modality") +
  theme_pub(base_size = 9)

discordance_tail_support_plot <- tail_rate_plot / tail_ess_plot +
  patchwork::plot_annotation(title = "Tail support and event rates for NIV/IMV")
```

```r
discordance_tail_png <- results_path("figs", "discordance_tail_support_niv_imv.png")
discordance_tail_pdf <- results_path("figs", "discordance_tail_support_niv_imv.pdf")
save_diag_plot(discordance_tail_support_plot, discordance_tail_png, width = 8.5, height = 6.2)
save_diag_plot(discordance_tail_support_plot, discordance_tail_pdf, width = 8.5, height = 6.2)
register_plot_file("discordance_tail_support_niv_imv", discordance_tail_png)
```

[1] TRUE

```r
render_inline_graphic(discordance_tail_png)
```



```r
## Setting and subgroup sensitivity analyses
site_candidates <- candidate_cols(
  names(subset_data),
  "(^site$|facility|hospital|health_system|institution)",
  exclude = c("location"),
  max_n = 10
)
site_field <- NA_character_
if (length(site_candidates)) {
  site_levels <- vapply(site_candidates, function(v) dplyr::n_distinct(subset_data[[v]], na.rm = TRUE), integer(1L))
  feasible <- site_candidates[site_levels >= 2L & site_levels <= 50L]
  if (length(feasible)) site_field <- feasible[[1L]]
}
setting_vars <- c("encounter_type", "location", site_field)
setting_vars <- setting_vars[!is.na(setting_vars) & setting_vars %in% names(subset_data)]

setting_rows <- list()
for (imp_index in seq_len(imp_n)) {
  d_imp <- get_imp(imp_index)
  for (group in discordance_groups) {
    treat_var <- if (identical(group, "ABG")) "has_abg" else "has_vbg"
    weights_i <- get_mi_weight(group, imp_index)
    tested <- d_imp[[treat_var]] == 1
    for (setting_var in setting_vars) {
      for (outcome_var in discordance_outcome_vars) {
        setting_rows[[length(setting_rows) + 1L]] <- tibble::tibble(
          setting_level = as.character(d_imp[[setting_var]]),
          y = to01(d_imp[[outcome_var]]),
          w = weights_i,
          tested = tested
        ) |>
          dplyr::filter(.data$tested, !is.na(.data$setting_level), nzchar(.data$setting_level)) |>
          dplyr::group_by(setting_level) |>
          dplyr::summarise(
            n_unweighted = dplyr::n(),
            weighted_n = sum(w, na.rm = TRUE),
```

```
            weighted_event_rate = weighted_mean_safe(y, w),
            ess = weighted_ess(w),
            .groups = "drop"
          ) |>
          dplyr::mutate(
            imp = imp_index,
            group = group,
            outcome = discordance_outcome_labels[[outcome_var]],
            setting_var = setting_var
          )
      }
    }
  }
  rm(d_imp)
  invisible(gc(FALSE))
}

discordance_setting_sensitivity <- dplyr::bind_rows(setting_rows) |>
  dplyr::group_by(setting_var, setting_level, group, outcome) |>
  dplyr::summarise(
    n_unweighted = stats::median(n_unweighted, na.rm = TRUE),
    weighted_n = stats::median(weighted_n, na.rm = TRUE),
    weighted_event_rate = stats::median(weighted_event_rate, na.rm = TRUE),
    ess = stats::median(ess, na.rm = TRUE),
    m_used = dplyr::n_distinct(imp),
    .groups = "drop"
  )
write_csv_safely(
  discordance_setting_sensitivity,
  results_path("discordance_setting_sensitivity.csv"),
  row_names = FALSE
)
discordance_setting_rate_ranges <- discordance_setting_sensitivity |>
  dplyr::group_by(setting_var, outcome, group) |>
  dplyr::summarise(
    n_setting_levels = dplyr::n(),
```

```
    min_weighted_event_rate = suppressWarnings(min(weighted_event_rate, na.rm = TRUE)),
    max_weighted_event_rate = suppressWarnings(max(weighted_event_rate, na.rm = TRUE)),
    rate_range = max_weighted_event_rate - min_weighted_event_rate,
    .groups = "drop"
  ) |>
  dplyr::mutate(
    dplyr::across(
      c(min_weighted_event_rate, max_weighted_event_rate, rate_range),
      ~ ifelse(is.infinite(.x), NA_real_, .x)
    )
  )
write_csv_safely(
  discordance_setting_rate_ranges,
  results_path("discordance_setting_rate_ranges.csv"),
  row_names = FALSE
)
record_discordance_status(
  "setting_sensitivity",
  "completed",
  ifelse(
    is.na(site_field),
    "Encounter-type and location sensitivity completed; no separate feasible site field was found.",
    paste0("Encounter-type, location, and site-like field sensitivity completed using ", site_field, ".")
  ),
  "Results/discordance_setting_sensitivity.csv; Results/discordance_setting_rate_ranges.csv"
)
if (is.na(site_field)) {
  discordance_site_sensitivity <- tibble::tibble(
    setting_var = NA_character_,
    setting_level = NA_character_,
    group = NA_character_,
    outcome = NA_character_,
    n_unweighted = NA_real_,
    weighted_n = NA_real_,
    weighted_event_rate = NA_real_,
    ess = NA_real_,
```

```
    m_used = NA_integer_,
    component_status = "skipped",
    reason = "No separate site/facility identifier with 2-50 levels was found."
  )
  record_discordance_status(
    "site_specific_sensitivity",
    "skipped",
    "No separate site/facility identifier with 2-50 levels was found.",
    "Results/discordance_site_sensitivity.csv",
    optional = TRUE
  )
} else {
  discordance_site_sensitivity <- discordance_setting_sensitivity |>
    dplyr::filter(.data$setting_var == .env$site_field) |>
    dplyr::mutate(
      component_status = "completed",
      reason = paste0("Site-like sensitivity used field ", .env$site_field, ".")
    )
  record_discordance_status(
    "site_specific_sensitivity",
    "completed",
    paste0("Site-like sensitivity used field ", site_field, "."),
    "Results/discordance_site_sensitivity.csv"
  )
}
write_csv_safely(
  discordance_site_sensitivity,
  results_path("discordance_site_sensitivity.csv"),
  row_names = FALSE
)

dual_rows <- list()
dual_tested <- subset_data$has_abg == 1 & subset_data$has_vbg == 1
if (sum(dual_tested, na.rm = TRUE) > 0L) {
  for (outcome_var in discordance_outcome_vars) {
    y <- to01(subset_data[[outcome_var]])
```

```r
    dual_rows[[length(dual_rows) + 1L]] <- tibble::tibble(
      modality = "ABG",
      co2_category = as.character(make_co2_cat3(subset_data$paco2, ABG_CO2_LOW, ABG_CO2_HIGH)),
      outcome = discordance_outcome_labels[[outcome_var]],
      y = y,
      dual_tested = dual_tested
    )
    dual_rows[[length(dual_rows) + 1L]] <- tibble::tibble(
      modality = "VBG",
      co2_category = as.character(make_co2_cat3(subset_data$vbg_co2, VBG_CO2_LOW, VBG_CO2_HIGH)),
      outcome = discordance_outcome_labels[[outcome_var]],
      y = y,
      dual_tested = dual_tested
    )
  }
  discordance_dual_tested_sensitivity <- dplyr::bind_rows(dual_rows) |>
    dplyr::filter(.data$dual_tested, !is.na(.data$co2_category)) |>
    dplyr::group_by(outcome, modality, co2_category) |>
    dplyr::summarise(
      n = dplyr::n(),
      events = sum(y == 1, na.rm = TRUE),
      event_rate = mean(y == 1, na.rm = TRUE),
      .groups = "drop"
    )
  record_discordance_status(
    "dual_tested_sensitivity",
    "completed",
    "Dual-tested encounter event rates by ABG/VBG CO2 category were summarized.",
    "Results/discordance_dual_tested_sensitivity.csv"
  )
} else {
  discordance_dual_tested_sensitivity <- tibble::tibble(
    outcome = NA_character_,
    modality = NA_character_,
    co2_category = NA_character_,
    n = 0L,
```

```
    events = 0L,
    event_rate = NA_real_,
    component_status = "skipped",
    reason = "no_dual_tested_encounters"
  )
  record_discordance_status(
    "dual_tested_sensitivity",
    "skipped",
    "No dual-tested encounters were available in this render subset.",
    "Results/discordance_dual_tested_sensitivity.csv",
    optional = TRUE
  )
}
write_csv_safely(
  discordance_dual_tested_sensitivity,
  results_path("discordance_dual_tested_sensitivity.csv"),
  row_names = FALSE
)

patient_field <- first_existing_col(
  c("patient_id", "person_id", "pat_id", "empi", "mrn", "member_id"),
  names(subset_data)
)
encounter_date_field <- first_existing_col(
  c("encounter_date", "encounter_datetime", "encounter_time", "enc_date"),
  names(subset_data)
)

if (!is.na(patient_field)) {
  first_df <- subset_data
  if (!is.na(encounter_date_field)) {
    first_df$.encounter_order <- suppressWarnings(as.POSIXct(first_df[[encounter_date_field]]))
  } else {
    first_df$.encounter_order <- seq_len(nrow(first_df))
  }
  first_df <- first_df |>
```

```
    dplyr::arrange(.data[[patient_field]], .data$.encounter_order) |>
    dplyr::group_by(.data[[patient_field]]) |>
    dplyr::slice_head(n = 1) |>
    dplyr::ungroup()
  discordance_first_encounter_sensitivity <- tidyr::crossing(
    outcome_var = discordance_outcome_vars,
    group = discordance_groups
  ) |>
    dplyr::rowwise() |>
    dplyr::mutate(
      outcome = discordance_outcome_labels[[outcome_var]],
      treat_var = ifelse(group == "ABG", "has_abg", "has_vbg"),
      n_tested = sum(first_df[[treat_var]] == 1, na.rm = TRUE),
      events = sum(to01(first_df[[outcome_var]]) == 1 & first_df[[treat_var]] == 1, na.rm = TRUE),
      event_rate = ifelse(n_tested > 0, events / n_tested, NA_real_),
      patient_field = patient_field,
      encounter_date_field = ifelse(is.na(encounter_date_field), "", encounter_date_field)
    ) |>
    dplyr::ungroup() |>
    dplyr::select(outcome, group, n_tested, events, event_rate, patient_field, encounter_date_field)
  record_discordance_status(
    "first_encounter_sensitivity",
    "completed",
    paste0("First-encounter sensitivity used patient field ", patient_field, "."),
    "Results/discordance_first_encounter_sensitivity.csv",
    optional = TRUE
  )
} else {
  discordance_first_encounter_sensitivity <- tibble::tibble(
    outcome = NA_character_,
    group = NA_character_,
    n_tested = NA_integer_,
    events = NA_integer_,
    event_rate = NA_real_,
    patient_field = NA_character_,
    encounter_date_field = NA_character_,
```

```
    component_status = "skipped",
    reason = "patient_identifier_unavailable"
  )
  record_discordance_status(
    "first_encounter_sensitivity",
    "skipped",
    "No patient identifier was available for first-encounter restriction.",
    "Results/discordance_first_encounter_sensitivity.csv",
    optional = TRUE
  )
}
write_csv_safely(
  discordance_first_encounter_sensitivity,
  results_path("discordance_first_encounter_sensitivity.csv"),
  row_names = FALSE
)

## Timing sensitivity
timing_names <- names(subset_data_raw)
gas_time_candidates <- candidate_cols(
  timing_names,
  "(abg|vbg|blood_gas|gas).*(date|time|timestamp|datetime)",
  max_n = 20
)
proc_time_candidates <- candidate_cols(
  timing_names,
  "(imv|niv).*(date|time|timestamp|datetime)",
  max_n = 20
)

if (length(gas_time_candidates) && length(proc_time_candidates)) {
  discordance_timing_status <- tibble::tibble(
    component = "timing_sensitivity",
    status = "available_fields_detected",
    gas_time_fields = paste(gas_time_candidates, collapse = "; "),
    procedure_time_fields = paste(proc_time_candidates, collapse = "; "),
```

```
    reason = "candidate_fields_available_manual_review_needed"
  )
  discordance_timing_sensitivity <- tibble::tibble(
    outcome = NA_character_,
    group = NA_character_,
    n_candidate_gas_time_fields = length(gas_time_candidates),
    n_candidate_procedure_time_fields = length(proc_time_candidates),
    status = "candidate_fields_detected",
    reason = "Field naming is not specific enough for an automated before-procedure restriction."
  )
  record_discordance_status(
    "timing_sensitivity",
    "skipped",
    "Candidate timing fields were detected, but no unambiguous gas/procedure timestamp pairs were available.",
    "Results/discordance_timing_status.csv",
    optional = TRUE
  )
} else {
  discordance_timing_status <- tibble::tibble(
    component = "timing_sensitivity",
    status = "skipped",
    gas_time_fields = paste(gas_time_candidates, collapse = "; "),
    procedure_time_fields = paste(proc_time_candidates, collapse = "; "),
    reason = "timestamp_fields_unavailable"
  )
  discordance_timing_sensitivity <- tibble::tibble(
    outcome = NA_character_,
    group = NA_character_,
    n_candidate_gas_time_fields = length(gas_time_candidates),
    n_candidate_procedure_time_fields = length(proc_time_candidates),
    status = "skipped",
    reason = "timestamp_fields_unavailable"
  )
  record_discordance_status(
    "timing_sensitivity",
    "skipped",
```

```r
    "Blood-gas/procedure timestamp fields were unavailable.",
    "Results/discordance_timing_status.csv",
    optional = TRUE
  )
}
write_csv_safely(
  discordance_timing_status,
  results_path("discordance_timing_status.csv"),
  row_names = FALSE
)
write_csv_safely(
  discordance_timing_sensitivity,
  results_path("discordance_timing_sensitivity.csv"),
  row_names = FALSE
)

## Summary of hypotheses and interpretation
standardization_supported <- any(
  is.finite(discordance_standardization_summary$gap_current_ref) &
    (
      abs(discordance_standardization_summary$gap_common_ref) <=
        0.75 * abs(discordance_standardization_summary$gap_current_ref) |
        abs(discordance_standardization_summary$gap_marginal_standardized) <=
          0.75 * abs(discordance_standardization_summary$gap_current_ref)
    ),
  na.rm = TRUE
)

scale_supported <- any(
  grepl("Relative odds are closer", discordance_scale_summary$comment, fixed = TRUE),
  na.rm = TRUE
)

capture_supported <- any(
  missingness_capture_summary$max_abs_abg_vbg_difference > 0.05,
  na.rm = TRUE
```

```r
)

setting_supported <- any(discordance_setting_rate_ranges$rate_range > 0.10, na.rm = TRUE)

sentinel_selected_n <- sum(discordance_sentinel_procedure_summary$selected %in% TRUE, na.rm = TRUE)
sentinel_inconclusive <- sentinel_selected_n < 2L
sentinel_supported <- !sentinel_inconclusive && any(
  abs(discordance_sentinel_procedure_summary$weighted_abs_diff) > 0.05 |
    discordance_sentinel_procedure_summary$date_missing_abs_diff > 0.05,
  na.rm = TRUE
)

tail_supported <- any(discordance_tail_support_summary$support_flag %in% TRUE, na.rm = TRUE)

dual_inconclusive <- any(
  dplyr::bind_rows(discordance_status_rows) |>
    dplyr::filter(.data$component == "dual_tested_sensitivity") |>
    dplyr::pull(status) == "skipped",
  na.rm = TRUE
)

timing_inconclusive <- any(
  dplyr::bind_rows(discordance_status_rows) |>
    dplyr::filter(.data$component == "timing_sensitivity") |>
    dplyr::pull(status) == "skipped",
  na.rm = TRUE
)

discordance_interpretation_summary <- tibble::tibble(
  hypothesis = c(
    "Reference-profile / standardization effect",
    "Setting or workflow heterogeneity",
    "Differential missingness or procedure capture",
    "Sentinel procedure capture/completeness",
    "Scale / noncollapsibility effect",
    "Same-encounter timing structure",
```

```
  "Tail support / positivity",
  "Intrinsic modality-specific measurement effect"
),
supported = c(
  standardization_supported,
  setting_supported,
  capture_supported,
  sentinel_supported,
  scale_supported,
  FALSE,
  tail_supported,
  FALSE
),
not_supported = c(
  !standardization_supported,
  !setting_supported,
  !capture_supported,
  !sentinel_supported & !sentinel_inconclusive,
  !scale_supported,
  !timing_inconclusive,
  !tail_supported,
  !dual_inconclusive
),
inconclusive = c(
  FALSE,
  FALSE,
  FALSE,
  sentinel_inconclusive,
  FALSE,
  timing_inconclusive,
  FALSE,
  dual_inconclusive
),
key_evidence = c(
  ifelse(standardization_supported, "Common or marginal standardization shrank at least one current probability gap
   ↪  by >=25%.", "Standardization did not shrink probability gaps by >=25%."),
```

```
    ifelse(setting_supported, "Within-modality weighted event-rate range exceeded 10 percentage points for at least
     ↪  one setting variable.", "Within-modality setting-specific weighted event-rate ranges were <=10 percentage
     ↪  points."),
    ifelse(capture_supported, "Detected missingness/capture summaries show >5 percentage-point or SMD-scale
     ↪  differences.", "Detected missingness/capture summaries did not show >5-point differences."),
    ifelse(sentinel_inconclusive, "Fewer than two suitable sentinel procedure concepts were selected.",
     ↪  ifelse(sentinel_supported, "At least one sentinel procedure showed >5-point capture or timestamp-completeness
     ↪  difference.", "Selected sentinel procedures did not show >5-point capture/completeness differences.")),
    ifelse(scale_supported, "At least one contrast had closer ABG-VBG agreement on OR scale than risk-difference
     ↪  scale.", "OR-scale agreement was not consistently closer than probability-scale agreement."),
    ifelse(timing_inconclusive, "Timestamp fields were unavailable or ambiguous.", "Timing fields were available for
     ↪  direct review."),
    ifelse(tail_supported, "At least one clinical/quantile bin had ESS <100 or weighted events <10.", "Tail bins did
     ↪  not trigger the ESS/event-count support flag."),
    ifelse(dual_inconclusive, "Dual-tested subset was unavailable in this render subset.", "Dual-tested subset event
     ↪  rates were generated for direct review.")
  ),
  recommended_next_step = c(
    "If supported, consider whether manuscript probability curves should use common or marginal standardization in a
     ↪  future methodological ticket.",
    "If supported, consider setting-stratified or setting-interaction sensitivity models.",
    "If supported, audit NIV/IMV source-field construction and coding-density definitions before changing models.",
    "If supported, inspect selected sentinel source fields and date-field definitions before attributing NIV/IMV gaps
     ↪  to physiology.",
    "Use risk differences/RRs beside ORs when communicating procedure outcomes.",
    "If timestamps become available, restrict to blood gases clearly preceding NIV/IMV.",
    "If supported, avoid overinterpreting tail probability gaps and consider trimming/support annotations.",
    "Compare dual-tested event-rate patterns against primary ABG/VBG gaps before attributing discordance to modality."
  )
)
write_csv_safely(
  discordance_interpretation_summary,
  results_path("discordance_interpretation_summary.csv"),
  row_names = FALSE
)
record_discordance_status(
  "interpretation_summary",
```

```
  "completed",
  "Hypothesis-level interpretation summary was generated.",
  "Results/discordance_interpretation_summary.csv"
)

status_df_pre_manifest <- write_discordance_status()

discordance_required_artifacts <- c(
  "discordance_validation_status.csv",
  "discordance_current_summary.csv",
  "discordance_gap_metrics.csv",
  "discordance_xref_common_prob_curves.csv",
  "discordance_marginal_standardized_prob_curves.csv",
  "discordance_marginal_standardization_status.csv",
  "discordance_standardization_summary.csv",
  "discordance_scale_summary.csv",
  "discordance_missingness_outcome_fields.csv",
  "discordance_procedure_capture_proxies.csv",
  "discordance_missing_indicator_balance.csv",
  "discordance_missingness_capture_summary.csv",
  "discordance_sentinel_procedure_inventory.csv",
  "discordance_sentinel_procedure_selection.csv",
  "discordance_sentinel_procedure_balance.csv",
  "discordance_sentinel_procedure_summary.csv",
  "discordance_tail_support_summary.csv",
  "discordance_tail_support_by_bin.csv",
  "discordance_setting_sensitivity.csv",
  "discordance_setting_rate_ranges.csv",
  "discordance_site_sensitivity.csv",
  "discordance_dual_tested_sensitivity.csv",
  "discordance_first_encounter_sensitivity.csv",
  "discordance_timing_sensitivity.csv",
  "discordance_timing_status.csv",
  "discordance_interpretation_summary.csv",
  "figs/discordance_probability_standardization_niv_imv.png",
  "figs/discordance_tail_support_niv_imv.png",
```

```r
  "discordance_artifact_manifest.csv"
)

discordance_artifact_manifest <- tibble::tibble(
  artifact_path = discordance_required_artifacts,
  required = TRUE,
  kind = dplyr::case_when(
    grepl("^figs/", artifact_path) ~ "figure",
    grepl("manifest|status", artifact_path) ~ "validation",
    TRUE ~ "csv"
  ),
  exists = vapply(
    artifact_path,
    function(path) {
      if (identical(path, "discordance_artifact_manifest.csv")) return(TRUE)
      file.exists(results_path(path))
    },
    logical(1L)
  ),
  purpose = "NIV/IMV probability-OR discordance diagnostic artifact."
)
write_csv_safely(
  discordance_artifact_manifest,
  results_path("discordance_artifact_manifest.csv"),
  row_names = FALSE
)

missing_discordance_artifacts <- discordance_artifact_manifest |>
  dplyr::filter(.data$required, !.data$exists) |>
  dplyr::pull(artifact_path)
if (length(missing_discordance_artifacts)) {
  stop(
    "Discordance artifact completeness check failed: ",
    paste(missing_discordance_artifacts, collapse = ", ")
  )
}
```

```r
record_discordance_status(
  "artifact_manifest",
  "completed",
  "All required discordance artifacts were present.",
  "Results/discordance_artifact_manifest.csv"
)
status_df <- write_discordance_status()

required_status_failures <- status_df |>
  dplyr::filter(!.data$optional, .data$status != "completed")
if (nrow(required_status_failures)) {
  stop(
    "Required discordance component(s) did not complete: ",
    paste(required_status_failures$component, collapse = ", ")
  )
}

render_table_pdf(
  discordance_interpretation_summary |>
    dplyr::mutate(
      supported = ifelse(supported, "yes", "no"),
      not_supported = ifelse(not_supported, "yes", "no"),
      inconclusive = ifelse(inconclusive, "yes", "no")
    ),
  caption = "Summary of plausible explanations for NIV/IMV probability-OR discordance",
  file_stub = "discordance_interpretation_summary_display",
  wide = TRUE,
  digits = 2,
  max_cols = 5,
  landscape = TRUE
)
```

Summary of plausible explanations for NIV/IMV probability-OR discordance

hypothesis	supported	not_supported	inconclusive	key_evidence
Reference-profile / standardization effect	yes	no	no	Common or marginal standardization shrank at least one current probability gap by >=25%.
Setting or workflow heterogeneity	yes	no	no	Within-modality weighted event-rate range exceeded 10 percentage points for at least one setting variable.
Differential missingness or procedure capture	yes	no	no	Detected missingness/capture summaries show >5 percentage-point or SMD-scale differences.
Sentinel procedure capture/completeness	yes	no	no	At least one sentinel procedure showed >5-point capture or timestamp-completeness difference.
Scale / noncollapsibility effect	no	yes	no	OR-scale agreement was not consistently closer than probability-scale agreement.
Same-encounter timing structure	no	no	yes	Timestamp fields were unavailable or ambiguous.
Tail support / positivity	no	yes	no	Tail bins did not trigger the ESS/event-count support flag.
Intrinsic modality-specific measurement effect	no	yes	no	Dual-tested subset event rates were generated for direct review.

recommended_next_step
If supported, consider whether manuscript probability curves should use common or marginal standardization in a future methodological ticket. If supported, consider setting-stratified or setting-interaction sensitivity models. If supported, audit NIV/IMV source-field construction and coding-density definitions before changing models. If supported, inspect selected sentinel source fields and date-field definitions before attributing NIV/IMV gaps to physiology. Use risk differences/RRs beside ORs when communicating procedure outcomes.
If timestamps become available, restrict to blood gases clearly preceding NIV/IMV. If supported, avoid overinterpreting tail probability gaps and consider trimming/support annotations. Compare dual-tested event-rate patterns against primary ABG/VBG gaps before attributing discordance to modality.

```r
cat(
  "Summary of plausible explanations for NIV/IMV probability-OR discordance\n\n",
  "- Required discordance diagnostics completed in run mode `", RUN_MODE, "` with `pilot_frac = ", PILOT_FRAC, "`.\n",
  "- Optional sentinel-procedure, timing, first-encounter, dual-tested, and site-specific diagnostics may be marked
   ↪  skipped when required fields or rows are unavailable.\n",
  "- Detailed artifacts are written under `Results/discordance_*` and `Results/figs/discordance_*`.\n",
  sep = ""
)
```

Summary of plausible explanations for NIV/IMV probability-OR discordance

- Required discordance diagnostics completed in run mode `pilot` with `pilot_frac = 0.25`.
- Optional sentinel-procedure, timing, first-encounter, dual-tested, and site-specific diagnostics may be marked skipped when required fields or rows are unavailable.
- Detailed artifacts are written under `Results/discordance_*` and `Results/figs/discordance_*`.

```r
# Purpose: diagnostics completeness.
expected_diag <- c(
  "runtime_log.csv", "runtime_summary.csv", "runtime_summary_top15.csv",
  "latex_codeblock_risk_scan.csv",
  "warnings_log.csv", "mi_warnings_log.csv", "mi_info_log.csv",
  "diagnostics_summary.csv", "diagnostics_missingness.csv",
  "diagnostics_missingness-by-strata.csv",
  "balance_target_imp_summary.csv", "balance_target_by_imp.csv",
  "balance_target_worst_rows.csv", "balance_max_smd_by_imp.csv",
  "balance_worst_terms.csv", "balance_worst10.csv", "balance_table.csv",
  "weight_summary.csv", "ps_overlap_summary.csv",
  "model_fit_diagnostics.csv", "mi_outcome_fit_diagnostics.csv",
  "mi_fit_issue_summary.csv", "mi_m_stability.csv", "mi_maxit_sensitivity.csv",
  "mi_obs_vs_imp_summary.csv", "mi_spline_curve_abg.csv",
  "mi_spline_curve_vbg.csv",
  "mi_cat3_prob_curve_abg.csv", "mi_cat3_prob_curve_vbg.csv",
  "mi_threelevel_results.csv",
  "mi_spline_prob_curve_abg.csv", "mi_spline_prob_curve_vbg.csv",
  "mi_spline_coef_abg.csv",
  "mi_spline_coef_vbg.csv", "diag-ps-shap-stability.csv",
  "diag_mi_shap_errors.csv", "diag_mi_shap_method_used.csv", "mi_shap_imp_diagnostics.csv",
```

```
  "mice_logged_events_raw.csv", "mice_logged_events_summary.csv",
  "mice_pred_width_preflight.csv", "mice_chain_diagnostics.csv",
  "mice_batches_log.csv", "missingness-by-strata.csv",
  "missingness-drivers.csv", "missingness-pattern.csv", "plot_drop_log.csv",
  "mice_smoketest.log",
  "discordance_validation_status.csv", "discordance_artifact_manifest.csv",
  "discordance_current_summary.csv", "discordance_gap_metrics.csv",
  "discordance_xref_common_prob_curves.csv",
  "discordance_marginal_standardized_prob_curves.csv",
  "discordance_marginal_standardization_status.csv",
  "discordance_standardization_summary.csv", "discordance_scale_summary.csv",
  "discordance_missingness_outcome_fields.csv",
  "discordance_procedure_capture_proxies.csv",
  "discordance_missing_indicator_balance.csv",
  "discordance_missingness_capture_summary.csv",
  "discordance_sentinel_procedure_inventory.csv",
  "discordance_sentinel_procedure_selection.csv",
  "discordance_sentinel_procedure_balance.csv",
  "discordance_sentinel_procedure_summary.csv",
  "discordance_tail_support_summary.csv",
  "discordance_tail_support_by_bin.csv",
  "discordance_setting_sensitivity.csv",
  "discordance_setting_rate_ranges.csv",
  "discordance_site_sensitivity.csv",
  "discordance_dual_tested_sensitivity.csv",
  "discordance_first_encounter_sensitivity.csv",
  "discordance_timing_sensitivity.csv", "discordance_timing_status.csv",
  "discordance_interpretation_summary.csv",
  "figs/discordance_probability_standardization_niv_imv.png",
  "figs/discordance_tail_support_niv_imv.png"
)
missing_files <- expected_diag[!file.exists(results_path(expected_diag))]
stopifnot(length(missing_files) == 0)

# Only a subset of diagnostics tables are guaranteed to be rewritten with the
# current run metadata on every render. Many analysis CSVs are checked here for
```

```r
# existence, but their contents may be intentionally preserved across reruns or
# written by helpers that do not stamp the current run_id into the first row.
# Restrict freshness checks to the end-of-run diagnostics that are explicitly
# regenerated in this session.
run_id_check_files <- c(
  "runtime_log.csv", "runtime_summary.csv", "runtime_summary_top15.csv",
  "warnings_log.csv", "mi_warnings_log.csv", "mi_info_log.csv",
  "diagnostics_summary.csv", "diagnostics_missingness.csv",
  "diagnostics_missingness-by-strata.csv", "plot_drop_log.csv"
)

run_id_mismatch <- character()
for (f in run_id_check_files) {
  path <- results_path(f)
  if (grepl("\\.csv$", f)) {
    df <- tryCatch(read.csv(path, nrows = 1), error = function(e) NULL)
    stopifnot(!is.null(df))
    stopifnot("run_id" %in% names(df))
    if (nrow(df) > 0 && !identical(as.character(df$run_id[1]), diag_run_id)) {
      run_id_mismatch <- c(run_id_mismatch, f)
    }
  }
}

if (length(missing_files) || length(run_id_mismatch)) {
  msg <- paste0(
    "Diagnostics completeness check failed: missing files [",
    paste(missing_files, collapse = ", "),
    "]; run_id mismatch [",
    paste(run_id_mismatch, collapse = ", "),
    "]."
  )
  stop(msg)
}
```

5.2 Save, export, and session info

```
# Purpose: diagnostics audit run.
audit_script <- here::here("R", "diagnostics_audit.R")
audit_md <- results_path("diagnostics_audit.md")
audit_summary <- results_path("diagnostics_audit_summary.csv")
audit_issues <- results_path("diagnostics_audit_issues.csv")
stopifnot(file.exists(audit_script))
old_diag_results_dir <- Sys.getenv("DIAG_RESULTS_DIR", unset = "")
on.exit(Sys.setenv(DIAG_RESULTS_DIR = old_diag_results_dir), add = TRUE)
Sys.setenv(DIAG_RESULTS_DIR = results_dir)
audit_env <- new.env(parent = globalenv())
audit_result <- tryCatch(
  {
    sys.source(audit_script, envir = audit_env)
    TRUE
  },
  error = function(e) e
)
if (inherits(audit_result, "error")) {
  stop("diagnostics_audit.R failed: ", conditionMessage(audit_result), call. = FALSE)
}

read_required_audit_csv <- function(path, required_cols, label) {
  if (!file.exists(path)) {
    stop(label, " missing after diagnostics audit: ", path, call. = FALSE)
  }
  df <- tryCatch(
    utils::read.csv(path, stringsAsFactors = FALSE),
    error = function(e) stop(label, " could not be read: ", conditionMessage(e), call. = FALSE)
  )
  missing_cols <- setdiff(required_cols, names(df))
  if (length(missing_cols)) {
    stop(
      label, " missing required columns [",
      paste(missing_cols, collapse = ", "),
```

```
      "]: ", path,
      call. = FALSE
    )
  }
  df
}

diagnostics_audit_summary_df <- read_required_audit_csv(
  audit_summary,
  required_cols = c("component", "metric", "value", "status", "severity", "evidence_file", "note", "run_id",
   ↪  "run_ts"),
  label = "diagnostics_audit_summary.csv"
)

diagnostics_audit_issues_df <- read_required_audit_csv(
  audit_issues,
  required_cols = c("severity", "component", "evidence_file", "evidence_snippet", "why_it_matters", "recommended_fix",
   ↪  "run_id", "run_ts"),
  label = "diagnostics_audit_issues.csv"
)
```

```
# Purpose: stage diagnostics audit outputs for the final validation section.
diagnostics_audit_issues_df <- diagnostics_audit_issues_df |>
  dplyr::arrange(factor(severity, levels = c("high", "medium", "low")))

diagnostics_audit_summary_df <- diagnostics_audit_summary_df |>
  dplyr::arrange(factor(status, levels = c("fail", "warn", "pass")))

invisible(list(
  summary = diagnostics_audit_summary_df,
  issues = diagnostics_audit_issues_df
))
```

```
# Purpose: render one canonical validation-oriented PDF section: build status,
# provenance, manuscript-facing outputs, and compact essential audits.
diagnostics_overall_status <- diagnostics_audit_summary_df |>
  dplyr::filter(metric == "overall_status") |>
```

```r
  dplyr::slice_head(n = 1) |>
  dplyr::pull(value)

if (!length(diagnostics_overall_status) || !nzchar(diagnostics_overall_status)) {
  stop("diagnostics_audit_summary.csv did not provide metric == 'overall_status'.", call. = FALSE)
}

build_status_components <- dplyr::bind_rows(
  validation_static_summary,
  tibble::tibble(
    component = "diagnostics_audit",
    status = diagnostics_overall_status,
    detail = paste0(
      sum(diagnostics_audit_issues_df$severity == "high", na.rm = TRUE), " high / ",
      sum(diagnostics_audit_issues_df$severity == "medium", na.rm = TRUE), " medium / ",
      sum(diagnostics_audit_issues_df$severity == "low", na.rm = TRUE), " low diagnostics issues."
    )
  )
)

final_build_status <- collapse_validation_status(build_status_components$status)

validation_build_display <- dplyr::bind_rows(
  tibble::tibble(
    component = "overall_build",
    status = final_build_status,
    detail = "Final validation status derived from artifact checks, glyph audit, duplicate audit, and diagnostics
     ↪  audit."
  ),
  build_status_components
)

provenance_summary_display <- artifact_provenance_manifest |>
  dplyr::count(section, kind, status, source_type, pdf_display, name = "n_assets") |>
  dplyr::arrange(section, kind, status)
```

```r
artifact_check_summary_display <- artifact_check_status |>
  dplyr::count(check_status, severity, allowed_missing, name = "n_rows") |>
  dplyr::arrange(dplyr::desc(n_rows), check_status)

diagnostics_audit_summary_display <- diagnostics_audit_summary_df |>
  dplyr::select(component, metric, status, severity, value, note)

render_table_pdf(
  validation_build_display,
  caption = "Validation build status",
  file_stub = "validation_build_status",
  wide = TRUE,
  digits = 0,
  max_cols = 3,
  landscape = FALSE
)
```

Validation build status

component	status	detail
overall_build	FAIL	Final validation status derived from artifact checks, glyph audit, duplicate audit, and diagnostics audit.
artifact_registry	PASS	0 missing required generated artifact rows.
external_manual_assets	PASS	0 declared external/manual rows.
glyph_audit	PASS	0 unsafe manuscript glyph rows.
duplicate_audit	PASS	0 conflicting duplicate/numbering rows.
diagnostics_audit	FAIL	1 high / 0 medium / 0 low diagnostics issues.

```r
render_table_pdf(
  provenance_summary_display,
  caption = "Artifact provenance summary",
  file_stub = "artifact_provenance_summary",
  wide = TRUE,
  digits = 0,
  max_cols = 6,
  landscape = TRUE
)
```

Artifact provenance summary

section	kind	status	source_type	pdf_display	n_assets
draft_only	figure	draft_only	suppressed/deprecated	FALSE	1
main	figure	generated	notebook_generated	TRUE	2
main	table	generated	notebook_generated	TRUE	2
supplement	figure	generated	notebook_generated	TRUE	8
supplement	table	generated	notebook_generated	TRUE	5

```
render_validation_manuscript_assets()
```

Figure 1. Cohort assembly and target population for inverse probability-of-sampling weighting

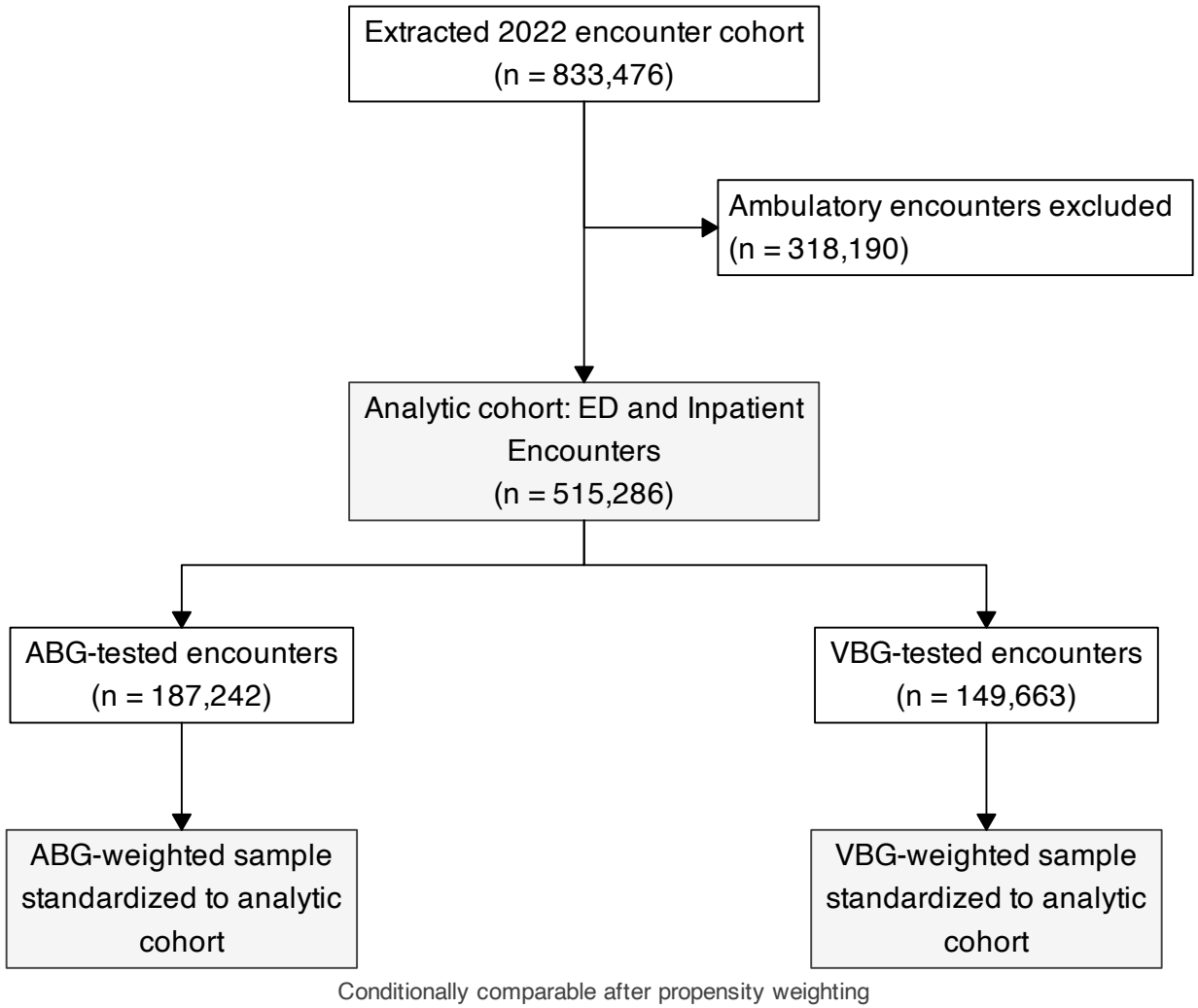
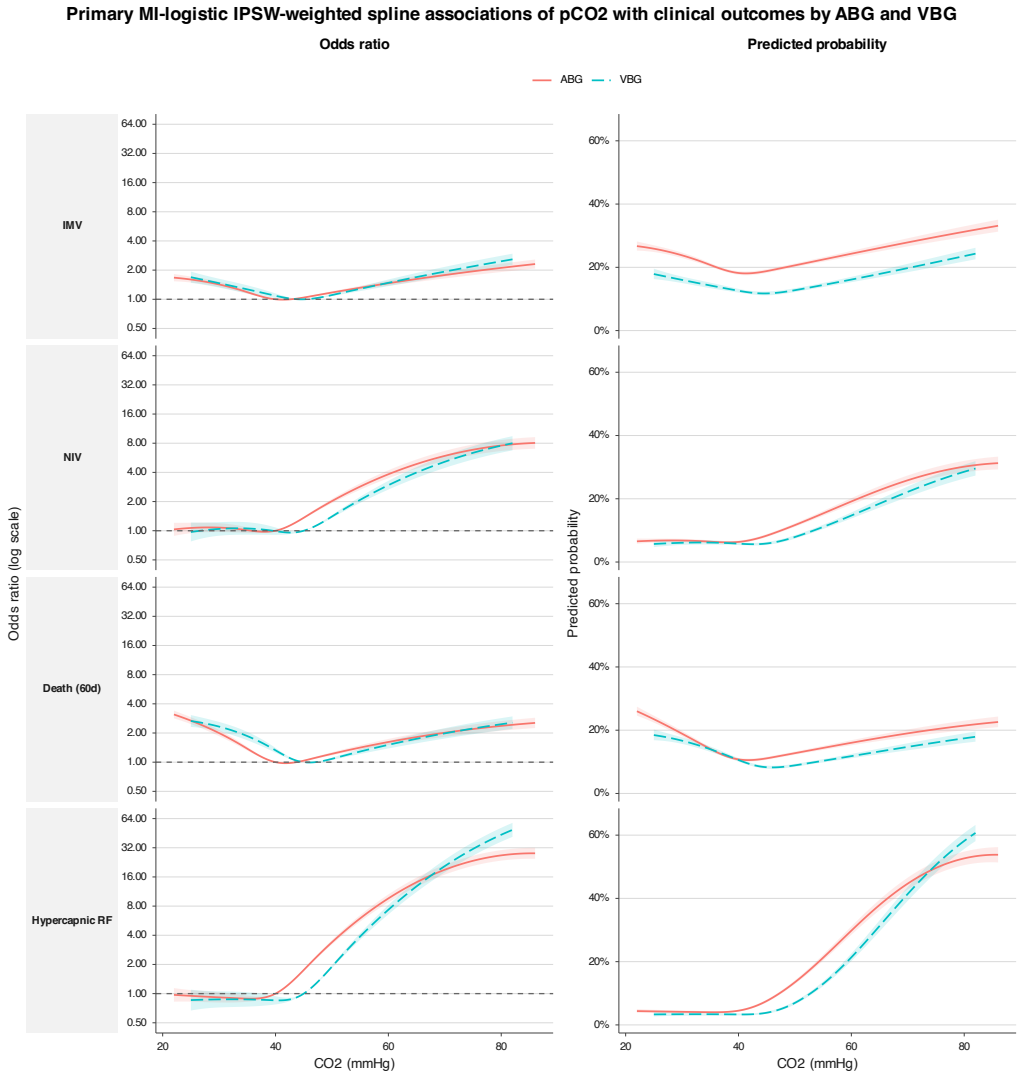


Figure 2. Primary MI-logistic IPSW-weighted spline associations of pCO2 with clinical outcomes by ABG and VBG



Note: These curves come from the primary MI-logistic IPSW analysis. Odds ratio curves are relative to the test-specific reference CO2 values (ABG 40 mmHg; VBG 45 mmHg). Predicted probability curves are marginally standardized to the common eligible source-population covariate distribution.

Table 1. Baseline characteristics of the analytic cohort overall and by receipt of arterial and venous blood gas testing

Characteristic	Everyone	Did not get ABG	Did get ABG	Did not get VBG	Did get VBG
Age	59.2 ± 17.7; 0.0/128,521.0 missing (0.0%)	58.1 ± 18.1; 0.0/81,868.0 missing (0.0%)	61.2 ± 16.9; 0.0/46,653.0 missing (0.0%)	59.4 ± 17.8; 0.0/91,138.0 missing (0.0%)	58.9 ± 17.6; 0.0/37,383.0 missing (0.0%)
BMI	31.1 ± 8.3; 73,272.0/128,521.0 missing (57.0%)	32.2 ± 8.7; 46,078.0/81,868.0 missing (56.3%)	29.0 ± 7.2; 27,194.0/46,653.0 missing (58.3%)	31.7 ± 8.5; 48,047.0/91,138.0 missing (52.7%)	28.8 ± 7.4; 25,225.0/37,383.0 missing (67.5%)
Sex					
Female	63,461 (49%)	42,242 (52%)	21,219 (45%)	46,152 (51%)	17,309 (46%)
Male	65,060 (51%)	39,626 (48%)	25,434 (55%)	44,986 (49%)	20,074 (54%)
Race/Ethnicity					
White	80,152 (62%)	49,867 (61%)	30,285 (65%)	60,106 (66%)	20,046 (54%)
Black or African American	22,349 (17%)	15,542 (19%)	6,807 (15%)	15,339 (17%)	7,010 (19%)
Hispanic	8,479 (6.6%)	5,992 (7.3%)	2,487 (5.3%)	5,795 (6.4%)	2,684 (7.2%)
Asian	2,085 (1.6%)	1,250 (1.5%)	835 (1.8%)	1,367 (1.5%)	718 (1.9%)
American Indian	968 (0.8%)	442 (0.5%)	526 (1.1%)	492 (0.5%)	476 (1.3%)
Pacific Islander	184 (0.1%)	122 (0.1%)	62 (0.1%)	150 (0.2%)	34 (<0.1%)
Unknown	14,304 (11%)	8,653 (11%)	5,651 (12%)	7,889 (8.7%)	6,415 (17%)
Location					
South	60,251 (47%)	34,356 (42%)	25,895 (56%)	48,895 (54%)	11,356 (30%)
Northeast	32,438 (25%)	23,477 (29%)	8,961 (19%)	16,441 (18%)	15,997 (43%)
Midwest	9,646 (7.5%)	5,732 (7.0%)	3,914 (8.4%)	6,223 (6.8%)	3,423 (9.2%)
West	26,186 (20%)	18,303 (22%)	7,883 (17%)	19,579 (21%)	6,607 (18%)
OSA	22,584 (18%)	15,183 (19%)	7,401 (16%)	16,508 (18%)	6,076 (16%)
Asthma	17,372 (14%)	12,083 (15%)	5,289 (11%)	12,437 (14%)	4,935 (13%)
COPD	24,996 (19%)	15,107 (18%)	9,889 (21%)	17,656 (19%)	7,340 (20%)
CHF	25,458 (20%)	15,053 (18%)	10,405 (22%)	17,459 (19%)	7,999 (21%)
NMD	5,122 (4.0%)	2,965 (3.6%)	2,157 (4.6%)	3,762 (4.1%)	1,360 (3.6%)
PHTN	10,542 (8.2%)	6,108 (7.5%)	4,434 (9.5%)	7,009 (7.7%)	3,533 (9.5%)
CKD	22,968 (18%)	13,740 (17%)	9,228 (20%)	15,382 (17%)	7,586 (20%)
Diabetes	37,231 (29%)	23,327 (28%)	13,904 (30%)	25,356 (28%)	11,875 (32%)
Encounter Type					
Emergency	42,859 (33%)	35,638 (44%)	7,221 (15%)	31,009 (34%)	11,850 (32%)
Inpatient	85,662 (67%)	46,230 (56%)	39,432 (85%)	60,129 (66%)	25,533 (68%)

Table 2. MI-pooled, MI-logistic IPSW-weighted 3-level categorical results for ABG and VBG

Panel A. Odds ratios

group	outcome	n_model	events	Low vs normal OR (95% CI)	High vs normal OR (95% CI)
ABG	IMV	46653	12008	1.38 (1.30, 1.46)	1.33 (1.25, 1.41)
ABG	NIV	46653	4690	0.98 (0.89, 1.09)	2.62 (2.41, 2.85)
ABG	Death (60d)	46653	8249	1.90 (1.77, 2.03)	1.38 (1.28, 1.48)
ABG	Hypercapnic RF	46653	4990	0.95 (0.84, 1.07)	6.39 (5.86, 6.97)
VBG	IMV	37383	5788	1.39 (1.28, 1.52)	1.56 (1.43, 1.70)
VBG	NIV	37383	2657	0.97 (0.84, 1.11)	2.85 (2.55, 3.19)
VBG	Death (60d)	37383	5087	1.74 (1.59, 1.91)	1.37 (1.25, 1.50)
VBG	Hypercapnic RF	37383	3323	0.88 (0.75, 1.03)	8.13 (7.23, 9.14)

Panel B. Predicted probabilities

group	outcome	Normal probability (95% CI)	Low probability (95% CI)	High probability (95% CI)
ABG	IMV	0.19 (0.18, 0.19)	0.24 (0.23, 0.25)	0.23 (0.23, 0.24)
ABG	NIV	0.07 (0.06, 0.07)	0.07 (0.06, 0.07)	0.15 (0.14, 0.16)
ABG	Death (60d)	0.11 (0.11, 0.12)	0.19 (0.18, 0.20)	0.15 (0.14, 0.16)
ABG	Hypercapnic RF	0.04 (0.04, 0.05)	0.04 (0.04, 0.05)	0.22 (0.21, 0.23)
VBG	IMV	0.12 (0.11, 0.12)	0.15 (0.14, 0.16)	0.17 (0.16, 0.18)
VBG	NIV	0.06 (0.06, 0.07)	0.06 (0.05, 0.07)	0.15 (0.14, 0.16)
VBG	Death (60d)	0.09 (0.09, 0.10)	0.14 (0.14, 0.15)	0.12 (0.11, 0.12)
VBG	Hypercapnic RF	0.04 (0.04, 0.04)	0.03 (0.03, 0.04)	0.23 (0.22, 0.24)

Table S1. Inclusion criteria into the enriched sample

step	criterion	Source field / rule	Excluded, n	Retained, n	notes
1	Source extract	TriNetX-derived 2022 extract	0	833,476	Initial local source cohort.
2	ED/inpatient encounters	encounter_type != ambulatory	318,190	515,286	Matches Figure 1 cohort-flow restriction.
3	ABG-tested cohort	has_abg == 1	328,044	187,242	Measured arterial CO2 outcome models.
4	VBG-tested cohort	has_vbg == 1	365,623	149,663	Measured venous CO2 outcome models.
5	Target population	ED/inpatient analytic cohort	0	515,286	Target for IPSW standardization.

Figure S1. Covariate balance after MI logistic inverse-probability weighting for ABG and VBG test-ordering models

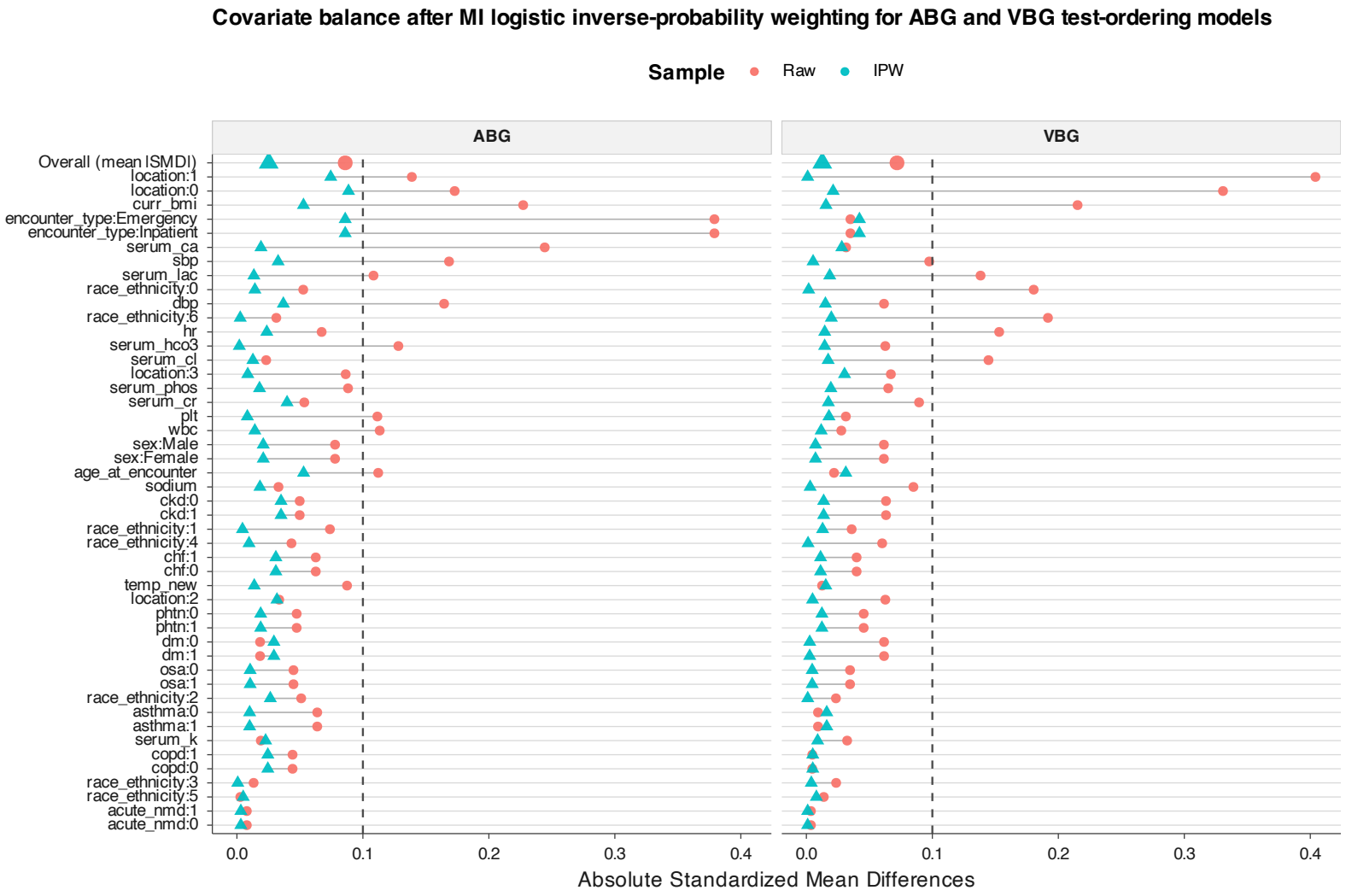


Figure S2. Propensity-score overlap for MI logistic ABG and VBG test-ordering models

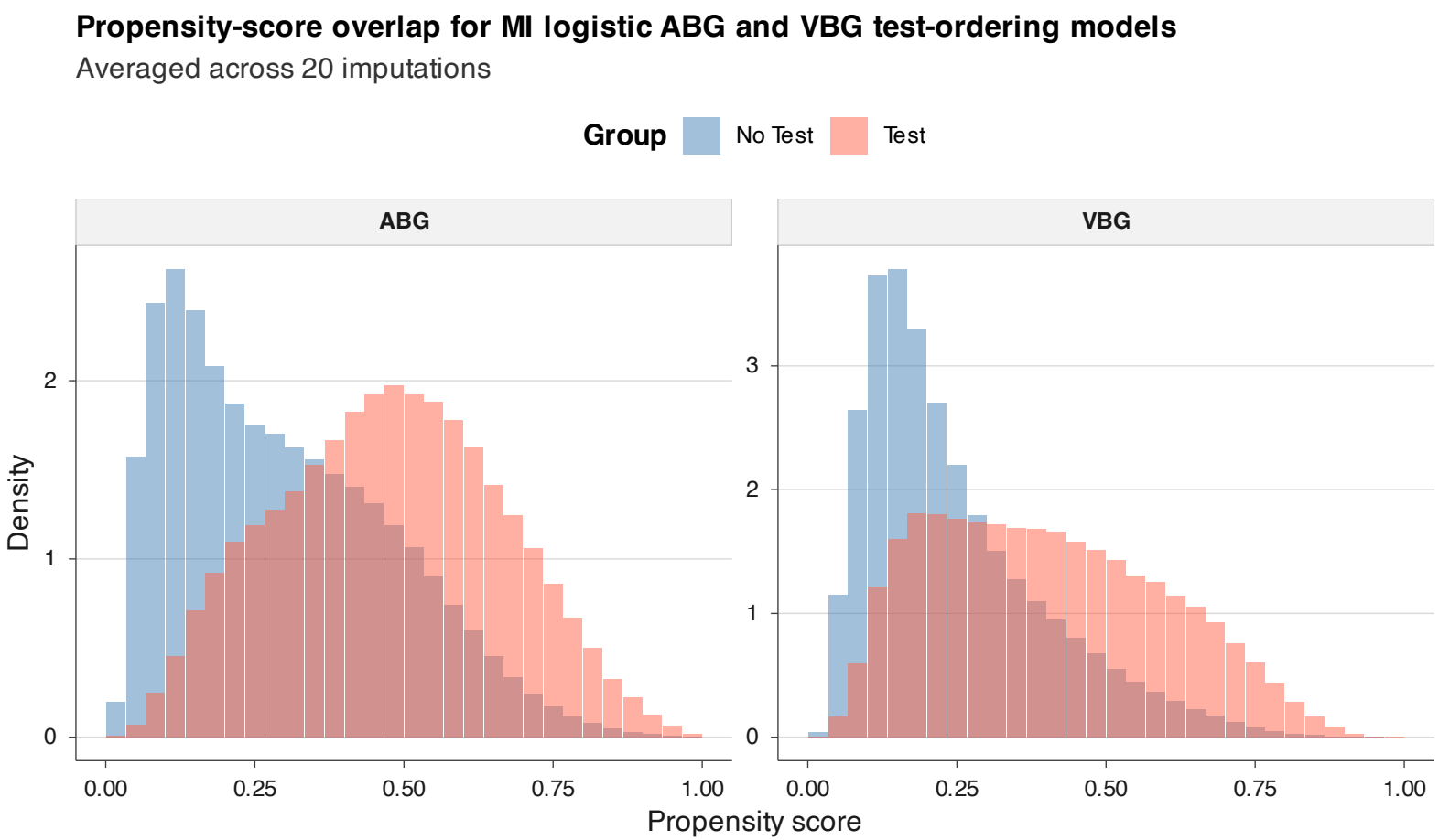


Figure S3. SHAP-style contribution summaries for MI logistic ABG and VBG test-ordering models

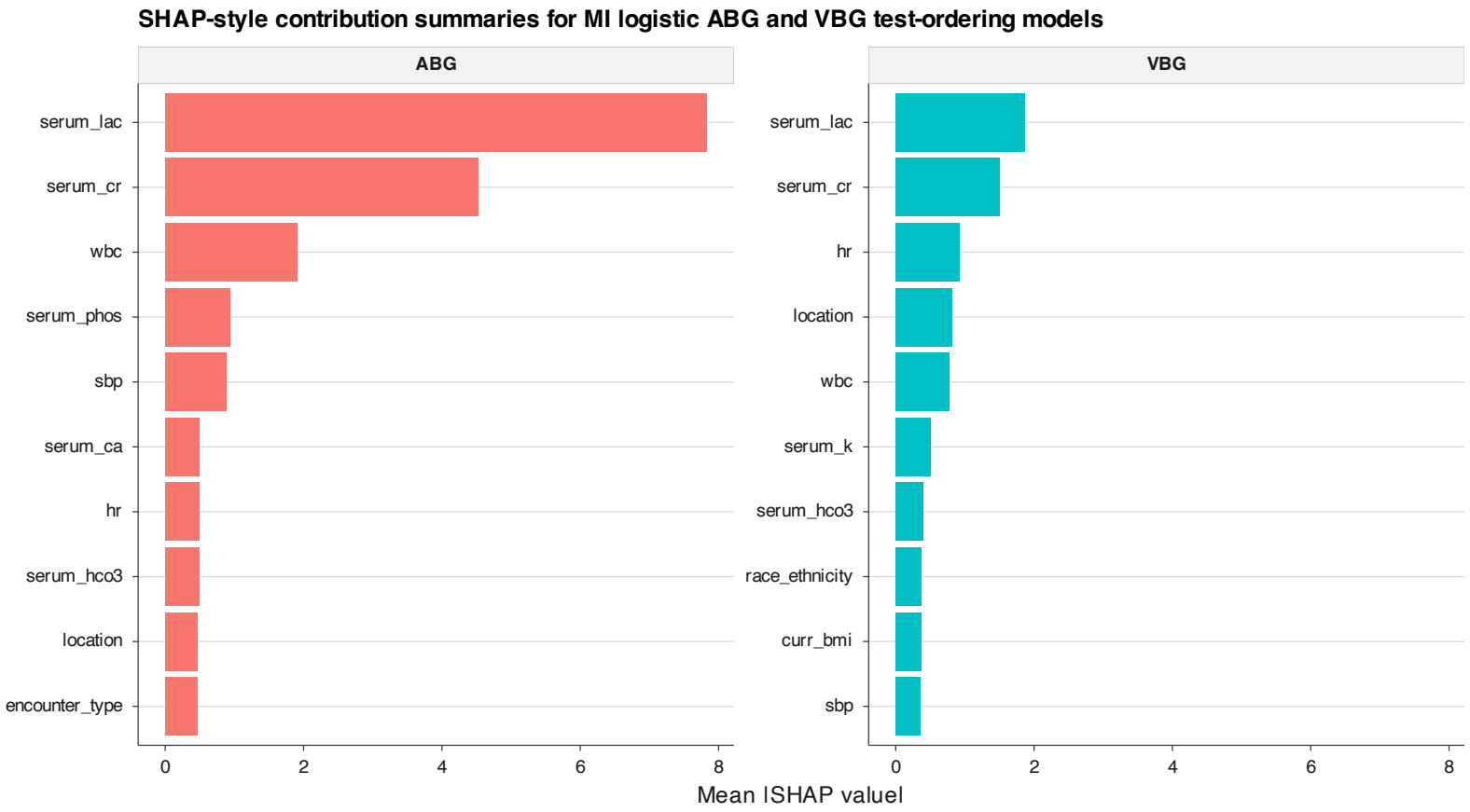
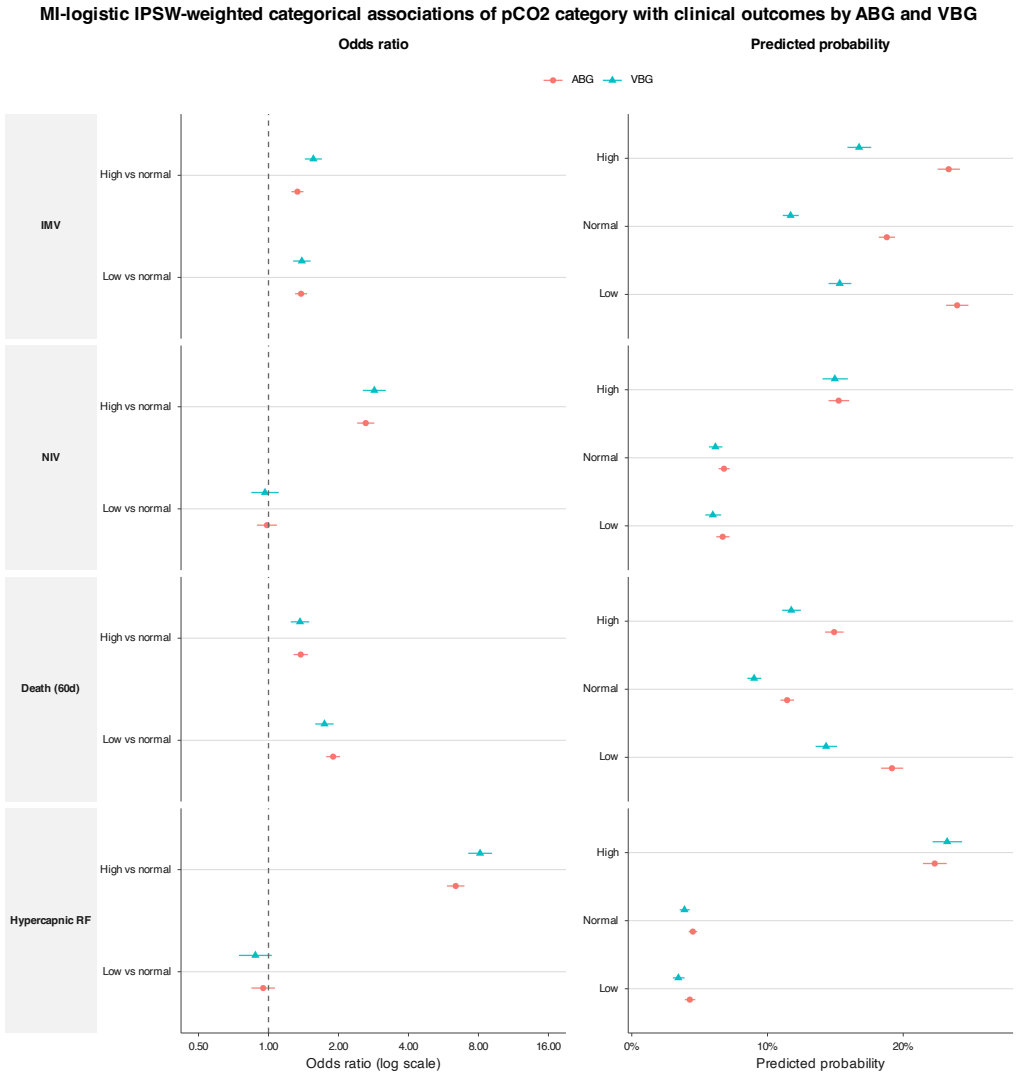
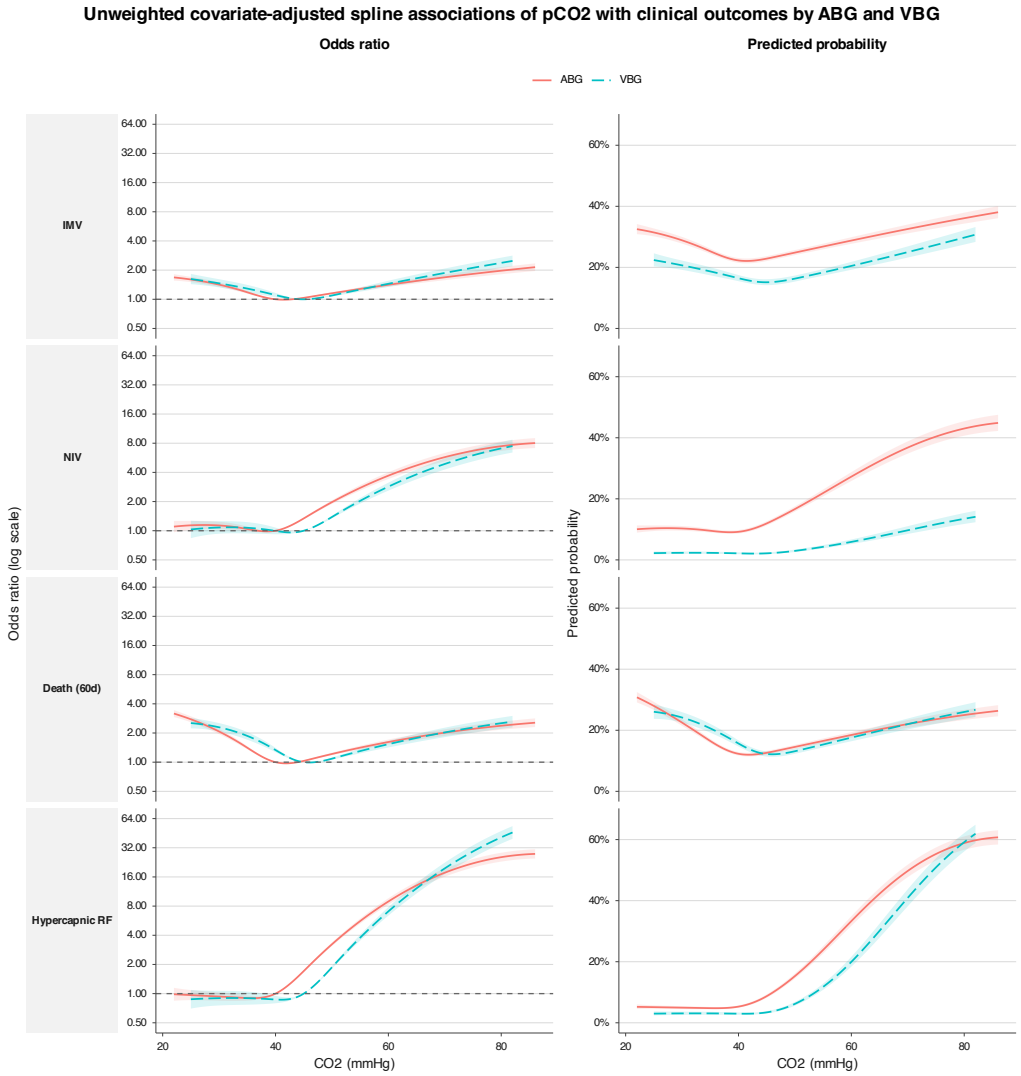


Figure S4. MI-logistic IPSW-weighted categorical associations of pCO2 category with clinical outcomes by ABG and VBG



Note: MI-pooled, MI-logistic IPSW-adjusted categorical odds ratios use the normal CO2 category as the reference. Predicted probabilities are pooled on the link scale and marginally standardized to the common eligible source-population covariate distribution.

Figure S5. Unweighted covariate-adjusted spline associations of pCO2 with clinical outcomes by ABG and VBG



Note: Odds ratio curves are relative to the test-specific reference CO2 values (ABG 40 mmHg; VBG 45 mmHg). Predicted probability curves are conditional on the reference covariate profile X_ref within each test-specific cohort.

Table S2. Crude associations of pCO2 category with clinical outcomes by ABG and VBG

group	outcome	contrast	OR (95% CI)
ABG	IMV	Low vs normal	1.36 (1.29, 1.43)
ABG	IMV	High vs normal	1.25 (1.19, 1.31)
ABG	NIV	Low vs normal	1.06 (0.97, 1.15)
ABG	NIV	High vs normal	2.41 (2.25, 2.58)
ABG	Death (60d)	Low vs normal	1.90 (1.79, 2.01)
ABG	Death (60d)	High vs normal	1.44 (1.35, 1.52)
ABG	Hypercapnic RF	Low vs normal	0.96 (0.87, 1.06)
ABG	Hypercapnic RF	High vs normal	5.87 (5.47, 6.32)
VBG	IMV	Low vs normal	1.53 (1.43, 1.64)
VBG	IMV	High vs normal	1.70 (1.59, 1.82)
VBG	NIV	Low vs normal	1.16 (1.04, 1.30)
VBG	NIV	High vs normal	2.75 (2.51, 3.01)
VBG	Death (60d)	Low vs normal	1.72 (1.60, 1.85)
VBG	Death (60d)	High vs normal	1.55 (1.45, 1.67)
VBG	Hypercapnic RF	Low vs normal	1.08 (0.95, 1.22)
VBG	Hypercapnic RF	High vs normal	8.02 (7.32, 8.81)

Table S3. GBM IPSW-weighted associations of pCO2 category with clinical outcomes by ABG and VBG

group	outcome	contrast	OR (95% CI)
ABG	IMV	Low vs normal	1.41 (1.33, 1.49)
ABG	IMV	High vs normal	1.30 (1.23, 1.38)
ABG	NIV	Low vs normal	1.04 (0.95, 1.15)
ABG	NIV	High vs normal	2.69 (2.48, 2.92)
ABG	Death (60d)	Low vs normal	1.99 (1.86, 2.12)
ABG	Death (60d)	High vs normal	1.43 (1.34, 1.53)
ABG	Hypercapnic RF	Low vs normal	0.99 (0.89, 1.11)
ABG	Hypercapnic RF	High vs normal	6.68 (6.15, 7.25)
VBG	IMV	Low vs normal	1.38 (1.28, 1.50)
VBG	IMV	High vs normal	1.55 (1.43, 1.67)
VBG	NIV	Low vs normal	0.99 (0.87, 1.12)
VBG	NIV	High vs normal	2.84 (2.56, 3.15)
VBG	Death (60d)	Low vs normal	1.72 (1.58, 1.86)
VBG	Death (60d)	High vs normal	1.40 (1.29, 1.52)
VBG	Hypercapnic RF	Low vs normal	0.92 (0.80, 1.07)
VBG	Hypercapnic RF	High vs normal	8.76 (7.88, 9.73)

Figure S6. Covariate balance after gradient-boosted propensity weighting for ABG and VBG test-ordering models

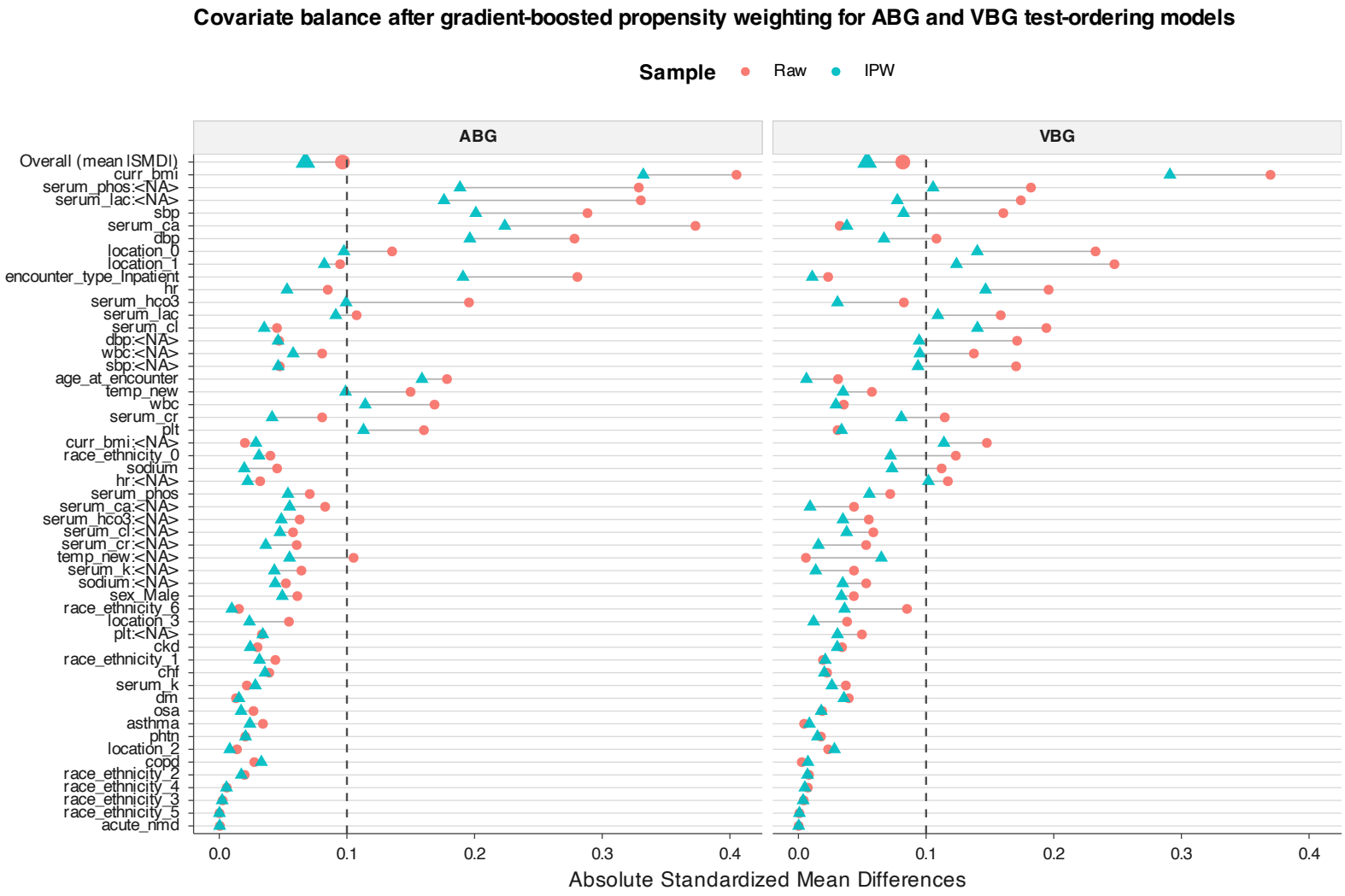


Figure S7. Propensity-score overlap for gradient-boosted ABG and VBG test-ordering models

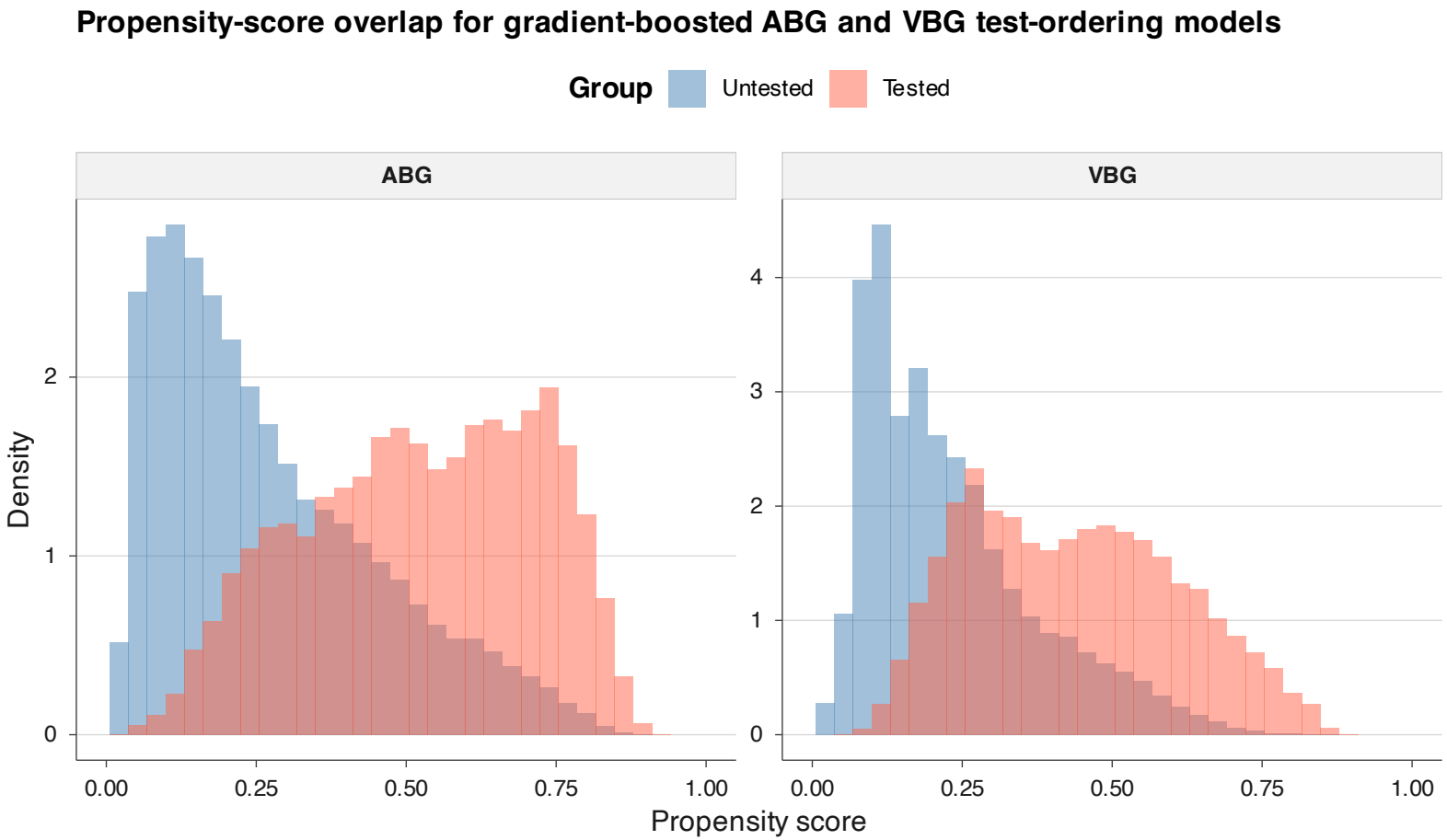


Figure S8. SHAP-style contribution summaries for gradient-boosted ABG and VBG test-ordering models

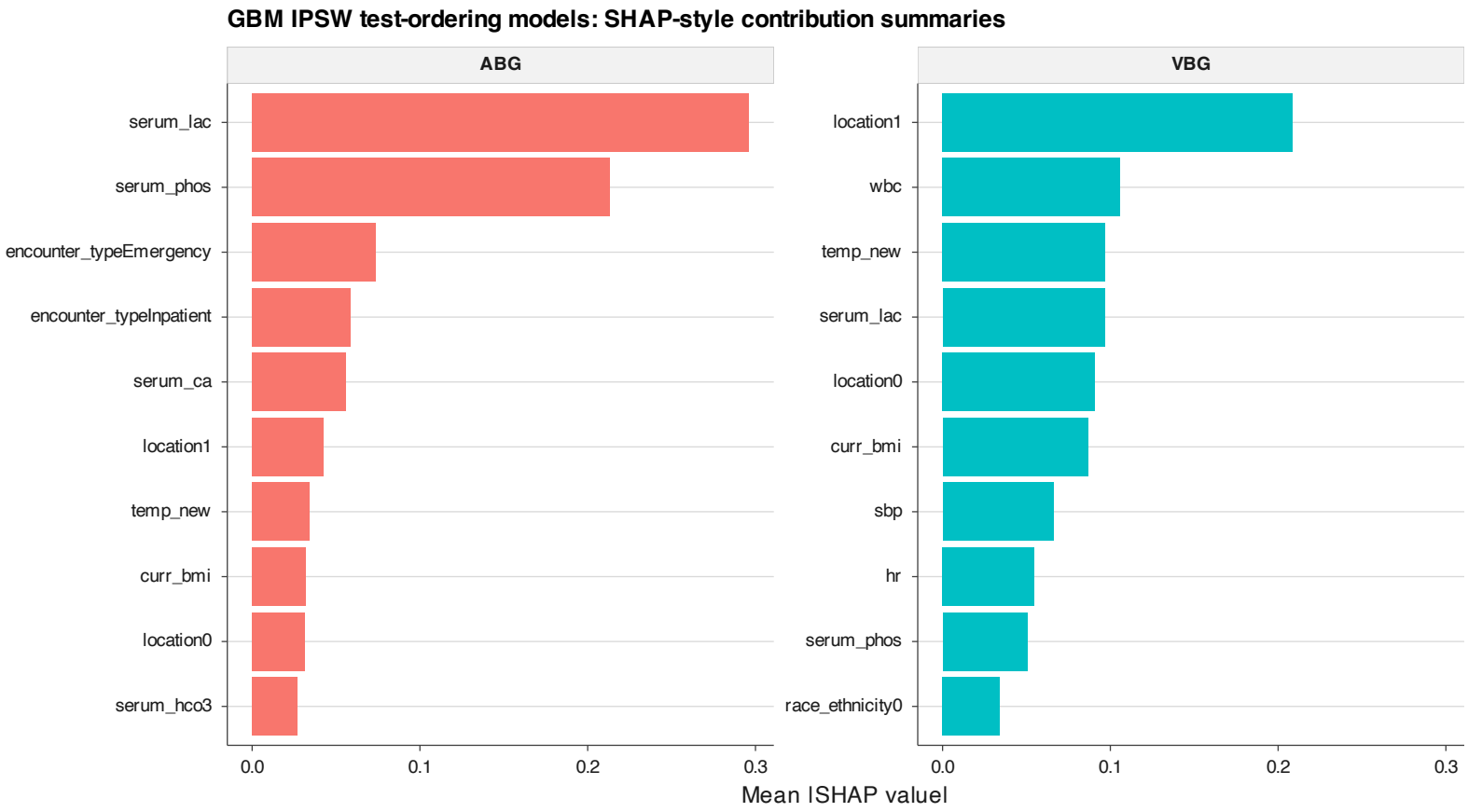


Table S4. Missingness of baseline covariates used in the primary analysis

variable	Analysis role	% missing	Imputed in primary MI
Age	direct adjustment; PS model	0.0%	Yes
Sex	direct adjustment; PS model	0.0%	Yes
Race/ethnicity	direct adjustment; PS model	0.0%	Yes
BMI	PS model	57.0%	Yes
COPD	PS model	0.0%	Yes
Asthma	PS model	0.0%	Yes
OSA	PS model	0.0%	Yes
CHF	PS model	0.0%	Yes
Neuromuscular disease	PS model	0.0%	Yes
Pulmonary hypertension	PS model	0.0%	Yes
Chronic kidney disease	PS model	0.0%	Yes
Diabetes	PS model	0.0%	Yes
Location	direct adjustment; PS model	0.0%	Yes
Encounter type	direct adjustment; PS model	0.0%	Yes
Temperature	PS model	48.0%	Yes
Systolic blood pressure	PS model	29.9%	Yes
Diastolic blood pressure	PS model	30.2%	Yes
Heart rate	PS model	36.3%	Yes
Sodium	PS model	5.3%	Yes
Serum creatinine	PS model	9.5%	Yes
Serum bicarbonate	PS model	5.8%	Yes
Serum chloride	PS model	5.9%	Yes
Serum lactate	PS model	59.9%	Yes
Serum potassium	PS model	8.0%	Yes
White blood cell count	PS model	17.7%	Yes
Platelet count	PS model	8.0%	Yes
Serum phosphate	PS model	52.9%	Yes
Serum calcium	PS model	10.3%	Yes

Table S5. Multiple-imputation diagnostic summary

domain	metric	value	interpretation
Imputation run	Completed imputations	20	Primary analysis imputation count used in this render.
Imputation run	Iterations per imputation	5	Configured MICE iteration count for this render.
Imputation run	Variables imputed	16	Variables with non-empty MICE imputation methods and observed missingness.
Monte Carlo error	Maximum MC error / SE in diagnostic model	0.170	Diagnostic model ratio; lower values indicate less simulation noise relative to model SE.
Chain diagnostics	Variables with available chain diagnostics	16	Imputed variables with traceable chain-mean diagnostics.
Chain diagnostics	Variables flagged for drift	6	Variables exceeding drift/slope screening thresholds.
MICE logged events	Total logged events	0	Rows reported by mice::loggedEvents across batches.
Captured warnings	Captured warning rows	0	Warnings captured by notebook-level diagnostics wrappers.
Outcome fit diagnostics	MI model fits audited	692	Outcome-model diagnostic rows captured for MI analyses.
Outcome fit diagnostics	Fits with nonconvergence flags	0	Potential convergence problems requiring review if nonzero.
Outcome fit diagnostics	Fits with separation/extreme-probability flags	172	Known separation/extreme-probability screen; interpret affected estimates cautiously.

```r
render_table_pdf(
  artifact_check_summary_display,
  caption = "Artifact check summary",
  file_stub = "artifact_check_summary",
  wide = FALSE,
  digits = 0,
  max_cols = 4,
  landscape = FALSE
)
```

Artifact check summary

check_status	severity	allowed_missing	n_rows
present	none	FALSE	68
suppressed_draft_only	none	TRUE	1

```r
render_table_pdf(
  diagnostics_audit_summary_display,
  caption = "Diagnostics audit summary",
  file_stub = "diagnostics_audit_summary_display",
  wide = TRUE,
  digits = 0,
  max_cols = 6,
  landscape = TRUE
)
```

Diagnostics audit summary

component	metric	status	severity	value	note
Audit	overall_status	fail	high	FAIL	Overall diagnostics audit status derived from the highest-severity unresolved issue.
Outcome	separation_flags_total	fail	high	172	Separation or near-separation can bias ORs and confidence intervals.
Inventory	expected_artifacts_missing	pass	none	0	Count of expected diagnostics artifacts missing from Results/.
MI	smoke_test_failed	pass	none	FALSE	Smoke test failure indicates the MI execution path is unstable.
MI	chain_diagnostics_issue	pass	none	FALSE	numeric_names=FALSE; drift_tail_na_frac=0.000
Balance	abg_max_abs_smd	pass	none	0.091	ABG target balance threshold is 0.10.
Balance	vbg_max_abs_smd	pass	none	0.045	VBG target balance threshold is 0.10.
Plotting	plot_drop_log_present	pass	none	TRUE	Plot drop logging should exist even if no rows are present.

```r
if (any(diagnostics_audit_issues_df$severity %in% c("high", "medium", "low"), na.rm = TRUE)) {
  render_table_pdf(
    diagnostics_audit_issues_df,
    caption = "Diagnostics audit issues",
    file_stub = "diagnostics_audit_issues",
    wide = TRUE,
    digits = 0,
    max_cols = 6,
    landscape = TRUE
  )
}
```

Diagnostics audit issues

severity	component	evidence_file	evidence_snippet	why_it_matters	recommended_fix
high	Outcome	Results/model_fit_diagnostics.csv	sep_flag TRUE for 172 / 692 fits	High rate of separation/near-separation can bias ORs and CIs.	Inspect flagged outcomes; consider penalized fits or check data sparsity.

run_id	run_ts
20260421_121048	2026-04-21 12:10:48.40139

```r
# Purpose: mi save exports.
stopifnot(
  exists("abg_curves"), exists("vbg_curves"),
  exists("abg_prob_curves"), exists("vbg_prob_curves"),
  exists("abg_coefs"), exists("vbg_coefs")
)
saveRDS(
  list(
    abg_curves = abg_curves,
    vbg_curves = vbg_curves,
    abg_prob_curves = abg_prob_curves,
    vbg_prob_curves = vbg_prob_curves,
    abg_coefs  = abg_coefs,
    vbg_coefs  = vbg_coefs
  ),
  mi_pooled_file
)
```

```r
# Purpose: mi session.
session_lines <- capture.output(sessionInfo())
writeLines(session_lines, results_path("sessionInfo.txt"))
writeLines(session_lines, results_path("session_info.txt"))
renv_status_lines <- if ("renv" %in% rownames(installed.packages())) {
  tryCatch(capture.output(renv::status()), error = function(e) c("renv::status() failed:", conditionMessage(e)))
} else {
  "renv package is not installed in this environment."
}
writeLines(renv_status_lines, results_path("renv_status.txt"))
```

```r
# Purpose: software statement.
cat("Software: R ", as.character(getRversion()),
    "; key packages: mice, WeightIt, cobalt, survey, rms.\n")
```

Software: R 4.5.3 ; key packages: mice, WeightIt, cobalt, survey, rms.

```r
# Purpose: record that render-path validation is deferred to wrapper postflight,
# which is the first point where the current render's TeX actually exists.
scan_out <- results_path("render_path_hygiene_scan.csv")
```

```r
scan_df <- data.frame(
  scan_status = "deferred_to_wrapper",
  line = "",
  run_id = diag_run_id,
  run_ts = diag_run_ts,
  stringsAsFactors = FALSE
)
write_csv_safely(scan_df, scan_out, row_names = FALSE)
```

```r
# Purpose: record that PDF hygiene validation is deferred to wrapper postflight,
# which is the first point where the current render's PDF actually exists.
scan_out <- results_path("pdf_hygiene_scan.csv")
scan_df <- data.frame(
  scan_status = "deferred_to_wrapper",
  scanner = "wrapper_postflight",
  line = "",
  run_id = diag_run_id,
  run_ts = diag_run_ts,
  stringsAsFactors = FALSE
)
write_csv_safely(scan_df, scan_out, row_names = FALSE)
```

```r
# Purpose: numbering hygiene scan.
qmd_file <- resolve_current_qmd()
src_lines <- readLines(qmd_file, warn = FALSE)
pattern_manual <- "(^#+\\s+[0-9]+(\\.[0-9]+)*\\s+[0-9]+\\))|(Table\\s+[0-9]+:\\s+Table\\s+[0-9]+)"
hits <- which(grepl(pattern_manual, src_lines))
scan_df <- data.frame(
  line_no = hits,
  line = src_lines[hits],
  stringsAsFactors = FALSE
)
write_csv_safely(scan_df, results_path("numbering_hygiene_scan.csv"), row_names = FALSE)
```

```r
# Purpose: remove transient MI scratch artifacts after a successful render.
cleanup_mi_transient_outputs()
```